THREE-LABEL DECREASING DIAGRAMS ARE COMPLETE FOR CONFLUENT REWRITING RELATIONS OF CARDINALITY AT MOST OMEGA1

IEVGEN IVANOV

Editorial note. Paper: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2025.25. Author Ievgen Ivanov; CC BY4.0. Supplement: https://doi.org/10.5281/zenodo.14254256 ; license metadata preserved in zenodo-record.json.

Editorial note. Author formal supplement is retained as received. Collection does not claim a new Isabelle check or independent mathematical certification.

Allowed standard prerequisites (editorial).

- ZFC/NBG set theory with choice, ordinal/cardinal arithmetic and HOL-Cardinals. Author §3 and references3,11,30; supplement imports standard Main and HOL-Cardinals theories only.

3 Preliminaries

A reader can assume that a background theory for understanding the natural language definitions and propositions given below is Von Neumann-Bernays-Gödel (NBG) set theory [30, 11] with the axiom of global choice [11, p. 133] (it allows one to directly reason about proper classes). Formal claims and proofs from Isabelle/HOL formalization [17] referenced in the paper can be transferred to the set-theoretic setting using a process similar to the one described in [25, 27]. The statement of the main result (Theorem 18) does not essentially depend on proper classes, so it and its proof can also be transferred to ZFC set theory.

3.1 General notation and terminology

We will use the following notation:

■

$\neg$, $\vee$, $\wedge$, $\Rightarrow$ are the logical negation, disjunction, conjunction, and implication respectively

■

$(a_i)_{i=0}^{\infty}$ is a sequence of elements indexed by non-negative integers

■

$(a_i)_{i\in I}$ is an indexed family indexed by elements of a set I.

If A is a set, any subset $R \subseteq A \times A$ is a (binary) relation on A. Any subset $R' \subseteq R$ is a subrelation of R. If A, B are any sets, n is a positive integer, and $R \subseteq A \times A$, then:

■

$\mathrm{Fld}(R) = \{a \mid \exists b\ (a,b) \in R\} \cup \{b \mid \exists a\ (a,b) \in R\}$, here $\mathrm{Fld}(R)$ is called the field of R

■

$R^{-1} = \{(b,a) \mid (a,b) \in R\}$

■

$\mathrm{Id}_A = \{(a,a) \mid a \in A\}$

■

Date: doi:10.4230/LIPIcs.FSCD.2025.25; CC BY 4.0 (paper and author Zenodo supplement).

$R^n = \{(a,b) \mid \exists a_0, a_1, ..., a_n \;\; a = a_0 \wedge b = a_n \wedge \forall i \in \{0,1,...,n-1\}\; (a_i, a_{i+1}) \in R\}$

■

$R^+ = \bigcup_{n=1}^{\infty} R^n$, i.e. R^+ is the transitive closure of R

■

$R|_B = R \cap (B \times B)$, i.e. $R|_B$ is the restriction of R to B.

► **Remark 1.**

We will usually use letters R, r, s or special symbols (e.g. $\rightarrow$) for relations. Sometimes we will use the infix notation for membership of a pair in a relation, e.g $a \rightarrow b$.

3.2 Ordinals and cardinality

In the paper we will assume that an ordinal [30] is a set α such that $\in$ well-orders α and each element of α is a subset of α.

► **Remark 2.**

In the supplementary material [17] we do not use the above mentioned definition of an ordinal. Instead, we use the HOL formalization [3] of ordinals (and cardinals) using typed well-orders that allows one to model reasoning that involves set-theoretic ordinals. Information on other related approaches to reasoning about ordinals can be found in [33].

If α, β are ordinals, then $\alpha < \beta$ denotes $\alpha \in \beta$ and $\alpha \leq \beta$ denotes $(\alpha = \beta) \vee (\alpha < \beta)$, Also, $\alpha > \beta$ and $\alpha \geq \beta$ denote $\beta < \alpha$ and $\beta \leq \alpha$ respectively.

► **Remark 3.**

For every ordinals α, β, $\alpha \leq \beta \vee \beta \leq \alpha$, and every nonempty set A of ordinals has a least element with respect to $\leq$ (i.e. an element $\alpha \in A$ such that $\forall \alpha' \in A \; \alpha \leq \alpha'$).

An ordinal α is

■

an (ordinal) successor of an ordinal α', if α is the least ordinal greater than α'

■

a successor ordinal, if α is a successor of some ordinal

■

a limit ordinal, if α is neither 0 (empty set), nor a successor ordinal.

For any ordinals α, β and a nonempty set of ordinals A:

■

$\alpha + 1$ is the (unique) ordinal successor of α

■

$[\alpha, \beta]$ and $[\alpha, \beta)$ denote $\{\alpha' \mid \alpha \leq \alpha' \leq \beta\}$ and $\{\alpha' \mid \alpha \leq \alpha' < \beta\}$ respectively

■

$[\alpha, \beta]_{0S}$ is the set of all $\alpha' \in [\alpha, \beta]$ such that α' is either 0, or is a successor ordinal

■

$[\alpha, \beta]_L = [\alpha, \beta] \backslash [\alpha, \beta]_{0S}$ is the set of all limit ordinals in $[\alpha, \beta]$

■

$\min A$ is the least element of A with respect to $\leq$.

For any set A we will denote:

■

$|A|$ (cardinality of A) is the least ordinal α (with respect to $\leq$) such that there exists a bijective function $f : \alpha \rightarrow A$.

► **Remark 4.**

Since we assume the axiom of choice, $|A|$ is defined for any set A [11, 30].

► **Remark 5.**

In [17] we use a formal notion of the cardinality of a set described in [3]. Comparisons of cardinalities of sets in [17] using special relations =o, <o, <=o described in [3] can be interpreted as comparisons of set-theoretic cardinalities defined above using $=, <, \leq$.

We will denote:

- ω is the least limit ordinal
- ω_1 is the least ordinal such that its cardinality is strictly greater than ω.

3.3 Decreasing Church-Rosser abstract rewriting systems

Semi-formally, a rewriting system is decreasing Church-Rosser (DCR), if it can be proven confluent using the decreasing diagrams method. Rigorous definitions are given below.

Definition 6.

1. An abstract rewriting system (ARS) is a pair $(A, \to)$ of a set A and a relation $\to \subseteq A \times A$.
2. An ARS $(A, \to)$ is confluent, if

$$\forall a, b, c \in A \ (a \to^* b \wedge a \to^* c \Rightarrow (\exists d \in A \ b \to^* d \wedge c \to^* d)),$$

where $\to^* = \mathrm{Id}_A \cup (\to)^+$, i.e. $\to^*$ is the reflexive transitive closure of $\to$ on A.

Definition 7.

A labelled rewriting system (LRS) is a triple $(A, I, (\to_i)_{i \in I})$, where A and I are sets and $(\to_i)_{i \in I}$ is an indexed family (with the set of indices I) such that $\to_i \subseteq A \times A$ for each $i \in I$. Elements of I are called labels.

Definitions 11 and 12 given below are based on [41, Definition 2.3.7] (there is an insignificant difference between the definitions given below and in [41], because [41] uses a slightly different terminology and notation explained in Remark 8).

▶ **Remark 8.**

[41, Definition 2.3.7] refers to [41, Theorem 2.3.5] that applies the term "well-founded partial order" to a relation denoted as $\succ$, and the well-foundness condition is defined in [41, Definition 1.2.3] for any transitive relation R via a variant of ascending chain condition: there is no infinite sequence $a_1, a_2, ...$ such that $(a_i, a_{i+1}) \in R \backslash (R \cap R^{-1})$. In other literature relations that satisfy this kind of condition are called converse well-founded or Noetherian. We will not use the terminology concerning well-founded relations from [41] in this paper and instead use Definition 9 below that relies on the descending chain condition (and apply it to a relation denoted as $\prec$ that can be thought of as a strict partial order that corresponds to the inverse of $\succ$ from [41, Definition 2.3.7]).

Definition 9.

A transitive well-founded relation on a set I is a binary relation $\prec \subseteq I \times I$ such that $\forall a, b, c \in I \ a \prec b \wedge b \prec c \Rightarrow a \prec c$ and there is no infinite sequence $(a_i)_{i=1}^{\infty}$ such that $a_{i+1} \prec a_i$ for every positive integer i.

▶ **Remark 10.**

A transitive well-founded relation is irreflexive and antisymmetric.

Definition 11.

An LRS $(A, I, (\to_i)_{i \in I})$ is decreasing Church-Rosser (DCR), if there exists a transitive well-founded relation $\prec$ on I and subsets $I_h \subseteq I$ and $I_v \subseteq I$ such that

$\left(\bigcup_{i\in I} \to_i\right) = \left(\bigcup_{i\in I_v} \to_i\right) = \left(\bigcup_{i\in I_h} \to_i\right)$ and for every $i \in I_v$, $j \in I_h$, and $a, b, c \in A$ such that $a \to_i b$ and $a \to_j c$ there exists elements $b', b'', c', c'', d \in A$ such that

$$\left(b \underset{(\Upsilon i)\cap I_h}{\overset{*}{\dashrightarrow}} b' \underset{\{j\}\cap I_h}{\overset{=}{\dashrightarrow}} b'' \underset{(\Upsilon i \cup \Upsilon j)\cap I_h}{\overset{*}{\dashrightarrow}} d\right) \wedge \left(c \underset{(\Upsilon j)\cap I_v}{\overset{*}{\dashrightarrow}} c' \underset{\{i\}\cap I_v}{\overset{=}{\dashrightarrow}} c'' \underset{(\Upsilon i \cup \Upsilon j)\cap I_v}{\overset{*}{\dashrightarrow}} d\right),$$

where

- for any $i \in I$, Υi denotes the set $\{k \in I \mid k \prec i\}$
- symbols of the form $\overset{*}{\underset{K}{\dashrightarrow}}$ and $\overset{=}{\underset{K}{\dashrightarrow}}$, where K is an expression, mean: $\overset{*}{\underset{K}{\dashrightarrow}} = \mathrm{Id}_A \cup \left(\bigcup_{k\in K} \to_k\right)^+$ and $\overset{=}{\underset{K}{\dashrightarrow}} = \mathrm{Id}_A \cup \left(\bigcup_{k\in K} \to_k\right)$.

Definition 12.

An ARS $(A, \to)$ is decreasing Church-Rosser (DCR), if there exists a set I and an indexed family $(\to_i)_{i\in I}$ (with the set of indices I) such that $\to_i \subseteq A \times A$ for each $i \in I$, $\to = \left(\bigcup_{i\in I} \to_i\right)$, and $(A, I, (\to_i)_{i\in I})$ is a decreasing Church-Rosser LRS.

3.4 DCR hierarchy and the class DCR3

For any ordinal α denote $\Upsilon\alpha = \{\beta \mid \beta < \alpha\}$ (the set of all ordinals below α).

► **Remark 13.**

In the sense of the usual set-theoretic definition of ordinals, $\Upsilon\alpha$ is the ordinal α itself. However, the notation $\Upsilon\alpha$ will be used to highlight that α is used as a set of labels.

The following definition is based on [10, Definition 4.4] and [10, Definition 4.2].

Definition 14 ([19]).

Let γ be an ordinal. An ARS $(A, \to)$ belongs to the class DCR_γ, if there exists an indexed family $(\to_\alpha)_{\alpha\in(\Upsilon\gamma)}$ of binary relations on A indexed by ordinals $\alpha < \gamma$ such that for every ordinals $\alpha, \beta \in (\Upsilon\gamma)$ and for every $a, b, c \in A$ the following holds: if $a \to_\alpha b \wedge a \to_\beta c$, then there exist elements $b', b'', c', c'', d \in A$ such that

$$\left(b \underset{\Upsilon\alpha}{\overset{*}{\dashrightarrow}} b' \underset{\{\beta\}}{\overset{=}{\dashrightarrow}} b'' \underset{\Upsilon\alpha\cup\Upsilon\beta}{\overset{*}{\dashrightarrow}} d\right) \wedge \left(c \underset{\Upsilon\beta}{\overset{*}{\dashrightarrow}} c' \underset{\{\alpha\}}{\overset{=}{\dashrightarrow}} c'' \underset{\Upsilon\beta\cup\Upsilon\alpha}{\overset{*}{\dashrightarrow}} d\right),$$

where symbols of the form $\overset{=}{\underset{K}{\dashrightarrow}}$ and $\overset{*}{\underset{K}{\dashrightarrow}}$, where K is an expression, mean:

$$\overset{=}{\underset{K}{\dashrightarrow}} = \{(a,a) \mid a \in A\} \cup \bigcup_{\kappa\in K} \to_\kappa \quad \text{and} \quad \overset{*}{\underset{K}{\dashrightarrow}} = \{(a,a) \mid a \in A\} \cup \left(\bigcup_{\kappa\in K} \to_\kappa\right)^+.$$

(1)

► **Remark 15.**

For a finite ordinal γ (that corresponds to a natural number), semi-formally, the class DCR_γ consists of ARS which can be proved confluent with help of the decreasing diagrams method using labels $0, 1, ..., \gamma - 1$ ordered in the usual way $(0 < 1 < ... < \gamma - 1)$.

Proposition 16.

If γ is an ordinal and $(A, \to)$ is an ARS in the class DCR_γ, then $(A, \to)$ is decreasing Church-Rosser in the sense of Definition 12.

Proof.

Follows from Definitions 12, 11 by assuming that $I = I_h = I_v = (\Upsilon\gamma)$ and that a (transitive well-founded) relation $\prec$ is the restriction of the usual order on ordinals to I. ◄

The next proposition makes the definition of the class DCR_3 explicit.

Proposition 17 ([19]).

An ARS $(A, \to)$ belongs to the class DCR_3 if and only if there exist binary relations $\to_0, \to_1, \to_2 \subseteq A \times A$ such that $\to = (\to_0 \cup \to_1 \cup \to_2)$ and the following conditions 1–6 hold, where

■

for $i = 0, 1, 2$ the symbol $\to_i^=$ denotes the reflexive closure of $\to_i$ on A

■

for $i = 0, 1, 2$ the symbol $\to_i^*$ denotes the reflexive-transitive closure of $\to_i$ on A

■

the symbol $\to_{01}^*$ denotes the reflexive-transitive closure of $\to_0 \cup \to_1$ on A:

1.

$\forall a, b, c \in A\ ((a \to_0 b \wedge a \to_0 c) \Rightarrow \exists d \in A\ (b \to_0^= d \wedge c \to_0^= d))$

2.

$\forall a, b, c \in A\ ((a \to_0 b \wedge a \to_1 c) \Rightarrow \exists b', d \in A\ (b \to_1^= b' \wedge b' \to_0^* d \wedge c \to_0^* d))$

3.

$\forall a, b, c \in A\ ((a \to_1 b \wedge a \to_1 c) \Rightarrow$
$\exists b', b'', c', c'', d \in A\ (b \to_0^* b' \wedge b' \to_1^= b'' \wedge b'' \to_0^* d \wedge c \to_0^* c' \wedge c' \to_1^= c'' \wedge c'' \to_0^* d))$

4.

$\forall a, b, c \in A\ ((a \to_0 b \wedge a \to_2 c) \Rightarrow \exists b', d \in A\ (b \to_2^= b' \wedge b' \to_{01}^* d \wedge c \to_{01}^* d))$

5.

$\forall a, b, c \in A\ ((a \to_1 b \wedge a \to_2 c) \Rightarrow \exists b', b'', d \in A\ (b \to_0^* b' \wedge b' \to_2^= b'' \wedge b'' \to_{01}^* d \wedge c \to_{01}^* d))$

6.

$\forall a, b, c \in A\ ((a \to_2 b \wedge a \to_2 c) \Rightarrow$
$\exists b', b'', c', c'', d \in A\ (b \to_{01}^* b' \wedge b' \to_2^= b'' \wedge b'' \to_{01}^* d \wedge c \to_{01}^* c' \wedge c' \to_2^= c'' \wedge c'' \to_{01}^* d))$.

Proof.

Follows directly from Definition 14 by assuming that $\gamma = 3$ and using symmetry of the cases $(\alpha, \beta) \in \{(0, 1), (1, 0)\}$, $(\alpha, \beta) \in \{(0, 2), (2, 0)\}$, $(\alpha, \beta) \in \{(1, 2), (2, 1)\}$.

◀

Appendix A Correspondence Between the Notions/Notation Used in the Paper and in the Supplementary Material

Below we give tables of correspondence between the formal notions used in the supplementary material [17] (Isabelle/HOL formalization) and their intended interpretations in terms of the notions based on set theory used in natural language text in this paper.

Syntax of the formal language that is used to express Isabelle/HOL theories is described in the documentation included in Isabelle software [14]. Information on semantic aspects of Isabelle language can be found in [35, 27, 26, 25].

A.1 General notation

Notation in [17]	Notation in the paper
$\neg$, $\vee$, $\wedge$, $\longrightarrow$	$\neg$, $\vee$, $\wedge$, $\Rightarrow$
$=$, $\in$, $\subseteq$	$=$, $\in$, $\subseteq$
$\cup$, $\cap$, $-$, $\times$	$\cup$, $\cap$, $\setminus$, $\times$
{}	$\emptyset$
f::'A ⇒ 'B	$f : A \to B\ (A, B \neq \emptyset)$
f x	$f(x)$
{x. P x}	$\{x \mid P(x)\}$
nat	$\{0, 1, 2, ...\}$

UNIV::'U	U $(U \neq \emptyset)$
x::'U	$x \in U$
A::'U set	$A \subseteq U$ $(U \neq \emptyset)$
r::'U rel	$r \subseteq U \times U$ $(U \neq \emptyset)$
Field r	$\mathrm{Fld}(r)$
Restr r B	$r\vert_B$
r-1, r^^n, r^+, r^=, r^*	r^{-1}, r^n, r^+, $r^=$, r^*
|A| =o |B|	$\vert A\vert = \vert B\vert$
|A| <o |B|	$\vert A\vert < \vert B\vert$
|A| <=o |B|	$\vert A\vert \leq \vert B\vert$
ω_ord	$\vert\mathbb{N}\vert$

A.2 Notation related to subsection 5.2

Lines in [17]	Notation in [17]	Notation in the paper
78–80	$\mathfrak{L}_1$ g α	$\mathfrak{L}_1(g, \alpha)$
82–84	$\mathfrak{L}_\vee$ g α β	$\mathfrak{L}_\vee(g, \alpha, \beta)$
86–88	$\mathfrak{D}$ g α β	$\mathfrak{D}(g, \alpha, \beta)$
116–118	CCR r	$r \in CCR(U)$
120–122	Conelike r	$r \in Conelike(U)$
90–93	DCR_generating g	g is DCR-generating
95–97	DCR n r	$r \in DCR_n(U)$
177–178	$\mathfrak{U}$ r	$\mathfrak{U}(r)$

A.3 Notation related to subsection 5.3 (part 1)

Lines in [17]	Notation in [17]	Notation in the paper
N/A	r \`\` A	$r\,[A]$
N/A	r \`\` {x}	$r\{x\}$
124–126	dncl r A	$\mathrm{dncl}(r, A)$
128–130	Inv r	$Inv(r)$
189–190	Span r	$Span(r)$
132–134	SF r	$SF(r)$
136–137	SCF r	$SCF(r)$
195–196	scf r	$\mathrm{scf}(r)$

A.4 Notation related to subsection 5.3 (part 2)

Lines in [17]	Notation in [17]	Notation in the paper
147–149	$\mathcal{F}$ r a b	$\mathcal{F}(r, a, b)$
198–200	w_dncl r A	$\mathrm{wdncl}(r, A)$
307–308	wrank r A	$\mathrm{wrank}(r, A)$
155–157	dnEsc r A a	$\mathrm{dn}_{\mathrm{Esc}}(r, A, a)$
159–161	dnesc r A a	$\mathrm{dn}_{\mathrm{esc}}(r, A, a)$
163–165	escl r A B	$\mathrm{escl}(r, A, B)$
167	clterm s′ r	$\mathrm{clterm}(s', r)$

A.5 Notation related to subsection 5.4

Lines in [17]	Notation in [17]	Notation in the paper
202–204	$\mathfrak{L}$ f α	$\mathfrak{L}(f, \alpha)$

206–208	∇ f α	$\nabla(f, \alpha)$
210–212	$\mathcal{Q}$ r f α	$\mathcal{Q}(r, f, \alpha)$
214–216	$\mathcal{W}$ r f α	$\mathcal{W}(r, f, \alpha)$
218–267	$\mathcal{N}i$ r $\alpha 0$	$\mathcal{N}_i(r, \alpha_0)$, i is an integer
269–274	$\mathcal{N}$ r Ps	$\mathcal{N}(r, Ps)$

Theorem 18 (Completeness of the decreasing diagrams method with 3 labels for proving confluence of ARS of the least uncountable cardinality).

If (U, r) is a confluent ARS and $|r| \leq \omega_1$, then (U, r) belongs to the class DCR_3.

5 Structure of the Formal Proof of the Main Result

The formal proof of Theorem 18 uses slightly more than 80 formal definitions and 260 formal lemmas located in one file in [17] (DCRN1.thy, file length is over 11000 lines). This file depends only on standard theories included in Isabelle software (Main and HOL-Cardinals).

In subsection 5.1 we describe in a semi-formal way a high-level idea of the proof. In subsequent subsections we describe the way in which the most important definitions and lemmas are formulated in [17] (correspondence between the notation used in the paper and in [17] is described in Appendix A). Auxiliary technical lemmas used in the formal proof of the main result are not described in the paper, but are given in [17].

An approximate correspondence between the content of subsection 5.1 and the rest of Section 5 is illustrated in Figure 2.

5.1 General idea of the proof of Theorem 18

Consider a confluent ARS (U, r) with $|r| \leq \omega_1$ as a directed graph.

Consider a connected component C of (U, r).

Case A.

If C (viewed as an ARS) has the cofinality property (defined in [9, Definition 7]), then it belongs to the class DCR_2 by [9, Theorem 19], so it also belongs to DCR_3.

Case B.

If C does not have the cofinality property, then perform the following steps.

Step B1.

Extract a subgraph S from C such that

- S has a shape illustrated in Figure 1
- nodes of S form a cofinal subset in (the preordered set induced by) C.

Step B2.

Show that S (viewed as an ARS) belongs to DCR_2. In Figure 1:

- label red arcs (that start at nodes with unique successor) with 0
- label green arcs (that start at nodes with non-unique successor) with 1.

Step B3.

Use the cofinality of S in C to conclude that C belongs to DCR_3.

Then (U, r) belongs to DCR_3, because all its connected components belong to DCR_3.

In more detail, the main step "B1" will involve the following sub-steps.

Sub-step B1a.

Extraction of a preliminary cofinal subgraph generally analogous to the one shown in Figure 1, but where red arcs may not form trees and there may be extra green arcs. It will be described using the relation r_1 in the proof of Lemma 29. This sub-step will be performed using an analog of Iwamura's lemma for connected Church-Rosser (CCR) relations that will be described in subsection 5.4. The latter allows one to represent a CCR ARS as a union of a (well-ordered) chain of CCR ARS of smaller cardinality (so, e.g., a CCR ARS of the cardinality $\aleph_1$ can be represented as a union of a chain of at most countable CCR ARS that have the cofinality property).

Sub-step B1b.

Refinement of a subgraph mentioned above to a connected cofinal subgraph that belongs to DCR_2. This sub-step will be described in the proof of Lemma 27.

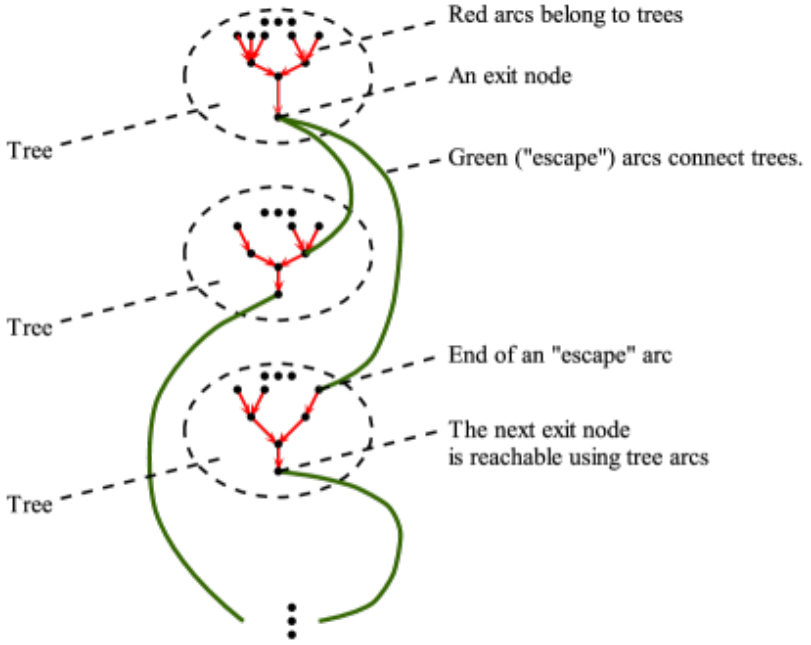


Figure 1: Structure of a connected cofinal subgraph of a connected component of a confluent ARS of the least uncountable cardinality, assuming this component lacks the cofinality property.

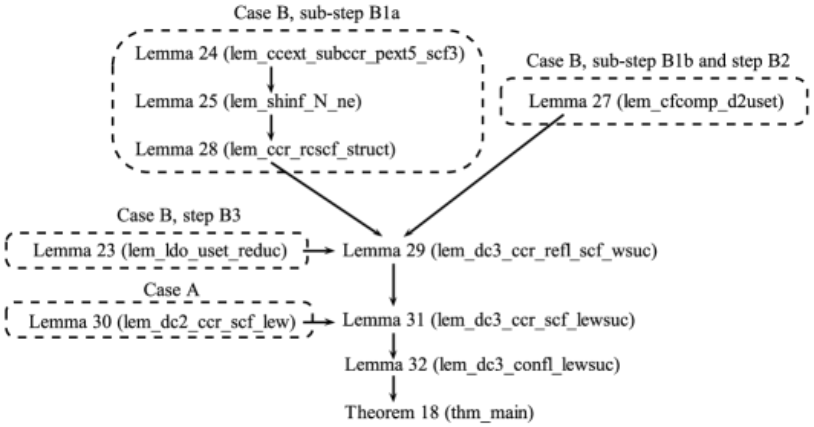


Figure 2: An approximate correspondence between the cases/steps from subsection 5.1 and lemmas used in the proof of the main result. Arrows $A \to B$ mean that

A is used in the proof of B. Lemmas are denoted as follows: "Name of a lemma in the paper (name of a formal version in [17])".

5.2 Formalization of finite levels of the DCR hierarchy

In this subsection we describe the way in which finite levels of the DCR hierarchy, i.e. classes DCR_α for ordinals $\alpha < \omega$, are formalized in [17].

For any relation $s \subseteq U \times U$ we will denote as $s^=$ and s^* the reflexive, and, respectively, reflexive transitive closure of s on the set U.

For any function $g : [0, \omega) \to 2^{U \times U}$ and any ordinals $\alpha, \beta \in [0, \omega)$ let us denote:

■

$\mathfrak{L}_1(g, \alpha) = \bigcup_{\alpha' \in [0,\alpha)} g(\alpha')$

■

$\mathfrak{L}_\vee(g, \alpha, \beta) = \bigcup_{\alpha' \in [0,\alpha) \cup [0,\beta)} g(\alpha')$

■

$\mathfrak{D}(g, \alpha, \beta)$ is the set of all tuples $(b, b', b'', d) \in U \times U \times U \times U$ such that

$$(b, b') \in (\mathfrak{L}_1(g, \alpha))^* \ \wedge \ (b', b'') \in (g(\beta))^= \ \wedge \ (b'', d) \in (\mathfrak{L}_\vee(g, \alpha, \beta))^*.$$

Definition 20.

A function $g : [0, \omega) \to 2^{U \times U}$ is called DCR-generating, if for every $\alpha, \beta \in [0, \omega)$ and every $a, b, c \in U$ the following implication holds: if $(a, b) \in g(\alpha) \wedge (a, c) \in g(\beta)$, then there exist $b', b'', c', c'', d \in U$ such that

$$(b, b', b'', d) \in \mathfrak{D}(g, \alpha, \beta) \wedge (c, c', c'', d) \in \mathfrak{D}(g, \beta, \alpha).$$

Let us denote:

■

$DCR_n(U)$, where $n \in [0, \omega)$, is the set of all relations $r \subseteq U \times U$ such that there exists a DCR-generating function g such that $r = \bigcup_{\alpha \in [0,n)} g(\alpha)$.

▶ **Remark 21.**

It is straightforward to check that for every $n \in [0, \omega)$, a relation r belongs to $DCR_n(U)$ if and only if the ARS (U, r) belongs to the class DCR_n.

Definition 22.

A binary relation $r \subseteq U \times U$ is called

1.

connected Church-Rosser (CCR), if $\forall a, b \in \mathrm{Fld}(r)\ \exists c \in \mathrm{Fld}(r)\ (a, c) \in r^* \wedge (b, c) \in r^*$

2.

cone-like, if $r = \emptyset \vee (\exists m \in \mathrm{Fld}(r)\ \forall a \in \mathrm{Fld}(r)\ (a, m) \in r^*)$.

We will denote:

■

$CCR(U)$ is the set of all CCR relations $r \subseteq U \times U$

■

$Conelike(U)$ is the set of all cone-like relations $r \subseteq U \times U$.

For any $r \subseteq U \times U$ let us denote:

■

$\mathfrak{U}(r) = \{s \in CCR(U) \mid s \subseteq r \ \wedge \ \forall a \in \mathrm{Fld}(r)\ \exists b \in \mathrm{Fld}(s)\ (a, b) \in r^*\}$. We will call any element of $\mathfrak{U}(r)$ a CCR residue of r.

Note that a CCR residue of r is a CCR subrelation $s \subseteq r$ such that the field of s is a cofinal subset in the preordered set $(\mathrm{Fld}(r), r^*|_{\mathrm{Fld}(r)})$.

The next lemma will be used to implement the step "B3" from subsection 5.1 (it can be considered as a restricted version of a theorem about DCR hierarchy presented in [20]).

Lemma 23 (DCR property inference using CCR residue argument).
If $n_0 \in [1,\omega)$, $r \subseteq U \times U$, $s \in \mathfrak{U}(r)$, and $s \in DCR_{n_0}(U)$, then $r \in DCR_{n_0+1}(U)$.

Proof.
A formal proof is available in [17, lines 7275–7598]. The proof involves extraction of an auxiliary spanning subrelation of r entitled r1 [17, line 7458, 7590], labeling r1 to show that $\texttt{r1} \in DCR_{n_0}(U)$ as described in [17, lines 7450–7453, 7588, 7591], and extension of a labeling of r1 to a labeling of r that ensures that $r \in DCR_{n_0+1}(U)$ [17, line 7597]. ◀

5.3 CCR relation subfield extension lemma

For any $r \subseteq U \times U$, $A \subseteq U$, and $x \in U$ let us denote:

■ $r\,[A] = \{b \mid \exists a \in A\ (a,b) \in r\}$, i.e. $r\,[A]$ is the image of A under r

■ $r\{x\} = \{y \mid (x,y) \in r\}$, i.e. $r\{x\}$ is the set of successors of x w.r.t. r

■ $\mathrm{dncl}(r,A) = ((r^*)^{-1})[A]$, i.e. $\mathrm{dncl}(r,A)$ is the downward closure of A in (U,r^*)

■ $Inv(r) = \{A \subseteq U \mid r\,[A] \subseteq A\ \}$, elements of $Inv(r)$ are called invariant sets of r

■ $Span(r) = \{s \subseteq r \mid \mathrm{Fld}(s) = \mathrm{Fld}(r)\}$, elements of $Span(r)$ are called spanning subrelations of r

■ $SF(r) = \{A \subseteq U \mid \mathrm{Fld}(r|_A) = A\ \}$, elements of $SF(r)$ are called subfields of r (note that A is a subfield of r if and only if A is the field of some subrelation $s \subseteq r$)

■ $SCF(r) = \{\, B \subseteq \mathrm{Fld}(r) \mid \forall a \in \mathrm{Fld}(r)\ \exists b \in B\ (a,b) \in r^*\}$, i.e. $SCF(r)$ is the set of all cofinal sets in the preordered set $(\mathrm{Fld}(r), r^*|_{\mathrm{Fld}(r)})$

■ $\mathrm{scf}(r) = \min\{\ |B| \mid B \in SCF(r)\ \}$, here $\mathrm{scf}(r)$ is called the sequential cofinality of r.

For any set X denote as $\mathcal{P}(X)$ the set of all subsets of X.

Let us fix a (choice) function $\mathrm{ch} : \mathcal{P}(\mathcal{P}(U))\backslash\{\emptyset\} \to \mathcal{P}(U)$ such that $\mathrm{ch}(A) \in A$ for every nonempty set $A \subseteq \mathcal{P}(U)$.

For any $r, s' \subseteq U \times U$, $A, B \subseteq U$, and $a, b \in U$ let us denote:

■ $\mathcal{F}(r,a,b)$ is the set of all (finite) subsets $F \subseteq U$ such that there exists a non-negative integer n and a finite sequence $(a_i)_{i=0}^n$ of elements of U such that $F = \{a_0, a_1, ..., a_n\}$, $a_0 = a$, $a_n = b$, and $(a_i, a_{i+1}) \in r$ for all $i = 0, 1, ..., n-1$

■ $\mathrm{wdncl}(r,A) = \{a \in \mathrm{dncl}(r,A) \mid \forall b \in U\ \forall F \in \mathcal{F}(r,a,b)\ (b \notin \mathrm{dncl}(r,A) \Rightarrow F \cap A \neq \emptyset)\}$

■ $\mathrm{wrank}(r,A) = \min\{\ |B| \mid A \subseteq \mathrm{wdncl}(r,B)\ \}$

■ $\mathrm{dn}_{\mathrm{Esc}}(r,A,a) = \{\ F \in \mathcal{P}(U) \mid \exists b \in U\backslash\mathrm{dncl}(r,A)\ F \in \mathcal{F}(r,a,b) \wedge (F \cap A = \emptyset)\ \}$

■ $\mathrm{dn}_{\mathrm{esc}}(r,A,a) = \begin{cases} \mathrm{ch}(\mathrm{dn}_{\mathrm{Esc}}(r,A,a)), & \text{if } \mathrm{dn}_{\mathrm{Esc}}(r,A,a) \neq \emptyset \\ \{a\}, & \text{if } \mathrm{dn}_{\mathrm{Esc}}(r,A,a) = \emptyset \end{cases}$

■ $\mathrm{escl}(r,A,B) = \bigcup_{b \in B} \mathrm{dn}_{\mathrm{esc}}(r,A,b)$

■

$\mathrm{clterm}(s', r) = (s' \in Conelike(U) \Rightarrow r \in Conelike(U))$.

Lemma 24.

Let $r \in CCR(U)$, $A \in SF(r)$, $B, B' \subseteq U$, $x \in U$, and $Ps \subseteq SCF(r)$.

Then there exists a set $A' \subseteq U$ that satisfies of the following 9 conditions:

1. $x \in \mathrm{Fld}(r) \Rightarrow x \in A'$
2. $A \subseteq A'$
3. $A' \in SF(r)$
4. $\forall a \in A\ ((r\{a\} \subseteq B)\ \vee\ (r\{a\} \cap (A' \backslash B) \neq \emptyset))$
5. $(\exists y \in U\ A' - B' \subseteq \{y\}) \Rightarrow (\mathrm{Fld}(r) \subseteq A' \cup B')$
6. $r|_{A'} \in CCR(U)$
7. $(|A| < \omega \Rightarrow |A'| < \omega) \wedge (|A| \geq \omega \Rightarrow |A'| = |A|)$
8. $((\exists P\ Ps = \{P\})\ \vee\ |Ps| \in [\omega, |A|\,]) \Rightarrow (\forall P \in Ps\ (A' \cap P) \in SCF(r|_{A'}))$
9. $|A| \geq \omega \Rightarrow \mathrm{escl}(r, A, A') \subseteq A' \wedge \mathrm{clterm}(r|_{A'}, r)$.

Proof.

A formal proof is available in [17, lines 4498–4538]. It depends on several auxiliary formal lemmas that have close formulations, but provide weaker/different guarantees (lem_Ccext_finsubccr_pext5_scf3 and lem_Ccext_infsubccr_pext5_scf3 in [17] and their dependencies). The idea of the proof of the current lemma and its dependencies is related to Löwenheim-Skolem theorem.

When A is infinite, the proof aims to define a required set A' as

$$\psi\left(\bigcup_{n=0}^{\infty} H^{(n)}(\phi(A))\right),$$

(2) for some suitable increasing set function H and set functions ϕ, ψ, where $H^{(n)}$ denotes the n-times composition of H with itself, such that H, ϕ, ψ ensure closure properties of A' required by the lemma's conditions (e.g. $r|_{A'} \in CCR(U)$), but keep the cardinalities of sets defined by subexpressions of (2) bounded, so that the condition 7, i.e. $(|A| < \omega \Rightarrow |A'| < \omega) \wedge (|A| \geq \omega \Rightarrow |A'| = |A|)$, holds. An explicit expression for $\phi(A)$ can be obtained by unfolding the notation on the right hand side of the definition in [17, line 3972], taking into account substitutions of values in place of parameters of the auxiliary lemma lem_Ccext_infsubccr_set_ext_scf3 specified in [17, line 4280]. Similarly, H can be expressed by unfolding the notation used in [17, lines 3964–3966], and as ψ one can take the identity function.

If A is a finite set, the proof is based on induction on the size of an auxiliary finite relation associated with A. Details can be found in [17, lines 4372–4496, 3332–3414, 2485–2680]. ◀

5.4 Analog of Iwamura's lemma on directed sets for CCR relations

In this subsection we describe a lemma that allows one to represent an infinite CCR relation r as a union of a chain of CCR relations of cardinalities smaller than the cardinality of r in a way reminiscent to Iwamura's lemma on directed sets [29, Theorem 1]. Unlike Iwamura's lemma, our lemma is applicable to a not-necessarily-transitive CCR relation r and guarantees that fields of elements of the

resulting chain satisfy additional constraints described below using the sets $\mathcal{N}_i$ that are used in the proof of the main result.

Let $\mathcal{U}$ be the set of all ordinals of the cardinality at most $|U|$.

For any $f : \mathcal{U} \to 2^U$ and ordinal $\alpha \in \mathcal{U}$ let us denote:

■ $\mathfrak{L}(f, \alpha) = \bigcup_{\alpha' \in [0,\alpha)} f(\alpha')$

■ $\nabla(f, \alpha) = f(\alpha) \backslash \mathfrak{L}(f, \alpha)$

■ $\mathcal{Q}(r, f, \alpha) = f(\alpha) \backslash \mathrm{dncl}(r, \mathfrak{L}(f, \alpha))$

■ $\mathcal{W}(r, f, \alpha) = f(\alpha) \backslash \mathrm{wdncl}(r, \mathfrak{L}(f, \alpha))$.

For any $r \subseteq U \times U$, $Ps \subseteq 2^U$, and ordinal $\alpha_0 \in \mathcal{U}$ let us denote:

■ $\mathcal{N}_1(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]\ \forall \alpha' \in [0, \alpha]\ f(\alpha') \subseteq f(\alpha)\ \}$

■ $\mathcal{N}_2(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]_L\ \nabla(f, \alpha) = \emptyset\ \}$

■ $\mathcal{N}_3(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]_{0S}\ \ |\mathfrak{L}(f, \alpha)| \geq \omega \Rightarrow$
$(\ \mathrm{escl}(r, \mathfrak{L}(f, \alpha), f(\alpha)) \subseteq f(\alpha)\ \wedge\ \mathrm{clterm}(r|_{f(\alpha)}, r)\ )\ \}$

■ $\mathcal{N}_4(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]_{0S}\ \forall a \in \mathfrak{L}(f, \alpha)\ (\ r\{a\} \subseteq \mathrm{wdncl}(r, \mathfrak{L}(f, \alpha)) \vee (r\{a\} \cap \mathcal{W}(r, f, \alpha) \neq \emptyset)\ )\ \}$

■ $\mathcal{N}_5(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]\ f(\alpha) \in SF(r)\ \}$

■ $\mathcal{N}_6(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]\ r|_{f(\alpha)} \in CCR(U)\ \}$

■ $\mathcal{N}_7(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]\ \ (\ (\alpha < \omega \Rightarrow |f(\alpha)| < \omega) \wedge (\alpha \geq \omega \Rightarrow |f(\alpha)| \leq \alpha)\ )\ \}$

■ $\mathcal{N}_8(r, Ps, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]_{0S}\ \ (\ (|Ps| = 1 \vee |Ps| \in [\omega, |f(\alpha)|\,]) \Rightarrow (\forall P \in Ps\ (f(\alpha) \cap P) \in SCF(r|_{f(\alpha)}))\ )\ \}$

■ $\mathcal{N}_9(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \alpha_0 \geq \omega \Rightarrow \mathrm{Fld}(r) \subseteq f(\alpha_0)\ \}$

■ $\mathcal{N}_{10}(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]\ ((\ \exists y \in U\ \mathcal{Q}(r, f, \alpha) = \{y\}\ ) \Rightarrow \mathrm{Fld}(r) \subseteq \mathrm{dncl}(r, f(\alpha))\ )\ \}$

■ $\mathcal{N}_{11}(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [0, \alpha_0]_{0S} \backslash \{0\}\ (\mathcal{Q}(r, f, \alpha) = \emptyset \Rightarrow \mathrm{Fld}(r) \subseteq \mathrm{dncl}(r, f(\alpha))\ )\ \}$

■ $\mathcal{N}_{12}(r, \alpha_0) = \{\ f : \mathcal{U} \to 2^U \mid \forall \alpha \in [\omega, \alpha_0]\ |\mathfrak{L}(f, \alpha)| \geq \omega\ \}$.

For every $r \subseteq U \times U$ and $Ps \subseteq 2^U$ let us denote:

$$\mathcal{N}(r, Ps) = \left(\bigcap_{i=1}^{7} \mathcal{N}_i(r, |\mathrm{Fld}(r)|) \right) \cap \mathcal{N}_8(r, Ps, |\mathrm{Fld}(r)|) \cap \left(\bigcap_{i=9}^{12} \mathcal{N}_i(r, |\mathrm{Fld}(r)|) \right).$$

(3) Note that $|\mathrm{Fld}(r)| \in [0, |U|\,] \subseteq \mathcal{U}$, because $\mathrm{Fld}(r) \subseteq U$.

Lemma 25 (Analog of Iwamura's lemma).
If $r \in CCR(U)$ and $Ps \subseteq SCF(r)$, then $\mathcal{N}(r, Ps) \neq \emptyset$.

Proof.

A formal proof is available in [17, lines 9533–9616]. The idea of the proof is to use Lemma 24 to define recursively a transfinite sequence of subsets of $\mathrm{Fld}(r)$ that gives values $f(\alpha)$, $\alpha \in [0, |\mathrm{Fld}(r)|]$ for some $f \in \mathcal{N}(r, Ps)$. ◀

► **Remark 26.**
If $f \in \mathcal{N}(r, Ps)$ for an infinite CCR relation r, then the relations $r|_{f(\alpha)}$, $\alpha \in [0, |\mathrm{Fld}(r)|)$ are smaller than r in cardinality and form a chain of CCR relations, the union of which is r (one can check this by unfolding the notations $\mathcal{N}_i(r, |\mathrm{Fld}(r)|)$ for $i = 1, 2, 5, 6, 7, 9$ and $\nabla(f, \alpha)$ and noting that if r is infinite, then $|\mathrm{Fld}(r)|$ is a limit ordinal).

5.5 Generation of a CCR residue in the class DCR2

Let us denote:

■

$T(U)$ is the set of all nonempty relations $t \in CCR(U)$ such that t is single-valued (i.e. $(a, b) \in t \wedge (a, c) \in t \Rightarrow b = c$), acyclic (i.e. t^+ is irreflexive), and total on its field (i.e. $\forall x \in \mathrm{Fld}(t)\ t\{x\} \neq \emptyset$).

Lemma 27.
Let $r \subseteq U \times U$ be a relation that is reflexive on $\mathrm{Fld}(r)$. Let S be a set of ordinals and $(W_\alpha)_{\alpha \in S}$, $(R_\alpha)_{\alpha \in S}$ be indexed families of subsets of U such that

1.
$S \subseteq [0, \omega_1)$, $|[0, \omega_1)| \leq |S|$, and $\forall \alpha \in S\ \exists \beta \in S\ \alpha < \beta$

2.
$\mathrm{Fld}(r) = \bigcup_{\alpha \in S} W_\alpha$ and $\forall \alpha, \beta \in S\ (\alpha \neq \beta \Rightarrow W_\alpha \cap W_\beta = \emptyset)$

3.
$\forall \alpha \in S\ R_\alpha \in T(U) \wedge R_\alpha \subseteq r \wedge |W_\alpha| \leq \omega \wedge \mathrm{Fld}(R_\alpha) = W_\alpha \wedge \neg Conelike(r|_{W_\alpha})$

4.
$\forall \alpha \in S\, \forall x \in W_\alpha\, \exists a \in U\ (\ (x, a) \in (r|_{W_\alpha})^* \wedge (\forall \beta \in S\ \alpha < \beta \Rightarrow r\{a\} \cap W_\beta \neq \emptyset)\ )$.

Then there exists $r' \in \mathfrak{U}(r)$ such that $r' \in DCR_2(U)$.

Proof.
A formal proof is available in [17, lines 4935–5746] (note that in the formulation of the lemma in [17] the assumptions are not numbered as above and the notation $\mathfrak{U}(r)$ in the conclusion is unfolded).

The idea of the proof is to define a relation $r' \in \mathfrak{U}(r)$ (a subrelation of r) such that, semi-formally, the graph $(\mathrm{Fld}(r'), r')$ has a shape similar to the one illustrated in Figure 1, and show that $r' \in DCR_2(U)$. The trees illustrated in Figure 1 are extracted from (at most countable) graphs $(W_\alpha, r|_{W_\alpha})$ for ordinals $\alpha \in S$.

In more detail this construction is described below (the relations R'_α correspond to sets of arcs of the trees extracted from $(W_\alpha, r|_{W_\alpha})$, the relation r'_e corresponds to the set of arcs that connect trees, and r' is the union of r'_e and R'_α for all $\alpha \in S$).

For every $\alpha \in S$ let us denote:

■

$EP_\alpha = \{a \in W_\alpha \mid \forall \beta \in S\ (\alpha < \beta \Rightarrow r\{\alpha\} \cap W_\beta \neq \emptyset)\}$, the elements of EP_α are called exit points from the ARS $(W_\alpha, r|_{W_\alpha})$

■

$h : \mathcal{U} \to \mathcal{U}$ is a function such that for every $\alpha, \beta \in S$ and $a \in EP_\alpha$, $b \in EP_\beta$ there exists $\gamma \in S$ and $a', b' \in W_\gamma$ such that $\alpha < \gamma$, $\beta < \gamma$, and $(a, a') \in r$, $(a', h(\gamma)) \in R^*_\gamma$, $(b, b') \in r$, and $(b', h(\gamma)) \in R^*_\gamma$

■

$(s_{\alpha,k})_{k=0}^\infty$ is some infinite cofinal reduction sequence in the (countable, CCR) ARS $(\mathrm{Fld}(R_\alpha), R_\alpha)$

■

$E_\alpha = \{(s_{\alpha,k+1}, s_{\alpha,k}) \notin (r|_{W_\alpha})^*\}$

■

$F_\alpha = \{s_{\alpha,k} \mid k \in E_\alpha\} \cap \{a \in W_\alpha \mid (h(\alpha) \in W_\alpha \Rightarrow (h(\alpha), a) \in R^*_\alpha) \wedge (a, h(\alpha)) \notin R^*_\alpha\}$

■

$re_{0,\alpha}$ and $re_{1,\alpha}$ is some pair of consecutive elements in the sequence $(s_{\alpha,k})_{k=0}^{\infty}$ such that $re_{0,\alpha}$ is located at some index $k \in E_\alpha$ in $(s_{\alpha,k})_{k=0}^{\infty}$ and $re_{0,\alpha} \in F_\alpha$

■

ep_α is some element of EP_α such that $(re_{1,\alpha}, ep_\alpha) \in (r|_{W_\alpha})^*$

■

spl_α is the length of a shortest reduction sequence in the ARS $(W_\alpha, r|_{W_\alpha})$ that starts at the element $re_{1,\alpha}$ and ends at the element ep_α

■

$sp_{\alpha,k}$ for $k = 0, 1, ..., (spl_\alpha - 1)$ is some reduction sequence that starts at the element $re_{1,\alpha}$, ends at the element ep_α, and has the length spl_α

■

$R'_\alpha = \{(a, b) \in R_\alpha \mid (b, re_{0,\alpha}) \in R^*_\alpha\} \cup \{(re_{0,\alpha}, re_{1,\alpha})\} \cup \{(sp_{\alpha,k}, sp_{\alpha,k+1}) \mid k < spl_\alpha\}$

■

$r'_e = \{(a, b) \in r \mid \exists \alpha, \beta \in S\ \alpha < \beta \wedge a = ep_\alpha \wedge b \in W_\beta \wedge (b, h(\beta)) \in R^*_\beta\}$

■

$r' = r'_e \cup \bigcup_{\alpha \in S} R'_\alpha$

■

$g : [0, \omega) \to 2^{U \times U}$ is a function such that for every $n \in [0, \omega)$:

$$g(n) = \begin{cases} \{(u, v) \in r' \mid r'\{u\} = \{v\}\}, & \text{if } n = 0 \\ \{(u, v) \in r' \mid r'\{u\} \neq \{v\}\}, & \text{if } n = 1 \\ \emptyset, & \text{if } n \notin \{0, 1\}. \end{cases}$$

Then it is shown that g is DCR-generating [17, lines 5659–5720] and $r' \in \mathfrak{U}(r)$. Then $r' \in \mathfrak{U}(r) \cap DCR_2(U)$, because $r' = \bigcup_{n<2} g(n)$. ◀

5.6 Completion of the proof of the main theorem

Below we give an important lemma (Lemma 28) that can be used to describe a structure of a confluent relation r when $\mathrm{scf}(r)$ is a regular cardinal. We will use it together with several other lemmas to complete the proof of Theorem 18.

Lemma 28.

Let $r \in CCR(U)$ be a relation such that r reflexive on $\mathrm{Fld}(r)$. Assume that $\mathrm{scf}(r)$ is a regular cardinal [3] such that $\omega < \mathrm{scf}(r) = |\mathrm{Fld}(r)|$.

Then there exists a set Ps and a function $f \in \mathcal{N}(r, Ps)$ such that for every successor ordinal $\alpha \in [0, |\mathrm{Fld}(r)|)$ such that $|\mathcal{L}(f, \alpha)| \geq \omega$ the following conditions 1–3 hold:

1.

$r|_{\mathcal{W}(r,f,\alpha)} \in CCR(U)$

2.

$|r|_{\mathcal{W}(r,f,\alpha)}| < |\mathrm{Fld}(r)|$

3.

for every $a \in \mathcal{W}(r, f, \alpha)$ there exists $b \in U$ (denoted in [17] as "wesc r f α a") such that $b \in \mathcal{W}(r, f, \alpha)$, $(a, b) \in (r|_{\mathcal{W}(r,f,\alpha)})^*$, and $r\{b\} \cap \mathcal{W}(r, f, \beta) \neq \emptyset$ for every successor ordinal β such that $\alpha < \beta < |\mathrm{Fld}(r)|$.

Proof.

A formal proof is available in [17, lines 10415–10460].

The idea of the proof is to define $Ps = \{P\}$, where

$$P = \{\ a \in \mathrm{Fld}(r) \mid \mathrm{scf}(r) \leq \mathrm{wrank}(r, r\{a\})\ \},$$

then obtain $f \in \mathcal{N}(r, Ps)$ using Lemma 25, and show that f satisfies the conditions 1–3. ◀

Lemma 29.
Let $r \subseteq U \times U$ be a relation that is reflexive on $\mathrm{Fld}(r)$.
Assume that $r \in CCR(U)$ and $\mathrm{scf}(r) = |\mathrm{Fld}(r)| = \omega_1$. Then $r \in DCR_3(U)$.

Proof.
A formal proof is available in [17, lines 10733–11021].
The proof proceeds as follows. Let Ps and $f \in \mathcal{N}(r, Ps)$ be a set and a function obtained from Lemma 28 applied to r. Let us denote:

■
S is the set of all successor ordinals in $[\omega, \omega_1)$

■
$(W_\alpha)_{\alpha \in S}$ is an indexed family of subsets of U such that $W_\alpha = \mathcal{W}(r, f, \alpha)$ for $\alpha \in S$

■
$(R_\alpha)_{\alpha \in S}$ is an indexed family of relations on U such that

$$R_\alpha \in Span(r|_{\mathcal{W}(r,f,\alpha)}) \cap T(U) \text{ for } \alpha \in S$$

■
$r_1 = r|_{\bigcup_{\alpha \in S} W_\alpha}$.
Let r' be a relation in $DCR_2(U)$ obtained using Lemma 27 applied to r_1, S, $(W_\alpha)_{\alpha \in S}$, $(R_\alpha)_{\alpha \in S}$. One can show that $r' \in \mathfrak{U}(r)$, so $r \in DCR_3(U)$ by Lemma 23. ◀

Lemma 30.
If $r \in CCR(U)$ and $\mathrm{scf}(r) \leq \omega$, then $r \in DCR_2(U)$.

Proof.
A formal proof is available in [17, lines 10687–10731]. The idea of the proof is similar to the idea of [9, Theorem 19]. ◀

Lemma 31.
If $r \in CCR(U)$ and $|\mathrm{Fld}(r)| \leq \omega_1$, then $r \in DCR_3(U)$.

Proof.
A formal proof is available in [17, lines 11023–11072].
The proof proceeds as follows. Consider two cases.
1.
If $\mathrm{scf}(r) \leq \omega$, then $r \in DCR_2(U)$ by Lemma 30, whence $r \in DCR_3(U)$.
2.
Assume $\mathrm{scf}(r) = \omega_1$. Let $r_1 = r \cup \{(x, x) \mid x \in \mathrm{Fld}(r)\}$. Then either $\mathrm{scf}(r_1) \leq \omega$, or $\mathrm{scf}(r_1) = |\mathrm{Fld}(r_1)|$. If $\mathrm{scf}(r_1) \leq \omega$, then $r_1 \in DCR_2(U)$ by Lemma 30. If $\mathrm{scf}(r_1) = |\mathrm{Fld}(r_1)|$, then $r_1 \in DCR_3(U)$ by Lemma 29. Thus in any case, $r_1 \in DCR_3(U)$. This is used to conclude that $r \in DCR_3(U)$.

◀

Lemma 32.
If $r \subseteq U \times U$ is a confluent relation and $|\mathrm{Fld}(r)| \leq \omega_1$, then $r \in DCR_3(U)$.

Proof.
A formal proof is available in [17, lines 11634–11652].
The idea of the proof is as follows. Consider (U, r) as a directed graph. For each connected component (C, s) of (U, r), the relation s satisfies the assumptions of Lemma 31, so $s \in DCR_3(U)$. This is used to conclude that $r \in DCR_3(U)$. ◀

Proof of Theorem 18 (i.e. if (U, r) is confluent and $|r| \leq \omega_1$, then (U, r) is in DCR_3).

A formal proof of Theorem 18 is available in [17, lines 11760–11807]. It follows from Lemma 32 by observing that $|r| \leq \omega_1$ implies $|\mathrm{Fld}(r)| \leq \omega_1$. ◀

Complete author formal proof supplement DCRN1.thy

Source: Ievgen Ivanov, Zenodo record 14254256, CC BY 4.0. This is the complete unchanged author file underlying the line references in the paper. No Isabelle rerun is claimed.

```
(*
This formalization continues and extends the results of the paper:
I. Ivanov. "On Non-triviality of the Hierarchy of Decreasing Church-Rosser Abstract Rewriting Systems",
      International Workshop on Confluence (IWC 2024), 2024, pp. 30-35.

Main result in this file (thm_main at line 11760):
if the cardinality of a confluent binary relation r does not exceed the first uncountable cardinal,
then confluence of r can be proved with the help of the decreasing diagrams method using no more than 3 labels (
      e.g. 0, 1, 2 ordered in the usual way).

This file can be checked using Isabelle 2024 proof assistant (HOL logic).
*)

theory DCRN1
imports Main "HOL-Cardinals.Cardinals"
begin

(* ------------------------------------------------------------------------ *)

section \<open>Auxiliary definitions\<close>

(* ------------------------------------------------------------------------ *)

definition jn00 :: "'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<Rightarrow> bool"
where
  "jn00 r0 b c \<equiv> (\<exists> d. (b,d) \<in> r0^= \<and> (c,d) \<in> r0^=)"

definition jn01 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<Rightarrow> bool"
where
  "jn01 r0 r1 b c \<equiv> (\<exists>b' d. (b,b') \<in> r1^= \<and> (b',d) \<in> r0^* \<and> (c,d) \<in> r0^*)"

definition jn10 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<Rightarrow> bool"
where
  "jn10 r0 r1 b c \<equiv> (\<exists>c' d. (b,d) \<in> r0^* \<and> (c,c') \<in> r1^= \<and> (c',d) \<in> r0^*)"

definition jn11 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<Rightarrow> bool"
where
  "jn11 r0 r1 b c \<equiv> (\<exists>b' b'' c' c'' d.  (b,b') \<in> r0^* \<and> (b',b'') \<in> r1^= \<and> (b'',
      d) \<in> r0^*
                                        \<and> (c,c') \<in> r0^* \<and> (c',c'') \<in> r1^= \<and> (c'',d) \<
      in> r0^*)"

definition jn02 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<
      Rightarrow> bool"
where
  "jn02 r0 r1 r2 b c \<equiv> (\<exists>b' d.  (b,b') \<in> r2^= \<and> (b',d) \<in> (r0 \<union> r1)^* \<and> (
      c,d) \<in> (r0 \<union> r1)^* )"

definition jn12 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<
      Rightarrow> bool"
where
  "jn12 r0 r1 r2 b c \<equiv> (\<exists>b' b'' d.  (b,b') \<in> (r0)^* \<and> (b',b'') \<in> r2^= \<and> (b'',d)
       \<in> (r0 \<union> r1)^*
                                        \<and> (c,d) \<in> (r0 \<union> r1)^*)"

definition jn22 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a \<Rightarrow> 'a \<
      Rightarrow> bool"
where
  "jn22 r0 r1 r2 b c \<equiv> (\<exists>b' b'' c' c'' d.  (b,b') \<in> (r0 \<union> r1)^* \<and> (b',b'') \<in>
      r2^= \<and> (b'',d) \<in> (r0 \<union> r1)^*
                                        \<and> (c,c') \<in> (r0 \<union> r1)^* \<and> (c',c'') \<in> r2^= \<
      and> (c'',d) \<in> (r0 \<union> r1)^*)"

definition LD2 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a rel \<Rightarrow> bool"
where
  "LD2 r r0 r1 \<equiv> ( r = r0 \<union> r1
              \<and> (\<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r0 \<longrightarrow> jn00 r0 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r1 \<longrightarrow> jn01 r0 r1 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r1 \<and> (a,c) \<in> r1 \<longrightarrow> jn11 r0 r1 b c) )"

definition LD3 :: "'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a rel \<Rightarrow> 'a rel \<Rightarrow> bool"
where
```

```
  "LD3 r r0 r1 r2 \<equiv> ( r = r0 \<union> r1 \<union> r2
              \<and> (\<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r0 \<longrightarrow> jn00 r0 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r1 \<longrightarrow> jn01 r0 r1 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r1 \<and> (a,c) \<in> r1 \<longrightarrow> jn11 r0 r1 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r2 \<longrightarrow> jn02 r0 r1 r2 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r1 \<and> (a,c) \<in> r2 \<longrightarrow> jn12 r0 r1 r2 b c)
              \<and> (\<forall> a b c. (a,b) \<in> r2 \<and> (a,c) \<in> r2 \<longrightarrow> jn22 r0 r1 r2 b c)
      )"

definition DCR2 :: "'a rel \<Rightarrow> bool"
where
  "DCR2 r \<equiv> ( \<exists>r0 r1. LD2 r r0 r1 )"

definition DCR3 :: "'a rel \<Rightarrow> bool"
where
  "DCR3 r \<equiv> ( \<exists>r0 r1 r2. LD3 r r0 r1 r2 )"

definition \<LL>1 :: "(nat \<Rightarrow> 'U rel) \<Rightarrow> nat \<Rightarrow> 'U rel"
where
  "\<LL>1 g \<alpha> \<equiv> \<Union> {A. \<exists> \<alpha>'. (\<alpha>' < \<alpha>) \<and> A = g \<alpha>'}"

definition \<LL>v :: "(nat \<Rightarrow> 'U rel) \<Rightarrow> nat \<Rightarrow> nat \<Rightarrow> 'U rel"
where
  "\<LL>v g \<alpha> \<beta> \<equiv> \<Union> {A. \<exists> \<alpha>'. (\<alpha>' < \<alpha> \<or> \<alpha>' <
      \<beta>) \<and> A = g \<alpha>'}"

definition \<DD> :: "(nat \<Rightarrow> 'U rel) \<Rightarrow> nat \<Rightarrow> nat \<Rightarrow> ('U \<times> '
      U \<times> 'U \<times> 'U) set"
where
  "\<DD> g \<alpha> \<beta> = {(b,b',b'',d). (b,b') \<in> (\<LL>1 g \<alpha>)^* \<and> (b',b'') \<in> (g \<beta
      >)^= \<and> (b'',d) \<in> (\<LL>v g \<alpha> \<beta>)^*}"

definition DCR_generating :: "(nat \<Rightarrow> 'U rel) \<Rightarrow> bool"
where
  "DCR_generating g \<equiv> (\<forall> \<alpha> \<beta> a b c. (a,b) \<in> (g \<alpha>) \<and> (a,c) \<in> (g
      \<beta>)
             \<longrightarrow>  (\<exists> b' b'' c' c'' d. (b,b',b'',d) \<in> (\<DD> g \<alpha> \<beta>) \<and>
       (c,c',c'',d) \<in> (\<DD> g \<beta> \<alpha>) ))"

definition DCR :: "nat \<Rightarrow> 'U rel \<Rightarrow> bool"
where
  "DCR n r \<equiv> (\<exists> g::(nat \<Rightarrow> 'U rel). DCR_generating g \<and> r = \<Union> { r'. \<
      exists> \<alpha>'. \<alpha>' < n \<and> r' = g \<alpha>' } )"

definition confl_rel
  where "confl_rel r \<equiv> (\<forall>a b c. (a,b) \<in> r^* \<and> (a,c) \<in> r^* \<longrightarrow> (\<
      exists> d. (b,d) \<in> r^* \<and> (c,d) \<in> r^*) )"

abbreviation \<omega>_ord where "\<omega>_ord \<equiv> natLeq"

definition sc_ord::"'U rel \<Rightarrow> 'U rel \<Rightarrow> bool"
  where "sc_ord \<alpha> \<alpha>' \<equiv> (\<alpha> <o \<alpha>' \<and> (\<forall> \<beta>::'U rel. \<alpha> <
      o \<beta> \<longrightarrow> \<alpha>' \<le>o \<beta>))"

definition lm_ord::"'U rel \<Rightarrow> bool"
  where "lm_ord \<alpha> \<equiv> Well_order \<alpha> \<and> \<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)"

definition nord :: "'U rel \<Rightarrow> 'U rel" where "nord \<alpha> = (SOME \<alpha>'::'U rel. \<alpha>' =o \<
      alpha>)"

definition \<O>::"'U rel set" where "\<O> \<equiv> nord ` {\<alpha>. Well_order \<alpha>}"

definition oord::"'U rel rel" where "oord \<equiv> (Restr ordLeq \<O>)"

definition CCR :: "'U rel \<Rightarrow> bool"
where
  "CCR r = (\<forall>a \<in> Field r. \<forall>b \<in> Field r. \<exists>c \<in> Field r. (a,c) \<in> r^* \<and>
       (b,c) \<in> r^*)"

definition Conelike :: "'U rel \<Rightarrow> bool"
where
  "Conelike r = (r = {} \<or> (\<exists> m \<in> Field r. \<forall> a \<in> Field r. (a,m) \<in> r^*))"

definition dncl :: "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set"
where
  "dncl r A = ((r^*)^-1)``A"

definition Inv :: "'U rel \<Rightarrow> 'U set set"
```

```
where
  "Inv r = { A :: 'U set . r `` A \<subseteq> A }"

definition SF :: "'U rel \<Rightarrow> 'U set set"
where
  "SF r = { A :: 'U set. Field (Restr r A) = A }"

definition SCF::"'U rel \<Rightarrow> ('U set) set" where
  "SCF r \<equiv> { B::('U set) . B \<subseteq> Field r \<and> (\<forall> a \<in> Field r. \<exists> b \<in> B.
      (a,b) \<in> r^*) }"

definition cfseq :: "'U rel \<Rightarrow> (nat \<Rightarrow> 'U) \<Rightarrow> bool"
where
  "cfseq r xi \<equiv> ((\<forall> a \<in> Field r. \<exists> i. (a, xi i) \<in> r^*) \<and> (\<forall> i. (xi i
      , xi (Suc i)) \<in> r))"

definition rpth :: "'U rel \<Rightarrow> 'U \<Rightarrow> 'U \<Rightarrow> nat \<Rightarrow> (nat \<Rightarrow>
      'U) set"
where
  "rpth r a b n \<equiv> { f::(nat \<Rightarrow> 'U). f 0 = a \<and> f n = b \<and> (\<forall>i<n. (f i, f(Suc i
      )) \<in> r) }"

definition \<F> :: "'U rel \<Rightarrow> 'U \<Rightarrow> 'U \<Rightarrow> 'U set set"
where
  "\<F> r a b \<equiv> { F::'U set. \<exists> n::nat. \<exists> f \<in> rpth r a b n. F = f`{i. i\<le>n} }"

definition \<ff> :: "'U rel \<Rightarrow> 'U \<Rightarrow> 'U \<Rightarrow> 'U set"
where
  "\<ff> r a b \<equiv> (if (\<F> r a b \<noteq> {}) then (SOME F. F \<in> \<F> r a b) else {})"

definition dnEsc ::  "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U \<Rightarrow> 'U set set"
where
  "dnEsc r A a \<equiv> { F. \<exists> b. ((b \<notin> dncl r A) \<and> (F \<in> \<F> r a b) \<and> (F \<inter>
      A = {})) }"

definition dnesc ::  "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U \<Rightarrow> 'U set"
where
  "dnesc r A a = (if (dnEsc r A a \<noteq> {}) then (SOME F. F \<in> dnEsc r A a) else { a })"

definition escl ::  "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set \<Rightarrow> 'U set"
where
  "escl r A B = \<Union> ((dnesc r A) ` B)"

definition clterm where "clterm s' r \<equiv> (Conelike s' \<longrightarrow> Conelike r)"

definition spthlen::"'U rel \<Rightarrow> 'U \<Rightarrow> 'U \<Rightarrow> nat"
where
  "spthlen r a b \<equiv> (LEAST n::nat. (a,b) \<in> r^^n)"

definition spth :: "'U rel \<Rightarrow> 'U \<Rightarrow> 'U \<Rightarrow> (nat \<Rightarrow> 'U) set"
where
  "spth r a b = rpth r a b (spthlen r a b)"

definition \<UU>::"'U rel \<Rightarrow> ('U rel) set" where
  "\<UU> r \<equiv> { s::('U rel) . CCR s \<and> s \<subseteq> r \<and> (\<forall> a \<in> Field r. \<exists> b
      \<in> Field s. (a,b) \<in> r^*) }"

definition RCC_rel :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> bool" where
  "RCC_rel r \<alpha> \<equiv> (\<UU> r = {} \<and> \<alpha> = {}) \<or> (\<exists> s \<in> \<UU> r. |s| =o \<
      alpha> \<and> ( \<forall> s' \<in> \<UU> r. |s| \<le>o |s'| ))"

definition RCC :: "'U rel \<Rightarrow> 'U rel" ("\<parallel>_\<parallel>")
  where "\<parallel>r\<parallel> \<equiv> (SOME \<alpha>. RCC_rel r \<alpha>)"

definition Den::"'U rel \<Rightarrow> ('U set) set" where
  "Den r \<equiv> { B::('U set) . B \<subseteq> Field r \<and> (\<forall> a \<in> Field r. \<exists> b \<in> B.
      (a,b) \<in> r^=) }"

definition Span::"'U rel \<Rightarrow> ('U rel) set" where
  "Span r \<equiv> { s. s \<subseteq> r \<and> Field s = Field r }"

definition scf_rel :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> bool" where
  "scf_rel r \<alpha> \<equiv> (\<exists> B \<in> SCF r. |B| =o \<alpha> \<and> ( \<forall> B' \<in> SCF r. |B|
      \<le>o |B'| ))"

definition scf :: "'U rel \<Rightarrow> 'U rel"
  where "scf r \<equiv> (SOME \<alpha>. scf_rel r \<alpha>)"
```

```
definition w_dncl :: "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set"
where
  "w_dncl r A = { a \<in> dncl r A. \<forall> b. \<forall> F \<in> \<F> r a b. ( b \<notin> dncl r A \<
      longrightarrow> F \<inter> A \<noteq> {} ) }"

definition \<LL> :: "('U rel \<Rightarrow> 'U set) \<Rightarrow> 'U rel \<Rightarrow> 'U set"
where
  "\<LL> f \<alpha> \<equiv> \<Union> {A. \<exists> \<alpha>'. \<alpha>' <o \<alpha> \<and> A = f \<alpha>'}"

definition Dbk :: "('U rel \<Rightarrow> 'U set) \<Rightarrow> 'U rel \<Rightarrow> 'U set" (" \<nabla> _ _ ")
where
  "\<nabla> f \<alpha> \<equiv> f \<alpha> - (\<LL> f \<alpha>)"

definition \<Q> :: "'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) \<Rightarrow> 'U rel \<Rightarrow> 'U set
      "
where
  "\<Q> r f \<alpha> \<equiv> (f \<alpha> - (dncl r (\<LL> f \<alpha>)))"

definition \<W> :: "'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) \<Rightarrow> 'U rel \<Rightarrow> 'U set
      "
where
  "\<W> r f \<alpha> \<equiv> (f \<alpha> - (w_dncl r (\<LL> f \<alpha>)))"

definition \<N>1 :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>1 r \<alpha>0 \<equiv> { f . \<forall>\<alpha> \<alpha>'. ( \<alpha> \<le>o \<alpha>0 \<and> \<alpha>' \<
      le>o \<alpha> ) \<longrightarrow> (f \<alpha>') \<subseteq> (f \<alpha>) }"

definition \<N>2:: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>2 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. ( \<alpha> \<le>o \<alpha>0 \<and> \<not> (\<alpha> = {}
      \<or> isSuccOrd \<alpha>) ) \<longrightarrow> (\<nabla> f \<alpha>) = {} }"

definition \<N>3:: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>3 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. ( \<alpha> \<le>o \<alpha>0 \<and> (\<alpha> = {} \<or>
      isSuccOrd \<alpha>) ) \<longrightarrow>
      ( \<omega>_ord \<le>o |\<LL> f \<alpha>| \<longrightarrow> ((escl r (\<LL> f \<alpha>) (f \<alpha>) \<
      subseteq> (f \<alpha>)) \<and> (clterm (Restr r (f \<alpha>)) r)) ) }"

definition \<N>4:: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>4 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. ( \<alpha> \<le>o \<alpha>0 \<and> (\<alpha> = {} \<or>
      isSuccOrd \<alpha>) ) \<longrightarrow>
         ( \<forall>a \<in> (\<LL> f \<alpha>). ( r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>) ) \<or> ( r``{a
      } \<inter> (\<W> r f \<alpha>)\<noteq>{} ) ) }"

definition \<N>5 :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>5 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. \<alpha> \<le>o \<alpha>0 \<longrightarrow> (f \<alpha>)
      \<in> SF r }"

definition \<N>6 :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>6 r \<alpha>0 \<equiv> { f. \<forall>\<alpha>. \<alpha> \<le>o \<alpha>0 \<longrightarrow> CCR (Restr r (
      f \<alpha>)) }"

definition \<N>7 :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>7 r \<alpha>0 \<equiv> { f. \<forall>\<alpha>. \<alpha> \<le>o \<alpha>0 \<longrightarrow> ( \<alpha> <o
      \<omega>_ord \<longrightarrow> |f \<alpha>| <o \<omega>_ord ) \<and> (\<omega>_ord \<le>o \<alpha> \<
      longrightarrow> |f \<alpha>| \<le>o \<alpha>) }"

definition \<N>8 :: "'U rel \<Rightarrow> 'U set set \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U
       set) set"
where
  "\<N>8 r Ps \<alpha>0 \<equiv> { f. \<forall>\<alpha>. \<alpha> \<le>o \<alpha>0 \<and> (\<alpha> = {} \<or>
      isSuccOrd \<alpha>) \<and> ( (\<exists> P. Ps = {P}) \<or> (\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha
      >| )) \<longrightarrow>
                          (\<forall> P \<in> Ps. ((f \<alpha>) \<inter> P) \<in> SCF (Restr r (f \<alpha>))) }"

definition \<N>9 :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>9 r \<alpha>0 \<equiv> { f . \<omega>_ord \<le>o \<alpha>0 \<longrightarrow> Field r \<subseteq> (f \<
      alpha>0) }"

definition \<N>10 :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
```

```
  "\<N>10 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. \<alpha> \<le>o \<alpha>0 \<longrightarrow> ((\<exists>
      y::'U. \<Q> r f \<alpha> = {y}) \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>))) }"

definition \<N>11:: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>11 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. ( \<alpha> \<le>o \<alpha>0 \<and> isSuccOrd \<alpha>)
      \<longrightarrow> \<Q> r f \<alpha> = {} \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>)) }"

definition \<N>12:: "'U rel \<Rightarrow> 'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N>12 r \<alpha>0 \<equiv> { f . \<forall>\<alpha>. \<alpha> \<le>o \<alpha>0 \<longrightarrow> \<omega>_ord
       \<le>o \<alpha> \<longrightarrow> \<omega>_ord \<le>o |\<LL> f \<alpha>| }"

definition \<N> :: "'U rel \<Rightarrow> 'U set set \<Rightarrow> ('U rel \<Rightarrow> 'U set) set"
where
  "\<N> r Ps \<equiv> { f \<in> (\<N>1 r |Field r| ) \<inter> (\<N>2 r |Field r| ) \<inter> (\<N>3 r |Field r| )
       \<inter> (\<N>4 r |Field r| )
       \<inter> (\<N>5 r |Field r| ) \<inter> (\<N>6 r |Field r| ) \<inter> (\<N>7 r |Field r| ) \<inter> (\<N
      >8 r Ps |Field r| )
       \<inter> (\<N>9 r |Field r| \<inter> \<N>10 r |Field r| \<inter> \<N>11 r |Field r| \<inter> \<N>12 r |
      Field r| ).
       (\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>) }"

definition \<T> :: "('U rel \<Rightarrow> 'U set \<Rightarrow> 'U set) \<Rightarrow> ('U rel \<Rightarrow> 'U
      set) set"
where
  "\<T> F \<equiv> { f::'U rel \<Rightarrow> 'U set .
          f {} = {}
       \<and> (\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> =
      F \<alpha>0 (f \<alpha>0)))
       \<and> (\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<
      beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }))
       \<and> (\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>) }"

definition \<E>p where "\<E>p r Ps A A' \<equiv>
               (((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| ))
                     \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A') ))"

definition \<E> :: "'U rel \<Rightarrow> 'U \<Rightarrow> 'U set \<Rightarrow> 'U set set \<Rightarrow> 'U set
      set"
where
  "\<E> r a A Ps \<equiv> { A'.
           (a \<in> Field r \<longrightarrow> a \<in> A') \<and> A \<subseteq> A'
          \<and> ( |A| <o \<omega>_ord \<longrightarrow> |A'| <o \<omega>_ord ) \<and> ( \<omega>_ord \<le>o |A
      | \<longrightarrow> |A'| \<le>o |A| )
          \<and> (A \<in> SF r \<longrightarrow> (
                A' \<in> SF r
             \<and> CCR (Restr r A')
             \<and> ( \<forall> a\<in>A. (r``{a} \<subseteq> w_dncl r A) \<or> (r``{a} \<inter> (A'-w_dncl r
      A) \<noteq> {}) )
             \<and> ((\<exists> y. A' - dncl r A \<subseteq> {y}) \<longrightarrow> (Field r \<subseteq> (
      dncl r A')))
             \<and> \<E>p r Ps A A'
             \<and> ( \<omega>_ord \<le>o |A| \<longrightarrow> escl r A A' \<subseteq> A' \<and> clterm (
      Restr r A') r)) ) }"

definition wbase::"'U rel \<Rightarrow> 'U set \<Rightarrow> ('U set) set" where
  "wbase r A \<equiv> { B::'U set. A \<subseteq> w_dncl r B }"

definition wrank_rel :: "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U rel \<Rightarrow> bool" where
  "wrank_rel r A \<alpha> \<equiv> (\<exists> B \<in> wbase r A. |B| =o \<alpha> \<and> ( \<forall> B' \<in>
      wbase r A. |B| \<le>o |B'| ))"

definition wrank :: "'U rel \<Rightarrow> 'U set \<Rightarrow> 'U rel"
  where "wrank r A \<equiv> (SOME \<alpha>. wrank_rel r A \<alpha>)"

definition Mwn :: "'U rel \<Rightarrow> 'U rel \<Rightarrow> 'U set"
where
  "Mwn r \<alpha> = { a \<in> Field r. \<alpha> <o wrank r (r ``{a}) }"

definition Mwnm :: "'U rel \<Rightarrow> 'U set"
where
  "Mwnm r = { a \<in> Field r. \<parallel>r\<parallel> \<le>o wrank r (r ``{a}) }"

definition wesc_rel :: "'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) \<Rightarrow> 'U rel \<Rightarrow> 'U
       \<Rightarrow> 'U \<Rightarrow> bool"
where
```

```
  "wesc_rel r f \<alpha> a b \<equiv> ( b \<in> \<W> r f \<alpha> \<and> (a,b) \<in> (Restr r (\<W> r f \<alpha
      >))^*
    \<and> (\<forall>\<beta>. \<alpha> <o \<beta> \<and> \<beta> <o |Field r| \<and> (\<beta> = {} \<or>
      isSuccOrd \<beta>) \<longrightarrow> (r``{b} \<inter> (\<W> r f \<beta>) \<noteq> {})) )"

definition wesc :: "'U rel \<Rightarrow> ('U rel \<Rightarrow> 'U set) \<Rightarrow> 'U rel \<Rightarrow> 'U \<
      Rightarrow> 'U"
where
  "wesc r f \<alpha> a \<equiv> (SOME b. wesc_rel r f \<alpha> a b)"

definition is_ars :: "'a set \<Rightarrow> ('a \<Rightarrow> 'a \<Rightarrow> bool) \<Rightarrow> bool"
  where "is_ars X \<sigma> \<equiv> (\<forall> x y. \<sigma> x y \<longrightarrow> x \<in> X \<and> y \<in> X)"

definition is_confl_ars
  where "is_confl_ars X \<sigma> \<equiv> is_ars X \<sigma> \<and>
         (\<forall>a\<in>X. \<forall>b\<in>X. \<forall>c\<in>X. (\<sigma>^** a b \<and> \<sigma>^** a c \<
      longrightarrow> (\<exists>d\<in>X. \<sigma>^** b d \<and> \<sigma>^** c d)))"

definition reflexive_on
  where "reflexive_on \<rho> X \<equiv> \<forall> x \<in> X. \<rho> x x"

definition transitive_on
  where "transitive_on \<rho> X \<equiv> \<forall> x \<in> X. \<forall> y \<in> X. \<forall> z \<in> X. (\<rho>
      x y \<and> \<rho> y z \<longrightarrow> \<rho> x z)"

definition is_dn_closed
  where "is_dn_closed A X \<rho> \<equiv> A \<subseteq> X \<and> (\<forall> a \<in> A . \<forall> x \<in> X. \<
      rho> x a \<longrightarrow> x \<in> A)"

definition is_up_closed
  where "is_up_closed A X \<rho> \<equiv> A \<subseteq> X \<and> (\<forall> a \<in> A . \<forall> x \<in> X. \<
      rho> a x \<longrightarrow> x \<in> A)"

abbreviation is_preorder
  where "is_preorder X \<rho> \<equiv> reflexive_on \<rho> X \<and> transitive_on \<rho> X"

definition has_diamprop
  where "has_diamprop X \<rho> \<equiv> \<forall>l\<in>X. \<forall>x1\<in>X. \<forall>x2\<in>X . (\<rho> l x1 \<
      and> \<rho> l x2 \<longrightarrow> (\<exists>u\<in>X. \<rho> x1 u \<and> \<rho> x2 u))"

definition is_dirset_wrt
  where "is_dirset_wrt A \<rho> \<equiv> A \<noteq> {} \<and> (\<forall>x\<in>A. \<forall>y\<in>A. \<exists>c\<
      in>A . \<rho> x c \<and> \<rho> y c)"

abbreviation is_dirset_in
  where "is_dirset_in A X \<rho> \<equiv> A \<subseteq> X \<and> (is_dirset_wrt A \<rho>)"

definition is_maxdirset_in
  where "is_maxdirset_in A X \<rho> \<equiv> (is_dirset_in A X \<rho>) \<and> (\<forall> B. (is_dirset_in B X \<
      rho>) \<and> A \<subseteq> B \<longrightarrow> A = B)"

definition cardLeN1::"'a set \<Rightarrow> bool"
where
  "cardLeN1 A \<equiv> (\<forall> B \<subseteq> A.
                    ( \<forall> C \<subseteq> B . ((\<exists> D f. D \<subset> C \<and> C \<subseteq> f`D ) \<
      longrightarrow> ( \<exists> f. B \<subseteq> f`C )) )
                \<or> ( \<exists> g . A \<subseteq> g`B ) )"

(* ------------------------------------------------------------------------ *)

section \<open>Auxiliary lemmas\<close>

(* ------------------------------------------------------------------------ *)

lemma lem_Ldo_ldogen_ord:
assumes "\<forall>\<alpha> \<beta> a b c. \<alpha> \<le> \<beta> \<longrightarrow> (a, b) \<in> g \<alpha> \<and
      > (a, c) \<in> g \<beta> \<longrightarrow>
       (\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c'', d) \<in> \<
      DD> g \<beta> \<alpha>)"
shows "DCR_generating g"
  using assms unfolding DCR_generating_def by (meson linear)

lemma lem_rtr_field: "(x,y) \<in> r^* \<Longrightarrow> (x = y) \<or> (x \<in> Field r \<and> y \<in> Field r)"
  by (metis Field_def Not_Domain_rtrancl Range.RangeI UnCI rtranclE)

lemma lem_fin_fl_rel: "finite (Field r) = finite r"
  using finite_Field finite_subset trancl_subset_Field2 by fastforce
```

```
lemma lem_Relprop_fld_sat:
fixes r s::"'U rel"
assumes a1: "s \<subseteq> r" and a2: "s' = Restr r (Field s)"
shows "s \<subseteq> s' \<and> Field s' = Field s"
proof -
  have "s \<subseteq> (Field s) \<times> (Field s)" unfolding Field_def by force
  then have "s \<subseteq> s'" using a1 a2 by blast
  moreover then have "Field s \<subseteq> Field s'" unfolding Field_def by blast
  moreover have "Field s' \<subseteq> Field s" using a2 unfolding Field_def by blast
  ultimately show ?thesis by blast
qed

lemma lem_Relprop_sat_un:
fixes r::"'U rel" and S::"'U set set" and A'::"'U set"
assumes a1: "\<forall>A\<in>S. Field (Restr r A) = A" and a2: "A' = \<Union> S"
shows "Field (Restr r A') = A'"
proof
  show "Field (Restr r A') \<subseteq> A'" unfolding Field_def by blast
next
  show "A' \<subseteq> Field (Restr r A')"
  proof
    fix x
    assume "x \<in> A'"
    then obtain A where "A \<in> S \<and> x \<in> A" using a2 by blast
    then have "x \<in> Field (Restr r A) \<and> A \<subseteq> A'" using a1 a2 by blast
    moreover then have "Field (Restr r A) \<subseteq> Field (Restr r A')" unfolding Field_def by blast
    ultimately show "x \<in> Field (Restr r A')" by blast
  qed
qed

lemma lem_nord_r: "Well_order \<alpha> \<Longrightarrow> nord \<alpha> =o \<alpha>" unfolding nord_def by (meson
       ordIso_reflexive someI_ex)

lemma lem_nord_l: "Well_order \<alpha> \<Longrightarrow> \<alpha> =o nord \<alpha>" unfolding nord_def by (meson
       ordIso_reflexive ordIso_symmetric someI_ex)

lemma lem_nord_eq: "\<alpha> =o \<beta> \<Longrightarrow> nord \<alpha> = nord \<beta>" unfolding nord_def using
       ordIso_symmetric ordIso_transitive by metis

lemma lem_nord_req: "Well_order \<alpha> \<Longrightarrow> Well_order \<beta> \<Longrightarrow> nord \<alpha> =
      nord \<beta> \<Longrightarrow> \<alpha> =o \<beta>"
  using lem_nord_l lem_nord_r ordIso_transitive by metis

lemma lem_Onord: "\<alpha> \<in> \<O> \<Longrightarrow> \<alpha> = nord \<alpha>" unfolding \<O>_def using
      lem_nord_r lem_nord_eq by blast

lemma lem_Oeq: "\<alpha> \<in> \<O> \<Longrightarrow> \<beta> \<in> \<O> \<Longrightarrow> \<alpha> =o \<beta>
      \<Longrightarrow> \<alpha> = \<beta>" using lem_Onord lem_nord_eq by metis

lemma lem_Owo: "\<alpha> \<in> \<O> \<Longrightarrow> Well_order \<alpha>" unfolding \<O>_def using lem_nord_r
      ordIso_Well_order_simp by blast

lemma lem_fld_oord: "Field oord = \<O>" using lem_Owo ordLeq_reflexive unfolding oord_def Field_def by blast

lemma lem_nord_less: "\<alpha> <o \<beta> \<Longrightarrow> nord \<beta> \<noteq> nord \<alpha> \<and> (nord \<
      alpha>, nord \<beta>) \<in> oord"
proof -
  assume b1: "\<alpha> <o \<beta>"
  then have "nord \<alpha> \<in> \<O> \<and> nord \<beta> \<in> \<O> \<and> nord \<alpha> =o \<alpha> \<and>
      nord \<beta> =o \<beta>"
    using lem_nord_r ordLess_Well_order_simp unfolding \<O>_def by blast
  moreover have "\<forall> r A a b. (a,b) \<in> Restr r A = (a \<in> A \<and> b \<in> A \<and> (a,b) \<in> r)"
    unfolding Field_def by force
  ultimately show "nord \<beta> \<noteq> nord \<alpha> \<and>(nord \<alpha>, nord \<beta>) \<in> oord" using b1
      unfolding oord_def
    by (metis not_ordLess_ordIso ordIso_iff_ordLeq ordLeq_iff_ordLess_or_ordIso ordLeq_transitive)
qed

lemma lem_nord_ls: "\<alpha> <o \<beta> \<Longrightarrow> nord \<alpha> <o nord \<beta>"
proof -
  assume a1: "\<alpha> <o \<beta>"
  then have "Well_order \<alpha> \<and> Well_order \<beta>" unfolding ordLess_def by blast
  then have "nord \<alpha> =o \<alpha>" and "nord \<beta> =o \<beta>" using lem_nord_r by blast+
  then show "nord \<alpha> <o nord \<beta>" using a1
    using ordIso_iff_ordLeq ordIso_ordLess_trans ordLess_ordLeq_trans by blast
qed

lemma lem_nord_le: "\<alpha> \<le>o \<beta> \<Longrightarrow> nord \<alpha> \<le>o nord \<beta>"
```

```
proof -
  assume a1: "\<alpha> \<le>o \<beta>"
  then have "Well_order \<alpha> \<and> Well_order \<beta>" unfolding ordLeq_def by blast
  then have "nord \<alpha> =o \<alpha>" and "nord \<beta> =o \<beta>" using lem_nord_r by blast+
  then show "nord \<alpha> \<le>o nord \<beta>" using a1 by (meson ordIso_iff_ordLeq ordLeq_transitive)
qed

lemma lem_nordO_ls_l: "\<alpha> <o \<beta> \<Longrightarrow> nord \<alpha> \<in> \<O>" using \<O>_def
      ordLess_Well_order_simp by blast

lemma lem_nordO_ls_r: "\<alpha> <o \<beta> \<Longrightarrow> nord \<beta> \<in> \<O>" using \<O>_def
      ordLess_Well_order_simp by blast

lemma lem_nordO_le_l: "\<alpha> \<le>o \<beta> \<Longrightarrow> nord \<alpha> \<in> \<O>" using \<O>_def
      ordLeq_Well_order_simp by blast

lemma lem_nordO_le_r: "\<alpha> \<le>o \<beta> \<Longrightarrow> nord \<beta> \<in> \<O>" using \<O>_def
      ordLeq_Well_order_simp by blast

lemma lem_nord_ls_r: "\<alpha> <o \<beta> \<Longrightarrow> \<alpha> <o nord \<beta>"
  using lem_nord_ls[of \<alpha> \<beta>] lem_nord_r[of \<beta>] lem_nord_l by (metis ordLess_ordIso_trans
      ordLess_Well_order_simp)

lemma lem_nord_ls_l: "\<alpha> <o \<beta> \<Longrightarrow> nord \<alpha> <o \<beta>"
  using lem_nord_ls[of \<alpha> \<beta>] lem_nord_r[of \<beta>] by (metis ordLess_ordIso_trans
      ordLess_Well_order_simp)

lemma lem_nord_le_r: "\<alpha> \<le>o \<beta> \<Longrightarrow> \<alpha> \<le>o nord \<beta>"
  using lem_nord_le[of \<alpha> \<beta>] lem_nord_r[of \<beta>] lem_nord_l by (metis ordLeq_ordIso_trans
      ordLeq_Well_order_simp)

lemma lem_nord_le_l: "\<alpha> \<le>o \<beta> \<Longrightarrow> nord \<alpha> \<le>o \<beta>"
  using lem_nord_le[of \<alpha> \<beta>] lem_nord_r[of \<beta>] by (metis ordLeq_ordIso_trans
      ordLeq_Well_order_simp)

lemma lem_oord_wo: "Well_order oord"
proof -
  let ?oleqO = "Restr ordLeq \<O>"
  have "Well_order ?oleqO"
  proof -
    have c1: "Field ordLeq = {\<alpha>::'U rel. Well_order \<alpha>}"
      using ordLeq_Well_order_simp ordLeq_reflexive unfolding Field_def by blast
    then have "Refl ordLeq" using ordLeq_refl_on by metis
    then have "Preorder ordLeq" using ordLeq_trans unfolding preorder_on_def by blast
    then have "Preorder ?oleqO" using Preorder_Restr by blast
    moreover have "\<forall>\<alpha> \<beta>::'U rel. (\<alpha>, \<beta>) \<in> ?oleqO \<longrightarrow> (\<beta
      >, \<alpha>) \<in> ?oleqO \<longrightarrow> \<alpha> = \<beta>"
    proof (intro allI impI)
      fix \<alpha> \<beta>::"'U rel"
      assume d1: "(\<alpha>, \<beta>) \<in> ?oleqO" and d2: "(\<beta>, \<alpha>) \<in> ?oleqO"
      then have "\<alpha> \<le>o \<beta> \<and> \<beta> \<le>o \<alpha>" by blast
      then have "\<alpha> =o \<beta>" using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> \<in> \<O> \<and> \<beta> \<in> \<O>" using d1 by blast
      ultimately show "\<alpha> = \<beta>" using lem_Oeq by blast
    qed
    moreover have "\<forall> \<alpha> \<in> Field (?oleqO::'U rel rel). \<forall> \<beta> \<in> Field ?oleqO. \<
      alpha> \<noteq> \<beta> \<longrightarrow>
                                        (\<alpha>, \<beta>) \<in> ?oleqO \<or> (\<beta>, \<alpha>) \<in> ?
      oleqO"
    proof (intro ballI impI)
      fix \<alpha> \<beta>::"'U rel"
      assume d1: "\<alpha> \<in> Field ?oleqO" and d2: "\<beta> \<in> Field ?oleqO" and "\<alpha> \<noteq> \<
      beta>"
      then have "Well_order \<alpha> \<and> Well_order \<beta>" using c1 unfolding Field_def
        by (metis (no_types, lifting) Field_Un Field_def Un_def mem_Collect_eq sup_inf_absorb)
      then have "\<alpha> \<le>o \<beta> \<or> \<beta> \<le>o \<alpha>"  using ordLess_imp_ordLeq
      ordLess_or_ordLeq by blast
      moreover have "\<alpha> \<in> \<O> \<and> \<beta> \<in> \<O>" using d1 d2 unfolding Field_def by blast
      ultimately show "(\<alpha>, \<beta>) \<in> ?oleqO \<or> (\<beta>, \<alpha>) \<in> ?oleqO" by blast
    qed
    ultimately have "Linear_order ?oleqO" unfolding linear_order_on_def
      partial_order_on_def total_on_def antisym_def preorder_on_def by blast
    moreover have "wf ((?oleqO::'U rel rel) - Id)"
    proof -
      have "Restr (ordLess::'U rel rel) \<O> \<subseteq> ?oleqO - Id"
        using not_ordLeq_ordLess ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "(?oleqO::'U rel rel) - Id \<subseteq> Restr ordLess \<O>"
        using lem_Oeq ordLeq_iff_ordLess_or_ordIso by blast
```

```
      ultimately have "(?oleqO::'U rel rel) - Id = Restr ordLess \<O>" by blast
      moreover have "wf (Restr ordLess \<O>)"
        using wf_ordLess Restr_subset wf_subset[of ordLess "Restr ordLess \<O>"] by blast
      ultimately show ?thesis by simp
    qed
    ultimately show ?thesis unfolding well_order_on_def by blast
  qed
  moreover have "Well_order |(UNIV - \<O>)::'U rel set|" using  card_of_Well_order by blast
  moreover have "Field (Restr ordLeq \<O>) \<inter> Field ( |(UNIV - \<O>)::'U rel set| ) = {}"
  proof -
    have "Field (Restr ordLeq \<O>) \<subseteq> \<O>" unfolding Field_def by blast
    moreover have "Field ( |(UNIV - \<O>)::'U rel set| ) \<subseteq> UNIV - \<O>" by simp
    ultimately show ?thesis by blast
  qed
  ultimately show ?thesis unfolding oord_def using Osum_Well_order by blast
qed

lemma lem_lmord_inf:
fixes \<alpha>::"'U rel"
assumes "lm_ord \<alpha>"
shows "\<not> finite (Field \<alpha>)"
proof -
  have "finite (Field \<alpha>) \<longrightarrow> False"
  proof
    assume c1: "finite (Field \<alpha>)"
    have c2: "Well_order \<alpha>" using assms unfolding lm_ord_def by blast
    have "\<alpha> \<noteq> {}" using assms lm_ord_def by blast
    then have "Field \<alpha> \<noteq> {}" unfolding Field_def by force
    then have "wo_rel.isMaxim \<alpha> (Field \<alpha>) (wo_rel.maxim \<alpha> (Field \<alpha>))"
      using c1 c2 wo_rel.maxim_isMaxim[of \<alpha> "Field \<alpha>"] unfolding wo_rel_def by blast
    then have "\<exists>j\<in>Field \<alpha>. \<forall>i\<in>Field \<alpha>. (i, j) \<in> \<alpha>"
      using c2 wo_rel.isMaxim_def[of \<alpha> "Field \<alpha>"] unfolding wo_rel_def by blast
    then have "isSuccOrd \<alpha>" using c2 wo_rel.isSuccOrd_def unfolding wo_rel_def by blast
    then show "False" using assms unfolding lm_ord_def by blast
  qed
  then show ?thesis by blast
qed

lemma lem_sucord_ex:
fixes \<alpha> \<beta>::"'U rel"
assumes "\<alpha> <o \<beta>"
shows "\<exists> \<alpha>'::'U rel. sc_ord \<alpha> \<alpha>'"
proof -
  obtain S::"'U rel set" where b1: "S = { \<gamma>::'U rel. \<alpha> <o \<gamma> }" by blast
  then have "S \<noteq> {} \<and> (\<forall> \<alpha> \<in> S. Well_order \<alpha>)" using assms
      ordLess_Well_order_simp by blast
  then obtain \<alpha>' where "\<alpha>' \<in> S \<and> (\<forall>\<alpha> \<in> S. \<alpha>' \<le>o \<alpha>)"
    using BNF_Wellorder_Constructions.exists_minim_Well_order[of S] by blast
  then show ?thesis unfolding b1 sc_ord_def by blast
qed

lemma lem_osucc_eq: "isSuccOrd \<alpha> \<Longrightarrow> \<alpha> =o \<beta> \<Longrightarrow> isSuccOrd \<beta
      >"
proof -
  assume a1: "isSuccOrd \<alpha>" and a2: "\<alpha> =o \<beta>"
  moreover then have a3: "wo_rel \<alpha>" and a4: "wo_rel \<beta>" unfolding ordIso_def wo_rel_def by blast+
  obtain j where a5: "j \<in> Field \<alpha>" and a6: "\<forall>i\<in>Field \<alpha>. (i, j) \<in> \<alpha>"
      using a1 a3 wo_rel.isSuccOrd_def by blast
  obtain f where a7: "iso \<alpha> \<beta> f" using a2 unfolding ordIso_def by blast
  have "(f j) \<in> Field \<beta>" using a5 a7 unfolding iso_def bij_betw_def by blast
  moreover have "\<forall> i' \<in> Field \<beta>. (i', f j) \<in> \<beta>"
  proof
    fix i'
    assume b1: "i' \<in> Field \<beta>"
    then obtain i where b2: "i \<in> Field \<alpha> \<and> i' = f i" using a7 unfolding iso_def bij_betw_def by
      blast
    then have "(i, j) \<in> \<alpha>" using a6 by blast
    then have "(f i, f j) \<in> \<beta>" using a2 a7 by (meson iso_oproj oproj_in ordIso_Well_order_simp)
    then show "(i', f j) \<in> \<beta>" using b2 by blast
  qed
  ultimately have "\<exists>j\<in>Field \<beta>. \<forall>i\<in>Field \<beta>. (i, j) \<in> \<beta>" by blast
  then show "isSuccOrd \<beta>" using a4 wo_rel.isSuccOrd_def by blast
qed

lemma lem_ord_subemp: "(\<alpha>::'a rel) \<le>o ({}::'b rel) \<Longrightarrow> \<alpha> = {}"
proof -
  assume "\<alpha> \<le>o ({}::'b rel)"
  then obtain f where "embed \<alpha> ({}::'b rel) f" unfolding ordLeq_def by blast
```

```
  then show "\<alpha> = {}" unfolding embed_def bij_betw_def Field_def under_def by force
qed

lemma lem_ordint_sucord:
fixes \<alpha>0::"'a rel" and \<alpha>::"'b rel"
assumes "\<alpha>0 <o \<alpha> \<and> (\<forall> \<gamma>::'b rel. \<alpha>0 <o \<gamma> \<longrightarrow> \<
      alpha> \<le>o \<gamma>)"
shows "isSuccOrd \<alpha>"
proof -
  have c1: "Well_order \<alpha>" using assms unfolding ordLess_def by blast
  obtain f where e3: "Well_order \<alpha>0 \<and> Well_order \<alpha> \<and> embedS \<alpha>0 \<alpha> f" using
      assms unfolding ordLess_def by blast
  moreover have e4: "f ` Field \<alpha>0 \<subseteq> Field \<alpha>" using e3 embed_in_Field[of \<alpha>0 \<
      alpha> f] unfolding embedS_def by blast
  have "f ` Field \<alpha>0 \<noteq> Field \<alpha>" using e3 embed_inj_on unfolding bij_betw_def embedS_def by
      blast
  then obtain j0 where e5: "j0 \<in> Field \<alpha> \<and> j0 \<notin> f ` Field \<alpha>0" using e4 by blast
  moreover have "\<forall> i \<in> Field \<alpha>. (i, j0) \<in> \<alpha>"
  proof
    fix i
    assume "i \<in> Field \<alpha>"
    moreover then have "(i, i) \<in> \<alpha>" using e3 unfolding well_order_on_def
      linear_order_on_def partial_order_on_def preorder_on_def refl_on_def by blast
    moreover have "(j0, i) \<in> \<alpha> \<longrightarrow> (i, j0) \<in> \<alpha>"
    proof
      assume g1: "(j0, i) \<in> \<alpha>"
      obtain \<gamma> where g2: "\<gamma> = Restr \<alpha> (under \<alpha> j0)" by blast
      then have g3: "Well_order \<gamma>" using e3 Well_order_Restr by blast
      have "\<alpha>0 <o \<gamma>"
      proof -
        have h1: "\<forall> a \<in> Field \<alpha>0. f a \<in> under \<alpha> j0"
        proof
          fix a
          assume i1: "a \<in> Field \<alpha>0"
          then have i2: "bij_betw f (under \<alpha>0 a) (under \<alpha> (f a))" using e3 unfolding embedS_def
      embed_def by blast
          have "(j0, f a) \<in> \<alpha> \<longrightarrow> False"
          proof
            assume "(j0, f a) \<in> \<alpha>"
            then obtain b where "j0 = f b \<and> b \<in> under \<alpha>0 a" using i2 unfolding under_def
      bij_betw_def by (simp, blast)
            moreover then have "b \<in> Field \<alpha>0" unfolding under_def Field_def by blast
            ultimately show "False" using e5 by blast
          qed
          moreover have i3: "j0 \<in> Field \<alpha>" using g1 unfolding Field_def by blast
          moreover have "f a \<in> Field \<alpha>" using i1 e3 embed_Field unfolding embedS_def by blast
          ultimately have i4: "(f a, j0) \<in> \<alpha>"
            using e3 unfolding well_order_on_def linear_order_on_def total_on_def
              partial_order_on_def preorder_on_def refl_on_def by metis
          then show "f a \<in> under \<alpha> j0" unfolding under_def by blast
        qed
        then have "compat \<alpha>0 \<gamma> f"
          using e3 g2 embed_compat unfolding Field_def embedS_def compat_def by blast
        moreover have "ofilter \<gamma> (f ` Field \<alpha>0)"
        proof -
          have "ofilter \<alpha> (under \<alpha> j0)" using e3 wo_rel.under_ofilter[of \<alpha>] unfolding
      wo_rel_def by blast
          moreover have "ofilter \<alpha> (f ` Field \<alpha>0)"
            using e3 embed_iff_compat_inj_on_ofilter[of \<alpha>0 \<alpha> f] unfolding embedS_def by blast
          moreover have "f ` Field \<alpha>0 \<subseteq> under \<alpha> j0" using h1 by blast
          ultimately show "ofilter \<gamma> (f ` Field \<alpha>0)"
            using g2 e3 ofilter_Restr_subset[of \<alpha> "f ` Field \<alpha>0" "under \<alpha> j0"] by blast
        qed
        moreover have "inj_on f (Field \<alpha>0)"
          using e3 embed_iff_compat_inj_on_ofilter[of \<alpha>0 \<alpha> f] unfolding embedS_def by blast
        ultimately have "embed \<alpha>0 \<gamma> f" using g3 e3 embed_iff_compat_inj_on_ofilter[of \<alpha>0 \<
      gamma> f] by blast
        moreover have "bij_betw f (Field \<alpha>0) (Field \<gamma>) \<longrightarrow> False"
        proof
          assume i1: "bij_betw f (Field \<alpha>0) (Field \<gamma>)"
          have "(j0, j0) \<in> \<alpha>" using e3 e5 unfolding well_order_on_def
            linear_order_on_def partial_order_on_def preorder_on_def refl_on_def by blast
          then have "j0 \<in> Field \<gamma>" using g2 unfolding under_def Field_def by blast
          then show "False" using i1 e5 unfolding bij_betw_def by blast
        qed
        ultimately have "embedS \<alpha>0 \<gamma> f"  unfolding embedS_def by blast
        then show ?thesis using g3 e3 unfolding ordLess_def by blast
      qed
```

```
      then have "\<alpha> =o \<gamma>" using assms g2 e3 under_Restr_ordLeq[of \<alpha> j0] ordIso_iff_ordLeq by
       blast
      then obtain f1 where "iso \<alpha> \<gamma> f1" unfolding ordIso_def by blast
      then have g4: "embed \<alpha> \<gamma> f1 \<and> bij_betw f1 (Field \<alpha>) (Field \<gamma>)" unfolding
      iso_def by blast
      then have "f1 ` under \<alpha> i = under \<gamma> (f1 i)" using g1 unfolding bij_betw_def embed_def
      Field_def by blast
      then have "(f1 i, j0) \<in> \<alpha>" using g1 unfolding g2 under_def by blast
      moreover have "f1 i = i"
      proof -
        have "Restr \<alpha> (Field \<alpha>) = \<alpha>" unfolding Field_def by force
        moreover have "ofilter \<alpha> (under \<alpha> j0)" using e3 wo_rel.under_ofilter[of \<alpha>]
      unfolding wo_rel_def by blast
        moreover have "ofilter \<alpha> (Field \<alpha>)" unfolding ofilter_def under_def Field_def by blast
        moreover have "under \<alpha> j0 \<subseteq> Field \<alpha>" unfolding under_def Field_def by blast
        ultimately have "embed \<gamma> \<alpha> id" using g2 e3 ofilter_subset_embed by metis
        then have "embed \<alpha> \<alpha> (id \<circ> f1)" using g4 e3 comp_embed by blast
        then have "embed \<alpha> \<alpha> f1" by simp
        moreover have "embed \<alpha> \<alpha> id" unfolding embed_def id_def bij_betw_def inj_on_def by blast
        ultimately have "\<forall> k \<in> Field \<alpha>. f1 k = k" using e3 embed_unique[of \<alpha> \<alpha>
      f1 id] unfolding id_def by blast
        moreover have "i \<in> Field \<alpha>" using g1 unfolding Field_def by blast
        ultimately show ?thesis by blast
      qed
      ultimately show "(i, j0) \<in> \<alpha>" by metis
    qed
    ultimately show "(i, j0) \<in> \<alpha>"
      using e3 e5 unfolding well_order_on_def linear_order_on_def total_on_def by metis
  qed
  ultimately show "isSuccOrd \<alpha>" using c1 wo_rel.isSuccOrd_def[of \<alpha>] unfolding wo_rel_def by blast
qed

lemma lem_sucord_ordint:
fixes \<alpha>::"'U rel"
assumes "Well_order \<alpha> \<and> isSuccOrd \<alpha>"
shows "\<exists> \<alpha>0::'U rel. \<alpha>0 <o \<alpha> \<and> (\<forall> \<gamma>::'U rel. \<alpha>0 <o \<
      gamma> \<longrightarrow> \<alpha> \<le>o \<gamma>)"
proof -
  obtain j where b1: "j \<in> Field \<alpha> \<and> (\<forall>i \<in> Field \<alpha>. (i, j) \<in> \<alpha>)"
    using assms wo_rel.isSuccOrd_def unfolding wo_rel_def by blast
  moreover obtain \<alpha>0 where b2: "\<alpha>0 = Restr \<alpha> (UNIV - {j})" by blast
  moreover have "\<forall> i. (j, i) \<in> \<alpha> \<longrightarrow> i = j" using assms b1 unfolding Field_def
      well_order_on_def
    linear_order_on_def partial_order_on_def antisym_def by blast
  ultimately have b3: "embedS \<alpha>0 \<alpha> id" unfolding Field_def embedS_def embed_def id_def
    bij_betw_def under_def inj_on_def apply simp by blast
  moreover have b4: "Well_order \<alpha>0" using assms b2 Well_order_Restr by blast
  ultimately have "\<alpha>0 <o \<alpha>" using assms unfolding ordLess_def by blast
  moreover have "\<forall> \<gamma>::'U rel. \<alpha>0 <o \<gamma> \<longrightarrow> \<alpha> \<le>o \<gamma>"
  proof (intro allI impI)
    fix \<gamma>::"'U rel"
    assume c1: "\<alpha>0 <o \<gamma>"
    then have c2: "Well_order \<gamma>" unfolding ordLess_def by blast
    obtain f where "embedS \<alpha>0 \<gamma> f" using c1 unfolding ordLess_def by blast
    then have c3: "embed \<alpha>0 \<gamma> f \<and> \<not> bij_betw f (Field \<alpha>0) (Field \<gamma>)"
      unfolding embedS_def by blast
    have "\<gamma> <o \<alpha> \<longrightarrow> False"
    proof
      assume d1: "\<gamma> <o \<alpha>"
      obtain g where "embedS \<gamma> \<alpha> g" using d1 unfolding ordLess_def by blast
      then have d3: "embed \<gamma> \<alpha> g \<and> \<not> bij_betw g (Field \<gamma>) (Field \<alpha>)"
      unfolding embedS_def by blast
      have d4: "j \<in> g ` Field \<gamma> \<longrightarrow> False"
      proof
        assume "j \<in> g ` Field \<gamma>"
        then obtain a where "a \<in> Field \<gamma> \<and> g a = j" by blast
        then have "bij_betw g (under \<gamma> a) (under \<alpha> j)" using d3 unfolding embed_def by blast
        moreover have "under \<alpha> j = Field \<alpha>" using b1 unfolding under_def Field_def by blast
        ultimately have "bij_betw g (under \<gamma> a) (Field \<alpha>)" by simp
        then have "g ` Field \<gamma> \<noteq> Field \<alpha> \<and> g ` Field \<gamma> \<subseteq> Field \<
      alpha> \<and> g ` under \<gamma> a = Field \<alpha>"
          using c2 d3 embed_inj_on[of \<gamma> \<alpha> g] embed_Field[of \<gamma> \<alpha> g] unfolding
      bij_betw_def by blast
        moreover have "under \<gamma> a \<subseteq> Field \<gamma>" unfolding under_def Field_def by blast
        ultimately show "False" by blast
      qed
      have "Field \<gamma> \<subseteq> f ` Field \<alpha>0"
      proof
```

```
        fix a
        assume e1: "a \<in> Field \<gamma>"
        then have "bij_betw g (under \<gamma> a) (under \<alpha> (g a))" using d3 unfolding embed_def by blast
        have "g a \<in> Field \<alpha> - {j}" using e1 c2 d3 d4 embed_Field by blast
        moreover then have "(g a, g a) \<in> \<alpha>" using assms unfolding Field_def well_order_on_def
          linear_order_on_def partial_order_on_def preorder_on_def refl_on_def by blast
        ultimately have e2: "g a \<in> Field \<alpha>0" using b2 unfolding Field_def by blast
        have "embed \<alpha>0 \<alpha> (g \<circ> f)" using b4 c3 d3 comp_embed[of \<alpha>0 \<gamma> f \<alpha>
       g] by blast
        then have "\<forall> x \<in> Field \<alpha>0. g (f x) = x" using assms b3 b4 embed_unique[of \<alpha>0
      \<alpha> "g \<circ> f" id]
          unfolding embedS_def comp_def id_def by blast
        then have "g (f (g a)) = g a" using e2 by blast
        moreover have "inj_on g (Field \<gamma>)" using c2 d3 embed_inj_on[of \<gamma> \<alpha> g] by blast
        moreover have "f (g a) \<in> Field \<gamma>" using e2 b4 c3 embed_Field[of \<alpha>0 \<gamma> f] by
      blast
        ultimately have "f (g a) = a" using e1 unfolding inj_on_def by blast
        then show "a \<in> f ` Field \<alpha>0" using e2 by force
      qed
      then have "bij_betw f (Field \<alpha>0) (Field \<gamma>)"
        using b4 c3 embed_inj_on[of \<alpha>0 \<gamma> f] embed_Field[of \<alpha>0 \<gamma> f] unfolding
      bij_betw_def by blast
      then show "False" using c3 by blast
    qed
    then show "\<alpha> \<le>o \<gamma>" using assms c2 by simp
  qed
  ultimately show ?thesis by blast
qed

lemma lem_sclm_ordind:
fixes P::"'U rel \<Rightarrow> bool"
assumes a1: "P {}"
    and a2: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<and> P \<alpha>0 \<
      longrightarrow> P \<alpha>)"
    and a3: "\<forall> \<alpha>. ((lm_ord \<alpha> \<and> (\<forall> \<beta>. \<beta> <o \<alpha> \<
      longrightarrow> P \<beta>)) \<longrightarrow> P \<alpha>)"
shows "\<forall> \<alpha>. Well_order \<alpha> \<longrightarrow> P \<alpha>"
proof -
  obtain Q where b1: "Q = (\<lambda> \<alpha>. Well_order \<alpha> \<longrightarrow> P \<alpha>)" by blast
  have "\<forall> \<alpha>. (\<forall> \<beta>. \<beta> <o \<alpha> \<longrightarrow> Q \<beta>) \<
      longrightarrow> Q \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "\<forall> \<beta>. \<beta> <o \<alpha> \<longrightarrow> Q \<beta>"
    then have c2: "\<forall> \<beta>. \<beta> <o \<alpha> \<longrightarrow> P \<beta>" unfolding b1 ordLess_def
      by blast
    show "Q \<alpha>"
    proof (cases "\<exists> \<alpha>0. sc_ord \<alpha>0 \<alpha>")
      assume "\<exists> \<alpha>0. sc_ord \<alpha>0 \<alpha>"
      then obtain \<alpha>0 where "sc_ord \<alpha>0 \<alpha>" by blast
      then show "Q \<alpha>" using c2 b1 a2 unfolding sc_ord_def by blast
    next
      assume "\<not> (\<exists> \<alpha>0. sc_ord \<alpha>0 \<alpha>)"
      then have "(\<not> Well_order \<alpha>) \<or> \<alpha> = {} \<or> lm_ord \<alpha>"
        using lem_sucord_ordint unfolding sc_ord_def lm_ord_def by blast
      moreover have "lm_ord \<alpha> \<longrightarrow> P \<alpha>" using c2 a3 by blast
      ultimately show "Q \<alpha>" using a1 b1 by blast
    qed
  qed
  then show ?thesis using b1 wf_induct[of ordLess Q] wf_ordLess by blast
qed

lemma lem_ordseq_rec_sets:
fixes E::"'U set" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set"
assumes "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> F \<alpha> = F \<beta>"
shows "\<exists> f::('U rel \<Rightarrow> 'U set).
           f {} = E
         \<and> (\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> =
       F \<alpha>0 (f \<alpha>0)))
         \<and> (\<forall> \<alpha>. lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<
      beta>. \<beta> <o \<alpha> \<and> D = f \<beta> })
         \<and> (\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>)"
proof -
  obtain cmp::"'U rel rel" where b1: "cmp = oord" by blast
  then interpret cmp: wo_rel cmp unfolding wo_rel_def using lem_oord_wo by blast
  obtain L where b2: "L = (\<lambda> g::'U rel \<Rightarrow> 'U set. \<lambda> \<alpha>::'U rel. \<Union> (g ` (
      underS cmp \<alpha>)))" by blast
  then have b3: "adm_woL cmp L" unfolding cmp.adm_woL_def by blast
```

```
obtain fo where b4: "fo = (worecZSL cmp E F L)" by blast
obtain f where b5: "f = (\<lambda> \<alpha>::'U rel. fo (nord \<alpha>))" by blast
have b6: "fo (zero cmp) = E" using b3 b4 cmp.worecZSL_zero by simp
have b7: "\<forall> \<alpha>. aboveS cmp \<alpha> \<noteq> {} \<longrightarrow> fo (succ cmp \<alpha>) = F \<
    alpha> (fo \<alpha>)"
  using b3 b4 cmp.worecZSL_succ by metis
have b8: "\<forall> \<alpha>. isLim cmp \<alpha> \<and> \<alpha> \<noteq> zero cmp \<longrightarrow> fo \<
    alpha> = \<Union> (fo ` (underS cmp \<alpha>))"
  using b2 b3 b4 cmp.worecZSL_isLim by metis
have b9: "zero cmp = {} \<and> nord ({}::'U rel) = {}"
proof -
  obtain isz where c1: "isz = (\<lambda> \<alpha>. \<alpha> \<in> Field cmp \<and> (\<forall>\<beta>\<in>Field
     cmp. (\<alpha>, \<beta>) \<in> cmp))" by blast
  have c2: "{} \<in> (\<O>::'U rel set)"
  proof -
    have "Well_order ({}::'U rel)" by simp
    moreover then have "nord ({}::'U rel) = {}" using lem_nord_r lem_ord_subemp ordIso_iff_ordLeq by blast
    ultimately show ?thesis unfolding \<O>_def by blast
  qed
  moreover have "\<forall> \<beta> \<in> \<O>::('U rel set). ({}, \<beta>) \<in> oord"
  proof
    fix \<beta>::"'U rel"
    assume d1: "\<beta> \<in> \<O>"
    then have "Well_order \<beta>" using lem_0wo by blast
    then have "{} \<le>o \<beta>" using ozero_ordLeq unfolding ozero_def by blast
    then show "({}, \<beta>) \<in> oord" using d1 c2 unfolding oord_def by blast
  qed
  ultimately have "isz {}" using c1 b1 lem_fld_oord by blast
  moreover have "\<forall> \<alpha>. isz \<alpha> \<longrightarrow> \<alpha> = {}"
  proof (intro allI impI)
    fix \<alpha>
    assume d1: "isz \<alpha>"
    then have d2: "\<alpha> \<in> \<O> \<and> (\<forall> \<beta> \<in> \<O>. (\<alpha>, \<beta>) \<in> oord)"
     using c1 b1 lem_fld_oord by blast
    have "Well_order ({}::'U rel)" by simp
    then have "\<alpha> \<le>o nord ({}::'U rel) \<and> nord ({}::'U rel) =o ({}::'U rel)"
      using d2 lem_nord_r unfolding oord_def \<O>_def by blast
    then have "\<alpha> \<le>o ({}::'U rel)" using ordLeq_ordIso_trans by blast
    then show "\<alpha> = {}" using lem_ord_subemp by blast
  qed
  ultimately have "(THE \<alpha>. isz \<alpha>) = {}" by (simp only: the_equality)
  then have "zero cmp = {}" unfolding c1 cmp.zero_def cmp.minim_def cmp.isMinim_def by blast
  moreover have "nord ({}::'U rel) = {}" using c2 lem_0nord by blast
  ultimately show ?thesis by blast
qed
have b10: "\<forall> \<alpha> \<alpha>'::'U rel. aboveS cmp \<alpha> \<noteq> {} \<and> \<alpha>' = succ cmp
    \<alpha> \<longrightarrow> (\<alpha> \<in> \<O> \<and> \<alpha>' \<in> \<O> \<and> \<alpha> <o \<alpha>'
    \<and> (\<forall> \<beta>::'U rel. \<alpha> <o \<beta> \<longrightarrow> \<alpha>' \<le>o \<beta>))"
proof (intro allI impI)
  fix \<alpha> \<alpha>'
  assume "aboveS cmp \<alpha> \<noteq> {} \<and> \<alpha>' = succ cmp \<alpha>"
  moreover then have "AboveS cmp {\<alpha>} \<subseteq> Field cmp \<and> AboveS cmp {\<alpha>} \<noteq> {}"
    unfolding AboveS_def aboveS_def Field_def by blast
  ultimately have c4: "isMinim cmp (AboveS cmp {\<alpha>}) \<alpha>'"
    using cmp.minim_isMinim unfolding cmp.succ_def cmp.suc_def by blast
  have c5: "(\<alpha>, \<alpha>') \<in> cmp \<and> \<alpha> \<noteq> \<alpha>'" using c4 lem_fld_oord
    unfolding cmp.isMinim_def AboveS_def by blast
  then have "\<alpha> \<le>o \<alpha>' \<and> \<not> (\<alpha> =o \<alpha>')" using b1 lem_0eq unfolding
    oord_def by blast
  then have "\<alpha> <o \<alpha>'" using ordLeq_iff_ordLess_or_ordIso by blast
  moreover have "\<forall> \<beta>::'U rel. \<alpha> <o \<beta> \<longrightarrow> \<alpha>' \<le>o \<beta>"
  proof (intro allI impI)
    fix \<beta>::"'U rel"
    assume d1: "\<alpha> <o \<beta>"
    have "nord \<beta> \<noteq> nord \<alpha> \<and> (nord \<alpha>, nord \<beta>) \<in> cmp" using d1 b1
     lem_nord_less by blast
    moreover then have "nord \<beta> \<in> Field cmp" unfolding Field_def by blast
    ultimately have "nord \<beta> \<in> AboveS cmp {nord \<alpha>}" unfolding AboveS_def by blast
    moreover have "\<alpha> = nord \<alpha>" using c5 b1 lem_0nord unfolding oord_def by blast
    ultimately have "(\<alpha>', nord \<beta>) \<in> cmp" using c4 unfolding cmp.isMinim_def by metis
    then have "\<alpha>' \<le>o nord \<beta>" unfolding b1 oord_def by blast
    moreover have "nord \<beta> =o \<beta>" using d1 lem_nord_r ordLess_Well_order_simp by blast
    ultimately show "\<alpha>' \<le>o \<beta>" using ordLeq_ordIso_trans by blast
  qed
  moreover have "\<alpha> \<in> \<O> \<and> \<alpha>' \<in> \<O>" using c5 b1 unfolding oord_def by blast
  ultimately show "\<alpha> \<in> \<O> \<and> \<alpha>' \<in> \<O> \<and> \<alpha> <o \<alpha>' \<and> (\<
    forall> \<beta>::'U rel. \<alpha> <o \<beta> \<longrightarrow> \<alpha>' \<le>o \<beta>)" by blast
qed
```

```
  then have b11: "\<forall> \<alpha>::'U rel. Well_order \<alpha> \<and> \<not> (\<alpha> = {} \<or> isSuccOrd
      \<alpha>) \<longrightarrow> isLim cmp \<alpha>"
    using lem_ordint_sucord unfolding cmp.isLim_def cmp.isSucc_def by metis
  have "f {} = E" using b5 b6 b9 by simp
  moreover have "(\<forall> \<alpha> \<alpha>'::'U rel. (\<alpha> <o \<alpha>' \<and> (\<forall> \<beta>::'U rel
      . \<alpha> <o \<beta> \<longrightarrow> \<alpha>' \<le>o \<beta>) \<longrightarrow> f \<alpha>' = F \<
      alpha> (f \<alpha>)))"
  proof (intro allI impI)
    fix \<alpha> \<alpha>'::"'U rel"
    assume c1: "\<alpha> <o \<alpha>' \<and> (\<forall> \<beta>::'U rel. \<alpha> <o \<beta> \<longrightarrow>
      \<alpha>' \<le>o \<beta>)"
    then have c2: "(aboveS cmp (nord \<alpha>)) \<noteq> {}" using lem_nord_less unfolding b1 aboveS_def by fast
    obtain \<gamma> where c3: "\<gamma> = succ cmp (nord \<alpha>)" by blast
    have c4: "\<gamma> \<in> \<O> \<and> (nord \<alpha>) <o \<gamma> \<and> (\<forall>\<beta>::'U rel. (nord \<
      alpha>) <o \<beta> \<longrightarrow> \<gamma> \<le>o \<beta>)" using c2 c3 b10 by blast
    moreover have "nord \<alpha> =o \<alpha>" using c1 lem_nord_r ordLess_Well_order_simp by blast
    ultimately have "\<alpha> <o \<gamma> \<and> (\<forall>\<beta>::'U rel. \<alpha> <o \<beta> \<longrightarrow
      > \<gamma> \<le>o \<beta>)" using ordIso_iff_ordLeq ordLeq_ordLess_trans by blast
    then have "\<alpha>' =o \<gamma>" using c1 ordIso_iff_ordLeq by blast
    then have "f \<alpha>' = f \<gamma>" using b5 lem_nord_eq by metis
    moreover have "\<gamma> = nord \<gamma>" using c4 lem_Onord by blast
    moreover have "fo \<gamma> = F (nord \<alpha>) (f \<alpha>)" using c2 c3 b5 b7 by blast
    moreover have "F (nord \<alpha>) (f \<alpha>) = F \<alpha> (f \<alpha>)" using assms c1 lem_nord_r
      ordLess_Well_order_simp by metis
    ultimately show "f \<alpha>' = F \<alpha> (f \<alpha>)" using b5 by metis
  qed
  moreover have "\<forall> \<alpha>. (Well_order \<alpha> \<and> \<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)
      ) \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta>
       }"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "Well_order \<alpha> \<and> \<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)"
    then have "Well_order (nord \<alpha>)" using lem_nord_l unfolding ordIso_def by blast
    moreover have "nord \<alpha> \<noteq> {} \<and> \<not> isSuccOrd (nord \<alpha>)"
      using c1 lem_ord_subemp ordIso_iff_ordLeq lem_osucc_eq[of "nord \<alpha>" \<alpha>] lem_nord_r[of \<alpha
      >] by metis
    ultimately have c2: "fo (nord \<alpha>) = \<Union> (fo ` (underS cmp (nord \<alpha>)))" using b8 b9 b11 by
      metis
    obtain A where c3: "A = \<Union> { D. \<exists> \<beta>::'U rel. \<beta> <o \<alpha> \<and> D = f \<beta> }"
       by blast
    have "\<forall> \<gamma> \<in> underS cmp (nord \<alpha>). \<exists> \<beta>::'U rel. \<beta> <o \<alpha> \<
      and> fo \<gamma> = f \<beta>"
    proof
      fix \<gamma>::"'U rel"
      assume "\<gamma> \<in> underS cmp (nord \<alpha>)"
      then have "\<gamma> \<noteq> nord \<alpha> \<and> (\<gamma>, nord \<alpha>) \<in> oord" unfolding b1
      underS_def by blast
      then have "\<gamma> \<le>o nord \<alpha> \<and> \<gamma> \<in> \<O> \<and> \<not> (\<gamma> =o nord \<
      alpha>)" using lem_Oeq unfolding oord_def by blast
      then have "\<gamma> <o nord \<alpha> \<and> \<gamma> = nord \<gamma>" using lem_Onord
      ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "nord \<alpha> =o \<alpha>" using c1 lem_nord_r by blast
      ultimately have "\<gamma> <o \<alpha> \<and> fo \<gamma> = f \<gamma>" unfolding b5 using
      ordIso_imp_ordLeq ordLess_ordLeq_trans by metis
      then show "\<exists> \<beta>::'U rel. \<beta> <o \<alpha> \<and> fo \<gamma> = f \<beta>" by blast
    qed
    then have c4: "f \<alpha> \<subseteq> A" unfolding c2 c3 b5 by blast
    have "\<forall> \<beta>::'U rel. \<beta> <o \<alpha> \<longrightarrow> (\<exists> \<gamma> \<in> underS cmp
      (nord \<alpha>). f \<beta> = fo \<gamma>)"
    proof (intro allI impI)
      fix \<beta>::"'U rel"
      assume "\<beta> <o \<alpha>"
      then have "(nord \<beta>, nord \<alpha>) \<in> cmp \<and> nord \<beta> \<noteq> nord \<alpha>" using b1
      lem_nord_less by blast
      then have "nord \<beta> \<in> underS cmp (nord \<alpha>)" unfolding underS_def by blast
      then show "\<exists> \<gamma> \<in> underS cmp (nord \<alpha>). f \<beta> = fo \<gamma>" unfolding b5 by
      blast
    qed
    then have "A \<subseteq> f \<alpha>" unfolding c2 c3 b5 by force
    then show "f \<alpha> = \<Union> { D. \<exists> \<beta>::'U rel. \<beta> <o \<alpha> \<and> D = f \<beta> }"
       using c3 c4 by blast
  qed
  moreover have "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using
       b5 lem_nord_eq by metis
  ultimately show ?thesis unfolding sc_ord_def lm_ord_def by blast
qed

lemma lem_lmord_prec:
```

```
fixes \<alpha>::"'a rel" and \<alpha>'::"'b rel"
assumes a1: "\<alpha>' <o \<alpha>" and a2: "isLimOrd \<alpha>"
shows "\<exists> \<beta>::('a rel). \<alpha>' <o \<beta> \<and> \<beta> <o \<alpha>"
proof -
  have "\<not> isSuccOrd \<alpha>" using a1 a2 wo_rel.isLimOrd_def unfolding ordLess_def wo_rel_def by blast
  then obtain \<beta>::"'a rel" where "\<alpha>' <o \<beta> \<and> \<not> (\<alpha> \<le>o \<beta>)" using a1
      lem_ordint_sucord[of \<alpha>' \<alpha>] by blast
  then have "\<alpha>' <o \<beta> \<and> \<beta> <o \<alpha>" using a1 ordIso_imp_ordLeq ordLess_Well_order_simp
    ordLess_imp_ordLeq ordLess_or_ordIso by metis
  then show ?thesis by blast
qed

lemma lem_inford_ge_w:
fixes \<alpha>::"'U rel"
assumes "Well_order \<alpha>" and "\<not> finite (Field \<alpha>)"
shows "\<omega>_ord \<le>o \<alpha>"
  using assms card_of_least infinite_iff_natLeq_ordLeq ordLeq_transitive by blast

lemma lem_ge_w_inford:
fixes \<alpha>::"'U rel"
assumes "\<omega>_ord \<le>o \<alpha>"
shows "\<not> finite (Field \<alpha>)"
  using assms cinfinite_def cinfinite_mono natLeq_cinfinite by blast

lemma lem_fin_card: "finite |A| = finite A"
proof
  assume "finite |A|"
  then show "finite A" using finite_Field by fastforce
next
  assume "finite A"
  then show "finite |A|" using lem_fin_fl_rel by fastforce
qed

lemma lem_cardord_emp: "Card_order ({}::'U rel)"
  by (metis Well_order_empty card_order_on_def ozero_def ozero_ordLeq well_order_on_Well_order)

lemma lem_card_emprel: "|{}::'U rel| =o ({}::'U rel)"
proof -
  have "({}::'U rel) =o |{}::'U set|" using lem_cardord_emp BNF_Cardinal_Order_Relation.card_of_unique by simp
  then show ?thesis using card_of_empty_ordIso ordIso_symmetric ordIso_transitive by blast
qed

lemma lem_cord_lin: "Card_order \<alpha> \<Longrightarrow> Card_order \<beta> \<Longrightarrow> ( \<alpha> \<le>
      o \<beta>) = ( \<not> ( \<beta> <o \<alpha> ) )" by simp

lemma lem_co_one_ne_min:
fixes \<alpha>::"'U rel" and a::"'a"
assumes "Well_order \<alpha>" and "\<alpha> \<noteq> {}"
shows "|{a}| \<le>o \<alpha>"
proof -
  have "Field \<alpha> \<noteq> {}" using assms unfolding Field_def by force
  then have "|{a}| \<le>o |Field \<alpha>|" using assms by simp
  moreover have "|Field \<alpha>| \<le>o \<alpha>" using assms card_of_least by blast
  ultimately show ?thesis using ordLeq_transitive by blast
qed

lemma lem_rel_inf_fld_card:
fixes r::"'U rel"
assumes "\<not> finite r"
shows "|Field r| =o |r|"
proof -
  obtain f1::"'U \<times> 'U \<Rightarrow> 'U" where b1: "f1 = (\<lambda> (x,y). x)" by blast
  obtain f2::"'U \<times> 'U \<Rightarrow> 'U" where b2: "f2 = (\<lambda> (x,y). y)" by blast
  then have "f1 ` r = Domain r \<and> f2 ` r = Range r" using b1 b2 by force
  then have b3: "|Domain r| \<le>o |r| \<and> |Range r| \<le>o |r|"
    using card_of_image[of f1 r] card_of_image[of f2 r] by simp
  have "|Domain r| \<le>o |Range r| \<or> |Range r| \<le>o |Domain r|" by (simp add: ordLeq_total)
  moreover have "|Domain r| \<le>o |Range r| \<longrightarrow> |Field r| \<le>o |r|"
  proof
    assume c1: "|Domain r| \<le>o |Range r|"
    moreover have "finite (Domain r) \<and> finite (Range r) \<longrightarrow> finite (Field r)" unfolding
      Field_def by blast
    ultimately have "\<not> finite (Range r)"
      using assms lem_fin_fl_rel card_of_ordLeq_finite by blast
    then have "|Field r| =o |Range r|" using c1 card_of_Un_infinite unfolding Field_def by blast
    then show "|Field r| \<le>o |r|" using b3 ordIso_ordLeq_trans by blast
  qed
  moreover have "|Range r| \<le>o |Domain r| \<longrightarrow> |Field r| \<le>o |r|"
```

```
  proof
    assume c1: "|Range r| \<le>o |Domain r|"
    moreover have "finite (Domain r) \<and> finite (Range r) \<longrightarrow> finite (Field r)" unfolding
      Field_def by blast
    ultimately have "\<not> finite (Domain r)"
      using assms lem_fin_fl_rel card_of_ordLeq_finite by blast
    then have "|Field r| =o |Domain r|" using c1 card_of_Un_infinite unfolding Field_def by blast
    then show "|Field r| \<le>o |r|" using b3 ordIso_ordLeq_trans by blast
  qed
  ultimately have "|Field r| \<le>o |r|" by blast
  moreover have "|r| \<le>o |Field r|"
  proof -
    have "r \<subseteq> (Field r) \<times> (Field r)" unfolding Field_def by force
    then have c1: "|r| \<le>o |Field r \<times> Field r|" by simp
    have "\<not> finite (Field r)" using assms lem_fin_fl_rel by blast
    then have c2: "|Field r \<times> Field r| =o |Field r|" by simp
    show ?thesis using c1 c2 using ordLeq_ordIso_trans by blast
  qed
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed

lemma lem_cardreleq_cardfldeq_inf:
fixes r1 r2:: "'U rel"
assumes a1: "|r1| =o |r2|" and a2: "\<not> finite r1 \<or> \<not> finite r2"
shows "|Field r1| =o |Field r2|"
proof -
  have "\<not> finite r1 \<and> \<not> finite r2" using a1 a2 by simp
  then have "|Field r1| =o |r1| \<and> |Field r2| =o |r2|" using lem_rel_inf_fld_card by blast
  then show "|Field r1| =o |Field r2|" using a1 by (meson ordIso_symmetric ordIso_transitive)
qed

lemma lem_card_un_bnd:
fixes S::"'a set set" and \<alpha>::"'U rel"
assumes a3: "\<forall>A\<in>S. |A| \<le>o \<alpha>" and a4: "|S| \<le>o \<alpha>" and a5: "\<omega>_ord \<le>o
      \<alpha>"
shows "| \<Union> S | \<le>o \<alpha>"
proof -
  obtain \<alpha>' where b0: "\<alpha>' = |Field \<alpha>|" by blast
  have a3': "\<forall>A\<in>S. |A| \<le>o \<alpha>'"
  proof
    fix A
    assume "A \<in> S"
    then have "|A| \<le>o \<alpha>" using a3 by blast
    moreover have "Card_order |A|" by simp
    ultimately show "|A| \<le>o \<alpha>'" using b0 card_of_unique card_of_mono2 ordIso_ordLeq_trans by blast
  qed
  have "Card_order |S|" by simp
  then have a4': "|S| \<le>o \<alpha>'" using b0 a4 card_of_unique card_of_mono2 ordIso_ordLeq_trans by blast
  have a5': "\<not> finite (Field \<alpha>')"
  proof -
    have "Card_order \<alpha>'" using b0 by simp
    then have "|Field \<alpha>| =o |Field \<alpha>'|" using b0 card_of_unique by blast
    moreover have "\<not> finite (Field \<alpha>)" using a5 lem_ge_w_inford by blast
    ultimately show "\<not> finite (Field \<alpha>')" by simp
  qed
  have a0': "\<alpha>' \<le>o \<alpha>" using b0 a4 by simp
  obtain r where b1: "r = \<Union> S" by blast
    have "\<forall> A \<in> S. |A| \<le>o \<alpha>'" using a3' ordIso_ordLeq_trans by blast
    moreover have "r = (\<Union>A\<in>S. A)" using b1 by blast
    moreover have "Card_order \<alpha>'" using b0 by simp
    ultimately have "|r| \<le>o \<alpha>'" using a4' a5' card_of_UNION_ordLeq_infinite_Field[of \<alpha>' S "\<
      lambda> x. x"] by blast
  then have "| \<Union> S | \<le>o \<alpha>'" unfolding b1 using ordLeq_transitive by blast
  then show "| \<Union> S | \<le>o \<alpha>" using a0' ordLeq_transitive by blast
qed

lemma lem_ord_suc_ge_w:
fixes \<alpha>0 \<alpha>::"'U rel"
assumes a1: "\<omega>_ord \<le>o \<alpha>" and a2: "sc_ord \<alpha>0 \<alpha>"
shows "\<omega>_ord \<le>o \<alpha>0"
proof -
  obtain N::"'U set" where b1: "|N| =o \<omega>_ord" using a1
    by (metis card_of_nat Field_natLeq card_of_mono2 internalize_card_of_ordLeq ordIso_symmetric
      ordIso_transitive)
  have "\<alpha>0 <o |N| \<longrightarrow> False"
  proof
    assume c1: "\<alpha>0 <o |N|"
    have "Well_order \<omega>_ord \<and> isLimOrd \<omega>_ord"
```

```
      by (metis natLeq_Well_order Field_natLeq card_of_nat card_order_infinite_isLimOrd
       infinite_iff_natLeq_ordLeq natLeq_Card_order ordIso_iff_ordLeq)
    then have "\<not> isSuccOrd \<omega>_ord" using wo_rel.isLimOrd_def unfolding wo_rel_def by blast
    then have "\<not> isSuccOrd |N|" using b1 lem_osucc_eq by blast
    then have "\<not> (\<forall>\<gamma>::'U rel. \<alpha>0 <o \<gamma> \<longrightarrow> |N| \<le>o \<gamma>)"
      using c1 unfolding sc_ord_def using lem_ordint_sucord[of \<alpha>0 "|N|"] by blast
    then obtain \<beta>::"'U rel" where "\<alpha>0 <o \<beta> \<and> \<beta> <o |N|"
      using card_of_Well_order not_ordLeq_iff_ordLess ordLess_Well_order_simp by blast
    moreover then have "\<alpha> \<le>o \<beta>" using a2 unfolding sc_ord_def by blast
    ultimately have "\<alpha> <o |N|" using ordLeq_ordLess_trans by blast
    then show "False" using a1 b1 using not_ordLess_ordLeq ordIso_iff_ordLeq ordLeq_transitive by blast
  qed
  moreover have "Well_order \<alpha>0" using a2 unfolding sc_ord_def ordLess_def by blast
  moreover have "Well_order |N|" by simp
  ultimately show ?thesis using b1 not_ordLess_iff_ordLeq ordIso_iff_ordLeq ordLeq_transitive by blast
qed

lemma lem_restr_ordbnd:
fixes r::"'U rel" and A::"'U set" and \<alpha>::"'U rel"
assumes a1: "\<omega>_ord \<le>o \<alpha>" and a2: "|A| \<le>o \<alpha>"
shows "|Restr r A| \<le>o \<alpha>"
proof (cases "finite A")
  assume "finite A"
  then have "finite (Restr r A)" by blast
  then have "|Restr r A| <o \<omega>_ord" using finite_iff_ordLess_natLeq by blast
  then show "|Restr r A| \<le>o \<alpha>" using a1 ordLeq_transitive ordLess_imp_ordLeq by blast
next
  assume "\<not> finite A"
  then have "|A \<times> A| =o |A|" by simp
  moreover have "|Restr r A| \<le>o |A \<times> A|" by simp
  ultimately show "|Restr r A| \<le>o \<alpha>" using a2 ordLeq_ordIso_trans ordLeq_transitive by blast
qed

lemma lem_card_inf_lim:
fixes r::"'U rel"
assumes a1: "Card_order \<alpha>" and a2: "\<omega>_ord \<le>o \<alpha>"
shows "\<not>( \<alpha> = {} \<or> isSuccOrd \<alpha> )"
proof -
  obtain s where "s = Field \<alpha>" by blast
  then have "|s| =o \<alpha>" using a1 card_of_Field_ordIso by blast
  moreover then have "\<not> ( |s| <o |UNIV :: nat set| )" using a2
    by (metis card_of_nat ordLess_ordIso_trans not_ordLess_ordIso ordLeq_iff_ordLess_or_ordIso
      ordLeq_ordLess_trans)
  ultimately have "\<not> finite (Field \<alpha>)" using lem_fin_card lem_fin_fl_rel by (metis
      finite_iff_cardOf_nat ordIso_finite_Field)
  moreover then have "\<alpha> \<noteq> {}" by force
  moreover have "wo_rel \<alpha>" using a1 unfolding wo_rel_def card_order_on_def by blast
  ultimately show ?thesis using a1 card_order_infinite_isLimOrd wo_rel.isLimOrd_def by blast
qed

lemma lem_card_nreg_inf_osetlm:
fixes \<alpha>::"'U rel"
assumes a1: "Card_order \<alpha>" and a2: "\<not> regularCard \<alpha>" and a3: "\<not> finite (Field \<alpha>)"
shows "\<exists> S::'U rel set. |S| <o \<alpha> \<and> (\<forall> \<alpha>'\<in>S. \<alpha>' <o \<alpha>) \<and>
       (\<forall> \<alpha>'::'U rel. \<alpha>' <o \<alpha> \<longrightarrow> (\<exists> \<beta> \<in> S. \<alpha
      >' \<le>o \<beta>))"
proof -
  obtain K::"'U set" where b1: "K \<subseteq> Field \<alpha> \<and> cofinal K \<alpha>" and b2: "\<not> |K| =o
      \<alpha>"
    using a2 unfolding regularCard_def by blast
  have b3: "|K| <o \<alpha>"
  proof -
    have "|K| \<le>o |Field \<alpha>|" using b1 by simp
    moreover have "|Field \<alpha>| =o \<alpha>" using a1 card_of_Field_ordIso by blast
    ultimately show "|K| <o \<alpha>" using a1 b2
      by (metis card_of_Well_order card_order_on_def not_ordLeq_ordLess ordIso_or_ordLess ordIso_ordLess_trans)
  qed
  have b4: "isLimOrd \<alpha>" using a1 a3 card_order_infinite_isLimOrd by blast
  obtain f::"'U \<Rightarrow> 'U rel" where b5: "f = (\<lambda> a. Restr \<alpha> (under \<alpha> a))" by blast
  obtain S::"'U rel set" where b6: "S = f ` K" by blast
  then have "|S| <o \<alpha>" using b3 card_of_image ordLeq_ordLess_trans by blast
  moreover have "\<forall> \<alpha>'\<in>S. \<alpha>' <o \<alpha>"
  proof
    fix \<alpha>'::"'U rel"
    assume c1: "\<alpha>' \<in> S"
    then obtain a where c2: "a \<in> K \<and> \<alpha>' = Restr \<alpha> (under \<alpha> a)" using b5 b6 by
      blast
```

```
  then have c3: "Well_order \<alpha>' \<and> Well_order \<alpha>" using a1 Well_order_Restr unfolding
    card_order_on_def by blast
  moreover have "embed \<alpha>' \<alpha> id"
  proof -
    have "ofilter \<alpha> (under \<alpha> a)" using c3 wo_rel.under_ofilter[of \<alpha>] unfolding wo_rel_def
     by blast
    moreover then have "under \<alpha> a \<subseteq> Field \<alpha>" unfolding ofilter_def by blast
    ultimately show ?thesis using c2 c3 ofilter_embed[of \<alpha> "under \<alpha> a"] by blast
  qed
  moreover have "bij_betw id (Field \<alpha>') (Field \<alpha>) \<longrightarrow> False"
  proof
    assume "bij_betw id (Field \<alpha>') (Field \<alpha>)"
    then have d1: "Field \<alpha>' = Field \<alpha>" unfolding bij_betw_def by simp
    have "a \<in> Field \<alpha>" using c2 b1 by blast
    then obtain b where d2: "b \<in> aboveS \<alpha> a"
      using b4 c3 wo_rel.isLimOrd_aboveS[of \<alpha> a] unfolding wo_rel_def by blast
    then have "b \<in> Field \<alpha>'" using d1 unfolding aboveS_def Field_def by blast
    then have "b \<in> under \<alpha> a" using c2 unfolding Field_def by blast
    then show "False" using a1 d2 unfolding under_def aboveS_def
      card_order_on_def well_order_on_def linear_order_on_def partial_order_on_def antisym_def by blast
  qed
  ultimately show "\<alpha>' <o \<alpha>" using embedS_def unfolding ordLess_def by blast
qed
moreover have "\<forall> \<alpha>'::'U rel. \<alpha>' <o \<alpha> \<longrightarrow> (\<exists> \<beta> \<in> S
    . \<alpha>' \<le>o \<beta>)"
proof (intro allI impI)
  fix \<alpha>'::"'U rel"
  assume c1: "\<alpha>' <o \<alpha>"
  then obtain g where c2: "embed \<alpha>' \<alpha> g \<and> \<not> bij_betw g (Field \<alpha>') (Field \<
    alpha>)"
    using embedS_def unfolding ordLess_def by blast
  then have "g ` Field \<alpha>' \<noteq> Field \<alpha>"
    using c1 embed_inj_on unfolding ordLess_def bij_betw_def by blast
  moreover have "g ` Field \<alpha>' \<subseteq> Field \<alpha>"
    using c1 c2 embed_in_Field[of \<alpha>' \<alpha> g] unfolding ordLess_def by fast
  ultimately obtain a where c3: "a \<in> Field \<alpha> - (g ` Field \<alpha>')" by blast
  then obtain b \<beta> where c4: "b \<in> K \<and> (a, b) \<in> \<alpha> \<and> \<beta> = f b" using b1
    unfolding cofinal_def by blast
  then have "\<beta> \<in> S" using b6 by blast
  moreover have "\<alpha>' \<le>o \<beta>"
  proof -
    have d1: "Well_order \<beta>" using c4 b5 a1 Well_order_Restr unfolding card_order_on_def by blast
    moreover have "embed \<alpha>' \<beta> g"
    proof -
      have e1: "\<forall>x y. (x, y) \<in> \<alpha>' \<longrightarrow> (g x, g y) \<in> \<beta>"
      proof (intro allI impI)
        fix x y
        assume f1: "(x, y) \<in> \<alpha>'"
        then have f2: "(g x, g y) \<in> \<alpha>" using c2 embed_compat unfolding compat_def by blast
        moreover have "g y \<in> under \<alpha> b"
        proof -
          have "(b, g y) \<in> \<alpha> \<longrightarrow> False"
          proof
            assume "(b, g y) \<in> \<alpha>"
            moreover have "(a, b) \<in> \<alpha>" using c4 by blast
            ultimately have "(a, g y) \<in> \<alpha>" using a1 unfolding under_def card_order_on_def
              well_order_on_def linear_order_on_def partial_order_on_def preorder_on_def trans_def by blast
            then have "a \<in> under \<alpha> (g y)" unfolding under_def by blast
            moreover have "bij_betw g (under \<alpha>' y) (under \<alpha> (g y))"
              using f1 c2 unfolding embed_def Field_def by blast
            ultimately obtain y' where "y' \<in> under \<alpha>' y \<and> a = g y'" unfolding bij_betw_def by
    blast
            moreover then have "y' \<in> Field \<alpha>'" unfolding under_def Field_def by blast
            ultimately have "a \<in> g ` Field \<alpha>'" by blast
            then show "False" using c3 by blast
          qed
          moreover have "g y \<in> Field \<alpha> \<and> b \<in> Field \<alpha>" using f2 c4 unfolding
    Field_def by blast
          ultimately have "(g y, b) \<in> \<alpha>" using a1 unfolding card_order_on_def well_order_on_def
            linear_order_on_def partial_order_on_def preorder_on_def refl_on_def total_on_def by metis
          then show ?thesis unfolding under_def by blast
        qed
        moreover then have "g x \<in> under \<alpha> b" using a1 f2 unfolding under_def card_order_on_def
          well_order_on_def linear_order_on_def partial_order_on_def preorder_on_def trans_def by blast
        ultimately have "(g x, g y) \<in> Restr \<alpha> (under \<alpha> b)" by blast
        then show "(g x, g y) \<in> \<beta>" using c4 b5 by blast
      qed
      have e2: "\<forall>x \<in> g ` Field \<alpha>'. under \<beta> x \<subseteq> g ` Field \<alpha>'"
```

```
        proof
          fix x
          assume "x \<in> g ` Field \<alpha>'"
          then obtain c where f1: "c \<in> Field \<alpha>' \<and> x = g c" by blast
          have "\<forall> x'. (x', x) \<in> \<beta> \<longrightarrow> x' \<in> g ` Field \<alpha>'"
          proof (intro allI impI)
            fix x'
            assume "(x', x) \<in> \<beta>"
            then have "(x', g c) \<in> Restr \<alpha> (under \<alpha> b)" using b5 f1 c4 by blast
            then have "x' \<in> under \<alpha> (g c)" unfolding under_def by blast
            moreover have "bij_betw g (under \<alpha>' c) (under \<alpha> (g c))" using f1 c2 unfolding
      embed_def by blast
            ultimately obtain c' where "x' = g c' \<and> c' \<in> under \<alpha>' c" unfolding bij_betw_def by
      blast
            moreover then have "c' \<in> Field \<alpha>'" unfolding under_def Field_def by blast
            ultimately show "x' \<in> g ` Field \<alpha>'" by blast
          qed
          then show "under \<beta> x \<subseteq> g ` Field \<alpha>'" unfolding under_def by blast
        qed
        have "compat \<alpha>' \<beta> g" using e1 unfolding compat_def by blast
        moreover then have "ofilter \<beta> (g ` Field \<alpha>')" using e2 unfolding ofilter_def compat_def
      Field_def by blast
        moreover have "inj_on g (Field \<alpha>')" using c1 c2 embed_inj_on unfolding ordLess_def by blast
        ultimately show ?thesis using d1 c1 embed_iff_compat_inj_on_ofilter[of \<alpha>' \<beta> g]
          unfolding ordLess_def by blast
      qed
      ultimately show ?thesis using c1 unfolding ordLess_def ordLeq_def by blast
    qed
    ultimately show "\<exists> \<beta> \<in> S. \<alpha>' \<le>o \<beta>" by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_card_un_bnd_stab:
fixes S::"'a set set" and \<alpha>::"'U rel"
assumes "stable \<alpha>" and "\<forall>A\<in>S. |A| <o \<alpha>" and "|S| <o \<alpha>"
shows "| \<Union> S | <o \<alpha>"
  using assms stable_UNION[of \<alpha> S "\<lambda> x. x"] by simp

lemma lem_finwo_cardord: "finite \<alpha> \<Longrightarrow> Well_order \<alpha> \<Longrightarrow> Card_order \<
      alpha>"
proof -
  assume a1: "finite \<alpha>" and a2: "Well_order \<alpha>"
  have "\<forall>r. well_order_on (Field \<alpha>) r \<longrightarrow> \<alpha> \<le>o r"
  proof (intro allI impI)
    fix r
    assume "well_order_on (Field \<alpha>) r"
    moreover have "well_order_on (Field \<alpha>) \<alpha>" using a2 by blast
    moreover have "finite (Field \<alpha>)" using a1 finite_Field by fastforce
    ultimately have "\<alpha> =o r" using finite_well_order_on_ordIso by blast
    then show "\<alpha> \<le>o r" using ordIso_iff_ordLeq by blast
  qed
  then show ?thesis using a2 unfolding card_order_on_def by blast
qed

lemma lem_finwo_le_w: "finite \<alpha> \<Longrightarrow> Well_order \<alpha> \<Longrightarrow> \<alpha> <o
      natLeq"
proof -
  assume a1: "finite \<alpha>" and a2: "Well_order \<alpha>"
  then have "|Field \<alpha>| =o \<alpha>" using lem_finwo_cardord by (metis card_of_Field_ordIso)
  moreover have "finite (Field \<alpha>)" using a1 finite_Field by fastforce
  moreover then have "|Field \<alpha>| <o natLeq" using finite_iff_ordLess_natLeq by blast
  ultimately show "\<alpha> <o natLeq" using ordIso_iff_ordLeq ordLeq_ordLess_trans by blast
qed

lemma lem_wolew_fin: "\<alpha> <o natLeq \<Longrightarrow> finite \<alpha>"
proof -
  assume a1: "\<alpha> <o natLeq"
  then have "Well_order \<alpha>" using a1 unfolding ordLess_def by blast
  then have "|Field \<alpha>| \<le>o \<alpha>" using card_of_least[of "Field \<alpha>" \<alpha>] by blast
  then have "\<not> (natLeq \<le>o |Field \<alpha>| )" using a1 by (metis BNF_Cardinal_Order_Relation.
      ordLess_Field not_ordLeq_ordLess)
  then have "finite (Field \<alpha>)" using infinite_iff_natLeq_ordLeq by blast
  then show "finite \<alpha>" using finite_subset trancl_subset_Field2 by fastforce
qed

lemma lem_wolew_nat:
assumes a1: "\<alpha> <o natLeq" and a2: "n = card (Field \<alpha>)"
```

```
shows "\<alpha> =o (natLeq_on n)"
proof -
  have b1: "Well_order \<alpha>" using a1 unfolding ordLess_def by blast
  have b2: "finite \<alpha>" using a1 lem_wolew_fin by blast
  then have "finite (Field \<alpha>)" using a1 finite_Field by fastforce
  then have "|Field \<alpha>| =o natLeq_on n" using a2 finite_imp_card_of_natLeq_on[of "Field \<alpha>"] by
      blast
  moreover have "|Field \<alpha>| =o \<alpha>" using b1 b2 lem_finwo_cardord by (metis card_of_Field_ordIso)
  ultimately show ?thesis using ordIso_symmetric ordIso_transitive by blast
qed

lemma lem_cntset_enum:  "|A| =o natLeq \<Longrightarrow> (\<exists> f. A = f ` (UNIV::nat set))"
proof -
  assume "|A| =o natLeq"
  moreover have "|UNIV::nat set| =o natLeq" using card_of_nat by blast
  ultimately have "|UNIV::nat set| =o |A|" by (meson ordIso_iff_ordLeq ordIso_ordLeq_trans)
  then obtain f where "bij_betw f (UNIV::nat set) A" using card_of_ordIso by blast
  then have "A = f ` (UNIV::nat set)" unfolding bij_betw_def by blast
  then show ?thesis by blast
qed

lemma lem_oord_int_card_le_inf:
fixes \<alpha>::"'U rel"
assumes "\<omega>_ord \<le>o \<alpha>"
shows "|{ \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> }| \<le>o \<alpha>"
proof -
  obtain f::"'U \<Rightarrow> 'U rel" where b1: "f = (\<lambda> a. nord (Restr \<alpha> (underS \<alpha> a)))"
      by blast
  have "\<forall> \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> \<longrightarrow> \<gamma> \<in> f ` (
      Field \<alpha>)"
  proof (intro ballI impI)
    fix \<gamma>::"'U rel"
    assume c1: "\<gamma> \<in> \<O>" and c2: "\<gamma> <o \<alpha>"
    have "\<exists> a \<in> Field \<alpha>. \<gamma> =o Restr \<alpha> (underS \<alpha> a)"
      using c2 ordLess_iff_ordIso_Restr[of \<alpha> \<gamma>] unfolding ordLess_def by blast
    then obtain a where "a \<in> Field \<alpha> \<and> \<gamma> =o Restr \<alpha> (underS \<alpha> a)" by blast
    moreover then have "\<gamma> = f a" using c1 b1 lem_nord_eq lem_Onord by blast
    ultimately show "\<gamma> \<in> f ` (Field \<alpha>)" by blast
  qed
  then have "{ \<gamma> \<in> \<O>::'U rel set. \<gamma><o \<alpha> } \<subseteq> f ` (Field \<alpha>)" by blast
  then have "|{ \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> }| \<le>o |f ` (Field \<alpha>)|" by simp
  moreover have "|f ` (Field \<alpha>)| \<le>o |Field \<alpha>|" by simp
  ultimately have "|{ \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> }| \<le>o |Field \<alpha>|" using
      ordLeq_transitive by blast
  moreover have "|Field \<alpha>| \<le>o \<alpha>" using assms by simp
  ultimately show ?thesis using ordLeq_transitive by blast
qed

lemma lem_oord_card_le_int_inf:
fixes \<alpha>::"'U rel"
assumes a1: "Card_order \<alpha>" and a2: "\<omega>_ord \<le>o \<alpha>"
shows "\<alpha> \<le>o |{ \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> }|"
proof -
  obtain \<alpha>' where b0: "\<alpha>' = |Field \<alpha>|" by blast
  then have b0': "Card_order \<alpha>' \<and> \<alpha> =o \<alpha>'" using a1 card_of_unique by simp
  then have b0'': "\<omega>_ord \<le>o \<alpha>'" using a2 ordLeq_ordIso_trans by blast
  obtain f::"'U \<Rightarrow> 'U rel" where b1: "f = (\<lambda> a. Restr \<alpha>' (under \<alpha>' a))" by
      blast
  have b2: "Well_order \<alpha>'" using b0 by simp
  have b3: "\<forall>a \<in> Field \<alpha>'. \<forall>b \<in> Field \<alpha>'. f a =o f b \<longrightarrow> a =
       b"
  proof (intro ballI impI)
    fix a b
    assume d1: "a \<in> Field \<alpha>'" and d2: "b \<in> Field \<alpha>'" and "f a =o f b"
    then have d3: "f a \<le>o f b \<and> f b \<le>o f a" using ordIso_iff_ordLeq by blast
    obtain A B where d4: "A = under \<alpha>' a \<and> B = under \<alpha>' b" by blast
    have d5: "Well_order \<alpha>'" using b0 by simp
    moreover then have "wo_rel.ofilter \<alpha>' A \<and> wo_rel.ofilter \<alpha>' B"
      using d4 wo_rel_def wo_rel.under_ofilter[of \<alpha>'] by blast
    moreover have "Restr \<alpha>' A \<le>o Restr \<alpha>' B" and "Restr \<alpha>' B \<le>o Restr \<alpha>' A"
      using d3 d4 b1 by blast+
    ultimately have "A = B" using ofilter_subset_ordLeq[of \<alpha>'] by blast
    then have "under \<alpha>' a = under \<alpha>' b" using d4 by blast
    moreover have "(a,a) \<in> \<alpha>' \<and> (b,b) \<in> \<alpha>'" using d1 d2 d5
      by (metis preorder_on_def partial_order_on_def linear_order_on_def
          well_order_on_def refl_on_def)
    ultimately have "(a,b) \<in> \<alpha>' \<and> (b,a) \<in> \<alpha>'" unfolding under_def by blast
    then show "a = b" using d5
```

```
      by (metis partial_order_on_def linear_order_on_def well_order_on_def antisym_def)
  qed
  have b4: "\<forall> a \<in> Field \<alpha>'. f a <o \<alpha>'"
  proof
    fix a
    assume c1: "a \<in> Field \<alpha>'"
    have "under \<alpha>' a \<subset> Field \<alpha>'"
    proof -
      have "\<not> finite \<alpha>'" using b0'' Field_natLeq finite_Field infinite_UNIV_nat ordLeq_finite_Field
       by metis
      then have "\<not> finite (Field \<alpha>')" using lem_fin_fl_rel by blast
      then obtain a' where "a' \<in> Field \<alpha>' \<and> a \<noteq> a' \<and> (a, a') \<in> \<alpha>'"
        using c1 b0' infinite_Card_order_limit[of \<alpha>' a] by blast
      moreover then have "(a', a) \<notin> \<alpha>'" using b2 unfolding well_order_on_def
        linear_order_on_def partial_order_on_def antisym_def by blast
      ultimately show ?thesis unfolding under_def Field_def by blast
    qed
    moreover have "ofilter \<alpha>' (under \<alpha>' a)"
      using b2 wo_rel.under_ofilter[of \<alpha>'] unfolding wo_rel_def by blast
    ultimately show "f a <o \<alpha>'" unfolding b1 using b2 ofilter_ordLess by blast
  qed
  obtain g where b5: "g = nord \<circ> f" by blast
  have "\<forall>x\<in>Field \<alpha>'. \<forall>y\<in>Field \<alpha>'. g x = g y \<longrightarrow> x = y"
  proof (intro ballI impI)
    fix x y
    assume c1: "x \<in> Field \<alpha>'" and c2: "y \<in> Field \<alpha>'" and "g x = g y"
    then have "Well_order (f x) \<and> Well_order (f y) \<and> nord (f x) = nord (f y)"
      using b4 b5 unfolding ordLess_def by simp
    then have "f x =o f y" using lem_nord_req by blast
    then show "x = y" using c1 c2 b3 by blast
  qed
  then have "inj_on g (Field \<alpha>')" unfolding inj_on_def by blast
  moreover have "\<forall> a \<in> Field \<alpha>'. g a \<in> \<O> \<and> g a <o \<alpha>'"
  proof
    fix a
    assume "a \<in> Field \<alpha>'"
    then have "f a <o \<alpha>'" using b4 by blast
    then have "nord (f a) <o \<alpha>' \<and> nord (f a) \<in> \<O>" using lem_nord_ls_l lem_nordO_ls_l by blast
    then show "g a \<in> \<O> \<and> g a <o \<alpha>'" using b5 by simp
  qed
  ultimately have "|Field \<alpha>'| \<le>o |{\<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha>'}|"
    using card_of_ordLeq[of "Field \<alpha>'" "{\<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha>'}"] by
      blast
  moreover have "\<alpha> =o |Field \<alpha>'|" using b0 a1 by simp
  moreover have "{\<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha>'} = {\<gamma> \<in> \<O>::'U rel set. \<
      gamma> <o \<alpha>}"
    using b0' using ordIso_iff_ordLeq ordLess_ordLeq_trans by blast
  ultimately show ?thesis using ordIso_ordLeq_trans by simp
qed

lemma lem_ord_int_card_le_inf:
fixes \<alpha>::"'U rel" and f :: "'U rel \<Rightarrow> 'a"
assumes "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" and "\<omega>
      _ord \<le>o \<alpha>"
shows "|f ` { \<gamma>::'U rel. \<gamma> <o \<alpha> }| \<le>o \<alpha>"
proof -
  obtain I where b1: "I = { \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> }" by blast
  have "f`{ \<gamma>::'U rel. \<gamma> <o \<alpha> } \<subseteq> f`I"
  proof
    fix a
    assume "a \<in> f`{ \<gamma>::'U rel. \<gamma> <o \<alpha> }"
    then obtain \<gamma> where "a = f \<gamma> \<and> \<gamma> <o \<alpha>" by blast
    moreover then have "nord \<gamma> =o \<gamma> \<and> nord \<gamma> \<in> I"
      using b1 lem_nord_r lem_nord_ls_l lem_nordO_ls_l ordLess_def by blast
    ultimately have "a = f (nord \<gamma>) \<and> nord \<gamma> \<in> I " using assms by metis
    then show "a \<in> f`I" by blast
  qed
  then have "|f`{ \<gamma>::'U rel. \<gamma> <o \<alpha> }| \<le>o |f`I|" by simp
  moreover have "|f`I| \<le>o |I|" by simp
  moreover have "|I| \<le>o \<alpha>" using b1 assms lem_oord_int_card_le_inf by blast
  ultimately show ?thesis using ordLeq_transitive by metis
qed

lemma lem_card_setcv_inf_stab:
fixes \<alpha>::"'U rel" and A::"'U set"
assumes a1: "Card_order \<alpha>" and a2: "\<omega>_ord \<le>o \<alpha>" and a3: "|A| \<le>o \<alpha>"
shows "\<exists> f::('U rel \<Rightarrow> 'U). A \<subseteq> f `{ \<gamma>::'U rel. \<gamma> <o \<alpha> } \<and
      > (\<forall> \<gamma>1 \<gamma>2. \<gamma>1 =o \<gamma>2 \<longrightarrow> f \<gamma>1 = f \<gamma>2)"
```

```
proof -
  obtain B where b1: "B = { \<gamma> \<in> \<O>::'U rel set. \<gamma> <o \<alpha> }" by blast
  then have "|A| \<le>o |B|"
    using a1 a2 a3 lem_oord_card_le_int_inf[of \<alpha>] ordLeq_transitive by blast
  then obtain g where b2: "A \<subseteq> g `B" by (metis card_of_ordLeq2 empty_subsetI order_refl)
  obtain f where b3: "f = g \<circ> nord" by blast
  have "A \<subseteq> f `{ \<gamma>::'U rel. \<gamma> <o \<alpha> }"
  proof
    fix a
    assume "a \<in> A"
    then obtain \<gamma>::"'U rel" where "\<gamma> \<in> \<O> \<and> \<gamma> <o \<alpha> \<and> a = g \<gamma>"
       using b1 b2 by blast
    moreover then have "f \<gamma> = g \<gamma>" using b3 lem_Onord by force
    ultimately show "a \<in> f `{ \<gamma>::'U rel. \<gamma> <o \<alpha> }" by force
  qed
  moreover have "\<forall> \<gamma>1 \<gamma>2. \<gamma>1 =o \<gamma>2 \<longrightarrow> f \<gamma>1 = f \<gamma
      >2" using b3 lem_nord_eq by force
  ultimately show ?thesis by blast
qed

lemma lem_jnfix_gen:
fixes I::"'i set" and leI::"'i rel" and L::"'l set"
  and t::"'i\<times>'l \<Rightarrow> 'i \<Rightarrow> 'n" and jnN::"'n \<Rightarrow> 'n \<Rightarrow> 'n"
assumes a1:"\<not> finite L"
    and a2: "|L| <o |I|"
    and a3: "\<forall>\<alpha>\<in>I. (\<alpha>,\<alpha>) \<in> leI"
    and a4: "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. \<forall>\<gamma>\<in>I. (\<alpha>,\<beta>)\<in>
      leI \<and> (\<beta>,\<gamma>)\<in>leI \<longrightarrow> (\<alpha>,\<gamma>)\<in>leI"
    and a5: "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. (\<alpha>,\<beta>) \<in> leI \<or> (\<beta>,\<
      alpha>) \<in> leI"
    and a6: "\<forall>\<beta>\<in>I. |{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o |L|"
    and a7: "\<forall>\<alpha>\<in>I. \<exists>\<alpha>'\<in>I. (\<alpha>,\<alpha>') \<in> leI \<and> (\<alpha
      >',\<alpha>) \<notin> leI"
shows "\<exists> h. \<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. \<forall>i\<in>L. \<forall>j\<in>L. \<
      exists> \<gamma>\<in>I. (\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>)\<in>leI \<and> (\<gamma>,\<
      alpha>)\<notin>leI \<and> (\<gamma>,\<beta>)\<notin>leI
           \<and> h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)"
proof -
  obtain inc where p1: "inc = (\<lambda> \<alpha>. SOME \<alpha>'. \<alpha>' \<in> I \<and> (\<alpha>,\<alpha>')
       \<in> leI \<and> (\<alpha>',\<alpha>) \<notin> leI)" by blast
  have p2: "\<And> \<alpha>. \<alpha> \<in> I \<Longrightarrow> (inc \<alpha>) \<in> I \<and> (\<alpha>, inc \<
      alpha>) \<in> leI \<and> (inc \<alpha>, \<alpha>) \<notin> leI"
  proof -
    fix \<alpha>
    assume "\<alpha> \<in> I"
    moreover obtain P where c1: "P = (\<lambda> \<alpha>'. \<alpha>' \<in> I \<and> (\<alpha>,\<alpha>') \<in>
      leI \<and> (\<alpha>',\<alpha>) \<notin> leI)" by blast
    ultimately have "\<exists> \<alpha>'. P \<alpha>'" using a7 by blast
    then have "P (SOME x. P x)" using someI_ex by metis
    moreover have "inc \<alpha> = (SOME x. P x)" using c1 p1 by blast
    ultimately show "(inc \<alpha>) \<in> I \<and> (\<alpha>,inc \<alpha>) \<in> leI \<and> (inc \<alpha>, \<
      alpha>) \<notin> leI" using c1 by simp
  qed
  obtain mxI where m0: "mxI = (\<lambda> \<alpha> \<beta>. (if ((\<alpha>,\<beta>) \<in> leI) then \<beta> else
      \<alpha>))" by blast
  then have m1: "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. mxI \<alpha> \<beta> \<in> I" by simp
  obtain maxI where b0: "maxI = (\<lambda> \<alpha> \<beta>. inc (mxI \<alpha> \<beta>))" by blast
  have q1: "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. maxI \<alpha> \<beta> \<in> I" using p2 b0 m0 by
      simp
  have q2: "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. (\<alpha>, maxI \<alpha> \<beta>) \<in> leI \<and>
      (\<beta>, maxI \<alpha> \<beta>) \<in> leI"
  proof (intro ballI)
    fix \<alpha> \<beta>
    assume c1: "\<alpha> \<in> I" and c2: "\<beta> \<in> I"
    moreover then have c3: "(\<alpha>, mxI \<alpha> \<beta>) \<in> leI \<and> (\<beta>, mxI \<alpha> \<beta>) \<
      in> leI \<and> mxI \<alpha> \<beta> \<in> I"
      using m0 m1 a5 by force+
    ultimately have "(mxI \<alpha> \<beta>, maxI \<alpha> \<beta>) \<in> leI \<and> maxI \<alpha> \<beta> \<in>
      I" using b0 p2 by blast
    then show "(\<alpha>, maxI \<alpha> \<beta>) \<in> leI \<and> (\<beta>, maxI \<alpha> \<beta>) \<in> leI"
      using c1 c2 c3 a4 by blast
  qed
  have q3: "\<forall> \<alpha>\<in>I. \<forall>\<beta>\<in>I. \<forall>\<gamma>\<in>I. (maxI \<alpha> \<beta>,
      \<gamma>) \<in> leI \<longrightarrow> (\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>)\<in>leI \<and>
       (\<gamma>,\<alpha>)\<notin>leI \<and> (\<gamma>,\<beta>)\<notin>leI"
  proof (intro ballI impI)
    fix \<alpha> \<beta> \<gamma>
```

```
  assume c1: "\<alpha>\<in>I" and c2: "\<beta>\<in>I" and c3: "\<gamma>\<in>I" and c4: "(maxI \<alpha> \<beta
    >, \<gamma>) \<in> leI"
  moreover then have c5: "(mxI \<alpha> \<beta>, maxI \<alpha> \<beta>) \<in> leI \<and> maxI \<alpha> \<beta>
     \<in> I
           \<and> (maxI \<alpha> \<beta>, mxI \<alpha> \<beta>) \<notin> leI \<and> mxI \<alpha> \<beta> \<in
    > I"  using b0 p2 m1 by blast
  ultimately have c6: "(mxI \<alpha> \<beta>, \<gamma>) \<in> leI" using a4 by blast
  have "(\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>)\<in>leI"
  proof (cases "(\<alpha>,\<beta>) \<in> leI")
    assume "(\<alpha>,\<beta>) \<in> leI"
    moreover then have "(\<beta>,\<gamma>) \<in> leI" using m0 c6 by simp
    ultimately show "(\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>)\<in>leI" using c1 c2 c3 a4 by blast
  next
    assume "(\<alpha>,\<beta>) \<notin> leI"
    then have "(\<beta>,\<alpha>) \<in> leI \<and> (\<alpha>,\<gamma>) \<in> leI" using m0 c1 c2 c6 a5 by
     force
    then show "(\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>)\<in>leI" using c1 c2 c3 a4 by blast
  qed
  moreover have "(\<gamma>,\<alpha>) \<in> leI \<longrightarrow> False"
  proof
    assume "(\<gamma>,\<alpha>) \<in> leI"
    moreover have "(\<alpha>, mxI \<alpha> \<beta>) \<in> leI \<and> mxI \<alpha> \<beta> \<in> I" using c1 c2
     m0 a5 by force
    ultimately have "(\<gamma>, mxI \<alpha> \<beta>) \<in> leI" using c1 c3 a4 by blast
    then show "False" using c3 c4 c5 a4 by blast
  qed
  moreover have "(\<gamma>,\<beta>) \<in> leI \<longrightarrow> False"
  proof
    assume "(\<gamma>,\<beta>) \<in> leI"
    moreover have "(\<beta>, mxI \<alpha> \<beta>) \<in> leI \<and> mxI \<alpha> \<beta> \<in> I" using c1 c2
     m0 a5 by force
    ultimately have "(\<gamma>, mxI \<alpha> \<beta>) \<in> leI" using c2 c3 a4 by blast
    then show "False" using c3 c4 c5 a4 by blast
  qed
  ultimately show "(\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>)\<in>leI \<and> (\<gamma>,\<alpha>)\<
    notin>leI \<and> (\<gamma>,\<beta>)\<notin>leI" by blast
qed
have "\<exists> d. d`I = I\<times>L\<times>I"
proof -
  have c1: "\<not> finite I" using a1 a2 by (metis card_of_ordLeq_infinite ordLess_imp_ordLeq)
  then have "I \<noteq> {} \<and> L \<noteq> {}" using a1 by blast
  moreover then have "|I| \<le>o |L\<times>I| \<and> |L\<times>I| =o |I| \<and> L \<noteq> {}"
    using c1 a1 a2 by (metis card_of_Times_infinite[of I L] ordLess_imp_ordLeq ordIso_iff_ordLeq)
  moreover then have "\<not> finite (L\<times>I)" using c1 a1 by (metis finite_cartesian_productD2)
  ultimately have "|I\<times>(L\<times>I)| \<le>o |I|"
    by (metis card_of_Times_infinite[of "L\<times>I" I] ordIso_transitive ordIso_iff_ordLeq)
  moreover have "I\<times>L\<times>I \<noteq> {}" using c1 a1 by force
  ultimately show ?thesis using card_of_ordLeq2[of "I\<times>(L\<times>I)" I] by blast
qed
then obtain d where b1: "d`I = I\<times>(L\<times>I)" by blast
obtain \<mu> where b2: "\<mu> = (\<lambda> \<gamma>. SOME m. m`L = ({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>
    leI}\<times>L)\<times>({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L) )" by blast
have b3: "\<And>\<gamma>. \<gamma> \<in> I \<Longrightarrow> (\<mu> \<gamma>)`L = ({\<alpha>\<in>I. (\<alpha
    >,\<gamma>)\<in>leI}\<times>L)\<times>({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L)"
proof -
  fix \<gamma>
  assume c1: "\<gamma> \<in> I"
  obtain A where c2: "A = {\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}" by blast
  have c3: "A \<noteq> {}" using c1 c2 a3 unfolding refl_on_def by blast
  moreover have "L \<noteq> {}" using a1 by blast
  ultimately have "(A\<times>L)\<times>(A\<times>L) \<noteq> {}" using a1 by simp
  moreover have "|(A\<times>L)\<times>(A\<times>L)| \<le>o |L|"
  proof -
    have "|A| \<le>o |L|" using c1 c2 a6 by blast
    then have "|A\<times>L| \<le>o |L|" using c3 a1 by (metis card_of_Times_infinite[of L A] ordIso_iff_ordLeq
    )
    moreover have "\<not> finite (A\<times>L)" using c3 a1 by (metis finite_cartesian_productD2)
    ultimately show ?thesis
      by (metis card_of_Times_same_infinite[of "A\<times>L"] ordIso_iff_ordLeq ordLeq_transitive)
  qed
  ultimately have "\<exists>m. m`L = ({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L)\<times>({\<alpha
    >\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L)"
    using c2 card_of_ordLeq2[of "(A\<times>L)\<times>(A\<times>L)" L] by blast
  then show "(\<mu> \<gamma>)`L = ({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L)\<times>({\<alpha>\<
    in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L)"
    using b2 someI_ex[of "\<lambda> m. m`L = ({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L)\<times
    >({\<alpha>\<in>I. (\<alpha>,\<gamma>)\<in>leI}\<times>L) "] by blast
qed
```

```
  obtain \<phi> where b4: "\<phi> = (\<lambda> x. \<mu> (fst (d x)) (fst (snd (d x))))" by blast
  obtain h where b5: "h = (\<lambda> x. jnN (t (fst (\<phi> x)) x) (t (snd (\<phi> x)) x))" by blast
  have "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. \<forall>i\<in>L. \<forall>j\<in>L. \<exists> \<gamma
      >\<in>I.
       (maxI \<alpha> \<beta>, \<gamma>) \<in> leI \<and> h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta
      >,j) \<gamma>)"
  proof (intro ballI)
    fix \<alpha> \<beta> i j
    assume c1: "\<alpha> \<in> I" and c2: "\<beta> \<in> I" and c3: "i \<in> L" and c4: "j \<in> L"
    obtain D where c5: "D = ({\<alpha>' \<in> I. (\<alpha>', maxI \<alpha> \<beta>) \<in> leI} \<times> L) \<
      times> {\<alpha>' \<in> I. (\<alpha>', maxI \<alpha> \<beta>) \<in> leI} \<times> L" by blast
    have c6: "maxI \<alpha> \<beta> \<in> I" using c1 c2 q1 by blast
    have "\<alpha> \<in> {\<alpha>' \<in> I. (\<alpha>', maxI \<alpha> \<beta>) \<in> leI}" using c1 c2 q2 by
      blast
    moreover have "\<beta> \<in> {\<alpha>' \<in> I. (\<alpha>', maxI \<alpha> \<beta>) \<in> leI}" using c1 c2
      q2 by blast
    ultimately have "((\<alpha>,i),(\<beta>,j)) \<in> D"  using c3 c4 c5 by blast
    moreover have "\<mu> (maxI \<alpha> \<beta>) ` L = D" using c5 c6 b3[of "maxI \<alpha> \<beta>"] by blast
    ultimately obtain v where c7: "v \<in> L \<and> (\<mu> (maxI \<alpha> \<beta>)) v = ((\<alpha>,i),(\<beta>,j
      ))" by force
    obtain A where c8: "A = {maxI \<alpha> \<beta>} \<times> ({v} \<times> I)" by blast
    then have "A \<subseteq> I \<times> L \<times> I" using c6 c7 by blast
    then have "\<forall>a\<in>A. \<exists> x\<in>I. d x = a" using b1 by (metis imageE set_rev_mp)
    moreover obtain X where c9: "X = { x\<in>I. d x \<in> A }" by blast
    ultimately have "A = d ` X" by force
    then have "|A| \<le>o |X|" by simp
    moreover have "|I| =o |A|"
    proof -
      obtain f where "f = (\<lambda> x::'i. (maxI \<alpha> \<beta>, v, x))" by blast
      then have "bij_betw f I A" using c8 unfolding bij_betw_def inj_on_def by force
      then show "|I| =o |A|" using card_of_ordIsoI[of f I A] by blast
    qed
    ultimately have c10: "|L| <o |X|" using a2 by (metis ordLess_ordIso_trans ordLess_ordLeq_trans)
    have "\<forall>y\<in>I. X \<subseteq> {x\<in>I. (x,y) \<in> leI} \<longrightarrow> False"
    proof (intro ballI impI)
      fix y
      assume "y \<in> I" and "X \<subseteq> {x\<in>I. (x,y) \<in> leI}"
      then have "y \<in> I \<and> X \<subseteq> {x\<in>I. (x,y) \<in> leI}" by blast
      moreover then have "|{x\<in>I. (x,y) \<in> leI}| \<le>o |L|" using a6 by blast
      ultimately have "|X| \<le>o |L|" using card_of_mono1 ordLeq_transitive by blast
      then show "False" using c10 by (metis not_ordLeq_ordLess)
    qed
    then obtain \<gamma> where c11: "\<gamma> \<in> X \<and> (\<gamma>, maxI \<alpha> \<beta>) \<notin> leI"
      using c6 c9 by blast
    then obtain w where c12: "\<gamma> \<in> I \<and>  d \<gamma> = (maxI \<alpha> \<beta>, v, w)" using c8 c9
      by blast
    moreover have "(maxI \<alpha> \<beta>, \<gamma>) \<in> leI" using c11 c12 c6 a5 by blast
    moreover have "h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)"
    proof -
      have "\<phi> \<gamma> = \<mu> (fst (d \<gamma>)) (fst (snd (d \<gamma>)))" using b4 by blast
      then have "\<phi> \<gamma> = \<mu> (maxI \<alpha> \<beta>) v" using c12 by simp
      then have "\<phi> \<gamma> = ((\<alpha>,i),(\<beta>,j))" using c7 by simp
      moreover have "h \<gamma> = jnN (t (fst (\<phi> \<gamma>)) \<gamma>) (t (snd (\<phi> \<gamma>)) \<gamma>)"
       using b5 by blast
      ultimately show "h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)" by simp
    qed
    ultimately show "\<exists> \<gamma>\<in>I. (maxI \<alpha> \<beta>, \<gamma>) \<in> leI \<and> h \<gamma> =
      jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)" by blast
  qed
  then show ?thesis using q3 by blast
qed

lemma lem_jnfix_card:
fixes \<kappa>::"'U rel" and L::"'l set" and t::"('U rel)\<times>'l \<Rightarrow> 'U rel \<Rightarrow> 'n" and
      jnN::"'n \<Rightarrow> 'n \<Rightarrow> 'n"
  and S::"'U rel set"
assumes a1: "Card_order \<kappa>" and a2: "\<not> finite L" and a3: "|L| <o \<kappa>"
    and a4: "\<forall> \<alpha> \<in> S. |Field \<alpha>| \<le>o |L|"
    and a5: "S \<subseteq> \<O>" and a6: "|{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |S|"
    and a7: "\<forall> \<alpha> \<in> S. \<exists> \<beta> \<in> S. \<alpha> <o \<beta>"
shows "\<exists> h. \<forall> \<alpha> \<in> S. \<forall> \<beta> \<in> S. \<forall>i\<in>L. \<forall>j\<in>L.
              (\<exists> \<gamma> \<in> S. \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma> \<and> h \<gamma> =
      jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>) )"
proof -
  obtain I::"('U rel) set" where c1: "I = S" by blast
  obtain leI::"'U rel rel" where c2: "leI = oord" by blast
  have "\<not> finite L" using a2 by blast
  moreover have "|L| <o |I|"
```

```
proof -
  have "\<omega>_ord \<le>o |L|" using a2 by (metis infinite_iff_natLeq_ordLeq)
  then have "\<omega>_ord \<le>o \<kappa>" using a3 by (metis ordLeq_ordLess_trans ordLess_imp_ordLeq)
  then obtain f::"'U rel \<Rightarrow> 'U" where
    d1: "Field \<kappa> \<subseteq> f ` {\<gamma>. \<gamma> <o \<kappa>}" and d2: "\<forall>\<gamma>1 \<gamma
    >2. \<gamma>1 =o \<gamma>2 \<longrightarrow> f \<gamma>1 = f \<gamma>2"
    using a1 lem_card_setcv_inf_stab[of \<kappa> "Field \<kappa>"] by (metis card_of_Field_ordIso
    ordIso_imp_ordLeq)
  then have "|Field \<kappa>| \<le>o |f ` {\<gamma>. \<gamma> <o \<kappa>}|" by simp
  then have "\<kappa> \<le>o |f ` {\<gamma>. \<gamma> <o \<kappa>}|" using a1
    by (metis card_of_Field_ordIso ordIso_imp_ordLeq ordLeq_transitive ordIso_symmetric)
  moreover have "|f ` {\<gamma>. \<gamma> <o \<kappa>}| \<le>o |{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o
    \<kappa>}|"
  proof -
    have "\<kappa> \<noteq> {}" using a2 a3
      using lem_cardord_emp by (metis Field_empty card_of_Field_ordIso card_of_empty not_ordLess_ordIso
    ordLeq_ordLess_trans)
    then have "({}::'U rel) <o \<kappa>" using a1
      by (metis ozero_def iso_ozero_empty card_order_on_well_order_on ordIso_symmetric
    ordLeq_iff_ordLess_or_ordIso ozero_ordLeq)
    then have e1: "f ` {\<gamma>. \<gamma> <o \<kappa>} \<noteq> {}" by blast
    moreover have "f ` {\<gamma>. \<gamma> <o \<kappa>} \<subseteq> f ` {\<alpha> \<in> \<O>. \<alpha> <o \<
    kappa>}"
    proof
      fix y
      assume "y \<in> f ` {\<gamma>. \<gamma> <o \<kappa>}"
      then obtain \<gamma> \<alpha> where f1: "\<gamma> <o \<kappa> \<and> y = f \<gamma> \<and> \<alpha> =
    nord \<gamma>" by blast
      moreover then have f2: "\<alpha> \<in> \<O> \<and> \<alpha> =o \<gamma>" using lem_nord_r unfolding \<O>
    _def ordLess_def by blast
      ultimately have "\<alpha> <o \<kappa>" using d2 ordIso_ordLess_trans by blast
      moreover have "y = f \<alpha>" using d2 f1 f2 by fastforce
      ultimately show "y \<in> f ` {\<alpha> \<in> \<O>. \<alpha> <o \<kappa>}" using f2 by blast
    qed
    ultimately have "f ` {\<alpha> \<in> \<O>. \<alpha> <o \<kappa>} = f ` {\<gamma>. \<gamma> <o \<kappa>}"
    by blast
    then show ?thesis using e1 card_of_ordLeq2[of "f ` {\<gamma>. \<gamma> <o \<kappa>}" "{\<alpha> \<in> \<O
    >::'U rel set. \<alpha> <o \<kappa>}"] by blast
  qed
  ultimately have "\<kappa> \<le>o |{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}|" using
    ordLeq_transitive by blast
  moreover have "I = S" using c1 by blast
  moreover then have "|{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |I|" using a6 by blast
  ultimately have "\<kappa> \<le>o |I|" using c1 using ordLeq_transitive by blast
  then show ?thesis using a3 by (metis ordLess_ordLeq_trans)
qed
moreover have "\<forall>\<alpha>\<in>I. (\<alpha>,\<alpha>) \<in> leI"
  using c1 c2 a5 lem_fld_oord lem_oord_wo unfolding well_order_on_def linear_order_on_def
    partial_order_on_def preorder_on_def refl_on_def by blast
moreover have "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. \<forall>\<gamma>\<in>I. (\<alpha>,\<beta>)\<
    in>leI \<and> (\<beta>,\<gamma>)\<in>leI \<longrightarrow> (\<alpha>,\<gamma>)\<in>leI"
  using c2 lem_oord_wo unfolding well_order_on_def linear_order_on_def
    partial_order_on_def preorder_on_def trans_def by blast
moreover have "\<forall>\<alpha>\<in>\<O>. \<forall>\<beta>\<in>\<O>. (\<alpha>,\<beta>) \<in> leI \<or> (\<
    beta>,\<alpha>) \<in> leI"
  using c1 c2 lem_fld_oord lem_oord_wo unfolding well_order_on_def linear_order_on_def total_on_def
    partial_order_on_def preorder_on_def refl_on_def by metis
moreover then have "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. (\<alpha>,\<beta>) \<in> leI \<or> (\<
    beta>,\<alpha>) \<in> leI" using c1 a5 by blast
moreover have "\<forall>\<beta>\<in>I. |{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o |L|"
proof
  fix \<beta>
  assume d1: "\<beta> \<in> I"
  show "|{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o |L|"
  proof (cases "\<omega>_ord \<le>o \<beta>")
    assume e1: "\<omega>_ord \<le>o \<beta>"
    obtain C where e2: "C = nord ` {\<alpha>::'U rel. \<alpha> <o \<beta>}" by blast
    have "{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI} \<subseteq> C \<union> {\<beta>}"
    proof
      fix \<gamma>
      assume "\<gamma> \<in> {\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}"
      then have "\<gamma> \<in> \<O> \<and> (\<gamma> <o \<beta> \<or> \<gamma> = \<beta>)"
        using c2 lem_0eq unfolding oord_def using ordLeq_iff_ordLess_or_ordIso by blast
      moreover then have "\<gamma> = nord \<gamma>" using lem_0nord by blast
      ultimately show "\<gamma> \<in> C \<union> {\<beta>}" using e2 by blast
    qed
    moreover have "|C \<union> {\<beta>}| \<le>o \<beta>"
    proof (cases "finite C")
```

```
      assume "finite C"
      then have "finite (C \<union> {\<beta>})" by blast
      then have "|C \<union> {\<beta>}| <o \<omega>_ord" using finite_iff_ordLess_natLeq by blast
      then show ?thesis using e1 ordLess_ordLeq_trans ordLess_imp_ordLeq by blast
    next
      assume "\<not> finite C"
      then have "|C \<union> {\<beta>}| =o |C|" by (metis card_of_singl_ordLeq finite.simps
    card_of_Un_infinite)
      then show ?thesis using e1 e2 lem_nord_eq lem_ord_int_card_le_inf[of nord \<beta>] ordIso_ordLeq_trans
    by blast
    qed
    ultimately have "|{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o \<beta>" by (meson card_of_mono1
    ordLeq_transitive)
    moreover have "\<And> A::'U rel set. |A| \<le>o \<beta> \<Longrightarrow> |A| \<le>o |Field \<beta>|"
      by (metis Field_card_of card_of_mono1 internalize_card_of_ordLeq)
    ultimately have "|{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o |Field \<beta>|" by blast
    moreover have "|Field \<beta>| \<le>o |L|" using d1 c1 a4 by blast
    ultimately show "|{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o |L|" using ordLeq_transitive by
    blast
  next
    assume "\<not> \<omega>_ord \<le>o \<beta>"
    then have e1: "\<beta> <o \<omega>_ord" using d1 c1 a5 using lem_Owo Field_natLeq natLeq_well_order_on by
    force
    then have e2: "\<beta> =o natLeq_on (card (Field \<beta>))" using lem_wolew_nat by blast
    obtain A where e3: "A = { n. n \<le> card (Field \<beta>) }" by blast
    obtain f where e4: "f = (\<lambda>n::nat. SOME \<alpha>. \<alpha> \<in> I \<and> \<alpha> <o \<omega>_ord
    \<and> card (Field \<alpha>) = n)" by blast
    have "{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI} \<subseteq> f ` A"
    proof
      fix \<gamma>
      assume f1: "\<gamma> \<in> {\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}"
      then have f2: "\<gamma> \<le>o \<beta>" using c2 oord_def by blast
      then have f3: "\<gamma> <o \<omega>_ord" using e1 ordLeq_ordLess_trans by blast
      then have f4: "\<gamma> =o natLeq_on (card (Field \<gamma>))" using lem_wolew_nat by blast
      then have "natLeq_on (card (Field \<gamma>)) \<le>o natLeq_on (card (Field \<beta>))"
        using f2 e2 by (meson ordIso_iff_ordLeq ordLeq_transitive)
      then have f5: "\<gamma> \<in> I \<and> card (Field \<gamma>) \<in> A" using f1 e3
    natLeq_on_ordLeq_less_eq by blast
      moreover obtain \<gamma>' where f6: "\<gamma>' = f (card (Field \<gamma>))" by blast
      ultimately have "\<gamma>' \<in> I \<and> \<gamma>' <o \<omega>_ord \<and> card (Field \<gamma>') = card
     (Field \<gamma>)"
        using f3 e4 someI_ex[of "\<lambda> \<alpha>. \<alpha> \<in> I \<and> \<alpha> <o \<omega>_ord \<and>
    card (Field \<alpha>) = card (Field \<gamma>)"] by blast
      moreover then have "\<gamma>' =o natLeq_on (card (Field \<gamma>))" using lem_wolew_nat by force
      ultimately have "\<gamma> \<in> \<O> \<and> \<gamma>' \<in> \<O> \<and> \<gamma>' =o \<gamma>" using f1
    f4 c1 a5 ordIso_symmetric ordIso_transitive by blast
      then have "\<gamma>' = \<gamma>" using lem_Oeq by blast
      moreover have "\<gamma>' \<in> f ` A" using f5 f6 by blast
      ultimately show "\<gamma> \<in> f ` A" by blast
    qed
    then have "finite {\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}" using e3 finite_subset by blast
    then show "|{\<alpha>\<in>I. (\<alpha>,\<beta>) \<in> leI}| \<le>o |L|" using a2 ordLess_imp_ordLeq by
    force
  qed
qed
moreover have "\<forall>\<alpha>\<in>I. \<exists>\<alpha>'\<in>I. (\<alpha>,\<alpha>') \<in> leI \<and> (\<
    alpha>',\<alpha>) \<notin> leI"
proof
  fix \<alpha>
  assume "\<alpha> \<in> I"
  then obtain \<alpha>' where d1: "\<alpha> \<in> S \<and> \<alpha>' \<in> S \<and> \<alpha> <o \<alpha>'"
    using c1 a7 by blast
  then have d2: "\<alpha> \<le>o \<alpha>' \<and> \<alpha> \<in> \<O> \<and> \<alpha>' \<in> \<O>" using a5
    ordLess_imp_ordLeq by blast
  then have "\<alpha>' \<in> I \<and> (\<alpha>,\<alpha>') \<in> leI" using d1 c1 c2 unfolding oord_def by
    blast
  moreover have "(\<alpha>',\<alpha>) \<in> leI \<longrightarrow> False"
  proof
    assume e1: "(\<alpha>',\<alpha>) \<in> leI"
    then have "\<alpha>' \<le>o \<alpha>" using c2 unfolding oord_def by blast
    then have "\<alpha>' = \<alpha>" using d2 lem_Oeq ordIso_iff_ordLeq by blast
    then show "False" using d1 ordLess_irreflexive by blast
  qed
  ultimately show "\<exists>\<alpha>'\<in>I. (\<alpha>,\<alpha>') \<in> leI \<and> (\<alpha>',\<alpha>) \<
    notin> leI" by blast
qed
ultimately obtain h where
```

```
    c3: "\<forall>\<alpha>\<in>I. \<forall>\<beta>\<in>I. \<forall>i\<in>L. \<forall>j\<in>L. \<exists> \<gamma
      >\<in>I.
        (\<alpha>,\<gamma>)\<in>leI \<and> (\<beta>,\<gamma>) \<in> leI \<and> (\<gamma>,\<alpha>)\<notin>leI \<
      and> (\<gamma>,\<beta>)\<notin>leI \<and> h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<
      gamma>)"
    using lem_jnfix_gen[of L I leI jnN t] by blast
  have "\<forall> \<alpha> \<in> S. \<forall> \<beta> \<in> S. \<forall>i\<in>L. \<forall>j\<in>L.
           (\<exists> \<gamma> \<in> S. \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma> \<and> h \<gamma> = jnN
       (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>))"
  proof (intro allI ballI impI)
    fix \<alpha>::"'U rel" and i::'l and \<beta>::"'U rel" and j::"'l"
    assume d2: "i \<in> L" and d3: "j \<in> L" and "\<alpha> \<in> S" and "\<beta> \<in> S"
    then have d4: "\<alpha> \<in> I \<and> \<beta> \<in> I" using c1 a5 by blast
    then obtain \<gamma> where "\<gamma> \<in> I" and "(\<alpha>,\<gamma>) \<in> leI \<and> (\<beta>,\<gamma>)
      \<in> leI" and "(\<gamma>,\<alpha>)\<notin>leI \<and> (\<gamma>,\<beta>)\<notin>leI"
      and d6: "h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)" using d2 d3 c3 by blast
    then have "\<gamma> \<in> \<O> \<inter> S \<and> \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma>"
      using d4 c1 c2 a5 lem_Oeq unfolding oord_def
        by (smt ordLeq_iff_ordLess_or_ordIso subsetCE Int_iff)
    moreover have "h \<gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)" using d2 d3 d6 by blast
    ultimately show "\<exists> \<gamma> \<in> S. \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma> \<and> h \<
      gamma> = jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>)" by blast
  qed
  then show ?thesis by blast
qed

lemma lem_cardsuc_ls_fldcard:
fixes \<kappa>::"'a rel" and \<alpha>::"'b rel"
assumes a1: "Card_order \<kappa>" and a2: "\<alpha> <o cardSuc \<kappa>"
shows "|Field \<alpha>| \<le>o \<kappa>"
proof -
  have "\<kappa> <o |Field \<alpha>| \<longrightarrow> False"
  proof
    assume "\<kappa> <o |Field \<alpha>|"
    moreover have "Card_order |Field \<alpha>|" by simp
    ultimately have "cardSuc \<kappa> \<le>o |Field \<alpha>|" using a1 cardSuc_least by blast
    moreover have "|Field \<alpha>| \<le>o \<alpha>" using a2 by simp
    ultimately have "cardSuc \<kappa> \<le>o \<alpha>" using ordLeq_transitive by blast
    then show "False" using a2 not_ordLeq_ordLess by blast
  qed
  then show "|Field \<alpha>| \<le>o \<kappa>" using a1 by simp
qed

lemma lem_jnfix_cardsuc:
fixes L::"'l set" and \<kappa>::"'U rel" and t::"('U rel)\<times>'l \<Rightarrow> 'U rel \<Rightarrow> 'n" and
      jnN::"'n \<Rightarrow> 'n \<Rightarrow> 'n"
  and S::"'U rel set"
assumes a1: "\<not> finite L" and a2: "\<kappa> =o cardSuc |L|"
    and a3: "S \<subseteq> {\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}" and a4: "|{\<alpha> \<in> \<
      O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |S|"
    and a5: "\<forall> \<alpha> \<in> S. \<exists> \<beta> \<in> S. \<alpha> <o \<beta>"
shows "\<exists> h. \<forall> \<alpha> \<in> S. \<forall> \<beta> \<in> S. \<forall>i\<in>L. \<forall>j\<in>L.
             (\<exists> \<gamma> \<in> S. \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma> \<and> h \<gamma> =
      jnN (t (\<alpha>,i) \<gamma>) (t (\<beta>,j) \<gamma>) )"
proof -
  have "Card_order \<kappa>" using a2 by (metis Card_order_ordIso cardSuc_Card_order card_of_Card_order)
  moreover have "|L| <o \<kappa>" using a2 cardSuc_greater[of "|L|"]
    by (metis Field_card_of card_of_card_order_on ordIso_iff_ordLeq ordLess_ordLeq_trans)
  moreover have "\<forall>\<alpha>::'U rel. \<alpha> <o \<kappa> \<longrightarrow> |Field \<alpha>| \<le>o |L|"
    using a2 using lem_cardsuc_ls_fldcard ordLess_ordIso_trans by force
  ultimately show ?thesis using a1 a3 a4 a5 lem_jnfix_card[of \<kappa> L S jnN t] by blast
qed

lemma lem_Relprop_cl_ccr:
fixes r::"'U rel"
shows "Conelike r \<Longrightarrow> CCR r"
  unfolding CCR_def Conelike_def by fastforce

lemma lem_Relprop_ccr_confl:
fixes r::"'U rel"
shows "CCR r \<Longrightarrow> confl_rel r"
  using lem_rtr_field[of _ _ r] unfolding CCR_def confl_rel_def by blast

lemma lem_Relprop_fin_ccr:
fixes r::"'U rel"
shows "finite r \<Longrightarrow> CCR r = Conelike r"
proof -
  assume a1: "finite r"
```

```
  have "r \<noteq> {} \<and> CCR r \<longrightarrow> Conelike r"
  proof
    assume b1: "r \<noteq> {} \<and> CCR r"
    have b2: "finite (Field r)" using a1 finite_Field by fastforce
    have "\<exists> xm \<in> Field r. \<forall> x \<in> Field r. (x, xm) \<in> r^*"
    proof -
      have "{} \<subseteq> Field r \<longrightarrow> (\<exists> xm \<in> Field r. \<forall> x \<in> {}. (x, xm)
       \<in> r^*)" using b1 Field_def by fastforce
      moreover have "\<And> x F. finite F \<Longrightarrow> x \<notin> F \<Longrightarrow>
        F \<subseteq> Field r \<longrightarrow> (\<exists> xm \<in> Field r. \<forall> x \<in> F. (x, xm) \<in>
      r^*) \<Longrightarrow>
        insert x F \<subseteq> Field r \<longrightarrow> (\<exists> xm \<in> Field r. \<forall> x \<in> insert x
       F. (x, xm) \<in> r^*)"
      proof
        fix x F
        assume c1: "finite F" and c2: "x \<notin> F" and c3: "F \<subseteq> Field r \<longrightarrow> (\<exists>
      xm\<in>Field r. \<forall>x\<in>F. (x, xm) \<in> r^*)"
           and c4: "insert x F \<subseteq> Field r"
        then obtain xm where c5: "xm \<in> Field r \<and> (\<forall>y\<in>F. (y, xm) \<in> r^*)" by blast
        then obtain xm' where "xm' \<in> Field r \<and> (x, xm') \<in> r^* \<and> (xm, xm') \<in> r^*"
          using b1 c4 unfolding CCR_def by blast
        moreover then have "\<forall>y\<in>insert x F. (y, xm') \<in> r^*" using c5 by force
        ultimately show "\<exists>xm\<in>Field r. \<forall>x\<in>insert x F. (x, xm) \<in> r^*" by blast
      qed
      ultimately have "(\<exists> xm \<in> Field r. \<forall> x \<in> Field r. (x, xm) \<in> r^*)"
        using b2 finite_induct[of "Field r" "\<lambda> A'. A' \<subseteq> Field r \<longrightarrow> (\<exists>
      xm \<in> Field r. \<forall> x \<in> A'. (x, xm) \<in> r^*)"] by simp
      then show "\<exists> xm \<in> Field r. \<forall> x \<in> Field r. (x, xm) \<in> r^*" by blast
    qed
    then show "Conelike r" using a1 b1 unfolding Conelike_def by blast
  qed
  then show "CCR r = Conelike r" using lem_Relprop_cl_ccr unfolding Conelike_def by blast
qed

lemma lem_Relprop_ccr_ch_un:
fixes S::"'U rel set"
assumes a1: "\<forall>s\<in>S. CCR s" and a2: "\<forall>s1\<in>S. \<forall>s2\<in>S. s1 \<subseteq> s2 \<or> s2
      \<subseteq> s1"
shows "CCR (\<Union> S)"
proof -
  have "\<forall>a\<in>Field (\<Union>S). \<forall>b\<in>Field (\<Union>S). \<exists>c\<in>Field (\<Union>S). (a
      , c) \<in> (\<Union>S)^* \<and> (b, c) \<in> (\<Union>S)^*"
  proof (intro ballI)
    fix a b
    assume c1: "a \<in> Field (\<Union>S)" and c2: "b \<in> Field (\<Union>S)"
    then obtain s1 s2 where c3: "s1 \<in> S \<and> a \<in> Field s1" and c4: "s2 \<in> S \<and> b \<in> Field s2
      "
      unfolding Field_def by blast
    show "\<exists>c\<in>Field (\<Union>S). (a,c) \<in> (\<Union>S)^* \<and> (b,c) \<in> (\<Union>S)^*"
    proof (cases "s1 \<subseteq> s2")
      assume "s1 \<subseteq> s2"
      then have "a \<in> Field s2" using c3 unfolding Field_def by blast
      then obtain c where "c \<in> Field s2 \<and> (a,c) \<in> s2^* \<and> (b,c) \<in> s2^*"
        using a1 c4 unfolding CCR_def by force
      moreover then have "c \<in> Field (\<Union>S)" using c4 unfolding Field_def by blast
      moreover have "s2^* \<subseteq> (\<Union>S)^*" using c4 Transitive_Closure.rtrancl_mono[of s2 "\<Union>S"]
       by blast
      ultimately show "\<exists>c\<in>Field (\<Union>S). (a,c) \<in> (\<Union>S)^* \<and> (b,c) \<in> (\<Union>S
      )^*" by blast
    next
      assume "\<not> s1 \<subseteq> s2"
      then have "s2 \<subseteq> s1" using a2 c3 c4 by blast
      then have "b \<in> Field s1" using c4 unfolding Field_def by blast
      then obtain c where "c \<in> Field s1 \<and> (a,c) \<in> s1^* \<and> (b,c) \<in> s1^*"
        using a1 c3 unfolding CCR_def by force
      moreover then have "c \<in> Field (\<Union>S)" using c3 unfolding Field_def by blast
      moreover have "s1^* \<subseteq> (\<Union>S)^*" using c3 Transitive_Closure.rtrancl_mono[of s1 "\<Union>S"]
       by blast
      ultimately show "\<exists>c\<in>Field (\<Union>S). (a,c) \<in> (\<Union>S)^* \<and> (b,c) \<in> (\<Union>S
      )^*" by blast
    qed
  qed
  then show ?thesis unfolding CCR_def by blast
qed

lemma lem_Relprop_restr_ch_un:
fixes C::"'U set set" and r::"'U rel"
assumes "\<forall>A1\<in>C. \<forall>A2\<in>C. A1 \<subseteq> A2 \<or> A2 \<subseteq> A1"
```

```
shows "Restr r (\<Union> C) = \<Union> { s. \<exists> A \<in> C. s = Restr r A }"
proof
  show "Restr r (\<Union> C) \<subseteq> \<Union> { s. \<exists> A \<in> C. s = Restr r A }"
  proof
    fix p
    assume "p \<in> Restr r (\<Union> C)"
    then obtain a b A1 A2 where "p = (a,b) \<and> a \<in> A1 \<and> b \<in> A2 \<and> p \<in> r \<and> A1 \<in>
      C \<and> A2 \<in> C" by blast
    moreover then have "A1 \<subseteq> A2 \<or> A2 \<subseteq> A1" using assms by blast
    ultimately show "p \<in> \<Union> { s. \<exists> A \<in> C. s = Restr r A }" by blast
  qed
next
  show "\<Union> { s. \<exists> A \<in> C. s = Restr r A } \<subseteq> Restr r (\<Union> C)" by blast
qed

lemma lem_Inv_restr_rtr:
fixes r::"'U rel" and A::"'U set"
assumes "A \<in> Inv r"
shows "r^* \<inter> (A\<times>(UNIV::'U set)) \<subseteq> (Restr r A)^*"
proof -
  have "\<forall> n. \<forall> a b. (a,b) \<in> r^^n \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)
      ^*"
  proof
    fix n
    show "\<forall> a b. (a,b) \<in> r^^n \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^*"
    proof (induct n)
      show "\<forall>a b. (a,b) \<in> r ^^ 0 \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^*" by
       simp
    next
      fix n
      assume d1: "\<forall>a b. (a,b) \<in> r ^^ n \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^*"
      show "\<forall>a b. (a,b) \<in> r ^^ (Suc n) \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^*"
      proof (intro allI impI)
        fix a b
        assume e1: "(a,b) \<in> r ^^ (Suc n) \<and> a \<in> A"
        moreover then obtain c where e2: "(a,c) \<in> r^^n \<and> (c,b) \<in> r" by force
        ultimately have e3: "(a,c) \<in> (Restr r A)^*" using d1 by blast
        moreover then have "c \<in> A" using e1 using rtranclE by force
        then have "(c,b) \<in> Restr r A" using assms e2 unfolding Inv_def by blast
        then show "(a,b) \<in> (Restr r A)^*" using e3 by (meson rtrancl.rtrancl_into_rtrancl)
      qed
    qed
  qed
  then show ?thesis using rtrancl_power by blast
qed

lemma lem_Inv_restr_rtr2:
fixes r::"'U rel" and A::"'U set"
assumes "A \<in> Inv r"
shows "r^* \<inter> (A\<times>(UNIV::'U set)) \<subseteq> (Restr r A)^* \<inter> ((UNIV::'U set)\<times>A)"
proof -
  have "\<forall> n. \<forall> a b. (a,b) \<in> r^^n \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)
      ^* \<inter> ((UNIV::'U set)\<times>A)"
  proof
    fix n
    show "\<forall> a b. (a,b) \<in> r^^n \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^* \<inter>
      ((UNIV::'U set)\<times>A)"
    proof (induct n)
      show "\<forall>a b. (a,b) \<in> r ^^ 0 \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^* \<
      inter> ((UNIV::'U set)\<times>A)" by simp
    next
      fix n
      assume d1: "\<forall>a b. (a,b) \<in> r ^^ n \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^*
      \<inter> ((UNIV::'U set)\<times>A)"
      show "\<forall>a b. (a,b) \<in> r ^^ (Suc n) \<and> a \<in> A \<longrightarrow> (a,b) \<in> (Restr r A)^*
      \<inter> ((UNIV::'U set)\<times>A)"
      proof (intro allI impI)
        fix a b
        assume e1: "(a,b) \<in> r ^^ (Suc n) \<and> a \<in> A"
        moreover then obtain c where e2: "(a,c) \<in> r^^n \<and> (c,b) \<in> r" by force
        ultimately have e3: "(a,c) \<in> (Restr r A)^*" using d1 by blast
        moreover then have "c \<in> A" using e1 using rtranclE by force
        then have e4: "(c,b) \<in> Restr r A" using assms e2 unfolding Inv_def by blast
        ultimately have "(a,b) \<in> (Restr r A)^*" using e3 by (meson rtrancl.rtrancl_into_rtrancl)
        then show "(a,b) \<in> (Restr r A)^* \<inter> ((UNIV::'U set)\<times>A)" using e4 by blast
      qed
    qed
  qed
```

```
  then show ?thesis using rtrancl_power by blast
qed

lemma lem_inv_rtr_mem:
fixes r::"'U rel" and A::"'U set" and a b::"'U"
assumes "A \<in> Inv r" and "a \<in> A" and "(a,b) \<in> r^*"
shows "b \<in> A"
  using assms lem_Inv_restr_rtr[of A r] rtranclE[of a b] by blast

lemma lem_Inv_ccr_restr:
fixes r::"'U rel" and A::"'U set"
assumes "CCR r" and "A \<in> Inv r"
shows "CCR (Restr r A)"
proof -
  have "\<forall>a \<in> Field (Restr r A). \<forall>b \<in> Field (Restr r A). \<exists>c \<in> Field (Restr r
      A). (a,c) \<in> (Restr r A)^* \<and> (b,c) \<in> (Restr r A)^*"
  proof (intro ballI)
    fix a b
    assume c1: "a \<in> Field (Restr r A)" and c2: "b \<in> Field (Restr r A)"
    moreover then obtain c where "c \<in> Field r" and "(a,c) \<in> r^* \<and> (b,c) \<in> r^*" using assms
      unfolding CCR_def Field_def by blast
    ultimately have "(a,c) \<in> r^* \<inter> (A\<times>(UNIV::'U set)) \<and> (b,c) \<in> r^* \<inter> (A\<
      times>(UNIV::'U set))" unfolding Field_def by blast
    then have "(a,c) \<in> (Restr r A)^* \<and> (b,c) \<in> (Restr r A)^*" using assms lem_Inv_restr_rtr by
      blast
    moreover then have "c \<in> Field (Restr r A)" using c1 lem_rtr_field[of a c] by blast
    ultimately show "\<exists>c \<in> Field (Restr r A). (a,c) \<in> (Restr r A)^* \<and> (b,c) \<in> (Restr r A
      )^*" by blast
  qed
  then show ?thesis unfolding CCR_def by blast
qed

lemma lem_Inv_cl_restr:
fixes r::"'U rel" and A::"'U set"
assumes "Conelike r" and "A \<in> Inv r"
shows "Conelike (Restr r A)"
proof(cases "r = {}")
  assume "r = {}"
  then show ?thesis unfolding Conelike_def by blast
next
  assume "r \<noteq> {}"
  then obtain m where b1: "\<forall> a \<in> Field r. (a,m) \<in> r^*" using assms unfolding Conelike_def by
      blast
  show "Conelike (Restr r A)"
  proof (cases "m \<in> Field (Restr r A)")
    assume "m \<in> Field (Restr r A)"
    moreover have "\<forall>a \<in> Field (Restr r A). (a,m) \<in> (Restr r A)^*"
      using assms lem_Inv_restr_rtr b1 unfolding Field_def by blast
    ultimately show "Conelike (Restr r A)" unfolding Conelike_def by blast
  next
    assume c1: "m \<notin> Field (Restr r A)"
    have "(Field r) \<inter> A \<subseteq> {m}"
    proof
      fix a0
      assume "a0 \<in> (Field r) \<inter> A"
      then have "(a0,m) \<in> r^* \<inter> (A\<times>(UNIV::'U set))" using b1 by blast
      then have "(a0,m) \<in> (Restr r A)^*" using assms lem_Inv_restr_rtr by blast
      then show "a0 \<in> {m}" using c1 lem_rtr_field by fast
    qed
    then show "Conelike (Restr r A)" unfolding Conelike_def Field_def by blast
  qed
qed

lemma lem_Inv_ccr_restr_invdiff:
fixes r::"'U rel" and A B::"'U set"
assumes a1: "CCR (Restr r A)" and a2: "B \<in> Inv (r^-1)"
shows "CCR (Restr r (A - B))"
proof -
  have "(Restr r A) `` (A-B) \<subseteq> (A-B)"
  proof
    fix b
    assume "b \<in> (Restr r A) `` (A-B)"
    then obtain a where c2: "a \<in> A-B \<and> (a,b) \<in> (Restr r A)" by blast
    moreover then have "b \<notin> B" using a2 unfolding Inv_def by blast
    ultimately show "b \<in> A - B" by blast
  qed
  then have "(A-B) \<in> Inv(Restr r A)" unfolding Inv_def by blast
  then have "CCR (Restr (Restr r A) (A - B))" using a1 lem_Inv_ccr_restr by blast
```

```
  moreover have "Restr (Restr r A) (A - B) = Restr r (A-B)" by blast
  ultimately show ?thesis by metis
qed

lemma lem_Inv_dncl_invbk: "dncl r A \<in> Inv (r^-1)"
  unfolding dncl_def Inv_def using converse_rtrancl_into_rtrancl by fastforce

lemma lem_inv_sf_ext:
fixes r::"'U rel" and A::"'U set"
assumes "A \<subseteq> Field r"
shows "\<exists> A' \<in> SF r. A \<subseteq> A' \<and> (finite A \<longrightarrow> finite A') \<and> ((\<not>
      finite A) \<longrightarrow> |A'| =o |A| )"
proof -
  obtain rs where b4: "rs = r \<union> (r^-1)" by blast
  obtain S where b1: "S = (\<lambda> a. rs``{a} )" by blast
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. (f a) \<in> (S' a)" by (metis someI_ex)
  then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
      longrightarrow> f a = a)"
    unfolding b2 by (clarsimp, metis singletonD)
  obtain A' where b5: "A' = A \<union> (f ` A)" by blast
  have "A \<union> (f ` A) \<subseteq> Field (Restr r A')"
  proof
    fix x
    assume "x \<in> A \<union> (f ` A)"
    then obtain a b where c1: "a \<in> A \<and> b = f a \<and> x \<in> {a,b}" by blast
    moreover then have "rs `` {a} \<noteq> {} \<longrightarrow> (a, b) \<in> rs" using assms b1 b3 by blast
    moreover have "rs `` {a} = {} \<longrightarrow> False" using assms c1 b4 unfolding Field_def by blast
    moreover have "(a,b) \<in> rs \<longrightarrow> {a,b} \<subseteq> Field (Restr r A')" using c1 b4 b5
      unfolding Field_def by blast
    ultimately show "x \<in> Field (Restr r A')" by blast
  qed
  then have "(A \<subseteq> A') \<and> (A' \<in> SF r)" using b5 unfolding SF_def Field_def by blast
  moreover have "finite A \<longrightarrow> finite A'" using b5 by blast
  moreover have "(\<not> finite A) \<longrightarrow> |A'| =o |A|" using b5 by simp
  ultimately show ?thesis by blast
qed

lemma lem_inv_sf_un:
assumes "S \<subseteq> SF r"
shows "(\<Union> S) \<in> SF r"
  using assms unfolding SF_def Field_def by blast

lemma lem_Inv_ccr_sf_inv_diff:
fixes r::"'U rel" and A B::"'U set"
assumes a1: "A \<in> SF r" and a2: "CCR (Restr r A)" and a3: "B \<in> Inv (r^-1)"
shows "(A-B) \<in> SF r \<or> (\<exists> y::'U. (A-B) = {y})"
proof -
  have "\<forall> a \<in> A - B. a \<notin> Field (Restr r (A-B)) \<longrightarrow> A - B = {a}"
  proof (intro ballI impI)
    fix a
    assume b1: "a \<in> A - B" and b2: "a \<notin> Field (Restr r (A-B))"
    then have "\<not> (\<exists> b \<in> A-B. (a,b) \<in> r \<or> (b,a) \<in> r)" unfolding Field_def by blast
    then have b3: "\<forall> b \<in> A. (a,b) \<notin> r" using a3 b1 unfolding Inv_def by blast
    have b4: "\<forall> x \<in> Field(Restr r A). (x,a) \<in> (Restr r A)^*"
    proof
      fix x
      assume "x \<in> Field(Restr r A)"
      moreover then have "a \<in> Field (Restr r A)" using b1 a1 unfolding SF_def by blast
      ultimately obtain y where c1: "(a,y) \<in> (Restr r A)^* \<and> (x,y) \<in> (Restr r A)^*"
        using a2 unfolding CCR_def by blast
      moreover have "(a,y) \<in> (Restr r A)^+ \<longrightarrow> False" using b3 tranclD by force
      ultimately have "a = y" using rtrancl_eq_or_trancl by metis
      then show "(x,a) \<in> (Restr r A)^*" using c1 by blast
    qed
    have "\<forall> b \<in> (A-B) - {a}. False"
    proof
      fix b
      assume c1: "b \<in> (A-B) - {a}"
      then have "b \<in> Field (Restr r A)" using a1 unfolding SF_def by blast
      then have "(b,a) \<in> (Restr r A)^*" using b4 by blast
      moreover have "(b,a) \<in> (Restr r A)^+ \<longrightarrow> False"
      proof
        assume "(b,a) \<in> (Restr r A)^+"
        then obtain b' where d1: "(b,b') \<in> (Restr r A)^* \<and> (b',a) \<in> Restr r A" using tranclD2 by
      metis
```

```
      have d2: "\<forall> r' a b. (a,b) \<in> Restr r' B = (a \<in> B \<and> b \<in> B \<and> (a,b) \<in> r')"
        unfolding Field_def by force
      have "(b,b') \<in> r^*" using d1 rtrancl_mono[of "Restr r A"] by blast
      then have "(b',b) \<in> (r^-1)^*" using rtrancl_converse by blast
      then have "b' \<in> B \<longrightarrow> (b',b) \<in> (Restr (r^-1) B)^*" using a3 lem_Inv_restr_rtr by
    blast
      then have "b' \<in> B \<longrightarrow> b \<in> B" using d2 by (metis rtrancl_eq_or_trancl tranclD2)
      then have "b' \<in> A - B" using d1 c1 by blast
      then have "(b',a) \<in> Restr r (A-B)" using b1 d1 by blast
      then have "a \<in> Field (Restr r (A-B))" unfolding Field_def by blast
      then show "False" using b2 by blast
    qed
    ultimately have "b = a" using rtrancl_eq_or_trancl[of b a] by blast
    then show "False" using c1 by blast
  qed
  then show "A - B = {a}" using b1 by blast
 qed
 then show ?thesis unfolding SF_def Field_def by blast
qed

lemma lem_Inv_ccr_sf_dn_diff:
fixes r::"'U rel" and A D A'::"'U set"
assumes a1: "A \<in> SF r" and a2: "CCR (Restr r A)" and a3: "A' = (A - (dncl r D))"
shows "((A' \<in> SF r) \<and> CCR (Restr r A')) \<or> (\<exists> y::'U. A' = {y})"
 using assms lem_Inv_ccr_restr_invdiff lem_Inv_ccr_sf_inv_diff lem_Inv_dncl_invbk by blast

lemma lem_rseq_tr:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U"
assumes "\<forall> i. (xi i, xi (Suc i)) \<in> r"
shows "\<forall> i j. i < j \<longrightarrow> (xi i \<in> Field r \<and> (xi i, xi j) \<in> r^+)"
proof -
 have "\<And> j. \<forall> i < j. xi i \<in> Field r \<and> (xi i, xi j) \<in> r^+"
 proof -
   fix j0
   show "\<forall> i < j0. xi i \<in> Field r \<and> (xi i, xi j0) \<in> r^+"
   proof (induct j0)
     show "\<forall>i<0. xi i \<in> Field r \<and> (xi i, xi 0) \<in> r^+" by blast
   next
     fix j
     assume d1: "\<forall>i<j. xi i \<in> Field r \<and> (xi i, xi j) \<in> r^+"
     show "\<forall>i<Suc j. xi i \<in> Field r \<and> (xi i, xi (Suc j)) \<in> r^+"
     proof (intro allI impI)
       fix i
       assume e1: "i < Suc j"
       have e2: "(xi j, xi (Suc j)) \<in> r" using assms by simp
       show "xi i \<in> Field r \<and> (xi i, xi (Suc j)) \<in> r^+"
       proof (cases "i < j")
         assume "i < j"
         then have "xi i \<in> Field r \<and> (xi i, xi j) \<in> r^+" using d1 by blast
         then show ?thesis using e2 by force
       next
         assume "\<not> i < j"
         then have "i = j" using e1 by simp
         then show ?thesis using e2 unfolding Field_def by blast
       qed
     qed
   qed
 qed
 then show ?thesis by blast
qed

lemma lem_rseq_rtr:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U"
assumes "\<forall> i. (xi i, xi (Suc i)) \<in> r"
shows "\<forall> i j. i \<le> j \<longrightarrow> (xi i \<in> Field r \<and> (xi i, xi j) \<in> r^*)"
proof (intro allI impI)
 fix i::nat and j::nat
 assume b1: "i \<le> j"
 then have "xi i \<in> Field r" using assms unfolding Field_def by blast
 moreover have "(xi i, xi j) \<in> r^*"
 proof (cases "i = j")
   assume "i = j"
   then show ?thesis by blast
 next
   assume "i \<noteq> j"
   then have "i < j" using b1 by simp
   moreover have "r^+ \<subseteq> r^*" by force
   ultimately show ?thesis using assms lem_rseq_tr[of xi r] by blast
```

```
  qed
  ultimately show "xi i \<in> Field r \<and> (xi i, xi j) \<in> r^*" by blast
qed

lemma lem_rseq_svacyc_inv_tr:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U" and a::"'U"
assumes a1: "single_valued r" and a2: "\<forall> i. (xi i, xi (Suc i)) \<in> r"
shows "\<And> i. (xi i, a) \<in> r^+ \<Longrightarrow> (\<exists> j. i<j \<and> a = xi j)"
proof -
  fix i
  assume "(xi i, a) \<in> r^+"
  moreover have "\<And> n. \<forall> i a. (xi i, a) \<in> r^^(Suc n) \<longrightarrow> (\<exists> j. i<j \<and>
      a = xi j)"
  proof -
    fix n
    show "\<forall> i a. (xi i, a) \<in> r^^(Suc n) \<longrightarrow> (\<exists> j. i<j \<and> a = xi j)"
    proof (induct n)
      show "\<forall>i a. (xi i, a) \<in> r^^(Suc 0) \<longrightarrow> (\<exists>j>i. a = xi j)"
      proof (intro allI impI)
        fix i a
        assume "(xi i, a) \<in> r^^(Suc 0)"
        then have "(xi i, a) \<in> r \<and> (xi i, xi (Suc i)) \<in> r" using a2 by simp
        then have "a = xi (Suc i)" using a1 unfolding single_valued_def by blast
        then show "\<exists>j>i. a = xi j" by force
      qed
    next
      fix n
      assume d1: "\<forall>i a. (xi i, a) \<in> r^^(Suc n) \<longrightarrow> (\<exists>j>i. a = xi j)"
      show "\<forall>i a. (xi i, a) \<in> r ^^ Suc (Suc n) \<longrightarrow> (\<exists>j>i. a = xi j)"
      proof (intro allI impI)
        fix i a
        assume "(xi i, a) \<in> r^^(Suc (Suc n))"
        then obtain b where "(xi i, b) \<in> r^^(Suc n) \<and> (b, a) \<in> r" by force
        moreover then obtain j where e1: "j > i \<and> b = xi j" using d1 by blast
        ultimately have "(xi j, a) \<in> r \<and> (xi j, xi (Suc j)) \<in> r" using a2 by blast
        then have "a = xi (Suc j)" using a1 unfolding single_valued_def by blast
        moreover have "Suc j > i" using e1 by force
        ultimately show "\<exists>j>i. a = xi j" by blast
      qed
    qed
  qed
  ultimately show "\<exists> j. i<j \<and> a = xi j" using trancl_power[of _ r] by (metis Suc_pred')
qed

lemma lem_rseq_svacyc_inv_rtr:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U" and a::"'U"
assumes a1: "single_valued r" and a2: "\<forall> i. (xi i, xi (Suc i)) \<in> r"
shows "\<And> i. (xi i, a) \<in> r^* \<Longrightarrow> (\<exists> j. i\<le>j \<and> a = xi j)"
proof -
  fix i
  assume b1: "(xi i, a) \<in> r^*"
  show "\<exists> j. i\<le>j \<and> a = xi j"
  proof (cases "xi i = a")
    assume "xi i = a"
    then show ?thesis by force
  next
    assume "xi i \<noteq> a"
    then have "(xi i, a) \<in> r^+" using b1 by (meson rtranclD)
    then obtain j where "i<j \<and> a = xi j" using assms lem_rseq_svacyc_inv_tr[of r xi i a] by blast
    then have "i \<le> j \<and> a = xi j" by force
    then show ?thesis by blast
  qed
qed

lemma lem_ccrsv_cfseq:
fixes r::"'U rel"
assumes a1: "r \<noteq> {}" and a2: "CCR r" and a3: "single_valued r" and a4: "\<forall>x\<in>Field r. r``{x} \<
      noteq> {}"
shows "\<exists> xi. cfseq r xi"
proof -
  have b1: "Field r \<noteq> {} \<and> (\<forall> x \<in> Field r. \<exists> y. (x,y) \<in> r)"
    using a1 a4 unfolding Field_def by force
  moreover obtain f where "f = (\<lambda> x. SOME y. (x,y) \<in> r)" by blast
  ultimately have b2: "\<forall> x \<in> Field r. (x, f x) \<in> r" by (metis someI_ex)
  obtain x0 where b3: "x0 \<in> Field r" using b1 unfolding Field_def by blast
  obtain xi::"nat \<Rightarrow> 'U" where b4: "xi = (\<lambda> n::nat. (f^^n) x0)" by blast
  obtain A where b5: "A = xi ` UNIV" by blast
  have "r `` A \<subseteq> A"
```

```
  proof
    fix a
    assume "a \<in> r``A"
    then obtain i where "(xi i, a) \<in> r" using b5 by blast
    moreover then have "(xi i, f (xi i)) \<in> r" using b2 unfolding Field_def by blast
    moreover have "f (xi i) = xi (Suc i)" using b4 by simp
    ultimately have "a = xi (Suc i)" using a3 unfolding single_valued_def by blast
    then show "a \<in> A" using b5 by blast
  qed
  then have b6: "A \<in> Inv r" unfolding Inv_def by blast
  have "\<forall> a \<in> Field r. \<exists> i. (a, xi i) \<in> r^*"
  proof
    fix a
    assume "a \<in> Field r"
    then obtain b where "(a,b) \<in> r^* \<and> (x0,b) \<in> r^*" using b3 a2 unfolding CCR_def by blast
    moreover have "x0 = xi 0" using b4 by simp
    ultimately have "(a,b) \<in> r^* \<and> b \<in> A" using b5 b6 lem_inv_rtr_mem[of A r x0 b] by blast
    then show "\<exists> i. (a, xi i) \<in> r^*" using b5 by blast
  qed
  moreover have "\<And> i. (xi i, xi (Suc i)) \<in> r"
  proof -
    fix i0
    show "(xi i0, xi (Suc i0)) \<in> r"
    proof (induct i0)
      show "(xi 0, xi (Suc 0)) \<in> r" using b2 b3 b4 by simp
    next
      fix i
      assume "(xi i, xi (Suc i)) \<in> r"
      then have "xi (Suc i) \<in> Field r" unfolding Field_def by blast
      then show "(xi (Suc i), xi (Suc (Suc i))) \<in> r" using b2 b3 b4 by simp
    qed
  qed
  ultimately show ?thesis unfolding cfseq_def by blast
qed

lemma lem_cfseq_fld: "cfseq r xi \<Longrightarrow> xi ` UNIV \<subseteq> Field r"
  using lem_rseq_rtr[of xi r] unfolding cfseq_def by blast

lemma lem_cfseq_inv: "cfseq r xi \<Longrightarrow> single_valued r \<Longrightarrow> xi ` UNIV \<in> Inv r"
  unfolding cfseq_def single_valued_def Inv_def by blast

lemma lem_scfinv_scf_int: "A \<in> SCF r \<inter> Inv r \<Longrightarrow> B \<in> SCF r \<Longrightarrow> (A \<
      inter> B) \<in> SCF r"
proof -
  assume a1: "A \<in> SCF r \<inter> Inv r" and a2: "B \<in> SCF r"
  moreover have "\<forall> a \<in> Field r. \<exists>b\<in>A \<inter> B. (a, b) \<in> r^*"
  proof
    fix a
    assume "a \<in> Field r"
    then obtain a' where b1: "a' \<in> A \<and> a' \<in> Field r \<and> (a,a') \<in> r^*" using a1 unfolding
      SCF_def by blast
    moreover then obtain b where b2: "b \<in> B \<and> (a',b) \<in> r^*" using a2 unfolding SCF_def by blast
    ultimately have "(a, b) \<in> r^*" by force
    moreover have "b \<in> A \<inter> B" using b1 b2 a1 lem_inv_rtr_mem[of A r a' b] by blast
    ultimately show "\<exists> b \<in> A \<inter> B. (a, b) \<in> r^*" by blast
  qed
  ultimately show "(A \<inter> B) \<in> SCF r" unfolding SCF_def Inv_def by blast
qed

lemma lem_scf_minr: "a \<in> Field r \<Longrightarrow> B \<in> SCF r \<Longrightarrow> \<exists> b \<in> B. (a,b
      ) \<in> (r \<inter> ((UNIV-B) \<times> UNIV))^*"
proof -
  assume a1: "a \<in> Field r" and a2: "B \<in> SCF r"
  then obtain b' where b1: "b' \<in> B \<and> (a,b') \<in> r^*" unfolding SCF_def by blast
  then obtain n where "(a,b') \<in> r^^n" using rtrancl_power by blast
  then obtain f where b2: "f (0::nat) = a \<and> f n = b'" and b3: "\<forall>i<n. (f i, f (Suc i)) \<in> r"
    using relpow_fun_conv[of a b'] by blast
  obtain N where b4: "N = { i. f i \<in> B }" by blast
  obtain s where b5: "s = r \<inter> ((UNIV-B) \<times> UNIV)" by blast
  obtain m where "m = (LEAST i. i \<in> N)" by blast
  moreover have "n \<in> N" using b1 b2 b4 by blast
  ultimately have "m \<in> N \<and> m \<le> n \<and> (\<forall> i \<in> N. m \<le> i)" by (metis LeastI Least_le
      )
  then have "m \<le> n \<and> f m \<in> B \<and> (\<forall> i < m. f i \<notin> B)" using b4 by force
  then have "f 0 = a \<and> f m \<in> B \<and> (\<forall>i<m. (f i, f (Suc i)) \<in> s)" using b2 b3 b5 by force
  then have "f m \<in> B \<and> (a, f m) \<in> s^*"
    using relpow_fun_conv[of a "f m"] rtrancl_power[of _ s] by metis
  then show "\<exists> b \<in> B. (a,b) \<in> (r \<inter> ((UNIV-B) \<times> UNIV))^*" using b5 by blast
```

```
qed

lemma lem_cfseq_ncl:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U"
assumes a1: "cfseq r xi" and a2: "\<not> Conelike r"
shows "\<forall> n. \<exists> k. n \<le> k \<and> (xi (Suc k), xi k) \<notin> r^*"
proof
  fix n
  have "(\<forall> k. n \<le> k \<longrightarrow> (xi (Suc k), xi k) \<in> r^*) \<longrightarrow> False"
  proof
    assume c1: "\<forall> k. n \<le> k \<longrightarrow> (xi (Suc k), xi k) \<in> r^*"
    have "\<And> k. n \<le> k \<longrightarrow> (xi k, xi n) \<in> r^*"
    proof -
      fix k
      show "n \<le> k \<longrightarrow> (xi k, xi n) \<in> r^*"
      proof (induct k)
        show "n \<le> 0 \<longrightarrow> (xi 0, xi n) \<in> r^*" by blast
      next
        fix k
        assume e1: "n \<le> k \<longrightarrow> (xi k, xi n) \<in> r^*"
        show "n \<le> Suc k \<longrightarrow> (xi (Suc k), xi n) \<in> r^*"
        proof
          assume f1: "n \<le> Suc k"
          show "(xi (Suc k), xi n) \<in> r^*"
          proof (cases "n = Suc k")
            assume "n = Suc k"
            then show ?thesis using c1 by blast
          next
            assume "n \<noteq> Suc k"
            then have "(xi k, xi n) \<in> r^* \<and> (xi (Suc k), xi k) \<in> r^*" using f1 e1 c1 by simp
            then show ?thesis by force
          qed
        qed
      qed
    qed
    moreover have "\<forall> k \<le> n. (xi k, xi n) \<in> r^*" using a1 lem_rseq_rtr unfolding cfseq_def by
      blast
    moreover have "\<forall> k::nat. k \<le> n \<or> n \<le> k" by force
    ultimately have b1: "\<forall> k. (xi k, xi n) \<in> r^*" by blast
    have "xi n \<in> Field r" using a1 unfolding cfseq_def Field_def by blast
    moreover have b2: "\<forall> a \<in> Field r. (a, xi n) \<in> r^*"
    proof
      fix a
      assume "a \<in> Field r"
      then obtain i where "(a, xi i) \<in> r^*" using a1 unfolding cfseq_def by blast
      moreover have "(xi i, xi n) \<in> r^*" using b1 by blast
      ultimately show "(a, xi n) \<in> r^*" by force
    qed
    ultimately have "Conelike r" unfolding Conelike_def by blast
    then show "False" using a2 by blast
  qed
  then show "\<exists> k. n \<le> k \<and> (xi (Suc k), xi k) \<notin> r^*" by blast
qed

lemma lem_cfseq_inj:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U"
assumes a1: "cfseq r xi" and a2: "acyclic r"
shows "inj xi"
proof -
  have "\<forall> i j. xi i = xi j \<longrightarrow> i = j"
  proof (intro allI impI)
    fix i j
    assume c1: "xi i = xi j"
    have "i < j \<longrightarrow> False"
    proof
      assume "i < j"
      then have "(xi i, xi j) \<in> r^+" using a1 lem_rseq_tr unfolding cfseq_def by blast
      then show "False" using c1 a2 unfolding acyclic_def by force
    qed
    moreover have "j < i \<longrightarrow> False"
    proof
      assume "j < i"
      then have "(xi j, xi i) \<in> r^+" using a1 lem_rseq_tr unfolding cfseq_def by blast
      then show "False" using c1 a2 unfolding acyclic_def by force
    qed
    ultimately show "i = j" by simp
  qed
  then show ?thesis unfolding inj_on_def by blast
```

```
qed

lemma lem_cfseq_rmon:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U"
assumes a1: "cfseq r xi" and a2: "single_valued r" and a3: "acyclic r"
shows "\<forall> i j. (xi i, xi j) \<in> r^+ \<longrightarrow> i < j"
proof (intro allI impI)
  fix i j
  assume c1: "(xi i, xi j) \<in> r^+"
  then obtain j' where c2: "i < j' \<and> xi j' = xi j"
    using a1 a2 lem_rseq_svacyc_inv_tr[of r xi i] unfolding cfseq_def by metis
  have "j \<le> i \<longrightarrow> False"
  proof
    assume d1: "j \<le> i"
    then have "(xi j, xi i) \<in> r^*" using c2 a1 lem_rseq_rtr unfolding cfseq_def by blast
    then have "(xi i, xi i) \<in> r^+" using c1 by force
    then show "False" using a3 unfolding acyclic_def by blast
  qed
  then show "i < j" by simp
qed

lemma lem_rseq_hd:
assumes "\<forall>i<n. (f i, f (Suc i)) \<in> r"
shows "\<forall>i\<le>n. (f 0, f i) \<in> r^*"
proof (intro allI impI)
  fix i
  assume "i \<le> n"
  then have "\<forall>j<i. (f j, f (Suc j)) \<in> r" using assms by force
  then have "(f 0, f i) \<in> r^^i" using relpow_fun_conv by metis
  then show "(f 0, f i) \<in> r^*" using relpow_imp_rtrancl by blast
qed

lemma lem_rseq_tl:
assumes "\<forall>i<n. (f i, f (Suc i)) \<in> r"
shows "\<forall>i\<le>n. (f i, f n) \<in> r^*"
proof (intro allI impI)
  fix i
  assume b1: "i \<le> n"
  obtain g where b2: "g = (\<lambda> j. f (i + j))" by blast
  then have "\<forall>j<n-i. (g j, g (Suc j)) \<in> r" using assms by force
  moreover have "g 0 = f i \<and> g (n-i) = f n" using b1 b2 by simp
  ultimately have "(f i, f n) \<in> r^^(n-i)" using relpow_fun_conv by metis
  then show "(f i, f n) \<in> r^*" using relpow_imp_rtrancl by blast
qed

lemma lem_ccext_ntr_rpth: "(a,b) \<in> r^^n = (rpth r a b n \<noteq> {})"
proof
  assume "rpth r a b n \<noteq> {}"
  then obtain f where "f \<in> rpth r a b n" by blast
  then show "(a,b) \<in> r^^n" unfolding rpth_def using relpow_fun_conv[of a b] by blast
next
  assume "(a,b) \<in> r^^n"
  then obtain f where "f \<in> rpth r a b n" unfolding rpth_def using relpow_fun_conv[of a b] by blast
  then show "rpth r a b n \<noteq> {}" by blast
qed

lemma lem_ccext_rtr_rpth: "(a,b) \<in> r^* \<Longrightarrow> \<exists> n. rpth r a b n \<noteq> {}"
  using rtrancl_power lem_ccext_ntr_rpth by metis

lemma lem_ccext_rpth_rtr: "rpth r a b n \<noteq> {} \<Longrightarrow> (a,b) \<in> r^*"
  using rtrancl_power lem_ccext_ntr_rpth by metis

lemma lem_ccext_rtr_Fne:
fixes r::"'U rel" and a b::'U
shows "(a,b) \<in> r^* = (\<F> r a b \<noteq> {})"
proof
  assume "(a,b) \<in> r^*"
  then obtain n f where "f \<in> rpth r a b n" using lem_ccext_rtr_rpth[of a b r] by blast
  then have "f`{i. i\<le>n} \<in> \<F> r a b" unfolding \<F>_def by blast
  then show "\<F> r a b \<noteq> {}" by blast
next
  assume "\<F> r a b \<noteq> {}"
  then obtain F where "F \<in> \<F> r a b" by blast
  then obtain n::nat and f::"nat \<Rightarrow> 'U" where "F = f`{i. i\<le>n} \<and> f \<in> rpth r a b n"
      unfolding \<F>_def by blast
  then show "(a,b) \<in> r^*" using lem_ccext_rpth_rtr[of r] by blast
qed
```

```
lemma lem_ccext_fprop: "\<F> r a b \<noteq> {} \<Longrightarrow> \<ff> r a b \<in> \<F> r a b" unfolding \<ff>
      _def using some_in_eq by metis

lemma lem_ccext_ffin: "finite (\<ff> r a b)"
proof (cases "\<F> r a b = {}")
  assume "\<F> r a b = {}"
  then show "finite (\<ff> r a b)" unfolding \<ff>_def by simp
next
  assume "\<F> r a b \<noteq> {}"
  then have "\<ff> r a b \<in> \<F> r a b" using lem_ccext_fprop[of r] by blast
  then show "finite (\<ff> r a b)" unfolding \<F>_def by force
qed

lemma lem_ccr_fin_subr_ext:
fixes r s::"'U rel"
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "finite s"
shows "\<exists> s'::('U rel). finite s' \<and> CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r"
proof -
  have "CCR {}" unfolding CCR_def Field_def by blast
  then have "{} \<subseteq> r \<longrightarrow> (\<exists> r''. CCR r'' \<and> {} \<subseteq> r'' \<and> r'' \<
      subseteq> r \<and> finite r'')" by blast
  moreover have "\<And> p R. finite R \<Longrightarrow> p \<notin> R \<Longrightarrow>
    R \<subseteq> r \<longrightarrow> (\<exists> r''. CCR r'' \<and> R \<subseteq> r'' \<and> r'' \<subseteq> r
      \<and> finite r'') \<Longrightarrow>
    insert p R \<subseteq> r \<longrightarrow> (\<exists> r''. CCR r'' \<and>  insert p R \<subseteq> r'' \<and>
       r'' \<subseteq> r \<and> finite r'')"
  proof
    fix p R
    assume c1: "finite R" and c2: "p \<notin> R"
      and c3: "R \<subseteq> r \<longrightarrow> (\<exists> r''. CCR r'' \<and> R \<subseteq> r'' \<and> r'' \<
      subseteq> r \<and> finite r'')" and c4: "insert p R \<subseteq> r"
    then obtain r'' where c5: "CCR r'' \<and> R \<subseteq> r'' \<and> r'' \<subseteq> r \<and> finite r''" by
      blast
    show "\<exists> r'''. CCR r''' \<and> insert p R \<subseteq> r''' \<and> r''' \<subseteq> r \<and> finite r
      '''"
    proof (cases "r'' = {}")
      assume "r'' = {}"
      then have "insert p R \<subseteq> {p}" using c5 by blast
      moreover have "CCR {p}" unfolding CCR_def Field_def by fastforce
      ultimately show "\<exists> r'''. CCR r''' \<and> insert p R \<subseteq> r''' \<and> r''' \<subseteq> r \<
      and> finite r'''"  using c4 by blast
    next
      assume d1: "r'' \<noteq> {}"
      then obtain xm where d2: "xm \<in> Field r'' \<and> (\<forall> x \<in> Field r''. (x, xm) \<in> r''^*)"
        using c5 lem_Relprop_fin_ccr[of r''] unfolding Conelike_def by blast
      then have d3: "xm \<in> Field r" using c5 unfolding Field_def by blast
      obtain xp yp where d4: "p = (xp, yp)" by force
      then have d5: "yp \<in> Field r" using c4 unfolding Field_def by blast
      then obtain t where d6: "t \<in> Field r \<and> (xm, t) \<in> r^* \<and> (yp, t) \<in> r^*" using a1 d3
      unfolding CCR_def by blast
      then obtain n m where d7: "(xm, t) \<in> r^^n \<and> (yp, t) \<in> r^^m" using rtrancl_power by blast
      obtain fn where d8: "fn (0::nat) = xm \<and> fn n = t \<and> (\<forall>i<n. (fn i, fn(Suc i)) \<in> r)"
      using d7 relpow_fun_conv[of xm t] by blast
      obtain fm where d9: "fm (0::nat) = yp \<and> fm m = t \<and> (\<forall>i<m. (fm i, fm(Suc i)) \<in> r)"
      using d7 relpow_fun_conv[of yp t] by blast
      obtain A where d10: "A = Field r'' \<union> { xp } \<union> { x. \<exists> i\<le>n. x = fn i } \<union> {
      x. \<exists> i\<le>m. x = fm i }" by blast
      obtain r''' where d11: "r''' = r \<inter> (A \<times> A)" by blast
      have d12: "r'' \<subseteq> r'''" using d10 d11 c5 unfolding Field_def by fastforce
      then have d13: "Field r'' \<subseteq> Field r'''" unfolding Field_def by blast
      have d14: "r''^* \<subseteq> r'''^*" using d12 rtrancl_mono by blast
      have d15: "\<forall> i. i<n \<longrightarrow> (fn i, fn(Suc i)) \<in> r'''"
      proof
        fix i
        show "i<n \<longrightarrow> (fn i, fn(Suc i)) \<in> r'''"
        proof (induct i)
          show "0 < n \<longrightarrow> (fn 0, fn (Suc 0)) \<in> r'''"
          proof
            assume "0 < n"
            moreover then have "(Suc 0) \<le> n" by force
            ultimately have "fn 0 \<in> A \<and> fn(Suc 0) \<in> A \<and> (fn 0, fn(Suc 0)) \<in> r" using d8
      d10 by fastforce
            then show "(fn 0, fn (Suc 0)) \<in> r'''" using d11 by blast
          qed
        next
          fix i
          assume g1: "i < n \<longrightarrow> (fn i, fn (Suc i)) \<in> r'''"
          show "Suc i < n \<longrightarrow> (fn (Suc i), fn (Suc (Suc i))) \<in> r'''"
```

```
    proof
      assume "Suc i < n"
      moreover then have "Suc (Suc i) \<le> n" by simp
      moreover then have "(fn i, fn (Suc i)) \<in> r'''" using g1 by simp
      ultimately show "(fn (Suc i), fn (Suc (Suc i))) \<in> r''''" using d8 d10 d11 by blast
    qed
  qed
qed
have d16: "\<forall> i. i<m \<longrightarrow> (fm i, fm(Suc i)) \<in> r''''"
proof
  fix i
  show "i<m \<longrightarrow> (fm i, fm(Suc i)) \<in> r''''"
  proof (induct i)
    show "0 < m \<longrightarrow> (fm 0, fm (Suc 0)) \<in> r''''"
    proof
      assume "0 < m"
      moreover then have "(Suc 0) \<le> m" by force
      ultimately have "fm 0 \<in> A \<and> fm(Suc 0) \<in> A \<and> (fm 0, fm(Suc 0)) \<in> r" using d9
d10 by fastforce
      then show "(fm 0, fm (Suc 0)) \<in> r''''" using d11 by blast
    qed
  next
    fix i
    assume g1: "i < m \<longrightarrow> (fm i, fm (Suc i)) \<in> r''''"
    show "Suc i < m \<longrightarrow> (fm (Suc i), fm (Suc (Suc i))) \<in> r''''"
    proof
      assume "Suc i < m"
      moreover then have "Suc (Suc i) \<le> m" by simp
      moreover then have "(fm i, fm (Suc i)) \<in> r'''" using g1 by simp
      ultimately show "(fm (Suc i), fm (Suc (Suc i))) \<in> r''''" using d9 d10 d11 by blast
    qed
  qed
qed
have d17: "(xm, t) \<in> r'''^*" using d8 d15 relpow_fun_conv[of xm t n r'''] rtrancl_power by blast
then have d18: "t \<in> Field r''''" using d2 d13 by (metis FieldI2 rtrancl.cases subsetCE)
have d19: "(yp, t) \<in> r'''^*" using d9 d16 relpow_fun_conv[of yp t m r'''] rtrancl_power by blast
have d20: "\<forall> j\<le>n. (fn j, t) \<in> r'''^*"
proof (intro allI impI)
  fix j
  assume "j \<le> n"
  moreover obtain f' where "f' = (\<lambda>k. fn (j + k))" by blast
  ultimately have "f' 0 = fn j \<and> f' (n - j) = t \<and> (\<forall>i < n - j. (f' i, f' (Suc i)) \<in>
r''')"
    using d8 d15 by simp
  then show "(fn j, t) \<in> r'''^*"
    using relpow_fun_conv[of "fn j" t "n - j" r'''] rtrancl_power by blast
qed
have d21: "\<forall> j\<le>m. (fm j, t) \<in> r'''^*"
proof (intro allI impI)
  fix j
  assume "j \<le> m"
  moreover obtain f' where "f' = (\<lambda>k. fm (j + k))" by blast
  ultimately have "f' 0 = fm j \<and> f' (m - j) = t \<and> (\<forall>i < m - j. (f' i, f' (Suc i)) \<in>
r''')"
    using d9 d16 by simp
  then show "(fm j, t) \<in> r'''^*"
    using relpow_fun_conv[of "fm j" t "m - j" r'''] rtrancl_power by blast
qed
have "r''' \<subseteq> r" using d11 by blast
moreover have d22: "insert p R \<subseteq> r''''"
proof -
  have "p \<in> r''''" using c4 d4 d9 d10 d11 by blast
  moreover have "R \<subseteq> r''''"
  proof
    fix p'
    assume "p' \<in> R"
    moreover then have "p' \<in> Field R \<times> Field R" using Restr_Field by blast
    moreover have "Field R \<subseteq> Field r''" using c5 unfolding Field_def by blast
    ultimately show "p' \<in> r''''" using c4 d10 d11 by blast
  qed
  ultimately show ?thesis by blast
qed
moreover have "finite r''''" using c5 d10 d11 finite_Field by fastforce
moreover have "CCR r''''"
proof -
  let ?jn = "\<lambda> a b. \<exists>c \<in> Field r'''. (a,c) \<in> r'''^* \<and> (b,c) \<in> r'''^*"
  have "\<forall>a \<in> Field r'''. \<forall>b \<in> Field r'''. ?jn a b"
  proof (intro ballI)
```

```
        fix a b
        assume f1: "a \<in> Field r'''" and f2: "b \<in> Field r'''"
        then have f3: "a \<in> A \<and> b \<in> A" using d11 unfolding Field_def by blast
        have f4: "(xp, t) \<in> r'''^*" using d4 d19 d22 by force
        have "a \<in> Field r'' \<longrightarrow> ?jn a b"
        proof
          assume g1: "a \<in> Field r'''"
          then have g2: "(a, t) \<in> r'''^*" using d2 d14 d17 by fastforce
          have "b \<in> Field r'' \<longrightarrow> ?jn a b" using c5 d13 d14 g1 unfolding CCR_def by blast
          moreover have "?jn a xp" using d4 d18 d19 d22 g2 by force
          moreover have "\<forall> j\<le>n. ?jn a (fn j)" using d18 d20 g2 by blast
          moreover have "\<forall> j\<le>m. ?jn a (fm j)" using d18 d21 g2 by blast
          ultimately show "?jn a b" using d10 f3 by blast
        qed
        moreover have "?jn xp b"
        proof -
          have "b \<in> Field r'' \<longrightarrow> ?jn xp b"
          proof
            assume "b \<in> Field r'''"
            then have "(b, xm) \<in> r'''^*" using d14 d2 by blast
            then show "?jn xp b" using d17 d18 f4 by force
          qed
          moreover have "?jn xp xp" using d4 d22 unfolding Field_def by blast
          moreover have "\<forall> j\<le>n. ?jn xp (fn j)" using d18 d20 f4 by blast
          moreover have "\<forall> j\<le>m. ?jn xp (fm j)" using d18 d21 f4 by blast
          ultimately show "?jn xp b" using d10 f3 by blast
        qed
        moreover have "\<forall>i\<le>n. ?jn (fn i) b"
        proof (intro allI impI)
          fix i
          assume g1: "i \<le> n"
          have "b \<in> Field r'' \<longrightarrow> ?jn (fn i) b"
          proof
            assume "b \<in> Field r'''"
            then have "(b, t) \<in> r'''^*" using d2 d14 d17 by fastforce
            then show "?jn (fn i) b" using d18 d20 g1 by blast
          qed
          moreover have "?jn (fn i) xp" using d18 d20 f4 g1 by blast
          moreover have "\<forall> j\<le>n. ?jn (fn i) (fn j)" using d18 d20 g1 by blast
          moreover have "\<forall> j\<le>m. ?jn (fn i) (fm j)" using d18 d20 d21 g1 by blast
          ultimately show "?jn (fn i) b" using d10 f3 by blast
        qed
        moreover have "\<forall>i\<le>m. ?jn (fm i) b"
        proof (intro allI impI)
          fix i
          assume g1: "i \<le> m"
          have "b \<in> Field r'' \<longrightarrow> ?jn (fm i) b"
          proof
            assume "b \<in> Field r'''"
            then have "(b, t) \<in> r'''^*" using d2 d14 d17 by fastforce
            then show "?jn (fm i) b" using d18 d21 g1 by blast
          qed
          moreover have "?jn (fm i) xp" using d18 d21 f4 g1 by blast
          moreover have "\<forall> j\<le>n. ?jn (fm i) (fn j)" using d18 d20 d21 g1 by blast
          moreover have "\<forall> j\<le>m. ?jn (fm i) (fm j)" using d18 d21 g1 by blast
          ultimately show "?jn (fm i) b" using d10 f3 by blast
        qed
        ultimately show "?jn a b" using d10 f3 by blast
      qed
      then show ?thesis unfolding CCR_def by blast
    qed
    ultimately show "\<exists> r'''. CCR r''' \<and> insert p R \<subseteq> r''' \<and> r''' \<subseteq> r \<
    and> finite r'''" by blast
  qed
 qed
 ultimately have "\<exists> r''. CCR r'' \<and> s \<subseteq> r'' \<and> r'' \<subseteq> r \<and> finite r''"
   using a2 a3 finite_induct[of s "\<lambda> h. h \<subseteq> r \<longrightarrow>  (\<exists> r''. CCR r'' \<
     and> h \<subseteq> r'' \<and> r'' \<subseteq> r \<and> finite r'')"] by simp
 then show ?thesis by blast
qed

lemma lem_Ccext_fint:
fixes r s::"'U rel" and a b::'U
assumes a1: "Restr r (\<ff> r a b) \<subseteq> s" and a2: "(a,b) \<in> r^*"
shows "{a, b} \<subseteq> \<ff> r a b \<and> (\<forall> c \<in> \<ff> r a b. (a,c) \<in> s^* \<and> (c,b) \<in>
      s^*)"
proof -
 obtain A where b1: "A = \<ff> r a b" by blast
```

```
  then have "A \<in> \<F> r a b" using a2 lem_ccext_rtr_Fne[of a b r] lem_ccext_fprop[of r] by blast
  then obtain n f where b2: "A = f ` {i. i \<le> n}" and b3: "f \<in> rpth r a b n" unfolding \<F>_def by blast
  then have "\<forall> i<n. (f i, f (Suc i)) \<in> Restr r A" unfolding rpth_def by simp
  then have b4: "\<forall> i<n. (f i, f (Suc i)) \<in> s" using a1 b1 by blast
  have "{a, b} \<subseteq> \<ff> r a b" using b1 b2 b3 unfolding rpth_def by blast
  moreover have "\<forall> c \<in> \<ff> r a b. (a,c) \<in> s^* \<and> (c,b) \<in> s^*"
  proof
    fix c
    assume "c \<in> \<ff> r a b"
    then obtain k where c1: "k \<le> n \<and> c = f k" using b1 b2 by blast
    have "f \<in> rpth s a c k" using c1 b3 b4 unfolding rpth_def by simp
    moreover have "(\<lambda> i. f (i + k)) \<in> rpth s c b (n - k)" using c1 b3 b4 unfolding rpth_def by simp
    ultimately show "(a,c) \<in> s^* \<and> (c,b) \<in> s^*" using lem_ccext_rpth_rtr[of s] by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_subccr_eqfld:
fixes r r'::"'U rel"
assumes "CCR r" and "r \<subseteq> r'" and "Field r' = Field r"
shows "CCR r'"
proof -
  have "\<forall>a\<in>Field r'. \<forall>b\<in>Field r'. \<exists>c\<in>Field r'. (a, c) \<in> r'^* \<and> (b,
      c) \<in> r'^*"
  proof (intro ballI)
    fix a b
    assume "a\<in>Field r'" and "b\<in>Field r'"
    then have "a \<in> Field r \<and> b \<in> Field r" using assms by blast
    then obtain c where "c \<in> Field r \<and> (a, c) \<in> r^* \<and> (b, c) \<in> r^*" using assms unfolding
      CCR_def by blast
    then have "c \<in> Field r' \<and> (a, c) \<in> r'^* \<and> (b, c) \<in> r'^*" using assms rtrancl_mono by
      blast
    then show "\<exists>c\<in>Field r'. (a, c) \<in> r'^* \<and> (b, c) \<in> r'^*" by blast
  qed
  then show "CCR r'" unfolding CCR_def by blast
qed

lemma lem_Ccext_finsubccr_pext:
fixes r s::"'U rel" and x::'U
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "finite s" and a5: "x \<in> Field r"
shows "\<exists> s'::('U rel). finite s' \<and> CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> x
      \<in> Field s'"
proof -
  obtain y where b1: "(x,y) \<in> r \<or> (y,x) \<in> r" using a5 unfolding Field_def by blast
  then obtain x' y' where b2: "{x',y'} = {x,y} \<and> (x',y') \<in> r" by blast
  obtain s1 where b3: "s1 = s \<union> {(x',y')}" by blast
  then have "finite s1" using a3 by blast
  moreover have "s1 \<subseteq> r" using b2 b3 a2 by blast
  ultimately obtain s' where b4: "finite s' \<and> CCR s' \<and> s1 \<subseteq> s' \<and> s' \<subseteq> r"
      using a1 lem_ccr_fin_subr_ext[of r s1] by blast
  moreover have "x \<in> Field s1" using b2 b3 unfolding Field_def by blast
  ultimately have "x \<in> Field s'" unfolding Field_def by blast
  then show ?thesis using b3 b4 by blast
qed

lemma lem_Ccext_finsubccr_dext:
fixes r::"'U rel" and A::"'U set"
assumes a1: "CCR r" and a2: "A \<subseteq> Field r" and a3: "finite A"
shows "\<exists> s::('U rel). finite s \<and> CCR s \<and> s \<subseteq> r \<and> A \<subseteq> Field s"
proof -
  have "finite {} \<and> {} \<subseteq> Field r \<longrightarrow> (\<exists>s. finite s \<and> CCR s \<and> s \<
      subseteq> r \<and> {} \<subseteq> Field s)" unfolding CCR_def Field_def by blast
  moreover have "\<forall> x F. finite F \<longrightarrow> x \<notin> F \<longrightarrow>
      finite F \<and> F \<subseteq> Field r \<longrightarrow> (\<exists>s. finite s \<and> CCR s \<and> s \<
      subseteq> r \<and> F \<subseteq> Field s) \<longrightarrow>
      finite (insert x F) \<and> insert x F \<subseteq> Field r \<longrightarrow>
    (\<exists>s. finite s \<and> CCR s \<and> s \<subseteq> r \<and> insert x F \<subseteq> Field s)"
  proof(intro allI impI)
    fix x F
    assume c1: "finite F" and c2: "x \<notin> F" and c3: "finite F \<and> F \<subseteq> Field r"
          and c4: "\<exists>s. finite s \<and> CCR s \<and> s \<subseteq> r \<and> F \<subseteq> Field s"
          and c5: "finite (insert x F) \<and> insert x F \<subseteq> Field r"
    then obtain s where c6: "finite s \<and> CCR s \<and> s \<subseteq> r \<and> F \<subseteq> Field s" by blast
    moreover have "x \<in> Field r" using c5 by blast
    ultimately obtain s' where "finite s' \<and> CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> x
       \<in> Field s'"
      using a1 lem_Ccext_finsubccr_pext[of r s x] by blast
    moreover then have "insert x F \<subseteq> Field s'" using c6 unfolding Field_def by blast
```

```
    ultimately show "\<exists>s'. finite s' \<and> CCR s' \<and> s' \<subseteq> r \<and> insert x F \<subseteq>
      Field s'" by blast
  qed
  ultimately have "finite A \<and> A \<subseteq> Field r \<longrightarrow> (\<exists>s. finite s \<and> CCR s \<
      and> s \<subseteq> r \<and> A \<subseteq> Field s)"
    using finite_induct[of A "\<lambda> A. finite A \<and> A \<subseteq> Field r \<longrightarrow> (\<exists> s.
       finite s \<and> CCR s \<and> s \<subseteq> r \<and> A \<subseteq> Field s)"]
    by simp
  then show ?thesis using a2 a3 by blast
qed

lemma lem_Ccext_infsubccr_pext:
fixes r s::"'U rel" and x::'U
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "\<not> finite s" and a5: "x \<in> Field r"
shows "\<exists> s'::('U rel). CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<and>
      x \<in> Field s'"
proof -
  obtain G::"'U set \<Rightarrow> 'U rel set" where b1: "G = (\<lambda> A. {t::'U rel. finite t \<and> CCR t \<
      and> t \<subseteq> r \<and> A \<subseteq> Field t})" by blast
  obtain g::"'U set \<Rightarrow> 'U rel" where b2: "g = (\<lambda> A. if A \<subseteq> Field r \<and> finite A
      then (SOME t. t \<in> G A) else {})" by blast
  have b3: "\<forall> A. A \<subseteq> Field r \<and> finite A \<longrightarrow> finite (g A) \<and> CCR (g A)
      \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)"
  proof (intro allI impI)
    fix A
    assume c1: "A \<subseteq> Field r \<and> finite A"
    then have "g A = (SOME t. t \<in> G A)" using b2 by simp
    moreover have "G A \<noteq> {}" using b1 a1 c1 lem_Ccext_finsubccr_dext[of r A] by blast
    ultimately have "g A \<in> G A" using some_in_eq by metis
    then show "finite (g A) \<and> CCR (g A) \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)" using
      b1 by blast
  qed
  have b4: "\<forall> A. \<not> (A \<subseteq> Field r \<and> finite A) \<longrightarrow> g A = {}" using b2 by
      simp
  obtain H::"'U set \<Rightarrow> 'U set"
    where b5: "H = (\<lambda> X. X \<union> \<Union> {S . \<exists> a\<in>X. \<exists>b\<in>X. S = Field (g {a,b
      })})" by blast
  obtain ax bx where b6: "(ax, bx) \<in> r \<and> x \<in> {ax, bx}" using a5 unfolding Field_def by blast
  obtain D0::"'U set" where b7: "D0 = Field s \<union> {ax, bx}" by blast
  obtain Di::"nat \<Rightarrow> 'U set" where b8: "Di = (\<lambda> n. (H^^n) D0)" by blast
  obtain D::"'U set" where b9: "D = \<Union> {X. \<exists> n. X = Di n}" by blast
  obtain s' where b10: "s' = Restr r D" by blast
  have b11: "\<forall> n. (\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
  proof
    fix n0
    show "(\<not> finite (Di n0)) \<and> |Di n0| \<le>o |s|"
    proof (induct n0)
      have "finite {ax, bx}" by blast
      moreover have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
      ultimately have "\<not> finite (Field s) \<and> |{ax, bx}| \<le>o |Field s|"
        using card_of_Well_order card_of_ordLeq_infinite ordLeq_total by metis
      then have "|D0| =o |Field s|" using b7 card_of_Un_infinite by blast
      moreover have "|Field s| =o |s|" using a3 lem_rel_inf_fld_card by blast
      ultimately have "|D0| \<le>o |s|" using ordIso_imp_ordLeq ordIso_transitive by blast
      moreover have "\<not> finite D0" using a3 b7 lem_fin_fl_rel by blast
      ultimately show "\<not> finite (Di 0) \<and> |Di 0| \<le>o |s|" using b8 by simp
    next
      fix n
      assume d1: "(\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
      moreover then have "|(Di n) \<times> (Di n)| =o |Di n|" by simp
      ultimately have d2: "|(Di n) \<times> (Di n)| \<le>o |s|" using ordIso_imp_ordLeq ordLeq_transitive by
       blast
      have d3: "\<forall> a \<in> (Di n). \<forall> b \<in> (Di n). |Field (g {a, b})| \<le>o |s|"
      proof (intro ballI)
        fix a b
        assume "a \<in> (Di n)" and "b \<in> (Di n)"
        have "finite (g {a, b})" using b3 b4 by (metis finite.emptyI)
        then have "finite (Field (g {a, b}))" using lem_fin_fl_rel by blast
        then have "|Field (g {a, b})| <o |s|" using a3 finite_ordLess_infinite2 by blast
        then show "|Field (g {a, b})| \<le>o |s|" using ordLess_imp_ordLeq by blast
      qed
      have d4: "Di (Suc n) = H (Di n)" using b8 by simp
      then have "Di n \<subseteq> Di (Suc n)" using b5 by blast
      then have "\<not> finite (Di (Suc n))" using d1 finite_subset by blast
      moreover have "|Di (Suc n)| \<le>o |s|"
      proof -
        obtain I where e1: "I = (Di n) \<times> (Di n)" by blast
        obtain f where e2: "f = (\<lambda> (a,b). Field (g {a,b}))" by blast
```

```
      have "|I| \<le>o |s|" using e1 d2 by blast
      moreover have "\<forall>i\<in>I. |f i| \<le>o |s|" using e1 e2 d3 by simp
      ultimately have "|\<Union> i\<in>I. f i| \<le>o |s|" using a3 card_of_UNION_ordLeq_infinite[of s I f] by
     blast
      moreover have "Di (Suc n) = (Di n) \<union> (\<Union> i\<in>I. f i)" using e1 e2 d4 b5 by blast
      ultimately show ?thesis using d1 a3 by simp
    qed
    ultimately show "(\<not> finite (Di (Suc n))) \<and> |Di (Suc n)| \<le>o |s|" by blast
  qed
qed
have b12: "\<forall> m. \<forall> n. n \<le> m \<longrightarrow> Di n \<le> Di m"
proof
  fix m0
  show "\<forall> n. n \<le> m0 \<longrightarrow> Di n \<le> Di m0"
  proof (induct m0)
    show "\<forall>n\<le>0. Di n \<subseteq> Di 0" by blast
  next
    fix m
    assume d1: "\<forall>n\<le>m. Di n \<subseteq> Di m"
    show "\<forall>n\<le>Suc m. Di n \<subseteq> Di (Suc m)"
    proof (intro allI impI)
      fix n
      assume e1: "n \<le> Suc m"
      have "Di (Suc m) = H (Di m)" using b8 by simp
      moreover have "Di m \<subseteq> H (Di m)" using b5 by blast
      ultimately have "n \<le> m \<longrightarrow> Di n \<subseteq> Di (Suc m)" using d1 by blast
      moreover have "n = (Suc m) \<or> n \<le> m" using e1 by force
      ultimately show "Di n \<subseteq> Di (Suc m)" by blast
    qed
  qed
qed
have "Di 0 \<subseteq> D" using b9 by blast
then have b13: "Field s \<subseteq> D" using b7 b8 by simp
then have b14: "s \<subseteq> s' \<and> s' \<subseteq> r" using a2 b10 unfolding Field_def by force
moreover have b15: "|D| \<le>o |s|"
proof -
  have "|UNIV::nat set| \<le>o |s|" using a3 infinite_iff_card_of_nat by blast
  then have "|\<Union> n. Di n| \<le>o |s|" using b11 a3 card_of_UNION_ordLeq_infinite[of s UNIV Di] by blast
  moreover have "D = (\<Union> n. Di n)" using b9 by force
  ultimately show ?thesis by blast
qed
moreover have "|s'| =o |s|"
proof -
  have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
  then have "\<not> finite D" using b13 finite_subset by blast
  then have "|D \<times> D| =o |D|" by simp
  moreover have "s' \<subseteq> D \<times> D" using b10 by blast
  ultimately have "|s'| \<le>o |s|" using b15 card_of_mono1 ordLeq_ordIso_trans ordLeq_transitive by metis
  moreover have "|s| \<le>o |s'|" using b14 by simp
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed
moreover have "x \<in> Field s'"
proof -
  have "Di 0 \<subseteq> D" using b9 by blast
  then have "{ax, bx} \<subseteq> D" using b7 b8 by simp
  then have "(ax, bx) \<in> s'" using b6 b10 by blast
  then show ?thesis using b6 unfolding Field_def by blast
qed
moreover have "CCR s'"
proof -
  have "\<forall> a \<in> Field s'. \<forall> b \<in> Field s'. \<exists> c \<in> Field s'. (a,c) \<in> (s')^*
     \<and> (b,c) \<in> (s')^*"
  proof (intro ballI)
    fix a b
    assume d1: "a \<in> Field s'" and d2: "b \<in> Field s'"
    then have d3: "a \<in> D \<and> b \<in> D" using b10 unfolding Field_def by blast
    then obtain ia ib where d4: "a \<in> Di ia \<and> b \<in> Di ib" using b9 by blast
    obtain k where d5: "k = (max ia ib)" by blast
    then have "ia \<le> k \<and> ib \<le> k" by simp
    then have d6: "a \<in> Di k \<and> b \<in> Di k" using d4 b12 by blast
    obtain p where d7: "p = g {a,b}" by blast
    have "Field p \<subseteq> H (Di k)" using b5 d6 d7 by blast
    moreover have "H (Di k) = Di (Suc k)" using b8 by simp
    moreover have "Di (Suc k) \<subseteq> D" using b9 by blast
    ultimately have d8: "Field p \<subseteq> D" by blast
    have "{a, b} \<subseteq> Field r" using d1 d2 b10 unfolding Field_def by blast
    moreover have "finite {a, b}" by simp
    ultimately have d9: "CCR p \<and> p \<subseteq> r \<and> {a,b} \<subseteq> Field p" using d7 b3 by blast
```

```
      then obtain c where d10: "c \<in> Field p \<and> (a,c) \<in> p^* \<and> (b,c) \<in> p^*" unfolding CCR_def
       by blast
      have "(p `` D) \<subseteq> D" using d8 unfolding Field_def by blast
      then have "D \<in> Inv p" unfolding Inv_def by blast
      then have "p^* \<inter> (D\<times>(UNIV::'U set)) \<subseteq> (Restr p D)^*" using lem_Inv_restr_rtr[of D
      p] by blast
      moreover have "Restr p D \<subseteq> s'" using d9 b10 by blast
      moreover have "(a,c) \<in> p^* \<inter> (D\<times>(UNIV::'U set)) \<and> (b,c) \<in> p^* \<inter> (D\<
      times>(UNIV::'U set))" using d10 d3 by blast
      ultimately have "(a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" using rtrancl_mono by blast
      moreover then have "c \<in> Field s'" using d1 lem_rtr_field by fastforce
      ultimately show "\<exists> c \<in> Field s'. (a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" by blast
    qed
    then show ?thesis unfolding CCR_def by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_finsubccr_set_ext:
fixes r s::"'U rel" and A::"'U set"
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "finite s" and a4: "A \<subseteq> Field r" and a5: "finite
       A"
shows "\<exists> s'::('U rel). CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> finite s' \<and> A
       \<subseteq> Field s'"
proof -
  obtain Pt::"'U \<Rightarrow> 'U rel" where p1: "Pt = (\<lambda> x. {p \<in> r. x = fst p \<or> x = snd p})" by
       blast
  obtain pt::"'U \<Rightarrow> 'U\<times>'U" where p2: "pt = (\<lambda> x. (SOME p. p \<in> Pt x))" by blast
  have "\<forall> x\<in>A. Pt x \<noteq> {}" using a4 unfolding p1 Field_def by force
  then have p3: "\<forall> x\<in>A. pt x \<in> Pt x" unfolding p2 by (metis (full_types) Collect_empty_eq
      Collect_mem_eq someI_ex)
  have b2: "pt`A \<subseteq> r" using p1 p3 by blast
  obtain s1 where b3: "s1 = s \<union> (pt`A)" by blast
  then have "finite s1" using a3 a5 by blast
  moreover have "s1 \<subseteq> r" using b2 b3 a2 by blast
  ultimately obtain s' where b4: "finite s' \<and> CCR s' \<and> s1 \<subseteq> s' \<and> s' \<subseteq> r"
      using a1 lem_ccr_fin_subr_ext[of r s1] by blast
  moreover have "A \<subseteq> Field s1"
  proof
    fix x
    assume c1: "x \<in> A"
    then have "pt x \<in> s1" using b3 by blast
    moreover obtain ax bx where c2: "pt x = (ax,bx)" by force
    ultimately have "ax \<in> Field s1 \<and> bx \<in> Field s1" unfolding Field_def by force
    then show "x \<in> Field s1" using c1 c2 p1 p3 by force
  qed
  ultimately have "A \<subseteq> Field s'" unfolding Field_def by blast
  then show ?thesis using b3 b4 by blast
qed

lemma lem_Ccext_infsubccr_set_ext:
fixes r s::"'U rel" and A::"'U set"
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "\<not> finite s" and a4: "A \<subseteq> Field r" and a5:
       "|A| \<le>o |Field s|"
shows "\<exists> s'::('U rel). CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<and>
       A \<subseteq> Field s'"
proof -
  obtain G::"'U set \<Rightarrow> 'U rel set" where b1: "G = (\<lambda> A. {t::'U rel. finite t \<and> CCR t \<
      and> t \<subseteq> r \<and> A \<subseteq> Field t})" by blast
  obtain g::"'U set \<Rightarrow> 'U rel" where b2: "g = (\<lambda> A. if A \<subseteq> Field r \<and> finite A
      then (SOME t. t \<in> G A) else {})" by blast
  have b3: "\<forall> A. A \<subseteq> Field r \<and> finite A \<longrightarrow> finite (g A) \<and> CCR (g A)
      \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)"
  proof (intro allI impI)
    fix A
    assume c1: "A \<subseteq> Field r \<and> finite A"
    then have "g A = (SOME t. t \<in> G A)" using b2 by simp
    moreover have "G A \<noteq> {}" using b1 a1 c1 lem_Ccext_finsubccr_dext[of r A] by blast
    ultimately have "g A \<in> G A" using some_in_eq by metis
    then show "finite (g A) \<and> CCR (g A) \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)" using
      b1 by blast
  qed
  have b4: "\<forall> A. \<not> (A \<subseteq> Field r \<and> finite A) \<longrightarrow> g A = {}" using b2 by
      simp
  obtain H::"'U set \<Rightarrow> 'U set"
    where b5: "H = (\<lambda> X. X \<union> \<Union> {S . \<exists> a\<in>X. \<exists>b\<in>X. S = Field (g {a,b
      })})" by blast
```

```
obtain Pt::"'U \<Rightarrow> 'U rel" where p1: "Pt = (\<lambda> x. {p \<in> r. x = fst p \<or> x = snd p})" by
     blast
obtain pt::"'U \<Rightarrow> 'U\<times>'U" where p2: "pt = (\<lambda> x. (SOME p. p \<in> Pt x))" by blast
have "\<forall> x\<in>A. Pt x \<noteq> {}" using a4 unfolding p1 Field_def by force
then have p3: "\<forall> x\<in>A. pt x \<in> Pt x" unfolding p2 by (metis (full_types) Collect_empty_eq
    Collect_mem_eq someI_ex)
obtain D0 where b7: "D0 = Field s \<union> fst`(pt`A) \<union> snd`(pt`A)" by blast
obtain Di::"nat \<Rightarrow> 'U set" where b8: "Di = (\<lambda> n. (H^^n) D0)" by blast
obtain D::"'U set" where b9: "D = \<Union> {X. \<exists> n. X = Di n}" by blast
obtain s' where b10: "s' = Restr r D" by blast
have b11: "\<forall> n. (\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
proof
  fix n0
  show "(\<not> finite (Di n0)) \<and> |Di n0| \<le>o |s|"
  proof (induct n0)
    have "|D0| =o |Field s|"
    proof -
      have "|fst`(pt`A)| \<le>o |(pt`A)| \<and> |(pt`A)| \<le>o |A|" by simp
      then have c1: "|fst`(pt`A)| \<le>o |A|" using ordLeq_transitive by blast
      have "|snd`(pt`A)| \<le>o |(pt`A)| \<and> |(pt`A)| \<le>o |A|" by simp
      then have c2: "|snd`(pt`A)| \<le>o |A|" using ordLeq_transitive by blast
      have "|fst`(pt`A)| \<le>o |Field s| \<and> |snd`(pt`A)| \<le>o |Field s|"
        using c1 c2 a5 ordLeq_transitive by blast
      moreover have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
      ultimately have c3: "|D0| \<le>o |Field s|" unfolding b7 by simp
      have "Field s \<subseteq> D0" unfolding b7 by blast
      then have "|Field s| \<le>o |D0|" by simp
      then show ?thesis using c3 ordIso_iff_ordLeq by blast
    qed
    moreover have "|Field s| =o |s|" using a3 lem_rel_inf_fld_card by blast
    ultimately have "|D0| \<le>o |s|" using ordIso_imp_ordLeq ordIso_transitive by blast
    moreover have "\<not> finite D0" using a3 b7 lem_fin_fl_rel by blast
    ultimately show "\<not> finite (Di 0) \<and> |Di 0| \<le>o |s|" using b8 by simp
  next
    fix n
    assume d1: "(\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
    moreover then have "|(Di n) \<times> (Di n)| =o |Di n|" by simp
    ultimately have d2: "|(Di n) \<times> (Di n)| \<le>o |s|" using ordIso_imp_ordLeq ordLeq_transitive by
     blast
    have d3: "\<forall> a \<in> (Di n). \<forall> b \<in> (Di n). |Field (g {a, b})| \<le>o |s|"
    proof (intro ballI)
      fix a b
      assume "a \<in> (Di n)" and "b \<in> (Di n)"
      have "finite (g {a, b})" using b3 b4 by (metis finite.emptyI)
      then have "finite (Field (g {a, b}))" using lem_fin_fl_rel by blast
      then have "|Field (g {a, b})| <o |s|" using a3 finite_ordLess_infinite2 by blast
      then show "|Field (g {a, b})| \<le>o |s|" using ordLess_imp_ordLeq by blast
    qed
    have d4: "Di (Suc n) = H (Di n)" using b8 by simp
    then have "Di n \<subseteq> Di (Suc n)" using b5 by blast
    then have "\<not> finite (Di (Suc n))" using d1 finite_subset by blast
    moreover have "|Di (Suc n)| \<le>o |s|"
    proof -
      obtain I where e1: "I = (Di n) \<times> (Di n)" by blast
      obtain f where e2: "f = (\<lambda> (a,b). Field (g {a,b}))" by blast
      have "|I| \<le>o |s|" using e1 d2 by blast
      moreover have "\<forall>i\<in>I. |f i| \<le>o |s|" using e1 e2 d3 by simp
      ultimately have "|\<Union> i\<in>I. f i| \<le>o |s|" using a3 card_of_UNION_ordLeq_infinite[of s I f] by
     blast
      moreover have "Di (Suc n) = (Di n) \<union> (\<Union> i\<in>I. f i)" using e1 e2 d4 b5 by blast
      ultimately show ?thesis using d1 a3 by simp
    qed
    ultimately show "(\<not> finite (Di (Suc n))) \<and> |Di (Suc n)| \<le>o |s|" by blast
  qed
qed
have b12: "\<forall> m. \<forall> n. n \<le> m \<longrightarrow> Di n \<le> Di m"
proof
  fix m0
  show "\<forall> n. n \<le> m0 \<longrightarrow> Di n \<le> Di m0"
  proof (induct m0)
    show "\<forall>n\<le>0. Di n \<subseteq> Di 0" by blast
  next
    fix m
    assume d1: "\<forall>n\<le>m. Di n \<subseteq> Di m"
    show "\<forall>n\<le>Suc m. Di n \<subseteq> Di (Suc m)"
    proof (intro allI impI)
      fix n
      assume e1: "n \<le> Suc m"
```

```
      have "Di (Suc m) = H (Di m)" using b8 by simp
      moreover have "Di m \<subseteq> H (Di m)" using b5 by blast
      ultimately have "n \<le> m \<longrightarrow> Di n \<subseteq> Di (Suc m)" using d1 by blast
      moreover have "n = (Suc m) \<or> n \<le> m" using e1 by force
      ultimately show "Di n \<subseteq> Di (Suc m)" by blast
    qed
  qed
qed
have "Di 0 \<subseteq> D" using b9 by blast
then have b13: "Field s \<subseteq> D" using b7 b8 by simp
then have b14: "s \<subseteq> s' \<and> s' \<subseteq> r" using a2 b10 unfolding Field_def by force
moreover have b15: "|D| \<le>o |s|"
proof -
  have "|UNIV::nat set| \<le>o |s|" using a3 infinite_iff_card_of_nat by blast
  then have "|\<Union> n. Di n| \<le>o |s|" using b11 a3 card_of_UNION_ordLeq_infinite[of s UNIV Di] by blast
  moreover have "D = (\<Union> n. Di n)" using b9 by force
  ultimately show ?thesis by blast
qed
moreover have "|s'| =o |s|"
proof -
  have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
  then have "\<not> finite D" using b13 finite_subset by blast
  then have "|D \<times> D| =o |D|" by simp
  moreover have "s' \<subseteq> D \<times> D" using b10 by blast
  ultimately have "|s'| \<le>o |s|" using b15 card_of_mono1 ordLeq_ordIso_trans ordLeq_transitive by metis
  moreover have "|s| \<le>o |s'|" using b14 by simp
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed
moreover have "A \<subseteq> Field s'"
proof
  fix x
  assume c1: "x \<in> A"
  obtain ax bx where c2: "ax = fst (pt x) \<and> bx = snd (pt x)" by blast
  have "pt x \<in> Pt x" using c1 p3 by blast
  then have c3: "(ax, bx) \<in> r \<and> x \<in> {ax,bx}" using c2 p1 by simp
  have "{ax, bx} \<subseteq> D0" using b7 c1 c2 by blast
  moreover have "Di 0 \<subseteq> D" using b9 by blast
  moreover have "Di 0 = D0" using b8 by simp
  ultimately have "{ax, bx} \<subseteq> D" by blast
  then have "(ax, bx) \<in> s'" using c3 b10 by blast
  then show "x \<in> Field s'" using c3 unfolding Field_def by blast
qed
moreover have "CCR s'"
proof -
  have "\<forall> a \<in> Field s'. \<forall> b \<in> Field s'. \<exists> c \<in> Field s'. (a,c) \<in> (s')^*
     \<and> (b,c) \<in> (s')^*"
  proof (intro ballI)
    fix a b
    assume d1: "a \<in> Field s'" and d2: "b \<in> Field s'"
    then have d3: "a \<in> D \<and> b \<in> D" using b10 unfolding Field_def by blast
    then obtain ia ib where d4: "a \<in> Di ia \<and> b \<in> Di ib" using b9 by blast
    obtain k where d5: "k = (max ia ib)" by blast
    then have "ia \<le> k \<and> ib \<le> k" by simp
    then have d6: "a \<in> Di k \<and> b \<in> Di k" using d4 b12 by blast
    obtain p where d7: "p = g {a,b}" by blast
    have "Field p \<subseteq> H (Di k)" using b5 d6 d7 by blast
    moreover have "H (Di k) = Di (Suc k)" using b8 by simp
    moreover have "Di (Suc k) \<subseteq> D" using b9 by blast
    ultimately have d8: "Field p \<subseteq> D" by blast
    have "{a, b} \<subseteq> Field r" using d1 d2 b10 unfolding Field_def by blast
    moreover have "finite {a, b}" by simp
    ultimately have d9: "CCR p \<and> p \<subseteq> r \<and> {a,b} \<subseteq> Field p" using d7 b3 by blast
    then obtain c where d10: "c \<in> Field p \<and> (a,c) \<in> p^* \<and> (b,c) \<in> p^*" unfolding CCR_def
     by blast
    have "(p `` D) \<subseteq> D" using d8 unfolding Field_def by blast
    then have "D \<in> Inv p" unfolding Inv_def by blast
    then have "p^* \<inter> (D\<times>(UNIV::'U set)) \<subseteq> (Restr p D)^*" using lem_Inv_restr_rtr[of D
    p] by blast
    moreover have "Restr p D \<subseteq> s'" using d9 b10 by blast
    moreover have "(a,c) \<in> p^* \<inter> (D\<times>(UNIV::'U set)) \<and> (b,c) \<in> p^* \<inter> (D\<
    times>(UNIV::'U set))" using d10 d3 by blast
    ultimately have "(a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" using rtrancl_mono by blast
    moreover then have "c \<in> Field s'" using d1 lem_rtr_field by fastforce
    ultimately show "\<exists> c \<in> Field s'. (a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" by blast
  qed
  then show ?thesis unfolding CCR_def by blast
qed
ultimately show ?thesis by blast
```

```
qed

lemma lem_Ccext_finsubccr_pext5:
fixes r::"'U rel" and A B::"'U set" and x::'U
assumes a1: "CCR r" and a2: "finite A" and a3: "A \<in> SF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR
       (Restr r A') \<and> finite A'
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B))"
proof -
  have q1: "Field (Restr r A) = A" using a3 unfolding SF_def by blast
  obtain s where "s = (Restr r A)" by blast
  then have q2: "s \<subseteq> r" and q3: "finite s" and q4: "A = Field s"
    using a2 q1 lem_fin_fl_rel by (blast, metis, blast)
  obtain S where b1: "S = (\<lambda> a. r``{a} - B )" by blast
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. f a \<in> S' a" by (metis someI_ex)
  then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
      longrightarrow> f a = a)"
    unfolding b2 by (clarsimp, metis singletonD)
  obtain y1 y2::'U where n1: "Field r \<noteq> {} \<longrightarrow> {y1, y2} \<subseteq> Field r"
                    and n2: "(\<not> (\<exists> y::'U. Field r - B \<subseteq> {y})) \<longrightarrow> y1 \<
      notin> B \<and> y2 \<notin> B \<and> y1 \<noteq> y2" by blast
  obtain A1 where b4: "A1 = ({x,y1,y2} \<inter> Field r) \<union> A \<union> (f ` A)" by blast
  have "A1 \<subseteq> Field r"
  proof -
    have c1: "A \<subseteq> Field r" using q4 q2 unfolding Field_def by blast
    moreover have "f ` A \<subseteq> Field r"
    proof
      fix x
      assume "x \<in> f ` A"
      then obtain a where d2: "a \<in> A \<and> x = f a" by blast
      show "x \<in> Field r"
      proof (cases "S a = {}")
        assume "S a = {}"
        then have "x = a" using c1 d2 b3 by blast
        then show "x \<in> Field r" using d2 c1 by blast
      next
        assume "S a \<noteq> {}"
        then have "x \<in> S a" using d2 b3 by blast
        then show "x \<in> Field r" using b1 unfolding Field_def by blast
      qed
    qed
    ultimately show "A1 \<subseteq> Field r" using b4 by blast
  qed
  moreover have s0: "finite A1" using b4 q3 q4 lem_fin_fl_rel by blast
  ultimately obtain s' where s1: "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> finite s' \<and>
       A1 \<subseteq> Field s'"
    using a1 q2 q3 lem_Ccext_finsubccr_set_ext[of r s A1] by blast
  obtain A' where s2: "A' = Field s'" by blast
  obtain s'' where s3: "s'' = Restr r A'" by blast
  then have s4: "s' \<subseteq> s'' \<and> Field s'' = A'" using s1 s2 lem_Relprop_fld_sat[of s' r s''] by blast
  have s5: "finite (Field s')" using s1 lem_fin_fl_rel by blast
  have "A1 \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 s1 s2 by blast
  moreover have "CCR (Restr r A')"
  proof -
    have "CCR s''" using s1 s2 s4 lem_Ccext_subccr_eqfld[of s' s''] by blast
    then show ?thesis using s3 by blast
  qed
  ultimately have b6: "A1 \<union> ({x} \<inter> Field r) \<subseteq> A' \<and> CCR (Restr r A')" by blast
  moreover then have "A \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 by blast
  moreover have "finite A'" using s2 s5 by blast
  moreover have "\<forall>a\<in>A. r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}"
  proof
    fix a
    assume c1: "a \<in> A"
    have "\<not> (r``{a} \<subseteq> B) \<longrightarrow> r``{a} \<inter> (A'-B) \<noteq> {}"
    proof
      assume "\<not> (r``{a} \<subseteq> B)"
      then have "S a \<noteq> {}" unfolding b1 by blast
      then have "f a \<in> r``{a} - B" using b1 b3 by blast
      moreover have "f a \<in> A'" using c1 b4 b6 by blast
      ultimately show "r``{a} \<inter> (A'-B) \<noteq> {}" by blast
    qed
    then show "r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}" by blast
```

```
  qed
  moreover have "A' \<in> SF r" using s3 s4 unfolding SF_def by blast
  moreover have "(\<exists> y::'U. A' - B = {y}) \<longrightarrow> Field r \<subseteq> (A' \<union> B)"
  proof
    assume c1: "\<exists> y::'U. A' - B = {y}"
    moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
    ultimately have "Field r \<noteq> {}" by blast
    then have "{y1, y2} \<subseteq> Field r" using n1 by blast
    then have "{y1, y2} \<subseteq> A'" using b4 s1 s2 by fast
    then have "\<not> (\<exists>y. Field r - B \<subseteq> {y}) \<longrightarrow> {y1, y2} \<subseteq> A' - B \<
      and> y1 \<noteq> y2" using n2 by blast
    moreover have "\<not> ({y1, y2} \<subseteq> A' - B \<and> y1 \<noteq> y2)" using c1 by force
    ultimately have "\<exists> y::'U. Field r - B \<subseteq> {y}" by blast
    then show "Field r \<subseteq> A' \<union> B" using c1 c2 by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_infsubccr_pext5:
fixes r::"'U rel" and A B::"'U set" and x::'U
assumes a1: "CCR r" and a2: "\<not> finite A" and a3: "A \<in> SF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR
       (Restr r A') \<and> |A'| =o |A|
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B))"
proof -
  have q1: "Field (Restr r A) = A" using a3 unfolding SF_def by blast
  obtain s where "s = (Restr r A)" by blast
  then have q2: "s \<subseteq> r" and q3: "\<not> finite s" and q4: "A = Field s"
    using a2 q1 lem_fin_fl_rel by (blast, metis, blast)
  obtain S where b1: "S = (\<lambda> a. r``{a} - B )" by blast
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. f a \<in> S' a" by (metis someI_ex)
  then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
      longrightarrow> f a = a)"
    unfolding b2 by (clarsimp, metis singletonD)
  obtain y1 y2::'U where n1: "Field r \<noteq> {} \<longrightarrow> {y1, y2} \<subseteq> Field r"
                    and n2: "(\<not> (\<exists> y::'U. Field r - B \<subseteq> {y})) \<longrightarrow> y1 \<
      notin> B \<and> y2 \<notin> B \<and> y1 \<noteq> y2" by blast
  obtain A1 where b4: "A1 = ({x, y1, y2} \<inter> Field r) \<union> A \<union> (f ` A)" by blast
  have "A1 \<subseteq> Field r"
  proof -
    have c1: "A \<subseteq> Field r" using q4 q2 unfolding Field_def by blast
    moreover have "f ` A \<subseteq> Field r"
    proof
      fix x
      assume "x \<in> f ` A"
      then obtain a where d2: "a \<in> A \<and> x = f a" by blast
      show "x \<in> Field r"
      proof (cases "S a = {}")
        assume "S a = {}"
        then have "x = a" using c1 d2 b3 by blast
        then show "x \<in> Field r" using d2 c1 by blast
      next
        assume "S a \<noteq> {}"
        then have "x \<in> S a" using d2 b3 by blast
        then show "x \<in> Field r" using b1 unfolding Field_def by blast
      qed
    qed
    ultimately show "A1 \<subseteq> Field r" using b4 by blast
  qed
  moreover have s0: "|A1| \<le>o |Field s|"
  proof -
    obtain C1 where c1: "C1 = {x,y1,y2} \<inter> Field r" by blast
    obtain C2 where c2: "C2 = A \<union> f ` A" by blast
    have "\<not> finite A" using q4 q3 lem_fin_fl_rel by blast
    then have "|C2| =o |A|" using c2 b4 q3 by simp
    then have "|C2| \<le>o |Field s|" unfolding q4 using ordIso_iff_ordLeq by blast
    moreover have c3: "\<not> finite (Field s)" using q3 lem_fin_fl_rel by blast
    moreover have "|C1| \<le>o |Field s|"
    proof -
      have "|{x,y1,y2}| \<le>o |Field s|" using c3
        by (meson card_of_Well_order card_of_ordLeq_finite finite.emptyI finite.insertI ordLeq_total)
      moreover have "|C1| \<le>o |{x,y1,y2}|" unfolding c1 by simp
      ultimately show ?thesis using ordLeq_transitive by blast
```

```
    qed
    ultimately have "|C1 \<union> C2| \<le>o |Field s|" unfolding b4 using card_of_Un_ordLeq_infinite by blast
    moreover have "A1 = C1 \<union> C2" using c1 c2 b4 by blast
    ultimately show ?thesis by blast
  qed
  ultimately obtain s' where s1: "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<
      and> A1 \<subseteq> Field s'"
    using a1 q2 q3 lem_Ccext_infsubccr_set_ext[of r s A1] by blast
  obtain A' where s2: "A' = Field s'" by blast
  obtain s'' where s3: "s'' = Restr r A'" by blast
  then have s4: "s' \<subseteq> s'' \<and> Field s'' = A'" using s1 s2 lem_Relprop_fld_sat[of s' r s''] by blast
  have s5: "|Field s'| =o |Field s|" using s1 q3 lem_cardreleq_cardfldeq_inf[of s' s] by blast
  have "A1 \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 s1 s2 by blast
  moreover have "CCR (Restr r A')"
  proof -
    have "CCR s''" using s1 s2 s4 lem_Ccext_subccr_eqfld[of s' s''] by blast
    then show ?thesis using s3 by blast
  qed
  moreover have "|A'| =o |A1|"
  proof -
    have "Field s \<subseteq> A1" using q4 b4 by blast
    then have "|Field s| \<le>o |A1|" by simp
    then have "|A'| \<le>o |A1|" using s2 s5 ordIso_ordLeq_trans by blast
    moreover have "|A1| \<le>o |A'|" using s1 s2 by simp
    ultimately show ?thesis using ordIso_iff_ordLeq by blast
  qed
  ultimately have b6: "A1 \<union> ({x} \<inter> Field r) \<subseteq> A' \<and> CCR (Restr r A') \<and> |A'| =o
      |A1|" by blast
  moreover then have "A \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 by blast
  moreover have "|A'| =o |A|" using s5 s2 q4 by blast
  moreover have "\<forall>a\<in>A. r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}"
  proof
    fix a
    assume c1: "a \<in> A"
    have "\<not> (r``{a} \<subseteq> B) \<longrightarrow> r``{a} \<inter> (A'-B) \<noteq> {}"
    proof
      assume "\<not> (r``{a} \<subseteq> B)"
      then have "S a \<noteq> {}" unfolding b1 by blast
      then have "f a \<in> r``{a} - B" using b1 b3 by blast
      moreover have "f a \<in> A'" using c1 b4 b6 by blast
      ultimately show "r``{a} \<inter> (A'-B) \<noteq> {}" by blast
    qed
    then show "r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}" by blast
  qed
  moreover have "A' \<in> SF r" using s3 s4 unfolding SF_def by blast
  moreover have "(\<exists> y::'U. A' - B = {y}) \<longrightarrow> Field r \<subseteq> (A' \<union> B)"
  proof
    assume c1: "\<exists> y::'U. A' - B = {y}"
    moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
    ultimately have "Field r \<noteq> {}" by blast
    then have "{y1, y2} \<subseteq> Field r" using n1 by blast
    then have "{y1, y2} \<subseteq> A'" using b4 s1 s2 by fast
    then have "\<not> (\<exists>y. Field r - B \<subseteq> {y}) \<longrightarrow> {y1, y2} \<subseteq> A' - B \<
      and> y1 \<noteq> y2" using n2 by blast
    moreover have "\<not> ({y1, y2} \<subseteq> A' - B \<and> y1 \<noteq> y2)" using c1 by force
    ultimately have "\<exists> y::'U. Field r - B \<subseteq> {y}" by blast
    then show "Field r \<subseteq> A' \<union> B" using c1 c2 by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_subccr_pext5:
fixes r::"'U rel" and A B::"'U set" and x::'U
assumes "CCR r" and "A \<in> SF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A')
                    \<and> A \<subseteq> A'
                    \<and> A' \<in> SF r
                    \<and> (\<forall>a\<in>A. ((r``{a}\<subseteq>B) \<or> (r``{a}\<inter>(A'-B) \<noteq> {})))
                    \<and> ((\<exists> y::'U. A'-B = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B))
                    \<and> CCR (Restr r A')
                    \<and> ((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow>
       |A'| =o |A| ))"
proof (cases "finite A")
  assume "finite A"
  then show ?thesis using assms lem_Ccext_finsubccr_pext5[of r A x B] by blast
next
  assume "\<not> finite A"
  then show ?thesis using assms lem_Ccext_infsubccr_pext5[of r A x B] by blast
```

```
qed

lemma lem_Ccext_finsubccr_set_ext_scf:
fixes r s::"'U rel" and A P::"'U set"
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "finite s" and a4: "A \<subseteq> Field r" and a5: "finite
       A"
    and a6: "P \<in> SCF r"
shows "\<exists> s'::('U rel). CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> finite s' \<and> A
      \<subseteq> Field s'
                     \<and> ((Field s' \<inter> P) \<in> SCF s')"
proof (cases "s = {} \<and> A = {}")
  assume "s = {} \<and> A = {}"
  moreover obtain s'::"'U rel" where "s' = {}" by blast
  ultimately have "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> finite s' \<and> A \<subseteq>
      Field s'
                  \<and> ((Field s' \<inter> P) \<in> SCF s')" unfolding CCR_def SCF_def Field_def by blast
  then show ?thesis by blast
next
  assume b1: "\<not> (s = {} \<and> A = {})"
  obtain Pt::"'U \<Rightarrow> 'U rel" where p1: "Pt = (\<lambda> x. {p \<in> r. x = fst p \<or> x = snd p})" by
       blast
  obtain pt::"'U \<Rightarrow> 'U\<times>'U" where p2: "pt = (\<lambda> x. (SOME p. p \<in> Pt x))" by blast
  have "\<forall> x\<in>A. Pt x \<noteq> {}" using a4 unfolding p1 Field_def by force
  then have p3: "\<forall> x\<in>A. pt x \<in> Pt x" unfolding p2 by (metis (full_types) Collect_empty_eq
      Collect_mem_eq someI_ex)
  have b2: "pt`A \<subseteq> r" using p1 p3 by blast
  obtain s1 where b3: "s1 = s \<union> (pt`A)" by blast
  then have "finite s1" using a3 a5 by blast
  moreover have "s1 \<subseteq> r" using b2 b3 a2 by blast
  ultimately obtain s2 where b4: "finite s2 \<and> CCR s2 \<and> s1 \<subseteq> s2 \<and> s2 \<subseteq> r"
      using a1 lem_ccr_fin_subr_ext[of r s1] by blast
  moreover have "A \<subseteq> Field s1"
  proof
    fix x
    assume c1: "x \<in> A"
    then have "pt x \<in> s1" using b3 by blast
    moreover obtain ax bx where c2: "pt x = (ax,bx)" by force
    ultimately have "ax \<in> Field s1 \<and> bx \<in> Field s1" unfolding Field_def by force
    then show "x \<in> Field s1" using c1 c2 p1 p3 by force
  qed
  ultimately have b5: "A \<subseteq> Field s2" unfolding Field_def by blast
  have "Conelike s2" using b4 lem_Relprop_fin_ccr by blast
  moreover have "s2 \<noteq> {}" using b1 b3 b4 unfolding Field_def by blast
  ultimately obtain m where b6: "m \<in> Field s2 \<and> (\<forall> a\<in>Field s2. (a,m) \<in> s2^*)" unfolding
       Conelike_def by blast
  then have "m \<in> Field r" using b4 unfolding Field_def by blast
  then obtain m' where b7: "m' \<in> P \<and> (m,m') \<in> r^*" using a6 unfolding SCF_def by blast
  obtain D where b8: "D = Field s2 \<union> (\<ff> r m m')" by blast
  obtain s' where b9: "s' = Restr r D" by blast
  have b10: "s2 \<subseteq> s'" using b4 b8 b9 unfolding Field_def by force
  have b11: "\<forall> a \<in> Field s'. (a,m') \<in> s'^*"
  proof
    fix a
    assume c1: "a \<in> Field s'"
    have c2: "Restr r (\<ff> r m m') \<subseteq> s'" using b8 b9 by blast
    then have c3: "(m,m') \<in> s'^*" using b7 lem_Ccext_fint[of r m m' s'] by blast
    show "(a,m') \<in> s'^*"
    proof (cases "a \<in> Field s2")
      assume "a \<in> Field s2"
      then have "(a,m) \<in> s2^*" using b6 by blast
      then have "(a,m) \<in> s'^*" using b10 rtrancl_mono by blast
      then show "(a,m') \<in> s'^*" using c3 by simp
    next
      assume "a \<notin> Field s2"
      then have "a \<in> (\<ff> r m m')" using c1 b8 b9 unfolding Field_def by blast
      then show "(a,m') \<in> s'^*" using c2 b7 lem_Ccext_fint[of r m m' s'] by blast
    qed
  qed
  have b12: "m' \<in> Field s'"
  proof -
    have "m \<in> Field s'" using b6 b10 unfolding Field_def by blast
    then have "m \<in> Field s' \<and> (m,m') \<in> s'^*" using b11 by blast
    then show "m' \<in> Field s'" using lem_rtr_field by force
  qed
  have "Field s \<subseteq> D" using b3 b4 b8 unfolding Field_def by blast
  then have "s \<subseteq> s'" using a2 b9 unfolding Field_def by force
  moreover have "s' \<subseteq> r" using b9 by blast
  moreover have "finite s'"
```

```
  proof -
    have "finite (Field s2)" using b4 lem_fin_fl_rel by blast
    then have "finite D" using b8 lem_ccext_ffin by simp
    then show ?thesis using b9 by blast
  qed
  moreover have "A \<subseteq> Field s'" using b5 b10 unfolding Field_def by blast
  moreover have "CCR s'"
  proof -
    have "Conelike s'" using b11 b12 unfolding Conelike_def by blast
    then show ?thesis using lem_Relprop_cl_ccr by blast
  qed
  moreover have "(Field s' \<inter> P) \<in> SCF s'" using b7 b11 b12 unfolding SCF_def by blast
  ultimately show ?thesis by blast
qed

lemma lem_ccext_scf_sat:
assumes "s \<subseteq> r" and "Field s = Field r"
shows "SCF s \<subseteq> SCF r"
  using assms rtrancl_mono unfolding SCF_def by blast

lemma lem_Ccext_infsubccr_set_ext_scf2:
fixes r s::"'U rel" and A::"'U set" and Ps::"'U set set"
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "\<not> finite s" and a4: "A \<subseteq> Field r"
    and a5: "|A| \<le>o |Field s|" and a6: "Ps \<subseteq> SCF r \<and> |Ps| \<le>o |Field s|"
shows "\<exists> s'::('U rel). CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<and>
      A \<subseteq> Field s'
            \<and> (\<forall> P\<in>Ps. (Field s' \<inter> P) \<in> SCF s')"
proof -
  obtain q where q0: "q = (\<lambda> P a. SOME p. p \<in> P \<and> (a, p) \<in> r^*)" by blast
  have q1: "\<forall> P\<in>Ps. \<forall> a\<in>Field r. (q P a) \<in> Field r \<and> (q P a) \<in> P \<and> (a,
       q P a) \<in> r^*"
  proof (intro ballI)
    fix P a
    assume "P \<in> Ps" and "a \<in> Field r"
    then show "(q P a) \<in> Field r \<and> (q P a) \<in> P \<and> (a, q P a) \<in> r^*"
      using q0 a6 someI_ex[of "\<lambda> p. p \<in> P \<and> (a,p) \<in> r^*"] unfolding SCF_def by blast
  qed
  obtain G::"'U set \<Rightarrow> 'U rel set" where b1: "G = (\<lambda> A. {t::'U rel. finite t \<and> CCR t \<
      and> t \<subseteq> r \<and> A \<subseteq> Field t})" by blast
  obtain g::"'U set \<Rightarrow> 'U rel" where b2: "g = (\<lambda> A. if A \<subseteq> Field r \<and> finite A
      then (SOME t. t \<in> G A) else {})" by blast
  have b3: "\<forall> A. A \<subseteq> Field r \<and> finite A \<longrightarrow> finite (g A) \<and> CCR (g A)
      \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)"
  proof (intro allI impI)
    fix A
    assume c1: "A \<subseteq> Field r \<and> finite A"
    then have "g A = (SOME t. t \<in> G A)" using b2 by simp
    moreover have "G A \<noteq> {}" using b1 a1 c1 lem_Ccext_finsubccr_dext[of r A] by blast
    ultimately have "g A \<in> G A" using some_in_eq by metis
    then show "finite (g A) \<and> CCR (g A) \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)" using
      b1 by blast
  qed
  have b4: "\<forall> A. \<not> (A \<subseteq> Field r \<and> finite A) \<longrightarrow> g A = {}" using b2 by
      simp
  obtain H::"'U set \<Rightarrow> 'U set"
    where b5: "H = (\<lambda> X. X \<union> \<Union> {S . \<exists> a\<in>X. \<exists>b\<in>X. S = Field (g {a,b
      })} \<union> \<Union> {S. \<exists> P\<in>Ps. \<exists> a\<in>X. S = \<ff> r a (q P a) })" by blast
  obtain Pt::"'U \<Rightarrow> 'U rel" where p1: "Pt = (\<lambda> x. {p \<in> r. x = fst p \<or> x = snd p})" by
       blast
  obtain pt::"'U \<Rightarrow> 'U\<times>'U" where p2: "pt = (\<lambda> x. (SOME p. p \<in> Pt x))" by blast
  have "\<forall> x\<in>A. Pt x \<noteq> {}" using a4 unfolding p1 Field_def by force
  then have p3: "\<forall> x\<in>A. pt x \<in> Pt x" unfolding p2 by (metis (full_types) Collect_empty_eq
      Collect_mem_eq someI_ex)
  obtain D0 where b7: "D0 = Field s \<union> fst`(pt`A) \<union> snd`(pt`A)" by blast
  obtain Di::"nat \<Rightarrow> 'U set" where b8: "Di = (\<lambda> n. (H^^n) D0)" by blast
  obtain D::"'U set" where b9: "D = \<Union> {X. \<exists> n. X = Di n}" by blast
  obtain s' where b10: "s' = Restr r D" by blast
  have b11: "\<forall> n. (\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
  proof
    fix n0
    show "(\<not> finite (Di n0)) \<and> |Di n0| \<le>o |s|"
    proof (induct n0)
      have "|D0| =o |Field s|"
      proof -
        have "|fst`(pt`A)| \<le>o |(pt`A)| \<and> |(pt`A)| \<le>o |A|" by simp
        then have c1: "|fst`(pt`A)| \<le>o |A|" using ordLeq_transitive by blast
        have "|snd`(pt`A)| \<le>o |(pt`A)| \<and> |(pt`A)| \<le>o |A|" by simp
        then have c2: "|snd`(pt`A)| \<le>o |A|" using ordLeq_transitive by blast
```

```
      have "|fst`(pt`A)| \<le>o |Field s| \<and> |snd`(pt`A)| \<le>o |Field s|"
        using c1 c2 a5 ordLeq_transitive by blast
      moreover have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
      ultimately have c3: "|D0| \<le>o |Field s|" unfolding b7 by simp
      have "Field s \<subseteq> D0" unfolding b7 by blast
      then have "|Field s| \<le>o |D0|" by simp
      then show ?thesis using c3 ordIso_iff_ordLeq by blast
    qed
    moreover have "|Field s| =o |s|" using a3 lem_rel_inf_fld_card by blast
    ultimately have "|D0| \<le>o |s|" using ordIso_imp_ordLeq ordIso_transitive by blast
    moreover have "\<not> finite D0" using a3 b7 lem_fin_fl_rel by blast
    ultimately show "\<not> finite (Di 0) \<and> |Di 0| \<le>o |s|" using b8 by simp
  next
    fix n
    assume d1: "(\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
    moreover then have "|(Di n) \<times> (Di n)| =o |Di n|" by simp
    ultimately have d2: "|(Di n) \<times> (Di n)| \<le>o |s|" using ordIso_imp_ordLeq ordLeq_transitive by
     blast
    have d3: "\<forall> a \<in> (Di n). \<forall> b \<in> (Di n). |Field (g {a, b})| \<le>o |s|"
    proof (intro ballI)
      fix a b
      assume "a \<in> (Di n)" and "b \<in> (Di n)"
      have "finite (g {a, b})" using b3 b4 by (metis finite.emptyI)
      then have "finite (Field (g {a, b}))" using lem_fin_fl_rel by blast
      then have "|Field (g {a, b})| <o |s|" using a3 finite_ordLess_infinite2 by blast
      then show "|Field (g {a, b})| \<le>o |s|" using ordLess_imp_ordLeq by blast
    qed
    have d4: "Di (Suc n) = H (Di n)" using b8 by simp
    then have "Di n \<subseteq> Di (Suc n)" using b5 by blast
    then have "\<not> finite (Di (Suc n))" using d1 finite_subset by blast
    moreover have "|Di (Suc n)| \<le>o |s|"
    proof -
      obtain I where e1: "I = (Di n) \<times> (Di n)" by blast
      obtain f where e2: "f = (\<lambda> (a,b). Field (g {a,b}))" by blast
      have "|I| \<le>o |s|" using e1 d2 by blast
      moreover have "\<forall>i\<in>I. |f i| \<le>o |s|" using e1 e2 d3 by simp
      ultimately have "|\<Union> i\<in>I. f i| \<le>o |s|" using a3 card_of_UNION_ordLeq_infinite[of s I f] by
       blast
      moreover have "Di (Suc n) = (Di n) \<union> (\<Union> i\<in>I. f i) \<union> (\<Union> P\<in>Ps. (\<
    Union> a\<in>(Di n). \<ff> r a (q P a)))"
        using e1 e2 d4 b5 by blast
      moreover have "|\<Union> P\<in>Ps. (\<Union> a\<in>(Di n). \<ff> r a (q P a))| \<le>o |s|"
      proof -
        have "\<And> P. P \<in> Ps \<Longrightarrow> \<forall>a\<in>(Di n). |\<ff> r a (q P a)| \<le>o |s|"
          using a3 lem_ccext_ffin by (metis card_of_Well_order card_of_ordLeq_infinite ordLeq_total)
        then have "\<And> P. P \<in> Ps \<Longrightarrow> |\<Union> a\<in>(Di n). \<ff> r a (q P a)| \<le>o |s
    |"
          using d1 a3 card_of_UNION_ordLeq_infinite[of s "Di n" "\<lambda> a. \<ff> r a (q _ a)"] by blast
        moreover have "|Ps| \<le>o |s|" using a3 a6 lem_rel_inf_fld_card[of s] lem_fin_fl_rel[of s]
          by (metis ordIso_iff_ordLeq ordLeq_transitive)
        ultimately show ?thesis
          using a3 card_of_UNION_ordLeq_infinite[of s Ps "\<lambda> P. \<Union> a\<in>(Di n). \<ff> r a (q P a
    )"] by blast
      qed
      ultimately show ?thesis using d1 a3 by simp
    qed
    ultimately show "(\<not> finite (Di (Suc n))) \<and> |Di (Suc n)| \<le>o |s|" by blast
  qed
qed
have b12: "\<forall> m. \<forall> n. n \<le> m \<longrightarrow> Di n \<le> Di m"
proof
  fix m0
  show "\<forall> n. n \<le> m0 \<longrightarrow> Di n \<le> Di m0"
  proof (induct m0)
    show "\<forall>n\<le>0. Di n \<subseteq> Di 0" by blast
  next
    fix m
    assume d1: "\<forall>n\<le>m. Di n \<subseteq> Di m"
    show "\<forall>n\<le>Suc m. Di n \<subseteq> Di (Suc m)"
    proof (intro allI impI)
      fix n
      assume e1: "n \<le> Suc m"
      have "Di (Suc m) = H (Di m)" using b8 by simp
      moreover have "Di m \<subseteq> H (Di m)" using b5 by blast
      ultimately have "n \<le> m \<longrightarrow> Di n \<subseteq> Di (Suc m)" using d1 by blast
      moreover have "n = (Suc m) \<or> n \<le> m" using e1 by force
      ultimately show "Di n \<subseteq> Di (Suc m)" by blast
    qed
```

```
  qed
qed
have "Di 0 \<subseteq> D" using b9 by blast
then have b13: "Field s \<subseteq> D" using b7 b8 by simp
then have b14: "s \<subseteq> s' \<and> s' \<subseteq> r" using a2 b10 unfolding Field_def by force
moreover have b15: "|D| \<le>o |s|"
proof -
  have "|UNIV::nat set| \<le>o |s|" using a3 infinite_iff_card_of_nat by blast
  then have "|\<Union> n. Di n| \<le>o |s|" using b11 a3 card_of_UNION_ordLeq_infinite[of s UNIV Di] by blast
  moreover have "D = (\<Union> n. Di n)" using b9 by force
  ultimately show ?thesis by blast
qed
moreover have "|s'| =o |s|"
proof -
  have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
  then have "\<not> finite D" using b13 finite_subset by blast
  then have "|D \<times> D| =o |D|" by simp
  moreover have "s' \<subseteq> D \<times> D" using b10 by blast
  ultimately have "|s'| \<le>o |s|" using b15 card_of_mono1 ordLeq_ordIso_trans ordLeq_transitive by metis
  moreover have "|s| \<le>o |s'|" using b14 by simp
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed
moreover have "A \<subseteq> Field s'"
proof
  fix x
  assume c1: "x \<in> A"
  obtain ax bx where c2: "ax = fst (pt x) \<and> bx = snd (pt x)" by blast
  have "pt x \<in> Pt x" using c1 p3 by blast
  then have c3: "(ax, bx) \<in> r \<and> x \<in> {ax,bx}" using c2 p1 by simp
  have "{ax, bx} \<subseteq> D0" using b7 c1 c2 by blast
  moreover have "Di 0 \<subseteq> D" using b9 by blast
  moreover have "Di 0 = D0" using b8 by simp
  ultimately have "{ax, bx} \<subseteq> D" by blast
  then have "(ax, bx) \<in> s'" using c3 b10 by blast
  then show "x \<in> Field s'" using c3 unfolding Field_def by blast
qed
moreover have "CCR s'"
proof -
  have "\<forall> a \<in> Field s'. \<forall> b \<in> Field s'. \<exists> c \<in> Field s'. (a,c) \<in> (s')^*
     \<and> (b,c) \<in> (s')^*"
  proof (intro ballI)
    fix a b
    assume d1: "a \<in> Field s'" and d2: "b \<in> Field s'"
    then have d3: "a \<in> D \<and> b \<in> D" using b10 unfolding Field_def by blast
    then obtain ia ib where d4: "a \<in> Di ia \<and> b \<in> Di ib" using b9 by blast
    obtain k where d5: "k = (max ia ib)" by blast
    then have "ia \<le> k \<and> ib \<le> k" by simp
    then have d6: "a \<in> Di k \<and> b \<in> Di k" using d4 b12 by blast
    obtain p where d7: "p = g {a,b}" by blast
    have "Field p \<subseteq> H (Di k)" using b5 d6 d7 by blast
    moreover have "H (Di k) = Di (Suc k)" using b8 by simp
    moreover have "Di (Suc k) \<subseteq> D" using b9 by blast
    ultimately have d8: "Field p \<subseteq> D" by blast
    have "{a, b} \<subseteq> Field r" using d1 d2 b10 unfolding Field_def by blast
    moreover have "finite {a, b}" by simp
    ultimately have d9: "CCR p \<and> p \<subseteq> r \<and> {a,b} \<subseteq> Field p" using d7 b3 by blast
    then obtain c where d10: "c \<in> Field p \<and> (a,c) \<in> p^* \<and> (b,c) \<in> p^*" unfolding CCR_def
     by blast
    have "(p `` D) \<subseteq> D" using d8 unfolding Field_def by blast
    then have "D \<in> Inv p" unfolding Inv_def by blast
    then have "p^* \<inter> (D\<times>(UNIV::'U set)) \<subseteq> (Restr p D)^*" using lem_Inv_restr_rtr[of D
    p] by blast
    moreover have "Restr p D \<subseteq> s'" using d9 b10 by blast
    moreover have "(a,c) \<in> p^* \<inter> (D\<times>(UNIV::'U set)) \<and> (b,c) \<in> p^* \<inter> (D\<
    times>(UNIV::'U set))" using d10 d3 by blast
    ultimately have "(a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" using rtrancl_mono by blast
    moreover then have "c \<in> Field s'" using d1 lem_rtr_field by fastforce
    ultimately show "\<exists> c \<in> Field s'. (a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" by blast
  qed
  then show ?thesis unfolding CCR_def by blast
qed
moreover have "\<forall> P\<in>Ps. (Field s' \<inter> P) \<in> SCF s'"
proof -
  have "\<forall> P \<in> Ps. \<forall>a\<in>Field s'. \<exists>b\<in>(Field s' \<inter> P). (a, b) \<in> s
    '^*"
  proof (intro ballI)
    fix P a
    assume d0: "P \<in> Ps" and d1: "a \<in> Field s'"
```

```
      then have "a \<in> D" using b10 unfolding Field_def by blast
      then obtain n where "a \<in> Di n" using b9 by blast
      then have "\<ff> r a (q P a) \<subseteq> H (Di n)" using d0 b5 by blast
      moreover have "H (Di n) = Di (Suc n)" using b8 by simp
      ultimately have d2: "\<ff> r a (q P a) \<subseteq> D" using b9 by blast
      have "a \<in> Field r" using d1 b10 unfolding Field_def by blast
      then have "q P a \<in> P \<and> (a, q P a) \<in> r^*" using d0 q1 by blast
      moreover have "Restr r (\<ff> r a (q P a)) \<subseteq> s'" using d0 d2 b10 by blast
      ultimately have "q P a \<in> P \<and> (a, q P a) \<in> s'^*" using lem_Ccext_fint[of r a "q P a" s'] by
       blast
      moreover then have "q P a \<in> Field s'" using d1 lem_rtr_field by fastforce
      ultimately show "\<exists>b\<in>(Field s' \<inter> P). (a, b) \<in> s'^*" by blast
    qed
    then show ?thesis unfolding SCF_def by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_finsubccr_pext5_scf2:
fixes r::"'U rel" and A B B'::"'U set" and x::'U and Ps::"'U set set"
assumes a1: "CCR r" and a2: "finite A" and a3: "A \<in> SF r" and a4: "Ps \<subseteq> SCF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR
       (Restr r A') \<and> finite A'
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B')
      )
                    \<and> ((\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<
      in> SCF (Restr r A')))"
proof -
  obtain P where p0: "P = (if (Ps \<noteq> {}) then (SOME P. P \<in> Ps) else Field r)" by blast
  moreover have "Field r \<in> SCF r" unfolding SCF_def by blast
  ultimately have p1: "P \<in> SCF r" using a4 by (metis contra_subsetD some_in_eq)
  have p2: "(\<exists> P. Ps = {P}) \<longrightarrow> Ps = {P}" using p0 by fastforce
  have q1: "Field (Restr r A) = A" using a3 unfolding SF_def by blast
  obtain s where "s = (Restr r A)" by blast
  then have q2: "s \<subseteq> r" and q3: "finite s" and q4: "A = Field s"
    using a2 q1 lem_fin_fl_rel by (blast, metis, blast)
  obtain S where b1: "S = (\<lambda> a. r``{a} - B )" by blast
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. f a \<in> S' a" by (metis someI_ex)
  then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
      longrightarrow> f a = a)"
    unfolding b2 by (clarsimp, metis singletonD)
  obtain y1 y2::'U where n1: "Field r \<noteq> {} \<longrightarrow> {y1, y2} \<subseteq> Field r"
                    and n2: "(\<not> (\<exists> y::'U. Field r - B' \<subseteq> {y})) \<longrightarrow> y1 \<
      notin> B' \<and> y2 \<notin> B' \<and> y1 \<noteq> y2" by blast
  obtain A1 where b4: "A1 = ({x,y1,y2} \<inter> Field r) \<union> A \<union> (f ` A)" by blast
  have "A1 \<subseteq> Field r"
  proof -
    have c1: "A \<subseteq> Field r" using q4 q2 unfolding Field_def by blast
    moreover have "f ` A \<subseteq> Field r"
    proof
      fix x
      assume "x \<in> f ` A"
      then obtain a where d2: "a \<in> A \<and> x = f a" by blast
      show "x \<in> Field r"
      proof (cases "S a = {}")
        assume "S a = {}"
        then have "x = a" using c1 d2 b3 by blast
        then show "x \<in> Field r" using d2 c1 by blast
      next
        assume "S a \<noteq> {}"
        then have "x \<in> S a" using d2 b3 by blast
        then show "x \<in> Field r" using b1 unfolding Field_def by blast
      qed
    qed
    ultimately show "A1 \<subseteq> Field r" using b4 by blast
  qed
  moreover have s0: "finite A1" using b4 q3 q4 lem_fin_fl_rel by blast
  ultimately obtain s' where s1: "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> finite s' \<and>
       A1 \<subseteq> Field s'"
                         and s1': "(\<exists> P. Ps = {P}) \<longrightarrow> (Field s' \<inter> P) \<in> SCF s'"
    using p1 a1 a4 q2 q3 lem_Ccext_finsubccr_set_ext_scf[of r s A1 P] by metis
  obtain A' where s2: "A' = Field s'" by blast
  obtain s'' where s3: "s'' = Restr r A'" by blast
```

```
  then have s4: "s' \<subseteq> s'' \<and> Field s'' = A'" using s1 s2 lem_Relprop_fld_sat[of s' r s''] by blast
  have s5: "finite (Field s')" using s1 lem_fin_fl_rel by blast
  have "A1 \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 s1 s2 by blast
  moreover have "CCR (Restr r A')"
  proof -
    have "CCR s''" using s1 s2 s4 lem_Ccext_subccr_eqfld[of s' s''] by blast
    then show ?thesis using s3 by blast
  qed
  ultimately have b6: "A1 \<union> ({x} \<inter> Field r) \<subseteq> A' \<and> CCR (Restr r A')" by blast
  moreover then have "A \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 by blast
  ultimately have "(x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR (Restr r A
      ')" by blast
  moreover have "finite A'" using s2 s5 by blast
  moreover have "\<forall>a\<in>A. r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}"
  proof
    fix a
    assume c1: "a \<in> A"
    have "\<not> (r``{a} \<subseteq> B) \<longrightarrow> r``{a} \<inter> (A'-B) \<noteq> {}"
    proof
      assume "\<not> (r``{a} \<subseteq> B)"
      then have "S a \<noteq> {}" unfolding b1 by blast
      then have "f a \<in> r``{a} - B" using b1 b3 by blast
      moreover have "f a \<in> A'" using c1 b4 b6 by blast
      ultimately show "r``{a} \<inter> (A'-B) \<noteq> {}" by blast
    qed
    then show "r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}" by blast
  qed
  moreover have "A' \<in> SF r" using s3 s4 unfolding SF_def by blast
  moreover have "(\<exists> y::'U. A' - B' = {y}) \<longrightarrow> Field r \<subseteq> (A' \<union> B')"
  proof
    assume c1: "\<exists> y::'U. A' - B' = {y}"
    moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
    ultimately have "Field r \<noteq> {}" by blast
    then have "{y1, y2} \<subseteq> Field r" using n1 by blast
    then have "{y1, y2} \<subseteq> A'" using b4 s1 s2 by fast
    then have "\<not> (\<exists>y. Field r - B' \<subseteq> {y}) \<longrightarrow> {y1, y2} \<subseteq> A' - B'
      \<and> y1 \<noteq> y2" using n2 by blast
    moreover have "\<not> ({y1, y2} \<subseteq> A' - B' \<and> y1 \<noteq> y2)" using c1 by force
    ultimately have "\<exists> y::'U. Field r - B' \<subseteq> {y}" by blast
    then show "Field r \<subseteq> A' \<union> B'" using c1 c2 by blast
  qed
  moreover have "(\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (
      Restr r A'))"
  proof -
    have c1: "s' \<subseteq> r" using s3 s4 by blast
    then have "Field s' = Field (Restr r (Field s'))" using lem_Relprop_fld_sat by blast
    moreover have "s' \<subseteq> Restr r (Field s')" using c1 unfolding Field_def by force
    ultimately have "SCF s' \<subseteq> SCF (Restr r (Field s'))" using lem_ccext_scf_sat[of s' "Restr r (Field
      s')"] by blast
    then show ?thesis using p2 s1' s2 by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_infsubccr_pext5_scf2:
fixes r::"'U rel" and A B B'::"'U set" and x::'U and Ps::"'U set set"
assumes a1: "CCR r" and a2: "\<not> finite A" and a3: "A \<in> SF r" and a4: "Ps \<subseteq> SCF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR
       (Restr r A') \<and> |A'| =o |A|
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B')
      )
                    \<and> ( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF
       (Restr r A')) )"
proof -
  obtain Ps' where p0: "Ps' = (if ( |Ps| \<le>o |A| ) then Ps else {})" by blast
  then have p1: "Ps' \<subseteq> SCF r \<and> |Ps'| \<le>o |A|" using a4 by simp
  have q1: "Field (Restr r A) = A" using a3 unfolding SF_def by blast
  obtain s where "s = (Restr r A)" by blast
  then have q2: "s \<subseteq> r" and q3: "\<not> finite s" and q4: "A = Field s"
    using a2 q1 lem_fin_fl_rel by (blast, metis, blast)
  obtain S where b1: "S = (\<lambda> a. r``{a} - B )" by blast
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. f a \<in> S' a" by (metis someI_ex)
```

```
then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
    longrightarrow> f a = a)"
  unfolding b2 by (clarsimp, metis singletonD)
obtain y1 y2::'U where n1: "Field r \<noteq> {} \<longrightarrow> {y1, y2} \<subseteq> Field r"
                   and n2: "(\<not> (\<exists> y::'U. Field r - B' \<subseteq> {y})) \<longrightarrow> y1 \<
    notin> B' \<and> y2 \<notin> B' \<and> y1 \<noteq> y2" by blast
obtain A1 where b4: "A1 = ({x, y1, y2} \<inter> Field r) \<union> A \<union> (f ` A)" by blast
have "A1 \<subseteq> Field r"
proof -
  have c1: "A \<subseteq> Field r" using q4 q2 unfolding Field_def by blast
  moreover have "f ` A \<subseteq> Field r"
  proof
    fix x
    assume "x \<in> f ` A"
    then obtain a where d2: "a \<in> A \<and> x = f a" by blast
    show "x \<in> Field r"
    proof (cases "S a = {}")
      assume "S a = {}"
      then have "x = a" using c1 d2 b3 by blast
      then show "x \<in> Field r" using d2 c1 by blast
    next
      assume "S a \<noteq> {}"
      then have "x \<in> S a" using d2 b3 by blast
      then show "x \<in> Field r" using b1 unfolding Field_def by blast
    qed
  qed
  ultimately show "A1 \<subseteq> Field r" using b4 by blast
qed
moreover have s0: "|A1| \<le>o |Field s|"
proof -
  obtain C1 where c1: "C1 = {x,y1,y2} \<inter> Field r" by blast
  obtain C2 where c2: "C2 = A \<union> f ` A" by blast
  have "\<not> finite A" using q4 q3 lem_fin_fl_rel by blast
  then have "|C2| =o |A|" using c2 b4 q3 by simp
  then have "|C2| \<le>o |Field s|" unfolding q4 using ordIso_iff_ordLeq by blast
  moreover have c3: "\<not> finite (Field s)" using q3 lem_fin_fl_rel by blast
  moreover have "|C1| \<le>o |Field s|"
  proof -
    have "|{x,y1,y2}| \<le>o |Field s|" using c3
      by (meson card_of_Well_order card_of_ordLeq_finite finite.emptyI finite.insertI ordLeq_total)
    moreover have "|C1| \<le>o |{x,y1,y2}|" unfolding c1 by simp
    ultimately show ?thesis using ordLeq_transitive by blast
  qed
  ultimately have "|C1 \<union> C2| \<le>o |Field s|" unfolding b4 using card_of_Un_ordLeq_infinite by blast
  moreover have "A1 = C1 \<union> C2" using c1 c2 b4 by blast
  ultimately show ?thesis by blast
qed
ultimately obtain s' where s1: "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<
    and> A1 \<subseteq> Field s'"
                       and s1': "(\<forall> P \<in> Ps'. (Field s' \<inter> P) \<in> SCF s')"
  using p1 a1 q2 q3 q4 lem_Ccext_infsubccr_set_ext_scf2[of r s A1 Ps'] by blast
obtain A' where s2: "A' = Field s'" by blast
obtain s'' where s3: "s'' = Restr r A'" by blast
then have s4: "s' \<subseteq> s'' \<and> Field s'' = A'" using s1 s2 lem_Relprop_fld_sat[of s' r s''] by blast
have s5: "|Field s'| =o |Field s|" using s1 q3 lem_cardreleq_cardfldeq_inf[of s' s] by blast
have "A1 \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 s1 s2 by blast
moreover have "CCR (Restr r A')"
proof -
  have "CCR s''" using s1 s2 s4 lem_Ccext_subccr_eqfld[of s' s''] by blast
  then show ?thesis using s3 by blast
qed
moreover have "|A'| =o |A1|"
proof -
  have "Field s \<subseteq> A1" using q4 b4 by blast
  then have "|Field s| \<le>o |A1|" by simp
  then have "|A'| \<le>o |A1|" using s2 s5 ordIso_ordLeq_trans by blast
  moreover have "|A1| \<le>o |A'|" using s1 s2 by simp
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed
ultimately have b6: "A1 \<union> ({x} \<inter> Field r) \<subseteq> A' \<and> CCR (Restr r A') \<and> |A'| =o
    |A1|" by blast
moreover then have "A \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 by blast
moreover have "|A'| =o |A|" using s5 s2 q4 by blast
moreover have "\<forall>a\<in>A. r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}"
proof
  fix a
  assume c1: "a \<in> A"
  have "\<not> (r``{a} \<subseteq> B) \<longrightarrow> r``{a} \<inter> (A'-B) \<noteq> {}"
```

```
    proof
      assume "\<not> (r``{a} \<subseteq> B)"
      then have "S a \<noteq> {}" unfolding b1 by blast
      then have "f a \<in> r``{a} - B" using b1 b3 by blast
      moreover have "f a \<in> A'" using c1 b4 b6 by blast
      ultimately show "r``{a} \<inter> (A'-B) \<noteq> {}" by blast
    qed
    then show "r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}" by blast
  qed
  moreover have "A' \<in> SF r" using s3 s4 unfolding SF_def by blast
  moreover have "(\<exists> y::'U. A' - B' = {y}) \<longrightarrow> Field r \<subseteq> (A' \<union> B')"
  proof
    assume c1: "\<exists> y::'U. A' - B' = {y}"
    moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
    ultimately have "Field r \<noteq> {}" by blast
    then have "{y1, y2} \<subseteq> Field r" using n1 by blast
    then have "{y1, y2} \<subseteq> A'" using b4 s1 s2 by fast
    then have "\<not> (\<exists>y. Field r - B' \<subseteq> {y}) \<longrightarrow> {y1, y2} \<subseteq> A' - B'
      \<and> y1 \<noteq> y2" using n2 by blast
    moreover have "\<not> ({y1, y2} \<subseteq> A' - B' \<and> y1 \<noteq> y2)" using c1 by force
    ultimately have "\<exists> y::'U. Field r - B' \<subseteq> {y}" by blast
    then show "Field r \<subseteq> A' \<union> B'" using c1 c2 by blast
  qed
  moreover have "( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A
      )) )"
  proof -
    have c1: "s' \<subseteq> r" using s3 s4 by blast
    then have "Field s' = Field (Restr r (Field s'))" using lem_Relprop_fld_sat by blast
    moreover have "s' \<subseteq> Restr r (Field s')" using c1 unfolding Field_def by force
    ultimately have "SCF s' \<subseteq> SCF (Restr r (Field s'))" using lem_ccext_scf_sat[of s' "Restr r (Field
      s')"] by blast
    moreover have "|Ps| \<le>o |A| \<longrightarrow> Ps' = Ps" using p0 by simp
    ultimately show ?thesis using s1' s2 by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_subccr_pext5_scf2:
fixes r::"'U rel" and A B B'::"'U set" and x::'U and Ps::"'U set set"
assumes "CCR r" and "A \<in> SF r" and "Ps \<subseteq> SCF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A')
                    \<and> A \<subseteq> A'
                    \<and> A' \<in> SF r
                    \<and> (\<forall>a\<in>A. ((r``{a}\<subseteq>B) \<or> (r``{a}\<inter>(A'-B) \<noteq> {})))
                    \<and> ((\<exists> y::'U. A'-B' = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B')
      )
                    \<and> CCR (Restr r A')
                    \<and> ((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow>
       |A'| =o |A| ))
                    \<and> ( ((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| )) \<
      longrightarrow>
                        (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A')))"
proof (cases "finite A")
  assume b1: "finite A"
  then obtain A'::"'U set" where b2: "(x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<
      and> CCR (Restr r A')
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B')
      )"
                    and b3: "finite A' \<and> ((\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps.
       (A' \<inter> P) \<in> SCF (Restr r A')))"
                    using assms lem_Ccext_finsubccr_pext5_scf2[of r A Ps x B B'] by metis
  have b4: "((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow> |A'| =o |A| ))"
   and b5: "( ((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| )) \<longrightarrow> (\<
      forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A')))"
       using b1 b3 card_of_ordLeq_finite by blast+
  show ?thesis apply (rule exI) using b2 b4 b5 by force
next
  assume b1: "\<not> finite A"
  then obtain A' where b2: "(x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR (
      Restr r A')
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' = {y}) \<longrightarrow> Field r \<subseteq> (A'\<union>B')
      )"
             and b3: "|A'| =o |A| \<and> ( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter
      > P) \<in> SCF (Restr r A')) )"
```

```
    using assms lem_Ccext_infsubccr_pext5_scf2[of r A Ps x B B'] by metis
  have b4: "((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow> |A'| =o |A| ))"
    using b1 b3 by metis
  have b5: "( ((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| )) \<longrightarrow> (\<
      forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A')))"
    using b1 b3 by (metis card_of_singl_ordLeq finite.simps)
  show ?thesis apply (rule exI) using b2 b4 b5 by force
qed

lemma lem_dnEsc_el: "F \<in> dnEsc r A a \<Longrightarrow> a \<in> F \<and> finite F" unfolding dnEsc_def \<F>
      _def rpth_def by blast

lemma lem_dnEsc_emp: "dnEsc r A a = {} \<Longrightarrow> dnesc r A a = { a }" unfolding dnesc_def by simp

lemma lem_dnEsc_ne: "dnEsc r A a \<noteq> {} \<Longrightarrow> dnesc r A a \<in> dnEsc r A a"
  unfolding dnesc_def using someI_ex[of "\<lambda> F. F \<in> dnEsc r A a"] by force

lemma lem_dnesc_in: "a \<in> dnesc r A a \<and> finite (dnesc r A a)"
  using lem_dnEsc_emp[of r A a] lem_dnEsc_el[of _ r A a] lem_dnEsc_ne[of r A a] by force

lemma lem_escl_incr: "B \<subseteq> escl r A B" using lem_dnesc_in[of _ r A] unfolding escl_def by blast

lemma lem_escl_card: "(finite B \<longrightarrow> finite (escl r A B)) \<and> (\<not> finite B \<longrightarrow>
       |escl r A B| \<le>o |B| )"
proof (intro conjI impI)
  assume "finite B"
  then show "finite (escl r A B)" using lem_dnesc_in[of _ r A] unfolding escl_def by blast
next
  assume b1: "\<not> finite B"
  moreover have "escl r A B = (\<Union>x\<in>B. ((dnesc r A) x))" unfolding escl_def by blast
  moreover have "\<forall> x. |(dnesc r A) x| \<le>o |B|"
  proof
    fix x
    have "finite (dnesc r A x)" using lem_dnesc_in[of _ r A] by blast
    then show "|dnesc r A x| \<le>o |B|" using b1 by (meson card_of_Well_order card_of_ordLeq_infinite
      ordLeq_total)
  qed
  ultimately show "|escl r A B| \<le>o |B|" by (simp add: card_of_UNION_ordLeq_infinite)
qed

lemma lem_Ccext_infsubccr_set_ext_scf3:
fixes r s::"'U rel" and A A0::"'U set" and Ps::"'U set set"
assumes a1: "CCR r" and a2: "s \<subseteq> r" and a3: "\<not> finite s" and a4: "A \<subseteq> Field r"
    and a5: "|A| \<le>o |Field s|" and a6: "Ps \<subseteq> SCF r \<and> |Ps| \<le>o |Field s|"
shows "\<exists> s'::('U rel). CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<and>
      A \<subseteq> Field s'
            \<and> (\<forall> P\<in>Ps. (Field s' \<inter> P) \<in> SCF s') \<and> (escl r A0 (Field s') \<
      subseteq> Field s')
            \<and> (\<exists> D. s' = Restr r D) \<and> (Conelike s' \<longrightarrow> Conelike r)"
proof -
  obtain w where w0: "w = (\<lambda> x. SOME y. y \<in> Field r - dncl r {x})" by blast
  have w1: "\<And> x. Field r - dncl r {x} \<noteq> {} \<Longrightarrow> w x \<in> Field r - dncl r {x}"
  proof -
    fix x
    assume "Field r - dncl r {x} \<noteq> {}"
    then show "w x \<in> Field r - dncl r {x}"
      using w0 someI_ex[of "\<lambda> y. y \<in> Field r - dncl r {x}"] by force
  qed
  obtain q where q0: "q = (\<lambda> P a. SOME p. p \<in> P \<and> (a, p) \<in> r^*)" by blast
  have q1: "\<forall> P\<in>Ps. \<forall> a\<in>Field r. (q P a) \<in> Field r \<and> (q P a) \<in> P \<and> (a,
       q P a) \<in> r^*"
  proof (intro ballI)
    fix P a
    assume "P \<in> Ps" and "a \<in> Field r"
    then show "(q P a) \<in> Field r \<and> (q P a) \<in> P \<and> (a, q P a) \<in> r^*"
      using q0 a6 someI_ex[of "\<lambda> p. p \<in> P \<and> (a,p) \<in> r^*"] unfolding SCF_def by blast
  qed
  obtain G::"'U set \<Rightarrow> 'U rel set" where b1: "G = (\<lambda> A. {t::'U rel. finite t \<and> CCR t \<
      and> t \<subseteq> r \<and> A \<subseteq> Field t})" by blast
  obtain g::"'U set \<Rightarrow> 'U rel" where b2: "g = (\<lambda> A. if A \<subseteq> Field r \<and> finite A
      then (SOME t. t \<in> G A) else {})" by blast
  have b3: "\<forall> A. A \<subseteq> Field r \<and> finite A \<longrightarrow> finite (g A) \<and> CCR (g A)
      \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)"
  proof (intro allI impI)
    fix A
    assume c1: "A \<subseteq> Field r \<and> finite A"
    then have "g A = (SOME t. t \<in> G A)" using b2 by simp
    moreover have "G A \<noteq> {}" using b1 a1 c1 lem_Ccext_finsubccr_dext[of r A] by blast
```

```
  ultimately have "g A \<in> G A" using some_in_eq by metis
  then show "finite (g A) \<and> CCR (g A) \<and> (g A) \<subseteq> r \<and> A \<subseteq> Field (g A)" using
    b1 by blast
qed
have b4: "\<forall> A. \<not> (A \<subseteq> Field r \<and> finite A) \<longrightarrow> g A = {}" using b2 by
    simp
obtain H::"'U set \<Rightarrow> 'U set"
  where b5: "H = (\<lambda> X. X \<union> \<Union> {S. \<exists> a\<in>X. \<exists>b\<in>X. S = Field (g {a,b
    })}
                        \<union> \<Union> {S. \<exists> P\<in>Ps. \<exists> a\<in>X. S = \<ff> r a (q P a) }
                        \<union> escl r A0 X \<union> (w`X) )" by blast

obtain Pt::"'U \<Rightarrow> 'U rel" where p1: "Pt = (\<lambda> x. {p \<in> r. x = fst p \<or> x = snd p})" by
     blast
obtain pt::"'U \<Rightarrow> 'U\<times>'U" where p2: "pt = (\<lambda> x. (SOME p. p \<in> Pt x))" by blast
have "\<forall> x\<in>A. Pt x \<noteq> {}" using a4 unfolding p1 Field_def by force
then have p3: "\<forall> x\<in>A. pt x \<in> Pt x" unfolding p2 by (metis (full_types) Collect_empty_eq
    Collect_mem_eq someI_ex)
obtain D0 where b7: "D0 = Field s \<union> fst`(pt`A) \<union> snd`(pt`A)" by blast
obtain Di::"nat \<Rightarrow> 'U set" where b8: "Di = (\<lambda> n. (H^^n) D0)" by blast
obtain D::"'U set" where b9: "D = \<Union> {X. \<exists> n. X = Di n}" by blast
obtain s' where b10: "s' = Restr r D" by blast
have b11: "\<forall> n. (\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
proof
  fix n0
  show "(\<not> finite (Di n0)) \<and> |Di n0| \<le>o |s|"
  proof (induct n0)
    have "|D0| =o |Field s|"
    proof -
      have "|fst`(pt`A)| \<le>o |(pt`A)| \<and> |(pt`A)| \<le>o |A|" by simp
      then have c1: "|fst`(pt`A)| \<le>o |A|" using ordLeq_transitive by blast
      have "|snd`(pt`A)| \<le>o |(pt`A)| \<and> |(pt`A)| \<le>o |A|" by simp
      then have c2: "|snd`(pt`A)| \<le>o |A|" using ordLeq_transitive by blast
      have "|fst`(pt`A)| \<le>o |Field s| \<and> |snd`(pt`A)| \<le>o |Field s|"
        using c1 c2 a5 ordLeq_transitive by blast
      moreover have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
      ultimately have c3: "|D0| \<le>o |Field s|" unfolding b7 by simp
      have "Field s \<subseteq> D0" unfolding b7 by blast
      then have "|Field s| \<le>o |D0|" by simp
      then show ?thesis using c3 ordIso_iff_ordLeq by blast
    qed
    moreover have "|Field s| =o |s|" using a3 lem_rel_inf_fld_card by blast
    ultimately have "|D0| \<le>o |s|" using ordIso_imp_ordLeq ordIso_transitive by blast
    moreover have "\<not> finite D0" using a3 b7 lem_fin_fl_rel by blast
    ultimately show "\<not> finite (Di 0) \<and> |Di 0| \<le>o |s|" using b8 by simp
  next
    fix n
    assume d1: "(\<not> finite (Di n)) \<and> |Di n| \<le>o |s|"
    moreover then have "|(Di n) \<times> (Di n)| =o |Di n|" by simp
    ultimately have d2: "|(Di n) \<times> (Di n)| \<le>o |s|" using ordIso_imp_ordLeq ordLeq_transitive by
    blast
    have d3: "\<forall> a \<in> (Di n). \<forall> b \<in> (Di n). |Field (g {a, b})| \<le>o |s|"
    proof (intro ballI)
      fix a b
      assume "a \<in> (Di n)" and "b \<in> (Di n)"
      have "finite (g {a, b})" using b3 b4 by (metis finite.emptyI)
      then have "finite (Field (g {a, b}))" using lem_fin_fl_rel by blast
      then have "|Field (g {a, b})| <o |s|" using a3 finite_ordLess_infinite2 by blast
      then show "|Field (g {a, b})| \<le>o |s|" using ordLess_imp_ordLeq by blast
    qed
    have d4: "Di (Suc n) = H (Di n)" using b8 by simp
    then have "Di n \<subseteq> Di (Suc n)" using b5 by blast
    then have "\<not> finite (Di (Suc n))" using d1 finite_subset by blast
    moreover have "|Di (Suc n)| \<le>o |s|"
    proof -
      obtain I where e1: "I = (Di n) \<times> (Di n)" by blast
      obtain f where e2: "f = (\<lambda> (a,b). Field (g {a,b}))" by blast
      have "|I| \<le>o |s|" using e1 d2 by blast
      moreover have "\<forall>i\<in>I. |f i| \<le>o |s|" using e1 e2 d3 by simp
      ultimately have "|\<Union> i\<in>I. f i| \<le>o |s|" using a3 card_of_UNION_ordLeq_infinite[of s I f] by
     blast
      moreover have "Di (Suc n) = (Di n) \<union> (\<Union> i\<in>I. f i)
          \<union> (\<Union> P\<in>Ps. (\<Union> a\<in>(Di n). \<ff> r a (q P a))) \<union> escl r A0 (Di n)
    \<union> (w`(Di n))"
        using e1 e2 d4 b5 by blast
      moreover have "|\<Union> P\<in>Ps. (\<Union> a\<in>(Di n). \<ff> r a (q P a))| \<le>o |s|"
      proof -
        have "\<And> P. P \<in> Ps \<Longrightarrow> \<forall>a\<in>(Di n). |\<ff> r a (q P a)| \<le>o |s|"
```

```
          using a3 lem_ccext_ffin by (metis card_of_Well_order card_of_ordLeq_infinite ordLeq_total)
        then have "\<And> P. P \<in> Ps \<Longrightarrow> |\<Union> a\<in>(Di n). \<ff> r a (q P a)| \<le>o |s
    |"
          using d1 a3 card_of_UNION_ordLeq_infinite[of s "Di n" "\<lambda> a. \<ff> r a (q _ a)"] by blast
        moreover have "|Ps| \<le>o |s|" using a3 a6 lem_rel_inf_fld_card[of s] lem_fin_fl_rel[of s]
          by (metis ordIso_iff_ordLeq ordLeq_transitive)
        ultimately show ?thesis
          using a3 card_of_UNION_ordLeq_infinite[of s Ps "\<lambda> P. \<Union> a\<in>(Di n). \<ff> r a (q P a
    )"] by blast
      qed
      moreover have "|escl r A0 (Di n)| \<le>o |s|"
        using d1 lem_escl_card[of "Di n" r A0] by (metis ordLeq_transitive)
      moreover have "|w`(Di n)| \<le>o |s|" using d1 using card_of_image ordLeq_transitive by blast
      ultimately show ?thesis using d1 a3 by simp
    qed
    ultimately show "(\<not> finite (Di (Suc n))) \<and> |Di (Suc n)| \<le>o |s|" by blast
  qed
qed
have b12: "\<forall> m. \<forall> n. n \<le> m \<longrightarrow> Di n \<le> Di m"
proof
  fix m0
  show "\<forall> n. n \<le> m0 \<longrightarrow> Di n \<le> Di m0"
  proof (induct m0)
    show "\<forall>n\<le>0. Di n \<subseteq> Di 0" by blast
  next
    fix m
    assume d1: "\<forall>n\<le>m. Di n \<subseteq> Di m"
    show "\<forall>n\<le>Suc m. Di n \<subseteq> Di (Suc m)"
    proof (intro allI impI)
      fix n
      assume e1: "n \<le> Suc m"
      have "Di (Suc m) = H (Di m)" using b8 by simp
      moreover have "Di m \<subseteq> H (Di m)" using b5 by blast
      ultimately have "n \<le> m \<longrightarrow> Di n \<subseteq> Di (Suc m)" using d1 by blast
      moreover have "n = (Suc m) \<or> n \<le> m" using e1 by force
      ultimately show "Di n \<subseteq> Di (Suc m)" by blast
    qed
  qed
qed
have "Di 0 \<subseteq> D" using b9 by blast
then have b13: "Field s \<subseteq> D" using b7 b8 by simp
then have b14: "s \<subseteq> s' \<and> s' \<subseteq> r" using a2 b10 unfolding Field_def by force
moreover have b15: "|D| \<le>o |s|"
proof -
  have "|UNIV::nat set| \<le>o |s|" using a3 infinite_iff_card_of_nat by blast
  then have "|\<Union> n. Di n| \<le>o |s|" using b11 a3 card_of_UNION_ordLeq_infinite[of s UNIV Di] by blast
  moreover have "D = (\<Union> n. Di n)" using b9 by force
  ultimately show ?thesis by blast
qed
moreover have "|s'| =o |s|"
proof -
  have "\<not> finite (Field s)" using a3 lem_fin_fl_rel by blast
  then have "\<not> finite D" using b13 finite_subset by blast
  then have "|D \<times> D| =o |D|" by simp
  moreover have "s' \<subseteq> D \<times> D" using b10 by blast
  ultimately have "|s'| \<le>o |s|" using b15 card_of_mono1 ordLeq_ordIso_trans ordLeq_transitive by metis
  moreover have "|s| \<le>o |s'|" using b14 by simp
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed
moreover have "A \<subseteq> Field s'"
proof
  fix x
  assume c1: "x \<in> A"
  obtain ax bx where c2: "ax = fst (pt x) \<and> bx = snd (pt x)" by blast
  have "pt x \<in> Pt x" using c1 p3 by blast
  then have c3: "(ax, bx) \<in> r \<and> x \<in> {ax,bx}" using c2 p1 by simp
  have "{ax, bx} \<subseteq> D0" using b7 c1 c2 by blast
  moreover have "Di 0 \<subseteq> D" using b9 by blast
  moreover have "Di 0 = D0" using b8 by simp
  ultimately have "{ax, bx} \<subseteq> D" by blast
  then have "(ax, bx) \<in> s'" using c3 b10 by blast
  then show "x \<in> Field s'" using c3 unfolding Field_def by blast
qed
moreover have "CCR s'"
proof -
  have "\<forall> a \<in> Field s'. \<forall> b \<in> Field s'. \<exists> c \<in> Field s'. (a,c) \<in> (s')^*
     \<and> (b,c) \<in> (s')^*"
  proof (intro ballI)
```

```
    fix a b
    assume d1: "a \<in> Field s'" and d2: "b \<in> Field s'"
    then have d3: "a \<in> D \<and> b \<in> D" using b10 unfolding Field_def by blast
    then obtain ia ib where d4: "a \<in> Di ia \<and> b \<in> Di ib" using b9 by blast
    obtain k where d5: "k = (max ia ib)" by blast
    then have "ia \<le> k \<and> ib \<le> k" by simp
    then have d6: "a \<in> Di k \<and> b \<in> Di k" using d4 b12 by blast
    obtain p where d7: "p = g {a,b}" by blast
    have "Field p \<subseteq> H (Di k)" using b5 d6 d7 by blast
    moreover have "H (Di k) = Di (Suc k)" using b8 by simp
    moreover have "Di (Suc k) \<subseteq> D" using b9 by blast
    ultimately have d8: "Field p \<subseteq> D" by blast
    have "{a, b} \<subseteq> Field r" using d1 d2 b10 unfolding Field_def by blast
    moreover have "finite {a, b}" by simp
    ultimately have d9: "CCR p \<and> p \<subseteq> r \<and> {a,b} \<subseteq> Field p" using d7 b3 by blast
    then obtain c where d10: "c \<in> Field p \<and> (a,c) \<in> p^* \<and> (b,c) \<in> p^*" unfolding CCR_def
     by blast
    have "(p `` D) \<subseteq> D" using d8 unfolding Field_def by blast
    then have "D \<in> Inv p" unfolding Inv_def by blast
    then have "p^* \<inter> (D\<times>(UNIV::'U set)) \<subseteq> (Restr p D)^*" using lem_Inv_restr_rtr[of D
    p] by blast
    moreover have "Restr p D \<subseteq> s'" using d9 b10 by blast
    moreover have "(a,c) \<in> p^* \<inter> (D\<times>(UNIV::'U set)) \<and> (b,c) \<in> p^* \<inter> (D\<
    times>(UNIV::'U set))" using d10 d3 by blast
    ultimately have "(a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" using rtrancl_mono by blast
    moreover then have "c \<in> Field s'" using d1 lem_rtr_field by fastforce
    ultimately show "\<exists> c \<in> Field s'. (a,c) \<in> (s')^* \<and> (b,c) \<in> (s')^*" by blast
  qed
  then show ?thesis unfolding CCR_def by blast
qed
moreover have "\<forall> P\<in>Ps. (Field s' \<inter> P) \<in> SCF s'"
proof -
  have "\<forall> P \<in> Ps. \<forall>a\<in>Field s'. \<exists>b\<in>(Field s' \<inter> P). (a, b) \<in> s
    '^*"
  proof (intro ballI)
    fix P a
    assume d0: "P \<in> Ps" and d1: "a \<in> Field s'"
    then have "a \<in> D" using b10 unfolding Field_def by blast
    then obtain n where "a \<in> Di n" using b9 by blast
    then have "\<ff> r a (q P a) \<subseteq> H (Di n)" using d0 b5 by blast
    moreover have "H (Di n) = Di (Suc n)" using b8 by simp
    ultimately have d2: "\<ff> r a (q P a) \<subseteq> D" using b9 by blast
    have "a \<in> Field r" using d1 b10 unfolding Field_def by blast
    then have "q P a \<in> P \<and> (a, q P a) \<in> r^*" using d0 q1 by blast
    moreover have "Restr r (\<ff> r a (q P a)) \<subseteq> s'" using d0 d2 b10 by blast
    ultimately have "q P a \<in> P \<and> (a, q P a) \<in> s'^*" using lem_Ccext_fint[of r a "q P a" s'] by
    blast
    moreover then have "q P a \<in> Field s'" using d1 lem_rtr_field by fastforce
    ultimately show "\<exists>b\<in>(Field s' \<inter> P). (a, b) \<in> s'^*" by blast
  qed
  then show ?thesis unfolding SCF_def by blast
qed
moreover have "escl r A0 (Field s') \<subseteq> Field s'"
proof
  fix x
  assume c1: "x \<in> escl r A0 (Field s')"
  then obtain F a where c2: "x \<in> F \<and> F = dnesc r A0 a \<and> a \<in> Field s'" unfolding escl_def by
    blast
  obtain n where "a \<in> Di n" using c2 b9 b10 unfolding Field_def by blast
  then have "F \<subseteq> H (Di n)" using c2 b5 unfolding escl_def by blast
  moreover have "H (Di n) = Di (Suc n)" using b8 b9 by simp
  ultimately have c3: "F \<subseteq> D" using b9 by blast
  show "x \<in> Field s'"
  proof (cases "dnEsc r A0 a = {}")
    assume "dnEsc r A0 a = {}"
    then have "x = a" using c2 lem_dnEsc_emp[of r A0] by blast
    then show ?thesis using c2 by blast
  next
    assume "dnEsc r A0 a \<noteq> {}"
    then have "F \<in> dnEsc r A0 a" using c2 lem_dnEsc_ne[of r A0 a] by blast
    then obtain b where "F \<in> \<F> r a b" unfolding dnEsc_def by blast
    then obtain f k where "f \<in> rpth r a b k \<and> F = f`{i. i\<le>k}" unfolding \<F>_def by blast
    moreover then obtain j where "j\<le>k \<and> x = f j" using c2 by blast
    ultimately have "f \<in> rpth (Restr r D) a x j" using c3 unfolding rpth_def by force
    then have "a \<in> Field s' \<and> (a,x) \<in> s'^*" using c2 b10 lem_ccext_rpth_rtr[of _ a x] by blast
    then show ?thesis using lem_rtr_field by fastforce
  qed
qed
```

```
  moreover have "\<exists> D. s' = Restr r D" using b10 by blast
  moreover have "\<not> Conelike r \<longrightarrow> \<not> Conelike s'"
  proof
    assume "\<not> Conelike r"
    then have c1: "\<forall> a \<in> Field r. Field r - dncl r {a} \<noteq> {}" unfolding Conelike_def dncl_def
      by blast
    have "\<forall> a \<in> Field s'. \<exists> a' \<in> Field s'. (a', a) \<notin> s'^*"
    proof
      fix a
      assume d1: "a \<in> Field s'"
      then have d2: "a \<in> Field r" using b10 unfolding Field_def by blast
      then have d3: "w a \<in> Field r - dncl r {a}" using c1 w1 by blast
      then have "(w a, a) \<notin> s'^*" unfolding dncl_def using b10 rtrancl_mono[of s' r] by blast
      moreover have "w a \<in> Field s'"
      proof -
        obtain n where "a \<in> Di n" using d1 b9 b10 unfolding Field_def by blast
        then have "a \<in> Di (Suc n) \<and> w a \<in> Di (Suc n)" using b5 b8 by simp
        then have e1: "Field (g {a, w a}) \<subseteq> H (Di (Suc n))" using b5 b8 by blast
        have e2: "{a, w a} \<subseteq> Field r \<and> finite {a, w a}" using d2 d3 by blast
        have "H (Di (Suc n)) = Di (Suc (Suc n))" using b8 by simp
        moreover have "Di (Suc (Suc n)) \<subseteq> D" using b9 by blast
        ultimately have "Field (g {a,w a}) \<subseteq> D" using e1 by blast
        moreover have "Restr (g {a,w a}) D \<subseteq> s'" using e2 b3 b10 by blast
        ultimately have "g {a,w a} \<subseteq> s'" unfolding Field_def by fastforce
        moreover have "w a \<in> Field (g {a, w a})" using e2 b3 by blast
        ultimately show "w a \<in> Field s'" unfolding Field_def by blast
      qed
      ultimately show "\<exists> a' \<in> Field s'. (a', a) \<notin> s'^*" by blast
    qed
    moreover have "s' \<noteq> {}" using b14 a3 by force
    ultimately show "\<not> Conelike s'" unfolding Conelike_def by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_infsubccr_pext5_scf3:
fixes r::"'U rel" and A B B'::"'U set" and x::'U and Ps::"'U set set"
assumes a1: "CCR r" and a2: "\<not> finite A" and a3: "A \<in> SF r" and a4: "Ps \<subseteq> SCF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR
       (Restr r A') \<and> |A'| =o |A|
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A
      '\<union>B'))
                    \<and> ( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF
       (Restr r A')) )
                    \<and> (escl r A A' \<subseteq> A') \<and> clterm (Restr r A') r"
proof -
  obtain Ps' where p0: "Ps' = (if ( |Ps| \<le>o |A| ) then Ps else {})" by blast
  then have p1: "Ps' \<subseteq> SCF r \<and> |Ps'| \<le>o |A|" using a4 by simp
  have q1: "Field (Restr r A) = A" using a3 unfolding SF_def by blast
  obtain s where "s = (Restr r A)" by blast
  then have q2: "s \<subseteq> r" and q3: "\<not> finite s" and q4: "A = Field s"
    using a2 q1 lem_fin_fl_rel by (blast, metis, blast)
  obtain S where b1: "S = (\<lambda> a. r``{a} - B )" by blast
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. f a \<in> S' a" by (metis someI_ex)
  then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
      longrightarrow> f a = a)"
    unfolding b2 by (clarsimp, metis singletonD)
  obtain y1 y2::'U where n1: "Field r \<noteq> {} \<longrightarrow> {y1, y2} \<subseteq> Field r"
                     and n2: "(\<not> (\<exists> y::'U. Field r - B' \<subseteq> {y})) \<longrightarrow> y1 \<
      notin> B' \<and> y2 \<notin> B' \<and> y1 \<noteq> y2" by blast
  obtain y3 where n3: "(\<not> (Field r - B' \<subseteq> {})) \<longrightarrow> y3 \<in> Field r - B'" by blast
  obtain A1 where b4: "A1 = ({x, y1, y2, y3} \<inter> Field r) \<union> A \<union> (f ` A)" by blast
  have "A1 \<subseteq> Field r"
  proof -
    have c1: "A \<subseteq> Field r" using q4 q2 unfolding Field_def by blast
    moreover have "f ` A \<subseteq> Field r"
    proof
      fix x
      assume "x \<in> f ` A"
      then obtain a where d2: "a \<in> A \<and> x = f a" by blast
      show "x \<in> Field r"
      proof (cases "S a = {}")
        assume "S a = {}"
```

```
      then have "x = a" using c1 d2 b3 by blast
      then show "x \<in> Field r" using d2 c1 by blast
    next
      assume "S a \<noteq> {}"
      then have "x \<in> S a" using d2 b3 by blast
      then show "x \<in> Field r" using b1 unfolding Field_def by blast
    qed
  qed
  ultimately show "A1 \<subseteq> Field r" using b4 by blast
qed
moreover have s0: "|A1| \<le>o |Field s|"
proof -
  obtain C1 where c1: "C1 = {x,y1,y2,y3} \<inter> Field r" by blast
  obtain C2 where c2: "C2 = A \<union> f ` A" by blast
  have "\<not> finite A" using q4 q3 lem_fin_fl_rel by blast
  then have "|C2| =o |A|" using c2 b4 q3 by simp
  then have "|C2| \<le>o |Field s|" unfolding q4 using ordIso_iff_ordLeq by blast
  moreover have c3: "\<not> finite (Field s)" using q3 lem_fin_fl_rel by blast
  moreover have "|C1| \<le>o |Field s|"
  proof -
    have "|{x,y1,y2,y3}| \<le>o |Field s|" using c3
      by (meson card_of_Well_order card_of_ordLeq_finite finite.emptyI finite.insertI ordLeq_total)
    moreover have "|C1| \<le>o |{x,y1,y2,y3}|" unfolding c1 by simp
    ultimately show ?thesis using ordLeq_transitive by blast
  qed
  ultimately have "|C1 \<union> C2| \<le>o |Field s|" unfolding b4 using card_of_Un_ordLeq_infinite by blast
  moreover have "A1 = C1 \<union> C2" using c1 c2 b4 by blast
  ultimately show ?thesis by blast
qed
ultimately obtain s' where s1: "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> |s'| =o |s| \<
    and> A1 \<subseteq> Field s'"
                      and s1': "(\<forall> P \<in> Ps'. (Field s' \<inter> P) \<in> SCF s')"
                      and s1'': "escl r A (Field s') \<subseteq> Field s'"
                      and s1''': "(\<exists> D. s' = Restr r D) \<and> (Conelike s' \<longrightarrow>
    Conelike r)"
  using p1 a1 q2 q3 q4 lem_Ccext_infsubccr_set_ext_scf3[of r s A1 Ps' A] by blast
obtain A' where s2: "A' = Field s'" by blast
obtain s'' where s3: "s'' = Restr r A'" by blast
then have s4: "s' \<subseteq> s'' \<and> Field s'' = A'" using s1 s2 lem_Relprop_fld_sat[of s' r s''] by blast
have s5: "|Field s'| =o |Field s|" using s1 q3 lem_cardreleq_cardfldeq_inf[of s' s] by blast
have "A1 \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 s1 s2 by blast
moreover have "CCR (Restr r A')"
proof -
  have "CCR s''" using s1 s2 s4 lem_Ccext_subccr_eqfld[of s' s''] by blast
  then show ?thesis using s3 by blast
qed
moreover have "|A'| =o |A1|"
proof -
  have "Field s \<subseteq> A1" using q4 b4 by blast
  then have "|Field s| \<le>o |A1|" by simp
  then have "|A'| \<le>o |A1|" using s2 s5 ordIso_ordLeq_trans by blast
  moreover have "|A1| \<le>o |A'|" using s1 s2 by simp
  ultimately show ?thesis using ordIso_iff_ordLeq by blast
qed
ultimately have b6: "A1 \<union> ({x} \<inter> Field r) \<subseteq> A' \<and> CCR (Restr r A') \<and> |A'| =o
    |A1|" by blast
moreover then have "A \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 by blast
moreover have "|A'| =o |A|" using s5 s2 q4 by blast
moreover have "\<forall>a\<in>A. r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}"
proof
  fix a
  assume c1: "a \<in> A"
  have "\<not> (r``{a} \<subseteq> B) \<longrightarrow> r``{a} \<inter> (A'-B) \<noteq> {}"
  proof
    assume "\<not> (r``{a} \<subseteq> B)"
    then have "S a \<noteq> {}" unfolding b1 by blast
    then have "f a \<in> r``{a} - B" using b1 b3 by blast
    moreover have "f a \<in> A'" using c1 b4 b6 by blast
    ultimately show "r``{a} \<inter> (A'-B) \<noteq> {}" by blast
  qed
  then show "r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}" by blast
qed
moreover have "A' \<in> SF r" using s3 s4 unfolding SF_def by blast
moreover have "(\<exists> y::'U. A' - B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A' \<union> B
    ')"
proof
  assume c0: "\<exists> y::'U. A' - B' \<subseteq> {y}"
  show "Field r \<subseteq> (A' \<union> B')"
```

```
    proof (cases "\<exists> y::'U. A' - B' = {y}")
      assume c1: "\<exists> y::'U. A' - B' = {y}"
      moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
      ultimately have "Field r \<noteq> {}" by blast
      then have "{y1, y2} \<subseteq> Field r" using n1 by blast
      then have "{y1, y2} \<subseteq> A'" using b4 s1 s2 by fast
      then have "\<not> (\<exists>y. Field r - B' \<subseteq> {y}) \<longrightarrow> {y1, y2} \<subseteq> A' - B
       ' \<and> y1 \<noteq> y2" using n2 by blast
      moreover have "\<not> ({y1, y2} \<subseteq> A' - B' \<and> y1 \<noteq> y2)" using c1 by force
      ultimately have "\<exists> y::'U. Field r - B' \<subseteq> {y}" by blast
      then show "Field r \<subseteq> A' \<union> B'" using c1 c2 by blast
    next
      assume "\<not> (\<exists> y::'U. A' - B' = {y})"
      then have c1: "A' - B' = {}" using c0 by blast
      show "Field r \<subseteq> (A' \<union> B')"
      proof (cases "Field r = {}")
        assume "Field r = {}"
        then show "Field r \<subseteq> (A' \<union> B')" by blast
      next
        assume "Field r \<noteq> {}"
        moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
        ultimately have "Field r \<noteq> {}" by blast
        then have "\<not> (Field r - B' \<subseteq> {}) \<longrightarrow> {y3} \<subseteq> Field r" using n3 by
       blast
        then have "\<not> (Field r - B' \<subseteq> {}) \<longrightarrow> {y3} \<subseteq> A'" using b4 s1 s2 by
       fast
        then have "\<not> (Field r - B' \<subseteq> {}) \<longrightarrow> {y3} \<subseteq> A' - B' " using n3 by
       blast
        moreover have "\<not> ({y3} \<subseteq> A' - B' )" using c1 by force
        ultimately have "Field r - B' \<subseteq> {}" by blast
        then show "Field r \<subseteq> A' \<union> B'" using c1 c2 by blast
      qed
    qed
  qed
  moreover have "( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A
      ')) )"
  proof -
    have c1: "s' \<subseteq> r" using s3 s4 by blast
    then have "Field s' = Field (Restr r (Field s'))" using lem_Relprop_fld_sat by blast
    moreover have "s' \<subseteq> Restr r (Field s')" using c1 unfolding Field_def by force
    ultimately have "SCF s' \<subseteq> SCF (Restr r (Field s'))" using lem_ccext_scf_sat[of s' "Restr r (Field
      s')"] by blast
    moreover have "|Ps| \<le>o |A| \<longrightarrow> Ps' = Ps" using p0 by simp
    ultimately show ?thesis using s1' s2 by blast
  qed
  moreover have "escl r A A' \<subseteq> A'" using s1'' s2 by blast
  moreover have "Conelike (Restr r A') \<longrightarrow> Conelike r"
  proof
    assume c1: "Conelike (Restr r A')"
    obtain D where "s' = Restr r D" using s1''' by blast
    then have "s' = Restr r (Field s')" unfolding Field_def by force
    then have "Conelike s'" using c1 s2 by simp
    then show "Conelike r" using s1''' by blast
  qed
  ultimately show ?thesis unfolding clterm_def by blast
qed

lemma lem_Ccext_finsubccr_pext5_scf3:
fixes r::"'U rel" and A B B'::"'U set" and x::'U and Ps::"'U set set"
assumes a1: "CCR r" and a2: "finite A" and a3: "A \<in> SF r" and a4: "Ps \<subseteq> SCF r"
shows "\<exists> A'::('U set). (x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR
       (Restr r A') \<and> finite A'
                     \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                     \<and> ((\<exists> y::'U. A'-B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A
      '\<union>B'))
                     \<and> ((\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<
      in> SCF (Restr r A')))"
proof -
  obtain P where p0: "P = (if (Ps \<noteq> {}) then (SOME P. P \<in> Ps) else Field r)" by blast
  moreover have "Field r \<in> SCF r" unfolding SCF_def by blast
  ultimately have p1: "P \<in> SCF r" using a4 by (metis contra_subsetD some_in_eq)
  have p2: "(\<exists> P. Ps = {P}) \<longrightarrow> Ps = {P}" using p0 by fastforce
  have q1: "Field (Restr r A) = A" using a3 unfolding SF_def by blast
  obtain s where "s = (Restr r A)" by blast
  then have q2: "s \<subseteq> r" and q3: "finite s" and q4: "A = Field s"
    using a2 q1 lem_fin_fl_rel by (blast, metis, blast)
  obtain S where b1: "S = (\<lambda> a. r``{a} - B )" by blast
```

```
  obtain S' where b2: "S' = (\<lambda> a. if (S a) \<noteq> {} then (S a) else {a})" by blast
  obtain f where "f = (\<lambda> a. SOME b. b \<in> S' a)" by blast
  moreover have "\<forall> a. \<exists> b. b \<in> (S' a)" unfolding b2 by force
  ultimately have "\<forall> a. f a \<in> S' a" by (metis someI_ex)
  then have b3: "\<forall> a. (S a \<noteq> {} \<longrightarrow> f a \<in> S a) \<and> (S a = {} \<
      longrightarrow> f a = a)"
    unfolding b2 by (clarsimp, metis singletonD)
  obtain y1 y2::'U where n1: "Field r \<noteq> {} \<longrightarrow> {y1, y2} \<subseteq> Field r"
                     and n2: "(\<not> (\<exists> y::'U. Field r - B' \<subseteq> {y})) \<longrightarrow> y1 \<
      notin> B' \<and> y2 \<notin> B' \<and> y1 \<noteq> y2" by blast
  obtain y3 where n3: "(\<not> (Field r - B' \<subseteq> {})) \<longrightarrow> y3 \<in> Field r - B'" by blast
  obtain A1 where b4: "A1 = ({x,y1,y2,y3} \<inter> Field r) \<union> A \<union> (f ` A)" by blast
  have "A1 \<subseteq> Field r"
  proof -
    have c1: "A \<subseteq> Field r" using q4 q2 unfolding Field_def by blast
    moreover have "f ` A \<subseteq> Field r"
    proof
      fix x
      assume "x \<in> f ` A"
      then obtain a where d2: "a \<in> A \<and> x = f a" by blast
      show "x \<in> Field r"
      proof (cases "S a = {}")
        assume "S a = {}"
        then have "x = a" using c1 d2 b3 by blast
        then show "x \<in> Field r" using d2 c1 by blast
      next
        assume "S a \<noteq> {}"
        then have "x \<in> S a" using d2 b3 by blast
        then show "x \<in> Field r" using b1 unfolding Field_def by blast
      qed
    qed
    ultimately show "A1 \<subseteq> Field r" using b4 by blast
  qed
  moreover have s0: "finite A1" using b4 q3 q4 lem_fin_fl_rel by blast
  ultimately obtain s' where s1: "CCR s' \<and> s \<subseteq> s' \<and> s' \<subseteq> r \<and> finite s' \<and>
       A1 \<subseteq> Field s'"
                         and s1': "(\<exists> P. Ps = {P}) \<longrightarrow> (Field s' \<inter> P) \<in> SCF s'"
    using p1 a1 a4 q2 q3 lem_Ccext_finsubccr_set_ext_scf[of r s A1 P] by metis
  obtain A' where s2: "A' = Field s'" by blast
  obtain s'' where s3: "s'' = Restr r A'" by blast
  then have s4: "s' \<subseteq> s'' \<and> Field s'' = A'" using s1 s2 lem_Relprop_fld_sat[of s' r s''] by blast
  have s5: "finite (Field s')" using s1 lem_fin_fl_rel by blast
  have "A1 \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 s1 s2 by blast
  moreover have "CCR (Restr r A')"
  proof -
    have "CCR s''" using s1 s2 s4 lem_Ccext_subccr_eqfld[of s' s''] by blast
    then show ?thesis using s3 by blast
  qed
  ultimately have b6: "A1 \<union> ({x} \<inter> Field r) \<subseteq> A' \<and> CCR (Restr r A')" by blast
  moreover then have "A \<union> ({x} \<inter> Field r) \<subseteq> A'" using b4 by blast
  ultimately have "(x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR (Restr r A
      ')" by blast
  moreover have "finite A'" using s2 s5 by blast
  moreover have "\<forall>a\<in>A. r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}"
  proof
    fix a
    assume c1: "a \<in> A"
    have "\<not> (r``{a} \<subseteq> B) \<longrightarrow> r``{a} \<inter> (A'-B) \<noteq> {}"
    proof
      assume "\<not> (r``{a} \<subseteq> B)"
      then have "S a \<noteq> {}" unfolding b1 by blast
      then have "f a \<in> r``{a} - B" using b1 b3 by blast
      moreover have "f a \<in> A'" using c1 b4 b6 by blast
      ultimately show "r``{a} \<inter> (A'-B) \<noteq> {}" by blast
    qed
    then show "r``{a} \<subseteq> B \<or> r``{a} \<inter> (A'-B) \<noteq> {}" by blast
  qed
  moreover have "A' \<in> SF r" using s3 s4 unfolding SF_def by blast
  moreover have "(\<exists> y::'U. A' - B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A' \<union> B
      ')"
  proof
    assume c0: "\<exists> y::'U. A' - B' \<subseteq> {y}"
    show "Field r \<subseteq> (A' \<union> B')"
    proof (cases "\<exists> y::'U. A' - B' = {y}")
      assume c1: "\<exists> y::'U. A' - B' = {y}"
      moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
      ultimately have "Field r \<noteq> {}" by blast
      then have "{y1, y2} \<subseteq> Field r" using n1 by blast
```

```
      then have "{y1, y2} \<subseteq> A'" using b4 s1 s2 by fast
      then have "\<not> (\<exists>y. Field r - B' \<subseteq> {y}) \<longrightarrow> {y1, y2} \<subseteq> A' - B
      ' \<and> y1 \<noteq> y2" using n2 by blast
      moreover have "\<not> ({y1, y2} \<subseteq> A' - B' \<and> y1 \<noteq> y2)" using c1 by force
      ultimately have "\<exists> y::'U. Field r - B' \<subseteq> {y}" by blast
      then show "Field r \<subseteq> A' \<union> B'" using c1 c2 by blast
    next
      assume "\<not> (\<exists> y::'U. A' - B' = {y})"
      then have c1: "A' - B' = {}" using c0 by blast
      show "Field r \<subseteq> (A' \<union> B')"
      proof (cases "Field r = {}")
        assume "Field r = {}"
        then show "Field r \<subseteq> (A' \<union> B')" by blast
      next
        assume "Field r \<noteq> {}"
        moreover have c2: "A' \<subseteq> Field r" using s1 s2 unfolding Field_def by blast
        ultimately have "Field r \<noteq> {}" by blast
        then have "\<not> (Field r - B' \<subseteq> {}) \<longrightarrow> {y3} \<subseteq> Field r" using n3 by
       blast
        then have "\<not> (Field r - B' \<subseteq> {}) \<longrightarrow> {y3} \<subseteq> A'" using b4 s1 s2 by
        fast
        then have "\<not> (Field r - B' \<subseteq> {}) \<longrightarrow> {y3} \<subseteq> A' - B' " using n3 by
        blast
        moreover have "\<not> ({y3} \<subseteq> A' - B' )" using c1 by force
        ultimately have "Field r - B' \<subseteq> {}" by blast
        then show "Field r \<subseteq> A' \<union> B'" using c1 c2 by blast
      qed
    qed
  qed
  moreover have "(\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (
      Restr r A'))"
  proof -
    have c1: "s' \<subseteq> r" using s3 s4 by blast
    then have "Field s' = Field (Restr r (Field s'))" using lem_Relprop_fld_sat by blast
    moreover have "s' \<subseteq> Restr r (Field s')" using c1 unfolding Field_def by force
    ultimately have "SCF s' \<subseteq> SCF (Restr r (Field s'))" using lem_ccext_scf_sat[of s' "Restr r (Field
      s')"] by blast
    then show ?thesis using p2 s1' s2 by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ccext_subccr_pext5_scf3:
fixes r::"'U rel" and A B B'::"'U set" and x::'U and Ps::"'U set set" and C::"'U set \<Rightarrow> bool"
assumes a1: "CCR r" and a2: "A \<in> SF r" and a3: "Ps \<subseteq> SCF r"
    and a4: "C = (\<lambda> A'::'U set. (x \<in> Field r \<longrightarrow> x \<in> A')
                    \<and> A \<subseteq> A'
                    \<and> A' \<in> SF r
                    \<and> (\<forall>a\<in>A. ((r``{a}\<subseteq>B) \<or> (r``{a}\<inter>(A'-B) \<noteq> {})))
                    \<and> ((\<exists> y::'U. A'-B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A
      '\<union>B'))
                    \<and> CCR (Restr r A')
                    \<and> ((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow>
       |A'| =o |A| ))
                    \<and> ( ((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| )) \<
      longrightarrow>
                        (\<forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A')))
                    \<and> ( (\<not> finite A) \<longrightarrow> ((escl r A A' \<subseteq> A') \<and> (clterm (
      Restr r A') r))) )"
shows "\<exists> A'::('U set). C A'"
proof (cases "finite A")
  assume b1: "finite A"
  then obtain A'::"'U set" where b2: "(x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<
      and> CCR (Restr r A')
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A
      '\<union>B'))"
                    and b3: "finite A' \<and> ((\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps.
       (A' \<inter> P) \<in> SCF (Restr r A')))"
                    using a1 a2 a3 lem_Ccext_finsubccr_pext5_scf3[of r A Ps x B B'] by metis
  have b4: "((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow> |A'| =o |A| ))"
   and b5: "( ((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| )) \<longrightarrow> (\<
      forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A')))"
       using b1 b3 card_of_ordLeq_finite by blast+
  show ?thesis apply (rule exI) unfolding a4 using b1 b2 b4 b5 by force
next
  assume b1: "\<not> finite A"
```

```
  then obtain A' where b2: "(x \<in> Field r \<longrightarrow> x \<in> A') \<and> A \<subseteq> A' \<and> CCR (
      Restr r A')
                    \<and> (\<forall>a\<in>A. r``{a}\<subseteq>B \<or> r``{a}\<inter>(A'-B) \<noteq> {}) \<and>
       A' \<in> SF r
                    \<and> ((\<exists> y::'U. A'-B' \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> (A
      '\<union>B'))"
             and b3: "|A'| =o |A| \<and> ( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter
      > P) \<in> SCF (Restr r A')) )"
             and b3': "(escl r A A' \<subseteq> A') \<and> clterm (Restr r A') r"
    using a1 a2 a3 lem_Ccext_infsubccr_pext5_scf3[of r A Ps x B B'] by metis
  have b4: "((finite A \<longrightarrow> finite A') \<and> ( (\<not> finite A) \<longrightarrow> |A'| =o |A| ))"
    using b1 b3 by metis
  have b5: "( ((\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| )) \<longrightarrow> (\<
      forall> P \<in> Ps. (A' \<inter> P) \<in> SCF (Restr r A')))"
    using b1 b3 by (metis card_of_singl_ordLeq finite.simps)
  have b6: "( (\<not> finite A) \<longrightarrow> ((escl r A A' \<subseteq> A') \<and> clterm (Restr r A') r))"
      using b3' by blast
  have "C A'" unfolding a4 using b2 b4 b5 b6 by simp
  then show ?thesis by blast
qed

lemma lem_acyc_un_emprd:
fixes r s:: "'U rel"
assumes a1: "acyclic r \<and> acyclic s" and a2: "(Range r) \<inter> (Domain s) = {}"
shows "acyclic (r \<union> s)"
proof -
  have "\<And> n. (r \<union> s)^^n \<subseteq> s^* O r^*"
  proof -
    fix n
    show "(r \<union> s)^^n \<subseteq> s^* O r^*"
    proof (induct n)
      show "(r \<union> s)^^0 \<subseteq> s^* O r^*" by force
    next
      fix n
      assume "(r \<union> s)^^n \<subseteq> s^* O r^*"
      moreover then have "(r \<union> s)^^n O r \<subseteq> s^* O r^*" by force
      moreover have "(s^* O r^*) O s \<subseteq> s^* O r^*"
      proof -
        have "r^+ O s = r^* O (r O s)" by (simp add: O_assoc trancl_unfold_right)
        moreover have "r O s = {}" using a2 by force
        ultimately have "s^* O (r^+ O s) = {}" by force
        moreover have "s^* O s \<subseteq> s^*" by force
        moreover have "r^* = Id \<union> r^+" by (metis rtrancl_unfold trancl_unfold_right)
        moreover then have "(s^* O r^*) O s = (s^* O s) \<union> (s^* O (r^+ O s))" by fastforce
        ultimately show ?thesis by fastforce
      qed
      moreover have "(r \<union> s)^^(Suc n) = ((((r \<union> s)^^n) O r) \<union> (((r \<union> s)^^n) O s))"
       by simp
      ultimately show "(r \<union> s) ^^ (Suc n) \<subseteq> s^* O r^*" by force
    qed
  qed
  then have b1: "(r \<union> s)^* \<subseteq> s^* O r^*" using rtrancl_power[of _ "r \<union> s"] by blast
  have "\<forall>x. (x,x) \<in> (r \<union> s)^+ \<longrightarrow> False"
  proof (intro allI impI)
    fix x
    assume "(x,x) \<in> (r \<union> s)^+"
    then have "(x,x) \<in> (r \<union> s)^* O (r \<union> s)" using trancl_unfold_right by blast
    then have "(x,x) \<in> ((s^* O r^*) O r) \<union> ((s^* O r^*) O s)" using b1 by fastforce
    moreover have "(x,x) \<in> ((s^* O r^*) O r) \<longrightarrow> False"
    proof
      assume "(x,x) \<in> ((s^* O r^*) O r)"
      then obtain u v where d1: "(x,u) \<in> s^* \<and> (u,v) \<in> r^* \<and> (v,x) \<in> r" by blast
      moreover then have "x \<notin> Domain s" using a2 by blast
      ultimately have "x = u" by (meson Not_Domain_rtrancl)
      then have "(x,x) \<in> r^+" using d1 by force
      then show "False" using a1 unfolding acyclic_def by blast
    qed
    moreover have "(x,x) \<in> ((s^* O r^*) O s) \<longrightarrow> False"
    proof
      assume "(x,x) \<in> ((s^* O r^*) O s)"
      then obtain u v where d1: "(x,u) \<in> s^* \<and> (u,v) \<in> r^* \<and> (v,x) \<in> s" by blast
      have "u = v \<longrightarrow> False"
      proof
        assume "u = v"
        then have "(x,x) \<in> s^+" using d1 by force
        then show "False" using a1 unfolding acyclic_def by blast
      qed
      then have "(u,v) \<in> r^+" using d1 by (meson rtranclD)
```

```
      then have "v \<in> Range r" using trancl_unfold_right[of r] by force
      moreover have "v \<in> Domain s" using d1 by blast
      ultimately show "False" using a2 by blast
    qed
    ultimately show "False" by blast
  qed
  then show ?thesis using a1 unfolding acyclic_def by blast
qed


lemma lem_spthlen_rtr: "(a,b) \<in> r^* \<Longrightarrow> (a,b) \<in> r^^(spthlen r a b)"
  using rtrancl_power unfolding spthlen_def by (metis LeastI_ex)

lemma lem_spthlen_tr: "(a,b) \<in> r^* \<and> a \<noteq> b \<Longrightarrow> (a,b) \<in> r^^(spthlen r a b) \<
      and> spthlen r a b > 0"
proof -
  assume "(a,b) \<in> r^* \<and> a \<noteq> b"
  moreover then have b1: "(a,b) \<in> r^^(spthlen r a b)" using lem_spthlen_rtr[of a b] by force
  ultimately have "spthlen r a b = 0 \<longrightarrow> False" by force
  then show ?thesis using b1 by blast
qed

lemma lem_spthlen_min: "(a,b) \<in> r^^n \<Longrightarrow> spthlen r a b \<le> n"
  unfolding spthlen_def by (metis Least_le)

lemma lem_spth_inj:
fixes r::"'U rel" and a b::"'U" and f::"nat \<Rightarrow> 'U" and n::"nat"
assumes a1: "f \<in> spth r a b" and a2: "n = spthlen r a b"
shows "inj_on f {i. i\<le>n}"
proof -
  have b1: "f \<in> rpth r a b n" using a1 a2 unfolding spth_def by blast
  have "\<forall> i j. i \<le> n \<and> j \<le> n \<and> i < j \<longrightarrow> f i = f j \<longrightarrow>
      False"
  proof (intro allI impI)
    fix i j
    assume c1: "i \<le> n \<and> j \<le> n \<and> i < j" and c2: "f i = f j"
    obtain l where c3: "l = j - i" by blast
    then have c4: "l \<noteq> 0" using c1 by simp
    obtain g where c5: "g = (\<lambda> k. if (k\<le>i) then (f k) else (f (k + l)))" by blast
    then have "g 0 = a" using b1 unfolding rpth_def by fastforce
    moreover have "g (n - l) = b"
    proof (cases "j < n")
      assume "j < n"
      then show ?thesis using c5 c3 b1 unfolding rpth_def by simp
    next
      assume "\<not> j < n"
      then have "j = n" using c1 by simp
      then show ?thesis using c5 c2 c3 c4 b1 unfolding rpth_def by simp
    qed
    moreover have "\<forall> k < n - l. (g k, g (Suc k)) \<in> r"
    proof (intro allI impI)
      fix k
      assume d1: "k < n - l"
      have "k \<noteq> i \<longrightarrow> (g k, g (Suc k)) \<in> r" using c5 d1 b1 unfolding rpth_def by
       fastforce
      moreover have "k = i \<longrightarrow> (g k, g (Suc k)) \<in> r"
      proof
        assume e1: "k = i"
        then have "(g k, g (Suc k)) = (f i, f ((Suc i) + l))" using c5 by simp
        moreover have "f i = f (i + l)" using c1 c2 c3 by simp
        moreover have "i + l < n" using d1 e1 by force
        ultimately show "(g k, g (Suc k)) \<in> r" using b1 unfolding rpth_def by simp
      qed
      ultimately show "(g k, g (Suc k)) \<in> r" by force
    qed
    ultimately have "g \<in> rpth r a b (n - l)" unfolding rpth_def by blast
    then have "spthlen r a b \<le> n - l"
      using lem_spthlen_min[of a b] lem_ccext_ntr_rpth[of a b] by blast
    then show "False" using a2 c1 c3 by force
  qed
  moreover then have "\<forall> i j. i \<le> n \<and> j \<le> n \<and> j < i \<longrightarrow> f i = f j \<
      longrightarrow> False" by metis
  ultimately show ?thesis unfolding inj_on_def by (metis linorder_neqE_nat mem_Collect_eq)
qed

lemma lem_rtn_rpth_inj: "(a,b) \<in> r^^n \<Longrightarrow> n = spthlen r a b \<Longrightarrow> \<exists> f . f
      \<in> rpth r a b n \<and> inj_on f {i. i \<le> n}"
proof -
```

```
  assume a1: "(a,b) \<in> r^^n" and a2: "n = spthlen r a b"
  then have "(a,b) \<in> r^^n" using lem_spthlen_rtr[of a b] rtrancl_power by blast
  then obtain f where b2: "f \<in> rpth r a b n" using lem_ccext_ntr_rpth[of a b] by blast
  then have "f \<in> spth r a b" using a2 unfolding spth_def by blast
  then have "inj_on f {i. i \<le> n}" using a2 lem_spth_inj[of f] by blast
  then show ?thesis using b2 by blast
qed

lemma lem_rtr_rpth_inj: "(a,b) \<in> r^* \<Longrightarrow> \<exists> f n . f \<in> rpth r a b n \<and> inj_on f
      {i. i \<le> n}"
  using lem_spthlen_rtr[of a b r] lem_rtn_rpth_inj[of a b _ r] by blast

lemma lem_sum_ind_ex:
assumes a1: "g = (\<lambda>n::nat. \<Sum>i<n. f i)"
    and a2:"\<forall>i::nat. f i > 0"
shows "\<exists> n k. (m::nat) = g n + k \<and> k < f n"
proof(induct m)
  have "0 = g 0 + 0 \<and> 0 < f 0" using a1 a2 by simp
  then show "\<exists>n k. (0::nat) = g n + k \<and> k < f n" by blast
next
  fix m
  assume "\<exists>n k. m = g n + k \<and> k < f n"
  then obtain n k where b1: "m = g n + k \<and> k < f n" by blast
  show "\<exists>n' k'. Suc m = g n' + k' \<and> k' < f n'"
  proof(cases "Suc k < f n")
    assume "Suc k < f n"
    then have "Suc m = g n + (Suc k) \<and> (Suc k) < f n" using b1 by simp
    then show "\<exists>n' k'. Suc m = g n' + k' \<and> k' < f n'" by blast
  next
    assume "\<not> Suc k < f n"
    then have "Suc m = g (Suc n) + 0 \<and> 0 < f (Suc n)" using a1 a2 b1 by simp
    then show "\<exists>n' k'. Suc m = g n' + k' \<and> k' < f n'" by blast
  qed
qed

lemma lem_sum_ind_un:
assumes a1: "g = (\<lambda>n::nat. \<Sum>i<n. f i)"
    and a2: "\<forall>i::nat. f i > 0"
    and a3: "(m::nat) = g n + k \<and> k < f n"
    and a4: "m = g n' + k' \<and> k' < f n'"
shows "n = n' \<and> k = k'"
proof -
  have b1: "\<forall> n1 n2. n1 \<le> n2 \<longrightarrow> g n1 \<le> g n2"
  proof(intro allI impI)
    fix n1::nat and n2::nat
    assume "n1 \<le> n2"
    moreover obtain t where "t = n2 - n1" by blast
    moreover have "g n1 \<le> g (n1 + t)" unfolding a1 by (induct t, simp+)
    ultimately show "g n1 \<le> g n2" by simp
  qed
  have "n < n' \<longrightarrow> False"
  proof
    assume "n < n'"
    then have "g (Suc n) \<le> g n'" using b1 by simp
    then have "g n + f n \<le> g n'" using a1 b1 by simp
    moreover have "g n' < g n + f n" using a3 a4 by simp
    ultimately show "False" by simp
  qed
  moreover have "n' < n \<longrightarrow> False"
  proof
    assume "n' < n"
    then have "g (Suc n') \<le> g n" using b1 by simp
    then have "g n' + f n' \<le> g n" using a1 b1 by simp
    moreover have "g n < g n' + f n'" using a3 a4 by simp
    ultimately show "False" by simp
  qed
  ultimately show "n = n' \<and> k = k'" using a3 a4 by simp
qed

lemma lem_flatseq:
fixes r::"'U rel" and xi::"nat \<Rightarrow> 'U"
assumes "\<forall>n. (xi n, xi (Suc n)) \<in> r^* \<and> (xi n \<noteq> xi (Suc n))"
shows "\<exists> g yi. ( \<forall>n. (yi n, yi (Suc n)) \<in> r )
             \<and> (\<forall> i::nat. \<forall> j::nat. i < j \<longleftrightarrow> g i < g j )
             \<and> (\<forall> i::nat. yi (g i) = xi i)
             \<and> (\<forall> i::nat. inj_on yi { k. g i \<le> k \<and> k \<le> g (Suc i) } )
             \<and> (\<forall> k::nat. \<exists> i::nat. g i \<le> k \<and> Suc k \<le> g (Suc i))
```

```
            \<and> (\<forall> k i i'. g i \<le> k \<and> Suc k \<le> g (Suc i) \<and> g i' \<le> k \<and> Suc k
       \<le> g (Suc i') \<longrightarrow> i = i' )"
proof -
  obtain P where b0: "P = (\<lambda> n m. m > 0 \<and> (xi n, xi (Suc n)) \<in> r^^m \<and> m = spthlen r (xi n)
       (xi (Suc n)))" by blast
  then have "\<forall>n. \<exists>m. P n m" using assms lem_spthlen_tr[of _ _ r] by blast
  then obtain f where "\<forall>n. P n (f n)" by metis
  then have b1: "\<forall> n. (f n) > 0 \<and> (xi n, xi (Suc n)) \<in> r^^(f n)"
       and b1': "\<forall> n. (f n) = spthlen r (xi n) (xi (Suc n))" using b0 by blast+
  have "\<forall> n. \<exists>yi. inj_on yi {i. i \<le> f n} \<and> (yi 0) = (xi n) \<and>
           (\<forall>k<(f n). (yi k, yi (Suc k)) \<in> r) \<and> (yi (f n)) = (xi (Suc n))"
  proof
    fix n
    have "(xi n, xi (Suc n)) \<in> r^^(f n)" and "(f n) = spthlen r (xi n) (xi (Suc n))"
      using b1 b1' by blast+
    then obtain yi where "yi \<in> rpth r (xi n) (xi (Suc n)) (f n) \<and> inj_on yi {i. i \<le> f n}"
      using lem_rtn_rpth_inj[of "xi n" "xi (Suc n)" "f n" r] by blast
    then show "\<exists>yi. inj_on yi {i. i \<le> f n} \<and> (yi 0) = (xi n) \<and> (\<forall>k<(f n). (yi k,
      yi (Suc k)) \<in> r)
             \<and> (yi (f n)) = (xi (Suc n)) " unfolding rpth_def by blast
  qed
  then obtain yin where b2: "\<forall> n. inj_on (yin n) {i. i \<le> f n} \<and> ((yin n) 0) = (xi n) \<and>
      (\<forall>k < (f n). ((yin n) k, (yin n) (Suc k)) \<in> r) \<and> ((yin n) (f n)) = (xi (Suc n))" by metis
  obtain g where b3: "g = (\<lambda>n. \<Sum>i<n. f i)" by blast
  obtain yi where b4: "yi = (\<lambda>m. let p =
                         (SOME p. m = (g (fst p)) + (snd p) \<and> (snd p) < (f (fst p)))
                         in (yin (fst p)) (snd p) )" by blast
  have b5: "\<And> m n k. m = (g n) + k \<and> k < f n \<Longrightarrow> yi m = yin n k"
  proof -
    fix m n k
    assume c0: "m = (g n) + k \<and> k < f n"
    have "\<exists> p . (m = (g (fst p)) + (snd p)) \<and> ((snd p) < (f (fst p)))"
      using b1 b3 lem_sum_ind_ex by force
    then obtain n' k' where "m = (g n') + k' \<and> k' < (f n') \<and> yi m = (yin n') k'"
      using b4 by (smt someI_ex)
    moreover then have "n' = n \<and> k' = k" using c0 b1 b3 lem_sum_ind_un[of g f m n' k' n k] by blast
    ultimately show "yi m = yin n k" by blast
  qed
  have "\<forall>m. (yi m, yi (Suc m)) \<in> r"
  proof
    fix m
    have "\<exists> p . (m = (g (fst p)) + (snd p)) \<and> ((snd p) < (f (fst p)))"
      using b1 b3 lem_sum_ind_ex by force
    then obtain n k where c1: "m = (g n) + k \<and> k < (f n) \<and> yi m = (yin n) k"
      using b4 by (smt someI_ex)
    have "\<exists> p . ((Suc m) = (g (fst p)) + (snd p)) \<and> ((snd p) < (f (fst p)))"
      using b1 b3 lem_sum_ind_ex by force
    then obtain n' k' where c2: "(Suc m) = (g n') + k' \<and> k' < (f n') \<and> yi (Suc m) = (yin n') k'"
      using b4 by (smt someI_ex)
    show "(yi m, yi (Suc m)) \<in> r"
    proof(cases "Suc k < f n")
      assume "Suc k < f n"
      then have "Suc m = g n + (Suc k) \<and> (Suc k) < f n" using c1 by simp
      then have "n' = n \<and> k' = Suc k" using b1 b3 c2 lem_sum_ind_un[of g] by blast
      then show "(yi m, yi (Suc m)) \<in> r" using b2 c1 c2 by force
    next
      assume d1: "\<not> Suc k < f n"
      then have "Suc m = g (Suc n) + 0 \<and> 0 < f (Suc n)" using b1 b3 c1 by simp
      then have "n' = Suc n \<and> k' = 0" using b1 b3 c2 lem_sum_ind_un[of g] by blast
      then show "(yi m, yi (Suc m)) \<in> r"
        using b2 c1 c2 d1 by (metis Suc_le_eq dual_order.antisym not_less)
    qed
  qed
  moreover have b6: "\<forall> j::nat. \<forall> i::nat. i < j \<longrightarrow> g i < g j"
  proof
    fix j0::"nat"
    show "\<forall> i::nat. i < j0 \<longrightarrow> g i < g j0"
    proof (induct j0)
      show "\<forall>i<0. g i < g 0" by blast
    next
      fix j::"nat"
      assume d1: "\<forall>i<j. g i < g j"
      show "\<forall>i<Suc j. g i < g (Suc j)"
      proof (intro allI impI)
        fix i::"nat"
        assume "i < Suc j"
        then have "i \<le> j" by force
        moreover have "g j < g (Suc j)" using b1 b3 by simp
```

```
      moreover then have "i < j \<longrightarrow> g i < g (Suc j)" using d1 by force
      ultimately show "g i < g (Suc j)" by force
    qed
  qed
qed
moreover have b7: "\<forall> j::nat. \<forall>i::nat. j \<le> i \<longrightarrow> g j \<le> g i"
proof (intro allI impI)
  fix j::"nat" and i::"nat"
  assume "j \<le> i"
  moreover have "j < i \<longrightarrow> g j \<le> g i" using b6 by force
  moreover have "j = i \<longrightarrow> g j \<le> g i" by blast
  ultimately show "g j \<le> g i" by force
qed
moreover have b8: "\<forall> j::nat. \<forall> i::nat. g i < g j \<longrightarrow> i < j"
proof (intro allI impI)
  fix j::"nat" and i::"nat"
  assume "g i < g j"
  moreover have "j \<le> i \<longrightarrow> g j \<le> g i" using b7 by blast
  ultimately show "i < j" by simp
qed
moreover have b9: "\<forall> i::nat. yi (g i) = xi i"
proof
  fix i::"nat"
  obtain p where "p = (i, 0::nat)" by blast
  then have "((g i) = (g (fst p)) + (snd p)) \<and> ((snd p) < (f (fst p)))" using b1 by force
  then obtain n k where c1: "(g i) = (g n) + k \<and> k < (f n) \<and> yi (g i) = (yin n) k"
    using b4 by (smt someI_ex)
  then have "g n \<le> g i" by simp
  moreover have "g n < g i \<longrightarrow> False"
  proof
    assume "g n < g i"
    then have "n < i" using b8 by blast
    then have "g (Suc n) \<le> g i" using b7 by simp
    then show "False" using c1 b3 b6 by force
  qed
  ultimately have "g i = g n" by force
  then have "\<not> i < n \<and> \<not> n < i" using b6 by force
  then have "i = n \<and> k = 0" using c1 by force
  then have "yi (g i) = (yin i) 0" using c1 by blast
  moreover have "(yin i) 0 = xi i" using b2 by blast
  ultimately show "yi (g i) = xi i" by simp
qed
moreover have "\<forall> i::nat. inj_on yi { k. g i \<le> k \<and> k \<le> g (Suc i) }"
proof
  fix i
  have c1: "inj_on (yin i) {k. k \<le> f i}" using b2 by blast
  have "\<forall> k1 k2. g i \<le> k1 \<and> k1 \<le> g (Suc i) \<longrightarrow> g i \<le> k2 \<and> k2 \<le>
     g (Suc i) \<longrightarrow> yi k1 = yi k2 \<longrightarrow> k1 = k2"
  proof (intro allI impI)
    fix k1 k2
    assume d1: "g i \<le> k1 \<and> k1 \<le> g (Suc i)"
       and d2: "g i \<le> k2 \<and> k2 \<le> g (Suc i)" and d3: "yi k1 = yi k2"
    have "g i \<le> k1 \<and> k1 \<le> g i + f i" using d1 b3 by simp
    then have "\<exists> t. k1 = g i + t \<and> t \<le> f i" by presburger
    then obtain t1 where d4: "k1 = g i + t1 \<and> t1 \<le> f i" by blast
    have "g i \<le> k2 \<and> k2 \<le> g i + f i" using d2 b3 by simp
    then have "\<exists> t. k2 = g i + t \<and> t \<le> f i" by presburger
    then obtain t2 where d5: "k2 = g i + t2 \<and> t2 \<le> f i" by blast
    have "t1 < f i \<and> t2 < f i \<longrightarrow> k1 = k2"
    proof
      assume "t1 < f i \<and> t2 < f i"
      then have "yi k1 = yin i t1 \<and> yi k2 = yin i t2" using d4 d5 b5 by blast
      then have "yin i t1 = yin i t2" using d3 by metis
      then show "k1 = k2" using c1 d4 d5 unfolding inj_on_def by blast
    qed
    moreover have "t1 = f i \<and> t2 < f i \<longrightarrow> False"
    proof
      assume e1: "t1 = f i \<and> t2 < f i"
      then have e2: "yi k2 = yin i t2" using d4 d5 b5 by blast
      have e3: "k1 = g (Suc i)" using e1 d4 b3 by simp
      then have "yi k1 = yin (Suc i) 0" using b1 b5[of k1 "Suc i" 0] by simp
      moreover have "yi k1 = yin i (f i)" using e3 b9 b2 by simp
      ultimately have "yin i t2 = yin i (f i)" using e2 d3 by metis
      then have "t2 = f i" using c1 d5 unfolding inj_on_def by blast
      then show "False" using e1 by force
    qed
    moreover have "t1 < f i \<and> t2 = f i \<longrightarrow> False"
    proof
```

```
        assume e1: "t1 < f i \<and> t2 = f i"
        then have e2: "yi k1 = yin i t1" using d4 d5 b5 by blast
        have e3: "k2 = g (Suc i)" using e1 d5 b3 by simp
        then have "yi k2 = yin (Suc i) 0" using b1 b5[of k2 "Suc i" 0] by simp
        moreover have "yi k2 = yin i (f i)" using e3 b9 b2 by simp
        ultimately have "yin i t1 = yin i (f i)" using e2 d3 by metis
        then have "t1 = f i" using c1 d4 unfolding inj_on_def by blast
        then show "False" using e1 by force
      qed
      ultimately show "k1 = k2" using d4 d5 by force
    qed
    then show "inj_on yi { k. g i \<le> k \<and> k \<le> g (Suc i) }" unfolding inj_on_def by blast
  qed
  moreover have "\<forall> m. \<exists> n. g n \<le> m \<and> Suc m \<le> g (Suc n)"
  proof
    fix m
    obtain n k where "m = g n + k \<and> k < f n" using b1 b3 lem_sum_ind_ex[of g f m] by blast
    then have "g n \<le> m \<and> Suc m \<le> g (Suc n)" using b3 by simp
    then show "\<exists> n. g n \<le> m \<and> Suc m \<le> g (Suc n)" by blast
  qed
  moreover have "\<forall> k i i'. g i \<le> k \<and> Suc k \<le> g (Suc i) \<and> g i' \<le> k \<and> Suc k \<
      le> g (Suc i') \<longrightarrow> i = i'"
  proof (intro allI impI)
    fix k i i'
    assume "g i \<le> k \<and> Suc k \<le> g (Suc i) \<and> g i' \<le> k \<and> Suc k \<le> g (Suc i')"
    moreover then have "k < g i + f i \<and> k < g i' + f i'" using b3 by simp
    ultimately have "\<exists> l1. k = g i + l1 \<and> l1 < f i" and "\<exists> l2. k = g i' + l2 \<and> l2 < f
      i'" by presburger+
    then obtain l1 l2 where "k = g i + l1 \<and> l1 < f i" and "k = g i' + l2 \<and> l2 < f i'" by blast
    then show "i = i'" using b1 b3 lem_sum_ind_un[of g f k i l1 i' l2] by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_sv_un3:
fixes r1 r2 r3::"'U rel"
assumes "single_valued (r1 \<union> r3)" and "single_valued (r2 \<union> r3)" and "Field r1 \<inter> Field r2 =
      {}"
shows "single_valued (r1 \<union> r2 \<union> r3)"
  using assms unfolding single_valued_def Field_def by blast

lemma lem_cfcomp_d2uset:
fixes \<kappa>::"'U rel" and r::"'U rel" and W::"'U rel \<Rightarrow> 'U set" and R::"'U rel \<Rightarrow> 'U
      rel"
    and S::"'U rel set"
assumes a1: "\<kappa> =o cardSuc |UNIV::nat set|"
    and a3: "T = { t::'U rel. t \<noteq> {} \<and> CCR t \<and> single_valued t \<and> acyclic t \<and> (\<
      forall>x\<in>Field t. t``{x} \<noteq> {}) }"
    and a4: "Refl r"

    and a5: "S \<subseteq> {\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}"
    and a6: "|{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |S|"
    and a7: "\<forall> \<alpha> \<in> S. \<exists> \<beta> \<in> S. \<alpha> <o \<beta>"

    and a8: "Field r = (\<Union>\<alpha>\<in>S. W \<alpha>)" and a9: "\<forall>\<alpha>\<in>S. \<forall> \<beta
      >\<in>S. \<alpha> \<noteq> \<beta> \<longrightarrow> W \<alpha> \<inter> W \<beta> = {}"
    and a10: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> R \<alpha> \<in> T \<and> R \<alpha> \<
      subseteq> r \<and> |W \<alpha>| \<le>o |UNIV::nat set|
                           \<and> Field (R \<alpha>) = W \<alpha> \<and> \<not> Conelike (Restr r (W \<alpha>)
      )"
    and a11: "\<And> \<alpha> x. \<alpha> \<in> S \<Longrightarrow> x \<in> W \<alpha> \<Longrightarrow> \<
      exists> a.
              ((x,a) \<in> (Restr r (W \<alpha>))^* \<and> (\<forall> \<beta> \<in> S. \<alpha> <o \<beta> \<
      longrightarrow> (r``{a} \<inter> W \<beta>) \<noteq> {}))"
shows "\<exists> r'. CCR r' \<and> DCR 2 r' \<and> r' \<subseteq> r \<and> (\<forall> a \<in> Field r. \<exists>
       b \<in> Field r'. (a,b) \<in> r^*)"
proof -
  obtain l :: "'U \<Rightarrow> 'U rel" where q1: "l = (\<lambda> a. SOME \<alpha>. \<alpha> \<in> S \<and> a \<
      in> W \<alpha>)" by blast
  have q2: "\<And> a. a \<in> Field r \<Longrightarrow> l a \<in> S \<and> a \<in> W (l a)"
  proof -
    fix a
    assume "a \<in> Field r"
    then obtain \<alpha> where "\<alpha> \<in> S \<and> a \<in> W \<alpha>" using q1 a8 by blast
    then show "l a \<in> S \<and> a \<in> W (l a)" using q1 someI_ex[of "\<lambda> \<alpha>. \<alpha> \<in> S \<
      and> a \<in> W \<alpha>"] by metis
  qed
```

```
have q3: "\<And> \<alpha> a. \<alpha> \<in> S \<Longrightarrow> a \<in> W \<alpha> \<Longrightarrow> l a = \<
    alpha>"
proof -
  fix \<alpha> a
  assume "\<alpha> \<in> S" and "a \<in> W \<alpha>"
  moreover then have "a \<in> W (l a) \<and> \<alpha> \<in> S \<and> l a \<in> S" using q2 a8 a10 by fast
  ultimately show "l a = \<alpha>" using a9 by blast
qed
have b1: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> (R \<alpha>) \<in> T" using a3 a10 by blast
have b4: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> (R \<alpha>) \<subseteq> r" using a10 by blast
have b7: "\<forall> \<alpha> \<in> S. \<forall> \<beta> \<in> S. \<exists> \<gamma>\<in>S. (\<alpha> <o \<
    gamma> \<or> \<alpha> = \<gamma>) \<and> (\<beta> <o \<gamma> \<or> \<beta> = \<gamma>)"
proof (intro ballI)
  fix \<alpha> \<beta>
  assume "\<alpha> \<in> S" and "\<beta> \<in> S"
  then have "Well_order \<alpha> \<and> Well_order \<beta>" and "\<alpha> \<in> S \<and> \<beta> \<in> S"
    using a5 unfolding ordLess_def by blast+
  moreover then have "\<alpha> <o \<beta> \<or> \<beta> <o \<alpha> \<or> \<alpha> =o \<beta>"
    using ordLeq_iff_ordLess_or_ordIso ordLess_or_ordLeq by blast
  ultimately show "\<exists> \<gamma> \<in> S. (\<alpha> <o \<gamma> \<or> \<alpha> = \<gamma>) \<and> (\<beta
    > <o \<gamma> \<or> \<beta> = \<gamma>)"
    using a3 a5 lem_Oeq[of \<alpha> \<beta>] by blast
qed
obtain s :: "'U rel \<Rightarrow> nat \<Rightarrow> 'U" where b8: "s = (\<lambda> \<alpha>. SOME xi. cfseq (R
    \<alpha>) xi)" by blast
moreover have "\<forall> \<alpha> \<in> S. \<exists> xi. cfseq (R \<alpha>) xi" using b1 a3 lem_ccrsv_cfseq by
     blast
ultimately have b9: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> cfseq (R \<alpha>) (s \<alpha>)" by (
    metis someI_ex)
obtain en where b_en: "en = (\<lambda> \<alpha>. SOME g :: nat \<Rightarrow> 'U. W \<alpha> \<subseteq> g`UNIV
    )" by blast
obtain ta :: "'U \<Rightarrow> 'U rel \<Rightarrow> 'U"
  where b10: "ta = (\<lambda> u \<alpha>'. SOME u'. (u,u') \<in> r \<and> u' \<in> W \<alpha>')" by blast
obtain t :: "('U rel) \<times> 'U \<Rightarrow> 'U rel \<Rightarrow> 'U"
  where b11: "t = (\<lambda> (\<alpha>,a) \<alpha>'. ta a \<alpha>')" by blast
obtain tm :: "('U rel) \<times> nat \<Rightarrow> 'U rel \<Rightarrow> 'U"
  where b12: "tm = (\<lambda> (\<alpha>,k) \<alpha>'. t (\<alpha>,(en \<alpha> k)) \<alpha>')" by blast
obtain jnN :: "'U \<Rightarrow> 'U \<Rightarrow> 'U"
  where b13: "jnN = (\<lambda> u u'. SOME v. (u,v) \<in> (R (l u))^* \<and> (u',v) \<in> (R (l u))^*)" by
    blast
obtain h where b20: "\<And> \<alpha> k1 \<beta> k2. \<alpha> \<in> S \<and> \<beta> \<in> S \<Longrightarrow>
         (\<exists> \<gamma> \<in> S. \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma> \<and> h \<gamma> = jnN
     (tm (\<alpha>,k1) \<gamma>) (tm (\<beta>,k2) \<gamma>) )"
  using a1 a5 a6 a7 lem_jnfix_cardsuc[of "UNIV::nat set" \<kappa> S jnN tm] by blast
define EP where "EP = (\<lambda> \<alpha>. { a \<in> W \<alpha>. \<forall> \<beta> \<in> S. \<alpha> <o \<beta
    > \<longrightarrow> (r``{a} \<inter> W \<beta>) \<noteq> {} })"
have b24: "\<And> \<alpha> k b. \<alpha> \<in> S \<Longrightarrow> (s \<alpha> k, b) \<in> (R \<alpha>)^* \<
    Longrightarrow> (\<exists> k'\<ge>k. b = s \<alpha> k')"
proof -
  fix \<alpha> k b
  assume c1: "\<alpha> \<in> S" and c2: "(s \<alpha> k, b) \<in> (R \<alpha>)^*"
  moreover then have "single_valued (R \<alpha>)" using b1 a3 by blast
  moreover have " \<forall>i. (s \<alpha> i, s \<alpha> (Suc i)) \<in> R \<alpha>" using c1 b9 unfolding
    cfseq_def by blast
  ultimately show "\<exists> k'\<ge>k. b = s \<alpha> k'"
    using lem_rseq_svacyc_inv_rtr[of "R \<alpha>" "s \<alpha>" k b] by blast
qed
have b25: "\<And> \<alpha> k b. \<alpha> \<in> S \<Longrightarrow> (s \<alpha> k, b) \<in> (R \<alpha>)^+ \<
    Longrightarrow> (\<exists> k'>k. b = s \<alpha> k')"
proof -
  fix \<alpha> k b
  assume c1: "\<alpha> \<in> S" and c2: "(s \<alpha> k, b) \<in> (R \<alpha>)^+"
  moreover then have "single_valued (R \<alpha>)" using b1 a3 by blast
  moreover have " \<forall>i. (s \<alpha> i, s \<alpha> (Suc i)) \<in> R \<alpha>" using c1 b9 unfolding
    cfseq_def by blast
  ultimately show "\<exists> k'>k. b = s \<alpha> k'" using lem_rseq_svacyc_inv_tr[of "R \<alpha>" "s \<alpha
    >" k b] by blast
qed
have b26: "\<And> \<alpha> a b c. \<alpha> \<in> S \<Longrightarrow> a \<in> W \<alpha> \<Longrightarrow> b \<
    in> W \<alpha> \<Longrightarrow>
         c = jnN a b \<Longrightarrow> c \<in> W \<alpha> \<and> (a, c) \<in> (R \<alpha>)^* \<and> (b, c) \<
    in> (R \<alpha>)^*"
proof -
  fix \<alpha> a b c
  assume c1: "\<alpha> \<in> S" and c2: "a \<in> W \<alpha>" and c3: "b \<in> W \<alpha>" and c4: "c = jnN a b
    "
  then have "CCR (R \<alpha>) \<and> a \<in> Field (R \<alpha>) \<and> b \<in> Field (R \<alpha>)" using c1 b1
     a3 a10 by blast
```

```
  then have "\<exists> c'. (a, c') \<in> (R \<alpha>)^* \<and> (b, c') \<in> (R \<alpha>)^*" unfolding CCR_def
     by blast
  moreover have "l a = \<alpha>" using c1 c2 q3 by blast
  moreover then have "c = (SOME c'. (a, c') \<in> (R \<alpha>)^* \<and> (b, c') \<in> (R \<alpha>)^*)" using
    c4 b13 by simp
  ultimately have c5: "(a, c) \<in> (R \<alpha>)^* \<and> (b, c) \<in> (R \<alpha>)^*"
    using someI_ex[of "\<lambda> c'. (a, c') \<in> (R \<alpha>)^* \<and> (b, c') \<in> (R \<alpha>)^*"] by
    force
  moreover have "W \<alpha> \<in> Inv (R \<alpha>)" using c1 a10[of \<alpha>] unfolding Field_def Inv_def by
    blast
  moreover then have "c \<in> W \<alpha>" using c2 c5 lem_Inv_restr_rtr2[of "W \<alpha>" "R \<alpha>"] by
    blast
  ultimately show "c \<in> W \<alpha> \<and> (a, c) \<in> (R \<alpha>)^* \<and> (b, c) \<in> (R \<alpha>)^*"
    by blast
qed
have b_enr: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> W \<alpha> \<subseteq> (en \<alpha>)`(UNIV::
    nat set)"
proof -
  fix \<alpha>
  assume "\<alpha> \<in> S"
  then have "|W \<alpha>| \<le>o |UNIV::nat set|" using a10 by blast
  then obtain g::"nat \<Rightarrow> 'U" where "W \<alpha> \<subseteq> g`UNIV"
    by (metis card_of_ordLeq2 empty_subsetI order_refl)
  then show "W \<alpha> \<subseteq> (en \<alpha>)`UNIV" unfolding b_en using someI_ex by metis
qed
have b_h: "\<And> \<alpha> a \<beta> b. \<alpha> \<in> S \<and> \<beta> \<in> S \<Longrightarrow> a \<in> EP
    \<alpha> \<and> b \<in> EP \<beta> \<Longrightarrow>
           (\<exists> \<gamma> \<in> S. \<exists> a' \<in> W \<gamma>. \<exists> b' \<in> W \<gamma>. \<alpha
    > <o \<gamma> \<and> \<beta> <o \<gamma>
             \<and> (a,a') \<in> r \<and> (a', h \<gamma>) \<in> (R \<gamma>)^* \<and> (b,b') \<in> r \<and>
    (b', h \<gamma>) \<in> (R \<gamma>)^*)"
proof -
  fix \<alpha> a \<beta> b
  assume c1: "\<alpha> \<in> S \<and> \<beta> \<in> S" and c2: "a \<in> EP \<alpha> \<and> b \<in> EP \<beta>"
  then have "a \<in> W \<alpha> \<and> b \<in> W \<beta>" unfolding EP_def by blast
  moreover then obtain k1 k2 where c3: "a = en \<alpha> k1 \<and> b = en \<beta> k2" using c1 b_enr by blast
  ultimately obtain \<gamma> where c4: "\<gamma> \<in> S \<and> \<alpha> <o \<gamma> \<and> \<beta> <o \<gamma
    >"
                       and c5: "h \<gamma> = jnN (tm (\<alpha>,k1) \<gamma>) (tm (\<beta>,k2) \<gamma>)"
    using c1 b20 by blast
  have "ta a \<gamma> = (SOME a'. (a, a') \<in> r \<and> a' \<in> W \<gamma>)" using b10 by simp
  moreover have "\<exists> x. (a, x) \<in> r \<and> x \<in> W \<gamma>" using c2 c4 unfolding EP_def by blast
  ultimately have c6: "(a, ta a \<gamma>) \<in> r \<and> ta a \<gamma> \<in> W \<gamma>"
    using someI_ex[of "\<lambda> a'. (a, a') \<in> r \<and> a' \<in> W \<gamma>"] by metis
  have "ta b \<gamma> = (SOME a'. (b, a') \<in> r \<and> a' \<in> W \<gamma>)" using b10 by simp
  moreover have "\<exists> x. (b, x) \<in> r \<and> x \<in> W \<gamma>" using c2 c4 unfolding EP_def by blast
  ultimately have c7: "(b, ta b \<gamma>) \<in> r \<and> ta b \<gamma> \<in> W \<gamma>"
    using someI_ex[of "\<lambda> a'. (b, a') \<in> r \<and> a' \<in> W \<gamma>"] by metis
  have "h \<gamma> = jnN (ta a \<gamma>) (ta b \<gamma>)" using c3 c5 b11 b12 by simp
  moreover have "ta a \<gamma> \<in> W \<gamma> \<and> ta b \<gamma> \<in> W \<gamma>" using c6 c7 by blast
  ultimately have "h \<gamma> \<in> W \<gamma> \<and> (ta a \<gamma>, h \<gamma>) \<in> (R \<gamma>)^* \<and>
    (ta b \<gamma>, h \<gamma>) \<in> (R \<gamma>)^*"
    using c4 b26[of \<gamma> "ta a \<gamma>" "ta b \<gamma>" "h \<gamma>"] by blast
  then show "\<exists> \<gamma> \<in> S. \<exists> a' \<in> W \<gamma>. \<exists> b' \<in> W \<gamma>. \<alpha
    > <o \<gamma> \<and> \<beta> <o \<gamma>
      \<and> (a,a') \<in> r \<and> (a', h \<gamma>) \<in> (R \<gamma>)^* \<and> (b,b') \<in> r \<and> (b', h
    \<gamma>) \<in> (R \<gamma>)^*"
    using c4 c6 c7 by blast
qed
have p1: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> R \<alpha> \<subseteq> Restr r (W \<alpha>)"
    using a10 unfolding Field_def by fastforce
have p2: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> Field (Restr r (W \<alpha>)) = W \<alpha>"
proof -
  fix \<alpha>
  assume "\<alpha> \<in> S"
  then have "W \<alpha> \<subseteq> Field r" using a10 unfolding Field_def by blast
  moreover have "SF r = {A. A \<subseteq> Field r}" using a4 unfolding SF_def refl_on_def Field_def by fast
  ultimately have "W \<alpha> \<in> SF r" by blast
  then show "Field (Restr r (W \<alpha>)) = W \<alpha>" unfolding SF_def by blast
qed
have p3: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> \<forall>n. \<exists>k\<ge>n. (s \<alpha> (Suc k
    ), s \<alpha> k) \<notin> (Restr r (W \<alpha>))^*"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have "\<forall>a\<in>Field (Restr r (W \<alpha>)). \<exists>i. (a, s \<alpha> i) \<in> (Restr r (W \<alpha>)
    )^*"
  proof
```

```
    fix a
    assume "a \<in> Field (Restr r (W \<alpha>))"
    then have "a \<in> Field (R \<alpha>)" using c1 a10[of \<alpha>] unfolding Field_def by blast
    then obtain i where "(a, s \<alpha> i) \<in> (R \<alpha>)^*" using c1 b9[of \<alpha>] unfolding cfseq_def
     by blast
    moreover have "R \<alpha> \<subseteq> Restr r (W \<alpha>)" using c1 p1 by blast
    ultimately show "\<exists>i. (a, s \<alpha> i) \<in> (Restr r (W \<alpha>))^*" using rtrancl_mono by blast
  qed
  moreover have "\<forall>i. (s \<alpha> i, s \<alpha> (Suc i)) \<in> Restr r (W \<alpha>)"
    using c1 p1 b9[of \<alpha>] unfolding cfseq_def using rtrancl_mono by blast
  ultimately have "cfseq (Restr r (W \<alpha>)) (s \<alpha>)" unfolding cfseq_def by blast
  then show "\<forall>n. \<exists>k\<ge>n. (s \<alpha> (Suc k), s \<alpha> k) \<notin> (Restr r (W \<alpha>))
    ^*"
    using c1 a10[of \<alpha>] lem_cfseq_ncl[of "Restr r (W \<alpha>)" "s \<alpha>"] by blast
qed
obtain E where b27: "E = (\<lambda> \<alpha>. { k. (s \<alpha> (Suc k), s \<alpha> k) \<notin> (Restr r (W \<
    alpha>))^* })" by blast
obtain P where b28: "P = (\<lambda> \<alpha>. (s \<alpha>)`(E \<alpha>) )" by blast
obtain K where b29: "K = (\<lambda> \<alpha>. { a \<in> W \<alpha>. (h \<alpha> \<in> W \<alpha> \<
    longrightarrow> (h \<alpha>, a) \<in> (R \<alpha>)^*)
                                      \<and> (a, h \<alpha>) \<notin> (R \<alpha>)^* })" by blast
let ?F = "\<lambda> \<alpha>. P \<alpha> \<inter> K \<alpha>"
have b31: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> P \<alpha> \<in> SCF (R \<alpha>)"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  then have "P \<alpha> \<subseteq> Field (R \<alpha>)" using b9 b28 lem_cfseq_fld by blast
  moreover have "\<forall> a \<in> Field (R \<alpha>). \<exists> b \<in> P \<alpha>. (a, b) \<in> (R \<alpha>)
    ^*"
  proof
    fix a
    assume "a \<in> Field (R \<alpha>)"
    then obtain i where d1: "(a, s \<alpha> i) \<in> (R \<alpha>)^*" using c1 b9[of \<alpha>] unfolding
     cfseq_def by blast
    then obtain k where "i\<le>k \<and> (s \<alpha> (Suc k), s \<alpha> k) \<notin> (Restr r (W \<alpha>))^*"
     using c1 p3[of \<alpha>] by blast
    moreover then have d2: "(s \<alpha> i, s \<alpha> k) \<in> (R \<alpha>)^*"
      using c1 b9[of \<alpha>] lem_rseq_rtr unfolding cfseq_def by blast
    ultimately have "s \<alpha> k \<in> P \<alpha>" using b27 b28 by blast
    moreover have "(a, s \<alpha> k) \<in> (R \<alpha>)^*" using d1 d2 by simp
    ultimately show "\<exists> b \<in> P \<alpha>. (a, b) \<in> (R \<alpha>)^*" by blast
  qed
  ultimately show "P \<alpha> \<in> SCF (R \<alpha>)" unfolding SCF_def by blast
qed
have b32: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> K \<alpha> \<in> SCF (R \<alpha>) \<inter> Inv
    (R \<alpha>)"
proof
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have "\<forall>a\<in>Field (R \<alpha>). \<exists>b\<in>K \<alpha>. (a, b) \<in> (R \<alpha>)^*"
  proof
    fix a
    assume d1: "a \<in> Field (R \<alpha>)"
    show "\<exists>b\<in>K \<alpha>. (a, b) \<in> (R \<alpha>)^*"
    proof (cases "h \<alpha> \<in> Field (R \<alpha>)")
      assume "h \<alpha> \<in> Field (R \<alpha>)"
      moreover have "CCR (R \<alpha>)" using c1 b1 a3 by blast
      ultimately obtain a' where "a' \<in> Field (R \<alpha>)"
                             and e1: "(a,a') \<in> (R \<alpha>)^* \<and> (h \<alpha>, a') \<in> (R \<alpha>)
    ^*"
        using d1 unfolding CCR_def by blast
      then obtain b where e2: "(a', b) \<in> (R \<alpha>)" using c1 b1 a3 by blast
      then have "b \<in> Field (R \<alpha>)" unfolding Field_def by blast
      moreover have "(h \<alpha>, b) \<in> (R \<alpha>)^*" using e1 e2 by force
      moreover have "(b, h \<alpha>) \<in> (R \<alpha>)^* \<longrightarrow> False"
      proof
        assume "(b, h \<alpha>) \<in> (R \<alpha>)^*"
        then have "(b, b) \<in> (R \<alpha>)^+" using e1 e2 by fastforce
        then show "False" using c1 b1 a3 unfolding acyclic_def by blast
      qed
      moreover have "(a, b) \<in> (R \<alpha>)^*" using e1 e2 by force
      ultimately show ?thesis using b29 c1 a10 by blast
    next
      assume "h \<alpha> \<notin> Field (R \<alpha>)"
      then have "(a, h \<alpha>) \<notin> (R \<alpha>)^* \<and> h \<alpha> \<notin> W \<alpha>" using d1 c1
    a10 lem_rtr_field[of a] by blast
      then have "a \<in> K \<alpha>" using d1 b29 c1 a10 by blast
      then show ?thesis by blast
```

```
    qed
  qed
  then show "K \<alpha> \<in> SCF (R \<alpha>)" using b29 c1 a10 unfolding SCF_def by blast
next
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have "\<forall> a b. a \<in> K \<alpha> \<and> (a,b) \<in> (R \<alpha>) \<longrightarrow> b \<in> K \<alpha
    >"
  proof (intro allI impI)
    fix a b
    assume d1: "a \<in> K \<alpha> \<and> (a,b) \<in> (R \<alpha>)"
    then have d3: "a \<in> Field (R \<alpha>)" and d4: "(a, h \<alpha>) \<notin> (R \<alpha>)\<^sup>*" using
     b29 c1 a10 by blast+
    have "b \<in> Field (R \<alpha>)" using d1 unfolding Field_def by blast
    moreover have "h \<alpha> \<in> W \<alpha> \<longrightarrow> (h \<alpha>, b) \<in> (R \<alpha>)^*" using
     d1 b29 by force
    moreover have "(b, h \<alpha>) \<in> (R \<alpha>)^* \<longrightarrow> False"
    proof
      assume "(b, h \<alpha>) \<in> (R \<alpha>)^*"
      then have "(a, h \<alpha>) \<in> (R \<alpha>)^*" using d1 by force
      then show "False" using d4 by blast
    qed
    ultimately show "b \<in> K \<alpha>" using b29 c1 a10 by blast
  qed
  then show "K \<alpha> \<in> Inv (R \<alpha>)" using b29 unfolding Inv_def by blast
qed
have b33: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> ?F \<alpha> \<in> SCF (R \<alpha>)"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have "K \<alpha> \<in> SCF (R \<alpha>) \<inter> Inv (R \<alpha>)" using c1 b31 b32 unfolding Inv_def by
    blast+
  moreover have "P \<alpha> \<in> SCF (R \<alpha>)" using c1 b31 b32 lem_scfinv_scf_int by blast
  ultimately have "K \<alpha> \<inter> P \<alpha> \<in> SCF (R \<alpha>)" using lem_scfinv_scf_int by blast
  moreover have "?F \<alpha> = K \<alpha> \<inter> P \<alpha>" by blast
  ultimately show "?F \<alpha> \<in> SCF (R \<alpha>)" by metis
qed
define rei where "rei = (\<lambda> \<alpha>. SOME k. k \<in> E \<alpha> \<and> (s \<alpha> k) \<in> ?F \<alpha
    >)"
define re0 where "re0 = (\<lambda> \<alpha>. s \<alpha> (rei \<alpha>))"
define re1 where "re1 = (\<lambda> \<alpha>. s \<alpha> (Suc (rei \<alpha>)))"
define ep where "ep = (\<lambda> \<alpha>. SOME b. (re1 \<alpha>, b) \<in> (Restr r (W \<alpha>))^* \<and> b
    \<in> EP \<alpha>)"
define spl where "spl = (\<lambda> \<alpha>. spthlen (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<alpha>))"
define sp where "sp = (\<lambda> \<alpha>. SOME f. f \<in> spth (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<
    alpha>))"
define R0 where "R0 = (\<lambda> \<alpha>. { (a,b) \<in> R \<alpha>. (b, re0 \<alpha>) \<in> (R \<alpha>)^* })
    "
define R2 where "R2 = (\<lambda> \<alpha>. { (a,b). \<exists> k < (spl \<alpha>). a = sp \<alpha> k \<and> b =
     sp \<alpha> (Suc k) })"
define R' where "R' = (\<lambda> \<alpha>. R0 \<alpha> \<union> R2 \<alpha> \<union> { (re0 \<alpha>, re1 \<
    alpha>) })"
define re' where "re' = ({ (a,b) \<in> r. \<exists> \<alpha> \<in> S. \<exists> \<beta> \<in> S. \<alpha> <o
    \<beta> \<and> a = ep \<alpha> \<and> b \<in> W \<beta> \<and> (b, h \<beta>) \<in> (R \<beta>)^* })"
define r' where "r' = (re' \<union> (\<Union>\<alpha>\<in>S. R' \<alpha>))"

have b_Fne: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> ?F \<alpha> \<noteq> {}"
proof -
  fix \<alpha>
  assume "\<alpha> \<in> S"
  then have "?F \<alpha> \<in> SCF (R \<alpha>) \<and> R \<alpha> \<noteq> {}" using b33 a3 a10 by blast
  then show "?F \<alpha> \<noteq> {}" unfolding SCF_def Field_def by force
qed
have b_re0: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> re0 \<alpha> \<in> ?F \<alpha> \<and> rei \<
    alpha> \<in> E \<alpha>"
proof -
  fix \<alpha>
  assume "\<alpha> \<in> S"
  then obtain k where "k \<in> E \<alpha> \<and> (s \<alpha> k) \<in> ?F \<alpha>" using b_Fne b28 by force
  then have "(s \<alpha> (rei \<alpha>)) \<in> ?F \<alpha> " and "rei \<alpha> \<in> E \<alpha>"
    using someI_ex[of "\<lambda> k. k \<in> E \<alpha> \<and> s \<alpha> k \<in> P \<alpha> \<inter> K \<alpha
    >"] unfolding rei_def by metis+
  then show "re0 \<alpha> \<in> ?F \<alpha> \<and> rei \<alpha> \<in> E \<alpha>" unfolding re0_def by blast
qed
have b_rs: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> s \<alpha> ` UNIV \<subseteq> W \<alpha>"
proof -
  fix \<alpha>
  assume "\<alpha> \<in> S"
```

```
  then have "cfseq (R \<alpha>) (s \<alpha>) \<and> Field (R \<alpha>) = W \<alpha>" using b9 a3 a10 by blast
  then show "s \<alpha> ` UNIV \<subseteq> W \<alpha>" using lem_rseq_rtr unfolding cfseq_def by blast
qed
have b_injs: "\<And> \<alpha> k1 k2. \<alpha> \<in> S \<Longrightarrow> s \<alpha> k1 = s \<alpha> k2 \<
    Longrightarrow> k1 = k2"
proof -
  fix \<alpha> k1 k2
  assume "\<alpha> \<in> S" and "s \<alpha> k1 = s \<alpha> k2"
  moreover then have "cfseq (R \<alpha>) (s \<alpha>) \<and> acyclic (R \<alpha>)" using b9 a3 a10 by blast
  moreover then have "inj (s \<alpha>)" using lem_cfseq_inj by blast
  ultimately show "k1 = k2" unfolding inj_on_def by blast
qed
have b_re1: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> re1 \<alpha> = s \<alpha> (Suc (rei \<alpha>)
    )"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  then have "re0 \<alpha> \<in> ?F \<alpha>" using b_re0[of \<alpha>] by blast
  then obtain k where c2: "re0 \<alpha> = s \<alpha> k \<and> k \<in> E \<alpha>" unfolding b28 by blast
  then have "(s \<alpha> (Suc k), s \<alpha> k) \<notin> (Restr r (W \<alpha>))^*" unfolding b27 by blast
  have "rei \<alpha> = k" using c1 c2 b_injs unfolding re0_def by blast
  moreover have "re1 \<alpha> = s \<alpha> (Suc (rei \<alpha>))" unfolding re1_def by blast
  ultimately show "re1 \<alpha> = s \<alpha> (Suc (rei \<alpha>))" by blast
qed
have b_re12: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> (re0 \<alpha>, re1 \<alpha>) \<in> R \<alpha
    > \<and> (re1 \<alpha>, re0 \<alpha>) \<notin> (Restr r (W \<alpha>))^*"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  then have "re0 \<alpha> = s \<alpha> (rei \<alpha>)" and "re1 \<alpha> = s \<alpha> (Suc (rei \<alpha>))"
        and "cfseq (R \<alpha>) (s \<alpha>)" using b9 b_re1 re0_def by blast+
  then have "(re0 \<alpha>, re1 \<alpha>) \<in> R \<alpha>" unfolding cfseq_def by simp
  moreover have "(re1 \<alpha>, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^* \<longrightarrow> False"
  proof
    assume "(re1 \<alpha>, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^*"
    then have "(s \<alpha> (Suc (rei \<alpha>)), s \<alpha> (rei \<alpha>)) \<in> (Restr r (W \<alpha>))^*"
      using c1 b_re1[of \<alpha>] unfolding re0_def by metis
    moreover have "(s \<alpha> (Suc (rei \<alpha>)), s \<alpha> (rei \<alpha>)) \<notin> (Restr r (W \<alpha>)
    )^*"
      using c1 b_re0[of \<alpha>] b27 by blast
    ultimately show "False" by blast
  qed
  ultimately show "(re0 \<alpha>, re1 \<alpha>) \<in> R \<alpha> \<and> (re1 \<alpha>, re0 \<alpha>) \<notin>
    (Restr r (W \<alpha>))^*" by blast
qed
have b_rw: "\<And> \<alpha> a b. \<alpha> \<in> S \<Longrightarrow> a \<in> W \<alpha> \<Longrightarrow> (a,b)
     \<in> (Restr r (W \<alpha>))^* \<Longrightarrow> b \<in> W \<alpha>"
proof -
  fix \<alpha> a b
  assume "\<alpha> \<in> S" and "a \<in> W \<alpha>" and "(a,b) \<in> (Restr r (W \<alpha>))^*"
  then show "b \<in> W \<alpha>" using lem_Inv_restr_rtr2[of _ "Restr r (W \<alpha>)"] unfolding Inv_def by
    blast
qed
have b_r0w: "\<And> \<alpha> a b. \<alpha> \<in> S \<Longrightarrow> a \<in> W \<alpha> \<Longrightarrow> (a,b
    ) \<in> (R \<alpha>)^* \<Longrightarrow> b \<in> W \<alpha>"
  using p1 b_rw rtrancl_mono by blast
have b_ep: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> (re1 \<alpha>, ep \<alpha>) \<in> (Restr r (W
    \<alpha>))^* \<and> ep \<alpha> \<in> EP \<alpha>"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  moreover then have c2: "re1 \<alpha> \<in> W \<alpha>" using b_rs[of \<alpha>] b_re1[of \<alpha>] by blast
  ultimately obtain b
    where c3: "(re1 \<alpha>, b) \<in> (Restr r (W \<alpha>))^* \<and> (\<forall>\<beta>\<in>S. \<alpha> <o \<
    beta> \<longrightarrow> r``{b} \<inter> W \<beta> \<noteq> {})"
    using a11[of \<alpha> "re1 \<alpha>"] by blast
  then have "b \<in> W \<alpha>" using c1 c2 b_rw[of \<alpha>] by blast
  moreover obtain L where c4: "L = (\<lambda> b. (re1 \<alpha>, b) \<in> (Restr r (W \<alpha>))^* \<and> b \<
    in> EP \<alpha>)" by blast
  ultimately have "L b" and "ep \<alpha> = (SOME b. L b)" using c3 unfolding EP_def ep_def by blast+
  then have "L (ep \<alpha>)" using someI_ex by metis
  then show "(re1 \<alpha>, ep \<alpha>) \<in> (Restr r (W \<alpha>))^* \<and> ep \<alpha> \<in> EP \<alpha>"
    using c4 by blast
qed
have b_sp: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> sp \<alpha> \<in> spth (Restr r (W \<alpha>))
    (re1 \<alpha>) (ep \<alpha>)"
proof -
  fix \<alpha>
```

```
  assume "\<alpha> \<in> S"
  then have "(re1 \<alpha>, ep \<alpha>) \<in> (Restr r (W \<alpha>))^*" using b_ep by blast
  then obtain f where "f \<in> spth (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<alpha>)"
    using lem_spthlen_rtr lem_rtn_rpth_inj unfolding spth_def by metis
  then show "sp \<alpha> \<in> spth (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<alpha>)"
    unfolding sp_def using someI_ex by metis
qed
have b_R0: "\<And> \<alpha> a. \<alpha> \<in> S \<Longrightarrow> (a,re0 \<alpha>) \<in> (R \<alpha>)^* \<
    Longrightarrow> (a,re0 \<alpha>) \<in> (R0 \<alpha>)^*"
proof -
  fix \<alpha> a
  assume "\<alpha> \<in> S" and "(a,re0 \<alpha>) \<in> (R \<alpha>)^*"
  then obtain g n where "g \<in> rpth (R \<alpha>) a (re0 \<alpha>) n" using lem_ccext_rtr_rpth[of a "re0 \<
    alpha>"] by blast
  then have c1: "g 0 = a \<and> g n = re0 \<alpha>" and c2: "\<forall>i<n. (g i, g (Suc i)) \<in> R \<alpha>"
    unfolding rpth_def by blast+
  then have "\<forall> i\<le>n. (g i, re0 \<alpha>) \<in> (R \<alpha>)^*" using lem_rseq_tl by metis
  then have "\<forall> i<n. (g i, g (Suc i)) \<in> R0 \<alpha>" using c2 unfolding R0_def by simp
  then show "(a, re0 \<alpha>) \<in> (R0 \<alpha>)^*"
    using c1 lem_ccext_rpth_rtr[of "R0 \<alpha>" a "re0 \<alpha>" n] unfolding rpth_def by blast
qed
have b_hr0: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> h \<alpha> \<in> W \<alpha> \<Longrightarrow>
     (h \<alpha>, re0 \<alpha>) \<in> (R0 \<alpha>)^*"
  using b_re0 b_R0 b29 by blast
have b_hf: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> h \<alpha> \<in> W \<alpha> \<Longrightarrow>
    h \<alpha> \<in> Field r'"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S" and "h \<alpha> \<in> W \<alpha>"
  then have "(h \<alpha>, re0 \<alpha>) \<in> (R0 \<alpha>)^*" using c1 b_hr0 by blast
  moreover have "R0 \<alpha> \<subseteq> R' \<alpha>" using c1 unfolding R'_def by blast
  ultimately have "(h \<alpha>, re0 \<alpha>) \<in> (R' \<alpha>)^*" using rtrancl_mono by blast
  moreover have "re0 \<alpha> \<in> Field (R' \<alpha>)" unfolding R'_def Field_def by blast
  ultimately have "h \<alpha> \<in> Field (R' \<alpha>)" using lem_rtr_field[of "h \<alpha>" "re0 \<alpha>"]
    by force
  moreover have "R' \<alpha> \<subseteq> r'" using c1 unfolding r'_def by blast
  ultimately show "h \<alpha> \<in> Field r'" unfolding Field_def by blast
qed
have b_fR': "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> Field (R' \<alpha>) \<subseteq> W \<alpha>"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  then have "Field (R0 \<alpha>) \<subseteq> W \<alpha>" using a10 unfolding R0_def Field_def by blast
  moreover have "Field (R2 \<alpha>) \<subseteq> W \<alpha>"
  proof
    fix a
    assume "a \<in> Field (R2 \<alpha>)"
    then obtain x y where d1: "(x,y) \<in> R2 \<alpha> \<and> (a = x \<or> a = y)" unfolding Field_def by
    blast
    then obtain k where "k < spl \<alpha> \<and> (x,y) = (sp \<alpha> k, sp \<alpha> (Suc k))" unfolding
    R2_def by blast
    then show "a \<in> W \<alpha>" using d1 c1 b_sp[of \<alpha>] unfolding spth_def rpth_def spl_def by blast
  qed
  moreover have "re0 \<alpha> \<in> W \<alpha>" using c1 b_re0[of \<alpha>] b29 by blast
  moreover have "re1 \<alpha> \<in> W \<alpha>" using c1 b_re12[of \<alpha>] a10[of \<alpha>] unfolding
    Field_def by blast
  ultimately show "Field (R' \<alpha>) \<subseteq> W \<alpha>" unfolding R'_def Field_def by fast
qed
have b_fR2: "\<And> \<alpha> a. \<alpha> \<in> S \<Longrightarrow> a \<in> Field (R2 \<alpha>) \<
    Longrightarrow> \<exists> k. k \<le> spl \<alpha> \<and> a = sp \<alpha> k"
proof -
  fix \<alpha> a
  assume "\<alpha> \<in> S" and "a \<in> Field (R2 \<alpha>)"
  then obtain x y where "(x,y) \<in> R2 \<alpha> \<and> (a = x \<or> a = y)" unfolding Field_def by blast
  moreover then obtain k' where "k' < spl \<alpha> \<and> x = sp \<alpha> k' \<and> y = sp \<alpha> (Suc k')"
    unfolding R2_def by blast
  ultimately show "\<exists> k. k \<le> spl \<alpha> \<and> a = sp \<alpha> k" by (metis Suc_leI
    less_or_eq_imp_le)
qed
have b_bhf: "\<And> \<alpha> a. \<alpha> \<in> S \<Longrightarrow> a \<in> W \<alpha> \<Longrightarrow> (a, h
    \<alpha>) \<in> (R \<alpha>)^* \<Longrightarrow> a \<in> Field (R' \<alpha>)"
proof -
  fix \<alpha> a
  assume c1: "\<alpha> \<in> S" and c2: "a \<in> W \<alpha>" and c3: "(a, h \<alpha>) \<in> (R \<alpha>)^*"
  then have "(h \<alpha>, re0 \<alpha>) \<in> (R0 \<alpha>)^*" using b_hr0[of \<alpha>] b_r0w[of \<alpha>] by
    blast
  moreover have "R0 \<alpha> \<subseteq> R \<alpha>" unfolding R0_def by blast
  ultimately have "(h \<alpha>, re0 \<alpha>) \<in> (R \<alpha>)^*" using c3 rtrancl_mono by blast
```

```
  then have "(a, re0 \<alpha>) \<in> (R \<alpha>)^*" using c3 by force
  then have "(a, re0 \<alpha>) \<in> (R0 \<alpha>)^*" using c1 c3 b_R0[of \<alpha>] by blast
  moreover have "R0 \<alpha> \<subseteq> R' \<alpha>" unfolding R'_def by blast
  ultimately have "(a, re0 \<alpha>) \<in> (R' \<alpha>)^*" using rtrancl_mono by blast
  moreover have "re0 \<alpha> \<in> Field (R' \<alpha>)" unfolding R'_def Field_def by blast
  ultimately show "a \<in> Field (R' \<alpha>)" using lem_rtr_field[of a "re0 \<alpha>"] by blast
qed
have b_clR': "\<And> \<alpha> a. \<alpha> \<in> S \<Longrightarrow> a \<in> Field (R' \<alpha>) \<
    Longrightarrow> (a, ep \<alpha>) \<in> (R' \<alpha>)^*"
proof -
  fix \<alpha> a
  assume c1: "\<alpha> \<in> S" and c2: "a \<in> Field (R' \<alpha>)"
  have c3: "sp \<alpha> 0 = re1 \<alpha>" using c1 b_sp[of \<alpha>] unfolding spth_def spl_def rpth_def by
    blast
  then have "a \<in> Field (R2 \<alpha>) \<or> a = re1 \<alpha> \<longrightarrow> (\<exists> k. k \<le> spl \<
    alpha> \<and> a = sp \<alpha> k)" using c1 b_fR2 by force
  moreover have "a \<in> Field (R0 \<alpha>) \<or> a = re0 \<alpha> \<longrightarrow> (a, re0 \<alpha>) \<in>
    (R \<alpha>)^*"
    unfolding R0_def Field_def by fastforce
  moreover have "a \<in> Field (R0 \<alpha>) \<or> a \<in> Field (R2 \<alpha>) \<or> a = re0 \<alpha> \<or> a
    = re1 \<alpha>"
    using c1 c2 unfolding R'_def Field_def by blast
  moreover have c4: "\<forall> k. (k \<le> spl \<alpha> \<longrightarrow> (sp \<alpha> k, ep \<alpha>) \<in> (
    R' \<alpha>)^*)"
  proof (intro allI impI)
    fix k
    assume "k \<le> spl \<alpha>"
    moreover have "sp \<alpha> (spl \<alpha>) = ep \<alpha>"
      using c1 b_sp[of \<alpha>] unfolding spth_def spl_def rpth_def by blast
    moreover have "\<forall> i < spl \<alpha>. (sp \<alpha> i, sp \<alpha> (Suc i)) \<in> R' \<alpha>"
      unfolding R'_def R2_def by blast
    ultimately show "(sp \<alpha> k, ep \<alpha>) \<in> (R' \<alpha>)^*" using lem_rseq_tl by metis
  qed
  moreover have "(a, re0 \<alpha>) \<in> (R \<alpha>)^* \<longrightarrow> (a, ep \<alpha>) \<in> (R' \<alpha>)
    ^*"
  proof
    assume "(a, re0 \<alpha>) \<in> (R \<alpha>)^*"
    then have "(a, re0 \<alpha>) \<in> (R0 \<alpha>)^*" using c1 b_R0 by blast
    moreover have "R0 \<alpha> \<subseteq> R' \<alpha>" using c1 unfolding R'_def by blast
    ultimately have "(a, re0 \<alpha>) \<in> (R' \<alpha>)^*" using rtrancl_mono by blast
    moreover have "(re0 \<alpha>, re1 \<alpha>) \<in> (R' \<alpha>)" using c1 unfolding R'_def by blast
    moreover have "(re1 \<alpha>, ep \<alpha>) \<in> (R' \<alpha>)^*" using c3 c4 by force
    ultimately show "(a, ep \<alpha>) \<in> (R' \<alpha>)^*" by simp
  qed
  ultimately show "(a, ep \<alpha>) \<in> (R' \<alpha>)^*" by blast
qed
have b_epr': "\<And> a. a \<in> Field r' \<Longrightarrow> \<exists> \<alpha> \<in> S. (a, ep \<alpha>) \<in>
    (R' \<alpha>)^*"
proof -
  fix a
  assume "a \<in> Field r'"
  then have "a \<in> Field re' \<or> (\<exists> \<alpha>\<in>S. a \<in> Field (R' \<alpha>))" unfolding r'_def
     Field_def by blast
  moreover have "a \<in> Field re' \<longrightarrow> (\<exists> \<alpha> \<in> S. (a, ep \<alpha>) \<in> (R'
    \<alpha>)^*)"
  proof
    assume "a \<in> Field re'"
    then obtain x y \<alpha> \<beta> where d1: "a = x \<or> a = y" and d2: "\<alpha> \<in> S \<and> \<beta> \<
    in> S \<and> \<alpha> <o \<beta>"
                        and d3: "x = ep \<alpha> \<and> y \<in> W \<beta> \<and> (y, h \<beta>) \<in> (R \<
    beta>)^*"
      unfolding re'_def Field_def by blast
    have "(x, ep \<alpha>) \<in> (R' \<alpha>)^*" using d3 by blast
    moreover have "(y, ep \<beta>) \<in> (R' \<beta>)^*" using d2 d3 b_bhf[of \<beta> y] b_clR'[of \<beta>] by
     blast
    ultimately show "\<exists> \<alpha> \<in> S. (a, ep \<alpha>) \<in> (R' \<alpha>)^*" using d1 d2 by blast
  qed
  ultimately show "\<exists> \<alpha> \<in> S. (a, ep \<alpha>) \<in> (R' \<alpha>)^*" using b_clR' by blast
qed
have b_svR': "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> single_valued (R' \<alpha>)"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have c2: "re0 \<alpha> \<in> Domain (R0 \<alpha>) \<longrightarrow> False"
  proof
    assume "re0 \<alpha> \<in> Domain (R0 \<alpha>)"
    then obtain b where "(re0 \<alpha>, b) \<in> R0 \<alpha>" by blast
```

```
    then have "(re0 \<alpha>, b) \<in> R \<alpha> \<and> (b, re0 \<alpha>) \<in> (R \<alpha>)^*" unfolding
    R0_def by blast
    then have "(re0 \<alpha>, re0 \<alpha>) \<in> (R \<alpha>)^+" by force
    moreover have "acyclic (R \<alpha>)" using c1 a10 a3 by blast
    ultimately show "False" unfolding acyclic_def by blast
  qed
  have c3: "re0 \<alpha> \<in> Domain (R2 \<alpha>) \<longrightarrow> False"
  proof
    assume "re0 \<alpha> \<in> Domain (R2 \<alpha>)"
    then obtain b where "(re0 \<alpha>, b) \<in> R2 \<alpha>" by blast
    then obtain k where d1: "k \<le> spl \<alpha> \<and> re0 \<alpha> = sp \<alpha> k \<and> b = sp \<alpha> (
    Suc k)"
      unfolding R2_def by force
    have "sp \<alpha> \<in> spth (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<alpha>)" using c1 b_sp by blast
    then have "sp \<alpha> 0 = re1 \<alpha>" and "\<forall>i<spl \<alpha>. (sp \<alpha> i, sp \<alpha> (Suc i)
    ) \<in> Restr r (W \<alpha>)"
      unfolding spth_def spl_def rpth_def by blast+
    then have "(re1 \<alpha>, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^*" using d1 lem_rseq_hd by metis
    then show "False" using c1 b_re12[of \<alpha>] by blast
  qed
  have c4: "\<forall> a \<in> Field (R0 \<alpha>) \<inter> Field (R2 \<alpha>). False"
  proof
    fix a
    assume d1: "a \<in> Field (R0 \<alpha>) \<inter> Field (R2 \<alpha>)"
    obtain k where d2: "k \<le> spl \<alpha> \<and> a = sp \<alpha> k" using d1 c1 b_fR2[of \<alpha> a] by
    blast
    have "sp \<alpha> \<in> spth (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<alpha>)" using c1 b_sp by blast
    then have "sp \<alpha> 0 = re1 \<alpha>" and "\<forall>i<spl \<alpha>. (sp \<alpha> i, sp \<alpha> (Suc i)
    ) \<in> Restr r (W \<alpha>)"
      unfolding spth_def spl_def rpth_def by blast+
    then have d3: "(re1 \<alpha>, a) \<in> (Restr r (W \<alpha>))^*"
      using d2 lem_rseq_hd unfolding spth_def rpth_def by metis
    have "(a, re0 \<alpha>) \<in> (R \<alpha>)^*" using d1 unfolding R0_def Field_def by force
    moreover have "R \<alpha> \<subseteq> Restr r (W \<alpha>)" using c1 a10 unfolding Field_def by fastforce
    ultimately have "(a, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^*" using rtrancl_mono by blast
    then have "(re1 \<alpha>, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^*" using d3 by force
    then show "False" using c1 b_re12[of \<alpha>] by blast
  qed
  have "R0 \<alpha> \<subseteq> R \<alpha>" unfolding R0_def by blast
  then have c5: "single_valued (R0 \<alpha>)" using c1 a3 a10[of \<alpha>] unfolding single_valued_def by
    blast
  have c6: "\<forall> a b c. (a,b) \<in> R2 \<alpha> \<and> (a,c) \<in> R2 \<alpha> \<longrightarrow> b = c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> R2 \<alpha> \<and> (a,c) \<in> R2 \<alpha>"
    then obtain k1 k2 where d1: "k1 < spl \<alpha> \<and> a = sp \<alpha> k1 \<and> b = sp \<alpha> (Suc k1)"
                        and d2: "k2 < spl \<alpha> \<and> a = sp \<alpha> k2 \<and> c = sp \<alpha> (Suc k2)"
      unfolding R2_def by blast
    then have "sp \<alpha> k1 = sp \<alpha> k2 \<and> k1 \<le> spl \<alpha> \<and> k2 \<le> spl \<alpha>" by
    force
    moreover have "inj_on (sp \<alpha>) {i. i\<le>spl \<alpha>}"
      using c1 b_sp[of \<alpha>] lem_spth_inj[of "sp \<alpha>"] unfolding spl_def by blast
    ultimately have "k1 = k2" unfolding inj_on_def by blast
    then show "b = c" using d1 d2 by blast
  qed
  have "single_valued (R0 \<alpha> \<union> {(re0 \<alpha>, re1 \<alpha>)})"
    using c2 c5 unfolding single_valued_def by blast
  moreover have "single_valued (R2 \<alpha> \<union> {(re0 \<alpha>, re1 \<alpha>)})"
    using c3 c6 unfolding single_valued_def by blast
  ultimately show "single_valued (R' \<alpha>)" using c4 lem_sv_un3 unfolding R'_def by blast
qed
have b_acR': "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> acyclic (R' \<alpha>)"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  obtain s where c2: "s = R0 \<alpha> \<union> {(re0 \<alpha>, re1 \<alpha>)}" by blast
  then have "s \<subseteq> R \<alpha>" using c1 b_re12[of \<alpha>] unfolding R0_def by blast
  moreover have "acyclic (R \<alpha>)" using c1 a3 a10 by blast
  ultimately have "acyclic s" using acyclic_subset by blast
  moreover have "acyclic (R2 \<alpha>)"
  proof -
    have "\<forall> a. (a,a) \<in> (R2 \<alpha>)^+ \<longrightarrow> False"
    proof (intro allI impI)
      fix a
      assume "(a,a) \<in> (R2 \<alpha>)^+"
      then obtain n where e1: "n > 0 \<and> (a,a) \<in> (R2 \<alpha>)^^n" using trancl_power by blast
      then obtain g where e2: "g 0 = a \<and> g n = a" and e3: "\<forall> i<n. (g i, g (Suc i)) \<in> R2 \<
    alpha>"
```

```
      using relpow_fun_conv[of a a n "R2 \<alpha>"] by blast
    then have "(g 0, g (Suc 0)) \<in> R2 \<alpha>" using e1 by force
    then obtain k0 where e4: "k0 < spl \<alpha> \<and> g 0 = sp \<alpha> k0" unfolding R2_def by blast
    have e5: "inj_on (sp \<alpha>) {i. i\<le>spl \<alpha>}"
      using c1 b_sp[of \<alpha>] lem_spth_inj[of "sp \<alpha>"] unfolding spl_def by blast
    have "\<forall> i\<le>n. k0 + i \<le> spl \<alpha> \<and> g i = sp \<alpha> (k0 + i)"
    proof
      fix i
      show "i \<le> n \<longrightarrow> k0 + i \<le> spl \<alpha> \<and> g i = sp \<alpha> (k0 + i)"
      proof (induct i)
        show "0 \<le> n \<longrightarrow> k0 + 0 \<le> spl \<alpha> \<and> g 0 = sp \<alpha> (k0 + 0)" using
   e4 by simp
      next
        fix i
        assume g1: "i \<le> n \<longrightarrow> k0 + i \<le> spl \<alpha> \<and> g i = sp \<alpha> (k0 + i)"
        show "Suc i \<le> n \<longrightarrow> k0 + Suc i \<le> spl \<alpha> \<and> g (Suc i) = sp \<alpha> (
  k0 + Suc i)"
        proof
          assume h1: "Suc i \<le> n"
          then have h2: "k0 + i \<le> spl \<alpha> \<and> g i = sp \<alpha> (k0 + i)" using g1 by simp
          moreover have "(g i, g (Suc i)) \<in> R2 \<alpha>" using h1 e3 by simp
          ultimately obtain k where
            h3: "k < spl \<alpha> \<and> sp \<alpha> (k0 + i) = sp \<alpha> k \<and> g (Suc i) = sp \<alpha>
   (Suc k)"
            unfolding R2_def by fastforce
          then have h4: "k0 + i = k" using h2 h3 e5 unfolding inj_on_def by simp
          then have "k0 + Suc i \<le> spl \<alpha>" using h3 by simp
          moreover have "g (Suc i) = sp \<alpha> (k0 + Suc i)" using h3 h4 by simp
          ultimately show "k0 + Suc i \<le> spl \<alpha> \<and> g (Suc i) = sp \<alpha> (k0 + Suc i)" by
  blast
        qed
      qed
    qed
    then have "k0 + n \<le> spl \<alpha> \<and> a = sp \<alpha> (k0 + n)" using e2 by simp
    moreover have "k0 \<le> spl \<alpha> \<and> a = sp \<alpha> k0" using e2 e4 by simp
    ultimately have "k0 + n = k0" using e5 unfolding inj_on_def by blast
    then show "False" using e1 by simp
  qed
  then show ?thesis unfolding acyclic_def by blast
qed
moreover have "\<forall> a \<in> (Range (R2 \<alpha>)) \<inter> (Domain s). False"
proof
  fix a
  assume e1: "a \<in> (Range (R2 \<alpha>)) \<inter> (Domain s)"
  then have e2: "a \<in> Field (R0 \<alpha>) \<or> a = re0 \<alpha>" using c2 unfolding Field_def by blast
  obtain k where e3: "k \<le> spl \<alpha> \<and> a = sp \<alpha> k" using e1 c1 b_fR2[of \<alpha> a]
   unfolding Field_def by blast
  have "sp \<alpha> \<in> spth (Restr r (W \<alpha>)) (re1 \<alpha>) (ep \<alpha>)" using c1 b_sp by blast
  then have "sp \<alpha> 0 = re1 \<alpha>" and "\<forall>i<spl \<alpha>. (sp \<alpha> i, sp \<alpha> (Suc i)
  ) \<in> Restr r (W \<alpha>)"
    unfolding spth_def spl_def rpth_def by blast+
  then have e4: "(re1 \<alpha>, a) \<in> (Restr r (W \<alpha>))^*"
    using e3 lem_rseq_hd unfolding spth_def rpth_def by metis
  have "(a, re0 \<alpha>) \<in> (R \<alpha>)^*" using e2 unfolding R0_def Field_def by force
  moreover have "R \<alpha> \<subseteq> Restr r (W \<alpha>)" using c1 a10 unfolding Field_def by fastforce
  ultimately have "(a, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^*" using rtrancl_mono by blast
  then have "(re1 \<alpha>, re0 \<alpha>) \<in> (Restr r (W \<alpha>))^*" using e4 by force
  then show "False" using c1 b_re12[of \<alpha>] by blast
qed
moreover have "R' \<alpha> = R2 \<alpha> \<union> s" using c2 unfolding R'_def by blast
ultimately show "acyclic (R' \<alpha>)" using lem_acyc_un_emprd[of "R2 \<alpha>" s] by force
qed
have b_dr': "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> Domain (R' \<alpha>) \<inter> Domain re' =
    {}"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have "\<forall> a b c. (a,b) \<in> (R' \<alpha>) \<and> (a,c) \<in> re' \<longrightarrow> False"
  proof (intro allI impI)
    fix a b c
    assume d1: "(a,b) \<in> (R' \<alpha>) \<and> (a,c) \<in> re'"
    then obtain \<alpha>' where d2: "\<alpha>' \<in> S \<and> a = ep \<alpha>'" unfolding re'_def by blast
    then have "a \<in> W \<alpha>'" using b_ep[of \<alpha>'] unfolding EP_def by blast
    moreover have "a \<in> W \<alpha>" using d1 c1 b_fR'[of \<alpha>] unfolding Field_def by blast
    ultimately have "\<alpha>' = \<alpha>" using d2 c1 a9 by blast
    then have "a = ep \<alpha>" using d2 by blast
    moreover have "(b, ep \<alpha>) \<in> (R' \<alpha>)^*" using d1 c1 b_clR' unfolding Field_def by blast
    ultimately have "(a, a) \<in> (R' \<alpha>)^+" using d1 by force
```

```
    then show "False" using c1 b_acR' unfolding acyclic_def by blast
  qed
  then show "Domain (R' \<alpha>) \<inter> Domain re' = {}" by blast
qed
have b_pkr': "\<And> a b1 b2. (a,b1) \<in> r' \<and> (a,b2) \<in> r' \<and> b1 \<noteq> b2 \<Longrightarrow>
    \<forall> b. (a,b) \<in> r' \<longrightarrow> (a,b) \<in> re'"
proof -
  fix a b1 b2
  assume c1: "(a,b1) \<in> r' \<and> (a,b2) \<in> r' \<and> b1 \<noteq> b2"
  moreover have "\<forall>\<alpha>\<in>S. \<forall>\<beta>\<in>S. (a,b1) \<in> R' \<alpha> \<and> (a,b2) \<in>
     R' \<beta> \<longrightarrow> False"
  proof (intro ballI impI)
    fix \<alpha> \<beta>
    assume "\<alpha> \<in> S" and "\<beta> \<in> S" and "(a,b1) \<in> R' \<alpha> \<and> (a,b2) \<in> R' \<
    beta>"
    moreover then have "\<alpha> = \<beta>" using b_fR'[of \<alpha>] b_fR'[of \<beta>] a9 unfolding Field_def
    by blast
    ultimately show "False" using c1 b_svR'[of \<alpha>] unfolding single_valued_def by blast
  qed
  ultimately have "(a,b1) \<in> re' \<or> (a,b2) \<in> re'" unfolding r'_def by blast
  then have "\<forall> \<alpha>\<in>S. a \<notin> Domain (R' \<alpha>)" using b_dr' by blast
  then show "\<forall> b. (a,b) \<in> r' \<longrightarrow> (a,b) \<in> re'" using c1 unfolding r'_def by blast
qed
have "r' \<subseteq> r"
proof
  fix p
  assume "p \<in> r'"
  moreover have "\<forall> \<alpha> \<in> S. p \<in> R' \<alpha> \<longrightarrow> p \<in> r"
  proof (intro ballI impI)
    fix \<alpha>
    assume d1: "\<alpha> \<in> S" and "p \<in> R' \<alpha>"
    moreover have "p \<in> R0 \<alpha> \<longrightarrow> p \<in> r" unfolding R0_def using d1 a10 by blast
    moreover have "p \<in> R2 \<alpha> \<longrightarrow> p \<in> r"
    proof
      assume "p \<in> R2 \<alpha>"
      then obtain k where "k<spl \<alpha> \<and> p = (sp \<alpha> k, sp \<alpha> (Suc k))" unfolding R2_def by
     blast
      then have "p \<in> Restr r (W \<alpha>)" using d1 b_sp[of \<alpha>] unfolding spth_def rpth_def spl_def
     by blast
      then show "p \<in> r" by blast
    qed
    moreover have "(re0 \<alpha>, re1 \<alpha>) \<in> r" using d1 b_re12 a10 by blast
    ultimately show "p \<in> r" unfolding R'_def by blast
  qed
  ultimately show "p \<in> r" unfolding r'_def re'_def by blast
qed
moreover have "\<forall>a\<in>Field r. \<exists>b\<in>Field r'. (a, b) \<in> r^*"
proof
  fix a
  assume "a \<in> Field r"
  then obtain \<alpha> where c1: "\<alpha> \<in> S \<and> a \<in> W \<alpha>" using a8 by blast
  then obtain a' where c2: "(a, a') \<in> (Restr r (W \<alpha>))^*"
                   and c3: "\<forall>\<beta>\<in>S. \<alpha> <o \<beta> \<longrightarrow> r``{a'} \<inter> W
    \<beta> \<noteq> {}" using a11[of \<alpha> a] by blast
  have "a' \<in> W \<alpha>" using c1 c2 lem_rtr_field[of a a'] unfolding Field_def by blast
  then have "a' \<in> EP \<alpha>" using c3 unfolding EP_def by blast
  then obtain \<gamma> a'' where c4: "\<gamma> \<in> S" and c5: "a'' \<in> W \<gamma> \<and> (a', a'') \<in> r
     \<and> (a'', h \<gamma>) \<in> (R \<gamma>)^*"
    using c1 b_h[of \<alpha> \<alpha> a' a'] by blast
  moreover then have "(a'', h \<gamma>) \<in> r^*" using p1 rtrancl_mono[of "R \<gamma>" r] by blast
  moreover have "(a, a') \<in> r^*" using c2 rtrancl_mono[of "Restr r (W \<alpha>)" r] by blast
  ultimately have "(a, h \<gamma>) \<in> r^*" by force
  moreover have "h \<gamma> \<in> W \<gamma>" using c4 c5 b_rOw by blast
  moreover then have "h \<gamma> \<in> Field r'" using c4 b_hf by blast
  ultimately show "\<exists>b\<in>Field r'. (a, b) \<in> r^*" by blast
qed
moreover have "DCR 2 r' \<and> CCR r'"
proof -
  obtain g0 where c1: "g0 = { (u,v) \<in> r'. r'``{u} = {v} }" by blast
  obtain g1 where c2: "g1 = r' - g0" by blast
  obtain g where c3: "g = (\<lambda>n::nat. (if (n=0) then g0 else (if (n=1) then g1 else {})))" by blast
  have c4: "\<forall> \<beta> \<in> S. R' \<beta> \<subseteq> g0"
  proof
    fix \<beta>
    assume d1: "\<beta> \<in> S"
    then have "R' \<beta> \<subseteq> r'" unfolding r'_def by blast
    moreover have "\<forall> a b c. (a,b) \<in> R' \<beta> \<and> (a,c) \<in> r' \<longrightarrow> b = c"
    proof (intro allI impI)
```

```
    fix a b c
    assume e1: "(a, b) \<in> R' \<beta> \<and> (a, c) \<in> r'"
    moreover then have "(a,b) \<in> r'" using d1 unfolding r'_def by blast
    ultimately have "b = c \<or> (a, b) \<in> re'" using b_pkr'[of a b c] by blast
    moreover have "(a,b) \<in> re' \<longrightarrow> False" using e1 d1 b_dr'[of \<beta>] by blast
    ultimately show "b = c" by blast
  qed
  ultimately show "R' \<beta> \<subseteq> g0" using c1 by blast
qed
have c5: "re' \<subseteq> g1"
proof -
  have "re' \<subseteq> r'" unfolding r'_def by blast
  moreover have "\<forall> a b. (a,b) \<in> re' \<and> (a,b) \<in> g0 \<longrightarrow> False"
  proof (intro allI impI)
    fix a b
    assume e1: "(a,b) \<in> re' \<and> (a,b) \<in> g0"
    then obtain \<alpha> where e2: "\<alpha> \<in> S \<and> a = ep \<alpha>" unfolding re'_def by blast
    then have e3: "a \<in> EP \<alpha>" using b_ep by blast
    obtain \<gamma>1 a1 where e4: "\<gamma>1 \<in> S \<and> \<alpha> <o \<gamma>1 \<and> a1 \<in> W \<gamma
  >1 \<and> (a,a1) \<in> re'"
      using e2 e3 b_h[of \<alpha> \<alpha> a a] b_bhf re'_def by blast
    then have "\<gamma>1 \<in> S \<and> ep \<gamma>1 \<in> EP \<gamma>1" using b_ep by blast
    then obtain \<gamma>2 a2 where e5: "\<gamma>2 \<in> S \<and> \<gamma>1 <o \<gamma>2 \<and> a2 \<in> W \<
  gamma>2 \<and> (a,a2) \<in> re'"
      using e2 e3 b_h[of \<alpha> \<gamma>1 a "ep \<gamma>1"] re'_def by blast
    then have "\<gamma>1 \<noteq> \<gamma>2" using ordLess_irrefl unfolding irrefl_def by blast
    then have "a1 \<noteq> a2" using e4 e5 a9 by blast
    moreover have "a1 \<in> r'``{a} \<and> a2 \<in> r'``{a}" using e4 e5 unfolding r'_def by blast
    moreover have "r'``{a} = {b}" using e1 c1 by blast
    ultimately have "a1 \<in> {b} \<and> a2 \<in> {b} \<and> a1 \<noteq> a2" by blast
    then show "False" by blast
  qed
  ultimately show ?thesis using c2 by force
qed
have "r' = \<Union>{r'. \<exists>\<alpha>'<2. r' = g \<alpha>'}"
proof
  have "r' \<subseteq> g0 \<union> g1" using c1 c2 by blast
  moreover have "g0 = g 0 \<and> g1 = g 1 \<and> (0::nat) < 2 \<and> (1::nat) < 2" using c3 by simp
  ultimately show "r' \<subseteq> \<Union>{r'. \<exists>\<alpha>'<2. r' = g \<alpha>'}" by blast
next
  have "\<And> \<alpha>. g \<alpha> \<subseteq> g0 \<union> g1" unfolding c3 by simp
  then show "\<Union>{r'. \<exists>\<alpha>'<2. r' = g \<alpha>'} \<subseteq> r'" using c1 c2 by blast
qed
moreover have "\<forall>l1 l2 u v w. l1 \<le> l2 \<longrightarrow> (u, v) \<in> g l1 \<and> (u, w) \<in> g
  l2 \<longrightarrow>
     (\<exists>v' v'' w' w'' d. (v, v', v'', d) \<in> \<DD> g l1 l2 \<and> (w, w', w'', d) \<in> \<DD> g l2
  l1)"
proof (intro allI impI)
  fix l1 l2 u v w
  assume d1: "l1 \<le> l2" and d2: "(u, v) \<in> g l1 \<and> (u, w) \<in> g l2"
  have d3: "g0 = g 0 \<and> g1 = g 1"
   and d4: "\<forall> \<alpha>. g \<alpha> \<noteq> {} \<longrightarrow> \<alpha> = 0 \<or> \<alpha> = 1"
  unfolding c3 by simp+
  have d5: "\<LL>1 g 1 = g0" and d6: "\<LL>v g 1 1 = g0"
   and d7: "\<LL>v g 1 0 = g0" and d8: "\<LL>v g 0 1 = g0" using d3 unfolding \<LL>1_def \<LL>v_def by blast
  +
  show "\<exists>v' v'' w' w'' d. (v, v', v'', d) \<in> \<DD> g l1 l2 \<and> (w, w', w'', d) \<in> \<DD> g
  l2 l1"
  proof -
    have "l1 = 0 \<and> l2 = 0 \<Longrightarrow> ?thesis"
    proof -
      assume "l1 = 0 \<and> l2 = 0"
      then have "r'``{u} = {v} \<and> r'``{u} = {w}" using c1 d2 d3 by blast
      then have "v = w" by blast
      then show ?thesis unfolding \<DD>_def by fastforce
    qed
    moreover have "l1 = 0 \<and> l2 = 1 \<Longrightarrow> False"
    proof -
      assume "l1 = 0 \<and> l2 = 1"
      then have "(u, v) \<in> r' \<and> (u, w) \<in> r'"
            and "r'``{u} = {v} \<and> r'``{u} \<noteq> {w}" using c1 c2 d2 d3 by blast+
      then show "False" by force
    qed
    moreover have "l1 = 1 \<and> l2 = 1 \<Longrightarrow> ?thesis"
    proof -
      assume f1: "l1 = 1 \<and> l2 = 1"
      then have "(u,v) \<in> g1 \<and> (u,w) \<in> g1" using d2 d3 by blast
      then have "(u,v) \<in> re' \<and> (u,w) \<in> re'" using c1 c2 b_pkr' by blast
```

```
        then obtain \<beta>1 \<beta>2 where f2: "\<beta>1 \<in> S \<and> \<beta>2 \<in> S"
          and "v \<in> W \<beta>1 \<and> (v, h \<beta>1) \<in> (R \<beta>1)^*"
          and "w \<in> W \<beta>2 \<and> (w, h \<beta>2) \<in> (R \<beta>2)^*" unfolding re'_def by blast
        then have "v \<in> Field (R' \<beta>1) \<and> w \<in> Field (R' \<beta>2)" using b_bhf by blast
        then have f3: "(v, ep \<beta>1) \<in> (R' \<beta>1)^* \<and> (w, ep \<beta>2) \<in> (R' \<beta>2)^*"
      using f2 b_clR' by blast
        then have "ep \<beta>1 \<in> EP \<beta>1 \<and> ep \<beta>2 \<in> EP \<beta>2" using f2 b_ep by blast
        then obtain \<gamma> v'' w'' where f4: "\<gamma> \<in> S \<and> \<beta>1 <o \<gamma> \<and> \<beta>2 <
      o \<gamma>"
                              and "v'' \<in> W \<gamma> \<and> (ep \<beta>1, v'') \<in> r \<and> (v'', h \<
      gamma>) \<in> (R \<gamma>)^*"
                              and "w'' \<in> W \<gamma> \<and> (ep \<beta>2, w'') \<in> r \<and> (w'', h \<
      gamma>) \<in> (R \<gamma>)^*"
            using f2 b_h[of \<beta>1 \<beta>2 "ep \<beta>1" "ep \<beta>2"] by blast
        then have "(ep \<beta>1, v'') \<in> re' \<and> (ep \<beta>2, w'') \<in> re'"
              and "(v'', ep \<gamma>) \<in> (R' \<gamma>)^* \<and> (w'', ep \<gamma>) \<in> (R' \<gamma>)^*"
          using f2 b_bhf b_clR' unfolding re'_def by blast+
        moreover obtain v' w' d where "v' = ep \<beta>1 \<and> w' = ep \<beta>2 \<and> d = ep \<gamma>" by
      blast
        ultimately have f5: "(v, v') \<in> (R' \<beta>1)^* \<and> (v', v'') \<in> re' \<and> (v'', d) \<in> (R
      ' \<gamma>)^*"
                    and f6: "(w, w') \<in> (R' \<beta>2)^* \<and> (w', w'') \<in> re' \<and> (w'', d) \<in> (R
      ' \<gamma>)^*"
                    using f3 by blast+
        have "(R' \<beta>1)^* \<subseteq> (\<LL>1 g l1)^*" using f1 f2 d5 c4 rtrancl_mono by blast
        moreover have "re' \<subseteq> g l2" using f1 d3 c5 by blast
        moreover have "(R' \<gamma>)^* \<subseteq> (\<LL>v g l1 l2)^*" using f1 f4 d6 c4 rtrancl_mono by blast
        moreover have "(R' \<beta>2)^* \<subseteq> (\<LL>1 g l2)^*" using f1 f2 d5 c4 rtrancl_mono by blast
        moreover have "re' \<subseteq> g l1" using f1 d3 c5 by blast
        moreover have "(R' \<gamma>)^* \<subseteq> (\<LL>v g l2 l1)^*" using f1 f4 d6 c4 rtrancl_mono by blast
        ultimately have "(v, v', v'', d) \<in> \<DD> g l1 l2 \<and> (w, w', w'', d) \<in> \<DD> g l2 l1 "
          using f5 f6 unfolding \<DD>_def by blast
        then show ?thesis by blast
      qed
      moreover have "(l1 = 0 \<or> l1 = 1) \<and> (l2 = 0 \<or> l2 = 1)" using d2 d4 by blast
      ultimately show ?thesis using d1 by fastforce
    qed
  qed
  ultimately have c9: "DCR 2 r'" using lem_Ldo_ldogen_ord unfolding DCR_def by blast
  have "\<forall>a\<in>Field r'. \<forall>b\<in>Field r'. \<exists>c \<in> Field r'. (a,c) \<in> r'^* \<and> (
    b,c) \<in> r'^*"
  proof (intro ballI impI)
    fix a b
    assume d1: "a \<in> Field r'" and d2: "b \<in> Field r'"
    obtain \<alpha> \<beta> where d3: "\<alpha> \<in> S \<and> \<beta> \<in> S"
      and d4: "(a, ep \<alpha>) \<in> (R' \<alpha>)^* \<and> (b, ep \<beta>) \<in> (R' \<beta>)^*"  using d1
    d2 b_epr' by blast
    then have "ep \<alpha> \<in> EP \<alpha> \<and> ep \<beta> \<in> EP \<beta>" using b_ep by blast
    then obtain \<gamma> a' b' where d5: "\<gamma> \<in> S \<and> \<alpha> <o \<gamma> \<and> \<beta> <o \<
    gamma>"
                          and d6: "a' \<in> W \<gamma> \<and> (ep \<alpha>, a') \<in> r \<and> (a', h \<gamma
    >) \<in> (R \<gamma>)^*"
                          and d7: "b' \<in> W \<gamma> \<and> (ep \<beta>, b') \<in> r \<and> (b', h \<gamma>)
     \<in> (R \<gamma>)^*"
      using d3 b_h[of \<alpha> \<beta> "ep \<alpha>" "ep \<beta>"] by blast
    then have "(a', ep \<gamma>) \<in> (R' \<gamma>)^* \<and> (b', ep \<gamma>) \<in> (R' \<gamma>)^*" using
    b_bhf b_clR' by blast
    moreover have "R' \<alpha> \<subseteq> r' \<and> R' \<beta> \<subseteq> r' \<and> R' \<gamma> \<subseteq>
    r'" using d3 d5 unfolding r'_def by blast
    ultimately have "(a, ep \<alpha>) \<in> r'^* \<and> (b, ep \<beta>) \<in> r'^*"
                and "(a', ep \<gamma>) \<in> r'^* \<and> (b', ep \<gamma>) \<in> r'^*" using d4 rtrancl_mono
    by blast+
    moreover have "(ep \<alpha>, a') \<in> r'" using d3 d5 d6 unfolding r'_def re'_def by blast
    moreover have "(ep \<beta>, b') \<in> r'" using d3 d5 d7 unfolding r'_def re'_def by blast
    ultimately have "(a, ep \<gamma>) \<in> r'^* \<and> (b, ep \<gamma>) \<in> r'^*" by force
    moreover then have "ep \<gamma> \<in> Field r'" using d1 lem_rtr_field by metis
    ultimately show "\<exists>c \<in> Field r'. (a,c) \<in> r'^* \<and> (b,c) \<in> r'^*" by blast
  qed
  then have "CCR r'" unfolding CCR_def by blast
  then show ?thesis using c9 by blast
 qed
 ultimately show ?thesis by blast
qed

lemma lem_uset_cl_ext:
fixes r::"'U rel" and s::"'U rel"
assumes "s \<in> \<UU> r" and "Conelike s"
shows "Conelike r"
```

```
proof (cases "s = {}")
  assume "s = {}"
  then have "r = {}" using assms unfolding \<UU>_def Field_def by fast
  then show "Conelike r" unfolding Conelike_def by blast
next
  assume "s \<noteq> {}"
  then obtain m where "m \<in> Field s \<and> (\<forall> a \<in> Field s. (a,m) \<in> s^*)" using assms
      unfolding Conelike_def by blast
  moreover have "s \<subseteq> r \<and> (\<forall> a \<in> Field r. \<exists> b \<in> Field s. (a,b) \<in> r^*)"
       using assms unfolding \<UU>_def by blast
  moreover then have "Field s \<subseteq> Field r \<and> s^* \<subseteq> r^*" unfolding Field_def using
      rtrancl_mono by blast
  ultimately have "(m \<in> Field r) \<and> (\<forall> a \<in> Field r. (a,m) \<in> r^*)" by (meson
      rtrancl_trans subsetCE)
  then show "Conelike r" unfolding Conelike_def by blast
qed

lemma lem_uset_cl_singleton:
fixes r::"'U rel"
assumes "Conelike r" and "r \<noteq> {}"
shows "\<exists> m::'U. \<exists> m'::'U. {(m',m)} \<in> \<UU> r"
proof -
  obtain m where b1: "m \<in> Field r \<and> (\<forall> a \<in> Field r. (a,m) \<in> r^*)" using assms unfolding
       Conelike_def by blast
  then obtain x where b2: "(m,x) \<in> r \<or> (x,m) \<in> r" unfolding Field_def by blast
  then have "(x,m) \<in> r^*" using b1 unfolding Field_def by blast
  then obtain m' where b3: "(m',m) \<in> r" using b2 by (metis rtranclE)
  have "CCR {(m',m)}" unfolding CCR_def Field_def by force
  moreover have "\<forall>a\<in>Field r. \<exists>b\<in>Field {(m',m)}. (a, b) \<in> r^*" using b1 unfolding
      Field_def by blast
  ultimately show ?thesis using b3 unfolding \<UU>_def by blast
qed

lemma lem_rcc_emp: "\<parallel>{}\<parallel> = {}"
  unfolding RCC_def RCC_rel_def \<UU>_def apply simp unfolding CCR_def apply simp
  using lem_card_emprel by (smt iso_ozero_empty ordIso_symmetric ozero_def someI_ex)

lemma lem_rcc_rccrel:
fixes r::"'U rel"
shows "RCC_rel r \<parallel>r\<parallel>"
proof -
  have "\<exists> \<alpha>. RCC_rel r \<alpha>"
  proof (cases "\<UU> r = {}")
    assume "\<UU> r = {}"
    then show "\<exists> \<alpha>. RCC_rel r \<alpha>" unfolding RCC_rel_def by blast
  next
    assume b1: "\<UU> r \<noteq> {}"
    obtain Q where b2: "Q = { \<alpha>::'U rel. \<exists> s \<in> \<UU> r. \<alpha> =o |s| }" by blast
    have b3: "\<forall> s \<in> \<UU> r. \<exists> \<alpha> \<in> Q. \<alpha> \<le>o |s|"
    proof
      fix s
      assume c1: "s \<in> \<UU> r"
      then have c2: "s \<subseteq> (UNIV::'U set) \<times> (UNIV::'U set)" unfolding \<UU>_def by simp
      then have c3: "|s| \<le>o |(UNIV::'U set) \<times> (UNIV::'U set)|" by simp
      show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |s|"
      proof (cases "finite (UNIV::'U set)")
        assume "finite (UNIV::'U set)"
        then have "finite s" using c2 finite_subset by blast
        moreover have "CCR s" using c1 unfolding \<UU>_def by blast
        ultimately have "Conelike s" using lem_Relprop_fin_ccr by blast
        then have d1: "Conelike r" using c1 lem_uset_cl_ext by blast
        show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |s|"
        proof (cases "r = {}")
          assume e1: "r = {}"
          obtain \<alpha> where e2: "\<alpha> = ({}::'U rel)" by blast
          then have "\<alpha> \<in> \<UU> r" using e1 unfolding \<UU>_def CCR_def Field_def by blast
          moreover have e3: "\<alpha> =o |({}::'U rel)|" using e2 lem_card_emprel ordIso_symmetric by blast
          ultimately have "\<alpha> \<in> Q" using b2 e2 by blast
          moreover have "\<alpha> \<le>o |s|" using e3 card_of_empty ordIso_ordLeq_trans by blast
          ultimately show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |s|" by blast
        next
          assume e1: "r \<noteq> {}"
          then obtain m m' where e2: "{(m',m)} \<in> \<UU> r" using d1 lem_uset_cl_singleton by blast
          obtain \<alpha> where e3: "\<alpha> = |{m}|" by blast
          then have "\<alpha> =o |{(m',m)}|" by (simp add: ordIso_iff_ordLeq)
          then have "\<alpha> \<in> Q" using b2 e2 by blast
          moreover have "s \<noteq> {}" using c1 e1 unfolding \<UU>_def Field_def by force
          moreover then have "\<alpha> \<le>o |s|" using e3 by simp
```

```
        ultimately show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |s|" by blast
       qed
     next
       assume "\<not> finite (UNIV::'U set)"
       then have "|(UNIV::'U set) \<times> (UNIV::'U set)| =o |UNIV::'U set|" using card_of_Times_same_infinite
      by blast
       then have "|s| \<le>o |UNIV::'U set|" using c3 using ordLeq_ordIso_trans by blast
       then obtain A::"'U set" where "|s| =o |A|" using internalize_card_of_ordLeq2 by fast
       moreover then obtain \<alpha>::"'U rel" where "\<alpha> = |A|" by blast
       ultimately have "\<alpha> \<in> Q \<and> \<alpha> =o |s|" using b2 c1 ordIso_symmetric by blast
       then show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |s|" using ordIso_iff_ordLeq by blast
     qed
   qed
   then have "Q \<noteq> {}" using b1 by blast
   then obtain \<alpha> where b4: "\<alpha> \<in> Q \<and> (\<forall>\<alpha>'. \<alpha>' <o \<alpha> \<
     longrightarrow> \<alpha>' \<notin> Q)" using wf_ordLess wf_eq_minimal[of "ordLess"] by blast
   moreover have "\<forall> \<alpha>' \<in> Q. Card_order \<alpha>'" using b2 using
     ordIso_card_of_imp_Card_order by blast
   ultimately have "\<forall> \<alpha>' \<in> Q. \<not> (\<alpha>' <o \<alpha>) \<longrightarrow> \<alpha> \<le
     >o \<alpha>'" by simp
   then have b5: "\<alpha> \<in> Q \<and> (\<forall> \<alpha>' \<in> Q. \<alpha> \<le>o \<alpha>')" using b4 by
      blast
   then obtain s where b6: "s \<in> \<UU> r \<and> |s| =o \<alpha>" using b2 ordIso_symmetric by blast
   moreover have "\<forall> s'\<in>\<UU> r. |s| \<le>o |s'|"
   proof
     fix s'
     assume "s' \<in> \<UU> r"
     then obtain \<alpha>' where "\<alpha>' \<in> Q \<and> \<alpha>' \<le>o |s'|" using b3 by blast
     moreover then have "|s| =o \<alpha> \<and> \<alpha> \<le>o \<alpha>'" using b5 b6 by blast
     ultimately show "|s| \<le>o |s'|" using ordIso_ordLeq_trans ordLeq_transitive by blast
   qed
   ultimately have "RCC_rel r \<alpha>" unfolding RCC_rel_def by blast
   then show "\<exists> \<alpha>. RCC_rel r \<alpha>" by blast
 qed
 then show ?thesis unfolding RCC_def by (metis someI2)
qed

lemma lem_rcc_uset_ne:
assumes "\<UU> r \<noteq> {}"
shows "\<exists> s \<in> \<UU> r. |s| =o \<parallel>r\<parallel> \<and> ( \<forall> s' \<in> \<UU> r. |s| \<le>o
       |s'| )"
 using assms lem_rcc_rccrel unfolding RCC_rel_def by blast

lemma lem_rcc_uset_emp:
assumes "\<UU> r = {}"
shows "\<parallel>r\<parallel> = {}"
 using assms lem_rcc_rccrel unfolding RCC_rel_def by blast

lemma lem_rcc_uset_mem_bnd:
assumes "s \<in> \<UU> r"
shows "\<parallel>r\<parallel> \<le>o |s|"
proof -
 obtain s0 where "s0 \<in> \<UU> r \<and> |s0| =o \<parallel>r\<parallel> \<and> ( \<forall> s' \<in> \<UU> r.
     |s0| \<le>o |s'| )" using assms lem_rcc_uset_ne by blast
 moreover then have "|s0| \<le>o |s|" using assms by blast
 ultimately show "\<parallel>r\<parallel> \<le>o |s|" by (metis ordIso_iff_ordLeq ordLeq_transitive)
qed

lemma lem_rcc_cardord: "Card_order \<parallel>r\<parallel>"
proof (cases "\<UU> r = {}")
 assume "\<UU> r = {}"
 then have "\<parallel>r\<parallel> = {}" using lem_rcc_uset_emp by blast
 then show "Card_order \<parallel>r\<parallel>" using lem_cardord_emp by simp
next
 assume "\<UU> r \<noteq> {}"
 then obtain s where "s \<in> \<UU> r \<and> |s| =o \<parallel>r\<parallel>" using lem_rcc_uset_ne by blast
 then show "Card_order \<parallel>r\<parallel>" using Card_order_ordIso2 card_of_Card_order by blast
qed

lemma lem_uset_ne_rcc_inf:
fixes r::"'U rel"
assumes "\<not> ( \<parallel>r\<parallel> <o \<omega>_ord )"
shows "\<UU> r \<noteq> {}"
proof -
 have "\<parallel>r\<parallel> = {} \<longrightarrow> \<parallel>r\<parallel> <o |UNIV :: nat set|"
   by (metis card_of_Well_order finite.emptyI infinite_iff_card_of_nat ordIso_ordLeq_trans ordIso_symmetric
     ordLeq_iff_ordLess_or_ordIso ozero_def ozero_ordLeq)
```

```
  then have "\<parallel>r\<parallel> = {} \<longrightarrow> \<parallel>r\<parallel> <o \<omega>_ord" using
      card_of_nat ordLess_ordIso_trans by blast
  then show "\<UU> r \<noteq> {}" using assms lem_rcc_uset_emp by blast
qed

lemma lem_rcc_inf: "( \<omega>_ord \<le>o \<parallel>r\<parallel> ) = ( \<not> ( \<parallel>r\<parallel> <o \<
      omega>_ord ) )"
  using lem_rcc_cardord lem_cord_lin by (metis Field_natLeq natLeq_card_order)

lemma lem_Rcc_eq1_12:
fixes r::"'U rel"
shows "CCR r \<Longrightarrow> r \<in> \<UU> r"
  unfolding \<UU>_def CCR_def by blast

lemma lem_Rcc_eq1_23:
fixes r::"'U rel"
assumes "r \<in> \<UU> r"
shows "(r = ({}::'U rel)) \<or> (({}::'U rel) <o \<parallel>r\<parallel>)"
proof -
  obtain s0 where a2: "s0 \<in> \<UU> r" and a3: "|s0| =o \<parallel>r\<parallel>" using assms lem_rcc_uset_ne
      by blast
  have "s0 = {} \<longrightarrow> r = {}" using a2 unfolding \<UU>_def Field_def by force
  moreover have "s0 \<noteq> {} \<longrightarrow> ({}::'U rel) <o \<parallel>r\<parallel>"
    using a3 lem_rcc_cardord lem_cardord_emp
       by (metis (no_types, lifting) Card_order_iff_ordIso_card_of Field_empty
          card_of_empty3 card_order_on_well_order_on not_ordLeq_iff_ordLess
          ordLeq_iff_ordLess_or_ordIso ordLeq_ordIso_trans ozero_def ozero_ordLeq)
  ultimately show ?thesis by blast
qed

lemma lem_Rcc_eq1_31:
fixes r::"'U rel"
assumes "(r = ({}::'U rel)) \<or> (({}::'U rel) <o \<parallel>r\<parallel>)"
shows "CCR r"
proof (cases "r = {}")
  assume "r = {}"
  then show "CCR r" unfolding CCR_def Field_def by blast
next
  assume b1: "r \<noteq> {}"
  then have b2: "({}::'U rel) <o \<parallel>r\<parallel>" using assms by blast
  then have "\<parallel>r\<parallel> \<noteq> ({}::'U rel)" using ordLess_irreflexive by fastforce
  then have "\<UU> r \<noteq> {}" using lem_rcc_uset_emp by blast
  then obtain s where b3: "s \<in> \<UU> r" and b4: "|s| =o \<parallel>r\<parallel>" and
    b5: "\<forall> s' \<in> \<UU> r. |s| \<le>o |s'|" using lem_rcc_uset_ne by blast
  have "s \<noteq> {}" using assms b1 b4 lem_card_emprel not_ordLess_ordIso ordIso_ordLess_trans by blast
  have "s \<subseteq> r" using b3 unfolding \<UU>_def by blast
  then have "Field s \<subseteq> Field r \<and> s^* \<subseteq> r^*" unfolding Field_def using rtrancl_mono by
      blast
  have "\<forall>a\<in>Field r. \<forall>b\<in>Field r. \<exists>c\<in>Field r. (a, c) \<in> r^* \<and> (b, c)
      \<in> r^*"
  proof (intro ballI)
    fix a b
    assume c1: "a \<in> Field r" and c2: "b \<in> Field r"
    then obtain a' b' where c3: "a' \<in> Field s \<and> b' \<in> Field s \<and> (a,a') \<in> r^* \<and> (b,b')
      \<in> r^*"
      using b3 unfolding \<UU>_def by blast
    then obtain c where c4: "c \<in> Field s \<and> (a',c) \<in> s^* \<and> (b',c) \<in> s^*" using b3 unfolding
       \<UU>_def CCR_def by blast
    have "a' \<in> Field r \<and> b' \<in> Field r \<and> c \<in> Field r" using b3 c3 c4 unfolding \<UU>_def
      Field_def by blast
    moreover have "(a',c) \<in> r^* \<and> (b',c) \<in> r^*" using b3 c4 unfolding \<UU>_def using rtrancl_mono
      by blast
    ultimately have "c \<in> Field r \<and> (a, c) \<in> r^* \<and> (b, c) \<in> r^*" using c3 by force
    then show "\<exists>c\<in>Field r. (a, c) \<in> r^* \<and> (b, c) \<in> r^*" by blast
  qed
  then show "CCR r" unfolding CCR_def by blast
qed

lemma lem_Rcc_eq2_12:
fixes r::"'U rel" and a::"'a"
assumes "Conelike r"
shows "\<parallel>r\<parallel> \<le>o |{a}|"
proof (cases "r = {}")
  assume "r = {}"
  then have "\<parallel>r\<parallel> = {}" using lem_rcc_emp by blast
  then show "\<parallel>r\<parallel> \<le>o |{a}|" by (metis card_of_Well_order ozero_def ozero_ordLeq)
next
  assume "r \<noteq> {}"
```

```
  then obtain m where b1: "m \<in> Field r \<and> (\<forall> a \<in> Field r. (a,m) \<in> r^*)" using assms
      unfolding Conelike_def by blast
  then obtain m' where b2: "(m,m') \<in> r \<or> (m',m) \<in> r" unfolding Field_def by blast
  then have "(m',m) \<in> r^*" using b1 by (meson FieldI2 r_into_rtrancl)
  then obtain x where "(x,m) \<in> r" using b2 by (metis rtranclE)
  moreover have "CCR {(x,m)}" unfolding CCR_def Field_def by blast
  ultimately have "{(x,m)} \<in> \<UU> r" using b1 unfolding \<UU>_def by simp
  then have "\<parallel>r\<parallel> \<le>o |{(x,m)}|" using lem_rcc_uset_mem_bnd by blast
  moreover have "|{(x,m)}| \<le>o |{a}|" by simp
  ultimately show "\<parallel>r\<parallel> \<le>o |{a}|" using ordLeq_transitive by blast
qed

lemma lem_Rcc_eq2_23:
fixes r::"'U rel" and a::"'a"
assumes "\<parallel>r\<parallel> \<le>o |{a}|"
shows "\<parallel>r\<parallel> <o \<omega>_ord"
proof -
  have "|{a}| <o |UNIV :: nat set|" using finite_iff_cardOf_nat by blast
  then show "\<parallel>r\<parallel> <o \<omega>_ord" using assms ordLeq_ordLess_trans card_of_nat
      ordLess_ordIso_trans by blast
qed

lemma lem_Rcc_eq2_31:
fixes r::"'U rel"
assumes "CCR r" and "\<parallel>r\<parallel> <o \<omega>_ord"
shows "Conelike r"
proof -
  have "r \<in> \<UU> r" using assms lem_Rcc_eq1_12 by blast
  then obtain s where b1: "s \<in> \<UU> r" and b2: "|s| =o \<parallel>r\<parallel>" using lem_rcc_uset_ne by
      blast
  have "|s| <o \<omega>_ord" using assms b2 using ordIso_imp_ordLeq ordLeq_ordLess_trans by blast
  then have "finite s" using finite_iff_ordLess_natLeq by blast
  moreover have "CCR s" using b1 unfolding \<UU>_def by blast
  ultimately have "Conelike s" using lem_Relprop_fin_ccr by blast
  then show "Conelike r" using b1 lem_uset_cl_ext by blast
qed

lemma lem_Rcc_range:
fixes r::"'U rel"
shows "\<parallel>r\<parallel> \<le>o |UNIV::('U set)|"
  by (simp add: lem_rcc_cardord)

lemma lem_rcc_nccr:
fixes r::"'U rel"
assumes "\<not> (CCR r)"
shows "\<parallel>r\<parallel> = {}"
proof -
  have "\<not> (({}::'U rel) <o \<parallel>r\<parallel>)" using assms lem_Rcc_eq1_31[of r] by blast
  moreover have "Well_order ({}::'U rel)" using Well_order_empty by blast
  moreover have "Well_order \<parallel>r\<parallel>" using lem_rcc_cardord unfolding card_order_on_def by blast
  ultimately have "\<parallel>r\<parallel> \<le>o ({}::'U rel)" by simp
  then show "\<parallel>r\<parallel> = {}" using lem_ord_subemp by blast
qed

lemma lem_Rcc_relcard_bnd:
fixes r::"'U rel"
shows "\<parallel>r\<parallel> \<le>o |r|"
proof(cases "CCR r")
  assume "CCR r"
  then show "\<parallel>r\<parallel> \<le>o |r|" using lem_Rcc_eq1_12 lem_rcc_uset_mem_bnd by blast
next
  assume "\<not> CCR r"
  then have "\<parallel>r\<parallel> = {}" using lem_rcc_nccr by blast
  then have "\<parallel>r\<parallel> \<le>o ({}::'U rel)" by (metis card_of_empty ordLeq_Well_order_simp
      ozero_def ozero_ordLeq)
  moreover have "({}::'U rel) \<le>o |r|" by (metis card_of_Well_order ozero_def ozero_ordLeq)
  ultimately show "\<parallel>r\<parallel> \<le>o |r|" using ordLeq_transitive by blast
qed

lemma lem_Rcc_inf_lim:
fixes r::"'U rel"
assumes "\<omega>_ord \<le>o \<parallel>r\<parallel>"
shows "\<not>( \<parallel>r\<parallel> = {} \<or> isSuccOrd \<parallel>r\<parallel> )"
  using assms lem_card_inf_lim lem_rcc_cardord by blast

lemma lem_rcc_uset_ne_ccr:
fixes r::"'U rel"
assumes "\<UU> r \<noteq> {}"
```

```
shows "CCR r"
proof -
  obtain s where b1: "s \<in> \<UU> r" using assms by blast
  have "\<forall>a\<in>Field r. \<forall>b\<in>Field r. \<exists>c\<in>Field r. (a, c) \<in> r^* \<and> (b, c)
      \<in> r^*"
  proof (intro ballI impI)
    fix a b
    assume "a\<in>Field r" and "b\<in>Field r"
    then obtain a' b' where c1: "a' \<in> Field s \<and> b' \<in> Field s \<and> (a,a') \<in> r^* \<and> (b,b')
      \<in> r^*"
      using b1 unfolding \<UU>_def by blast
    then obtain c where "c \<in> Field s \<and> (a',c) \<in> s^* \<and> (b',c) \<in> s^*" using b1 unfolding \<
      UU>_def CCR_def by blast
    moreover have "s \<subseteq> r" using b1 unfolding \<UU>_def by blast
    ultimately have "c \<in> Field r \<and> (a',c) \<in> r^* \<and> (b',c) \<in> r^*" using rtrancl_mono
      unfolding Field_def by blast
    moreover then have "(a,c) \<in> r^* \<and> (b,c) \<in> r^*" using c1 by force
    ultimately show "\<exists>c\<in>Field r. (a, c) \<in> r^* \<and> (b, c) \<in> r^*" by blast
  qed
  then show ?thesis unfolding CCR_def by blast
qed

lemma lem_rcc_uset_tr:
fixes r s t::"'U rel"
assumes a1: "s \<in> \<UU> r" and a2: "t \<in> \<UU> s"
shows "t \<in> \<UU> r"
proof -
  have "\<forall>a\<in>Field r. \<exists>b\<in>Field t. (a, b) \<in> r^*"
  proof
    fix a
    assume "a \<in> Field r"
    then obtain b' where "b' \<in> Field s \<and> (a,b') \<in> r^*" using a1 unfolding \<UU>_def by blast
    moreover then obtain b where "b \<in> Field t \<and> (b',b) \<in> s^*" using a2 unfolding \<UU>_def by blast
    moreover have "s \<subseteq> r" using a1 unfolding \<UU>_def by blast
    ultimately have "b \<in> Field t \<and> (a,b') \<in> r^* \<and> (b',b) \<in> r^*" using rtrancl_mono by
      blast
    then have "b \<in> Field t \<and> (a,b) \<in> r^*" by force
    then show "\<exists>b\<in>Field t. (a, b) \<in> r^*" by blast
  qed
  then show ?thesis using a1 a2 unfolding \<UU>_def by blast
qed

lemma lem_scf_emp: "scf {} = {}"
  unfolding scf_def scf_rel_def SCF_def apply simp
  using lem_card_emprel by (smt card_of_empty_ordIso iso_ozero_empty ordIso_symmetric ozero_def someI_ex)

lemma lem_scf_scfrel:
fixes r::"'U rel"
shows "scf_rel r (scf r)"
proof -
  have b1: "SCF r \<noteq> {}" unfolding SCF_def by blast
  obtain Q where b2: "Q = { \<alpha>::'U rel. \<exists> A \<in> SCF r. \<alpha> =o |A| }" by blast
  have b3: "\<forall> A \<in> SCF r. \<exists> \<alpha> \<in> Q. \<alpha> \<le>o |A|"
  proof
    fix A
    assume "A \<in> SCF r"
    then have "|A| \<in> Q \<and> |A| =o |A|" using b2 ordIso_symmetric by force
    then show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |A|" using ordIso_iff_ordLeq by blast
  qed
  then have "Q \<noteq> {}" using b1 by blast
  then obtain \<alpha> where b4: "\<alpha> \<in> Q \<and> (\<forall>\<alpha>'. \<alpha>' <o \<alpha> \<
      longrightarrow> \<alpha>' \<notin> Q)" using wf_ordLess wf_eq_minimal[of "ordLess"] by blast
  moreover have "\<forall> \<alpha>' \<in> Q. Card_order \<alpha>'" using b2 using ordIso_card_of_imp_Card_order
       by blast
  ultimately have "\<forall> \<alpha>' \<in> Q. \<not> (\<alpha>' <o \<alpha>) \<longrightarrow> \<alpha> \<le>o
       \<alpha>'" by simp
  then have b5: "\<alpha> \<in> Q \<and> (\<forall> \<alpha>' \<in> Q. \<alpha> \<le>o \<alpha>')" using b4 by
      blast
  then obtain A where b6: "A \<in> SCF r \<and> |A| =o \<alpha>" using b2 ordIso_symmetric by blast
  moreover have "\<forall> B\<in>SCF r. |A| \<le>o |B|"
  proof
    fix B
    assume "B \<in> SCF r"
    then obtain \<alpha>' where "\<alpha>' \<in> Q \<and> \<alpha>' \<le>o |B|" using b3 by blast
    moreover then have "|A| =o \<alpha> \<and> \<alpha> \<le>o \<alpha>'" using b5 b6 by blast
    ultimately show "|A| \<le>o |B|" using ordIso_ordLeq_trans ordLeq_transitive by blast
  qed
  ultimately have "scf_rel r \<alpha>" unfolding scf_rel_def by blast
```

```
  then show ?thesis unfolding scf_def by (metis someI2)
qed

lemma lem_scf_uset:
shows "\<exists> A \<in> SCF r. |A| =o scf r \<and> ( \<forall> B \<in> SCF r. |A| \<le>o |B| )"
  using lem_scf_scfrel unfolding scf_rel_def by blast

lemma lem_scf_uset_mem_bnd:
assumes "B \<in> SCF r"
shows "scf r \<le>o |B|"
proof -
  obtain A where "A \<in> SCF r \<and> |A| =o scf r \<and> ( \<forall> A' \<in> SCF r. |A| \<le>o |A'| )" using
      assms lem_scf_uset by blast
  moreover then have "|A| \<le>o |B|" using assms by blast
  ultimately show ?thesis by (metis ordIso_iff_ordLeq ordLeq_transitive)
qed

lemma lem_scf_cardord: "Card_order (scf r)"
proof -
  obtain A where "A \<in> SCF r \<and> |A| =o scf r" using lem_scf_uset by blast
  then show "Card_order (scf r)" using Card_order_ordIso2 card_of_Card_order by blast
qed

lemma lem_scf_inf: "( \<omega>_ord \<le>o (scf r) ) = ( \<not> ( (scf r) <o \<omega>_ord ) )"
  using lem_scf_cardord lem_cord_lin by (metis Field_natLeq natLeq_card_order)

lemma lem_scf_eq1_12:
fixes r::"'U rel"
shows "Field r \<in> SCF r"
  unfolding SCF_def by blast

lemma lem_scf_range:
fixes r::"'U rel"
shows "(scf r) \<le>o |UNIV::('U set)|"
  by (simp add: lem_scf_cardord)

lemma lem_scf_relfldcard_bnd:
fixes r::"'U rel"
shows "(scf r) \<le>o |Field r|"
  using lem_scf_eq1_12 lem_scf_uset_mem_bnd by blast

lemma lem_scf_ccr_scf_rcc_eq:
fixes r::"'U rel"
assumes "CCR r"
shows "\<parallel>r\<parallel> =o (scf r)"
proof -
  obtain B where b1: "B \<in> SCF r \<and> |B| =o scf r" using lem_scf_scfrel[of r] unfolding scf_rel_def by
      blast
  have "B \<subseteq> Field r" using b1 unfolding SCF_def by blast
  then obtain A where  b2: "B \<subseteq> A \<and> A \<in> SF r"
                  and b3: "(finite B \<longrightarrow> finite A) \<and> ((\<not> finite B) \<longrightarrow> |A
      | =o |B| )"
                using lem_inv_sf_ext[of B r] by blast
  then obtain A' where b4: "A \<subseteq> A' \<and> A' \<in> SF r \<and> CCR (Restr r A')"
                   and b5: "(finite A \<longrightarrow> finite A') \<and> ((\<not> finite A) \<longrightarrow> |
      A'| =o |A| )"
    using assms lem_Ccext_subccr_pext5[of r A _ "{}"] by metis
  have "Restr r A' \<in> \<UU> r"
  proof -
    have "\<forall>a\<in>Field r. \<exists>b\<in>Field (Restr r A'). (a, b) \<in> r^*"
    proof
      fix a
      assume "a \<in> Field r"
      then obtain b where "b \<in> B \<and> (a,b) \<in> r^*" using b1 unfolding SCF_def by blast
      moreover then have "b \<in> Field (Restr r A')" using b2 b4 unfolding SF_def by blast
      ultimately show "\<exists>b\<in>Field (Restr r A'). (a, b) \<in> r^*" by blast
    qed
    then show "Restr r A' \<in> \<UU> r" unfolding \<UU>_def using b4 by blast
  qed
  then have b6: "\<parallel>r\<parallel> \<le>o |Restr r A'|" using lem_rcc_uset_mem_bnd by blast
  obtain x0::"'U" where "True" by blast
  have b7: "\<parallel>r\<parallel> \<le>o (scf r)"
  proof (cases "finite B")
    assume "finite B"
    then have "finite (Restr r A')" using b3 b5 by blast
    then have "Conelike r"
      using assms b6 lem_Rcc_eq2_31[of r] finite_iff_ordLess_natLeq[of "Restr r A'"] ordLeq_ordLess_trans by
      blast
```

```
    then have c1: "\<parallel>r\<parallel> \<le>o |{x0}|" using lem_Rcc_eq2_12[of r x0] by blast
    show ?thesis
    proof (cases "r = {}")
      assume "r = {}"
      then have "scf r = {} \<and> \<parallel>r\<parallel> = {}" using lem_scf_emp lem_rcc_emp by blast
      then show "\<parallel>r\<parallel> \<le>o (scf r)" using b1 lem_ord_subemp ordIso_iff_ordLeq by metis
    next
      assume "r \<noteq> {}"
      then have "B \<noteq> {}" using b1 unfolding SCF_def Field_def by force
      then have "|{x0}| \<le>o |B|" using card_of_singl_ordLeq by metis
      then show ?thesis using c1 b1 ordLeq_transitive ordIso_imp_ordLeq by metis
    qed
  next
    assume c1: "\<not> finite B"
    then have "|A| =o |B| \<and> |A'| =o |A|" using b3 b5 finite_subset by simp
    then have "|A'| =o scf r" using b1 using ordIso_transitive by blast
    moreover have "\<omega>_ord \<le>o scf r" using c1 b1 infinite_iff_natLeq_ordLeq ordLeq_ordIso_trans by
      blast
    ultimately have "|Restr r A'| \<le>o scf r" using lem_restr_ordbnd[of "scf r" A' r] ordIso_imp_ordLeq by
      blast
    then show "\<parallel>r\<parallel> \<le>o (scf r)" using b6 ordLeq_transitive by blast
  qed
  moreover have "(scf r) \<le>o \<parallel>r\<parallel>"
  proof -
    obtain s where b1: "s \<in> \<UU> r \<and> |s| =o \<parallel>r\<parallel> \<and> (\<forall>s'\<in>\<UU> r. |
      s| \<le>o |s'| )"
      using assms lem_Rcc_eq1_12[of r] lem_rcc_uset_ne[of r] by blast
    then have "Field s \<subseteq> Field r \<and> (\<forall>a\<in>Field r. \<exists>b\<in>Field s. (a, b) \<in>
      r^*)"
      unfolding \<UU>_def Field_def by blast
    then have "Field s \<in> SCF r" unfolding SCF_def by blast
    then have b2: "scf r \<le>o |Field s|" using lem_scf_uset_mem_bnd by blast
    show ?thesis
    proof (cases "finite s")
      assume "finite s"
      then have "\<parallel>r\<parallel> <o \<omega>_ord"
        using b1 finite_iff_ordLess_natLeq not_ordLeq_ordLess ordIso_iff_ordLeq ordIso_transitive
       ordLeq_iff_ordLess_or_ordIso ordLeq_transitive by metis
      then have c1: "Conelike r" using assms lem_Rcc_eq2_31 by blast
      show ?thesis
      proof (cases "r = {}")
        assume "r = {}"
        then have "scf r = {} \<and> \<parallel>r\<parallel> = {}" using lem_scf_emp lem_rcc_emp by blast
        then show ?thesis using b7 by simp
      next
        assume d1: "r \<noteq> {}"
        then obtain m where "m \<in> Field r \<and> (\<forall> a \<in> Field r. (a,m) \<in> r^*)" using c1
       unfolding Conelike_def by blast
        then have "{m} \<in> SCF r" unfolding SCF_def by blast
        then have d2: "scf r \<le>o |{m}|" using lem_scf_uset_mem_bnd by blast
        have "({}::'U rel) <o \<parallel>r\<parallel>" using d1 assms lem_Rcc_eq1_23 lem_Rcc_eq1_12 by blast
        then have "|{m}| \<le>o \<parallel>r\<parallel>" using lem_co_one_ne_min by (metis card_of_empty3
       card_of_empty4 insert_not_empty ordLess_Well_order_simp)
        then show ?thesis using d2 ordLeq_transitive by blast
      qed
    next
      assume "\<not> finite s"
      then have "|Field s| =o |s|" using lem_rel_inf_fld_card by blast
      then show ?thesis using b1 b2 ordIso_iff_ordLeq ordLeq_transitive by metis
    qed
  qed
  ultimately show ?thesis using not_ordLeq_ordLess ordLeq_iff_ordLess_or_ordIso by blast
qed

lemma lem_scf_ccr_scf_uset:
fixes r::"'U rel"
assumes "CCR r" and "\<not> Conelike r"
shows "\<exists> s \<in> \<UU> r. (\<not> finite s) \<and> |Field s| =o (scf r)"
proof -
  have "\<parallel>r\<parallel> =o (scf r)" using assms lem_scf_ccr_scf_rcc_eq by blast
  moreover then obtain s where b1: "s \<in> \<UU> r \<and> |s| =o \<parallel>r\<parallel>" using assms
      lem_Rcc_eq1_12 lem_rcc_uset_ne[of r] by blast
  moreover have "(\<not> finite s) \<longrightarrow> |Field s| =o |s|" using lem_rel_inf_fld_card by blast
  moreover have "finite s \<longrightarrow> False"
  proof
    assume "finite s"
    then have "|s| <o \<omega>_ord" using finite_iff_ordLess_natLeq by blast
    then have "\<parallel>r\<parallel> <o \<omega>_ord" using b1
```

```
      by (meson not_ordLess_ordIso ordIso_iff_ordLeq ordIso_transitive ordLeq_iff_ordLess_or_ordIso
      ordLeq_transitive)
    then show "False" using assms lem_Rcc_eq2_31 by blast
  qed
  ultimately show ?thesis using ordIso_transitive by metis
qed

lemma lem_Scf_scfprops:
fixes r::"'U rel"
shows "( (scf r) \<le>o |UNIV::('U set)| ) \<and> ( (scf r) \<le>o |Field r| )"
  using lem_scf_range lem_scf_relfldcard_bnd by blast

lemma lem_scf_ccr_finscf_cl:
assumes "CCR r"
shows "finite (Field (scf r)) = Conelike r"
proof
  assume "finite (Field (scf r))"
  then have "finite \<parallel>r\<parallel>" using assms lem_scf_ccr_scf_rcc_eq lem_fin_fl_rel
      ordIso_finite_Field by blast
  then have "\<parallel>r\<parallel> <o \<omega>_ord" using lem_rcc_cardord lem_fin_fl_rel
    by (metis card_of_Field_ordIso finite_iff_ordLess_natLeq ordIso_iff_ordLeq ordLeq_ordLess_trans)
  then show "Conelike r" using assms lem_Rcc_eq2_31 by blast
next
  assume "Conelike r"
  then have "finite (Field \<parallel>r\<parallel>)" using lem_Rcc_eq2_12[of r] by (metis Field_card_of finite.
      emptyI finite_insert ordLeq_finite_Field)
  then show "finite (Field (scf r))" using assms lem_scf_ccr_scf_rcc_eq ordIso_finite_Field by blast
qed

lemma lem_sv_uset_sv_span:
fixes r s::"'U rel"
assumes a1: "s \<in> \<UU> r" and a2: "single_valued s"
shows "\<exists> r1. r1 \<in> Span r \<and> CCR r1 \<and> single_valued r1 \<and> s \<subseteq> r1 \<and> (
      acyclic s \<longrightarrow> acyclic r1)"
proof -
  have b0: "s \<subseteq> r" using a1 unfolding \<UU>_def by blast
  obtain isd where b3: "isd = (\<lambda> a i. \<exists> b \<in> Field s. (a, b) \<in> r^^i \<and> (\<forall> i'.
       (\<exists> b \<in> Field s. (a, b) \<in> r^^(i')) \<longrightarrow> i \<le> i'))" by blast
  obtain d where b4: "d = (\<lambda> a. SOME i. isd a i)" by blast
  obtain B where b5: "B = (\<lambda> a. { a'. (a, a') \<in> r })" by blast
  obtain H where b6: "H = (\<lambda> a. { a' \<in> B a. \<forall> a'' \<in> B a. (d a') \<le> (d a'') })" by
      blast
  obtain D where b7: "D = { a \<in> Field r - Field s. H a \<noteq> {}}" by blast
  obtain h where "h = (\<lambda> a. SOME a'. a' \<in> H a)" by blast
  then have b8: "\<forall> a \<in> D. h a \<in> H a" using b7 someI_ex[of "\<lambda> a'. a' \<in> H _"] by force
  have q1: "\<And> a. a \<in> Field r \<Longrightarrow> isd a (d a)"
  proof -
    fix a
    assume c1: "a \<in> Field r"
    then obtain b where c2: "b \<in> Field s \<and> (a,b) \<in> r^*" using a1 unfolding \<UU>_def by blast
    moreover obtain N where c3: "N = {i. \<exists> b \<in> Field s. (a, b) \<in> r^^i}" by blast
    ultimately have "N \<noteq> {}" using rtrancl_imp_relpow by blast
    then obtain m where "m \<in> N \<and> (\<forall> i \<in> N. m \<le> i)"
      using LeastI[of "\<lambda> x. x \<in> N"] Least_le[of "\<lambda> x. x \<in> N"] by blast
    then have "isd a m" using c2 c3 unfolding b3 by blast
    then show "isd a (d a)" using b4 someI_ex by metis
  qed
  have q2: "\<And> a. B a \<noteq> {} \<Longrightarrow> H a \<noteq> {}"
  proof -
    fix a
    assume "B a \<noteq> {}"
    moreover obtain N where c1: "N = d ` (B a)" by blast
    ultimately have "N \<noteq> {}"  by blast
    then obtain m where c2: "m \<in> N \<and> (\<forall> i \<in> N. m \<le> i)"
      using LeastI[of "\<lambda> x. x \<in> N"] Least_le[of "\<lambda> x. x \<in> N"] by blast
    then obtain a' where c3: "m = d a' \<and> a' \<in> B a" using c1 by blast
    moreover then have "\<forall> a'' \<in> B a. d a' \<le> d a''" using c1 c2 by force
    ultimately have "a' \<in> H a" unfolding b6 by blast
    then show "H a \<noteq> {}" by blast
  qed
  have q3: "\<forall> a \<in> Field r - Field s. d a = 1 \<or> d a > 1"
  proof
    fix a
    assume c1: "a \<in> Field r - Field s"
    then have "isd a (d a)" using q1 by blast
    then obtain b where "b \<in> Field s \<and> (a, b) \<in> r^^(d a)" using b3 by blast
    then have "d a = 0 \<longrightarrow> False" using c1 by force
    then show "d a = 1 \<or> d a > 1" by force
```

```
  qed
  have "Field r - Field s \<subseteq> D"
  proof
    fix a
    assume c1: "a \<in> Field r - Field s"
    moreover have "H a = {} \<longrightarrow> False"
    proof
      assume "H a = {}"
      then have "B a = {}" using q2 by blast
      moreover obtain b where "b \<in> Field s \<and> (a, b) \<in> r^*" using a1 c1 unfolding \<UU>_def by blast
      ultimately have "a \<in> Field s" unfolding b5 by (metis Collect_empty_eq converse_rtranclE)
      then show "False" using c1 by blast
    qed
    ultimately show "a \<in> D" using b7 by blast
  qed
  then have q4: "D = Field r - Field s" using b5 b6 b7 by blast
  have q5: "\<forall> a \<in> D. d a > 1 \<longrightarrow> d a = Suc (d (h a)) \<and> (d (h a) > 1 \<
      longrightarrow> h a \<in> D)"
  proof (intro ballI impI)
    fix a
    assume c1: "a \<in> D" and c2: "d a > 1"
    then obtain b where c3: "b \<in> Field s" and c4: "(a, b) \<in> r^^(d a)"
                    and c5: "\<forall> i'. (\<exists> b \<in> Field s. (a, b) \<in> r^^(i')) \<longrightarrow> (
      d a) \<le> i'"
                    using b3 b7 q1 by blast
    have c6: "d a \<ge> 1" using c1 c4 b7 q3 by force
    then have "d a = Suc ((d a) - 1)" by simp
    then obtain a' where c7: "(a,a') \<in> r \<and> (a',b) \<in> r^^((d a) - 1)"
      using c4 relpow_Suc_D2[of a b "d a - 1" r] by metis
    moreover then have "a' \<notin> Field s" using c2 c5 by (metis less_Suc_eq_le not_less_eq relpow_1)
    ultimately have "(a,a') \<in> r \<and> a' \<in> Field r - Field s" unfolding Field_def by blast
    then have "a' \<in> B a" unfolding b5 by blast
    moreover have "h a \<in> H a" using c1 b8 by blast
    ultimately have "d (h a) \<le> d a'" unfolding b6 by blast
    moreover have "Suc (d a') \<le> d a"
    proof -
      have "d a' \<le> d a - 1" using q1 b3 c7 c3 unfolding Field_def by blast
      then show ?thesis using c6 by force
    qed
    moreover have "d a \<le> (Suc (d (h a)))"
    proof -
      have d1: "(a, h a) \<in> r" using c1 b5 b6 b8 by blast
      then have "h a \<in> Field r" unfolding Field_def by blast
      then obtain b' where "b' \<in> Field s \<and> ((h a), b') \<in> r^^(d (h a))" using b3 q1 by blast
      moreover then have "(a,b') \<in> r^^(Suc (d (h a)))" using d1 c7 by (meson relpow_Suc_I2)
      ultimately show "d a \<le> (Suc (d (h a)))" using c5 by blast
    qed
    ultimately have "d a = Suc (d (h a))" by force
    moreover have "d (h a) > 1 \<longrightarrow> h a \<in> D"
    proof
      assume d1: "d (h a) > 1"
      then have d2: "(a, h a) \<in> r" using c1 b5 b6 b8 by simp
      then have "isd (h a) (d (h a))" using d1 q1 unfolding Field_def by force
      then have "(h a) \<notin> Field s" using d1 b3 by force
      then show "h a \<in> D" using d2 q4 unfolding Field_def by blast
    qed
    ultimately show "d a = Suc (d (h a)) \<and> (d (h a) > 1 \<longrightarrow> h a \<in> D)" by blast
  qed
  obtain g1 where b9:  "g1 = { (a, b). a \<in> D \<and> b = h a }" by blast
  have q6: "\<forall> a \<in> D. \<exists> a' \<in> D. d a' = 1 \<and> (a,a') \<in> g1^*"
  proof -
    have "\<forall> n. \<forall> a \<in> D. d a = Suc n \<longrightarrow> ((h^^n) a) \<in> D \<and> d ((h^^n) a)
       = 1"
    proof
      fix n0
      show "\<forall> a \<in> D. d a = Suc n0 \<longrightarrow> ((h^^n0) a) \<in> D \<and> d ((h^^n0) a) = 1"
      proof (induct n0)
        show "\<forall>a\<in>D. d a = Suc 0 \<longrightarrow> ((h^^0) a) \<in> D \<and> d ((h ^^ 0) a) = 1"
          using q4 by force
      next
        fix n
        assume d1: "\<forall>a\<in>D. d a = Suc n \<longrightarrow> ((h^^n) a) \<in> D \<and> d ((h ^^ n) a) =
      1"
        show "\<forall>a\<in>D. d a = Suc (Suc n) \<longrightarrow> ((h^^(Suc n)) a) \<in> D \<and> d ((h ^^ Suc
       n) a) = 1"
        proof (intro ballI impI)
          fix a
          assume e1: "a \<in> D" and e2: "d a = Suc (Suc n)"
```

```
      then have "d a = Suc (d (h a)) \<and> (d (h a) > 1 \<longrightarrow> h a \<in> D)" using q5 by simp
      moreover then have e3: "d (h a) = Suc n" using e2 by simp
      ultimately have "d (h a) > 1 \<longrightarrow> ((h^^n) (h a)) \<in> D \<and> d ((h^^n) (h a)) = 1"
   using d1 by blast
      moreover have "(h^^n) (h a) = (h^^(Suc n)) a" by (metis comp_apply funpow_Suc_right)
      moreover have e4: "d (h a) = 1 \<longrightarrow> d ((h^^(Suc n)) a) = 1" using e3 by simp
      moreover have "d (h a) = 1 \<longrightarrow> ((h^^(Suc n)) a) \<in> D"
      proof
        assume f1: "d (h a) = 1"
        then have f2: "n = 0 \<and> (a, h a) \<in> r" using e1 e3 b5 b6 b8 by simp
        then have "isd (h a) 1" using f1 q1 unfolding Field_def by force
        then have "(h a) \<notin> Field s" using b3 by force
        then have "(h a) \<in> D" using q4 f2 unfolding Field_def by blast
        then show "((h^^(Suc n)) a) \<in> D" using f2 by simp
      qed
      moreover have "d (h a) = 1 \<or> d (h a) > 1" using e3 by force
      ultimately show "((h^^(Suc n)) a) \<in> D \<and> d ((h ^^ (Suc n)) a) = 1" by force
    qed
  qed
qed
moreover have "\<forall> i. \<forall> a \<in> D. d a > i \<longrightarrow> (a, (h^^i) a) \<in> g1^*"
proof
  fix i0
  show "\<forall> a \<in> D. d a > i0 \<longrightarrow> (a, (h^^i0) a) \<in> g1^*"
  proof (induct i0)
    show "\<forall>a\<in>D. d a > 0 \<longrightarrow> (a, (h^^0) a) \<in> g1^*" by force
  next
    fix i
    assume d1: "\<forall>a\<in>D. d a > i \<longrightarrow> (a, (h^^i) a) \<in> g1^*"
    show "\<forall>a\<in>D. d a > (Suc i) \<longrightarrow> (a, (h^^(Suc i)) a) \<in> g1^*"
    proof (intro ballI impI)
      fix a
      assume e1: "a \<in> D" and e2: "d a > (Suc i)"
      then have e3: "d a = Suc (d (h a)) \<and> (d (h a) > 1 \<longrightarrow> h a \<in> D)" using q5 by
   simp
      moreover then have e4: "d (h a) > i" using e2 by simp
      ultimately have "d (h a) > 1 \<longrightarrow> (h a, (h^^i) (h a)) \<in> g1^*" using d1 by simp
      moreover have "(h^^i) (h a) = (h^^(Suc i)) a" by (metis comp_apply funpow_Suc_right)
      moreover have "d (h a) = 1 \<longrightarrow> (h^^(Suc i)) a = (h a)" using e4 by force
      moreover have "d (h a) = 1 \<or> d (h a) > 1" using e4 by force
      moreover then have "(a, h a) \<in> g1" using e1 e3 unfolding b9 by simp
      ultimately show "(a, (h^^(Suc i)) a) \<in> g1^*"
        by (metis converse_rtrancl_into_rtrancl r_into_rtrancl)
    qed
  qed
qed
ultimately have "\<forall>n. \<forall>a\<in>D. d a = Suc n \<longrightarrow> (h^^n) a \<in> D \<and> d ((h^^
  n) a) = 1 \<and> (a, (h ^^ n) a) \<in> g1^*"
  by simp
then have "\<forall>n. \<forall>a\<in>D. d a = Suc n \<longrightarrow> (\<exists> a' \<in> D. d a' = 1 \<and
  > (a,a') \<in> g1^* )"
  by blast
moreover have "\<forall> a \<in> D. \<exists> n. d a = Suc n" using q3 q4 q5 by force
ultimately show ?thesis by blast
qed
obtain r1 where b19: "r1 = s \<union> g1" by blast
have t1: "g1 \<subseteq> r1" using b19 by blast
have b20: "s \<subseteq> r1" using b19 by blast
have b21: "r1 \<subseteq> r"
proof -
  have "\<forall> a \<in> D. (a, h a) \<in> r" using b5 b6 b8 by blast
  then have "g1 \<subseteq> r" using b9 by blast
  then show ?thesis using b0 b19 by blast
qed
have b22: "\<forall>a \<in> Field r1 - Field s. \<exists>b \<in> Field s. (a, b) \<in> r1^*"
proof
  fix a
  assume d1: "a \<in> Field r1 - Field s"
  then have "a \<in> D" using q4 b21 unfolding Field_def by blast
  then obtain a' where d2: "a' \<in> D \<and> d a' = 1 \<and> (a, a') \<in> g1^*" using q6 by blast
  then have d3: "(a', h a') \<in> r1 \<and> h a' \<in> H a'" using b8 b9 t1 by blast
  obtain b where "b \<in> Field s \<and> (a',b) \<in> r" using d2 q1 q4 b3 by force
  moreover then have "isd b (d b)" using q1 unfolding Field_def by blast
  ultimately have "b \<in> B a' \<and> d b = 0" using b3 b5 by force
  then have "d (h a') = 0" using d3 b6 by force
  then have "isd (h a') 0" using q1 d3 b21 unfolding Field_def by force
  then have "h a' \<in> Field s" using b3 by force
  moreover have "(a, a') \<in> r1^*" using d2 t1 rtrancl_mono[of g1 r1] by blast
```

```
  ultimately have "(h a') \<in> Field s \<and> (a, h a') \<in> r1^*" using d3 by force
  then show "\<exists>b \<in> Field s. (a, b) \<in> r1^*" by blast
qed
have b23: "Field r \<subseteq> Field r1"
proof -
  have "(Field r - Field s) \<subseteq> Field r1" using q4 b9 t1 unfolding Field_def by blast
  moreover have "Field s \<subseteq> Field r1" using b20 unfolding Field_def by blast
  ultimately show "Field r \<subseteq> Field r1" by blast
qed
have "Field r1 \<subseteq> Field r" using b21 unfolding Field_def by blast
then have "r1 \<in> Span r" using b21 b23 unfolding Span_def by blast
moreover have "CCR r1"
proof -
  have "s \<in> \<UU> r1" using b20 b22 a1 unfolding \<UU>_def by blast
  then show "CCR r1" using lem_rcc_uset_ne_ccr by blast
qed
moreover have "single_valued r1"
proof -
  have "\<forall> a b c. (a,b) \<in> r1 \<and> (a,c) \<in> r1 \<longrightarrow> b = c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> r1 \<and> (a,c) \<in> r1"
    moreover have "(a,b) \<in> s \<and> (a,c) \<in> s \<longrightarrow> b = c" using a2 unfolding
    single_valued_def by blast
    moreover have "(a,b) \<in> s \<and> (a,c) \<in> g1 \<longrightarrow> False" using b9 b7 unfolding
    Field_def by blast
    moreover have "(a,b) \<in> g1 \<and> (a,c) \<in> s \<longrightarrow> b = c" using b9 b7 unfolding
    Field_def by blast
    moreover have "(a,b) \<in> g1 \<and> (a,c) \<in> g1 \<longrightarrow> b = c" using b9 by blast
    ultimately show "b = c" using b19 by blast
  qed
  then show ?thesis unfolding single_valued_def by blast
qed
moreover have "acyclic s \<longrightarrow> acyclic r1"
proof
  assume c1: "acyclic s"
  have c2: "\<forall> a' \<in> D. d a' = 1 \<longrightarrow> d (h a') = 0"
  proof (intro ballI impI)
    fix a'
    assume d1: "a' \<in> D" and d2: "d a' = 1"
    then have d3: "(a', h a') \<in> r1 \<and> h a' \<in> H a'" using b8 b9 t1 by blast
    obtain b where "b \<in> Field s \<and> (a',b) \<in> r" using d1 d2 q1 q4 b3 by force
    moreover then have "isd b (d b)" using q1 unfolding Field_def by blast
    ultimately have "b \<in> B a' \<and> d b = 0" using b3 b5 by force
    then show "d (h a') = 0" using d3 b6 by force
  qed
  have c3: "\<forall> a b. (a,b) \<in> g1 \<longrightarrow> d b < d a"
  proof (intro allI impI)
    fix a b
    assume "(a,b) \<in> g1"
    then have d1: "a \<in> D \<and> b = h a" using b9 by blast
    then have "d a > 1 \<or> d a = 1" and "d a > 1 \<longrightarrow> d b < d a" using q3 q4 q5 by force+
    moreover have "d a = 1 \<longrightarrow> d b < d a" using d1 c2 by force
    ultimately show "d b < d a" by blast
  qed
  have c4: "\<forall> n. \<forall> a b. (a,b) \<in> g1^^(Suc n) \<longrightarrow> d b < d a"
  proof
    fix n
    show "\<forall> a b. (a,b) \<in> g1^^(Suc n) \<longrightarrow> d b < d a"
    proof (induct n)
      show "\<forall>a b. (a, b) \<in> g1 ^^ (Suc 0) \<longrightarrow> d b < d a" using c3 by force
    next
      fix n
      assume e1: "\<forall>a b. (a, b) \<in> g1 ^^ (Suc n) \<longrightarrow> d b < d a"
      show "\<forall>a b. (a, b) \<in> g1 ^^ (Suc (Suc n)) \<longrightarrow> d b < d a"
      proof (intro allI impI)
        fix a b
        assume "(a, b) \<in> g1 ^^ (Suc (Suc n))"
        then obtain c where "(a,c) \<in> g1^^(Suc n) \<and> (c,b) \<in> g1" by force
        then have "d c < d a \<and> d b < d c" using e1 c3 by blast
        then show "d b < d a" by simp
      qed
    qed
  qed
  have "\<forall> x. (x,x) \<in> g1^+ \<longrightarrow> False"
  proof (intro allI impI)
    fix x
    assume "(x,x) \<in> g1^+"
```

```
      then obtain m::nat where "m > 0 \<and> (x,x) \<in> g1^^m" using trancl_power by blast
      moreover then obtain n where "m = Suc n" using less_imp_Suc_add by blast
      ultimately have "d x < d x" using c4 by blast
      then show "False" by blast
    qed
    then have "acyclic g1" unfolding acyclic_def by blast
    moreover have "\<forall> a b c. (a,b) \<in> s \<and> (b,c) \<in> g1 \<longrightarrow> False" using b9 b7
      unfolding Field_def by blast
    moreover have "r1 = s \<union> g1" using b19 by blast
    ultimately show "acyclic r1" using c1 lem_acyc_un_emprd by blast
  qed
  ultimately show ?thesis using b20 by blast
qed

lemma lem_incrfun_nat: "\<forall> i::nat. f i < f (Suc i) \<Longrightarrow> \<forall> i j. i \<le> j \<
      longrightarrow> f i + (j-i) \<le> f j"
proof -
  assume a1: "\<forall> i::nat. f i < f (Suc i)"
  have "\<forall> j. \<forall> i. i\<le>j \<longrightarrow> f i + (j-i) \<le> f j"
  proof
    fix j0
    show "\<forall> i. i\<le>j0 \<longrightarrow> f i + (j0-i) \<le> f j0"
    proof (induct j0)
      show "\<forall>i\<le>0. f i + (0 - i) \<le> f 0" by simp
    next
      fix j
      assume c1: "\<forall>i\<le>j. f i + (j - i) \<le> f j"
      show "\<forall>i\<le>Suc j. f i + (Suc j - i) \<le> f (Suc j)"
      proof (intro allI impI)
        fix i
        assume d1: "i \<le> Suc j"
        show "f i + (Suc j - i) \<le> f (Suc j)"
        proof (cases "i \<le> j")
          assume "i \<le> j"
          moreover then have "f i + (j - i) \<le> f j" using c1 by blast
          ultimately show ?thesis using a1
            by (metis Suc_diff_le Suc_le_eq add_Suc_right not_le order_trans)
        next
          assume "\<not> i \<le> j"
          then have "i = Suc j" using d1 by simp
          then show ?thesis by simp
        qed
      qed
    qed
  qed
  then show "\<forall> i j. i \<le> j \<longrightarrow> f i + (j-i) \<le> f j" by blast
qed

lemma lem_sv_uset_rcceqw:
fixes r::"'U rel"
assumes a1: "\<parallel>r\<parallel> =o \<omega>_ord"
shows "\<exists> r1 \<in> \<UU> r. single_valued r1 \<and> acyclic r1 \<and> (\<forall> x \<in> Field r1. r1``{x
      } \<noteq> {})"
proof -
  have "\<not> ( \<parallel>r\<parallel> <o \<omega>_ord )" using a1 by (metis not_ordLess_ordIso)
  then obtain s where b1: "s \<in> \<UU> r \<and> |s| =o \<parallel>r\<parallel>" using lem_rcc_uset_ne
      lem_uset_ne_rcc_inf by blast
  then have "|Field s| =o \<omega>_ord"
    using a1 lem_rel_inf_fld_card[of s] by (metis ordIso_natLeq_infinite1 ordIso_transitive)
  then obtain ai where b2: "Field s = ai ` (UNIV::nat set)" using lem_cntset_enum by blast
  obtain f where b3: "f = (\<lambda> x. SOME y. (x,y) \<in> r^* \<and> y \<in> Field s )" by blast
  obtain g where b4: "g = (\<lambda> A. SOME y. y \<in> Field r \<and> A \<subseteq> dncl r {y})" by blast
  obtain h where b5: "h = (\<lambda> A. SOME y. y \<in> Field r - dncl r A)" by blast
  have b6: "\<And> x. x \<in> Field r \<Longrightarrow> (x, f x) \<in> r^* \<and> f x \<in> Field s"
  proof -
    fix x
    assume "x \<in> Field r"
    then have "\<exists> y. (x,y) \<in> r^* \<and> y \<in> Field s" using b1 unfolding \<UU>_def by blast
    then show "(x,f x) \<in> r^* \<and> f x \<in> Field s"
      using b3 someI_ex[of "\<lambda> y. (x,y) \<in> r^* \<and> y \<in> Field s "] by blast
  qed
  have b7: "\<And> A. finite A \<and> A \<subseteq> Field r \<Longrightarrow> g A \<in> Field r \<and> A \<
      subseteq> dncl r {g A}"
  proof -
    fix A::"'U set"
    assume c1: "finite A \<and> A \<subseteq> Field r"
    moreover have "CCR r" using b1 lem_rcc_uset_ne_ccr by blast
    ultimately obtain s where c2: "finite s \<and> CCR s \<and> s \<subseteq> r \<and> A \<subseteq> Field s"
```

```
    using lem_Ccext_finsubccr_dext[of r A] by blast
  then have c3: "Conelike s" using lem_Relprop_fin_ccr by blast
  have "\<exists> y. y \<in> Field r \<and> A \<subseteq> dncl r {y}"
  proof (cases "A = {}")
    assume "A = {}"
    moreover have "r \<noteq> {}" using a1 lem_rcc_emp lem_Rcc_inf_lim by (metis ordIso_iff_ordLeq)
    moreover then have "Field r \<noteq> {}" unfolding Field_def by force
    ultimately show ?thesis unfolding dncl_def by blast
  next
    assume d1: "A \<noteq> {}"
    then have "s \<noteq> {}" using c2 unfolding Field_def by blast
    then obtain y where "\<forall>x\<in>A. (x, y) \<in> s^*" using c2 c3 unfolding Conelike_def by blast
    then have d2: "\<forall> x \<in> A. (x,y) \<in> r^*" using c2 rtrancl_mono by blast
    obtain x0 where "x0 \<in> A \<inter> Field r" using d1 c1 c2 by blast
    moreover then have "(x0, y) \<in> r^*" using d2 by blast
    ultimately have "y \<in> Field r" using lem_rtr_field[of x0 y r] by blast
    then show ?thesis using d2 unfolding dncl_def by blast
  qed
  then show "g A \<in> Field r \<and> A \<subseteq> dncl r {g A}"
    using b4 someI_ex[of "\<lambda> y. y \<in> Field r \<and> A \<subseteq> dncl r {y}"] by blast
qed
have b8: "\<And> A::'U set. finite A \<Longrightarrow> (h A) \<in> Field r - dncl r A"
proof -
  fix A::"'U set"
  assume c1: "finite A"
  have "Field r - dncl r A = {} \<longrightarrow> False"
  proof
    assume "Field r - dncl r A = {}"
    then have "\<forall> x \<in> Field r. \<exists> y \<in> A \<inter> Field r. (x,y) \<in> r^*"
      using lem_rtr_field[of _ _ r] unfolding dncl_def by blast
    then have "A \<inter> Field r \<in> SCF r" unfolding SCF_def by blast
    then have "scf r \<le>o |A \<inter> Field r|" using lem_scf_uset_mem_bnd by blast
    moreover have "|A \<inter> Field r| <o \<omega>_ord" using c1 finite_iff_ordLess_natLeq by blast
    ultimately have "scf r <o \<omega>_ord" by (metis ordLeq_ordLess_trans)
    moreover have "\<parallel>r\<parallel> =o scf r" using b1 lem_scf_ccr_scf_rcc_eq[of r] lem_rcc_uset_ne_ccr
     [of r] by blast
    ultimately show "False" using a1
      by (meson not_ordLeq_ordLess ordIso_iff_ordLeq ordLess_ordLeq_trans)
  qed
  then show "(h A) \<in> Field r - dncl r A"
    using b5 someI_ex[of "\<lambda> y. y \<in> Field r - dncl r A"] by blast
qed
obtain Ci where b9: "Ci = rec_nat { ai 0 } (\<lambda> n B. B \<union> {f(g({(h B)} \<union> B \<union> ai`{k.
    k\<le>n})})})" by blast
then have b10: "Ci 0 = {ai 0}"
      and b11: "\<And> n. Ci (Suc n) = Ci n \<union> {f(g({(h (Ci n))} \<union> Ci n \<union> ai`{k. k\<le>n})
    )}" by simp+
have b12: "Field s \<subseteq> Field r" using b1 unfolding \<UU>_def Field_def by blast
have b13: "\<And> n. Ci n \<subseteq> Field s \<and> finite (Ci n)"
proof -
  fix n
  show "Ci n \<subseteq> Field s \<and> finite (Ci n)"
  proof (induct n)
    show "Ci 0 \<subseteq> Field s \<and> finite (Ci 0)" using b2 b10 by simp
  next
    fix n
    assume "Ci n \<subseteq> Field s \<and> finite (Ci n)"
    moreover then have "{h (Ci n)} \<union> Ci n \<union> ai ` {k. k \<le> n} \<subseteq> Field r" using b2 b8
     b12 by blast
    ultimately show "Ci (Suc n) \<subseteq> Field s \<and> finite (Ci (Suc n))" using b6 b7 b11 by simp
  qed
qed
have b14: "\<And> n. \<exists> m\<in>(Ci n). Ci n \<union> ai`{k. k\<le>n-1} \<subseteq> dncl r {m}"
proof -
  fix n
  show "\<exists> m\<in>(Ci n). Ci n \<union> ai`{k. k\<le>n-1} \<subseteq> dncl r {m}"
  proof (induct n)
    show "\<exists>m\<in>Ci 0. Ci 0 \<union> ai`{k. k\<le>0-1} \<subseteq> dncl r {m}" using b10 unfolding
    dncl_def by simp
  next
    fix n
    assume "\<exists>m\<in>Ci n. Ci n \<union> ai`{k. k\<le>n-1} \<subseteq> dncl r {m}"
    obtain A where d1: "A = {(h (Ci n))} \<union> Ci n \<union> ai`{k. k\<le>n}" by blast
    obtain m where d2: "m = f(g(A))" by blast
    have "finite A \<and> A \<subseteq> Field r" using d1 b2 b8 b12 b13 by force
    then have d3: "g A \<in> Field r \<and> A \<subseteq> dncl r {g A}" using b7 by blast
    then have d4: "(g A, m) \<in> r^* \<and> m \<in> Field s" using d2 b6 by blast
    have "m \<in> Ci (Suc n)" using d1 d2 b11 by blast
```

```
    moreover have "ai`{k. k\<le>n} \<subseteq> dncl r {m}" using d1 d3 d4 unfolding dncl_def by force
    moreover have "Ci n \<subseteq> dncl r {m}" using d1 d3 d4 unfolding dncl_def by force
    moreover then have "Ci (Suc n) \<subseteq> dncl r {m}" using d1 d2 b11 unfolding dncl_def by blast
    ultimately show "\<exists>m\<in>Ci (Suc n). Ci (Suc n) \<union> ai`{k. k\<le>(Suc n)-1} \<subseteq> dncl r
     {m}" by force
  qed
qed
obtain ci where b15: "ci = (\<lambda> n. SOME m. m \<in> Ci n \<and> Ci n \<subseteq> dncl r {m})" by blast
have b16: "\<And> n. (ci n) \<in> Ci n \<and> Ci n \<subseteq> dncl r {ci n}"
proof -
  fix n
  have "\<exists> m\<in>(Ci n). Ci n \<subseteq> dncl r {m}" using b14 by blast
  then show "(ci n) \<in> Ci n \<and> Ci n \<subseteq> dncl r {ci n}"
    using b15 someI_ex[of "\<lambda> m. m \<in> Ci n \<and> Ci n \<subseteq> dncl r {m}"] by blast
qed
have b17: "\<And> n. ci (Suc n) \<notin> dncl r (Ci n)"
proof -
  fix n
  obtain A where c1: "A = {(h (Ci n))} \<union> Ci n \<union> ai`{k. k\<le>n}" by blast
  then have c2: "finite A \<and> A \<subseteq> Field r" using b2 b8[of "Ci n"] b13[of n] b12 by blast
  then have c3: "g A \<in> Field r \<and> A \<subseteq> dncl r {g A}" using b7 by simp
  then have "(h (Ci n), g A) \<in> r^*" using c1 unfolding dncl_def by blast
  moreover have "(g A, f (g A)) \<in> r^*" using c3 b6[of "g A"] by blast
  moreover have "(f (g A), ci (Suc n)) \<in> r^*" using c1 b11 b16 unfolding dncl_def by blast
  ultimately have "(h (Ci n), ci (Suc n)) \<in> r^*" by force
  moreover have "h (Ci n) \<notin> dncl r (Ci n)" using b8[of "Ci n"] b13[of n] by blast
  ultimately show "ci (Suc n) \<notin> dncl r (Ci n)" unfolding dncl_def
    by (meson Image_iff converse_iff rtrancl_trans)
qed
have "\<forall> n. (ci n, ci (Suc n)) \<in> r^* \<and> ci n \<noteq> ci (Suc n)"
proof
  fix n
  have "(ci n, ci (Suc n)) \<in> r^*" using b11 b16 unfolding dncl_def by blast
  moreover have "ci n \<noteq> ci (Suc n)" using b16[of n] b17[of "n"] unfolding dncl_def by fastforce
  ultimately show "(ci n, ci (Suc n)) \<in> r^* \<and> ci n \<noteq> ci (Suc n)" by blast
qed
then obtain l yi where
         b18: "\<forall>n. (yi n, yi (Suc n)) \<in> r"
     and b19: "\<forall>i j. (i < j) = (l i < l j)"
     and b20: "\<forall>i. yi (l i) = ci i"
     and b21: "\<forall>i. inj_on yi {k. l i \<le> k \<and> k \<le> l (Suc i)}"
     and b22: "\<forall> k. \<exists> i. l i \<le> k \<and> Suc k \<le> l (Suc i)"
  using lem_flatseq[of ci r] by blast
obtain r' where b23: "r' = { (x,y). \<exists> i. x = yi i \<and> y = yi (Suc i) }" by blast
have b24: "\<forall> j. \<forall> i. i \<le> j \<longrightarrow> (yi i, yi j) \<in> r'^*"
proof
  fix j
  show "\<forall> i. i \<le> j \<longrightarrow> (yi i, yi j) \<in> r'^*"
  proof (induct j)
    show "\<forall>i \<le> 0. (yi i, yi 0) \<in> r'^*" by blast
  next
    fix j
    assume d1: "\<forall>i \<le> j. (yi i, yi j) \<in> r'^*"
    show "\<forall>i \<le> Suc j. (yi i, yi (Suc j)) \<in> r'^*"
    proof (intro allI impI)
      fix i
      assume e1: "i \<le> Suc j"
      show "(yi i, yi (Suc j)) \<in> r'^*"
      proof (cases "i \<le> j")
        assume "i \<le> j"
        then have "(yi i, yi j) \<in> r'^*" using d1 by blast
        moreover have "(yi j, yi (Suc j)) \<in> r'" using b23 by blast
        ultimately show ?thesis by simp
      next
        assume "\<not> i \<le> j"
        then have "i = Suc j" using e1 by simp
        then show ?thesis using e1 by blast
      qed
    qed
  qed
qed
have b25: "\<forall> j. (\<forall> i. i \<le> j \<longrightarrow> Ci i \<subseteq> Ci j)"
proof
  fix j
  show "\<forall> i. i \<le> j \<longrightarrow> Ci i \<subseteq> Ci j"
  proof (induct j)
    show "\<forall>i\<le>0. Ci i \<subseteq> Ci 0" by force
  next
```

```
    fix j
    assume "\<forall>i\<le>j. Ci i \<subseteq> Ci j"
    moreover have "Ci j \<subseteq> Ci (Suc j)" using b11 by blast
    ultimately show "\<forall>i\<le>Suc j. Ci i \<subseteq> Ci (Suc j)" using le_Suc_eq by fastforce
  qed
qed
have b26: "\<forall> k1 k2. k1 < k2 \<longrightarrow> yi k1 = yi k2 \<longrightarrow> (\<exists> i. l i \<le>
    k1 \<and> k2 \<le> l (i+2))"
proof (intro allI impI)
  fix k1::nat and k2::nat
  assume d1: "k1 < k2" and d2: "yi k1 = yi k2"
  obtain i1 i2 where d3: "l i1 \<le> k1 \<and> Suc k1 \<le> l (Suc i1)"
                 and d4: "l i2 \<le> k2 \<and> Suc k2 \<le> l (Suc i2)" using b22 by blast
  have "i1 = i2 \<longrightarrow> False"
  proof
    assume "i1 = i2"
    then have "l i1 \<le> k2 \<and> k2 \<le> l (Suc i1)" using d4 by simp
    moreover have "l i1 \<le> k1 \<and> k1 \<le> l (Suc i1)" using d3 by simp
    ultimately show "False" using d1 d2 b21 unfolding inj_on_def by blast
  qed
  moreover have "i2 < i1 \<longrightarrow> False"
  proof
    assume "i2 < i1"
    then have "Suc i2 = i1 \<or> Suc i2 < i1" by fastforce
    then have "l (Suc i2) = l i1 \<or> l (Suc i2) < l i1" using b19 by blast
    then have "l (Suc i2) \<le> l i1" by fastforce
    moreover have "l i1 < l (Suc i2)" using d1 d3 d4 by simp
    ultimately show "False" by simp
  qed
  moreover have "Suc i1 < i2 \<longrightarrow> False"
  proof
    assume e1: "Suc i1 < i2"
    have "k1 \<le> l (Suc i1) \<and> l i2 \<le> k2" using d3 d4 by force
    then have "(yi k1, yi (l (Suc i1))) \<in> r^*" and "(yi (l i2), yi k2) \<in> r^*"
      using b18 b23 b24 rtrancl_mono[of r' r] by blast+
    then have e2: "(yi k1, ci (Suc i1)) \<in> r^*" and e3: "(ci i2, yi k1) \<in> r^*" using d2 b20 by force+
    have "Suc i1 \<le> i2-1 \<and> i2-1 \<le> i2" and "Suc (i2-1) = i2" using e1 by simp+
    then have e4: "ci i2 \<notin> dncl r (Ci (i2 - 1))" and e5: "ci (Suc i1) \<in> Ci (i2-1)"
      using b16[of "Suc i1"] b17[of "i2 - 1"] b25 by fastforce+
    have "yi k1 \<notin> dncl r (Ci (i2-1))" using e3 e4 unfolding dncl_def
      by (meson Image_iff converse_iff rtrancl_trans)
    moreover have "yi k1 \<in> dncl r (Ci (i2-1))" using e2 e5 unfolding dncl_def by blast
    ultimately show "False" by blast
  qed
  ultimately have "Suc i1 = i2" by simp
  moreover then have "l (Suc i1) = l i2" using b19 by blast
  ultimately have "l i1 \<le> k1 \<and> k2 \<le> l (i1 + 2)" using d3 d4 by simp
  then show "\<exists> i. l i \<le> k1 \<and> k2 \<le> l (i+2)" by blast
qed
obtain w where b27: "w = (\<lambda> k. k + l ((GREATEST j. l j \<le> k) + 2))" by blast
have b28: "\<And> k. \<forall> k'. yi k = yi k' \<longrightarrow> k' < Suc (w k)"
proof -
  fix k
  show "\<forall> k'. yi k = yi k' \<longrightarrow> k' < Suc (w k)"
  proof (cases "\<exists> k' > k. yi k' = yi k")
    assume d1: "\<exists> k' > k. yi k' = yi k"
    have d2: "\<forall> k'. k < k' \<longrightarrow> yi k = yi k' \<longrightarrow> (\<exists> i. l i \<le> k
    \<and> k' \<le> l (i+2))" using b26 by blast
    have d3: "\<forall> i. i \<le> l i"
    proof
      fix i
      show "i \<le> l i"
      proof (induct i)
        show "0 \<le> l 0" by blast
      next
        fix i
        assume "i \<le> l i"
        moreover have "l i < l (Suc i)" using b19 by blast
        ultimately show "Suc i \<le> l (Suc i)" by simp
      qed
    qed
    obtain i0 where d4: "i0 = (GREATEST j. l j \<le> k)" by blast
    obtain t where d5: "t = k + l (i0+2)" by blast
    then have "t \<ge> k" by force
    moreover have "\<forall> k'. yi k' = yi k \<longrightarrow> k' \<le> t"
    proof (intro allI impI)
      fix k'
      assume e1: "yi k' = yi k"
```

```
      have "k < k' \<longrightarrow> k' \<le> t"
      proof
        assume "k < k'"
        then obtain i where f1: "l i \<le> k \<and> k' \<le> l (i+2)" using e1 d2 by metis
        moreover have "\<forall>y. l y \<le> k \<longrightarrow> y < Suc k" using d3 less_Suc_eq_le
    order_trans by blast
        ultimately have "i \<le> i0" using d4 Greatest_le_nat[of "\<lambda> j. l j \<le> k" i "Suc k"] by
    force
        then have "l (i+2) \<le> l(i0+2)" using b19 by (metis Suc_less_eq add_2_eq_Suc' not_le)
        then show "k' \<le> t" using f1 d5 by fastforce
      qed
      then show "k' \<le> t" using d5 by fastforce
    qed
    ultimately show ?thesis using d4 d5 b27 by fastforce
  next
    assume "\<not> (\<exists> k' > k. yi k' = yi k)"
    then have "\<forall> k'. yi k' = yi k \<longrightarrow> k' \<le> k" using leI by blast
    then show ?thesis using b27 by fastforce
  qed
qed
obtain q where b29: "q = (\<lambda> k. GREATEST k'. yi k = yi k')" by blast
have b30: "\<And> k. yi k = yi (q k)"
proof -
  fix k
  show "yi k = yi (q k)" using b28[of k] b29 GreatestI_nat[of "\<lambda> k'. yi k = yi k'" k "Suc (w k)" ] by
    force
qed
have b31: "\<And> k k'. yi k' = yi (q k) \<longrightarrow> k' \<le> q k"
proof
  fix k k'
  assume "yi k' = yi (q k)"
  then show "k' \<le> q k" using b28[of k] b29 b30 Greatest_le_nat[of "\<lambda> k'. yi k = yi k'" k' "Suc (w
    k)"] by force
qed
obtain p where b32: "p = rec_nat (q 0) (\<lambda> n y. q (Suc y))" by blast
obtain r1 where b33: "r1 = { (x,y). \<exists> i. x = yi (p i) \<and> y = yi (Suc (p i)) }" by blast
have b34: "\<And> i. p i = q (p i)"
proof -
  fix i
  show "p i = q (p i)"
  proof (induct i)
    show "p 0 = q (p 0)" using b29 b30 b32 by simp
  next
    fix i
    assume "p i = q (p i)"
    then show "p (Suc i) = q (p (Suc i))" using b29 b30 b32 by simp
  qed
qed
have b35: "\<And> i j. i\<le>j \<longrightarrow> p i + (j-i) \<le> p j"
proof -
  fix i j
  have "\<And> k. q k = k \<longrightarrow> q k < q (Suc k)" using b30 b31 by (metis less_eq_Suc_le)
  then have "\<forall> i. p i < p (Suc i)" using b32 b34 by simp
  then show "i\<le>j \<longrightarrow> p i + (j-i) \<le> p j" using lem_incrfun_nat[of p] by blast
qed
have b36: "\<forall> i j. p i = p j \<longrightarrow> i = j"
proof (intro allI impI)
  fix i j
  assume "p i = p j"
  then have "i\<le>j \<longrightarrow> i = j" and "j\<le>i \<longrightarrow> j = i" using b35 by fastforce+
  then show "i = j" by fastforce
qed
have b37: "\<forall> i j. yi (p i) = yi (p j) \<longrightarrow> i = j" using b29 b34 b36 by metis
have b38: "\<forall> x \<in> Field r1. \<exists> i. x = yi (p i)"
proof
  fix x
  assume "x \<in> Field r1"
  moreover have "\<forall> i. yi (Suc (p i)) = yi (p (Suc i))" using b30 b32 by simp
  ultimately show "\<exists> i. x = yi (p i)" using b33 unfolding Field_def by force
qed
have b39: "\<And> i. (yi (p i), yi (p (Suc i))) \<in> r1" using b30 b32 b33 by fastforce
have b40: "\<forall> j. \<forall> i. i \<le> j \<longrightarrow> (yi (p i), yi (p j)) \<in> r1^*"
proof
  fix j0
  show "\<forall> i. i \<le> j0 \<longrightarrow> (yi (p i), yi (p j0)) \<in> r1^*"
  proof (induct j0)
    show "\<forall>i\<le>0. (yi (p i), yi (p 0)) \<in> r1^*" by blast
  next
```

```
    fix j
    assume d1: "\<forall>i\<le>j. (yi (p i), yi (p j)) \<in> r1^*"
    show "\<forall>i\<le>Suc j. (yi (p i), yi (p (Suc j))) \<in> r1^*"
    proof (intro allI impI)
      fix i
      assume e1: "i\<le>Suc j"
      show "(yi (p i), yi (p (Suc j))) \<in> r1^*"
      proof (cases "i = Suc j")
        assume "i = Suc j"
        then show ?thesis by force
      next
        assume "i \<noteq> Suc j"
        then have "(yi (p i), yi (p j)) \<in> r1^*" using e1 d1 by simp
        then show ?thesis using e1 d1 b39[of j] by simp
      qed
    qed
  qed
qed
have "r1 \<subseteq> r'" using b23 b33 by blast
moreover have "\<forall> a \<in> Field r'. \<exists> b \<in> Field r1. (a, b) \<in> r'^*"
proof
  fix a
  assume "a \<in> Field r'"
  then obtain k where "a = yi k" using b23 unfolding Field_def by blast
  moreover have "k \<le> p k" using b35[of 0 k] by fastforce
  ultimately have "(a, yi (p k)) \<in> r'^*" using b24 by blast
  moreover have "yi (p k) \<in> Field r1" using b33 unfolding Field_def by blast
  ultimately show "\<exists> b \<in> Field r1. (a, b) \<in> r'^*" by blast
qed
moreover have "CCR r1"
proof -
  have "\<forall>a\<in>Field r1. \<forall>b\<in>Field r1. \<exists>c\<in>Field r1. (a, c) \<in> r1^* \<and> (b
    , c) \<in> r1^*"
  proof (intro ballI)
    fix a b
    assume d1: "a \<in> Field r1" and d2: "b \<in> Field r1"
    then obtain i j where "a = yi (p i) \<and> b = yi (p j)" using b38 by blast
    then have "i \<le> j \<longrightarrow> (a,b) \<in> r1^*" and "j \<le> i \<longrightarrow> (b,a) \<in> r1
    ^*" using b40 by blast+
    then show "\<exists>c\<in>Field r1. (a, c) \<in> r1^* \<and> (b, c) \<in> r1^*" using d1 d2 by fastforce
  qed
  then show "CCR r1" unfolding CCR_def by blast
qed
ultimately have b41: "r1 \<in> \<UU> r'" unfolding \<UU>_def by blast
then have "CCR r'" using lem_rcc_uset_ne_ccr by blast
moreover have "r' \<subseteq> r" using b18 b23 by blast
moreover have "\<forall> x \<in> Field r. \<exists>y \<in> Field r'. (x, y) \<in> r^*"
proof
  fix x
  assume c1: "x \<in> Field r"
  then obtain y where c2: "y \<in> Field s \<and> (x,y) \<in> r^*" using b1 unfolding \<UU>_def by blast
  then obtain n where "y = ai n" using b2 by blast
  then obtain m where "y \<in> dncl r {m} \<and> m \<in> Ci (Suc n)" using b14[of "Suc n"] by force
  then have "(y, m) \<in> r^* \<and> (m, ci (Suc n)) \<in> r^*" using b16 unfolding dncl_def by blast
  then have "(x, ci (Suc n)) \<in> r^*" using c2 by force
  moreover obtain y' where c2: "y' = yi (l (Suc n))" by blast
  ultimately have c3: "(x,y') \<in> r^*" using b20 by metis
  have "(y', yi (Suc (l (Suc n)))) \<in> r'" using c2 b23 by blast
  then have "y' \<in> Field r'" unfolding Field_def by blast
  then show "\<exists>y \<in> Field r'. (x, y) \<in> r^*" using c3 by blast
qed
ultimately have "r' \<in> \<UU> r" unfolding \<UU>_def by blast
then have "r1 \<in> \<UU> r" using b41 lem_rcc_uset_tr by blast
moreover have "single_valued r1" using b33 b37 unfolding single_valued_def by blast
moreover have "acyclic r1"
proof -
  have c1: "\<forall> n. \<forall> i j. (yi (p i), yi (p j)) \<in> r1^^(Suc n) \<longrightarrow> i < j"
  proof
    fix n0
    show "\<forall> i j. (yi (p i), yi (p j)) \<in> r1^^(Suc n0) \<longrightarrow> i < j"
    proof (induct n0)
      show "\<forall>i j. (yi (p i), yi (p j)) \<in> r1 ^^ (Suc 0) \<longrightarrow> i < j"
      proof (intro allI impI)
        fix i j
        assume "(yi (p i), yi (p j)) \<in> r1^^(Suc 0)"
        then obtain i' j'::nat where "yi (p i) = yi (p i') \<and> yi (p j) = yi (Suc (p i'))" using b33 by
    force
        then have "i = i' \<and> j = Suc i'" using b30 b32 b37 by simp
```

```
          then show "i < j" by blast
        qed
      next
        fix n
        assume d1: "\<forall>i j. (yi (p i), yi (p j)) \<in> r1 ^^ (Suc n) \<longrightarrow> i < j"
        show "\<forall>i j. (yi (p i), yi (p j)) \<in> r1 ^^ Suc (Suc n) \<longrightarrow> i < j"
        proof (intro allI impI)
          fix i j
          assume "(yi (p i), yi (p j)) \<in> r1 ^^ Suc (Suc n)"
          then obtain x where "(yi (p i), x) \<in> r1 ^^ (Suc n) \<and> (x, yi (p j)) \<in> r1" by force
          moreover then obtain k where "x = yi (p k)" using b38 unfolding Field_def by blast
          ultimately have e1: "i < k \<and> (yi (p k), yi (p j)) \<in> r1" using d1 by blast
          then obtain i' j'::nat where "yi (p k) = yi (p i') \<and> yi (p j) = yi (Suc (p i'))" using b33 by
      force
          then have "k = i' \<and> j = Suc i'" using b30 b32 b37 by simp
          then have "k < j" by blast
          then show "i < j" using e1 by simp
        qed
      qed
    qed
    have "\<forall> x. (x,x) \<in> r1^+ \<longrightarrow> False"
    proof (intro allI impI)
      fix x
      assume d1: "(x,x) \<in> r1^+"
      then have "x \<in> Field r1" by (metis FieldI2 Field_def trancl_domain trancl_range)
      then obtain i where "x = yi (p i)" using b38 by blast
      moreover obtain m::nat where "m > 0 \<and> (x,x) \<in> r1^^m" using d1 trancl_power by blast
      moreover then obtain n where "m = Suc n" using less_imp_Suc_add by blast
      ultimately have "n < n" using c1 by blast
      then show "False" by blast
    qed
    then show ?thesis unfolding acyclic_def by blast
  qed
  moreover have "\<forall> x \<in> Field r1. r1``{x} \<noteq> {}"
  proof
    fix x
    assume "x \<in> Field r1"
    then obtain i where "x = yi (p i)" using b38 by blast
    moreover then obtain y where "y = yi (Suc (p i))" by blast
    ultimately have "(x,y) \<in> r1" using b33 by blast
    then show "r1``{x} \<noteq> {}" by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_sv_span_scflew:
fixes r::"'U rel"
assumes "CCR r" and "scf r \<le>o \<omega>_ord"
shows "\<exists> r1. r1 \<in> Span r \<and> CCR r1 \<and> single_valued r1"
proof (cases "\<parallel>r\<parallel> =o \<omega>_ord")
  assume "\<parallel>r\<parallel> =o \<omega>_ord"
  then obtain s where "s \<in> \<UU> r \<and> single_valued s" using lem_sv_uset_rcceqw by blast
  then show ?thesis using lem_sv_uset_sv_span by blast
next
  assume "\<not> (\<parallel>r\<parallel> =o \<omega>_ord)"
  then have "\<parallel>r\<parallel> <o \<omega>_ord" using assms lem_scf_ccr_scf_rcc_eq[of r]
    by (metis ordIso_ordLess_trans ordIso_transitive ordLeq_iff_ordLess_or_ordIso)
  then have b1: "Conelike r" using assms lem_Rcc_eq2_31 by blast
  have "\<exists> s. s \<in> \<UU> r \<and> single_valued s"
  proof (cases "r = {}")
    assume "r = {}"
    then have "{} \<in> \<UU> r" unfolding \<UU>_def CCR_def Field_def by blast
    moreover have "single_valued {}" unfolding single_valued_def by blast
    ultimately show ?thesis by blast
  next
    assume "r \<noteq> {}"
    then obtain m where c1: "m \<in> Field r \<and> (\<forall> a \<in> Field r. (a, m) \<in> r^*)" using b1
      unfolding Conelike_def by blast
    then obtain u v where c2: "(u, v) \<in> r \<and> (u = m \<or> v = m)" unfolding Field_def by blast
    obtain s where c3: "s = {(u,v)}" by blast
    have "s \<subseteq> r" using c2 c3 by blast
    moreover have "CCR s" using c3 unfolding CCR_def by fastforce
    moreover have "\<forall>a\<in>Field r. \<exists>b\<in>Field s. (a, b) \<in> r^*"
    proof
      fix a
      assume "a \<in> Field r"
      moreover have "m \<in> Field s" using c2 c3 unfolding Field_def by fastforce
      ultimately show "\<exists>b\<in>Field s. (a, b) \<in> r^*" using c1 by blast
```

```
    qed
    ultimately have "s \<in> \<UU> r" unfolding \<UU>_def by blast
    moreover have "single_valued s" using c3 unfolding single_valued_def by blast
    ultimately show ?thesis by blast
  qed
  then show ?thesis using lem_sv_uset_sv_span by blast
qed

lemma lem_sv_span_scfeqw:
fixes r::"'U rel"
assumes "CCR r" and "scf r =o \<omega>_ord"
shows "\<exists> r1. r1 \<in> Span r \<and> r1 \<noteq> {} \<and> CCR r1 \<and> single_valued r1 \<and> acyclic
      r1 \<and> (\<forall>x\<in>Field r1. r1``{x} \<noteq> {})"
proof -
  have b1: "\<parallel>r\<parallel> =o \<omega>_ord" using assms lem_scf_ccr_scf_rcc_eq[of r] by (metis
      ordIso_transitive)
  then obtain s where "s \<in> \<UU> r \<and> single_valued s \<and> acyclic s \<and> (\<forall>x\<in>Field s. s
      ``{x} \<noteq> {})"
    using lem_sv_uset_rcceqw by blast
  then obtain r1 where b2: "r1 \<in> Span r \<and> CCR r1 \<and> single_valued r1 \<and> s \<subseteq> r1 \<and>
       acyclic r1"
    using lem_sv_uset_sv_span[of s r] by blast
  moreover have "r1 = {} \<longrightarrow> False"
  proof
    assume "r1 = {}"
    then have "r = {}" using b2 unfolding Span_def Field_def by force
    then show "False" using b1 lem_Rcc_inf_lim lem_rcc_emp lem_rcc_inf by (metis not_ordLess_ordIso)
  qed
  moreover have "\<forall>x\<in>Field r1. r1``{x} = {} \<longrightarrow> False"
  proof (intro ballI impI)
    fix x
    assume c1: "x \<in> Field r1" and c2: "r1``{x} = {}"
    have "\<forall>a\<in>Field r1. (a, x) \<in> r1^*"
    proof
      fix a
      assume "a \<in> Field r1"
      then obtain t where "(x,t) \<in> r1^* \<and> (a,t) \<in> r1^*" using c1 b2 unfolding CCR_def by blast
      moreover then have "x = t" using c2 by (metis Image_singleton_iff converse_rtranclE empty_iff)
      ultimately show "(a,x) \<in> r1^*" by blast
    qed
    then have "Conelike r1" using c1 unfolding Conelike_def by blast
    moreover have "r1 \<in> \<UU> r" using b2 unfolding \<UU>_def Span_def by blast
    ultimately have "Conelike r" using lem_uset_cl_ext[of r1 r] by blast
    then show "False" using b1 lem_Rcc_eq2_12[of r] lem_Rcc_eq2_23[of r] by (metis not_ordLess_ordIso)
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Ldo_den_ccr_uset:
fixes r s::"'U rel"
assumes "CCR s" and "s \<subseteq> r \<and> Field s \<in> Den r"
shows "s \<in> \<UU> r"
  using assms unfolding Den_def \<UU>_def by blast

lemma lem_Ldo_ds_reduc:
fixes r s::"'U rel" and n0::nat
assumes a1: "CCR s \<and> DCR n0 s" and a2: "s \<subseteq> r" and a3: "Field s \<in> Den r" and a4: "Field s \<
      in> Inv (r - s)"
shows "CCR r \<and> DCR (Suc n0) r"
proof -
  obtain g0 where b1: "DCR_generating g0"
                and b2: "s = \<Union> {r'. \<exists>\<alpha>'. \<alpha>' < n0 \<and> r' = g0 \<alpha>'}"
    using a1 unfolding DCR_def by blast
  obtain g :: "nat \<Rightarrow> 'U rel"
           where b8: "g = (\<lambda> \<alpha>. if (\<alpha> < n0) then (g0 \<alpha>) else (r- s))" by blast
  obtain n :: nat where b9: "n = (Suc n0)" by blast
  have b11: "\<And> \<alpha>. \<alpha> < n0 \<Longrightarrow> g \<alpha> = (g0 \<alpha>)" using b8 by simp
  have b12: "\<And> \<alpha>. \<not> (\<alpha> < n0) \<Longrightarrow> g \<alpha> = (r- s)" using b8 by force
  have "\<forall>\<alpha> \<beta> a b c.
       \<alpha> \<le> \<beta> \<longrightarrow> (a, b) \<in> g \<alpha> \<and> (a, c) \<in> g \<beta> \<
      longrightarrow>
       (\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c'', d) \<in> \<
      DD> g \<beta> \<alpha>)"
  proof (intro allI impI)
    fix \<alpha> \<beta> a b c
    assume c0: "\<alpha> \<le> \<beta>" and c1: "(a, b) \<in> g \<alpha> \<and> (a, c) \<in> g \<beta>"
    have "\<alpha> < n0 \<and> \<beta> < n0
```

```
  \<longrightarrow> (\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c',
   c'', d) \<in> \<DD> g \<beta> \<alpha>)"
proof
  assume d1: "\<alpha> < n0 \<and> \<beta> < n0"
  moreover then have "(a, b) \<in> g0 \<alpha> \<and> (a, c) \<in> g0 \<beta>" using c1 b11 by blast
  then obtain b' b'' c' c'' d where d2: "(b, b', b'', d) \<in> \<DD> g0 \<alpha> \<beta> \<and> (c, c', c'',
   d) \<in> \<DD> g0 \<beta> \<alpha>"
    using b1 unfolding DCR_generating_def by blast
  have "(b, b', b'', d) \<in> \<DD> g \<alpha> \<beta>"
  proof -
    have "(b, b') \<in> (\<LL>1 g \<alpha>)^*"
    proof -
      have "\<forall> \<alpha>'. \<alpha>' < \<alpha> \<longrightarrow> g \<alpha>' = g0 \<alpha>'" using d1
   b11 by force
      then have "\<LL>1 g \<alpha> = \<LL>1 g0 \<alpha>" unfolding \<LL>1_def by blast
      moreover have "(b,b') \<in> (\<LL>1 g0 \<alpha>)^*" using d2 unfolding \<DD>_def by blast
      ultimately show ?thesis by metis
    qed
    moreover have "(b', b'') \<in> (g \<beta>)^="
    proof -
      have "g \<beta> = g0 \<beta>" using d1 b11 by blast
      moreover have "(b',b'') \<in> (g0 \<beta>)^=" using d2 unfolding \<DD>_def by blast
      ultimately show ?thesis by metis
    qed
    moreover have "(b'', d) \<in> (\<LL>v g \<alpha> \<beta>)^*"
    proof -
      have "\<forall> \<alpha>'. \<alpha>' < \<alpha> \<or> \<alpha>' < \<beta> \<longrightarrow> g \<alpha
  >' = g0 \<alpha>'" using d1 b11 by force
      then have "\<LL>v g \<alpha> \<beta> = \<LL>v g0 \<alpha> \<beta>" unfolding \<LL>v_def by blast
      moreover have "(b'',d) \<in> (\<LL>v g0 \<alpha> \<beta>)^*" using d2 unfolding \<DD>_def by blast
      ultimately show ?thesis by metis
    qed
    ultimately show ?thesis unfolding \<DD>_def by blast
  qed
  moreover have "(c, c', c'', d) \<in> \<DD> g \<beta> \<alpha>"
  proof -
    have "(c, c') \<in> (\<LL>1 g \<beta>)^*"
    proof -
      have "\<forall> \<alpha>'. \<alpha>' < \<beta> \<longrightarrow> g \<alpha>' = g0 \<alpha>'" using d1
   b11 by force
      then have "\<LL>1 g \<beta> = \<LL>1 g0 \<beta>" unfolding \<LL>1_def by blast
      moreover have "(c,c') \<in> (\<LL>1 g0 \<beta>)^*" using d2 unfolding \<DD>_def by blast
      ultimately show ?thesis by metis
    qed
    moreover have "(c', c'') \<in> (g \<alpha>)^="
    proof -
      have "g \<alpha> = g0 \<alpha>" using d1 b11 by blast
      moreover have "(c',c'') \<in> (g0 \<alpha>)^=" using d2 unfolding \<DD>_def by blast
      ultimately show ?thesis by metis
    qed
    moreover have "(c'', d) \<in> (\<LL>v g \<beta> \<alpha>)^*"
    proof -
      have "\<forall> \<alpha>'. \<alpha>' < \<alpha> \<or> \<alpha>' < \<beta> \<longrightarrow> g \<alpha
  >' = g0 \<alpha>'" using d1 b11 by force
      then have "\<LL>v g \<beta> \<alpha> = \<LL>v g0 \<beta> \<alpha>" unfolding \<LL>v_def by blast
      moreover have "(c'',d) \<in> (\<LL>v g0 \<beta> \<alpha>)^*" using d2 unfolding \<DD>_def by blast
      ultimately show ?thesis by metis
    qed
    ultimately show ?thesis unfolding \<DD>_def by blast
  qed
  ultimately show "\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c
   '', d) \<in> \<DD> g \<beta> \<alpha>" by blast
qed
moreover have "\<alpha> < n0 \<and> \<not> (\<beta> < n0)
  \<longrightarrow> (\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c',
   c'', d) \<in> \<DD> g \<beta> \<alpha>)"
proof
  assume d1: "\<alpha> < n0 \<and> \<not> (\<beta> < n0)"
  then have d2: "(a, b) \<in> g0 \<alpha> \<and> (g \<beta>) = (r - s)" using c1 b11 b12 by blast
  have d3: "(a,b) \<in> s \<and> (a,c) \<in> r - s" using d1 d2 c1 b2 unfolding Field_def by blast
  then have "b \<in> Field s \<and> c \<in> Field s" using a4 unfolding Field_def Inv_def by blast
  then obtain d where d6: "d \<in> Field s \<and> (b,d) \<in> s^* \<and> (c,d) \<in> s^*"
    using a1 unfolding CCR_def by blast
  have "\<forall> \<alpha>'. \<alpha>' < n0 \<longrightarrow> \<alpha>' < \<beta>" using d1 by force
  then have "s \<subseteq> \<LL>v g \<alpha> \<beta> \<and> s \<subseteq> \<LL>v g \<beta> \<alpha>" using
  b2 b11 unfolding \<LL>v_def by blast
  then have "(b,d) \<in> (\<LL>v g \<alpha> \<beta>)^* \<and> (c,d) \<in> (\<LL>v g \<beta> \<alpha>)^*"
  using d6 rtrancl_mono by blast
```

```
      then show "\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c'', d)
       \<in> \<DD> g \<beta> \<alpha>"
        unfolding \<DD>_def by blast
    qed
    moreover have "\<not> (\<alpha> < n0) \<and> (\<beta> < n0) \<longrightarrow> False" using c0 by force
    moreover have "\<not> (\<alpha> < n0) \<and> \<not> (\<beta> < n0)
      \<longrightarrow> (\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c',
       c'', d) \<in> \<DD> g \<beta> \<alpha>)"
    proof
      assume d1: "\<not> (\<alpha> < n0) \<and> \<not> (\<beta> < n0)"
      then have d2: "(g \<alpha>) = (r - s) \<and> (g \<beta>) = (r - s)" using b12 by blast
      then have d3: "b \<in> Field r \<and> c \<in> Field r" using c1 unfolding Field_def by blast
      obtain b'' where d4: "b'' \<in> Field s \<and> (b,b'') \<in> r^= \<and> ((b,b'') \<in> s \<longrightarrow>
       b = b'')"
        using a3 d3 unfolding Den_def
        by (cases "\<exists> b''. (b,b'') \<in> s", metis Domain.DomainI Field_def UnCI pair_in_Id_conv, blast)
      obtain c'' where d5: "c'' \<in> Field s \<and> (c,c'') \<in> r^= \<and> ((c,c'') \<in> s \<longrightarrow>
       c = c'')"
        using a3 d3 unfolding Den_def
        by (cases "\<exists> c''. (c,c'') \<in> s", metis Domain.DomainI Field_def UnCI pair_in_Id_conv, blast)
      obtain d where d6: "d \<in> Field s \<and> (b'',d) \<in> s^* \<and> (c'',d) \<in> s^*"
        using d4 d5 a1 unfolding CCR_def by blast
      have "\<forall> \<alpha>'. \<alpha>' < n0 \<longrightarrow> \<alpha>' < \<alpha>" using d1 by force
      then have "s \<subseteq> \<LL>v g \<alpha> \<beta> \<and> s \<subseteq> \<LL>v g \<beta> \<alpha>" using
       b2 b11 unfolding \<LL>v_def by blast
      then have "(b'',d) \<in> (\<LL>v g \<alpha> \<beta>)^* \<and> (c'',d) \<in> (\<LL>v g \<beta> \<alpha>)^*"
       using d6 rtrancl_mono by blast
      moreover have "(b,b'') \<in> (g \<beta>)^=" using d2 d4 by blast
      moreover have "(c,c'') \<in> (g \<alpha>)^=" using d2 d5 by blast
      ultimately show "\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c
      '', d) \<in> \<DD> g \<beta> \<alpha>"
        unfolding \<DD>_def by blast
    qed
    ultimately show "\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c
      '', d) \<in> \<DD> g \<beta> \<alpha>" by blast
  qed
  then have "DCR_generating g" using lem_Ldo_ldogen_ord by blast
  moreover have "r = \<Union> {r'. \<exists>\<alpha>'. \<alpha>' < n \<and> r' = g \<alpha>'}"
  proof -
    have "r \<subseteq> \<Union> {r'. \<exists>\<alpha>'. \<alpha>' < n \<and> r' = g \<alpha>'}"
    proof
      fix p
      assume c1: "p \<in> r"
      have "\<exists> \<alpha>'. \<alpha>' < n \<and> p \<in> g \<alpha>'"
      proof (cases "p \<in> s")
        assume "p \<in> s"
        then obtain \<alpha>' where "\<alpha>' < n0 \<and> p \<in> g \<alpha>'" using b2 b11 by blast
        moreover then have "\<alpha>' < n" using b9 by force
        ultimately show "\<exists> \<alpha>'. \<alpha>' < n \<and> p \<in> g \<alpha>'" by blast
      next
        assume "p \<notin> s"
        moreover have "\<not> ( n < n0)" using b9 by simp
        ultimately have "p \<in> g n0" using c1 b12 by blast
        then show "\<exists> \<alpha>'. \<alpha>' < n \<and> p \<in> g \<alpha>'" using b9 by blast
      qed
      then show "p \<in> \<Union> {r'. \<exists>\<alpha>'. \<alpha>' < n \<and> r' = g \<alpha>'}" by blast
    qed
    moreover have "\<forall> \<alpha>'. g \<alpha>' \<subseteq> r"
    proof
      fix \<alpha>'
      have "\<alpha>' < n0 \<longrightarrow> g0 \<alpha>' \<subseteq> r" using a2 b2 by blast
      then show "g \<alpha>' \<subseteq> r" using b8 by (cases "\<alpha>' < n0", force+)
    qed
    ultimately show ?thesis by force
  qed
  moreover have "CCR r" using a1 a2 a3 lem_Ldo_den_ccr_uset lem_rcc_uset_ne_ccr by blast
  ultimately show ?thesis unfolding b9 DCR_def by blast
qed

lemma lem_Ldo_sat_reduc:
fixes r s::"'U rel" and n::nat
assumes a1: "s \<in> Span r" and a2: "CCR s \<and> DCR n s"
shows "CCR r \<and> DCR (Suc n) r"
proof -
  have "Field s \<in> Inv (r - s)" using a1 unfolding Span_def Inv_def Field_def by blast
  moreover have "s \<subseteq> r" and "Field s \<in> Den r" using a1 unfolding Span_def Den_def by blast+
  ultimately show ?thesis using a2 lem_Ldo_ds_reduc by blast
qed
```

```
lemma lem_Ldo_uset_reduc:
fixes r s::"'U rel" and n0::nat
assumes a1: "s \<in> \<UU> r" and a2: "DCR n0 s" and a3: "n0 \<noteq> 0"
shows "DCR (Suc n0) r"
proof -
  have b0: "s \<subseteq> r" using a1 unfolding \<UU>_def by blast
  obtain g0 where b1: "DCR_generating g0"
             and b2: "s = \<Union> {r'. \<exists>\<alpha>'. \<alpha>' < n0 \<and> r' = g0 \<alpha>'}"
    using a2 unfolding DCR_def by blast
  obtain isd where b3: "isd = (\<lambda> a i. \<exists> b \<in> Field s. (a, b) \<in> r^^i \<and> (\<forall> i'.
       (\<exists> b \<in> Field s. (a, b) \<in> r^^(i')) \<longrightarrow> i \<le> i'))" by blast
  obtain d where b4: "d = (\<lambda> a. SOME i. isd a i)" by blast
  obtain B where b5: "B = (\<lambda> a. { a'. (a, a') \<in> r })" by blast
  obtain H where b6: "H = (\<lambda> a. { a' \<in> B a. \<forall> a'' \<in> B a. (d a') \<le> (d a'') })" by
      blast
  obtain D where b7: "D = { a \<in> Field r - Field s. H a \<noteq> {}}" by blast
  obtain h where "h = (\<lambda> a. SOME a'. a' \<in> H a)" by blast
  then have b8: "\<forall> a \<in> D. h a \<in> H a" using b7 someI_ex[of "\<lambda> a'. a' \<in> H _"] by force
  have q1: "\<And> a. a \<in> Field r \<Longrightarrow> isd a (d a)"
  proof -
    fix a
    assume c1: "a \<in> Field r"
    then obtain b where c2: "b \<in> Field s \<and> (a,b) \<in> r^*" using a1 unfolding \<UU>_def by blast
    moreover obtain N where c3: "N = {i. \<exists> b \<in> Field s. (a, b) \<in> r^^i}" by blast
    ultimately have "N \<noteq> {}" using rtrancl_imp_relpow by blast
    then obtain m where "m \<in> N \<and> (\<forall> i \<in> N. m \<le> i)"
      using LeastI[of "\<lambda> x. x \<in> N"] Least_le[of "\<lambda> x. x \<in> N"] by blast
    then have "isd a m" using c2 c3 unfolding b3 by blast
    then show "isd a (d a)" using b4 someI_ex by metis
  qed
  have q2: "\<And> a. B a \<noteq> {} \<Longrightarrow> H a \<noteq> {}"
  proof -
    fix a
    assume "B a \<noteq> {}"
    moreover obtain N where c1: "N = d ` (B a)" by blast
    ultimately have "N \<noteq> {}"  by blast
    then obtain m where c2: "m \<in> N \<and> (\<forall> i \<in> N. m \<le> i)"
      using LeastI[of "\<lambda> x. x \<in> N"] Least_le[of "\<lambda> x. x \<in> N"] by blast
    then obtain a' where c3: "m = d a' \<and> a' \<in> B a" using c1 by blast
    moreover then have "\<forall> a'' \<in> B a. d a' \<le> d a''" using c1 c2 by force
    ultimately have "a' \<in> H a" unfolding b6 by blast
    then show "H a \<noteq> {}" by blast
  qed
  have q3: "\<forall> a \<in> Field r - Field s. d a = 1 \<or> d a > 1"
  proof
    fix a
    assume c1: "a \<in> Field r - Field s"
    then have "isd a (d a)" using q1 by blast
    then obtain b where "b \<in> Field s \<and> (a, b) \<in> r^^(d a)" using b3 by blast
    then have "d a = 0 \<longrightarrow> False" using c1 by force
    then show "d a = 1 \<or> d a > 1" by force
  qed
  have "Field r - Field s \<subseteq> D"
  proof
    fix a
    assume c1: "a \<in> Field r - Field s"
    moreover have "H a = {} \<longrightarrow> False"
    proof
      assume "H a = {}"
      then have "B a = {}" using q2 by blast
      moreover obtain b where "b \<in> Field s \<and> (a, b) \<in> r^*" using a1 c1 unfolding \<UU>_def by blast
      ultimately have "a \<in> Field s" unfolding b5 by (metis Collect_empty_eq converse_rtranclE)
      then show "False" using c1 by blast
    qed
    ultimately show "a \<in> D" using b7 by blast
  qed
  then have q4: "D = Field r - Field s" using b5 b6 b7 by blast
  have q5: "\<forall> a \<in> D. d a > 1 \<longrightarrow> d a = Suc (d (h a)) \<and> (d (h a) > 1 \<
      longrightarrow> h a \<in> D)"
  proof (intro ballI impI)
    fix a
    assume c1: "a \<in> D" and c2: "d a > 1"
    then obtain b where c3: "b \<in> Field s" and c4: "(a, b) \<in> r^^(d a)"
                    and c5: "\<forall> i'. (\<exists> b \<in> Field s. (a, b) \<in> r^^(i')) \<longrightarrow> (
      d a) \<le> i'"
                    using b3 b7 q1 by blast
    have c6: "d a \<ge> 1" using c1 c4 b7 q3 by force
```

```
  then have "d a = Suc ((d a) - 1)" by simp
  then obtain a' where c7: "(a,a') \<in> r \<and> (a',b) \<in> r^^((d a) - 1)"
    using c4 relpow_Suc_D2[of a b "d a - 1" r] by metis
  moreover then have "a' \<notin> Field s" using c2 c5 by (metis less_Suc_eq_le not_less_eq relpow_1)
  ultimately have "(a,a') \<in> r \<and> a' \<in> Field r - Field s" unfolding Field_def by blast
  then have "a' \<in> B a" unfolding b5 by blast
  moreover have "h a \<in> H a" using c1 b8 by blast
  ultimately have "d (h a) \<le> d a'" unfolding b6 by blast
  moreover have "Suc (d a') \<le> d a"
  proof -
    have "d a' \<le> d a - 1" using q1 b3 c7 c3 unfolding Field_def by blast
    then show ?thesis using c6 by force
  qed
  moreover have "d a \<le> (Suc (d (h a)))"
  proof -
    have d1: "(a, h a) \<in> r" using c1 b5 b6 b8 by blast
    then have "h a \<in> Field r" unfolding Field_def by blast
    then obtain b' where "b' \<in> Field s \<and> ((h a), b') \<in> r^^(d (h a))" using b3 q1 by blast
    moreover then have "(a,b') \<in> r^^(Suc (d (h a)))" using d1 c7 by (meson relpow_Suc_I2)
    ultimately show "d a \<le> (Suc (d (h a)))" using c5 by blast
  qed
  ultimately have "d a = Suc (d (h a))" by force
  moreover have "d (h a) > 1 \<longrightarrow> h a \<in> D"
  proof
    assume d1: "d (h a) > 1"
    then have d2: "(a, h a) \<in> r" using c1 b5 b6 b8 by simp
    then have "isd (h a) (d (h a))" using d1 q1 unfolding Field_def by force
    then have "(h a) \<notin> Field s" using d1 b3 by force
    then show "h a \<in> D" using d2 q4 unfolding Field_def by blast
  qed
  ultimately show "d a = Suc (d (h a)) \<and> (d (h a) > 1 \<longrightarrow> h a \<in> D)" by blast
qed
obtain g1 where b9:  "g1 = { (a, b). a \<in> D \<and> b = h a }" by blast
have q6: "\<forall> a \<in> D. \<exists> a' \<in> D. d a' = 1 \<and> (a,a') \<in> g1^*"
proof -
  have "\<forall> n. \<forall> a \<in> D. d a = Suc n \<longrightarrow> ((h^^n) a) \<in> D \<and> d ((h^^n) a)
     = 1"
  proof
    fix n0
    show "\<forall> a \<in> D. d a = Suc n0 \<longrightarrow> ((h^^n0) a) \<in> D \<and> d ((h^^n0) a) = 1"
    proof (induct n0)
      show "\<forall>a\<in>D. d a = Suc 0 \<longrightarrow> ((h^^0) a) \<in> D \<and> d ((h ^^ 0) a) = 1"
        using q4 by force
    next
      fix n
      assume d1: "\<forall>a\<in>D. d a = Suc n \<longrightarrow> ((h^^n) a) \<in> D \<and> d ((h ^^ n) a) =
    1"
      show "\<forall>a\<in>D. d a = Suc (Suc n) \<longrightarrow> ((h^^(Suc n)) a) \<in> D \<and> d ((h ^^ Suc
     n) a) = 1"
      proof (intro ballI impI)
        fix a
        assume e1: "a \<in> D" and e2: "d a = Suc (Suc n)"
        then have "d a = Suc (d (h a)) \<and> (d (h a) > 1 \<longrightarrow> h a \<in> D)" using q5 by simp
        moreover then have e3: "d (h a) = Suc n" using e2 by simp
        ultimately have "d (h a) > 1 \<longrightarrow> ((h^^n) (h a)) \<in> D \<and> d ((h^^n) (h a)) = 1"
    using d1 by blast
        moreover have "(h^^n) (h a) = (h^^(Suc n)) a" by (metis comp_apply funpow_Suc_right)
        moreover have e4: "d (h a) = 1 \<longrightarrow> d ((h^^(Suc n)) a) = 1" using e3 by simp
        moreover have "d (h a) = 1 \<longrightarrow> ((h^^(Suc n)) a) \<in> D"
        proof
          assume f1: "d (h a) = 1"
          then have f2: "n = 0 \<and> (a, h a) \<in> r" using e1 e3 b5 b6 b8 by simp
          then have "isd (h a) 1" using f1 q1 unfolding Field_def by force
          then have "(h a) \<notin> Field s" using b3 by force
          then have "(h a) \<in> D" using q4 f2 unfolding Field_def by blast
          then show "((h^^(Suc n)) a) \<in> D" using f2 by simp
        qed
        moreover have "d (h a) = 1 \<or> d (h a) > 1" using e3 by force
        ultimately show "((h^^(Suc n)) a) \<in> D \<and> d ((h ^^ (Suc n)) a) = 1" by force
      qed
    qed
  qed
  moreover have "\<forall> i. \<forall> a \<in> D. d a > i \<longrightarrow> (a, (h^^i) a) \<in> g1^*"
  proof
    fix i0
    show "\<forall> a \<in> D. d a > i0 \<longrightarrow> (a, (h^^i0) a) \<in> g1^*"
    proof (induct i0)
      show "\<forall>a\<in>D. d a > 0 \<longrightarrow> (a, (h^^0) a) \<in> g1^*" by force
```

```
    next
      fix i
      assume d1: "\<forall>a\<in>D. d a > i \<longrightarrow> (a, (h^^i) a) \<in> g1^*"
      show "\<forall>a\<in>D. d a > (Suc i) \<longrightarrow> (a, (h^^(Suc i)) a) \<in> g1^*"
      proof (intro ballI impI)
        fix a
        assume e1: "a \<in> D" and e2: "d a > (Suc i)"
        then have e3: "d a = Suc (d (h a)) \<and> (d (h a) > 1 \<longrightarrow> h a \<in> D)" using q5 by
    simp
        moreover then have e4: "d (h a) > i" using e2 by simp
        ultimately have "d (h a) > 1 \<longrightarrow> (h a, (h^^i) (h a)) \<in> g1^*" using d1 by simp
        moreover have "(h^^i) (h a) = (h^^(Suc i)) a" by (metis comp_apply funpow_Suc_right)
        moreover have "d (h a) = 1 \<longrightarrow> (h^^(Suc i)) a = (h a)" using e4 by force
        moreover have "d (h a) = 1 \<or> d (h a) > 1" using e4 by force
        moreover then have "(a, h a) \<in> g1" using e1 e3 unfolding b9 by simp
        ultimately show "(a, (h^^(Suc i)) a) \<in> g1^*"
          by (metis converse_rtrancl_into_rtrancl r_into_rtrancl)
      qed
    qed
  qed
  ultimately have "\<forall>n. \<forall>a\<in>D. d a = Suc n \<longrightarrow> (h^^n) a \<in> D \<and> d ((h^^
    n) a) = 1 \<and> (a, (h ^^ n) a) \<in> g1^*"
    by simp
  then have "\<forall>n. \<forall>a\<in>D. d a = Suc n \<longrightarrow> (\<exists> a' \<in> D. d a' = 1 \<and
    > (a,a') \<in> g1^* )"
    by blast
  moreover have "\<forall> a \<in> D. \<exists> n. d a = Suc n" using q3 q4 q5 by force
  ultimately show ?thesis by blast
qed
let ?cond1 = "\<lambda> \<alpha>. \<alpha> = 0"
let ?cond3 = "\<lambda> \<alpha>. (1 \<le> \<alpha> \<and> \<alpha> < n0)"
obtain g :: "nat \<Rightarrow> 'U rel"
          where b12: "g = (\<lambda> \<alpha>. if (?cond1 \<alpha>) then (g0 \<alpha>) \<union> g1
                         else (if (?cond3 \<alpha>) then (g0 \<alpha>)
                         else {} ))" by blast
obtain n :: nat where b13: "n = n0" by blast
then have b14: "\<And> \<alpha>. \<alpha> < n \<Longrightarrow> (?cond1 \<alpha> \<or> ?cond3 \<alpha>)" by
    force
have b15: "\<And> \<alpha>. ?cond1 \<alpha> \<Longrightarrow> g \<alpha> = (g0 \<alpha>) \<union> g1" using
    b12 by simp
have b17: "\<And> \<alpha>. ?cond3 \<alpha> \<Longrightarrow> g \<alpha> = (g0 \<alpha>)" using b12 by force
obtain r1 where b19: "r1 = \<Union> {r'. \<exists>\<alpha>'. \<alpha>' < n \<and> r' = g \<alpha>'}" by blast
have t1: "g1 \<subseteq> r1" using b15 b19 b13 a3 by blast
have b20: "s \<subseteq> r1"
proof
  fix p
  assume "p \<in> s"
  then obtain \<alpha>' where c1: "\<alpha>' < n0 \<and> p \<in> g0 \<alpha>'" using b2 by blast
  then have c2: "\<alpha>' < n" unfolding b13 by fastforce
  then have "?cond1 \<alpha>' \<or> ?cond3 \<alpha>'" using b14 by blast
  then have "g0 \<alpha>' \<subseteq> g \<alpha>'" using b12 by fastforce
  then show "p \<in> r1" using c1 c2 b19 by blast
qed
have b21: "r1 \<subseteq> r"
proof -
  have "\<forall> r' \<alpha>'. \<alpha>' < n \<longrightarrow> g \<alpha>' \<subseteq> r"
  proof (intro allI impI)
    fix r' \<alpha>'
    assume d1: "\<alpha>' < n"
    have "\<forall> a \<in> D. (a, h a) \<in> r" using b5 b6 b8 by blast
    then have d2: "g1 \<subseteq> r" using b9 by blast
    have "(\<alpha>' = 0) \<longrightarrow> g \<alpha>' \<subseteq> r" using d2 b0 b2 b15[of \<alpha>'] a3 by
     blast
    moreover have "1 \<le> \<alpha>' \<longrightarrow> g \<alpha>' \<subseteq> r" using b17 b0 b2 b13 d1 by
     blast
    ultimately show "g \<alpha>' \<subseteq> r" using d1 b14 by blast
  qed
  then show "r1 \<subseteq> r" unfolding b19 by fast
qed
have b22: "\<forall>a \<in> Field r1 - Field s. \<exists>b \<in> Field s. (a, b) \<in> r1^*"
proof
  fix a
  assume d1: "a \<in> Field r1 - Field s"
  then have "a \<in> D" using q4 b21 unfolding Field_def by blast
  then obtain a' where d2: "a' \<in> D \<and> d a' = 1 \<and> (a, a') \<in> g1^*" using q6 by blast
  then have d3: "(a', h a') \<in> r1 \<and> h a' \<in> H a'" using q4 b8 b9 t1 a3 by blast
  obtain b where "b \<in> Field s \<and> (a',b) \<in> r" using d2 q1 q4 b3 by force
  moreover then have "isd b (d b)" using q1 unfolding Field_def by blast
```

```
  ultimately have "b \<in> B a' \<and> d b = 0" using b3 b5 by force
  then have "d (h a') = 0" using d3 b6 by force
  then have "isd (h a') 0" using q1 d3 b21 a3 unfolding Field_def by force
  then have "h a' \<in> Field s" using b3 by force
  moreover have "(a, a') \<in> r1^*" using d2 t1 rtrancl_mono[of g1 r1] a3 by blast
  ultimately have "(h a') \<in> Field s \<and> (a, h a') \<in> r1^*" using d3 by force
  then show "\<exists>b \<in> Field s. (a, b) \<in> r1^*" by blast
qed
have b23: "Field r \<subseteq> Field r1"
proof -
  have "(Field r - Field s) \<subseteq> Field r1" using q4 b9 t1 unfolding Field_def by blast
  moreover have "Field s \<subseteq> Field r1" using b20 unfolding Field_def by blast
  ultimately show "Field r \<subseteq> Field r1" by blast
qed
have "\<forall>\<alpha> \<beta> a b c. \<alpha> \<le> \<beta> \<longrightarrow> (a,b) \<in> g \<alpha> \<and>
    (a,c) \<in> g \<beta> \<longrightarrow>
     (\<exists>b' b'' c' c'' d. (b,b',b'',d) \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d) \<in> \<DD> g
    \<beta> \<alpha>)"
proof (intro allI impI)
  fix \<alpha> \<beta> a b c
  assume c1: "\<alpha> \<le> \<beta>" and c2: "(a,b) \<in> g \<alpha> \<and> (a,c) \<in> g \<beta>"
  obtain c123 where c0: "c123 = (\<lambda> \<alpha>::nat. ?cond1 \<alpha> \<or> ?cond3 \<alpha>)" by blast
  have c3: "\<And> \<alpha>'. c123 \<alpha>' \<Longrightarrow> g0 \<alpha>' \<subseteq> s"
  proof -
    fix \<alpha>'
    assume "c123 \<alpha>'"
    moreover have "?cond1 \<alpha>' \<longrightarrow> g0 \<alpha>' \<subseteq> s" using a3 unfolding b2 by
    force
    moreover have "?cond3 \<alpha>' \<longrightarrow> g0 \<alpha>' \<subseteq> s" using b2 by force
    ultimately show "g0 \<alpha>' \<subseteq> s" using c0 by blast
  qed
  have c4: "\<And>\<alpha>'. \<And> p. p \<in> g \<alpha>' \<longrightarrow> (?cond1 \<alpha>' \<and> p \<in>
    (g0 \<alpha>' \<union> g1)) \<or> (?cond3 \<alpha>' \<and> p \<in> (g0 \<alpha>'))"
  proof (intro impI)
    fix \<alpha>' p
    assume "p \<in> g \<alpha>'"
    then show "(?cond1 \<alpha>' \<and> p \<in> (g0 \<alpha>' \<union> g1)) \<or> (?cond3 \<alpha>' \<and> p
    \<in> (g0 \<alpha>'))"
      using b12 by (cases "?cond1 \<alpha>'", simp, cases "?cond3 \<alpha>'", force+)
  qed
  have c5: "\<And> \<alpha>' \<beta>'. \<alpha>' \<le> \<beta>' \<Longrightarrow> c123 \<beta>' \<
    Longrightarrow> c123 \<alpha>'" unfolding c0 using b14 by force
  have c6: "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<notin> g0 \<beta> \<longrightarrow> \<not> c123 \<beta>"
  proof
    assume d1: "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<notin> g0 \<beta>"
    then have "(a,c) \<in> g1" using c2 c4 by blast
    then have "a \<in> Field r - Field s" using b7 b9 by blast
    then have "\<not> c123 \<alpha>" using d1 c3 unfolding Field_def by blast
    then show "\<not> c123 \<beta>" using c1 c5 by blast
  qed
  have c7: "(a,b) \<notin> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta> \<longrightarrow> \<not> c123 \<beta>"
  proof
    assume d1: "(a,b) \<notin> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta>"
    then have "(a,b) \<in> g1" using c2 c4 by blast
    then have "a \<in> Field r - Field s" using b7 b9 by blast
    then show " \<not> c123 \<beta>" using d1 c3 unfolding Field_def by blast
  qed
  have c8: "\<And> \<alpha>'. c123 \<alpha>' \<Longrightarrow> g0 \<alpha>' \<subseteq> g \<alpha>'"
  proof -
    fix \<alpha>'
    assume "c123 \<alpha>'"
    then show "g0 \<alpha>' \<subseteq> g \<alpha>'" unfolding c0 using b15[of \<alpha>'] b17[of \<alpha>'] by
     blast
  qed
  then have c9: "\<And> \<alpha>' \<alpha>''. c123 \<alpha>' \<Longrightarrow> \<alpha>'' < \<alpha>' \<
    Longrightarrow> g0 \<alpha>'' \<subseteq> g \<alpha>''"
    using c5 less_or_eq_imp_le by blast
  have c10: "\<And> \<alpha>' \<beta>'. c123 \<alpha>' \<Longrightarrow> c123 \<beta>' \<Longrightarrow> \<DD>
     g0 \<alpha>' \<beta>' \<subseteq> \<DD> g \<alpha>' \<beta>'"
  proof -
    fix \<alpha>' \<beta>'
    assume d1: "c123 \<alpha>'" and d2: "c123 \<beta>'"
    have "\<LL>1 g0 \<alpha>' \<subseteq> \<LL>1 g \<alpha>'" using d1 c9 unfolding \<LL>1_def by blast
    moreover have "\<LL>v g0 \<alpha>' \<beta>' \<subseteq> \<LL>v g \<alpha>' \<beta>'" using d1 d2 c9
    unfolding \<LL>v_def by blast
    ultimately have "(\<LL>1 g0 \<alpha>')^* \<subseteq> (\<LL>1 g \<alpha>')^* \<and> (\<LL>v g0 \<alpha>' \<
    beta>')^* \<subseteq> (\<LL>v g \<alpha>' \<beta>')^*"
      using rtrancl_mono by blast
```

```
      moreover have "g0 \<beta>' \<subseteq> g \<beta>'" using d2 c8 by blast
      ultimately show "\<DD> g0 \<alpha>' \<beta>' \<subseteq> \<DD> g \<alpha>' \<beta>'" unfolding \<DD>_def
      by blast
    qed
    show "\<exists>b' b'' c' c'' d'. (b,b',b'',d') \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d') \<in> \<
      DD> g \<beta> \<alpha>"
    proof (cases "c123 \<beta>")
      assume d1: "c123 \<beta>"
      show ?thesis
      proof (cases "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta>")
        assume e1: "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta>"
        then obtain b' b'' c' c'' d' where "(b, b', b'', d') \<in> \<DD> g0 \<alpha> \<beta> \<and> (c, c', c'',
       d') \<in> \<DD> g0 \<beta> \<alpha>"
          using b1 unfolding DCR_generating_def by blast
        moreover have "c123 \<alpha>" using d1 c1 c5 by blast
        ultimately have "(b, b', b'', d') \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c'', d') \<in> \<DD> g
      \<beta> \<alpha>" using d1 c10 by blast
        then show ?thesis by blast
      next
        assume "\<not> ((a,b) \<in> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta>)"
        then have "(a,b) \<notin> g0 \<alpha> \<and> (a,c) \<notin> g0 \<beta>" using d1 c6 c7 by blast
        moreover have "c123 \<alpha>" using d1 c1 c5 by blast
        ultimately have "(a,b) \<in> g1 \<and> (a,c) \<in> g1" using d1 c0 c2 c4 by blast
        then have "b = c" using b9 by blast
        then show ?thesis unfolding \<DD>_def by blast
      qed
    next
      assume d1: "\<not> c123 \<beta>"
      then have d2: "False" using c2 c4 unfolding c0 by blast
      then show ?thesis by blast
    qed
  qed
  then have b24: "DCR_generating g" using a3 lem_Ldo_ldogen_ord by blast
  moreover then have "Field r1 \<subseteq> Field r" using b21 unfolding Field_def by blast
  ultimately have "r1 \<in> Span r" using b21 b23 unfolding Span_def by blast
  moreover have "DCR n r1" using b19 b24 unfolding DCR_def by blast
  moreover have "CCR r1"
  proof -
    have "s \<in> \<UU> r1" using b20 b22 a1 unfolding \<UU>_def by blast
    then show "CCR r1" using lem_rcc_uset_ne_ccr by blast
  qed
  ultimately show "DCR (Suc n0) r" using b13 a3 lem_Ldo_sat_reduc by blast
qed

lemma lem_Ldo_addid:
fixes r::"'U rel" and r'::"'U rel" and n0::nat and A::"'U set"
assumes a1: "DCR n0 r" and a2: "r' = r \<union> {(a,b). a = b \<and> a \<in> A}" and a3: "n0 \<noteq> 0"
shows "DCR n0 r'"
proof -
  obtain g0 where b1: "DCR_generating g0" and b2: "r = \<Union>{r'. \<exists>\<alpha>'<n0. r' = g0 \<alpha>'}"
      using a1 unfolding DCR_def by blast
  obtain g :: "nat \<Rightarrow> 'U rel" where b3: "g = (\<lambda> \<alpha>. (g0 \<alpha>) \<union> {(a,b). a =
      b \<and> a \<in> A})" by blast
  have "\<forall>\<alpha> \<beta> a b c. \<alpha> \<le> \<beta> \<longrightarrow> (a,b) \<in> g \<alpha> \<and>
      (a,c) \<in> g \<beta> \<longrightarrow>
       (\<exists>b' b'' c' c'' d. (b,b',b'',d) \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d) \<in> \<DD> g
      \<beta> \<alpha>)"
  proof (intro allI impI)
    fix \<alpha> \<beta> a b c
    assume c1: "\<alpha> \<le> \<beta>" and c2: "(a,b) \<in> g \<alpha> \<and> (a,c) \<in> g \<beta>"
    have c3: "\<And> \<alpha>' \<beta>'. \<DD> g0 \<alpha>' \<beta>' \<subseteq> \<DD> g \<alpha>' \<beta>'"
    proof -
      fix \<alpha>' \<beta>'
      have "\<LL>1 g0 \<alpha>' \<subseteq> (\<LL>1 g \<alpha>')^=" unfolding \<LL>1_def b3 by (clarsimp, auto)
      moreover have "\<LL>v g0 \<alpha>' \<beta>' \<subseteq> (\<LL>v g \<alpha>' \<beta>')^=" unfolding \<LL>
      v_def b3 by (clarsimp, auto)
      ultimately have "(\<LL>1 g0 \<alpha>')^* \<subseteq> (\<LL>1 g \<alpha>')^* \<and> (\<LL>v g0 \<alpha>' \<
      beta>')^* \<subseteq> (\<LL>v g \<alpha>' \<beta>')^*" using rtrancl_reflcl rtrancl_mono by blast
      moreover have "(g0 \<beta>')^= \<subseteq> (g \<beta>')^=" unfolding b3 by force
      ultimately show "\<DD> g0 \<alpha>' \<beta>' \<subseteq> \<DD> g \<alpha>' \<beta>'" unfolding \<DD>_def
      by blast
    qed
    have c4: "((a,b) \<in> g0 \<alpha> \<or> a = b) \<and> ((a,c) \<in> g0 \<beta> \<or> a = c)" using c1 c2 b3
      by blast
    moreover then have "a = b \<or> a = c \<longrightarrow> (\<exists>b' b'' c' c'' d. (b,b',b'',d) \<in> \<DD>
      g \<alpha> \<beta> \<and> (c,c',c'',d) \<in> \<DD> g \<beta> \<alpha>)"
      using b3 unfolding \<DD>_def by blast
```

```
    moreover have "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta> \<longrightarrow> (\<exists>b' b'' c' c
      '' d. (b,b',b'',d) \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d) \<in> \<DD> g \<beta> \<alpha>)"
    proof
      assume "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta>"
      then obtain b' b'' c' c'' d' where "(b, b', b'', d') \<in> \<DD> g0 \<alpha> \<beta> \<and> (c, c', c'', d
      ') \<in> \<DD> g0 \<beta> \<alpha>"
        using b1 unfolding DCR_generating_def by blast
      then have "(b, b', b'', d') \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c'', d') \<in> \<DD> g \<beta>
      \<alpha>" using c3 by blast
      then show "\<exists>b' b'' c' c'' d'. (b,b',b'',d') \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d') \<
      in> \<DD> g \<beta> \<alpha>" by blast
    qed
    ultimately show "\<exists>b' b'' c' c'' d. (b,b',b'',d) \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d)
      \<in> \<DD> g \<beta> \<alpha>" by blast
  qed
  then have "DCR_generating g" using lem_Ldo_ldogen_ord by blast
  moreover have "r' = \<Union>{s. \<exists>\<alpha>'<n0. s = g \<alpha>'}" unfolding b2 b3 a2 using a3 by blast
  ultimately show "DCR n0 r'" unfolding DCR_def by blast
qed

lemma lem_Ldo_removeid:
fixes r::"'U rel" and r'::"'U rel" and n0::nat
assumes a1: "DCR n0 r" and a2: "r' = r - {(a,b). a = b}"
shows "DCR n0 r'"
proof -
  obtain g0 where b1: "DCR_generating g0" and b2: "r = \<Union>{r'. \<exists>\<alpha>'<n0. r' = g0 \<alpha>'}"
      using a1 unfolding DCR_def by blast
  obtain g :: "nat \<Rightarrow> 'U rel" where b3: "g = (\<lambda> \<alpha>. (g0 \<alpha>) - {(a,b). a = b })"
      by blast
  have "\<forall>\<alpha> \<beta> a b c. \<alpha> \<le> \<beta> \<longrightarrow> (a,b) \<in> g \<alpha> \<and>
      (a,c) \<in> g \<beta> \<longrightarrow>
       (\<exists>b' b'' c' c'' d. (b,b',b'',d) \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d) \<in> \<DD> g
      \<beta> \<alpha>)"
  proof (intro allI impI)
    fix \<alpha> \<beta> a b c
    assume c1: "\<alpha> \<le> \<beta>" and c2: "(a,b) \<in> g \<alpha> \<and> (a,c) \<in> g \<beta>"
    have c3: "\<And> \<alpha>' \<beta>'. \<DD> g0 \<alpha>' \<beta>' \<subseteq> \<DD> g \<alpha>' \<beta>'"
    proof -
      fix \<alpha>' \<beta>'
      have "\<LL>1 g0 \<alpha>' \<subseteq> (\<LL>1 g \<alpha>')^=" unfolding \<LL>1_def b3 by (clarsimp, auto)
      moreover have "\<LL>v g0 \<alpha>' \<beta>' \<subseteq> (\<LL>v g \<alpha>' \<beta>')^=" unfolding \<LL>
      v_def b3 by (clarsimp, auto)
      ultimately have "(\<LL>1 g0 \<alpha>')^* \<subseteq> (\<LL>1 g \<alpha>')^* \<and> (\<LL>v g0 \<alpha>' \<
      beta>')^* \<subseteq> (\<LL>v g \<alpha>' \<beta>')^*" using rtrancl_reflcl rtrancl_mono by blast
      moreover have "(g0 \<beta>')^= \<subseteq> (g \<beta>')^=" unfolding b3 by force
      ultimately show "\<DD> g0 \<alpha>' \<beta>' \<subseteq> \<DD> g \<alpha>' \<beta>'" unfolding \<DD>_def
      by blast
    qed
    have "(a,b) \<in> g0 \<alpha> \<and> (a,c) \<in> g0 \<beta>" using c1 c2 b3 by blast
    then obtain b' b'' c' c'' d' where "(b, b', b'', d') \<in> \<DD> g0 \<alpha> \<beta> \<and> (c, c', c'', d')
       \<in> \<DD> g0 \<beta> \<alpha>"
      using b1 unfolding DCR_generating_def by blast
    then have "(b, b', b'', d') \<in> \<DD> g \<alpha> \<beta> \<and> (c, c', c'', d') \<in> \<DD> g \<beta> \<
      alpha>" using c3 by blast
    then show "\<exists>b' b'' c' c'' d'. (b,b',b'',d') \<in> \<DD> g \<alpha> \<beta> \<and> (c,c',c'',d') \<in
      > \<DD> g \<beta> \<alpha>" by blast
  qed
  then have "DCR_generating g" using lem_Ldo_ldogen_ord by blast
  moreover have "r' = \<Union>{s. \<exists>\<alpha>'<n0. s = g \<alpha>'}" unfolding b2 b3 a2 by blast
  ultimately show "DCR n0 r'" unfolding DCR_def by blast
qed

lemma lem_Ldo_eqid:
fixes r::"'U rel" and r'::"'U rel" and n::nat
assumes a1: "DCR n r" and a2: "r' - {(a,b). a = b} = r - {(a,b). a = b}" and a3: "n \<noteq> 0"
shows "DCR n r'"
proof -
  obtain r'' where b1: "r'' = r' - {(a,b). a = b}" by blast
  then have "DCR n r''" using a1 a2 lem_Ldo_removeid by blast
  moreover have "r' = r'' \<union> {(a,b). a = b \<and> (a,a) \<in> r'}" using b1 by blast
  ultimately show "DCR n r'" using lem_Ldo_addid[of n r'' r' "{a . (a,a) \<in> r'}"] a3 by blast
qed

lemma lem_wdn_range_lb: "A \<subseteq> w_dncl r A"
  unfolding w_dncl_def dncl_def \<F>_def rpth_def by fastforce

lemma lem_wdn_range_ub: "w_dncl r A \<subseteq> dncl r A" unfolding w_dncl_def by blast
```

```
lemma lem_wdn_mon: "A \<subseteq> A' \<Longrightarrow> w_dncl r A \<subseteq> w_dncl r A'" unfolding w_dncl_def
      dncl_def by fast

lemma lem_wdn_compl:
fixes r::"'U rel" and A::"'U set"
shows "UNIV - w_dncl r A = {a. \<exists> b. b \<notin> dncl r A \<and> (a,b) \<in> (Restr r (UNIV-A))^*}"
proof
  show "UNIV - w_dncl r A \<subseteq> {a. \<exists> b. b \<notin> dncl r A \<and> (a,b) \<in> (Restr r (UNIV-A))
      ^*}"
  proof
    fix x
    assume c1: "x \<in> UNIV - w_dncl r A"
    show "x \<in> {a. \<exists> b. b \<notin> dncl r A \<and> (a,b) \<in> (Restr r (UNIV-A))^*}"
    proof (cases "x \<in> dncl r A")
      assume "x \<in> dncl r A"
      then obtain b F where d1: "F \<in> \<F> r x b \<and> b \<notin> dncl r A \<and> F \<inter> A = {}"
        using c1 unfolding w_dncl_def by blast
      then obtain f n where "f \<in> rpth r x b n \<and> F = f ` {i. i\<le>n}" unfolding \<F>_def by blast
      moreover then have "\<forall>i\<le>n. f i \<notin> A" using d1 unfolding rpth_def by blast
      ultimately have "f \<in> rpth (Restr r (UNIV-A)) x b n" unfolding rpth_def by force
      then have "(x,b) \<in> (Restr r (UNIV-A))^*" using lem_ccext_rpth_rtr[of "Restr r (UNIV-A)"] by blast
      then show ?thesis using d1 by blast
    next
      assume "x \<notin> dncl r A"
      then show ?thesis unfolding w_dncl_def by blast
    qed
  qed
next
  show "{a. \<exists> b. b \<notin> dncl r A \<and> (a,b) \<in> (Restr r (UNIV-A))^*} \<subseteq> UNIV - w_dncl
      r A"
  proof
    fix x
    assume "x \<in> {a. \<exists> b. b \<notin> dncl r A \<and> (a,b) \<in> (Restr r (UNIV-A))^*}"
    then obtain y where c1: "y \<notin> dncl r A \<and> (x,y) \<in> (Restr r (UNIV-A))^*" by blast
    obtain f n where c2: "f \<in> rpth (Restr r (UNIV-A)) x y n" using c1 lem_ccext_rtr_rpth[of x y] by blast
    then have c3: "f \<in> rpth r x y n" unfolding rpth_def by blast
    obtain F where c4: "F = f`{i. i\<le>n}" by blast
    have "n = 0 \<longrightarrow> f 0 \<notin> A" using c1 c3 unfolding rpth_def dncl_def by blast
    moreover have "\<forall> i<n. f i \<notin> A \<and> f (Suc i) \<notin> A" using c2 unfolding rpth_def by
      blast
    moreover have "\<forall> i\<le>n. (n = 0 \<or> (\<exists> j<n. (j=i \<or> i=Suc j)))"
      by (metis le_eq_less_or_eq lessI less_Suc_eq_0_disj)
    ultimately have "\<forall> i\<le>n. f i \<notin> A" by blast
    then have "F \<inter> A = {}" using c4 by blast
    moreover have "F \<in> \<F> r x y" using c3 c4 unfolding \<F>_def by blast
    ultimately show "x \<in> UNIV - w_dncl r A" using c1 unfolding w_dncl_def by blast
  qed
qed

lemma lem_cowdn_uset:
fixes r::"'U rel" and A A' W::"'U set"
assumes a1: "CCR (Restr r A')" and a2: "escl r A A' \<subseteq> A'"
    and a3: "Q = A' - dncl r A" and a4: "W = A' - w_dncl r A" and a5: "Q \<in> SF r"
shows "Restr r Q \<in> \<UU> (Restr r W)"
proof -
  have "CCR (Restr r Q)" using a1 a3 lem_Inv_ccr_restr_invdiff lem_Inv_dncl_invbk by blast
  moreover have "Restr r Q \<subseteq> Restr r W" using a3 a4 lem_wdn_range_ub[of r] by blast
  moreover have "\<forall>a\<in>Field (Restr r W). \<exists>b\<in>Field (Restr r Q). (a, b) \<in> (Restr r W)^*"
  proof
    fix a
    assume "a \<in> Field (Restr r W)"
    then have c1: "a \<in> W" unfolding Field_def by blast
    show "\<exists>b\<in>Field (Restr r Q). (a, b) \<in> (Restr r W)^*"
    proof (cases "a \<in> Q")
      assume "a \<in> Q"
      then show ?thesis using a5 unfolding SF_def by blast
    next
      assume "a \<notin> Q"
      then obtain b F where d1: "a \<in> A' \<and> F \<in> \<F> r a b \<and> b \<notin> dncl r A \<and> F \<
       inter> A = {}"
        using c1 a3 a4 unfolding w_dncl_def by blast
      then have d2: "dnesc r A a \<subseteq> escl r A A'" unfolding escl_def by blast
      obtain E where d3: "E = dnesc r A a" by blast
      have "dnEsc r A a \<noteq> {}" using d1 unfolding dnEsc_def by blast
      then have "E \<in> dnEsc r A a" using d3 lem_dnEsc_ne[of r A] by blast
      then obtain b' where d4: "b' \<notin> dncl r A \<and> E \<in> \<F> r a b' \<and> E \<inter> A = {}"
        unfolding dnEsc_def by blast
      have d5: "E \<subseteq> A'" using d2 d3 a2 by blast
```

```
      have "b' \<in> E" using d4 unfolding \<F>_def rpth_def by blast
      then have "b' \<in> Field (Restr r Q)" using d4 d5 a3 a5 unfolding SF_def by blast
      moreover have "(a, b') \<in> (Restr r W)^*"
      proof -
        obtain f n where e1: "f \<in> rpth r a b' n" and e2: "E = f ` {i. i \<le> n}"
          using d4 unfolding \<F>_def by blast
        have e3: "\<forall> i\<le>n. f i \<in> W"
        proof (intro allI impI)
          fix i
          assume f1: "i \<le> n"
          obtain g where f2: "g = (\<lambda> k. f (k + i))" by blast
          have "g 0 = f i" using f2 by simp
          moreover have "g (n - i) = b'" using f1 f2 e1 unfolding rpth_def by simp
          moreover have "\<forall>k<n-i. (g k, g (Suc k)) \<in> Restr r (UNIV - A)"
          proof (intro allI impI)
            fix k
            assume "k < n-i"
            then have "(g k, g (Suc k)) \<in> (Restr r E)" using f2 e1 e2 unfolding rpth_def by simp
            then show "(g k, g (Suc k)) \<in> Restr r (UNIV - A)" using d4 by blast
          qed
          ultimately have "g \<in> rpth (Restr r (UNIV-A)) (f i) b' (n-i)" unfolding rpth_def by blast
          then have "(f i, b') \<in> (Restr r (UNIV-A))^*" using lem_ccext_rpth_rtr[of _ "f i" b'] by blast
          then have "f i \<notin> w_dncl r A" using d4 lem_wdn_compl[of r A] by blast
          then show "f i \<in> W" using f1 e2 d5 a4 by blast
        qed
        have "\<forall>i<n. (f i, f (Suc i)) \<in> Restr r W"
        proof (intro allI impI)
          fix i
          assume "i < n"
          moreover then have "f i \<in> W \<and> f (Suc i) \<in> W" using e2 e3 by force
          ultimately show "(f i, f (Suc i)) \<in> Restr r W" using e1 unfolding rpth_def by blast
        qed
        then have "E \<in> \<F> (Restr r W) a b'" using e1 e2 unfolding rpth_def \<F>_def by blast
        then show ?thesis using lem_ccext_rtr_Fne[of a b'] by blast
      qed
      ultimately show ?thesis by blast
    qed
  qed
  ultimately show ?thesis unfolding \<UU>_def by blast
qed

lemma lem_shrel_L_eq:
fixes f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel" and \<beta>::"'U rel"
assumes "\<alpha> =o \<beta>"
shows "\<LL> f \<alpha> = \<LL> f \<beta>"
proof
  show "\<LL> f \<alpha> \<subseteq> \<LL> f \<beta>" using assms ordLess_ordIso_trans unfolding \<LL>_def by
       fastforce
next
  have "\<beta> =o \<alpha>" using assms ordIso_symmetric by blast
  then show "\<LL> f \<beta> \<subseteq> \<LL> f \<alpha>" using ordLess_ordIso_trans unfolding \<LL>_def by
       fastforce
qed

lemma lem_shrel_dbk_eq:
fixes f::"'U rel \<Rightarrow> 'U set" and Ps::"'U set set" and \<alpha>::"'U rel" and \<beta>::"'U rel"
assumes "f \<in> \<N> r Ps" and "\<alpha> =o \<beta>" and "\<alpha> \<le>o |Field r|" and "\<beta> \<le>o |Field
        r|"
shows "(\<nabla> f \<alpha>) = (\<nabla> f \<beta>)"
proof -
  have "\<alpha> \<le>o \<beta> \<and> \<beta> \<le>o \<alpha>" using assms ordIso_iff_ordLeq by blast
  then have "f \<alpha> = f \<beta>" using assms unfolding \<N>_def \<N>1_def by blast
  moreover have "\<LL> f \<alpha> = \<LL> f \<beta>" using assms lem_shrel_L_eq by blast
  ultimately show ?thesis unfolding Dbk_def by blast
qed

lemma lem_L_emp:  "\<alpha> =o ({}::'U rel) \<Longrightarrow> \<LL> f \<alpha> = {}"
proof -
  assume "\<alpha> =o ({}::'U rel)"
  then have "\<forall> \<alpha>'. \<alpha>' <o \<alpha> \<longrightarrow> False" using lem_ord_subemp
    by (metis iso_ozero_empty not_ordLess_ordIso ordLess_imp_ordLeq ozero_def)
  then show "\<LL> f \<alpha> = {}" unfolding \<LL>_def by blast
qed

lemma lem_der_qinv1:
fixes r::"'U rel" and \<alpha>::"'U rel" and x y::'U
assumes a1: "x \<in> \<Q> r f \<alpha>" and a2: "(x,y) \<in> r^*" and a3: "y \<in> (f \<alpha>)"
shows "y \<in> \<Q> r f \<alpha>"
```

```
proof -
  obtain A where b1: "A = (\<LL> f \<alpha>)" by blast
  have "\<forall> x y. y \<in> dncl r A \<longrightarrow> (x,y) \<in> r \<longrightarrow> x \<in> dncl r A"
  proof (intro allI impI)
    fix x y
    assume "y \<in> dncl r A" and "(x,y) \<in> r"
    moreover then obtain a where "a \<in> A \<and> (y,a) \<in> r^*" unfolding dncl_def by blast
    ultimately have "a \<in> A \<and> (x,a) \<in> r^*" by force
    then show "x \<in> dncl r A" unfolding dncl_def by blast
  qed
  then have "(UNIV - dncl r A) \<in> Inv r" unfolding Inv_def by blast
  moreover have "x \<in> UNIV - (dncl r A)" using b1 a1 unfolding \<Q>_def by blast
  ultimately have "y \<in> UNIV - (dncl r A)" using a2 lem_Inv_restr_rtr2[of "UNIV - dncl r A" r] by blast
  then show ?thesis using b1 a3 unfolding \<Q>_def by blast
qed

lemma lem_der_qinv2:
fixes r::"'U rel" and \<alpha>::"'U rel" and x y::'U
assumes a1: "x \<in> \<Q> r f \<alpha>" and a2: "(x,y) \<in> (Restr r (f \<alpha>))^*" and a3: "y \<in> (f \<
      alpha>)"
shows "(x,y) \<in> (Restr r (\<Q> r f \<alpha>))^*"
proof -
  obtain Q where b1: "Q = \<Q> r f \<alpha>" by blast
  have "\<forall> a b. a \<in> Q \<longrightarrow> (a,b) \<in> Restr r (f \<alpha>) \<longrightarrow> b \<in> Q"
    using lem_der_qinv1[of _ r f \<alpha> _] unfolding b1 by blast
  then have "Q \<in> Inv (Restr r (f \<alpha>))" unfolding Inv_def by blast
  moreover have "x \<in> Q" using b1 a1 by blast
  ultimately have "(x,y) \<in> (Restr (Restr r (f \<alpha>)) Q)^*"
    using a2 lem_Inv_restr_rtr[of Q "Restr r (f \<alpha>)"] by blast
  moreover have "Restr (Restr r (f \<alpha>)) Q \<subseteq> Restr r (\<Q> r f \<alpha>)" using b1 by blast
  ultimately show ?thesis using rtrancl_mono by blast
qed

lemma lem_der_qinv3:
fixes r::"'U rel" and \<alpha>::"'U rel"
assumes a1: "A \<subseteq> (f \<alpha>)" and a2: "\<forall> x \<in> (f \<alpha>). \<exists> y \<in> A. (x,y) \<
      in> (Restr r (f \<alpha>))^*"
shows "\<forall> x \<in> (\<Q> r f \<alpha>). \<exists> y \<in> (A \<inter> (\<Q> r f \<alpha>)). (x,y) \<in> (
      Restr r (\<Q> r f \<alpha>))^*"
proof
  fix x
  assume b1: "x \<in> (\<Q> r f \<alpha>)"
  then have b2: "x \<in> (f \<alpha>)" unfolding \<Q>_def by blast
  then obtain y where b3: "y \<in> A \<and> (x,y) \<in> (Restr r (f \<alpha>))^*" using a2 by blast
  then have "(x, y) \<in> (Restr r (\<Q> r f \<alpha>))^*" using a1 b1 lem_der_qinv2[of x r f \<alpha> y] by
      blast
  moreover then have "y \<in> (\<Q> r f \<alpha>)" using b1 IntE mem_Sigma_iff rtranclE[of x y] by metis
  ultimately show "\<exists> y \<in> (A \<inter> (\<Q> r f \<alpha>)). (x,y) \<in> (Restr r (\<Q> r f \<alpha>))
      ^*" using b3 by blast
qed

lemma lem_der_inf_qrestr_ccr1:
fixes r::"'U rel" and Ps::"'U set set" and \<alpha>::"'U rel"
assumes "f \<in> \<N> r Ps" and "\<alpha> \<le>o |Field r|"
shows "CCR (Restr r (\<Q> r f \<alpha>))"
proof -
  have "CCR (Restr r (f \<alpha>))" using assms unfolding \<N>_def \<N>6_def by blast
  moreover have "dncl r (\<LL> f \<alpha>) \<in> Inv (r^-1)" using lem_Inv_dncl_invbk by blast
  ultimately show ?thesis unfolding \<Q>_def using lem_Inv_ccr_restr_invdiff by blast
qed

lemma lem_Nfdn_aemp:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "CCR r" and a2: "f \<in> \<N> r Ps" and a3: "\<alpha> <o scf r" and a4: "Field r \<subseteq> dncl r
      (f \<alpha>)"
shows "\<alpha> = {}"
proof (cases "finite r")
  assume "finite r"
  then have "scf r <o \<omega>_ord" using lem_scf_relfldcard_bnd lem_fin_fl_rel
    by (metis finite_iff_ordLess_natLeq ordLeq_ordLess_trans)
  then have "finite (Field (scf r))" using finite_iff_ordLess_natLeq by force
  then have "Conelike r" using a1 lem_scf_ccr_finscf_cl by blast
  moreover obtain a::'U where "True" by blast
  ultimately have "\<alpha> <o |{a}|" using a1 a3 lem_Rcc_eq2_12 lem_scf_ccr_scf_rcc_eq
    by (metis ordIso_iff_ordLeq ordLess_ordLeq_trans)
  then have b1: "\<alpha> =o |{}::'U set|" using lem_co_one_ne_min
    by (metis card_of_card_order_on card_of_empty3 card_of_unique insert_not_empty
        not_ordLeq_ordLess ordIso_Well_order_simp ordLess_Well_order_simp)
```

```
  then have "\<alpha> \<le>o |Field r|" using card_of_empty ordIso_ordLeq_trans by blast
  then have b2: "f \<alpha> \<in> SF r" using a2 unfolding \<N>_def \<N>5_def by blast
  have "\<not> (\<exists> \<alpha>'::'U rel. \<alpha>' <o \<alpha>)" using b1
    by (metis BNF_Cardinal_Order_Relation.ordLess_Field card_of_empty5 ordLess_ordIso_trans)
  then show "\<alpha> = {}" using a3 b1 using lem_co_one_ne_min
    by (metis card_of_empty card_of_empty3 insert_not_empty
        ordIso_ordLeq_trans ordLeq_transitive ordLess_Well_order_simp)
next
  assume q0: "\<not> finite r"
  have b0: "\<alpha> <o \<parallel>r\<parallel>" using a1 a3 lem_scf_ccr_scf_rcc_eq by (metis ordIso_iff_ordLeq
      ordLess_ordLeq_trans)
  obtain A' where b1: "A' = \<Q> r f \<alpha>" by blast
  have "\<parallel>r\<parallel> \<le>o |r|" using lem_Rcc_relcard_bnd by blast
  moreover have "|Field r| =o |r|" using q0 lem_rel_inf_fld_card by blast
  ultimately have "\<parallel>r\<parallel> \<le>o |Field r|" using ordIso_symmetric ordLeq_ordIso_trans by blast
  then have b2: "\<alpha> \<le>o |Field r|" using b0 ordLeq_transitive ordLess_imp_ordLeq by blast
  then have b3: "f \<alpha> \<in> SF r \<and> CCR (Restr r (f \<alpha>))"
      using b1 a2 unfolding \<N>_def \<N>5_def \<N>10_def \<N>6_def by blast+
  have b5: "(A' \<in> SF r ) \<or> (\<exists>y::'U. A' = {y})"
    using b1 b3 unfolding \<Q>_def using lem_Inv_ccr_sf_dn_diff[of "f \<alpha>" r A' "\<LL> f \<alpha>"] by
      blast
  have "\<forall>a\<in>Field r. \<exists>b\<in>Field (Restr r (f \<alpha>)). (a, b) \<in> r^*"
  proof
    fix a
    assume "a \<in> Field r"
    then have "a \<in> dncl r (f \<alpha>)" using a4 by blast
    then obtain b::'U where "(a, b) \<in> r^* \<and> b \<in> f \<alpha>" unfolding dncl_def by blast
    moreover have "(f \<alpha>) \<in> SF r" using b3 by blast
    ultimately have "b \<in> Field (Restr r (f \<alpha>)) \<and> (a, b) \<in> r^*" unfolding SF_def by blast
    then show "\<exists>b\<in>Field (Restr r (f \<alpha>)). (a, b) \<in> r^*" by blast
  qed
  moreover have "CCR (Restr r (f \<alpha>))" using b3 by blast
  ultimately have "Restr r (f \<alpha>) \<in> \<UU> r" unfolding \<UU>_def by blast
  then have d3: "\<parallel>r\<parallel> \<le>o |Restr r (f \<alpha>)|" using lem_rcc_uset_mem_bnd by blast
  obtain x::'U where d4: "True" by blast
  have "\<omega>_ord \<le>o \<alpha> \<longrightarrow> False"
  proof
    assume e1: "\<omega>_ord \<le>o \<alpha>"
    then have "|f \<alpha>| \<le>o \<alpha>" using b2 a2 unfolding \<N>_def \<N>7_def by blast
    moreover then have "|Restr r (f \<alpha>)| \<le>o \<alpha>" using e1 lem_restr_ordbnd by blast
    ultimately have "\<parallel>r\<parallel> \<le>o \<alpha>" using d3 ordLeq_transitive by blast
    then show "False" using b0 not_ordLess_iff_ordLeq ordLess_Well_order_simp by blast
  qed
  then have "\<alpha> <o \<omega>_ord" using b0 natLeq_Well_order not_ordLess_iff_ordLeq ordLess_Well_order_simp
      by blast
  then have "|f \<alpha>| <o \<omega>_ord" using b2 a2 unfolding \<N>_def \<N>7_def by blast
  then have "finite (f \<alpha>)" using finite_iff_ordLess_natLeq by blast
  then have "finite (Restr r (f \<alpha>))" by blast
  then have "|Restr r (f \<alpha>)| <o \<omega>_ord" using finite_iff_ordLess_natLeq by blast
  then have d5: "\<parallel>r\<parallel> <o \<omega>_ord" using d3 ordLeq_ordLess_trans by blast
  have "\<parallel>r\<parallel> \<le>o |{x}|"
  proof (cases "CCR r")
    assume "CCR r"
    then show "\<parallel>r\<parallel> \<le>o |{x}|" using d5 lem_Rcc_eq2_31[of r] lem_Rcc_eq2_12[of r x] by
      blast
  next
    assume "\<not> CCR r"
    moreover then have "\<parallel>r\<parallel> = {}" using lem_rcc_nccr by blast
    moreover have "{} \<le>o |{x}|" by (metis card_of_Well_order ozero_def ozero_ordLeq)
    ultimately show "\<parallel>r\<parallel> \<le>o |{x}|" by metis
  qed
  then have "\<alpha> <o |{x}|" using b0 ordLess_ordLeq_trans by blast
  then show "\<alpha> = {}" by (meson lem_co_one_ne_min not_ordLeq_ordLess ordLess_Well_order_simp)
qed

lemma lem_der_qccr_lscf_sf:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "CCR r" and a2: "f \<in> \<N> r Ps" and a3: "\<alpha> <o scf r"
shows "(\<Q> r f \<alpha>) \<in> SF r"
proof (cases "finite r")
  assume "finite r"
  then have "scf r <o \<omega>_ord" using lem_scf_relfldcard_bnd lem_fin_fl_rel
    by (metis finite_iff_ordLess_natLeq ordLeq_ordLess_trans)
  then have "finite (Field (scf r))" using finite_iff_ordLess_natLeq by force
  then have "Conelike r" using a1 lem_scf_ccr_finscf_cl by blast
  moreover obtain a::'U where "True" by blast
  ultimately have "\<alpha> <o |{a}|" using a1 a3 lem_Rcc_eq2_12 lem_scf_ccr_scf_rcc_eq
    by (metis ordIso_iff_ordLeq ordLess_ordLeq_trans)
```

```
  then have b1: "\<alpha> =o |{}::'U set|" using lem_co_one_ne_min
    by (metis card_of_card_order_on card_of_empty3 card_of_unique insert_not_empty
        not_ordLeq_ordLess ordIso_Well_order_simp ordLess_Well_order_simp)
  then have "\<alpha> \<le>o |Field r|" using card_of_empty ordIso_ordLeq_trans by blast
  then have b2: "f \<alpha> \<in> SF r" using a2 unfolding \<N>_def \<N>5_def by blast
  have "\<not> (\<exists> \<alpha>'::'U rel. \<alpha>' <o \<alpha>)" using b1
    by (metis BNF_Cardinal_Order_Relation.ordLess_Field card_of_empty5 ordLess_ordIso_trans)
  then have "\<LL> f \<alpha> = {}" unfolding \<LL>_def by blast
  then have "\<Q> r f \<alpha> = f \<alpha>" unfolding \<Q>_def dncl_def by blast
  then show ?thesis using b2 by metis
next
  assume q0: "\<not> finite r"
  have b0: "\<alpha> <o \<parallel>r\<parallel>" using a1 a3 lem_scf_ccr_scf_rcc_eq by (metis ordIso_iff_ordLeq
      ordLess_ordLeq_trans)
  obtain A' where b1: "A' = \<Q> r f \<alpha>" by blast
  have "\<parallel>r\<parallel> \<le>o |r|" using lem_Rcc_relcard_bnd by blast
  moreover have "|Field r| =o |r|" using q0 lem_rel_inf_fld_card by blast
  ultimately have "\<parallel>r\<parallel> \<le>o |Field r|" using ordIso_symmetric ordLeq_ordIso_trans by blast
  then have b2: "\<alpha> \<le>o |Field r|" using b0 ordLeq_transitive ordLess_imp_ordLeq by blast
  then have b3: "f \<alpha> \<in> SF r \<and> CCR (Restr r (f \<alpha>))"
       and b4: "(\<exists>y::'U. A' = {y}) \<longrightarrow> Field r \<subseteq> dncl r (f \<alpha>)"
       using b1 a2 unfolding \<N>_def \<N>5_def \<N>10_def \<N>6_def by blast+
  have b5: "(A' \<in> SF r ) \<or> (\<exists>y::'U. A' = {y})"
    using b1 b3 unfolding \<Q>_def using lem_Inv_ccr_sf_dn_diff[of "f \<alpha>" r A' "\<LL> f \<alpha>"] by
      blast
  show "(\<Q> r f \<alpha>) \<in> SF r"
  proof (cases "Field r \<subseteq> dncl r (f \<alpha>)")
    assume c1: "Field r \<subseteq> dncl r (f \<alpha>)"
    have "\<forall>a\<in>Field r. \<exists>b\<in>Field (Restr r (f \<alpha>)). (a, b) \<in> r^*"
    proof
      fix a
      assume "a \<in> Field r"
      then have "a \<in> dncl r (f \<alpha>)" using c1 by blast
      then obtain b::'U where "(a, b) \<in> r^* \<and> b \<in> f \<alpha>" unfolding dncl_def by blast
      moreover have "(f \<alpha>) \<in> SF r" using b3 by blast
      ultimately have "b \<in> Field (Restr r (f \<alpha>)) \<and> (a, b) \<in> r^*" unfolding SF_def by blast
      then show "\<exists>b\<in>Field (Restr r (f \<alpha>)). (a, b) \<in> r^*" by blast
    qed
    moreover have "CCR (Restr r (f \<alpha>))" using b3 by blast
    ultimately have "Restr r (f \<alpha>) \<in> \<UU> r" unfolding \<UU>_def by blast
    then have d3: "\<parallel>r\<parallel> \<le>o |Restr r (f \<alpha>)|" using lem_rcc_uset_mem_bnd by blast
    obtain x::'U where d4: "True" by blast
    have "\<omega>_ord \<le>o \<alpha> \<longrightarrow> False"
    proof
      assume e1: "\<omega>_ord \<le>o \<alpha>"
      then have "|f \<alpha>| \<le>o \<alpha>" using b2 a2 unfolding \<N>_def \<N>7_def by blast
      moreover then have "|Restr r (f \<alpha>)| \<le>o \<alpha>" using e1 lem_restr_ordbnd by blast
      ultimately have "\<parallel>r\<parallel> \<le>o \<alpha>" using d3 ordLeq_transitive by blast
      then show "False" using b0 not_ordLess_iff_ordLeq ordLess_Well_order_simp by blast
    qed
    then have "\<alpha> <o \<omega>_ord" using b0 natLeq_Well_order not_ordLess_iff_ordLeq
      ordLess_Well_order_simp by blast
    then have "|f \<alpha>| <o \<omega>_ord" using b2 a2 unfolding \<N>_def \<N>7_def by blast
    then have "finite (f \<alpha>)" using finite_iff_ordLess_natLeq by blast
    then have "finite (Restr r (f \<alpha>))" by blast
    then have "|Restr r (f \<alpha>)| <o \<omega>_ord" using finite_iff_ordLess_natLeq by blast
    then have d5: "\<parallel>r\<parallel> <o \<omega>_ord" using d3 ordLeq_ordLess_trans by blast
    have "\<parallel>r\<parallel> \<le>o |{x}|"
    proof (cases "CCR r")
      assume "CCR r"
      then show "\<parallel>r\<parallel> \<le>o |{x}|" using d5 lem_Rcc_eq2_31[of r] lem_Rcc_eq2_12[of r x] by
        blast
    next
      assume "\<not> CCR r"
      moreover then have "\<parallel>r\<parallel> = {}" using lem_rcc_nccr by blast
      moreover have "{} \<le>o |{x}|" by (metis card_of_Well_order ozero_def ozero_ordLeq)
      ultimately show "\<parallel>r\<parallel> \<le>o |{x}|" by metis
    qed
    then have "\<alpha> <o |{x}|" using b0 ordLess_ordLeq_trans by blast
    then have "\<alpha> = {}" by (meson lem_co_one_ne_min not_ordLeq_ordLess ordLess_Well_order_simp)
    then have "\<forall> \<alpha>'. \<alpha>' <o \<alpha> \<longrightarrow> False" using lem_ord_subemp by (
      metis iso_ozero_empty not_ordLess_ordIso ordLess_imp_ordLeq ozero_def)
    then have "dncl r (\<LL> f \<alpha>) = {}" unfolding dncl_def \<LL>_def by blast
    then have "\<Q> r f \<alpha> = f \<alpha>" unfolding \<Q>_def by blast
    then show "(\<Q> r f \<alpha>) \<in> SF r" using b3 by metis
  next
    assume "\<not> (Field r \<subseteq> dncl r (f \<alpha>))"
    then have "A' \<in> SF r" using b4 b5 by blast
```

```
    then show "(\<Q> r f \<alpha>) \<in> SF r" using b1 by blast
  qed
qed

lemma lem_der_q_uset:
fixes r::"'U rel" and Ps::"'U set set" and \<alpha>::"'U rel"
assumes a1: "CCR r" and a2: "f \<in> \<N> r Ps" and a3: "\<alpha> <o scf r" and a4: "isSuccOrd \<alpha>"
shows "Restr r (\<Q> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))"
proof -
  have b1: "\<alpha> \<le>o |Field r|" using a3 lem_scf_relfldcard_bnd
    by (metis ordLess_ordLeq_trans ordLess_imp_ordLeq)
  have a4: "\<Q> r f \<alpha> = {} \<longrightarrow> False"
  proof
    assume "\<Q> r f \<alpha> = {}"
    then have "Field r \<subseteq> dncl r (f \<alpha>)" using b1 a2 a4 unfolding \<N>_def \<N>11_def by blast
    then have "\<alpha> = {}" using a1 a2 a3 lem_Nfdn_aemp by blast
    then show "False" using a4 using wo_rel_def wo_rel.isSuccOrd_def unfolding Field_def by force
  qed
  have "(\<Q> r f \<alpha>) \<in> SF r" using a1 a2 a3 lem_der_qccr_lscf_sf by blast
  then have b2: "Field (Restr r (\<Q> r f \<alpha>)) \<noteq> {}" using a4 unfolding SF_def by blast
  have "Restr r (\<Q> r f \<alpha>) \<subseteq> Restr r (f \<alpha>)" unfolding \<Q>_def by blast
  moreover have "CCR (Restr r (\<Q> r f \<alpha>))" using b1 a2 lem_der_inf_qrestr_ccr1 by blast
  moreover have "\<forall>a\<in>Field (Restr r (f \<alpha>)). \<exists>b\<in>Field (Restr r (\<Q> r f \<alpha>))
      . (a,b) \<in> (Restr r (f \<alpha>))^*"
  proof
    fix a
    assume c1: "a \<in> Field (Restr r (f \<alpha>))"
    obtain b where c2: "b \<in> Field (Restr r (\<Q> r f \<alpha>))" using b2 by blast
    then have c3: "b \<in> f \<alpha> \<and> b \<in> \<Q> r f \<alpha>" unfolding \<Q>_def Field_def by blast
    have "f \<alpha> \<in> SF r" using b1 a2 unfolding \<N>_def \<N>5_def by blast
    then have "b \<in> Field (Restr r (f \<alpha>))" using c3 unfolding SF_def by blast
    moreover have "CCR (Restr r (f \<alpha>))" using b1 a2 unfolding \<N>_def \<N>6_def by blast
    ultimately obtain c where "c \<in> Field (Restr r (f \<alpha>))"
      and c4: "(a,c) \<in> (Restr r (f \<alpha>))^* \<and> (b,c) \<in> (Restr r (f \<alpha>))^*"
      using c1 unfolding CCR_def by blast
    moreover then have "c \<in> f \<alpha>" unfolding Field_def by blast
    ultimately have "(b, c) \<in> (Restr r (\<Q> r f \<alpha>))^*" using c3 lem_der_qinv2[of b r f \<alpha> c]
      by blast
    moreover have "Field (Restr r (\<Q> r f \<alpha>)) \<in> Inv (Restr r (\<Q> r f \<alpha>))"
      unfolding Inv_def Field_def by blast
    ultimately have "c \<in> Field (Restr r (\<Q> r f \<alpha>))"
      using c2 lem_Inv_restr_rtr2[of "Field (Restr r (\<Q> r f \<alpha>))"] by blast
    then show "\<exists>b\<in>Field (Restr r (\<Q> r f \<alpha>)). (a, b) \<in> (Restr r (f \<alpha>))^*" using
      c4 by blast
  qed
  ultimately show "Restr r (\<Q> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))" unfolding \<UU>_def by blast
qed

lemma lem_qw_range: "f \<in> \<N> r Ps \<Longrightarrow> \<alpha> \<le>o |Field r| \<Longrightarrow> \<W> r f \<
      alpha> \<subseteq> Field r"
  unfolding \<N>_def \<N>5_def SF_def Field_def \<W>_def by blast

lemma lem_der_qw_eq:
fixes r::"'U rel" and Ps::"'U set set" and \<alpha> \<beta>::"'U rel"
assumes "f \<in> \<N> r Ps" and "\<alpha> =o \<beta>"
shows "\<W> r f \<alpha> = \<W> r f \<beta>"
proof -
  have "f \<alpha> = f \<beta>" using assms unfolding \<N>_def by blast
  moreover have "\<LL> f \<alpha> = \<LL> f \<beta>" using assms lem_shrel_L_eq by blast
  ultimately show ?thesis unfolding \<W>_def by simp
qed

lemma lem_Der_inf_qw_disj:
fixes r::"'U rel" and \<alpha> \<beta>::"'U rel"
assumes "Well_order \<alpha>" and "Well_order \<beta>"
shows "(\<not> (\<alpha> =o \<beta>)) \<longrightarrow> (\<W> r f \<alpha>) \<inter> (\<W> r f \<beta>) = {}"
proof
  assume b1: "\<not> (\<alpha> =o \<beta>)"
  obtain W where b2: "W = (\<lambda> \<alpha>. \<W> r f \<alpha>)" by blast
  have "\<alpha> <o \<beta> \<or> \<beta> <o \<alpha>" using b1 assms by (meson not_ordLeq_iff_ordLess
      ordLeq_iff_ordLess_or_ordIso)
  moreover have "\<forall> \<alpha>' \<beta>'. \<alpha>' <o \<beta>' \<longrightarrow> (W \<alpha>' \<inter> W
      \<beta>' \<noteq> {}) \<longrightarrow> False"
  proof (intro allI impI)
    fix \<alpha>' \<beta>'::"'U rel"
    assume d1: "\<alpha>' <o \<beta>'" and "W \<alpha>' \<inter> W \<beta>' \<noteq> {}"
    then obtain a where d2: "a \<in> W \<alpha>' \<inter> W \<beta>'" by blast
    then have "a \<in> f \<alpha>'" using b2 unfolding \<W>_def by blast
```

```
    then have "a \<in> \<LL> f \<beta>'" using d1 unfolding \<LL>_def by blast
    then have "a \<notin> W \<beta>'" using b2 lem_wdn_range_lb[of _ r] unfolding \<W>_def by blast
    then show "False" using d2 by blast
  qed
  ultimately show "(\<W> r f \<alpha>) \<inter> (\<W> r f \<beta>) = {}" unfolding b2 by blast
qed

lemma lem_der_inf_qw_restr_card:
fixes r::"'U rel" and Ps::"'U set set" and \<alpha>::"'U rel"
assumes a1: "\<not> finite r" and a2: "f \<in> \<N> r Ps" and a3: "\<alpha> <o |Field r|"
shows "|Restr r (\<W> r f \<alpha>)| <o |Field r|"
proof -
  have b0: "|Field r| =o |r|" using a1 lem_rel_inf_fld_card by blast
  obtain W where b2: "W = (\<lambda> \<alpha>. \<W> r f \<alpha>)" by blast
  have "\<alpha> \<le>o |Field r|" using a3 b0 ordLess_imp_ordLeq ordIso_iff_ordLeq ordLeq_transitive by blast
  then have "(\<alpha> <o \<omega>_ord \<longrightarrow> |f \<alpha>| <o \<omega>_ord) \<and> (\<omega>_ord \<le
      >o \<alpha> \<longrightarrow> |f \<alpha>| \<le>o \<alpha>)"
    using a2 unfolding \<N>_def \<N>7_def by blast
  moreover have c2: "\<alpha> <o \<omega>_ord \<or> \<omega>_ord \<le>o \<alpha>" using a3 Field_natLeq
      natLeq_well_order_on by force
  moreover have c3: "|f \<alpha>| <o \<omega>_ord \<longrightarrow> |Restr r (W \<alpha>)| <o |Field r|"
  proof
    assume "|f \<alpha>| <o \<omega>_ord"
    then have "finite (f \<alpha>)" using finite_iff_ordLess_natLeq by blast
    then have "finite (Restr r (W \<alpha>))" unfolding b2 \<W>_def by blast
    then have "|Restr r (W \<alpha>)| <o \<omega>_ord" using finite_iff_ordLess_natLeq by blast
    moreover have "\<omega>_ord \<le>o |r|" using a1 infinite_iff_natLeq_ordLeq by blast
    moreover then have "\<omega>_ord \<le>o |Field r|" using lem_rel_inf_fld_card
      by (metis card_of_ordIso_finite infinite_iff_natLeq_ordLeq)
    ultimately show "|Restr r (W \<alpha>)| <o |Field r|" using ordLess_ordLeq_trans by blast
  qed
  moreover have "\<omega>_ord \<le>o \<alpha> \<and> |f \<alpha>| \<le>o \<alpha> \<longrightarrow> |Restr r (W
      \<alpha>)| <o |Field r|"
  proof
    assume d1: "\<omega>_ord \<le>o \<alpha> \<and> |f \<alpha>| \<le>o \<alpha>"
    moreover have "|W \<alpha>| \<le>o |f \<alpha>|" unfolding b2 \<W>_def by simp
    ultimately have "|W \<alpha>| \<le>o \<alpha>" using ordLeq_transitive by blast
    then have "|Restr r (W \<alpha>)| \<le>o \<alpha>" using d1 lem_restr_ordbnd[of \<alpha> "W \<alpha>" r] by
      blast
    then show "|Restr r (W \<alpha>)| <o |Field r|" using a3 ordLeq_ordLess_trans by blast
  qed
  ultimately show ?thesis using b2 by blast
qed

lemma lem_QS_subs_WS: "\<Q> r f \<alpha> \<subseteq> \<W> r f \<alpha>"
  unfolding \<Q>_def \<W>_def using lem_wdn_range_ub by force

lemma lem_WS_limord:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "\<not> finite r" and a2: "f \<in> \<N> r Ps" and a3: "\<alpha> <o |Field r|"
    and a4: "\<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)"
shows "\<W> r f \<alpha> = {}"
proof -
  have "\<alpha> \<le>o |Field r|" using a3 ordLess_imp_ordLeq by blast
  then have "f \<alpha> \<subseteq> \<LL> f \<alpha>" using a2 a4 unfolding \<N>_def \<N>2_def Dbk_def by blast
  then have "w_dncl r (f \<alpha>) \<subseteq> w_dncl r (\<LL> f \<alpha>)" using lem_wdn_mon by blast
  moreover have "f \<alpha> \<subseteq> w_dncl r (f \<alpha>)" using lem_wdn_range_lb[of "f \<alpha>" r] by
      metis
  ultimately have "f \<alpha> \<subseteq> w_dncl r (\<LL> f \<alpha>)" by blast
  then show ?thesis unfolding \<W>_def by blast
qed

lemma lem_der_inf_qw_restr_uset:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "Refl r \<and> \<not> finite r" and a2: "f \<in> \<N> r Ps"
    and a3: "\<alpha> <o |Field r|" and a4: "\<omega>_ord \<le>o |\<LL> f \<alpha>|"
shows "Restr r (\<Q> r f \<alpha>) \<in> \<UU> (Restr r (\<W> r f \<alpha>))"
proof (cases "\<alpha> = {} \<or> isSuccOrd \<alpha>")
  assume "\<alpha> = {} \<or> isSuccOrd \<alpha>"
  moreover have "|Field r| =o |r|" using a1 lem_rel_inf_fld_card by blast
  then have b1: "\<alpha> \<le>o |Field r|" using a3 ordLess_imp_ordLeq ordIso_iff_ordLeq ordLeq_transitive by
      blast
  ultimately have b2: "escl r (\<LL> f \<alpha>) (f \<alpha>) \<subseteq> f \<alpha>" using a2 a4 unfolding \<N>
      _def \<N>3_def by blast
  moreover have b3: "CCR (Restr r (f \<alpha>))" using b1 a2 unfolding \<N>_def \<N>6_def by blast
  moreover have "SF r = {A. A \<subseteq> Field r}" using a1 unfolding SF_def refl_on_def Field_def by fast
  moreover then have "\<W> r f \<alpha> \<in> SF r" and "\<Q> r f \<alpha> \<in> SF r"
    using a2 a3 lem_qw_range[of f r Ps \<alpha>] lem_QS_subs_WS[of r f \<alpha>] ordLess_imp_ordLeq by fast+
```

```
  ultimately show ?thesis
    using a1 lem_cowdn_uset[of r "f \<alpha>" "\<LL> f \<alpha>"] \<Q>_def[of r f \<alpha>] \<W>_def[of r f \<
      alpha>] by blast
next
  assume "\<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)"
  then have "\<W> r f \<alpha> = {} \<and> \<Q> r f \<alpha> = {}"
    using assms lem_WS_limord lem_QS_subs_WS[of r f \<alpha>] by blast
  then show ?thesis unfolding \<UU>_def CCR_def Field_def by blast
qed

lemma lem_der_inf_qw_restr_ccr:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "Refl r \<and> \<not> finite r" and a2: "f \<in> \<N> r Ps"
    and a3: "\<alpha> <o |Field r|" and a4: "\<omega>_ord \<le>o |\<LL> f \<alpha>|"
shows "CCR (Restr r (\<W> r f \<alpha>))"
  using assms lem_der_inf_qw_restr_uset lem_rcc_uset_ne_ccr by blast

lemma lem_der_qw_uset:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "CCR r \<and> Refl r \<and> \<not> finite r" and a2: "f \<in> \<N> r Ps"
    and a3: "\<alpha> <o scf r" and a4: "\<omega>_ord \<le>o |\<LL> f \<alpha>|" and a5: "isSuccOrd \<alpha>"
shows "Restr r (\<W> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))"
proof -
  have b1: "\<alpha> <o |Field r|" using a3 lem_scf_relfldcard_bnd by (metis ordLess_ordLeq_trans)
  have "\<Q> r f \<alpha> \<subseteq> \<W> r f \<alpha>" using lem_QS_subs_WS[of r f \<alpha>] by blast
  then have "Field (Restr r (\<Q> r f \<alpha>)) \<subseteq> Field (Restr r (\<W> r f \<alpha>))" unfolding
      Field_def by blast
  moreover have "Restr r (\<Q> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))"
    using a1 a2 a3 a5 lem_der_q_uset ordLess_imp_ordLeq by blast
  ultimately have "\<forall>a\<in>Field (Restr r (f \<alpha>)). \<exists>b\<in>Field (Restr r (\<W> r f \<alpha
      >)).
    (a,b) \<in> (Restr r (f \<alpha>))^*" unfolding \<UU>_def by blast
  moreover have "Restr r (\<W> r f \<alpha>) \<subseteq> Restr r (f \<alpha>)" unfolding \<W>_def by blast
  moreover have "CCR (Restr r (\<W> r f \<alpha>))" using assms b1 lem_der_inf_qw_restr_ccr by blast
  ultimately show ?thesis unfolding \<UU>_def by blast
qed

lemma lem_Shinf_N1:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>1 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "f \<in> \<N>1 r {}" using b2 unfolding \<N>1_def by (clarsimp, metis lem_ord_subemp)
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>1 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>1 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>1 r \<alpha>0"
    then have c2: "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using b3 by blast
    have "\<forall>\<alpha>' \<alpha>''. \<alpha>' \<le>o \<alpha> \<and> \<alpha>'' \<le>o \<alpha>' \<
      longrightarrow> f \<alpha>'' \<subseteq> f \<alpha>'"
    proof (intro allI impI)
      fix \<alpha>' \<alpha>''::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> \<alpha>'' \<le>o \<alpha>'"
      moreover then have "\<alpha>'' \<le>o \<alpha>" using ordLeq_transitive by blast
      ultimately have "(\<alpha>'' \<le>o \<alpha>0 \<or> \<alpha>'' =o \<alpha>) \<and> (\<alpha>' \<le>o \<
      alpha>0 \<or> \<alpha>' =o \<alpha>)" using c1 unfolding sc_ord_def
        by (meson not_ordLess_iff_ordLeq ordLeq_iff_ordLess_or_ordIso ordLess_Well_order_simp)
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> f \<alpha>'' \<subseteq> f \<alpha>'" using d1
       c1 unfolding \<N>1_def by blast
      moreover have "\<alpha>' =o \<alpha> \<and> \<alpha>'' =o \<alpha> \<longrightarrow> f \<alpha>'' \<
      subseteq> f \<alpha>'" using b5 by blast
      moreover have "\<alpha>' =o \<alpha> \<and> \<alpha>'' \<le>o \<alpha>0 \<longrightarrow> f \<alpha>'' \<
      subseteq> f \<alpha>'"
      proof
        assume e1: "\<alpha>' =o \<alpha> \<and> \<alpha>'' \<le>o \<alpha>0"
        moreover then have "\<alpha>0 \<le>o \<alpha>0" using ordLeq_Well_order_simp ordLeq_reflexive by blast
        ultimately have "f \<alpha>'' \<subseteq> f \<alpha>0" using c1 unfolding \<N>1_def by blast
        moreover have "f \<alpha>0 \<subseteq> f \<alpha>" using a1 c2 e1 ordLeq_Well_order_simp by blast
        ultimately show "f \<alpha>'' \<subseteq> f \<alpha>'" using b5 e1 by blast
```

```
      qed
      ultimately show "f \<alpha>'' \<subseteq> f \<alpha>'" by blast
    qed
    then show "f \<in> \<N>1 r \<alpha>" unfolding \<N>1_def by blast
  qed
  moreover have " \<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>1 r \<beta>) \<longrightarrow> f \<in> \<N>1 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>1 r
      \<beta>)"
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have "\<forall>\<alpha>' \<alpha>''. \<alpha>' \<le>o \<alpha> \<and> \<alpha>'' \<le>o \<alpha>' \<
      longrightarrow> f \<alpha>'' \<subseteq> f \<alpha>'"
    proof (intro allI impI)
      fix \<alpha>' \<alpha>''::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> \<alpha>'' \<le>o \<alpha>'"
      then have "(\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>) \<and> (\<alpha>'' <o \<alpha>' \<or> \<
      alpha>'' =o \<alpha>')" using ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> f \<alpha>'' \<subseteq> f \<alpha>'"
        using d1 c1 ordLeq_Well_order_simp ordLeq_reflexive unfolding \<N>1_def by blast
      moreover have "\<alpha>' =o \<alpha> \<and> \<alpha>'' <o \<alpha>' \<longrightarrow> f \<alpha>'' \<
      subseteq> f \<alpha>'"
        using c2 b5 ordLess_ordIso_trans by blast
      moreover have "\<alpha>' =o \<alpha> \<and> \<alpha>'' =o \<alpha>' \<longrightarrow> f \<alpha>'' \<
      subseteq> f \<alpha>'" using b5 by blast
      ultimately show "f \<alpha>'' \<subseteq> f \<alpha>'" by blast
    qed
    then show "f \<in> \<N>1 r \<alpha>" unfolding \<N>1_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>1 r \<alpha>"] by blast
qed

lemma lem_Shinf_N2:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>2 r \<alpha>"
proof -
  have b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "f \<in> \<N>2 r {}" using lem_ord_subemp unfolding \<N>2_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>2 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>2 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>2 r \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> \<not> (\<alpha>' = {} \<or> isSuccOrd \<
      alpha>') \<longrightarrow> (\<nabla> f \<alpha>') = {}"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> \<not> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> (\<nabla> f \<alpha>') = {}" using d1 c1
      unfolding \<N>2_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> False"
      proof
        assume "\<alpha> =o \<alpha>'"
        moreover have "isSuccOrd \<alpha>" using c1 lem_ordint_sucord[of \<alpha>0 \<alpha>] unfolding
      sc_ord_def by blast
        ultimately have "isSuccOrd \<alpha>'" using lem_osucc_eq by blast
        then show "False" using d1 by blast
      qed
      ultimately show "(\<nabla> f \<alpha>') = {}" by blast
    qed
    then show "f \<in> \<N>2 r \<alpha>" unfolding \<N>2_def by blast
  qed
  moreover have " \<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>2 r \<beta>) \<longrightarrow> f \<in> \<N>2 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>2 r
      \<beta>)"
```

```
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> \<not> (\<alpha>' = {} \<or> isSuccOrd \<
      alpha>') \<longrightarrow> (\<nabla> f \<alpha>') = {}"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> \<not> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')"
      then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> (\<nabla> f \<alpha>') = {}"
      proof
        assume "\<alpha>' <o \<alpha>"
        moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
        ultimately show "(\<nabla> f \<alpha>') = {}" using c1 d1 unfolding \<N>2_def by blast
      qed
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> (\<nabla> f \<alpha>') = {}"
      proof
        assume "\<alpha>' =o \<alpha>"
        moreover have "(\<nabla> f \<alpha>) = {}" using c2 unfolding Dbk_def \<LL>_def by blast
        ultimately show "(\<nabla> f \<alpha>') = {}" using b5 lem_shrel_L_eq unfolding Dbk_def by blast
      qed
      ultimately show "(\<nabla> f \<alpha>') = {}" by blast
    qed
    then show "f \<in> \<N>2 r \<alpha>" unfolding \<N>2_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>2 r \<alpha>"] by blast
qed

lemma lem_Shinf_N3:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a5: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
    and a3: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<in> SF r \<longrightarrow>
               (\<omega>_ord \<le>o |A| \<longrightarrow> escl r A (F \<alpha> A) \<subseteq> (F \<alpha> A) \<
      and> clterm (Restr r (F \<alpha> A)) r)"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>3 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "\<LL> f {} = {}" unfolding \<LL>_def using b2 lem_ord_subemp ordLess_imp_ordLeq by blast
  then have "\<not> \<omega>_ord \<le>o |\<LL> f {}|" using ctwo_ordLess_natLeq finite_iff_ordLess_natLeq
      ordLeq_transitive by auto
  then have "f \<in> \<N>3 r {}" using b2 lem_ord_subemp unfolding \<N>3_def Field_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>3 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>3 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>3 r \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha
      >') \<longrightarrow> (\<omega>_ord \<le>o |\<LL> f \<alpha>'| \<longrightarrow>
      escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (f \<alpha>')) r)"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')" and d2: "\<omega>
      _ord \<le>o |\<LL> f \<alpha>'|"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> (\<omega>_ord \<le>o |\<LL> f \<alpha>'| \<
      longrightarrow>
                                  escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and>
      clterm (Restr r (f \<alpha>')) r)"
        using d1 c1 unfolding \<N>3_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> (\<omega>_ord \<le>o |\<LL> f \<alpha>'| \<
      longrightarrow>
                    escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (f
      \<alpha>')) r)"
      proof (intro impI)
        assume e1: "\<alpha> =o \<alpha>'" and e2: "\<omega>_ord \<le>o |\<LL> f \<alpha>'|"
        have "\<LL> f \<alpha> \<subseteq> f \<alpha>0"
        proof
          fix p
```

```
      assume "p \<in> \<LL> f \<alpha>"
      then obtain \<beta>::"'U rel" where "\<beta> <o \<alpha> \<and> p \<in> f \<beta>" unfolding \<LL>_def
     by blast
      moreover then have "\<beta> \<le>o \<alpha>0 \<and> \<alpha>0 \<le>o \<alpha>0" using c1 unfolding
    sc_ord_def
        using not_ordLess_iff_ordLeq ordLess_Well_order_simp by blast
      moreover then have "f \<in> \<N>1 r \<alpha>0" using a0 a1 lem_Shinf_N1[of f F] ordLeq_Well_order_simp
     by metis
      ultimately show "p \<in> f \<alpha>0" unfolding \<N>1_def by blast
     qed
     moreover have "f \<alpha>0 \<subseteq> \<LL> f \<alpha>" using c1 unfolding sc_ord_def \<LL>_def by
    blast
     ultimately have e3: "\<LL> f \<alpha> = f \<alpha>0" by blast
     then have "\<omega>_ord \<le>o |f \<alpha>0|" using e1 e2 lem_shrel_L_eq by metis
     moreover have "Well_order \<alpha>0" using c1 unfolding sc_ord_def ordLess_def by blast
     moreover then have "(f \<alpha>0) \<in> SF r"
         using a5 unfolding \<N>5_def using ordLeq_reflexive by blast
     moreover have "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c1 b3 by blast
     ultimately have e4: "escl r (f \<alpha>0) (f \<alpha>) \<subseteq> f \<alpha> \<and> clterm (Restr r (f
    \<alpha>)) r" using a3 by metis
     then have "escl r (\<LL> f \<alpha>) (f \<alpha>) \<subseteq> f \<alpha>" using e3 by simp
     then have "escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>'" using e1 b5 lem_shrel_L_eq
    by metis
     moreover have "clterm (Restr r (f \<alpha>')) r" using e1 e4 b5 by metis
     ultimately show "escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r
    (f \<alpha>')) r" by blast
    qed
    ultimately show "escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (
    f \<alpha>')) r" using d2 by blast
   qed
   then show "f \<in> \<N>3 r \<alpha>" unfolding \<N>3_def by blast
  qed
  moreover have "\<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>3 r \<beta>) \<longrightarrow> f \<in> \<N>3 r \<alpha>"
  proof (intro allI impI)
   fix \<alpha>::"'U rel"
   assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>3 r
     \<beta>)"
   then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
     using b4 by blast
   have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha
     >') \<longrightarrow> (\<omega>_ord \<le>o |\<LL> f \<alpha>'| \<longrightarrow>
          escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (f \<alpha
     >')) r)"
   proof (intro allI impI)
     fix \<alpha>'::"'U rel"
     assume d1: "\<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')" and d2: "\<omega>
     _ord \<le>o |\<LL> f \<alpha>'|"
     then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
     moreover have "\<alpha>' <o \<alpha> \<longrightarrow> (\<omega>_ord \<le>o |\<LL> f \<alpha>'| \<
     longrightarrow>
         escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (f \<alpha>'))
      r)"
     proof
       assume "\<alpha>' <o \<alpha>"
       moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
       ultimately show "(\<omega>_ord \<le>o |\<LL> f \<alpha>'| \<longrightarrow> escl r (\<LL> f \<alpha>') (
     f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (f \<alpha>')) r)"
         using c1 d1 unfolding \<N>3_def by blast
     qed
     moreover have "\<alpha>' =o \<alpha> \<longrightarrow> False"
     proof
       assume "\<alpha>' =o \<alpha>"
       moreover then have "\<alpha>' = {} \<or> isSuccOrd \<alpha>" using d1 lem_osucc_eq by blast
       moreover have "\<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)" using c1 unfolding lm_ord_def by blast
       ultimately have "\<alpha>' =o \<alpha> \<and> \<alpha>' = {} \<and> \<alpha> \<noteq> {}" by blast
       then show "False" by (metis iso_ozero_empty ordIso_symmetric ozero_def)
     qed
     ultimately show "escl r (\<LL> f \<alpha>') (f \<alpha>') \<subseteq> f \<alpha>' \<and> clterm (Restr r (
     f \<alpha>')) r" using d2 by blast
   qed
   then show "f \<in> \<N>3 r \<alpha>" unfolding \<N>3_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>3 r \<alpha>"] by blast
qed

lemma lem_Shinf_N4:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
```

```
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a5: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
    and a4: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<in> SF r \<longrightarrow> (\<
      forall> a\<in>A. r``{a} \<subseteq> w_dncl r A \<or> r``{a} \<inter> (F \<alpha> A - w_dncl r A) \<noteq>
      {})"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>4 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "\<LL> f {} = {}" unfolding \<LL>_def using lem_ord_subemp ordLeq_iff_ordLess_or_ordIso
      ordLess_irreflexive by blast
  then have "f \<in> \<N>4 r {}" using lem_ord_subemp unfolding \<N>4_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>4 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>4 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>4 r \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha
      >') \<longrightarrow>
        ( \<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>') \<or> r``{a}\<
      inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<noteq>{} )"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> ( \<forall>a \<in> (\<LL> f \<alpha>'). r``{a}
       \<subseteq> w_dncl r (\<LL> f \<alpha>') \<or> r``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))
      \<noteq>{} )"
        using d1 c1 unfolding \<N>4_def Dbk_def \<W>_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> ( \<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<
      subseteq> w_dncl r (\<LL> f \<alpha>') \<or> r``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<
      noteq>{} )"
      proof
        assume e1: "\<alpha> =o \<alpha>'"
        have "Well_order \<alpha>0" using c1 unfolding sc_ord_def ordLess_def by blast
        moreover then have "(f \<alpha>0) \<in> SF r"
            using a5 unfolding \<N>5_def using ordLeq_reflexive by blast
        moreover have "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c1 b3 by blast
        ultimately have e2: "\<forall>a \<in> (f \<alpha>0). r``{a} \<subseteq> w_dncl r (f \<alpha>0) \<or> r
      ``{a}\<inter>(f \<alpha> - w_dncl r (f \<alpha>0))\<noteq>{}"
          using a4 by metis
        have "\<LL> f \<alpha> \<subseteq> f \<alpha>0"
        proof
          fix p
          assume "p \<in> \<LL> f \<alpha>"
          then obtain \<beta>::"'U rel" where "\<beta> <o \<alpha> \<and> p \<in> f \<beta>" unfolding \<LL>_def
       by blast
          moreover then have "\<beta> \<le>o \<alpha>0 \<and> \<alpha>0 \<le>o \<alpha>0" using c1 unfolding
      sc_ord_def
             using not_ordLess_iff_ordLeq ordLess_Well_order_simp by blast
          moreover then have "f \<in> \<N>1 r \<alpha>0" using a0 a1 lem_Shinf_N1[of f F] ordLeq_Well_order_simp
       by metis
          ultimately show "p \<in> f \<alpha>0" unfolding \<N>1_def by blast
        qed
        moreover have "f \<alpha>0 \<subseteq> \<LL> f \<alpha>" using c1 unfolding sc_ord_def \<LL>_def by
      blast
        ultimately have "\<LL> f \<alpha> = f \<alpha>0" by blast
        then have "\<LL> f \<alpha>' = f \<alpha>0" using e1 lem_shrel_L_eq by blast
        then show "\<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>') \<or> r
      ``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<noteq>{}"
          using e2 e1 b5 by metis
      qed
      ultimately show "\<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>') \<
      or> r``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<noteq>{}" by blast
    qed
    then show "f \<in> \<N>4 r \<alpha>" unfolding \<N>4_def Dbk_def \<W>_def by blast
  qed
  moreover have " \<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>4 r \<beta>) \<longrightarrow> f \<in> \<N>4 r \<alpha>"
```

```
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>4 r
      \<beta>)"
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha
      >') \<longrightarrow>
      ( \<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>') \<or> r``{a}\<
      inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<noteq>{} )"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')"
      then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> ( \<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<
       subseteq> w_dncl r (\<LL> f \<alpha>') \<or> r``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<
       noteq>{} )"
       proof
         assume "\<alpha>' <o \<alpha>"
         moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
         ultimately show "( \<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>')
        \<or> r``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<noteq>{} )"
           using c1 d1 unfolding \<N>4_def Dbk_def \<W>_def by blast
       qed
       moreover have "\<alpha>' =o \<alpha> \<longrightarrow> False"
       proof
         assume "\<alpha>' =o \<alpha>"
         moreover then have "\<alpha>' = {} \<or> isSuccOrd \<alpha>" using d1 lem_osucc_eq by blast
         moreover have "\<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)" using c1 unfolding lm_ord_def by blast
         ultimately have "\<alpha>' =o \<alpha> \<and> \<alpha>' = {} \<and> \<alpha> \<noteq> {}" by blast
         then show "False" by (metis iso_ozero_empty ordIso_symmetric ozero_def)
       qed
       ultimately show "\<forall>a \<in> (\<LL> f \<alpha>'). r``{a} \<subseteq> w_dncl r (\<LL> f \<alpha>') \<
       or> r``{a}\<inter>(f \<alpha>' - w_dncl r (\<LL> f \<alpha>'))\<noteq>{}" by blast
    qed
    then show "f \<in> \<N>4 r \<alpha>" unfolding \<N>4_def Dbk_def \<W>_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>4 r \<alpha>"] by blast
qed

lemma lem_Shinf_N5:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
assumes a5: "\<forall> \<alpha> A. (Well_order \<alpha> \<and> A \<in> SF r) \<longrightarrow> (F \<alpha> A) \<
       in> SF r"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
       alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
        \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
       unfolding \<T>_def by blast+
  have "f \<in> \<N>5 r {}" using b2 lem_ord_subemp unfolding \<N>5_def SF_def Field_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>5 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>5 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>5 r \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<longrightarrow> (f \<alpha>') \<in> SF r"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha>"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> Field (Restr r (f \<alpha>')) = (f \<alpha>')"
       using c1 unfolding \<N>5_def SF_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> (f \<alpha>') \<in> SF r"
      proof
        assume "\<alpha> =o \<alpha>'"
        moreover have "(f \<alpha>) \<in> SF r"
        proof -
          have "\<alpha>0 \<le>o \<alpha>0" using c1 unfolding sc_ord_def
            using ordLess_Well_order_simp ordLeq_reflexive by blast
          then have "(f \<alpha>0) \<in> SF r" using c1 unfolding \<N>5_def by blast
```

```
        moreover have "Well_order \<alpha>0" using c1 unfolding sc_ord_def using ordLess_Well_order_simp by
      blast
        moreover have "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c1 b3 by blast
        ultimately show "(f \<alpha>) \<in> SF r" using a5 by metis
       qed
       ultimately show "(f \<alpha>') \<in> SF r" using b5 by metis
      qed
      ultimately show "(f \<alpha>') \<in> SF r" unfolding SF_def by blast
    qed
    then show "f \<in> \<N>5 r \<alpha>" unfolding \<N>5_def by blast
  qed
  moreover have " \<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>5 r \<beta>) \<longrightarrow> f \<in> \<N>5 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>5 r
      \<beta>)"
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<longrightarrow> (f \<alpha>') \<in> SF r"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha>"
      then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> Field (Restr r (f \<alpha>')) = (f \<alpha>')"
      proof
        assume "\<alpha>' <o \<alpha>"
        moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
        ultimately show "Field (Restr r (f \<alpha>')) = (f \<alpha>')" using c1 d1 unfolding \<N>5_def SF_def
       by blast
      qed
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> (f \<alpha>') \<in> SF r"
      proof
        assume "\<alpha>' =o \<alpha>"
        moreover have "(f \<alpha>) \<in> SF r"
        proof -
          have "\<forall> \<beta>. \<beta> <o \<alpha> \<longrightarrow> (f \<beta>) \<in> SF r" using c1
       unfolding \<N>5_def
            using ordLess_Well_order_simp ordLeq_reflexive by blast
          then show ?thesis using c2 lem_Relprop_sat_un[of "{D. \<exists>\<beta>. \<beta> <o \<alpha> \<and> D =
         f \<beta>}" r "f \<alpha>"] unfolding SF_def by blast
        qed
        ultimately show "(f \<alpha>') \<in> SF r" using b5 by metis
      qed
      ultimately show "(f \<alpha>') \<in> SF r" unfolding SF_def by blast
    qed
    then show "f \<in> \<N>5 r \<alpha>" unfolding \<N>5_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>5 r \<alpha>"] by blast
qed

lemma lem_Shinf_N6:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a5: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
    and a6: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<in> SF r \<longrightarrow> CCR (
      Restr r (F \<alpha> A))"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>6 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "f \<in> \<N>6 r {}" using b2 lem_ord_subemp unfolding \<N>6_def CCR_def Field_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>6 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>6 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>6 r \<alpha>0"
    then have c2: "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using b3 by blast
    have "\<forall>\<alpha>'. \<alpha>' \<le>o \<alpha> \<longrightarrow> CCR (Restr r (f \<alpha>'))"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume "\<alpha>' \<le>o \<alpha>"
```

```
      then have "\<alpha>' \<le>o \<alpha>0 \<or> \<alpha>' =o \<alpha>" using c1 unfolding sc_ord_def
        by (meson ordIso_iff_ordLeq ordLeq_Well_order_simp ordLess_Well_order_simp ordLess_or_ordLeq)
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> CCR (Restr r (f \<alpha>'))" using c1
      unfolding \<N>6_def by blast
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> CCR (Restr r (f \<alpha>'))"
      proof
        assume "\<alpha>' =o \<alpha>"
        moreover have "CCR (Restr r (f \<alpha>))"
        proof -
          have "Well_order \<alpha>0"
            using c1 ordLess_Well_order_simp unfolding sc_ord_def by blast
          moreover then have "(f \<alpha>0) \<in> SF r"
            using a5 unfolding \<N>5_def using ordLeq_reflexive by blast
          ultimately show "CCR (Restr r (f \<alpha>))" unfolding c2 using a6 by blast
        qed
        ultimately show "CCR (Restr r (f \<alpha>'))" using b5 by metis
      qed
      ultimately show "CCR (Restr r (f \<alpha>'))" by blast
    qed
    then show "f \<in> \<N>6 r \<alpha>" unfolding \<N>6_def by blast
  qed
  moreover have "\<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>6 r \<beta>) \<longrightarrow> f \<in> \<N>6 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>6 r
      \<beta>)"
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have c3: "\<forall>\<alpha>'. \<alpha>' \<le>o \<alpha> \<longrightarrow> CCR (Restr r (f \<alpha>'))"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume "\<alpha>' \<le>o \<alpha>"
      then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordIso_iff_ordLeq
      ordLeq_Well_order_simp ordLess_or_ordLeq by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> CCR (Restr r (f \<alpha>'))" using c1 unfolding \<N
      >6_def
        using ordLess_Well_order_simp ordLeq_reflexive by blast
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> CCR (Restr r (f \<alpha>'))"
      proof
        assume "\<alpha>' =o \<alpha>"
        moreover have "CCR (Restr r (f \<alpha>))"
        proof -
          obtain C where f1: "C = { A. \<exists>  \<beta>::'U rel. \<beta> <o \<alpha> \<and> A = f \<beta> }"
      by blast
          obtain S where f2: "S = { s. \<exists> A \<in> C. s = Restr r A }" by blast
          have f3: "\<forall>A1 \<in> C. \<forall>A2 \<in> C. A1 \<subseteq> A2 \<or> A2 \<subseteq> A1"
          proof (intro ballI)
            fix A1 A2
            assume "A1 \<in> C" and "A2 \<in> C"
            then obtain \<beta>1 \<beta>2::"'U rel" where "A1 = f \<beta>1 \<and> A2 = f \<beta>2 \<and> \<beta
      >1 <o \<alpha> \<and> \<beta>2 <o \<alpha>" using f1 by blast
            moreover then have "(\<beta>1 \<le>o \<beta>2 \<or> \<beta>2 \<le>o \<beta>1) \<and> \<beta>1 \<le>o
       \<alpha> \<and> \<beta>2 \<le>o \<alpha>"
              using ordLeq_total ordLess_Well_order_simp ordLess_imp_ordLeq by blast
            moreover have "f \<in> \<N>1 r \<alpha>" using a0 a1 c1 lem_Shinf_N1[of f F r] unfolding lm_ord_def
      by blast
            ultimately show "A1 \<subseteq> A2 \<or> A2 \<subseteq> A1" unfolding \<N>1_def by blast
          qed
          have "\<forall> s \<in> S. CCR s" using f1 f2 c1 unfolding \<N>6_def
            using ordLess_Well_order_simp ordLeq_reflexive by blast
          moreover have "\<forall>s1\<in>S. \<forall>s2\<in>S. s1 \<subseteq> s2 \<or> s2 \<subseteq> s1" using
      f2 f3 by blast
          ultimately have "CCR (\<Union> S)" using lem_Relprop_ccr_ch_un[of S] by blast
          moreover have "Restr r ( \<Union> {D. \<exists>\<beta>. \<beta> <o \<alpha> \<and> D = f \<beta>} ) =
      \<Union> S"
            using f1 f2 f3 lem_Relprop_restr_ch_un[of C r] by blast
          ultimately show ?thesis unfolding c2 by simp
        qed
        ultimately show "CCR (Restr r (f \<alpha>'))" using b5 by metis
      qed
      ultimately show "CCR (Restr r (f \<alpha>'))" by blast
    qed
    then show "f \<in> \<N>6 r \<alpha>" unfolding \<N>6_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>6 r \<alpha>"] by blast
qed
```

```
lemma lem_Shinf_N7:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a7: "\<forall> \<alpha> A. ( |A| <o \<omega>_ord \<longrightarrow> |F \<alpha> A| <o \<omega>_ord )
                  \<and> ( \<omega>_ord \<le>o |A| \<longrightarrow> |F \<alpha> A| \<le>o |A| ) "
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>7 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
       alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
        \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
       unfolding \<T>_def by blast+
  have "\<forall>\<alpha>::'U rel. \<alpha> \<le>o {} \<longrightarrow> |f \<alpha>| \<le>o \<alpha> \<and> |f
       \<alpha>| <o \<omega>_ord"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume "\<alpha> \<le>o {}"
    moreover then have "(f \<alpha>) = {}" using b2 lem_ord_subemp by blast
    ultimately show "|f \<alpha>| \<le>o \<alpha> \<and> |f \<alpha>| <o \<omega>_ord" using lem_ord_subemp
      by (metis Field_natLeq card_of_empty1 card_of_empty5 ctwo_def ctwo_ordLess_natLeq natLeq_well_order_on
      not_ordLeq_iff_ordLess ordLeq_Well_order_simp)
  qed
  then have "f \<in> \<N>7 r {}" unfolding \<N>7_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>7 r \<alpha>0  \<
       longrightarrow> f \<in> \<N>7 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>7 r \<alpha>0"
    then have c2: "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using b3 by blast
    have "\<forall>\<alpha>'. \<alpha>' \<le>o \<alpha> \<and> \<omega>_ord \<le>o \<alpha>' \<longrightarrow> |
      f \<alpha>'| \<le>o \<alpha>'"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> \<omega>_ord \<le>o \<alpha>'"
      then have "\<alpha>' \<le>o \<alpha>0 \<or> \<alpha>' =o \<alpha>" using c1 unfolding sc_ord_def
        by (meson ordIso_iff_ordLeq ordLeq_Well_order_simp ordLess_Well_order_simp ordLess_or_ordLeq)
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> |f \<alpha>'| \<le>o \<alpha>'" using c1 d1
      unfolding \<N>7_def by blast
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> |f \<alpha>'| \<le>o \<alpha>'"
      proof
        assume e1: "\<alpha>' =o \<alpha>"
        then have e2: "\<omega>_ord \<le>o \<alpha>" using d1 b5 ordLeq_transitive by blast
        then have e3: "\<omega>_ord \<le>o \<alpha>0" using c1 lem_ord_suc_ge_w by blast
        then have "Well_order \<alpha>0 \<and> |f \<alpha>0| \<le>o \<alpha>0"
          using c1 unfolding sc_ord_def \<N>7_def using ordLess_Well_order_simp ordLeq_reflexive by blast
        moreover then have "|f \<alpha>| \<le>o |f \<alpha>0| \<or> |f \<alpha>| <o \<omega>_ord" unfolding c2
      using a7
          using finite_iff_ordLess_natLeq infinite_iff_natLeq_ordLeq by blast
        moreover have "\<alpha>0 \<le>o \<alpha>" using c1 unfolding sc_ord_def using ordLess_imp_ordLeq by
      blast
        ultimately have "|f \<alpha>| \<le>o \<alpha>" using e3 ordLeq_transitive ordLess_imp_ordLeq by metis
        then show "|f \<alpha>'| \<le>o \<alpha>'" using b5 e1 ordIso_iff_ordLeq ordLeq_transitive by metis
      qed
      ultimately show "|f \<alpha>'| \<le>o \<alpha>'" by blast
    qed
    moreover have "\<forall>\<alpha>'. \<alpha>' \<le>o \<alpha> \<and> \<alpha>' <o \<omega>_ord \<
      longrightarrow> |f \<alpha>'| <o \<omega>_ord"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> \<alpha>' <o \<omega>_ord"
      then have "\<alpha>' \<le>o \<alpha>0 \<or> \<alpha>' =o \<alpha>" using c1 unfolding sc_ord_def
        by (meson ordIso_iff_ordLeq ordLeq_Well_order_simp ordLess_Well_order_simp ordLess_or_ordLeq)
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> |f \<alpha>'| <o \<omega>_ord" using c1 d1
      unfolding \<N>7_def by blast
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> |f \<alpha>'| <o \<omega>_ord"
      proof
        assume e1: "\<alpha>' =o \<alpha>"
        then have e2: "\<alpha> <o \<omega>_ord" using d1 ordIso_iff_ordLeq ordIso_ordLess_trans by blast
        then have e3: "\<alpha>0 <o \<omega>_ord" using c1 unfolding sc_ord_def using ordLeq_ordLess_trans
      ordLess_imp_ordLeq by blast
        then have "Well_order \<alpha>0 \<and> |f \<alpha>0| <o \<omega>_ord"
          using c1 unfolding sc_ord_def \<N>7_def using ordLess_Well_order_simp ordLeq_reflexive by blast
        then have "|f \<alpha>| <o \<omega>_ord" unfolding c2 using a7 by blast
        then show "|f \<alpha>'| <o \<omega>_ord" using b5 e1 by metis
      qed
```

```
    ultimately show "|f \<alpha>'| <o \<omega>_ord" by blast
  qed
  ultimately show "f \<in> \<N>7 r \<alpha>" unfolding \<N>7_def by blast
qed
moreover have "\<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
    longrightarrow> f \<in> \<N>7 r \<beta>) \<longrightarrow> f \<in> \<N>7 r \<alpha>"
proof (intro allI impI)
  fix \<alpha>::"'U rel"
  assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>7 r
    \<beta>)"
  then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
    using b4 by blast
  have "\<forall>\<alpha>'. \<alpha>' \<le>o \<alpha> \<and> \<omega>_ord \<le>o \<alpha>' \<longrightarrow> |
    f \<alpha>'| \<le>o \<alpha>'"
  proof (intro allI impI)
    fix \<alpha>'::"'U rel"
    assume e1: "\<alpha>' \<le>o \<alpha> \<and> \<omega>_ord \<le>o \<alpha>'"
    then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordIso_iff_ordLeq
    ordLeq_Well_order_simp ordLess_or_ordLeq by blast
    moreover have "\<alpha>' <o \<alpha> \<longrightarrow> |f \<alpha>'| \<le>o \<alpha>'" using c1 e1
    unfolding \<N>7_def
      using ordLess_Well_order_simp ordLeq_reflexive by blast
    moreover have "\<alpha>' =o \<alpha> \<longrightarrow> |f \<alpha>'| \<le>o \<alpha>'"
    proof
      assume "\<alpha>' =o \<alpha>"
      moreover have "|f \<alpha>| \<le>o \<alpha>"
      proof -
        obtain S where f1: "S = { A. \<exists>  \<beta>::'U rel. \<beta> <o \<alpha> \<and> A = f \<beta> }"
    by blast
        have f2: "\<omega>_ord \<le>o \<alpha>" using c1 lem_lmord_inf lem_inford_ge_w unfolding lm_ord_def by
     blast
        have f3: "\<forall> s \<in> S. |s| \<le>o \<alpha>"
        proof
          fix s
          assume "s \<in> S"
          then obtain \<beta> where "\<beta> <o \<alpha> \<and> s = f \<beta>" using f1 by blast
          then show "|s| \<le>o \<alpha>" using c1 f2 unfolding \<N>7_def
            apply clarsimp
            by (metis card_of_Well_order natLeq_Well_order not_ordLess_ordLeq ordLeq_reflexive
    ordLess_Well_order_simp ordLess_or_ordLeq ordLess_transitive)
        qed
        moreover have "|S| \<le>o \<alpha>"
        proof -
          have "f ` {\<gamma>. \<gamma> <o \<alpha>} = S" using f1 by force
          then show ?thesis using f1 f2 b5 lem_ord_int_card_le_inf[of f \<alpha> ] by blast
        qed
        ultimately have "|\<Union> S| \<le>o \<alpha>" using f2 lem_card_un_bnd[of S \<alpha>] by blast
        then show ?thesis unfolding f1 c2 by blast
      qed
      ultimately show "|f \<alpha>'| \<le>o \<alpha>'" using b5 ordIso_iff_ordLeq ordLeq_transitive by metis
    qed
    ultimately show "|f \<alpha>'| \<le>o \<alpha>'" by blast
  qed
  moreover have "\<forall>\<alpha>'. \<alpha>' \<le>o \<alpha> \<and> \<alpha>' <o \<omega>_ord \<
    longrightarrow> |f \<alpha>'| <o \<omega>_ord"
  proof (intro allI impI)
    fix \<alpha>'::"'U rel"
    assume e1: "\<alpha>' \<le>o \<alpha> \<and> \<alpha>' <o \<omega>_ord"
    then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordIso_iff_ordLeq
    ordLeq_Well_order_simp ordLess_or_ordLeq by blast
    moreover have "\<alpha>' <o \<alpha> \<longrightarrow> |f \<alpha>'| <o \<omega>_ord" using c1 e1
    unfolding \<N>7_def
      using ordLess_Well_order_simp ordLeq_reflexive by blast
    moreover have "\<alpha>' =o \<alpha> \<longrightarrow> |f \<alpha>'| <o \<omega>_ord"
    proof
      assume "\<alpha>' =o \<alpha>"
      moreover have "|f \<alpha>| \<le>o \<alpha>"
      proof -
        obtain S where f1: "S = { A. \<exists>  \<beta>::'U rel. \<beta> <o \<alpha> \<and> A = f \<beta> }"
    by blast
        have f2: "\<omega>_ord \<le>o \<alpha>" using c1 lem_lmord_inf lem_inford_ge_w unfolding lm_ord_def by
     blast
        have f3: "\<forall> s \<in> S. |s| \<le>o \<alpha>"
        proof
          fix s
          assume "s \<in> S"
          then obtain \<beta> where "\<beta> <o \<alpha> \<and> s = f \<beta>" using f1 by blast
          then show "|s| \<le>o \<alpha>" using c1 f2 unfolding \<N>7_def
```

```
            apply clarsimp
            by (metis card_of_Well_order natLeq_Well_order not_ordLess_ordLeq ordLeq_reflexive
      ordLess_Well_order_simp ordLess_or_ordLeq ordLess_transitive)
        qed
        moreover have "|S| \<le>o \<alpha>"
        proof -
          have "f ` {\<gamma>. \<gamma> <o \<alpha>} = S" using f1 by force
          then show ?thesis using f1 f2 b5 lem_ord_int_card_le_inf[of f \<alpha> ] by blast
        qed
        ultimately have "|\<Union> S| \<le>o \<alpha>" using f2 lem_card_un_bnd[of S \<alpha>] by blast
        then show ?thesis unfolding f1 c2 by blast
      qed
      ultimately show "|f \<alpha>'| <o \<omega>_ord" using e1 b5 ordIso_iff_ordLeq ordLeq_transitive
        by (metis card_of_Well_order natLeq_Well_order not_ordLess_ordLeq ordLess_or_ordLeq)
    qed
    ultimately show "|f \<alpha>'| <o \<omega>_ord" by blast
  qed
  ultimately show "f \<in> \<N>7 r \<alpha>" unfolding \<N>7_def by blast
 qed
 ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>7 r \<alpha>"] by blast
qed

lemma lem_Shinf_N8:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
       and Ps::"'U set set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a5: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
    and a7: "\<forall> \<alpha> A. ( |A| <o \<omega>_ord \<longrightarrow> |F \<alpha> A| <o \<omega>_ord )
                \<and> ( \<omega>_ord \<le>o |A| \<longrightarrow> |F \<alpha> A| \<le>o |A| ) "
    and a8: "\<forall>\<alpha> A. A \<in> SF r \<longrightarrow> \<E>p r Ps A (F \<alpha> A)"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>8 r Ps \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "f \<in> \<N>8 r Ps {}" using b2 lem_ord_subemp unfolding \<N>8_def SCF_def Field_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>8 r Ps \<alpha>0 \<
      longrightarrow> f \<in> \<N>8 r Ps \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>8 r Ps \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha
      >') \<longrightarrow>
        ((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha>'| )) \<longrightarrow>
      (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))"
    proof (intro allI, rule impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> ((\<exists>P. Ps = {P}) \<or> (\<not> finite
      Ps \<and> |Ps| \<le>o |f \<alpha>'| )) \<longrightarrow>
                (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))"
        using d1 c1 unfolding \<N>8_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> ((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<
      and> |Ps| \<le>o |f \<alpha>'| )) \<longrightarrow>
                (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))"
      proof (intro ballI impI)
        fix P
        assume e1: "\<alpha> =o \<alpha>'" and e2: "(\<exists>P'. Ps = {P'}) \<or> (\<not> finite Ps \<and> |Ps|
       \<le>o |f \<alpha>'| )" and e3: "P \<in> Ps"
        have e4: "f \<alpha>' = f \<alpha>" using b5 e1 by blast
        have "Well_order \<alpha>0" using c1 unfolding sc_ord_def ordLess_def by blast
        then have "(f \<alpha>0) \<in> SF r" using a5 unfolding \<N>5_def using ordLeq_reflexive by blast
        moreover have e5: "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c1 b3 by blast
        moreover have "\<not> (\<exists>P'. Ps = {P'}) \<longrightarrow> (\<not> finite Ps \<and> |Ps| \<le>o |f
       \<alpha>0| )"
        proof
          assume f1: "\<not> (\<exists>P'. Ps = {P'})"
          then have f2: "\<omega>_ord \<le>o |Ps| \<and> |Ps| \<le>o |f \<alpha>|" using e2 e4
      infinite_iff_natLeq_ordLeq by metis
```

```
        then have "\<not> |F \<alpha>0 (f \<alpha>0)| <o \<omega>_ord" using e5
          by (metis finite_ordLess_infinite2 infinite_iff_natLeq_ordLeq not_ordLess_ordLeq)
        then have "\<not> |f \<alpha>0| <o \<omega>_ord" using a7 by blast
        then have "\<omega>_ord \<le>o |f \<alpha>0|" by (metis finite_iff_ordLess_natLeq
     infinite_iff_natLeq_ordLeq)
        then have "|F \<alpha>0 (f \<alpha>0)| \<le>o |f \<alpha>0|" using a7 by blast
        then have "|Ps| \<le>o |f \<alpha>0|" using f2 e5 ordLeq_transitive by metis
        then show "\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha>0|" using f1 e2 by blast
       qed
       ultimately show "f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>'))" using e3 e4 a8 unfolding \<E>
     p_def by metis
     qed
     ultimately show "((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha>'| )) \<
     longrightarrow> (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))" by blast
   qed
   then show "f \<in> \<N>8 r Ps \<alpha>" unfolding \<N>8_def by blast
 qed
 moreover have "\<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
     longrightarrow> f \<in> \<N>8 r Ps \<beta>) \<longrightarrow> f \<in> \<N>8 r Ps \<alpha>"
 proof (intro allI impI)
   fix \<alpha>::"'U rel"
   assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>8 r
     Ps \<beta>)"
   then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
     using b4 by blast
   have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha
     >') \<longrightarrow>
     ((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha>'| )) \<longrightarrow> (\<
     forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))"
   proof (intro allI, rule impI)
     fix \<alpha>'::"'U rel"
     assume d1: "\<alpha>' \<le>o \<alpha> \<and> (\<alpha>' = {} \<or> isSuccOrd \<alpha>')"
     then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
     moreover have "\<alpha>' <o \<alpha> \<longrightarrow> ((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<
     and> |Ps| \<le>o |f \<alpha>'| )) \<longrightarrow>
               (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))"
     proof
       assume "\<alpha>' <o \<alpha>"
       moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
       ultimately show "((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha>'| )) \<
     longrightarrow>
               (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))"
         using c1 d1 unfolding \<N>8_def by blast
     qed
     moreover have "\<alpha>' =o \<alpha> \<longrightarrow> False"
     proof
       assume "\<alpha>' =o \<alpha>"
       moreover then have "\<alpha>' = {} \<or> isSuccOrd \<alpha>" using d1 lem_osucc_eq by blast
       moreover have "\<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)" using c1 unfolding lm_ord_def by blast
       ultimately have "\<alpha>' =o \<alpha> \<and> \<alpha>' = {} \<and> \<alpha> \<noteq> {}" by blast
       then show "False" by (metis iso_ozero_empty ordIso_symmetric ozero_def)
     qed
     ultimately show "((\<exists>P. Ps = {P}) \<or> (\<not> finite Ps \<and> |Ps| \<le>o |f \<alpha>'| )) \<
     longrightarrow>
               (\<forall>P\<in>Ps. f \<alpha>' \<inter> P \<in> SCF (Restr r (f \<alpha>')))" by blast
   qed
   then show "f \<in> \<N>8 r Ps \<alpha>" unfolding \<N>8_def by blast
 qed
 ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>8 r Ps \<alpha>"] by blast
qed

lemma lem_Shinf_N9:
fixes r::"'U rel" and g::"'U rel \<Rightarrow> 'U"
  and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a2: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> g \<alpha> \<in> Field r \<
      longrightarrow> g \<alpha> \<in> F \<alpha> A"
    and a11: "\<omega>_ord \<le>o |Field r| \<longrightarrow> Field r \<subseteq> g ` { \<gamma>::'U rel. \<
      gamma> <o |Field r| }"
shows "f \<in> \<N>9 r |Field r|"
proof -
  have b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))" using a0 unfolding \<T>_def by blast+
  have "\<forall> a \<in> Field r. \<omega>_ord \<le>o |Field r| \<longrightarrow> a \<in> f |Field r|"
  proof (intro ballI impI)
    fix a
    assume c1: "a \<in> Field r" and c2: "\<omega>_ord \<le>o |Field r|"
```

```
    then obtain \<alpha>0::"'U rel" where c4: "\<alpha>0 <o |Field r| \<and> g \<alpha>0 = a" using a11 by blast
    moreover then obtain \<alpha> where c5: "sc_ord \<alpha>0 \<alpha>" using lem_sucord_ex[of \<alpha>0 "|Field
       r|"] by blast
    ultimately have c6: "\<alpha> \<le>o |Field r|" unfolding sc_ord_def by blast
    have "Well_order |Field r|" by simp
    then have "f \<in> \<N>1 r |Field r|" using a0 a1 lem_Shinf_N1 unfolding card_order_on_def by metis
    moreover have c7: "|Field r| \<le>o |Field r|" by simp
    moreover have "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c5 b3 by blast
    moreover have "a \<in> F \<alpha>0 (f \<alpha>0)" using a2 c4 c1 ordLess_Well_order_simp by blast
    ultimately show "a \<in> f |Field r|" using c6 unfolding \<N>1_def by blast
  qed
  then show ?thesis unfolding \<N>9_def by blast
qed

lemma lem_Shinf_N10:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a5: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
    and a10: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<in> SF r \<longrightarrow>
            ((\<exists>y.  (F \<alpha> A) - dncl r A \<subseteq> {y}) \<longrightarrow> (Field r \<subseteq>
      dncl r (F \<alpha> A)))"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>10 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
       alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
        \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
       unfolding \<T>_def by blast+
  have "f \<in> \<N>10 r {}" using b2 lem_ord_subemp unfolding \<N>10_def \<Q>_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>10 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>10 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>10 r \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<longrightarrow>
        ((\<exists>y.  (f \<alpha>') - dncl r (\<LL> f \<alpha>') = {y}) \<longrightarrow> (Field r \<subseteq>
      dncl r (f \<alpha>')))"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha>" and d2: "\<exists>y. (f \<alpha>') - dncl r (\<LL> f \<alpha>') = {
      y}"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> ((\<exists>y.  (f \<alpha>') - dncl r (\<LL> f
       \<alpha>') = {y}) \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>')))"
        using d1 c1 unfolding \<N>10_def \<Q>_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>'))"
      proof
        assume e1: "\<alpha> =o \<alpha>'"
        have "Well_order \<alpha>0" using c1 unfolding sc_ord_def ordLess_def by blast
        moreover then have "(f \<alpha>0) \<in> SF r"
          using a5 unfolding \<N>5_def using ordLeq_reflexive by blast
        moreover have "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c1 b3 by blast
        ultimately have e2: "((\<exists>y.  (f \<alpha>) - dncl r (f \<alpha>0) \<subseteq> {y}) \<
      longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>)))"
          using a10 by metis
        have "\<LL> f \<alpha> \<subseteq> f \<alpha>0"
        proof
          fix p
          assume "p \<in> \<LL> f \<alpha>"
          then obtain \<beta>::"'U rel" where "\<beta> <o \<alpha> \<and> p \<in> f \<beta>" unfolding \<LL>_def
       by blast
          moreover then have "\<beta> \<le>o \<alpha>0 \<and> \<alpha>0 \<le>o \<alpha>0" using c1 unfolding
      sc_ord_def
            using not_ordLess_iff_ordLeq ordLess_Well_order_simp by blast
          moreover then have "f \<in> \<N>1 r \<alpha>0" using a0 a1 lem_Shinf_N1[of f F] ordLeq_Well_order_simp
       by metis
          ultimately show "p \<in> f \<alpha>0" unfolding \<N>1_def by blast
        qed
        moreover have "f \<alpha>0 \<subseteq> \<LL> f \<alpha>" using c1 unfolding sc_ord_def \<LL>_def by
      blast
        ultimately have "\<LL> f \<alpha> = f \<alpha>0" by blast
        then have "\<LL> f \<alpha>' = f \<alpha>0" using e1 lem_shrel_L_eq by blast
```

```
        then show "Field r \<subseteq> dncl r (f \<alpha>')" using d2 e2 e1 b5 by force
      qed
      ultimately show "Field r \<subseteq> dncl r (f \<alpha>')" using d2 by blast
    qed
    then show "f \<in> \<N>10 r \<alpha>" unfolding \<N>10_def \<Q>_def by blast
  qed
  moreover have " \<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>10 r \<beta>) \<longrightarrow> f \<in> \<N>10 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>10 r
       \<beta>)"
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha>  \<longrightarrow>
      ((\<exists>y.  (f \<alpha>') - dncl r (\<LL> f \<alpha>') = {y}) \<longrightarrow> (Field r \<subseteq>
      dncl r (f \<alpha>')))"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha>" and d2: "\<exists>y. (f \<alpha>') - dncl r (\<LL> f \<alpha>') = {
      y}"
      then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>'))"
      proof
        assume "\<alpha>' <o \<alpha>"
        moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
        ultimately show "Field r \<subseteq> dncl r (f \<alpha>')" using c1 d1 d2 unfolding \<N>10_def \<Q>_def
      by blast
      qed
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> False"
      proof
        assume e1: "\<alpha>' =o \<alpha>"
        moreover then have e2: "\<LL> f \<alpha>' = \<LL> f \<alpha>" using lem_shrel_L_eq by blast
        ultimately have "\<exists>y. (f \<alpha>) - dncl r (\<LL> f \<alpha>) = {y}" using d2 b5 by metis
        moreover have "f \<alpha> \<subseteq> \<LL> f \<alpha>" using c2 unfolding \<LL>_def by blast
        ultimately show "False" unfolding dncl_def by blast
      qed
      ultimately show "Field r \<subseteq> dncl r (f \<alpha>')" using d2 by blast
    qed
    then show "f \<in> \<N>10 r \<alpha>" unfolding \<N>10_def \<Q>_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>10 r \<alpha>"] by blast
qed

lemma lem_Shinf_N11:
fixes r::"'U rel" and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<subseteq> F \<alpha> A"
    and a5: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>"
    and a10: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> A \<in> SF r \<longrightarrow>
            ((\<exists>y.  (F \<alpha> A) - dncl r A \<subseteq> {y}) \<longrightarrow> (Field r \<subseteq>
      dncl r (F \<alpha> A)))"
shows "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>11 r \<alpha>"
proof -
  have b2: "f {} = {}"
   and b3: "\<forall> \<alpha>0 \<alpha>::'U rel. (sc_ord \<alpha>0 \<alpha> \<longrightarrow> f \<alpha> = F \<
      alpha>0 (f \<alpha>0))"
   and b4: "\<forall> \<alpha>. (lm_ord \<alpha> \<longrightarrow> f \<alpha> = \<Union> { D. \<exists> \<beta>.
       \<beta> <o \<alpha> \<and> D = f \<beta> })"
   and b5: "\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>" using a0
      unfolding \<T>_def by blast+
  have "\<not> isSuccOrd ({}::'U rel)"
    using wo_rel_def wo_rel.isSuccOrd_def unfolding Field_def by force
  then have "f \<in> \<N>11 r {}" using lem_ord_subemp unfolding \<N>11_def by blast
  moreover have "\<forall>\<alpha>0 \<alpha>. sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>11 r \<alpha>0 \<
      longrightarrow> f \<in> \<N>11 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>0 \<alpha>::"'U rel"
    assume c1: "sc_ord \<alpha>0 \<alpha> \<and> f \<in> \<N>11 r \<alpha>0"
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (isSuccOrd \<alpha>') \<longrightarrow>
       (( f \<alpha>') - dncl r (\<LL> f \<alpha>') = {}) \<longrightarrow> (Field r \<subseteq> dncl r (f \<
      alpha>')))"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> (isSuccOrd \<alpha>')"
         and d2: "(f \<alpha>') - dncl r (\<LL> f \<alpha>') = {}"
      then have "\<alpha>0 <o \<alpha>' \<or> \<alpha>' \<le>o \<alpha>0" using c1 unfolding sc_ord_def
        using not_ordLeq_iff_ordLess ordLeq_Well_order_simp ordLess_Well_order_simp by blast
```

```
      moreover have "\<alpha>' \<le>o \<alpha>0 \<longrightarrow> (((f \<alpha>') - dncl r (\<LL> f \<alpha>') =
       {}) \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>')))"
        using d1 c1 unfolding \<N>11_def \<Q>_def by blast
      moreover have "\<alpha>0 <o \<alpha>' \<longrightarrow> \<alpha> =o \<alpha>'" using d1 c1 unfolding
      sc_ord_def using ordIso_iff_ordLeq by blast
      moreover have "\<alpha> =o \<alpha>' \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>'))"
      proof
        assume e1: "\<alpha> =o \<alpha>'"
        have "Well_order \<alpha>0" using c1 unfolding sc_ord_def ordLess_def by blast
        moreover then have "(f \<alpha>0) \<in> SF r"
          using a5 unfolding \<N>5_def using ordLeq_reflexive by blast
        moreover have "f \<alpha> = F \<alpha>0 (f \<alpha>0)" using c1 b3 by blast
        ultimately have e2: "(((f \<alpha>) - dncl r (f \<alpha>0) = {}) \<longrightarrow> (Field r \<subseteq>
      dncl r (f \<alpha>)))"
           using a10 by fastforce
        have "\<LL> f \<alpha> \<subseteq> f \<alpha>0"
        proof
          fix p
          assume "p \<in> \<LL> f \<alpha>"
          then obtain \<beta>::"'U rel" where "\<beta> <o \<alpha> \<and> p \<in> f \<beta>" unfolding \<LL>_def
       by blast
          moreover then have "\<beta> \<le>o \<alpha>0 \<and> \<alpha>0 \<le>o \<alpha>0" using c1 unfolding
      sc_ord_def
            using not_ordLess_iff_ordLeq ordLess_Well_order_simp by blast
          moreover then have "f \<in> \<N>1 r \<alpha>0" using a0 a1 lem_Shinf_N1[of f F] ordLeq_Well_order_simp
       by metis
          ultimately show "p \<in> f \<alpha>0" unfolding \<N>1_def by blast
        qed
        moreover have "f \<alpha>0 \<subseteq> \<LL> f \<alpha>" using c1 unfolding sc_ord_def \<LL>_def by
      blast
        ultimately have "\<LL> f \<alpha> = f \<alpha>0" by blast
        then have "\<LL> f \<alpha>' = f \<alpha>0" using e1 lem_shrel_L_eq by blast
        then show "Field r \<subseteq> dncl r (f \<alpha>')" using d2 e2 e1 b5 by force
      qed
      ultimately show "Field r \<subseteq> dncl r (f \<alpha>')" using d2 by blast
    qed
    then show "f \<in> \<N>11 r \<alpha>" unfolding \<N>11_def \<Q>_def by blast
  qed
  moreover have " \<forall>\<alpha>. lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<
      longrightarrow> f \<in> \<N>11 r \<beta>) \<longrightarrow> f \<in> \<N>11 r \<alpha>"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "lm_ord \<alpha> \<and> (\<forall>\<beta>. \<beta> <o \<alpha> \<longrightarrow> f \<in> \<N>11 r
       \<beta>)"
    then have c2: "f \<alpha> = \<Union> { D. \<exists> \<beta>. \<beta> <o \<alpha> \<and> D = f \<beta> }"
      using b4 by blast
    have "\<forall>\<alpha>'::'U rel. \<alpha>' \<le>o \<alpha> \<and> (isSuccOrd \<alpha>') \<longrightarrow>
      (((f \<alpha>') - dncl r (\<LL> f \<alpha>') = {}) \<longrightarrow> (Field r \<subseteq> dncl r (f \<
      alpha>')))"
    proof (intro allI impI)
      fix \<alpha>'::"'U rel"
      assume d1: "\<alpha>' \<le>o \<alpha> \<and> (isSuccOrd \<alpha>')"
         and d2: "(f \<alpha>') - dncl r (\<LL> f \<alpha>') = {}"
      then have "\<alpha>' <o \<alpha> \<or> \<alpha>' =o \<alpha>" using ordLeq_iff_ordLess_or_ordIso by blast
      moreover have "\<alpha>' <o \<alpha> \<longrightarrow> (Field r \<subseteq> dncl r (f \<alpha>'))"
      proof
        assume "\<alpha>' <o \<alpha>"
        moreover then have "\<alpha>' \<le>o \<alpha>'" using ordLess_Well_order_simp ordLeq_reflexive by blast
        ultimately show "Field r \<subseteq> dncl r (f \<alpha>')" using c1 d1 d2 unfolding \<N>11_def \<Q>_def
       by blast      qed
      moreover have "\<alpha>' =o \<alpha> \<longrightarrow> False"
      proof
        assume "\<alpha>' =o \<alpha>"
        moreover then have "\<alpha>' = {} \<or> isSuccOrd \<alpha>" using d1 lem_osucc_eq by blast
        moreover have "\<not> (\<alpha> = {} \<or> isSuccOrd \<alpha>)" using c1 unfolding lm_ord_def by blast
        ultimately have "\<alpha>' =o \<alpha> \<and> \<alpha>' = {} \<and> \<alpha> \<noteq> {}" by blast
        then show "False" by (metis iso_ozero_empty ordIso_symmetric ozero_def)
      qed
      ultimately show "Field r \<subseteq> dncl r (f \<alpha>')" using d2 by blast
    qed
    then show "f \<in> \<N>11 r \<alpha>" unfolding \<N>11_def \<Q>_def by blast
  qed
  ultimately show ?thesis using lem_sclm_ordind[of "\<lambda> \<alpha>. f \<in> \<N>11 r \<alpha>"] by blast
qed

lemma lem_Shinf_N12:
fixes r::"'U rel" and g::"'U rel \<Rightarrow> 'U"
  and F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" and f::"'U rel \<Rightarrow> 'U set"
```

```
assumes a0: "f \<in> \<T> F"
    and a1: "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>1 r \<alpha>"
    and a2: "\<forall> \<alpha> A. Well_order \<alpha> \<longrightarrow> g \<alpha> \<in> Field r \<
      longrightarrow> g \<alpha> \<in> F \<alpha> A"
    and a11: "\<omega>_ord \<le>o |Field r| \<longrightarrow> Field r = g ` { \<gamma>::'U rel. \<gamma> <o |
      Field r| }"
    and a2': "\<forall>\<alpha>::'U rel. \<omega>_ord \<le>o \<alpha> \<and> \<alpha> \<le>o |Field r| \<
      longrightarrow> \<omega>_ord \<le>o |g ` {\<gamma>. \<gamma> <o \<alpha>}|"
shows "f \<in> \<N>12 r |Field r|"
proof -
  have b1: "\<forall>\<alpha>. \<omega>_ord =o \<alpha> \<and> \<alpha> \<le>o |Field r| \<longrightarrow> \<
      omega>_ord \<le>o |\<LL> f \<alpha>|"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "\<omega>_ord =o \<alpha> \<and> \<alpha> \<le>o |Field r|"
    then have c2: "\<omega>_ord \<le>o |g`{\<gamma>. \<gamma> <o \<alpha>}|" using a2' ordIso_imp_ordLeq by
      blast
    have "g`{\<gamma>. \<gamma> <o \<alpha>} \<subseteq> g`{\<gamma>. \<gamma> <o |Field r|}" using c1
      ordLess_ordLeq_trans by force
    then have "g`{\<gamma>. \<gamma> <o \<alpha>} \<subseteq> Field r"
      using c1 a11 ordLeq_transitive ordIso_imp_ordLeq[of \<omega>_ord] by metis
    have "g`{\<gamma>. \<gamma> <o \<alpha>} \<subseteq> \<LL> f \<alpha>"
    proof
      fix a
      assume "a \<in> g`{\<gamma>. \<gamma> <o \<alpha>}"
      then obtain \<gamma> where d1: "a = g \<gamma> \<and> \<gamma> <o \<alpha>" by blast
      obtain \<gamma>' where d2: "sc_ord \<gamma> \<gamma>'" using d1 lem_sucord_ex by blast
      then have "f \<gamma>' = F \<gamma> (f \<gamma>)" using a0 unfolding \<T>_def by blast
      moreover have "Well_order \<gamma>" using d2 unfolding sc_ord_def using ordLess_def by blast
      moreover have "g \<gamma> \<in> Field r" using d1 c1 a11 ordIso_ordLeq_trans ordLess_ordLeq_trans by blast
      ultimately have "a \<in> f \<gamma>'" using d1 a2 by blast
      moreover have "\<gamma>' <o \<alpha>"
      proof -
        have "isLimOrd \<omega>_ord" by (simp add: Field_natLeq card_order_infinite_isLimOrd natLeq_card_order)
        then have "\<not> isSuccOrd \<alpha>"
          using c1 lem_osucc_eq ordIso_symmetric
          using natLeq_Well_order wo_rel.isLimOrd_def wo_rel_def by blast
        then obtain \<beta>::"'U rel" where "\<gamma> <o \<beta> \<and> \<not> (\<alpha> \<le>o \<beta>)" using
       d1 lem_ordint_sucord by blast
        then have "\<gamma> <o \<beta> \<and> \<beta> <o \<alpha>" using d1
          by (metis ordIso_imp_ordLeq ordLess_Well_order_simp ordLess_imp_ordLeq ordLess_or_ordIso)
        then show "\<gamma>' <o \<alpha>" using d2 unfolding sc_ord_def using ordLeq_ordLess_trans by blast
      qed
      ultimately show "a \<in> \<LL> f \<alpha>" unfolding \<LL>_def by blast
    qed
    then have "|g`{\<gamma>. \<gamma> <o \<alpha>}| \<le>o |\<LL> f \<alpha>|" by simp
    then show "\<omega>_ord \<le>o |\<LL> f \<alpha>|" using c2 ordLeq_transitive by blast
  qed
  have "\<forall>\<alpha>. \<omega>_ord \<le>o \<alpha> \<and> \<alpha> \<le>o |Field r| \<longrightarrow> \<
      omega>_ord \<le>o |\<LL> f \<alpha>|"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume "\<omega>_ord \<le>o \<alpha> \<and> \<alpha> \<le>o |Field r|"
    moreover then obtain \<alpha>0::"'U rel" where d1: "\<omega>_ord =o \<alpha>0 \<and> \<alpha>0 \<le>o \<
      alpha>"
      using internalize_ordLeq[of \<omega>_ord \<alpha>] by blast
    ultimately have "\<omega>_ord =o \<alpha>0 \<and> \<alpha>0 \<le>o |Field r|" using ordLeq_transitive by
      blast
    then have "\<omega>_ord \<le>o |\<LL> f \<alpha>0|" using b1 by blast
    moreover have "\<LL> f \<alpha>0 \<subseteq> \<LL> f \<alpha>" using d1 unfolding \<LL>_def using
      ordLess_ordLeq_trans by blast
    moreover then have "|\<LL> f \<alpha>0| \<le>o |\<LL> f \<alpha>|" by simp
    ultimately show "\<omega>_ord \<le>o |\<LL> f \<alpha>|" using ordLeq_transitive by blast
  qed
  then show ?thesis unfolding \<N>12_def by blast
qed

lemma lem_Shinf_E_ne:
fixes r::"'U rel" and a0::"'U" and A::"'U set" and Ps::"'U set set"
assumes  a2: "CCR r" and a3: "Ps \<subseteq> SCF r"
shows "\<E> r a0 A Ps \<noteq> {}"
proof (cases "A \<in> SF r")
  assume b0: "A \<in> SF r"
  show "\<E> r a0 A Ps \<noteq> {}"
  proof (cases "finite A")
    assume b1: "finite A"
    then obtain A' where "(a0 \<in> Field r \<longrightarrow> a0 \<in> A')" and b2: "A \<subseteq> A'" and b3: "
      CCR (Restr r A') \<and> finite A'"
```

```
                  and "(\<forall>a\<in>A. r``{a}\<subseteq>w_dncl r A \<or> r``{a}\<inter>(A'-w_dncl r A) \<
      noteq> {})"
                  and "A' \<in> SF r" and b4: "(\<exists>y. A' - dncl r A \<subseteq> {y}) \<longrightarrow>
      Field r \<subseteq> A' \<union> dncl r A"
                  and b5: "(\<exists> P. Ps = {P}) \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P \<
      in> SCF (Restr r A')))"
                   using b0 a2 a3
                   lem_Ccext_finsubccr_pext5_scf3[of r A Ps a0 "w_dncl r A" "dncl r A"]
                   by metis
    moreover have "|A'| <o \<omega>_ord" using b3 finite_iff_ordLess_natLeq by blast
    moreover have "\<not> ( \<omega>_ord \<le>o |A| )" using b1 infinite_iff_natLeq_ordLeq by blast
    moreover have "(\<exists>y. A' - dncl r A \<subseteq> {y}) \<longrightarrow> Field r \<subseteq> dncl r A' "
       using b2 b4 unfolding dncl_def by blast
    moreover have "(\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| ) \<longrightarrow>
      (\<exists> P. Ps = {P})"
      using b1 card_of_ordLeq_finite by blast
    ultimately have "A' \<in> \<E> r a0 A Ps" unfolding \<E>_def \<E>p_def by fast
    then show ?thesis by blast
  next
    assume b1: "\<not> finite A"
    then obtain A' where b2: "(a0 \<in> Field r \<longrightarrow> a0 \<in> A')" and b3: "A \<subseteq> A'" and
      b4: "CCR (Restr r A')"
                  and b5: "|A'| =o |A|" and b6: "(\<forall>a\<in>A. r``{a}\<subseteq>w_dncl r A \<or> r``{a
      }\<inter>(A'-w_dncl r A) \<noteq> {})"
                  and b7: "A' \<in> SF r" and b8: "(\<exists>y. A' - dncl r A \<subseteq> {y}) \<
      longrightarrow> Field r \<subseteq> A' \<union> dncl r A"
                  and b9: "( |Ps| \<le>o |A| \<longrightarrow> (\<forall> P \<in> Ps. (A' \<inter> P) \<in>
      SCF (Restr r A')) )"
                  and b10: "escl r A A' \<subseteq> A'" and b11: "clterm (Restr r A') r"
       using b0 a2 a3
            lem_Ccext_infsubccr_pext5_scf3[of r A Ps a0 "w_dncl r A" "dncl r A"] by metis
    then have "(\<omega>_ord \<le>o |A| \<longrightarrow> |A'| \<le>o |A| )" using ordIso_iff_ordLeq by blast
    moreover have "( |A| <o \<omega>_ord \<longrightarrow> |A'| <o \<omega>_ord)" using b1
      finite_iff_ordLess_natLeq by blast
    moreover have "(\<exists>y. A' - dncl r A \<subseteq> {y}) \<longrightarrow> (Field r \<subseteq> dncl r A')
      " using b3 b8 unfolding dncl_def by blast
    moreover have "(\<exists> P. Ps = {P}) \<or> ((\<not> finite Ps) \<and> |Ps| \<le>o |A| ) \<longrightarrow>
      |Ps| \<le>o |A|"
      using b1 by (metis card_of_singl_ordLeq finite.simps)
    ultimately have "A' \<in> \<E> r a0 A Ps" unfolding \<E>_def \<E>p_def
      using b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 by fast
    then show ?thesis by blast
  qed
next
  assume "A \<notin> SF r"
  moreover obtain A' where b1: "A' = A \<union> {a0}" by blast
  moreover then have "|A| <o \<omega>_ord \<longrightarrow> |A'| <o \<omega>_ord" using
      finite_iff_ordLess_natLeq by blast
  moreover have "\<omega>_ord \<le>o |A| \<longrightarrow> |A'| \<le>o |A|"
  proof
    assume "\<omega>_ord \<le>o |A|"
    then have "\<not> finite A" using finite_iff_ordLess_natLeq not_ordLeq_ordLess by blast
    then have "|A'| =o |A|" unfolding b1 using infinite_card_of_insert by simp
    then show "|A'| \<le>o |A|" using ordIso_imp_ordLeq by blast
  qed
  ultimately have "A' \<in> \<E> r a0 A Ps" unfolding \<E>_def by blast
  then show "\<E> r a0 A Ps \<noteq> {}" by blast
qed

lemma lem_oseq_fin_inj:
fixes g::"'U rel \<Rightarrow> 'a" and I::"'U rel \<Rightarrow> 'U rel set" and A::"'a set"
assumes a1: "I = (\<lambda> \<alpha>'. { \<alpha>::'U rel. \<alpha> <o \<alpha>' })"
    and a2: "\<omega>_ord \<le>o |A|"
    and a3: "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> g \<alpha> = g \<beta>"
shows "\<exists> h. (\<forall> \<alpha>'. g`(I \<alpha>') \<subseteq> h`(I \<alpha>') \<and> h`(I \<alpha>') \<
      subseteq> g`(I \<alpha>') \<union> A)
          \<and> (\<forall> \<alpha>'. \<omega>_ord \<le>o \<alpha>' \<longrightarrow> \<omega>_ord \<le>o |h`(I
       \<alpha>')| )
          \<and> (\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> h \<alpha> = h \<beta>)"
proof(cases "\<exists> \<alpha>::'U rel. \<omega>_ord \<le>o \<alpha>")
  assume "\<exists> \<alpha>::'U rel. \<omega>_ord \<le>o \<alpha>"
  then obtain \<alpha>m::"'U rel" where b1: "\<omega>_ord =o \<alpha>m" by (metis internalize_ordLeq)
  obtain f::"nat \<Rightarrow> 'U rel" where b2: "f = (\<lambda> n. SOME \<alpha>. \<alpha> =o (natLeq_on n))"
      by blast
  have "|UNIV::nat set| \<le>o |A|" using a2 using card_of_nat ordIso_imp_ordLeq ordLeq_transitive by blast
  then obtain xi::"nat \<Rightarrow> 'a" where b3: "inj xi \<and> xi ` UNIV \<subseteq> A" by (meson
      card_of_ordLeq)
```

```
obtain yi where b4: "yi = (\<lambda> n. if (\<exists> i<n. g (f n) = g (f i)) then (xi n) else (g (f n)))" by
    blast
obtain h where b5: "h = (\<lambda> \<alpha>. if (\<exists> n. \<alpha> =o f n) then (yi (SOME n. (\<alpha> =o
    f n))) else (g \<alpha>))" by blast
have b6: "\<And> n::nat. f n =o (natLeq_on n)"
proof -
  fix n
  have "natLeq_on n <o \<alpha>m" using b1 natLeq_on_ordLess_natLeq ordLess_ordIso_trans by blast
  then obtain \<alpha>::"'U rel" where "\<alpha> =o (natLeq_on n)"
    using internalize_ordLess ordIso_symmetric by fastforce
  then show "f n =o natLeq_on n" using b2 someI_ex[of "\<lambda>\<alpha>::'U rel. \<alpha> =o (natLeq_on n)"]
    by blast
qed
then have b7: "\<And> n m. n \<le> m \<Longrightarrow> f n \<le>o f m"
  by (metis (no_types, lifting) natLeq_on_ordLeq_less_eq ordIso_imp_ordLeq ordIso_symmetric ordLeq_transitive)
have b8: "\<And> n m. f n =o f m \<Longrightarrow> n = m"
proof -
  fix n m
  assume "f n =o f m"
  moreover then have "natLeq_on n =o f m" using b6 ordIso_transitive ordIso_symmetric by blast
  ultimately have "natLeq_on n =o natLeq_on m" using b6 ordIso_transitive by blast
  then show "n = m" using natLeq_on_injective_ordIso by blast
qed
have b9: "\<And> \<alpha> n. \<alpha> =o f n \<Longrightarrow> h \<alpha> = yi n"
proof -
  fix \<alpha>::"'U rel" and n::nat
  assume "\<alpha> =o f n"
  moreover obtain m where "m = (SOME n. (\<alpha> =o f n))" by blast
  ultimately have "h \<alpha> = yi m \<and> \<alpha> =o f m \<and> \<alpha> =o f n" using b5 someI_ex[of "\<
    lambda> n. \<alpha> =o f n"] by fastforce
  moreover then have "m = n" using b8 ordIso_transitive ordIso_symmetric by blast
  ultimately show "h \<alpha> = yi n" by blast
qed
have b10: "\<And> n. yi`{k. k \<le> n} \<subseteq> g`(f`({k. k \<le> n})) \<union> A"
proof -
  fix n0
  show "yi`{k. k \<le> n0} \<subseteq> g`(f`({k. k \<le> n0})) \<union> A"
  proof (induct n0)
    show "yi`{k. k \<le> 0} \<subseteq> g`(f`{k. k \<le> 0}) \<union> A" using b4 by simp
  next
    fix n
    assume d1: "yi`{k. k \<le> n} \<subseteq> g`(f`({k. k \<le> n})) \<union> A"
    show "yi`{k. k \<le> Suc n} \<subseteq> g`(f`({k. k \<le> (Suc n)})) \<union> A"
    proof (cases "\<exists> i<Suc n. g (f (Suc n)) = g (f i)")
      assume "\<exists> i<Suc n. g (f (Suc n)) = g (f i)"
      then obtain i where "i<Suc n \<and> g (f (Suc n)) = g (f i)" by blast
      then have "i \<le> n \<and> yi (Suc n) = xi (Suc n)" using b4 by force
      then have "yi (Suc n) \<in> g`(f`({k. k \<le> Suc n})) \<union> A" using b3 by blast
      moreover have "yi`{k. k \<le> n} \<subseteq> g`(f`({k. k \<le> Suc n})) \<union> A" using d1 by
    fastforce
      moreover have "\<And> k. k \<le> Suc n \<longleftrightarrow> (k \<le>n \<or> k = Suc n)" by linarith
      moreover then have "yi`{k. k \<le> Suc n} = yi`{k. k \<le> n} \<union> {yi (Suc n)}" by fastforce
      ultimately show ?thesis by blast
    next
      assume "\<not> (\<exists> i<Suc n. g (f (Suc n)) = g (f i))"
      then have "yi (Suc n) = g (f (Suc n))" using b4 by force
      then have "yi (Suc n) \<in> g`(f`({k. k \<le> Suc n})) \<union> A" by blast
      moreover have "yi`{k. k \<le> n} \<subseteq> g`(f`({k. k \<le> Suc n})) \<union> A" using d1 by
    fastforce
      moreover have "\<And> k. k \<le> Suc n \<longleftrightarrow> (k \<le>n \<or> k = Suc n)" by linarith
      moreover then have "yi`{k. k \<le> Suc n} = yi`{k. k \<le> n} \<union> {yi (Suc n)}" by fastforce
      ultimately show ?thesis by blast
    qed
  qed
qed
have "\<forall> \<alpha>'. g`(I \<alpha>') \<subseteq> h`(I \<alpha>') \<and> h`(I \<alpha>') \<subseteq> g`(I
     \<alpha>') \<union> A"
proof
  fix \<alpha>'::"'U rel"
  have "g`(I \<alpha>') \<subseteq> h`(I \<alpha>')"
  proof
    fix a
    assume "a \<in> g`(I \<alpha>')"
    then obtain \<beta> where d1: "\<beta> <o \<alpha>' \<and> a = g \<beta>" using a1 by blast
    show "a \<in> h`(I \<alpha>')"
    proof (cases "\<exists> n. \<beta> =o f n")
      assume "\<exists> n. \<beta> =o f n"
      then obtain n where e1: "\<beta> =o f n" by blast
```

```
      then have e2: "a = g (f n) \<and> h \<beta> = yi n" using d1 b9 a3 by blast
      obtain P where e3: "P = (\<lambda> i. i\<le>n \<and> g (f n) = g (f i))" by blast
      obtain k where "k = (LEAST i. P i)" by blast
      moreover have "P n" using e3 by blast
      ultimately have "P k \<and> (\<forall> i. P i \<longrightarrow> k \<le> i)" using LeastI Least_le by
    metis
      then have "k \<le> n \<and> g (f n) = g (f k) \<and> \<not> (\<exists> i<k. g (f k) = g (f i))"
        using e3 by (metis leD less_le_trans less_or_eq_imp_le)
      then have "a = yi k \<and> f k \<le>o f n" using e2 b4 b7 by fastforce
      moreover then have "f k <o \<alpha>'"
        using e1 d1 by (metis ordIso_symmetric ordLeq_ordIso_trans ordLeq_ordLess_trans)
      ultimately have "f k \<in> I \<alpha>' \<and> h (f k) = a" using a1 b7 b9 ordIso_iff_ordLeq by blast
      then show ?thesis by blast
    next
      assume "\<not> (\<exists> n. \<beta> =o f n)"
      then have "h \<beta> = g \<beta>" using b5 by simp
      then show ?thesis using d1 a1 by force
    qed
  qed
  moreover have "h`(I \<alpha>') \<subseteq> g`(I \<alpha>') \<union> A"
  proof
    fix a
    assume "a \<in> h`(I \<alpha>')"
    then obtain \<beta> where d1: "\<beta> <o \<alpha>' \<and> a = h \<beta>" using a1 by blast
    show "a \<in> g`(I \<alpha>') \<union> A"
    proof (cases "\<exists> n. \<beta> =o f n")
      assume "\<exists> n. \<beta> =o f n"
      then obtain n where e1: "\<beta> =o f n" by blast
      then have "a = yi n" using d1 b9 by blast
      then have "a \<in> g`(f`({k. k \<le> n})) \<union> A" using b10 by blast
      moreover have "\<forall> k. k \<le> n \<longrightarrow> f k \<in> I \<alpha>'"
      proof (intro allI impI)
        fix k
        assume "k \<le> n"
        then have "f k \<le>o f n" using b7 by blast
        then show "f k \<in> I \<alpha>'" using e1 a1 d1
          using ordIso_symmetric ordLeq_ordIso_trans ordLeq_ordLess_trans by fastforce
      qed
      ultimately show ?thesis by blast
    next
      assume "\<not> (\<exists> n. \<beta> =o f n)"
      then show ?thesis using d1 a1 b5 by force
    qed
  qed
  ultimately show "g`(I \<alpha>') \<subseteq> h`(I \<alpha>') \<and> h`(I \<alpha>') \<subseteq> g`(I \<alpha
    >') \<union> A" by blast
qed
moreover have "\<forall> \<alpha>'. \<omega>_ord \<le>o \<alpha>' \<longrightarrow> \<omega>_ord \<le>o |h`(I
    \<alpha>')|"
proof (intro allI impI)
  fix \<alpha>'::"'U rel"
  assume "\<omega>_ord \<le>o \<alpha>'"
  then have "I \<alpha>m \<subseteq> I \<alpha>'"
    using a1 b1 by (smt mem_Collect_eq not_ordLess_ordIso ordIso_symmetric
        ordLeq_iff_ordLess_or_ordIso ordLeq_ordLess_trans ordLeq_transitive subsetI)
  moreover have "f`UNIV \<subseteq> I \<alpha>m" using b1 a1
    using b6 natLeq_on_ordLess_natLeq ordIso_ordLess_trans ordLess_ordIso_trans by fastforce
  ultimately have "h`(f`UNIV) \<subseteq> h`(I \<alpha>')" by blast
  then have "|h`(f`UNIV)| \<le>o |h`(I \<alpha>')|" by simp
  moreover have "\<omega>_ord \<le>o |h`(f`UNIV)|"
  proof -
    have "\<forall> n. h (f n) = yi n" using b7 b9 ordIso_iff_ordLeq by blast
    then have "yi`UNIV \<subseteq> h`(f`UNIV)" by (smt imageE image_eqI subset_eq)
    then have "|yi`UNIV| \<le>o |h`(f`UNIV)|" by simp
    moreover have "\<omega>_ord \<le>o |yi`UNIV|"
    proof (cases "finite (g`(f`UNIV))")
      assume e1: "finite(g`(f`UNIV))"
      obtain J where e3: "J = {n. \<exists>i<n. g (f n) = g (f i)}" by blast
      have "(\<forall> m. \<exists> n>m. n \<notin> J) \<longrightarrow> False"
      proof
        assume f1: "\<forall> m. \<exists> n>m. n \<notin> J"
        obtain w where f2: "w = (\<lambda> m. SOME n. n>m \<and> n \<notin> J)" by blast
        have f3: "\<forall> m. w m > m \<and> w m \<notin> J"
        proof
          fix m
          show "w m > m \<and> w m \<notin> J" using f1 f2 someI_ex[of "\<lambda> n. n>m \<and> n \<notin> J"]
     by metis
        qed
```

```
    obtain p where f4: "p = (\<lambda> k::nat. (w^^k) 0)" by blast
    have f5: "\<forall> k. k \<noteq> 0 \<longrightarrow> p k \<notin> J"
    proof
      fix k
      show "k \<noteq> 0 \<longrightarrow> p k \<notin> J"
      proof (induct k)
        show "0 \<noteq> 0 \<longrightarrow> p 0 \<notin> J" by blast
      next
        fix k
        assume "k \<noteq> 0 \<longrightarrow> p k \<notin> J"
        show "Suc k \<noteq> 0 \<longrightarrow> p (Suc k) \<notin> J" using f3 f4 by simp
      qed
    qed
    have "\<forall> j. \<forall> i<j. p i < p j"
    proof
      fix j
      show "\<forall>i<j. p i < p j"
      proof (induct j)
        show "\<forall>i<0. p i < p 0" by blast
      next
        fix j
        assume "\<forall>i<j. p i < p j"
        moreover have "p j < p (Suc j)" using f3 f4 by force
        ultimately show "\<forall>i<Suc j. p i < p (Suc j)" by (metis less_antisym less_trans)
      qed
    qed
    then have "inj p" unfolding inj_on_def by (metis nat_neq_iff)
    then have "\<not> finite (p`UNIV)" using finite_imageD by blast
    moreover obtain P where f6: "P = p`{k. k \<noteq> 0}" by blast
    moreover have "UNIV = {0} \<union> {k::nat. k \<noteq> 0}" by blast
    moreover then have "p`UNIV = p`{0} \<union> P \<and> finite (p`{0})" using f6 by fastforce
    ultimately have f7: "\<not> finite P" using finite_UnI by metis
    have "\<forall> n \<in> P. \<forall> m \<in> P. g (f n) = g (f m) \<longrightarrow> n = m"
    proof (intro ballI impI)
      fix n m
      assume g1: "n \<in> P" and g2: "m \<in> P" and g3: "g (f n) = g (f m)"
      have "n < m \<longrightarrow> False"
      proof
        assume "n < m"
        moreover then have "m \<notin> J" using g2 f5 f6 by blast
        ultimately show "False" using g3 e3 by force
      qed
      moreover have "m < n \<longrightarrow> False"
      proof
        assume "m < n"
        moreover then have "n \<notin> J" using g1 f5 f6 by blast
        ultimately show "False" using g3 e3 by force
      qed
      ultimately show "n = m" by force
    qed
    then have "inj_on (g \<circ> f) P" unfolding inj_on_def by simp
    then have "\<not> finite ((g \<circ> f)`UNIV)" using f7
      by (metis finite_imageD infinite_iff_countable_subset subset_UNIV subset_image_iff)
    moreover have "(g \<circ> f)`UNIV = g`(f`UNIV)" by force
    ultimately show "False" using e1 by simp
  qed
  then obtain m where "\<forall> n>m. n \<in> J" by blast
  then have "\<forall> n>m. yi n = xi n" using e3 b4 by force
  then have e4: "xi`{n. n>m} \<subseteq> yi`UNIV" by (metis image_Collect_subsetI rangeI)
  have e5: "|xi`{n. n>m}| =o |{n. n>m}|" using b3 by (metis card_of_image image_inv_f_f ordIso_iff_ordLeq)
  have "finite {n. n\<le>m} \<and> (\<not> finite (UNIV::nat set)) \<and> {n. n\<le>m} \<union> {n. n>m} =
 UNIV" by force
  then have "\<not> finite {n. n>m}" using finite_UnI by metis
  then have "|xi`{n. n>m}| =o \<omega>_ord" using e5 by (meson card_of_UNIV card_of_nat
    finite_iff_cardOf_nat ordIso_transitive ordLeq_iff_ordLess_or_ordIso)
  then show ?thesis using e4
    by (metis finite_subset infinite_iff_natLeq_ordLeq ordIso_natLeq_infinite1)
next
  assume "\<not> finite (g`(f`UNIV))"
  moreover have "g`(f`UNIV) \<subseteq> yi`UNIV"
  proof
    fix a
    assume "a \<in> g`(f`UNIV)"
    then obtain n where e1: "a = g (f n)" by blast
    obtain P where e3: "P = (\<lambda> i. i\<le>n \<and> g (f n) = g (f i))" by blast
    obtain k where "k = (LEAST i. P i)" by blast
    moreover have "P n" using e3 by blast
```

```
        ultimately have "P k \<and> (\<forall> i. P i \<longrightarrow> k \<le> i)" using LeastI Least_le by
      metis
        then have "g (f n) = g (f k) \<and> \<not> (\<exists> i<k. g (f k) = g (f i))"
          using e3 by (metis leD less_le_trans less_or_eq_imp_le)
        then have "yi k = a" using e1 b4 b7 by fastforce
        then show "a \<in> yi`UNIV" by blast
      qed
      ultimately have "\<not> finite (yi`UNIV)" using finite_subset by metis
      then show ?thesis using infinite_iff_natLeq_ordLeq by blast
    qed
    ultimately show ?thesis using ordLeq_transitive by blast
  qed
  ultimately show "\<omega>_ord \<le>o |h`(I \<alpha>')|" using ordLeq_transitive by blast
qed
moreover have "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> h \<alpha> = h \<beta>"
proof (intro allI impI)
  fix \<alpha>::"'U rel" and \<beta>::"'U rel"
  assume c1: "\<alpha> =o \<beta>"
  show "h \<alpha> = h \<beta>"
  proof (cases "\<exists> n. \<alpha> =o f n")
    assume "\<exists> n. \<alpha> =o f n"
    moreover then have "\<exists> n. \<beta> =o f n" using c1 ordIso_transitive ordIso_symmetric by metis
    moreover have "\<forall> n. (\<alpha> =o f n) = (\<beta> =o f n)" using c1 ordIso_transitive
     ordIso_symmetric by metis
    ultimately show "h \<alpha> = h \<beta>" using b5 by simp
  next
    assume "\<not> (\<exists> n. \<alpha> =o f n)"
    moreover then have "\<not> (\<exists> n. \<beta> =o f n)" using c1 ordIso_transitive by metis
    ultimately show "h \<alpha> = h \<beta>" using b5 c1 a3 by simp
  qed
qed
ultimately show ?thesis by blast
next
  assume "\<not> (\<exists> \<alpha>::'U rel. \<omega>_ord \<le>o \<alpha>)"
  then show ?thesis using a3 by blast
qed

lemma lem_Shinf_N_ne:
fixes r::"'U rel" and Ps::"'U set set"
assumes "CCR r" and "Ps \<subseteq> SCF r"
shows "\<N> r Ps \<noteq> {}"
proof -
  obtain E :: "'U \<Rightarrow> 'U set \<Rightarrow> 'U set" where "E = (\<lambda> a A. SOME A'. A' \<in> \<E> r
       a A Ps)" by blast
  moreover have "\<forall> a A. \<exists> A'. A' \<in> \<E> r a A Ps" using assms lem_Shinf_E_ne[of r Ps] by
      blast
  ultimately have b1: "\<forall> a A. E a A \<in> \<E> r a A Ps" by (meson someI_ex)
  have "\<exists> g::'U rel \<Rightarrow> 'U. (\<omega>_ord \<le>o |Field r| \<longrightarrow> Field r = g ` {\<
      gamma>. \<gamma> <o |Field r|}) \<and>
        (\<forall>\<alpha>'::'U rel. \<omega>_ord \<le>o \<alpha>' \<and> \<alpha>' \<le>o |Field r| \<
      longrightarrow> \<omega>_ord \<le>o |g ` {\<gamma>. \<gamma> <o \<alpha>'}| ) \<and>
        (\<forall>\<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> g \<alpha> = g \<beta>)"
  proof(cases "\<omega>_ord \<le>o |Field r|")
    assume c1: "\<omega>_ord \<le>o |Field r|"
    moreover have "Card_order |Field r| \<and> |Field r| \<le>o |Field r|" by simp
    ultimately obtain g0::"'U rel \<Rightarrow> 'U" where
           c2: "Field r \<subseteq> g0 ` {\<gamma>. \<gamma> <o |Field r| }"
        and c3: "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> g0 \<alpha> = g0 \<beta>"
        using c1 lem_card_setcv_inf_stab[of "|Field r|" "Field r"] by blast
    have "Field r \<noteq> {}" using c1 by (metis finite.emptyI infinite_iff_natLeq_ordLeq)
    then obtain a0 where "a0 \<in> Field r" by blast
    moreover obtain t where "t = (\<lambda> a. if (a \<in> Field r) then a else a0)" by blast
    moreover obtain g1 where "g1 = (\<lambda> \<alpha>. t (g0 \<alpha>))" by blast
    ultimately have c4: "Field r \<subseteq> g1`{\<gamma> . \<gamma> <o |Field r| }"
               and c5: "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> g1 \<alpha> = g1 \<
      beta>" and c6: "g1`UNIV \<subseteq> Field r" using c2 c3 by force+
    obtain I where c7: "I = (\<lambda>\<alpha>'::'U rel. {\<alpha>::'U rel. \<alpha> <o \<alpha>'})" by blast
    then obtain g where c8: "(\<forall> \<alpha>'. g1`(I \<alpha>') \<subseteq> g`(I \<alpha>') \<and> g`(I \<
      alpha>') \<subseteq> g1`(I \<alpha>') \<union> (Field r))"
        and c9: "\<forall> \<alpha>'. \<omega>_ord \<le>o \<alpha>' \<longrightarrow> \<omega>_ord \<le>o |g`(
      I \<alpha>')|"
        and c10: "(\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> g \<alpha> = g \<beta>)"
      using c1 c5 lem_oseq_fin_inj[of I "Field r" g1] by blast
    have "g1`(I |Field r| ) \<subseteq> Field r" using c6 by blast
    then have "g ` { \<gamma>. \<gamma> <o |Field r| } \<subseteq> Field r" using c7 c8 by blast
    moreover have "Field r \<subseteq> g`{ \<gamma>. \<gamma> <o |Field r| }" using c4 c7 c8 by force
    ultimately have "\<omega>_ord \<le>o |Field r| \<longrightarrow> Field r = g`{ \<gamma>. \<gamma> <o |Field
      r| }" by blast
```

```
    then show ?thesis using c7 c9 c10 by blast
  next
    assume "\<not> \<omega>_ord \<le>o |Field r|"
    moreover then have "\<forall>\<alpha>'::'U rel. \<not> (\<omega>_ord \<le>o \<alpha>' \<and> \<alpha>' \<le>
      o |Field r| )" using ordLeq_transitive by blast
    moreover have "\<exists> g::'U rel \<Rightarrow> 'U. (\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<
      longrightarrow> g \<alpha> = g \<beta>)" by force
    ultimately show ?thesis by blast
  qed
  then obtain g::"'U rel \<Rightarrow> 'U" where
         b4: "\<omega>_ord \<le>o |Field r| \<longrightarrow> Field r = g ` { \<gamma>::'U rel. \<gamma> <o |
      Field r| }"
     and b4': "\<forall>\<alpha>'::'U rel. \<omega>_ord \<le>o \<alpha>' \<and> \<alpha>' \<le>o |Field r| \<
      longrightarrow> \<omega>_ord \<le>o |g ` {\<gamma>. \<gamma> <o \<alpha>'}|"
     and b5: "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> g \<alpha> = g \<beta>" by blast
  obtain F::"'U rel \<Rightarrow> 'U set \<Rightarrow> 'U set" where b6: "F = (\<lambda> \<alpha> A. E (g \<
      alpha>) A)" by blast
  then have "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> F \<alpha> = F \<beta>" using b5
      by fastforce
  then obtain f::"'U rel \<Rightarrow> 'U set" where b7: "f \<in> \<T> F"
    unfolding \<T>_def using lem_ordseq_rec_sets[of F "{}"] by clarsimp
  have b8: "Well_order |Field r|" by simp
  have "\<N> r Ps \<noteq> {}"
  proof -
    have c0: "\<forall> \<alpha> A. A \<in> SF r \<longrightarrow> F \<alpha> A \<in> SF r" using b6 b1
      unfolding \<E>_def by simp
    have c1: "\<forall> \<alpha> A. A \<subseteq> F \<alpha> A" using b6 b1 unfolding \<E>_def by simp
    have c2: "\<forall> \<alpha> A. (g \<alpha> \<in> Field r \<longrightarrow> g \<alpha> \<in> F \<alpha> A)"
      using b6 b1 unfolding \<E>_def by blast
    have c3: "\<forall> \<alpha> A. A \<in> SF r \<longrightarrow> \<omega>_ord \<le>o |A| \<longrightarrow>
      escl r A (F \<alpha> A) \<subseteq> (F \<alpha> A) \<and> clterm (Restr r (F \<alpha> A)) r"
      using b6 b1 unfolding \<E>_def by blast
    have c4: "\<forall> \<alpha> A. A \<in> SF r \<longrightarrow>
                ( \<forall> a\<in>A. r `` {a} \<subseteq> w_dncl r A \<or> r `` {a} \<inter> (F \<alpha> A -
      w_dncl r A) \<noteq> {} )"
      using b6 b1 unfolding \<E>_def by blast
    have c6: "\<forall> \<alpha> A. A \<in> SF r \<longrightarrow> CCR (Restr r (F \<alpha> A))"
              using b6 b1 unfolding \<E>_def by blast
    have c7: "\<forall> \<alpha> A. ( |A| <o \<omega>_ord \<longrightarrow> |F \<alpha> A| <o \<omega>_ord) \<
      and> ( \<omega>_ord \<le>o |A| \<longrightarrow> |F \<alpha> A| \<le>o |A| )"
              using b6 b1 unfolding \<E>_def by blast
    have c8: "\<forall> \<alpha> A. A \<in> SF r \<longrightarrow> \<E>p r Ps A (F \<alpha> A)" using b6 b1
      unfolding \<E>_def \<E>p_def by blast
    have c10: "\<forall> \<alpha> A. A \<in> SF r \<longrightarrow> ((\<exists>y.  (F \<alpha> A) - dncl r A \<
      subseteq> {y}) \<longrightarrow> (Field r \<subseteq> dncl r (F \<alpha> A)))"
      using b6 b1 unfolding \<E>_def by blast
    have c1': "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>1 r \<alpha>" using b7 b8 c1
       lem_Shinf_N1[of f F r] by blast
    have c5': "\<forall>\<alpha>. Well_order \<alpha> \<longrightarrow> f \<in> \<N>5 r \<alpha>" using b7 b8 c0
       lem_Shinf_N5[of f F r] by blast
    have "f \<in> \<N>1 r |Field r|" using b7 b8 c1 lem_Shinf_N1[of f F r] by blast
    moreover have "f \<in> \<N>2 r |Field r|" using b7 b8 lem_Shinf_N2[of f F r] by blast
    moreover have "f \<in> \<N>3 r |Field r|" using b7 b8 c1 c3 c5' lem_Shinf_N3[of f F r] by blast
    moreover have "f \<in> \<N>4 r |Field r|" using b7 b8 c1 c4 c5' lem_Shinf_N4[of f F r] by blast
    moreover have "f \<in> \<N>5 r |Field r|" using b7 b8 c0 lem_Shinf_N5[of f F r] by blast
    moreover have "f \<in> \<N>6 r |Field r|" using b7 b8 c1 c6 c5' lem_Shinf_N6[of f F r] by blast
    moreover have "f \<in> \<N>7 r |Field r|" using b7 b8 c1 c7 lem_Shinf_N7[of f F r] by blast
    moreover have "f \<in> \<N>8 r Ps |Field r|" using b7 b8 c1 c7 c8 c5' lem_Shinf_N8[of f F r Ps] by blast
    moreover have "f \<in> \<N>9 r |Field r|" using b7 b4 c1 c2 lem_Shinf_N9[of f F g r] by blast
    moreover have "f \<in> \<N>10 r |Field r|" using b7 b8 c1 c10 c5' lem_Shinf_N10[of f F r] by metis
    moreover have "f \<in> \<N>11 r |Field r|" using b7 b8 c1 c10 c5' lem_Shinf_N11[of f F r] by metis
    moreover have "f \<in> \<N>12 r |Field r|" using b7 c1' c2 b4 b4' lem_Shinf_N12[of f F r g] by blast
    moreover have "\<forall> \<alpha> \<beta>. \<alpha> =o \<beta> \<longrightarrow> f \<alpha> = f \<beta>"
      using b7 unfolding \<T>_def by blast
    ultimately show ?thesis unfolding \<N>_def by blast
  qed
  then show ?thesis by blast
qed

lemma lem_wrankrel_eq: "wrank_rel r A0 \<alpha> \<Longrightarrow> \<alpha> =o \<beta> \<Longrightarrow>
      wrank_rel r A0 \<beta>"
proof -
  assume a1: "wrank_rel r A0 \<alpha>" and a2: "\<alpha> =o \<beta>"
  then obtain B where "B \<in> wbase r A0 \<and> |B| =o \<alpha> \<and> ( \<forall> B' \<in> wbase r A0. |B| \<
      le>o |B'| )"  unfolding wrank_rel_def by blast
  moreover then have "|B| =o \<beta>" using a2 by (metis ordIso_transitive)
  ultimately show "wrank_rel r A0 \<beta>" unfolding wrank_rel_def by blast
qed
```

```
lemma lem_wrank_wrankrel:
fixes r::"'U rel" and A0::"'U set"
shows "wrank_rel r A0 (wrank r A0)"
proof -
  have b1: "wbase r A0 \<noteq> {}" using lem_wdn_range_lb[of A0 r] unfolding wbase_def by blast
  obtain Q where b2: "Q = { \<alpha>::'U rel. \<exists> A \<in> wbase r A0. \<alpha> =o |A| }" by blast
  have b3: "\<forall> A \<in> wbase r A0. \<exists> \<alpha> \<in> Q. \<alpha> \<le>o |A|"
  proof
    fix A
    assume "A \<in> wbase r A0"
    then have "|A| \<in> Q \<and> |A| =o |A|" using b2 ordIso_symmetric by force
    then show "\<exists> \<alpha> \<in> Q. \<alpha> \<le>o |A|" using ordIso_iff_ordLeq by blast
  qed
  then have "Q \<noteq> {}" using b1 by blast
  then obtain \<alpha> where b4: "\<alpha> \<in> Q \<and> (\<forall>\<alpha>'. \<alpha>' <o \<alpha> \<
      longrightarrow> \<alpha>' \<notin> Q)" using wf_ordLess wf_eq_minimal[of "ordLess"] by blast
  moreover have "\<forall> \<alpha>' \<in> Q. Card_order \<alpha>'" using b2 using ordIso_card_of_imp_Card_order
       by blast
  ultimately have "\<forall> \<alpha>' \<in> Q. \<not> (\<alpha>' <o \<alpha>) \<longrightarrow> \<alpha> \<le>o
       \<alpha>'" by simp
  then have b5: "\<alpha> \<in> Q \<and> (\<forall> \<alpha>' \<in> Q. \<alpha> \<le>o \<alpha>')" using b4 by
      blast
  then obtain A where b6: "A \<in> wbase r A0 \<and> |A| =o \<alpha>" using b2 ordIso_symmetric by blast
  moreover have "\<forall> B\<in>wbase r A0. |A| \<le>o |B|"
  proof
    fix B
    assume "B \<in> wbase r A0"
    then obtain \<alpha>' where "\<alpha>' \<in> Q \<and> \<alpha>' \<le>o |B|" using b3 by blast
    moreover then have "|A| =o \<alpha> \<and> \<alpha> \<le>o \<alpha>'" using b5 b6 by blast
    ultimately show "|A| \<le>o |B|" using ordIso_ordLeq_trans ordLeq_transitive by blast
  qed
  ultimately have "wrank_rel r A0 \<alpha>" unfolding wrank_rel_def by blast
  then show ?thesis unfolding wrank_def by (metis someI2)
qed

lemma lem_wrank_uset:
fixes r::"'U rel" and A0::"'U set"
shows "\<exists> A \<in> wbase r A0. |A| =o wrank r A0 \<and> ( \<forall> B \<in> wbase r A0. |A| \<le>o |B| )"
  using lem_wrank_wrankrel unfolding wrank_rel_def by blast

lemma lem_wrank_uset_mem_bnd:
fixes r::"'U rel" and A0 B::"'U set"
assumes "B \<in> wbase r A0"
shows "wrank r A0 \<le>o |B|"
proof -
  obtain A where "A \<in> wbase r A0 \<and> |A| =o wrank r A0 \<and> ( \<forall> A' \<in> wbase r A0. |A| \<le>o
       |A'| )" using assms lem_wrank_uset by blast
  moreover then have "|A| \<le>o |B|" using assms by blast
  ultimately show ?thesis by (metis ordIso_iff_ordLeq ordLeq_transitive)
qed

lemma lem_wrank_cardord: "Card_order (wrank r A0)"
proof -
  obtain A where "A \<in> wbase r A0 \<and> |A| =o wrank r A0" using lem_wrank_uset by blast
  then show "Card_order (wrank r A0)" using Card_order_ordIso2 card_of_Card_order by blast
qed

lemma lem_wrank_ub: "wrank r A0 \<le>o |A0|"
  using lem_wdn_range_lb[of A0 r] lem_wrank_uset_mem_bnd unfolding wbase_def by blast

lemma lem_card_un2_bnd: "\<omega>_ord \<le>o \<alpha> \<Longrightarrow> |A| \<le>o \<alpha> \<Longrightarrow> |B
      | \<le>o \<alpha> \<Longrightarrow> |A \<union> B| \<le>o \<alpha>"
proof -
  assume "\<omega>_ord \<le>o \<alpha>" and "|A| \<le>o \<alpha>" and "|B| \<le>o \<alpha>"
  moreover have "|{A, B}| \<le>o \<omega>_ord" using finite_iff_ordLess_natLeq ordLess_imp_ordLeq by blast
  ultimately have "|\<Union>{A, B}| \<le>o \<alpha>" using lem_card_un_bnd[of "{A,B}"] ordLeq_transitive by
      blast
  then show "|A \<union> B| \<le>o \<alpha>" by simp
qed

lemma lem_card_un2_lsbnd: "\<omega>_ord \<le>o \<alpha> \<Longrightarrow> |A| <o \<alpha> \<Longrightarrow> |B|
      <o \<alpha> \<Longrightarrow> |A \<union> B| <o \<alpha>"
proof -
  assume b1: "\<omega>_ord \<le>o \<alpha>" and b2: "|A| <o \<alpha>" and b3: "|B| <o \<alpha>"
  have "\<not> finite A \<longrightarrow> |A \<union> B| <o \<alpha>"
  proof
    assume c1: "\<not> finite A"
```

```
    show "|A \<union> B| <o \<alpha>"
    proof (cases "|A| \<le>o |B|")
      assume "|A| \<le>o |B|"
      then have "|A \<union> B| =o |B|" using c1 by (metis card_of_Un_infinite card_of_ordLeq_finite)
      then show ?thesis using b3 by (metis ordIso_ordLess_trans)
    next
      assume "\<not> |A| \<le>o |B|"
      then have "|B| \<le>o |A|" by (metis card_of_Well_order ordLeq_total)
      then have "|A \<union> B| =o |A|" using c1 by (metis card_of_Un_infinite)
      then show ?thesis using b2 by (metis ordIso_ordLess_trans)
    qed
  qed
  moreover have "\<not> finite B \<longrightarrow> |A \<union> B| <o \<alpha>"
  proof
    assume c1: "\<not> finite B"
    show "|A \<union> B| <o \<alpha>"
    proof (cases "|A| \<le>o |B|")
      assume "|A| \<le>o |B|"
      then have "|A \<union> B| =o |B|" using c1 by (metis card_of_Un_infinite)
      then show ?thesis using b3 by (metis ordIso_ordLess_trans)
    next
      assume "\<not> |A| \<le>o |B|"
      then have "|B| \<le>o |A|" by (metis card_of_Well_order ordLeq_total)
      then have "|A \<union> B| =o |A|" using c1 by (metis card_of_Un_infinite card_of_ordLeq_finite)
      then show ?thesis using b2 by (metis ordIso_ordLess_trans)
    qed
  qed
  moreover have "finite A \<and> finite B \<longrightarrow> |A \<union> B| <o \<alpha>"
  proof
    assume "finite A \<and> finite B"
    then have "finite (A \<union> B)" by blast
    then show "|A \<union> B| <o \<alpha>" using b1
      by (meson card_of_nat finite_iff_cardOf_nat ordIso_imp_ordLeq ordLess_ordLeq_trans)
  qed
  ultimately show ?thesis by blast
qed

lemma lem_wrank_un_bnd:
fixes r::"'U rel" and S::"'U set set" and \<alpha>::"'U rel"
assumes a1: "\<forall> A\<in>S. wrank r A \<le>o \<alpha>" and a2: "|S| \<le>o \<alpha>" and a3: "\<omega>_ord
      \<le>o \<alpha>"
shows "wrank r (\<Union> S) \<le>o \<alpha>"
proof -
  obtain h where b1: "h = (\<lambda> A B. B \<in> wbase r A \<and> |B| =o wrank r A)" by blast
  obtain Bi where b2: "Bi = (\<lambda> A. SOME B. h A B)" by blast
  have "\<forall>A\<in>S. \<exists> B. h A B" using b1 lem_wrank_uset[of r] by blast
  then have "\<forall>A\<in>S. h A (Bi A)" using b2 by (metis someI_ex)
  then have b3: "\<forall>A\<in>S. (Bi A) \<in> wbase r A \<and> |Bi A| =o wrank r A" using b1 by blast
  then have b4: "\<forall> A \<in> S. |Bi A| \<le>o \<alpha>" using assms ordIso_ordLeq_trans by blast
  obtain S' where b5: "S' = Bi ` S" by blast
  then have "|S'| \<le>o |S| \<and> (\<forall> X \<in> S'. |X| \<le>o \<alpha>)" using b4 by simp
  moreover then have "|S'| \<le>o \<alpha>" using a2 by (metis ordLeq_transitive)
  ultimately have "|\<Union>S'| \<le>o \<alpha>" using a3 lem_card_un_bnd[of S' \<alpha>] by blast
  moreover obtain B where b6: "B = (\<Union>A\<in>S. Bi A)" by blast
  ultimately have b7: "|B| \<le>o \<alpha>" using b5 by simp
  have "\<forall>A\<in>S. A \<subseteq> w_dncl r (Bi A)" using b3 unfolding wbase_def by blast
  then have "\<Union>S \<subseteq> w_dncl r B" using b6 lem_wdn_mon[of _ B r] by blast
  then have "B \<in> wbase r (\<Union>S)" unfolding wbase_def by blast
  then have "wrank r (\<Union>S) \<le>o |B|" using lem_wrank_uset_mem_bnd by blast
  then show ?thesis using b7 by (metis ordLeq_transitive)
qed

lemma lem_wrank_un_bnd_stab:
fixes r::"'U rel" and S::"'U set set" and \<alpha>::"'U rel"
assumes a1: "\<forall> A\<in>S. wrank r A <o \<alpha>" and a2: "|S| <o \<alpha>" and a3: "stable \<alpha>"
shows "wrank r (\<Union> S) <o \<alpha>"
proof -
  obtain h where b1: "h = (\<lambda> A B. B \<in> wbase r A \<and> |B| =o wrank r A)" by blast
  obtain Bi where b2: "Bi = (\<lambda> A. SOME B. h A B)" by blast
  have "\<forall>A\<in>S. \<exists> B. h A B" using b1 lem_wrank_uset[of r] by blast
  then have "\<forall>A\<in>S. h A (Bi A)" using b2 by (metis someI_ex)
  then have b3: "\<forall>A\<in>S. (Bi A) \<in> wbase r A \<and> |Bi A| =o wrank r A" using b1 by blast
  then have b4: "\<forall> A \<in> S. |Bi A| <o \<alpha>" using assms ordIso_ordLess_trans by blast
  obtain S' where b5: "S' = Bi ` S" by blast
  then have "|S'| \<le>o |S| \<and> (\<forall> X \<in> S'. |X| <o \<alpha>)" using b4 by simp
  moreover then have "|S'| <o \<alpha>" using a2 by (metis ordLeq_ordLess_trans)
  ultimately have "|\<Union>S'| <o \<alpha>" using a3 lem_card_un_bnd_stab[of \<alpha> S'] by blast
  moreover obtain B where b6: "B = (\<Union>A\<in>S. Bi A)" by blast
```

```
  ultimately have b7: "|B| <o \<alpha>" using b5 by simp
  have "\<forall>A\<in>S. A \<subseteq> w_dncl r (Bi A)" using b3 unfolding wbase_def by blast
  then have "\<Union>S \<subseteq> w_dncl r B" using b6 lem_wdn_mon[of _ B r] by blast
  then have "B \<in> wbase r (\<Union>S)" unfolding wbase_def by blast
  then have "wrank r (\<Union>S) \<le>o |B|" using lem_wrank_uset_mem_bnd by blast
  then show ?thesis using b7 by (metis ordLeq_ordLess_trans)
qed

lemma lem_wrank_fw:
fixes r::"'U rel" and K::"'U set" and \<alpha>::"'U rel"
assumes a1: "\<omega>_ord \<le>o \<alpha>" and a2: "wrank r K \<le>o \<alpha>" and a3: "\<forall> b\<in>K. wrank
       r (r``{b}) \<le>o \<alpha>"
shows "wrank r (\<Union>b\<in>K. (r``{b})) \<le>o \<alpha>"
proof -
  obtain h where b1: "h = (\<lambda> A B. B \<in> wbase r A \<and> |B| =o wrank r A)" by blast
  obtain Bi where b2: "Bi = (\<lambda> b. SOME B. h (r``{b}) B)" by blast
  have "\<forall>b\<in>K. \<exists> B. h (r``{b}) B" using b1 lem_wrank_uset[of r] by blast
  then have "\<forall>b\<in>K. h (r``{b}) (Bi b)" using b2 by (metis someI_ex)
  then have b3: "\<forall>b\<in>K. (Bi b) \<in> wbase r (r``{b}) \<and> |Bi b| =o wrank r (r``{b})" using b1 by
      blast
  obtain BK where b4: "BK \<in> wbase r K \<and> |BK| =o wrank r K" using lem_wrank_uset[of r K] by blast
  obtain BU where b5: "BU = BK \<union> (\<Union>b\<in>(K\<inter>BK). Bi b)" by blast
  obtain S where b6: "S = (\<Union>b\<in>K. (r``{b}))" by blast
  have b7: "\<forall> b \<in> K\<inter>BK. (r``{b}) \<subseteq> w_dncl r BU"
  proof
    fix b
    assume "b \<in> K \<inter> BK"
    then have "Bi b \<subseteq> BU \<and> (Bi b) \<in> wbase r (r``{b})" using b3 b5 by blast
    then show "r``{b} \<subseteq> w_dncl r BU" using lem_wdn_mon unfolding wbase_def by blast
  qed
  have "BU \<in> wbase r S"
  proof -
    have "\<forall> b \<in> K. r``{b} \<subseteq> dncl r BU"
    proof
      fix b
      assume d1: "b \<in> K"
      show "r``{b} \<subseteq> dncl r BU"
      proof (cases "b \<in> BK")
        assume "b \<in> BK"
        then show ?thesis using d1 b7 unfolding w_dncl_def by blast
      next
        assume e1: "b \<notin> BK"
        have "\<forall> t \<in> r``{b}. t \<notin> dncl r BU \<longrightarrow> False"
        proof (intro ballI impI)
          fix t
          assume f1: "t \<in> r``{b}" and f2: "t \<notin> dncl r BU"
          then have f3: "t \<notin> dncl r BK" using b5 unfolding dncl_def by blast
          moreover have "b \<in> w_dncl r BK" using d1 b4 unfolding wbase_def by blast
          ultimately have f4: "\<forall>F \<in> \<F> r b t. F \<inter> BK \<noteq> {}" unfolding w_dncl_def by
      blast
          obtain f where f5: "f = (\<lambda> n::nat. if (n = 0) then b else t)" by blast
          then have "f 0 = b \<and> f 1 = t" by simp
          moreover then have "\<forall>i<1. (f i, f (Suc i)) \<in> r" using f1 by simp
          ultimately have "f \<in> rpth r b t 1 \<and> {b, t} = f ` {i. i \<le> 1}"
             using f5 unfolding rpth_def by force
          then have "{b, t} \<in> \<F> r b t" unfolding \<F>_def by blast
          then have "{b, t} \<inter> BK \<noteq> {}" using f4 by blast
          then show "False" using e1 f3 unfolding dncl_def by blast
        qed
        then show ?thesis by blast
      qed
    qed
    then have c1: "S \<subseteq> dncl r BU" using b6 by blast
    moreover have "\<forall> x \<in> S. \<forall>c. \<forall>F\<in>\<F> r x c. c \<notin> dncl r BU \<
      longrightarrow> F \<inter> BU \<noteq> {}"
    proof (intro ballI allI impI)
      fix x c F
      assume d1: "x \<in> S" and d2: "F \<in> \<F> r x c" and d3: "c \<notin> dncl r BU"
      then obtain b where d4: "b \<in> K \<and> (b,x) \<in> r" using b6 by blast
      show "F \<inter> BU \<noteq> {}"
      proof (cases "b \<in> BK")
        assume "b \<in> BK"
        then have "x \<in> w_dncl r BU" using b7 d4 by blast
        then show ?thesis using d2 d3 unfolding w_dncl_def by blast
      next
        assume e1: "b \<notin> BK"
        have e2: "b \<in> w_dncl r BK" using d4 b4 unfolding wbase_def by blast
        obtain f n where e3: "f \<in> rpth r x c n" and e4: "F = f ` {i. i\<le>n}"
```

```
          using d2 unfolding \<F>_def by blast
        obtain g where e5: "g = (\<lambda> k::nat. if (k=0) then b else (f (k-1)))" by blast
        then have "g \<in> rpth r b c (Suc n)"
          using e3 d4 unfolding rpth_def
          by (simp, metis Suc_le_eq diff_Suc_Suc diff_zero gr0_implies_Suc less_Suc_eq_le)
        then have "g ` {i. i \<le> (Suc n)} \<in> \<F> r b c \<and> c \<notin> dncl r BK"
          using d3 b5 unfolding \<F>_def dncl_def by blast
        then have "g ` {i. i \<le> (Suc n)} \<inter> BK \<noteq> {}" using e2 unfolding w_dncl_def by blast
        moreover have "g ` {i. i \<le> (Suc n)} \<subseteq> F \<union> {b}"
        proof
          fix a
          assume "a \<in> g ` {i. i \<le> (Suc n)}"
          then obtain i where "i \<le> (Suc n) \<and> a = g i" by blast
          then show "a \<in> F \<union> {b}" using e4 e5 by force
        qed
        ultimately have "(F \<union> {b}) \<inter> BK \<noteq> {}" by blast
        then show ?thesis using e1 b5 by blast
      qed
    qed
    ultimately have "S \<subseteq> w_dncl r BU" unfolding w_dncl_def by blast
    then show ?thesis unfolding wbase_def by blast
  qed
  moreover have "|BU| \<le>o \<alpha>"
  proof -
    have c1: "|BK| \<le>o \<alpha>" using b4 a2 by (metis ordIso_ordLeq_trans)
    then have "|K \<inter> BK| \<le>o \<alpha>" by (meson card_of_mono1 inf_le2 ordLeq_transitive)
    then have "|Bi ` (K \<inter> BK)| \<le>o \<alpha>" by (metis card_of_image ordLeq_transitive)
    moreover have "\<forall> b\<in>(K\<inter>BK). |Bi b| \<le>o \<alpha>" using b3 a3 by (meson Int_iff
      ordIso_ordLeq_trans)
    ultimately have "|\<Union>(Bi ` (K \<inter> BK))| \<le>o \<alpha>" using a1 lem_card_un_bnd[of "Bi`(K\<inter
      >BK)" \<alpha>] by blast
    then show "|BU| \<le>o \<alpha>" using c1 b5 a1 lem_card_un2_bnd[of \<alpha> BK "\<Union>(Bi ` (K \<inter>
      BK))"] by simp
  qed
  ultimately have "wrank r S \<le>o \<alpha>" using b6 lem_wrank_uset_mem_bnd ordLeq_transitive by blast
  then show ?thesis using b6 by blast
qed

lemma lem_wrank_fw_stab:
fixes r::"'U rel" and K::"'U set" and \<alpha>::"'U rel"
assumes a1: "\<omega>_ord \<le>o \<alpha> \<and> stable \<alpha>" and a2: "wrank r K <o \<alpha>" and a3: "\<
      forall> b\<in>K. wrank r (r``{b}) <o \<alpha>"
shows "wrank r (\<Union>b\<in>K. (r``{b})) <o \<alpha>"
proof -
  obtain h where b1: "h = (\<lambda> A B. B \<in> wbase r A \<and> |B| =o wrank r A)" by blast
  obtain Bi where b2: "Bi = (\<lambda> b. SOME B. h (r``{b}) B)" by blast
  have "\<forall>b\<in>K. \<exists> B. h (r``{b}) B" using b1 lem_wrank_uset[of r] by blast
  then have "\<forall>b\<in>K. h (r``{b}) (Bi b)" using b2 by (metis someI_ex)
  then have b3: "\<forall>b\<in>K. (Bi b) \<in> wbase r (r``{b}) \<and> |Bi b| =o wrank r (r``{b})" using b1 by
      blast
  obtain BK where b4: "BK \<in> wbase r K \<and> |BK| =o wrank r K" using lem_wrank_uset[of r K] by blast
  obtain BU where b5: "BU = BK \<union> (\<Union>b\<in>(K\<inter>BK). Bi b)" by blast
  obtain S where b6: "S = (\<Union>b\<in>K. (r``{b}))" by blast
  have b7: "\<forall> b \<in> K\<inter>BK. (r``{b}) \<subseteq> w_dncl r BU"
  proof
    fix b
    assume "b \<in> K \<inter> BK"
    then have "Bi b \<subseteq> BU \<and> (Bi b) \<in> wbase r (r``{b})" using b3 b5 by blast
    then show "r``{b} \<subseteq> w_dncl r BU" using lem_wdn_mon unfolding wbase_def by blast
  qed
  have "BU \<in> wbase r S"
  proof -
    have "\<forall> b \<in> K. r``{b} \<subseteq> dncl r BU"
    proof
      fix b
      assume d1: "b \<in> K"
      show "r``{b} \<subseteq> dncl r BU"
      proof (cases "b \<in> BK")
        assume "b \<in> BK"
        then show ?thesis using d1 b7 unfolding w_dncl_def by blast
      next
        assume e1: "b \<notin> BK"
        have "\<forall> t \<in> r``{b}. t \<notin> dncl r BU \<longrightarrow> False"
        proof (intro ballI impI)
          fix t
          assume f1: "t \<in> r``{b}" and f2: "t \<notin> dncl r BU"
          then have f3: "t \<notin> dncl r BK" using b5 unfolding dncl_def by blast
          moreover have "b \<in> w_dncl r BK" using d1 b4 unfolding wbase_def by blast
```

```
        ultimately have f4: "\<forall>F \<in> \<F> r b t. F \<inter> BK \<noteq> {}" unfolding w_dncl_def by
     blast
        obtain f where f5: "f = (\<lambda> n::nat. if (n = 0) then b else t)" by blast
        then have "f 0 = b \<and> f 1 = t" by simp
        moreover then have "\<forall>i<1. (f i, f (Suc i)) \<in> r" using f1 by simp
        ultimately have "f \<in> rpth r b t 1 \<and> {b, t} = f ` {i. i \<le> 1}"
          using f5 unfolding rpth_def by force
        then have "{b, t} \<in> \<F> r b t" unfolding \<F>_def by blast
        then have "{b, t} \<inter> BK \<noteq> {}" using f4 by blast
        then show "False" using e1 f3 unfolding dncl_def by blast
      qed
      then show ?thesis by blast
    qed
  qed
  then have c1: "S \<subseteq> dncl r BU" using b6 by blast
  moreover have "\<forall> x \<in> S. \<forall>c. \<forall>F\<in>\<F> r x c. c \<notin> dncl r BU \<
    longrightarrow> F \<inter> BU \<noteq> {}"
  proof (intro ballI allI impI)
    fix x c F
    assume d1: "x \<in> S" and d2: "F \<in> \<F> r x c" and d3: "c \<notin> dncl r BU"
    then obtain b where d4: "b \<in> K \<and> (b,x) \<in> r" using b6 by blast
    show "F \<inter> BU \<noteq> {}"
    proof (cases "b \<in> BK")
      assume "b \<in> BK"
      then have "x \<in> w_dncl r BU" using b7 d4 by blast
      then show ?thesis using d2 d3 unfolding w_dncl_def by blast
    next
      assume e1: "b \<notin> BK"
      have e2: "b \<in> w_dncl r BK" using d4 b4 unfolding wbase_def by blast
      obtain f n where e3: "f \<in> rpth r x c n" and e4: "F = f ` {i. i\<le>n}"
        using d2 unfolding \<F>_def by blast
      obtain g where e5: "g = (\<lambda> k::nat. if (k=0) then b else (f (k-1)))" by blast
      then have "g \<in> rpth r b c (Suc n)"
        using e3 d4 unfolding rpth_def
        by (simp, metis Suc_le_eq diff_Suc_Suc diff_zero gr0_implies_Suc less_Suc_eq_le)
      then have "g ` {i. i \<le> (Suc n)} \<in> \<F> r b c \<and> c \<notin> dncl r BK"
        using d3 b5 unfolding \<F>_def dncl_def by blast
      then have "g ` {i. i \<le> (Suc n)} \<inter> BK \<noteq> {}" using e2 unfolding w_dncl_def by blast
      moreover have "g ` {i. i \<le> (Suc n)} \<subseteq> F \<union> {b}"
      proof
        fix a
        assume "a \<in> g ` {i. i \<le> (Suc n)}"
        then obtain i where "i \<le> (Suc n) \<and> a = g i" by blast
        then show "a \<in> F \<union> {b}" using e4 e5 by force
      qed
      ultimately have "(F \<union> {b}) \<inter> BK \<noteq> {}" by blast
      then show ?thesis using e1 b5 by blast
    qed
  qed
  ultimately have "S \<subseteq> w_dncl r BU" unfolding w_dncl_def by blast
  then show ?thesis unfolding wbase_def by blast
qed
moreover have "|BU| <o \<alpha>"
proof -
  have c1: "|BK| <o \<alpha>" using b4 a2 by (metis ordIso_imp_ordLeq ordLeq_ordLess_trans)
  then have "|K \<inter> BK| <o \<alpha>" by (meson Int_iff card_of_mono1 ordLeq_ordLess_trans subsetI)
  then have "|Bi ` (K \<inter> BK)| <o \<alpha>" by (metis card_of_image ordLeq_ordLess_trans)
  moreover have "\<forall> b\<in>(K\<inter>BK). |Bi b| <o \<alpha>" using b3 a3 by (meson Int_iff
    ordIso_ordLess_trans)
  ultimately have "|\<Union>(Bi ` (K \<inter> BK))| <o \<alpha>" using a1 lem_card_un_bnd_stab[of \<alpha> "Bi
    `(K\<inter>BK)"] by blast
  then show "|BU| <o \<alpha>" using c1 b5 a1 lem_card_un2_lsbnd[of \<alpha> BK "\<Union>(Bi ` (K \<inter> BK)
    )"] by simp
qed
ultimately have "wrank r S <o \<alpha>" using b6 lem_wrank_uset_mem_bnd[of BU r S] by (metis
    ordLeq_ordLess_trans)
then show ?thesis using b6 by blast
qed

lemma lem_wnb_neib:
fixes r::"'U rel" and \<alpha>::"'U rel"
assumes a1: "\<omega>_ord \<le>o \<alpha>" and a2: "\<alpha> <o \<parallel>r\<parallel>"
shows "\<forall> a \<in> Field r. \<exists> b \<in> Mwn r \<alpha>. (a,b) \<in> r^*"
proof
  fix a
  assume b1: "a \<in> Field r"
  have "\<not> (\<exists> b \<in> Mwn r \<alpha>. (a,b) \<in> r^*) \<longrightarrow> False"
  proof
```

```
assume c1: "\<not> (\<exists> b \<in> Mwn r \<alpha>. (a,b) \<in> r^*)"
obtain B where c2: "B = (r^*)``{a}" by blast
obtain S where c3: "S = ( (\<lambda> n. (r^^n)``{a}) ` (UNIV::nat set) )" by blast
have c4: "\<forall> b \<in> B. wrank r (r``{b}) \<le>o \<alpha>"
proof
  fix b
  assume d1: "b \<in> B"
  then obtain k where "b \<in> (r^^k)``{a}" using c2 rtrancl_power by blast
  moreover have "\<forall> n. (r^^n) `` {a} \<subseteq> Field r"
  proof
    fix n
    show "(r^^n) `` {a} \<subseteq> Field r" using b1
      by (induct n, force, meson FieldI2 Image_singleton_iff relpow_Suc_E subsetI)
  qed
  ultimately have "b \<in> Field r" by blast
  moreover have "b \<notin> Mwn r \<alpha>" using d1 c1 c2 by blast
  ultimately have "b \<in> Field r - Mwn r \<alpha>" by blast
  moreover have "Well_order \<alpha>" using assms unfolding ordLess_def by blast
  moreover have "Well_order (wrank r (r``{b}))" using lem_wrank_cardord by (metis
   card_order_on_well_order_on)
  ultimately show "wrank r (r``{b}) \<le>o \<alpha>" unfolding Mwn_def by simp
qed
have "\<forall> n. wrank r ((r^^n)``{a}) \<le>o \<alpha>"
proof
  fix n0
  show "wrank r ((r^^n0)``{a}) \<le>o \<alpha>"
  proof (induct n0)
    have "|{a}| \<le>o \<omega>_ord" using card_of_Well_order finite.emptyI
      infinite_iff_natLeq_ordLeq natLeq_Well_order ordLeq_total by blast
    then have "|(r^^0)``{a}| \<le>o \<omega>_ord" by simp
    then show "wrank r ((r^^0)``{a}) \<le>o \<alpha>"
      using a1 lem_wrank_ub[of r "(r^^0)``{a}"] by (metis ordLeq_transitive)
  next
    fix n
    assume e1: "wrank r ((r^^n)``{a}) \<le>o \<alpha>"
    obtain K where e2: "K = (r^^n)``{a}" by blast
    obtain S' where e3: "S' = ((\<lambda> b. r``{b}) ` K)" by blast
    have "wrank r K \<le>o \<alpha>" using e1 e2 by blast
    moreover have "\<forall>A\<in>S'. wrank r A \<le>o \<alpha>"
    proof
      fix A
      assume "A \<in> S'"
      then obtain b where "b \<in> K \<and> A = r``{b}" using e3 by blast
      moreover then have "b \<in> B" using c2 e2 rtrancl_power by blast
      ultimately show "wrank r A \<le>o \<alpha>" using c4 by blast
    qed
    ultimately have e4: "wrank r (\<Union> S') \<le>o \<alpha>"
      using a1 e3 lem_wrank_fw[of \<alpha> r K] by fastforce
    have "(r^^(Suc n))``{a} = r``K" using e2 by force
    moreover have "r``K = \<Union> S'" using e3 by blast
    ultimately have "(r^^(Suc n))``{a} = \<Union> S'" using e2 by blast
    then show "wrank r ((r^^(Suc n))``{a}) \<le>o \<alpha>" using e4 by simp
  qed
qed
then have "\<forall>A\<in>S. wrank r A \<le>o \<alpha>" using c3 by blast
moreover have "B = \<Union> S" using c2 c3 rtrancl_power apply (simp) by blast
moreover have "|S| \<le>o \<alpha>"
proof -
  have "|S| \<le>o |UNIV::nat set|" using c3 by simp
  moreover have "|UNIV::nat set| =o \<omega>_ord" using card_of_nat by blast
  ultimately show ?thesis using a1 ordLeq_ordIso_trans ordLeq_transitive by blast
qed
ultimately have "wrank r B \<le>o \<alpha>" using a1 lem_wrank_un_bnd[of S r \<alpha>] by blast
moreover obtain B0 where "B0 \<in> wbase r B \<and> |B0| =o wrank r B" using lem_wrank_uset[of r B] by blast
ultimately have c5: "B \<subseteq> dncl r B0 \<and> |B0| \<le>o \<alpha>"
  unfolding wbase_def w_dncl_def using ordIso_ordLeq_trans by blast
have "(({}::'U rel) <o \<parallel>r\<parallel>)" using a2 by (metis ordLeq_ordLess_trans
   ordLess_Well_order_simp ozero_def ozero_ordLeq)
then have c6: "CCR r" using lem_Rcc_eq1_31 by blast
obtain B1 where c7: "B1 = B0 \<inter> Field r" by blast
then have c8: "|B1| \<le>o \<alpha>" using c5 by (meson IntE card_of_mono1 ordLeq_transitive subsetI)
have "B1 \<subseteq> Field r" using c7 by blast
moreover have "\<forall>x \<in> Field r. \<exists>y \<in> B1. (x, y) \<in> r^*"
proof
  fix x
  assume e1: "x \<in> Field r"
  then obtain y where "(x,y) \<in> r^* \<and> (a,y) \<in> r^*" using c6 b1 unfolding CCR_def by blast
  moreover then have "y \<in> B" unfolding c2 by blast
```

```
      moreover then obtain y' where "y' \<in> B0 \<and> (y,y') \<in> r^*" using c5 unfolding dncl_def by blast
      ultimately have "y' \<in> B0 \<and> (x,y') \<in> r^*" by force
      moreover then have "x = y' \<or> y' \<in> Field r" using lem_rtr_field[of x y'] by blast
      ultimately have "y' \<in> B1 \<and> (x,y') \<in> r^*" using e1 c7 by blast
      then show "\<exists>y\<in>B1. (x, y) \<in> r^*" by blast
    qed
    ultimately have "B1 \<in> SCF r" unfolding SCF_def by blast
    then have "scf r \<le>o |B1|" using lem_scf_uset_mem_bnd by blast
    then have "scf r \<le>o \<alpha>" using c8 by (metis ordLeq_transitive)
    moreover have "\<parallel>r\<parallel> =o scf r" using c6 lem_scf_ccr_scf_rcc_eq[of r] by blast
    ultimately show "False" using a2 by (metis not_ordLeq_ordLess ordIso_ordLeq_trans)
  qed
  then show "\<exists> b \<in> Mwn r \<alpha>. (a,b) \<in> r^*" by blast
qed

lemma lem_wnb_neib3:
fixes r::"'U rel"
assumes a1: "\<omega>_ord <o \<parallel>r\<parallel>" and a2: "stable \<parallel>r\<parallel>"
shows "\<forall> a \<in> Field r. \<exists> b \<in> Mwnm r. (a,b) \<in> r^*"
proof
  fix a
  assume b1: "a \<in> Field r"
  have "\<not> (\<exists> b \<in> Mwnm r. (a,b) \<in> r^*) \<longrightarrow> False"
  proof
    assume c1: "\<not> (\<exists> b \<in> Mwnm r. (a,b) \<in> r^*)"
    obtain B where c2: "B = (r^*)``{a}" by blast
    obtain S where c3: "S = ( (\<lambda> n. (r^^n)``{a}) ` (UNIV::nat set) )" by blast
    have c4: "\<forall> b \<in> B. wrank r (r ``{b}) <o \<parallel>r\<parallel>"
    proof
      fix b
      assume d1: "b \<in> B"
      then obtain k where "b \<in> (r^^k)``{a}" using c2 rtrancl_power by blast
      moreover have "\<forall> n. (r^^n) `` {a} \<subseteq> Field r"
      proof
        fix n
        show "(r^^n) `` {a} \<subseteq> Field r" using b1
          by (induct n, force, meson FieldI2 Image_singleton_iff relpow_Suc_E subsetI)
      qed
      ultimately have "b \<in> Field r" by blast
      moreover have "b \<notin> Mwnm r" using d1 c1 c2 by blast
      ultimately have "b \<in> Field r - Mwnm r" by blast
      moreover have "Well_order (wrank r (r``{b}))" using lem_wrank_cardord by (metis
      card_order_on_well_order_on)
      moreover have "Well_order \<parallel>r\<parallel>" using lem_rcc_cardord unfolding card_order_on_def by
      blast
      ultimately show "wrank r (r``{b}) <o \<parallel>r\<parallel>" unfolding Mwnm_def by simp
    qed
    have "\<forall> n. wrank r ((r^^n)``{a}) <o \<parallel>r\<parallel>"
    proof
      fix n0
      show "wrank r ((r^^n0)``{a}) <o \<parallel>r\<parallel>"
      proof (induct n0)
        have "|{a}| \<le>o \<omega>_ord" using card_of_Well_order finite.emptyI
          infinite_iff_natLeq_ordLeq natLeq_Well_order ordLeq_total by blast
        then have "|(r^^0)``{a}| \<le>o \<omega>_ord" by simp
        then show "wrank r ((r^^0)``{a}) <o \<parallel>r\<parallel>"
          using a1 lem_wrank_ub[of r "(r^^0)``{a}"] by (metis ordLeq_ordLess_trans)
      next
        fix n
        assume e1: "wrank r ((r^^n)``{a}) <o \<parallel>r\<parallel>"
        obtain K where e2: "K = (r^^n)``{a}" by blast
        obtain S' where e3: "S' = ((\<lambda> b. r``{b}) ` K)" by blast
        have "wrank r K <o \<parallel>r\<parallel>" using e1 e2 by blast
        moreover have "\<forall>A\<in>S'. wrank r A <o \<parallel>r\<parallel>"
        proof
          fix A
          assume "A \<in> S'"
          then obtain b where "b \<in> K \<and> A = r``{b}" using e3 by blast
          moreover then have "b \<in> B" using c2 e2 rtrancl_power by blast
          ultimately show "wrank r A <o \<parallel>r\<parallel>" using c4 by blast
        qed
        moreover have "\<omega>_ord \<le>o \<parallel>r\<parallel>" using a1 by (metis ordLess_imp_ordLeq)
        ultimately have e4: "wrank r (\<Union> S') <o \<parallel>r\<parallel>"
          using e3 a2 lem_wrank_fw_stab[of "\<parallel>r\<parallel>" r K] by fastforce
        have "(r^^(Suc n))``{a} = r``K" using e2 by force
        moreover have "r``K = \<Union> S'" using e3 by blast
        ultimately have "(r^^(Suc n))``{a} = \<Union> S'" using e2 by blast
        then show "wrank r ((r^^(Suc n)) `` {a}) <o \<parallel>r\<parallel>" using e4 by simp
```

```
      qed
    qed
    then have "\<forall>A\<in>S. wrank r A <o \<parallel>r\<parallel>" using c3 by blast
    moreover have "B = \<Union> S" using c2 c3 rtrancl_power apply (simp) by blast
    moreover have "|S| <o \<parallel>r\<parallel>"
    proof -
      have "|S| \<le>o |UNIV::nat set|" using c3 by simp
      moreover have "|UNIV::nat set| =o \<omega>_ord" using card_of_nat by blast
      ultimately show ?thesis using a1 ordLeq_ordIso_trans ordLeq_ordLess_trans by blast
    qed
    ultimately have "wrank r B <o \<parallel>r\<parallel>" using a2 lem_wrank_un_bnd_stab[of S r "\<parallel>r\<
      parallel>"] by blast
    moreover obtain B0 where "B0 \<in> wbase r B \<and> |B0| =o wrank r B" using lem_wrank_uset[of r B] by blast
    ultimately have c5: "B \<subseteq> dncl r B0 \<and> |B0| <o \<parallel>r\<parallel>"
      unfolding wbase_def w_dncl_def
      by (metis (no_types, lifting) mem_Collect_eq ordIso_ordLess_trans subsetI subset_trans)
    have "(({}::'U rel) <o \<parallel>r\<parallel>)" using a1 by (metis ordLeq_ordLess_trans
      ordLess_Well_order_simp ozero_def ozero_ordLeq)
    then have c6: "CCR r" using lem_Rcc_eq1_31 by blast
    obtain B1 where c7: "B1 = B0 \<inter> Field r" by blast
    then have c8: "|B1| <o \<parallel>r\<parallel>" using c5 by (meson IntE card_of_mono1 ordLeq_ordLess_trans
      subsetI)
    have "B1 \<subseteq> Field r" using c7 by blast
    moreover have "\<forall>x \<in> Field r. \<exists>y \<in> B1. (x, y) \<in> r^*"
    proof
      fix x
      assume e1: "x \<in> Field r"
      then obtain y where "(x,y) \<in> r^* \<and> (a,y) \<in> r^*" using c6 b1 unfolding CCR_def by blast
      moreover then have "y \<in> B" unfolding c2 by blast
      moreover then obtain y' where "y' \<in> B0 \<and> (y,y') \<in> r^*" using c5 unfolding dncl_def by blast
      ultimately have "y' \<in> B0 \<and> (x,y') \<in> r^*" by force
      moreover then have "x = y' \<or> y' \<in> Field r" using lem_rtr_field[of x y'] by blast
      ultimately have "y' \<in> B1 \<and> (x,y') \<in> r^*" using e1 c7 by blast
      then show "\<exists>y\<in>B1. (x, y) \<in> r^*" by blast
    qed
    ultimately have "B1 \<in> SCF r" unfolding SCF_def by blast
    then have "scf r \<le>o |B1|" using lem_scf_uset_mem_bnd by blast
    then have "scf r <o \<parallel>r\<parallel>" using c8 by (metis ordLeq_ordLess_trans)
    moreover have "\<parallel>r\<parallel> =o scf r" using c6 lem_scf_ccr_scf_rcc_eq[of r] by blast
    ultimately show "False" by (metis not_ordLess_ordIso ordIso_symmetric)
  qed
  then show "\<exists> b \<in> Mwnm r. (a,b) \<in> r^*" by blast
qed

lemma lem_scfgew_ncl: "\<omega>_ord \<le>o scf r \<Longrightarrow> \<not> Conelike r"
proof (cases "CCR r")
  assume "\<omega>_ord \<le>o scf r" and "CCR r"
  then have "\<omega>_ord \<le>o \<parallel>r\<parallel>" using lem_scf_ccr_scf_rcc_eq[of r]
    by (metis ordIso_iff_ordLeq ordLeq_transitive)
  then have "\<forall> a. \<not> ( \<parallel>r\<parallel> \<le>o |{a}| )" using finite_iff_ordLess_natLeq
    ordLess_ordLeq_trans[of _ \<omega>_ord "\<parallel>r\<parallel>"] not_ordLess_ordLeq[of _ "\<parallel>r\<
      parallel>"] by blast
  then show "\<not> Conelike r" using lem_Rcc_eq2_12[of r] by metis
next
  assume "\<omega>_ord \<le>o scf r" and "\<not> CCR r"
  then show "\<not> Conelike r" unfolding CCR_def Conelike_def by fastforce
qed

lemma lem_wnb_P_ncl_reg_grw:
fixes r::"'U rel"
assumes a1: "CCR r" and a2: "\<omega>_ord <o scf r" and a3: "regularCard (scf r)"
shows "\<exists> P \<in> SCF r. (\<forall> \<alpha>::'U rel. \<alpha> <o scf r \<longrightarrow> (\<forall> a \<
      in> P. \<alpha> <o wrank r (r``{a}) ))"
proof -
  have "\<not> Conelike r" using a2 lem_scfgew_ncl ordLess_imp_ordLeq by blast
  moreover obtain P where b1: "P = { a \<in> Field r. scf r \<le>o wrank r (r ``{a}) }" by blast
  ultimately have "stable (scf r)"
    using a1 a3 lem_scf_ccr_finscf_cl lem_scf_cardord regularCard_stable by blast
  then have "stable \<parallel>r\<parallel>" using a1 lem_scf_ccr_scf_rcc_eq stable_ordIso1 by blast
  moreover have "\<omega>_ord <o \<parallel>r\<parallel>" using a1 a2 lem_scf_ccr_scf_rcc_eq[of r]
    by (metis ordIso_iff_ordLeq ordLess_ordLeq_trans)
  ultimately have "\<forall>a\<in>Field r. \<exists>b \<in> Mwnm r. (a, b) \<in> r^*" using lem_wnb_neib3 by
      blast
  moreover have "Mwnm r \<subseteq> P"  unfolding b1 Mwnm_def using a1 lem_scf_ccr_scf_rcc_eq[of r]
    by (clarsimp, metis ordIso_ordLeq_trans ordIso_symmetric)
  moreover have "P \<subseteq> Field r" using b1 by blast
  ultimately have "P \<in> SCF r" unfolding SCF_def by blast
```

```
  moreover have "\<forall> \<alpha>::'U rel. \<alpha> <o scf r \<longrightarrow> (\<forall> a \<in> P. \<alpha>
      <o wrank r (r``{a}) )"
    using b1 ordLess_ordLeq_trans by blast
  ultimately show ?thesis by blast
qed

lemma lem_wnb_P_ncl_nreg:
fixes r::"'U rel"
assumes a1: "CCR r" and a2: "\<omega>_ord \<le>o scf r" and a3: "\<not> regularCard (scf r)"
shows "\<exists> Ps::'U set set. Ps \<subseteq> SCF r \<and> |Ps| <o scf r
                       \<and> (\<forall> \<alpha>::'U rel. \<alpha> <o scf r \<longrightarrow> (\<exists> P \<in>
       Ps. \<forall> a \<in> P. \<alpha> <o wrank r (r``{a}) ))"
proof -
  have "\<not> Conelike r" using a2 lem_scfgew_ncl by blast
  then have b1: "\<not> finite (Field (scf r))" using a1 lem_scf_ccr_finscf_cl by blast
  have b2: "\<And> \<alpha>::'U rel. \<omega>_ord \<le>o \<alpha> \<Longrightarrow> \<alpha> <o scf r \<
      Longrightarrow> { a \<in> Field r. \<alpha> <o wrank r (r ``{a}) } \<in> SCF r"
  proof -
    fix \<alpha>::"'U rel"
    assume c1: "\<omega>_ord \<le>o \<alpha>" and c2: "\<alpha> <o scf r"
    have "\<alpha> <o \<parallel>r\<parallel>" using a1 c2 lem_scf_ccr_scf_rcc_eq ordIso_iff_ordLeq
      ordLess_ordLeq_trans by blast
    then have "\<forall> a \<in> Field r. \<exists> b \<in> Mwn r \<alpha>. (a,b) \<in> r^*" using c1
      lem_wnb_neib by blast
    then show "{ a \<in> Field r. \<alpha> <o wrank r (r ``{a}) } \<in> SCF r" unfolding SCF_def Mwn_def by
      blast
  qed
  have b3: "\<omega>_ord <o scf r"
  proof -
    have c1: "\<not> stable (scf r)" using b1 a3 lem_scf_cardord stable_regularCard by blast
    have "\<omega>_ord \<le>o scf r" using b1 lem_inford_ge_w lem_scf_cardord unfolding card_order_on_def by
      blast
    moreover have "\<omega>_ord =o scf r \<longrightarrow> False" using c1 stable_ordIso stable_natLeq by blast
    ultimately show ?thesis using ordLeq_iff_ordLess_or_ordIso by blast
  qed
  obtain S::"'U rel set" where b4: "|S| <o scf r" and b5: "\<forall>\<alpha>\<in>S. \<alpha> <o scf r"
                          and b6: "\<forall>\<alpha>::('U rel). \<alpha> <o scf r \<longrightarrow> (\<exists
      >\<beta>\<in>S. \<alpha> \<le>o \<beta>)"
    using b1 a3 lem_scf_cardord[of r] lem_card_nreg_inf_osetlm[of "scf r"] by blast
  obtain S1::"'U rel set" where b7: "S1 = { \<alpha> \<in> S. \<omega>_ord \<le>o \<alpha> }" by blast
  obtain f::"'U rel \<Rightarrow> 'U set" where b8: "f = (\<lambda> \<alpha>. { a \<in> Field r. \<alpha> <o
      wrank r (r ``{a}) })" by blast
  obtain Ps::"'U set set" where b9: "Ps = f ` S1" by blast
  have "Ps \<subseteq> SCF r" using b2 b5 b7 b8 b9 by blast
  moreover have "|Ps| <o scf r"
  proof -
    have "|Ps| \<le>o |S1|" using b9 by simp
    moreover have "|S1| \<le>o |S|" using b7 card_of_mono1[of S1 S] by blast
    ultimately show ?thesis using b4 ordLeq_ordLess_trans ordLeq_transitive by blast
  qed
  moreover have "\<forall> \<alpha>::'U rel. \<alpha> <o scf r \<longrightarrow> (\<exists> P \<in> Ps. \<forall
      > a \<in> P. \<alpha> <o wrank r (r``{a}) )"
  proof (intro allI impI)
    fix \<alpha>::"'U rel"
    assume c1: "\<alpha> <o scf r"
    have "\<exists> \<alpha>m::('U rel). \<omega>_ord \<le>o \<alpha>m \<and> \<alpha> \<le>o \<alpha>m \<and>
      \<alpha>m <o scf r"
    proof (cases "\<omega>_ord \<le>o \<alpha>")
      assume "\<omega>_ord \<le>o \<alpha>"
      then show ?thesis using c1 ordLeq_reflexive unfolding ordLeq_def by blast
    next
      assume "\<not> (\<omega>_ord \<le>o \<alpha>)"
      then have d1: "\<alpha> \<le>o \<omega>_ord" using c1 natLeq_Well_order ordLess_Well_order_simp
        ordLess_imp_ordLeq ordLess_or_ordLeq by blast
      have "isLimOrd (scf r)"
        using b1 lem_scf_cardord[of r] card_order_infinite_isLimOrd[of "scf r"] by blast
      then obtain \<alpha>m::"'U rel" where "\<omega>_ord \<le>o \<alpha>m \<and> \<alpha>m <o scf r"
        using b3 lem_lmord_prec[of \<omega>_ord "scf r"] ordLess_imp_ordLeq by blast
      then show ?thesis using d1 ordLeq_transitive by blast
    qed
    then obtain \<alpha>m::"'U rel" where "\<omega>_ord \<le>o \<alpha>m \<and> \<alpha> \<le>o \<alpha>m \<and>
       \<alpha>m <o scf r" by blast
    moreover then obtain \<beta>::"'U rel" where "\<beta> \<in> S \<and> \<alpha>m \<le>o \<beta>" using b6 by
      blast
    ultimately have c2: "\<alpha> \<le>o \<beta>" and c3: "\<beta> \<in> S1" using b7 ordLeq_transitive by blast
      +
    obtain P where c4: "P = f \<beta>" by blast
    then have "P \<in> Ps" using c3 b9 by blast
```

```
    moreover have "\<forall> a \<in> P. \<alpha> <o wrank r (r``{a})" using c2 c4 b8 ordLeq_ordLess_trans by
      blast
    ultimately show "\<exists> P \<in> Ps. \<forall> a \<in> P. \<alpha> <o wrank r (r``{a})" by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Wf_ext_arc:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel" and a::"'U"
assumes a1: "scf r =o |Field r|" and a2: "f \<in> \<N> r Ps"
    and a3: "\<forall>\<gamma>::'U rel. \<gamma> <o scf r \<longrightarrow> (\<forall>a \<in> P. \<gamma> <o
      wrank r (r``{a}))"
    and a4: "\<omega>_ord \<le>o \<alpha>" and a5: "a \<in> f \<alpha> \<inter> P"
shows "\<And> \<beta>. \<alpha> <o \<beta> \<and> \<beta> <o |Field r| \<and> (\<beta> = {} \<or> isSuccOrd \<
      beta>) \<Longrightarrow> (r``{a} \<inter> (\<W> r f \<beta>) \<noteq> {})"
proof (elim conjE)
  fix \<beta>::"'U rel"
  assume b1: "\<alpha> <o \<beta>" and b2: "\<beta> <o |Field r|" and b3: "\<beta> = {} \<or> isSuccOrd \<beta>"
  have b4: "\<omega>_ord \<le>o \<beta>" using b1 a4 by (metis ordLeq_ordLess_trans ordLess_imp_ordLeq)
  have b5: "a \<in> (\<LL> f \<beta>) \<inter> P" using b1 a5 unfolding \<LL>_def by blast
  show "r``{a} \<inter> (\<W> r f \<beta>) \<noteq> {}"
  proof -
    have "r``{a} \<subseteq> w_dncl r (\<LL> f \<beta>) \<or> ( r``{a} \<inter> (\<W> r f \<beta>)\<noteq>{})"
      using b2 b3 b5 a2 unfolding \<N>_def \<N>4_def using ordLess_imp_ordLeq by blast
    moreover have "r``{a} \<subseteq> w_dncl r (\<LL> f \<beta>) \<longrightarrow> False"
    proof
      assume "r``{a} \<subseteq> w_dncl r (\<LL> f \<beta>)"
      then have "\<LL> f \<beta> \<in> wbase r (r``{a})" unfolding wbase_def by blast
      then have d1: "wrank r (r``{a}) \<le>o |\<LL> f \<beta>|" using lem_wrank_uset_mem_bnd by blast
      have "\<LL> f \<beta> \<subseteq> f \<beta>" using b2 a2 unfolding \<N>_def \<N>1_def \<LL>_def using
       ordLess_imp_ordLeq by blast
      then have "|\<LL> f \<beta>| \<le>o |f \<beta>|" by simp
      moreover have "|f \<beta>| \<le>o \<beta>" using a2 b2 b4 unfolding \<N>_def \<N>7_def using
       ordLess_imp_ordLeq by blast
      ultimately have "wrank r (r``{a}) \<le>o \<beta>"  using d1 ordLeq_transitive by blast
      moreover have "\<beta> <o wrank r (r `` {a})" using b2 b5 a1 a3 by (meson IntE ordIso_symmetric
       ordLess_ordIso_trans)
      ultimately show "False" by (metis not_ordLeq_ordLess)
    qed
    ultimately show ?thesis by blast
  qed
qed

lemma lem_Wf_esc_pth:
fixes r::"'U rel" and Ps::"'U set set" and f::"'U rel \<Rightarrow> 'U set" and \<alpha>::"'U rel"
assumes a1: "Refl r \<and> \<not> finite r" and a2: "f \<in> \<N> r Ps"
    and a3: "\<omega>_ord \<le>o |\<LL> f \<alpha>|" and a4: "\<alpha> <o |Field r|"
shows "\<And> F. F \<in> SCF (Restr r (f \<alpha>)) \<Longrightarrow>
           \<forall> a \<in> \<W> r f \<alpha>. \<exists> b \<in> (F \<inter> (\<W> r f \<alpha>)). (a,b) \<in>
       (Restr r (\<W> r f \<alpha>))^*"
proof -
  fix F
  assume a5: "F \<in> SCF (Restr r (f \<alpha>))"
  show "\<forall> a \<in> (\<W> r f \<alpha>). \<exists> b \<in> (F \<inter> (\<W> r f \<alpha>)). (a,b) \<in> (
      Restr r (\<W> r f \<alpha>))^*"
  proof
    fix a
    assume b1: "a \<in> \<W> r f \<alpha>"
    have b2: "SF r = {A. A \<subseteq> Field r}" using a1 unfolding SF_def refl_on_def Field_def by fast
    moreover have "f \<alpha> \<subseteq> Field r"
      using a2 a4 unfolding \<N>_def \<N>5_def SF_def Field_def using ordLess_imp_ordLeq by blast
    ultimately have "\<forall>x \<in> f \<alpha>. \<exists>y \<in> f \<alpha> \<inter> F. (x, y) \<in> (Restr r
      (f \<alpha>))^*"
      using a5 unfolding SF_def SCF_def by blast
    then have b3: "\<forall>x \<in> \<Q> r f \<alpha>. \<exists>y \<in> (f \<alpha> \<inter> F \<inter> \<Q> r f
       \<alpha>). (x, y) \<in> (Restr r (\<Q> r f \<alpha>))^*"
      using lem_der_qinv3[of "(f \<alpha>) \<inter> F" f \<alpha> r] by blast
    have b4: "Restr r (\<Q> r f \<alpha>) \<in> \<UU> (Restr r (\<W> r f \<alpha>))"
      using a1 a2 a3 a4 lem_der_inf_qw_restr_uset[of r f Ps \<alpha>] by blast
    moreover have "a \<in> Field (Restr r (\<W> r f \<alpha>))"
    proof -
      have "\<W> r f \<alpha> \<subseteq> Field r" using a2 a4 lem_qw_range ordLess_imp_ordLeq by blast
      then have "\<W> r f \<alpha> \<in> SF r" using b2 by blast
      then show ?thesis using b1 unfolding SF_def by blast
    qed
    ultimately obtain a' where b5: "a' \<in> \<Q> r f \<alpha> \<and> (a, a') \<in> (Restr r (\<W> r f \<alpha>)
      )^*"
      unfolding \<UU>_def Field_def by blast
```

```
    then obtain b where b6: "b \<in> (f \<alpha> \<inter> F \<inter> \<Q> r f \<alpha>) \<and> (a', b) \<in> (
      Restr r (\<Q> r f \<alpha>))^*" using b3 by blast
    then have "b \<in> (F \<inter> (\<W> r f \<alpha>)) \<and> (a, b) \<in> (Restr r (\<W> r f \<alpha>))^*"
      using b5 lem_QS_subs_WS[of r f \<alpha>] rtrancl_mono[of "Restr r (\<Q> r f \<alpha>)" "Restr r (\<W> r f
      \<alpha>)"] by force
    then show "\<exists> b \<in> (F \<inter> (\<W> r f \<alpha>)). (a,b) \<in> (Restr r (\<W> r f \<alpha>))^*"
      by blast
  qed
qed

lemma lem_Nf_lewfbnd:
assumes a1: "f \<in> \<N> r Ps" and a2: "\<alpha> \<le>o |Field r|" and a3: "\<omega>_ord \<le>o |\<LL> f \<
      alpha>|"
shows "\<omega>_ord \<le>o \<alpha>"
proof -
  have "\<LL> f \<alpha> \<subseteq> f \<alpha>" using a1 a2 unfolding \<N>_def \<N>1_def \<LL>_def using
      ordLess_imp_ordLeq by blast
  then have "\<omega>_ord \<le>o |f \<alpha>|" using a3 by (metis card_of_mono1 ordLeq_transitive)
  moreover have "\<alpha> <o \<omega>_ord \<longrightarrow> |f \<alpha>| <o \<omega>_ord" using a1 a2 unfolding
      \<N>_def \<N>7_def by blast
  ultimately show ?thesis using a2 not_ordLess_ordLeq by force
qed

lemma lem_regcard_iso: "\<kappa> =o \<kappa>' \<Longrightarrow> regularCard \<kappa>' \<Longrightarrow>
      regularCard \<kappa>"
proof -
  assume a1: "\<kappa> =o \<kappa>'" and a2: "regularCard \<kappa>'"
  then obtain f where b1: "iso \<kappa> \<kappa>' f" unfolding ordIso_def by blast
  have "\<forall>K. K \<subseteq> Field \<kappa> \<and> cofinal K \<kappa> \<longrightarrow> |K| =o \<kappa>"
  proof (intro allI impI)
    fix K
    assume c1: "K \<subseteq> Field \<kappa> \<and> cofinal K \<kappa>"
    moreover then obtain K' where c2: "K' = f `` K" by blast
    ultimately have "K' \<subseteq> Field \<kappa>'" using b1 unfolding iso_def bij_betw_def by blast
    moreover have "cofinal K' \<kappa>'"
    proof -
      have "\<forall>a'\<in>Field \<kappa>'. \<exists>b'\<in>K'. a' \<noteq> b' \<and> (a', b') \<in> \<kappa>'"
      proof
        fix a'
        assume "a' \<in> Field \<kappa>'"
        then obtain a where e1: "a' = f a \<and> a \<in> Field \<kappa>" using b1 unfolding iso_def bij_betw_def
       by blast
        then obtain b where e2: "b \<in> K \<and> a \<noteq> b \<and> (a, b) \<in> \<kappa>" using c1 unfolding
      cofinal_def by blast
        then have "f b \<in> K'" using c2 by blast
        moreover have "a' \<noteq> f b" using e1 e2 c1 b1 unfolding iso_def bij_betw_def inj_on_def by blast
        moreover have "(a', f b) \<in> \<kappa>'"
        proof -
          have "(a,b) \<in> \<kappa>" using e2 by blast
          moreover have "embed \<kappa> \<kappa>' f" using b1 unfolding iso_def by blast
          ultimately have "(f a, f b) \<in> \<kappa>'" using compat_def embed_compat by metis
          then show ?thesis using e1 by blast
        qed
        ultimately show "\<exists>b'\<in>K'. a' \<noteq> b' \<and> (a', b') \<in> \<kappa>'" by blast
      qed
      then show ?thesis unfolding cofinal_def by blast
    qed
    ultimately have c3: "|K'| =o \<kappa>'" using a2 unfolding regularCard_def by blast
    have "inj_on f K" using c1 b1 unfolding iso_def bij_betw_def inj_on_def by blast
    then have "bij_betw f K K'" using c2 unfolding bij_betw_def by blast
    then have "|K| =o |K'|" using card_of_ordIsoI by blast
    then have "|K| =o \<kappa>'" using c3 ordIso_transitive by blast
    then show "|K| =o \<kappa>" using a1 ordIso_symmetric ordIso_transitive by blast
  qed
  then show "regularCard \<kappa>" unfolding regularCard_def by blast
qed

lemma lem_cardsuc_inf_gwreg: "\<not> finite A \<Longrightarrow> \<kappa> =o cardSuc |A| \<Longrightarrow> \<
      omega>_ord <o \<kappa> \<and> regularCard \<kappa>"
proof -
  assume a1: "\<not> finite A" and a2: "\<kappa> =o cardSuc |A|"
  moreover then have "regularCard (cardSuc |A| )" using infinite_cardSuc_regularCard by force
  ultimately have a3: "regularCard \<kappa>" using lem_regcard_iso ordIso_transitive by blast
  have "|A| <o cardSuc |A|" by simp
  then have "|A| <o \<kappa>" using a2 ordIso_symmetric ordLess_ordIso_trans by blast
  moreover have "\<omega>_ord \<le>o |A|" using a1 infinite_iff_natLeq_ordLeq by blast
  ultimately have "\<omega>_ord <o \<kappa>" using ordLeq_ordLess_trans by blast
  then show ?thesis using a3 by blast
```

```
qed

lemma lem_ccr_rcscf_struct:
fixes r::"'U rel"
assumes a1: "Refl r" and a2: "CCR r" and a3: "\<omega>_ord <o scf r" and a4: "regularCard (scf r)"
    and a5: "scf r =o |Field r|"
shows "\<exists> Ps. \<exists> f \<in> \<N> r Ps.
          \<forall>\<alpha>. \<omega>_ord \<le>o |\<LL> f \<alpha>| \<and> \<alpha> <o |Field r| \<and>
      isSuccOrd \<alpha> \<longrightarrow>
          CCR (Restr r (\<W> r f \<alpha>)) \<and> |Restr r (\<W> r f \<alpha>)| <o |Field r|
        \<and> (\<forall>a \<in> \<W> r f \<alpha>. wesc_rel r f \<alpha> a (wesc r f \<alpha> a))"
proof -
  obtain P where b1: "P \<in> SCF r"
             and b2: "\<forall>\<alpha>::'U rel. \<alpha> <o scf r \<longrightarrow> (\<forall>a \<in> P. \<
      alpha> <o wrank r (r``{a}))"
    using a2 a3 a4 lem_wnb_P_ncl_reg_grw[of r] by blast
  then obtain f where b3: "f \<in> \<N> r {P}" using a1 a2 lem_Shinf_N_ne[of r "{P}"] by blast
  moreover have "\<forall>\<alpha>. \<omega>_ord \<le>o |\<LL> f \<alpha>| \<and> \<alpha> <o |Field r| \<and>
      (\<alpha> = {} \<or> isSuccOrd \<alpha>) \<longrightarrow>
          CCR (Restr r (\<W> r f \<alpha>)) \<and> |Restr r (\<W> r f \<alpha>)| <o |Field r|
        \<and> (\<forall>a \<in> \<W> r f \<alpha>. wesc_rel r f \<alpha> a (wesc r f \<alpha> a))"
  proof (intro allI impI)
    fix \<alpha>
    assume c1: "\<omega>_ord \<le>o |\<LL> f \<alpha>| \<and> \<alpha> <o |Field r| \<and> (\<alpha> = {} \<or>
      isSuccOrd \<alpha>)"
    then have c2: "(f \<alpha> \<inter> P) \<in> SCF (Restr r (f \<alpha>))"
      using b3 unfolding \<N>_def \<N>8_def using ordLess_imp_ordLeq by blast
    have c3: "\<not> finite r" using a2 a3 lem_scfgew_ncl lem_scf_ccr_scf_uset[of r]
      unfolding \<UU>_def using ordLess_imp_ordLeq finite_subset[of _ r] by blast
    have "CCR (Restr r (\<W> r f \<alpha>))" using c1 c3 b3 a1 lem_der_inf_qw_restr_ccr[of r f "{P}" \<alpha>]
      by blast
    moreover have "|Restr r (\<W> r f \<alpha>)| <o |Field r|" using c1 c3 b3 lem_der_inf_qw_restr_card[of r f
      "{P}" \<alpha>] by blast
    moreover have "\<forall>a \<in> \<W> r f \<alpha>. wesc_rel r f \<alpha> a (wesc r f \<alpha> a)"
    proof
      fix a
      assume "a \<in> \<W> r f \<alpha>"
      then obtain b where d1: "b \<in> (P \<inter> (\<W> r f \<alpha>))" and d2: "(a,b) \<in> (Restr r (\<W> r f
       \<alpha>))^*"
        using c1 c2 c3 b3 a1 lem_Wf_esc_pth[of r f "{P}" \<alpha> "f \<alpha> \<inter> P"] by blast
      moreover then have "b \<in> (f \<alpha>) \<inter> P" unfolding \<W>_def by blast
      moreover have "\<omega>_ord \<le>o \<alpha>" using c1 b3 lem_Nf_lewfbnd[of f r "{P}" \<alpha>]
      ordLess_imp_ordLeq by blast
      ultimately have "\<forall> \<beta>. \<alpha> <o \<beta> \<and> \<beta> <o |Field r| \<and> (\<beta> = {}
      \<or> isSuccOrd \<beta>) \<longrightarrow> r `` {b} \<inter> \<W> r f \<beta> \<noteq> {}"
        using b2 b3 a5 lem_Wf_ext_arc[of r f "{P}" P \<alpha> b] by blast
      then have "wesc_rel r f \<alpha> a b" using d1 d2 unfolding wesc_rel_def by blast
      then have "\<exists> b. wesc_rel r f \<alpha> a b" by blast
      then show "wesc_rel r f \<alpha> a (wesc r f \<alpha> a)"
        using someI_ex[of "\<lambda> b. wesc_rel r f \<alpha> a b"] unfolding wesc_def by blast
    qed
    ultimately show "CCR (Restr r (\<W> r f \<alpha>))
            \<and> |Restr r (\<W> r f \<alpha>)| <o |Field r|
            \<and> (\<forall>a \<in> \<W> r f \<alpha>. wesc_rel r f \<alpha> a (wesc r f \<alpha> a))" by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_oint_infcard_sc_cf:
fixes \<alpha>0::"'a rel" and \<kappa>::"'U rel" and S::"'U rel set"
assumes a1: "Card_order \<kappa>" and a2: "\<omega>_ord \<le>o \<kappa>"
    and a3: "S = {\<alpha> \<in> \<O>::'U rel set. \<alpha>0 \<le>o \<alpha> \<and> isSuccOrd \<alpha> \<and> \<
      alpha> <o \<kappa>}"
shows "\<forall> \<alpha> \<in> S. \<exists> \<beta> \<in> S. \<alpha> <o \<beta>"
proof
  fix \<alpha>
  assume b1: "\<alpha> \<in> S"
  then have "\<alpha> <o \<kappa>" using a3 by blast
  then obtain \<beta> where b2: "sc_ord \<alpha> \<beta>" using lem_sucord_ex by blast
  obtain \<beta>' where b3: "\<beta>' = nord \<beta>" by blast
  have b4: "isSuccOrd \<beta>" using b2 unfolding sc_ord_def using lem_ordint_sucord by blast
  moreover have "\<beta> =o \<beta>'" using b2 b3 lem_nord_l unfolding sc_ord_def ordLess_def by blast
  ultimately have "isSuccOrd \<beta>'" using lem_osucc_eq by blast
  moreover have "\<beta>' \<in> \<O>" using b2 b3 lem_nordO_ls_r unfolding sc_ord_def by blast
  moreover have "\<alpha>0 \<le>o \<beta>'" using b1 b2 b3 a3 unfolding sc_ord_def
    using lem_nord_le_r ordLeq_ordLess_trans ordLess_imp_ordLeq by blast
  moreover have "\<beta>' <o \<kappa>"
  proof -
```

```
    have "\<beta> \<le>o \<kappa>" using b1 b2 a3 unfolding sc_ord_def by blast
    moreover have "\<beta> =o \<kappa> \<longrightarrow> False"
    proof
      assume "\<beta> =o \<kappa>"
      then have "isSuccOrd \<kappa>" using b4 lem_osucc_eq by blast
      moreover have "isLimOrd \<kappa>" using a1 a2 lem_ge_w_inford by (metis card_order_infinite_isLimOrd)
      moreover have "Well_order \<kappa>" using a1 unfolding card_order_on_def by blast
      ultimately show "False" using wo_rel.isLimOrd_def unfolding wo_rel_def by blast
    qed
    ultimately have "\<beta> <o \<kappa>" using ordLeq_iff_ordLess_or_ordIso by blast
    then show ?thesis using b3 lem_nord_ls_l by blast
  qed
  moreover have "\<alpha> <o \<beta>'" using b2 b3 lem_nord_ls_r unfolding sc_ord_def by blast
  ultimately have "\<beta>' \<in> S \<and> \<alpha> <o \<beta>'" using a3 by blast
  then show "\<exists> \<beta> \<in> S. \<alpha> <o \<beta>" by blast
qed

lemma lem_oint_infcard_gew_sc_cfbnd:
fixes \<alpha>0::"'a rel" and \<kappa>::"'U rel" and S::"'U rel set"
assumes a1: "Card_order \<kappa>" and a2: "\<omega>_ord \<le>o \<kappa>"  and a3: "\<alpha>0 <o \<kappa>" and a4
      : "\<alpha>0 =o \<omega>_ord"
    and a5: "S = {\<alpha> \<in> \<O>::'U rel set. \<alpha>0 \<le>o \<alpha> \<and> isSuccOrd \<alpha> \<and> \<
      alpha> <o \<kappa>}"
shows "|{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |S|
    \<and> (\<exists> f. (\<forall> \<alpha> \<in> \<O>::'U rel set. \<alpha>0 \<le>o \<alpha> \<and> \<alpha> <
      o \<kappa> \<longrightarrow> \<alpha> \<le>o f \<alpha> \<and> f \<alpha> \<in> S))"
proof -
  have "|UNIV::nat set| <o \<kappa>" using a3 a4 by (meson card_of_nat ordIso_ordLess_trans ordIso_symmetric)
  then obtain N where "N \<subseteq> Field \<kappa> \<and> |UNIV::nat set| =o |N|"
    using internalize_card_of_ordLess[of "UNIV::nat set" \<kappa>] by force
  moreover obtain \<alpha>0'::"'U rel" where "\<alpha>0' = |N|" by blast
  ultimately have b0: "\<alpha>0' =o \<omega>_ord" using card_of_nat ordIso_symmetric ordIso_transitive by blast
  then have b0': "\<alpha>0' <o \<kappa>" using a3 a4 ordIso_symmetric ordIso_ordLess_trans by metis
  have b0'': "\<alpha>0 =o \<alpha>0'" using b0 a4 ordIso_symmetric ordIso_transitive by blast
  obtain S1 where b1: "S1 = {\<alpha> \<in> \<O>::'U rel set. \<alpha>0 \<le>o \<alpha> \<and> \<alpha> <o \<
      kappa>}" by blast
  obtain f where "f = (\<lambda>\<alpha>::'U rel. SOME \<beta>. sc_ord \<alpha> \<beta>)" by blast
  moreover have "\<forall> \<alpha> \<in> S1. \<exists> \<beta>. sc_ord \<alpha> \<beta>" using b1 lem_sucord_ex
       by blast
  ultimately have b2: "\<And> \<alpha>. \<alpha> \<in> S1 \<Longrightarrow> sc_ord \<alpha> (f \<alpha>)" using
      someI_ex by metis
  have b3: "(nord \<circ> f) ` S1 \<subseteq> S"
  proof
    fix \<alpha>
    assume "\<alpha> \<in> (nord \<circ> f) ` S1"
    then obtain \<alpha>' where c1: "\<alpha>' \<in> S1 \<and> \<alpha> = nord (f \<alpha>')" by force
    then have c2: "sc_ord \<alpha>' (f \<alpha>')" using b2 by blast
    then have c3: "isSuccOrd (f \<alpha>')" unfolding sc_ord_def using lem_ordint_sucord by blast
    moreover have "f \<alpha>' =o \<alpha>" using c1 c2 lem_nord_l unfolding sc_ord_def ordLess_def by blast
    ultimately have c4: "isSuccOrd \<alpha>" using lem_osucc_eq by blast
    have "\<alpha>0 \<le>o \<alpha>' \<and> \<alpha>' <o \<kappa>" using c1 b1 by blast
    then have c5: "\<alpha>0 \<le>o (f \<alpha>') \<and> (f \<alpha>') \<le>o \<kappa>"
      using c1 b2 unfolding sc_ord_def using ordLeq_ordLess_trans ordLess_imp_ordLeq by blast
    then have c6: "\<alpha>0 \<le>o \<alpha>" using c1 lem_nord_le_r by blast
    have c7: "\<alpha> \<in> \<O>" using c1 c2 lem_nord0_ls_r unfolding sc_ord_def by blast
    have "(f \<alpha>') =o \<kappa> \<longrightarrow> False"
    proof
      assume "(f \<alpha>') =o \<kappa>"
      then have "isSuccOrd \<kappa>" using c3 lem_osucc_eq by blast
      moreover have "isLimOrd \<kappa>" using a1 a2 lem_ge_w_inford by (metis card_order_infinite_isLimOrd)
      moreover have "Well_order \<kappa>" using a1 unfolding card_order_on_def by blast
      ultimately show "False" using wo_rel.isLimOrd_def unfolding wo_rel_def by blast
    qed
    then have "f \<alpha>' <o \<kappa>" using c5 using ordLeq_iff_ordLess_or_ordIso by blast
    then have "\<alpha> <o \<kappa>" using c1 lem_nord_ls_l by blast
    then show "\<alpha> \<in> S" using c4 c6 c7 a5 by blast
  qed
  moreover have "inj_on (nord \<circ> f) S1"
  proof -
    have "\<forall>\<alpha>\<in>S1. \<forall>\<beta>\<in>S1. (nord \<circ> f) \<alpha> = (nord \<circ> f) \<beta
      > \<longrightarrow> \<alpha> = \<beta>"
    proof (intro ballI impI)
      fix \<alpha> \<beta>
      assume d1: "\<alpha> \<in> S1" and d2: "\<beta> \<in> S1" and "(nord \<circ> f) \<alpha> = (nord \<circ> f
      ) \<beta>"
      then have "nord (f \<alpha>) = nord (f \<beta>)" by simp
      moreover have "Well_order (f \<alpha>) \<and> Well_order (f \<beta>)"
        using d1 d2 b2 unfolding sc_ord_def ordLess_def by blast
```

```
    ultimately have d3: "f \<alpha> =o f \<beta>" using lem_nord_req by blast
    have d4: "sc_ord \<alpha> (f \<alpha>) \<and> sc_ord \<beta> (f \<beta>)" using d1 d2 b2 by blast
    have "Well_order \<alpha> \<and> Well_order \<beta>" using d1 d2 b1 unfolding ordLess_def by blast
    moreover have "\<alpha> <o \<beta> \<longrightarrow> False"
    proof
      assume "\<alpha> <o \<beta>"
      then have "f \<alpha> \<le>o \<beta> \<and> \<beta> <o f \<beta>" using d4 unfolding sc_ord_def by blast
      then show "False" using d3 using not_ordLess_ordIso ordLeq_ordLess_trans by blast
    qed
    moreover have "\<beta> <o \<alpha> \<longrightarrow> False"
    proof
      assume "\<beta> <o \<alpha>"
      then have "f \<beta> \<le>o \<alpha> \<and> \<alpha> <o f \<alpha>" using d4 unfolding sc_ord_def by
    blast
      then show "False" using d3 using not_ordLess_ordIso ordLeq_ordLess_trans ordIso_symmetric by blast
    qed
    ultimately have "\<alpha> =o \<beta>" using ordIso_or_ordLess by blast
    then show "\<alpha> = \<beta>" using d1 d2 b1 lem_0eq by blast
  qed
  then show ?thesis unfolding inj_on_def by blast
qed
ultimately have b4: "|S1| \<le>o |S|" using card_of_ordLeq by blast
obtain S2 where b5: "S2 = { \<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<alpha>0 }" by blast
have b6: "|UNIV::nat set| \<le>o |S1|"
proof -
  obtain xi where c1: "xi = (\<lambda> i::nat. ((nord \<circ> f)^^i) (nord \<alpha>0'))" by blast
  have c2: "\<forall> i. xi i \<in> S1"
  proof
    fix i0
    show "xi i0 \<in> S1"
    proof (induct i0)
      have "\<alpha>0' \<le>o nord \<alpha>0'"
        using b0' lem_nord_l unfolding ordLess_def using ordIso_iff_ordLeq by blast
      then have "\<alpha>0 \<le>o nord \<alpha>0'" using b0'' ordIso_ordLeq_trans by blast
      moreover then have "nord \<alpha>0' <o \<kappa> \<and> nord \<alpha>0' \<in> \<O>"
        using b0' lem_nordO_ls_l lem_nord_ls_l ordLeq_ordLess_trans by blast
      ultimately show "xi 0 \<in> S1" using c1 b1 by simp
    next
      fix i
      assume "xi i \<in> S1"
      then have "(nord \<circ> f) (xi i) \<in> S" using b3 by blast
      then show "xi (Suc i) \<in> S1" using c1 b1 a5 by simp
    qed
  qed
  have c3: "\<forall> j. \<forall> i<j. xi i <o xi j"
  proof
    fix j0
    show "\<forall>i<j0. xi i <o xi j0"
    proof (induct j0)
      show "\<forall>i<0. xi i <o xi 0" by blast
    next
      fix j
      assume e1: "\<forall>i<j. xi i <o xi j"
      show "\<forall>i<Suc j. xi i <o xi (Suc j)"
      proof(intro allI impI)
        fix i
        assume f1: "i < Suc j"
        have "xi j <o nord (f (xi j))" using c2 b2 unfolding sc_ord_def using lem_nord_ls_r by blast
        then have "xi j <o xi (Suc j)" using c1 by simp
        moreover then have "i < j \<longrightarrow> xi i <o xi (Suc j)" and "i = j \<longrightarrow> xi i <o
    xi (Suc j)"
          using e1 ordLess_transitive by blast+
        moreover have "i < j \<or> i = j" using f1 by force
        ultimately show "xi i <o xi (Suc j)" by blast
      qed
    qed
  qed
  then have "\<forall> i j. xi i = xi j \<longrightarrow> i = j" by (metis linorder_neqE_nat
    ordLess_irreflexive)
  then have "inj xi" unfolding inj_on_def by blast
  moreover have "xi ` UNIV \<subseteq> S1" using c2 by blast
  ultimately show "|UNIV::nat set| \<le>o |S1|" using card_of_ordLeq by blast
qed
then have "\<not> finite S1" using infinite_iff_card_of_nat by blast
moreover have "|S1| \<le>o |S2| \<or> |S2| \<le>o |S1|"
  using card_of_Well_order ordLess_imp_ordLeq ordLess_or_ordLeq by blast
ultimately have "|S1 \<union> S2| \<le>o |S1| \<or> |S1 \<union> S2| \<le>o |S2|"
  by (metis card_of_Un1 card_of_Un_ordLeq_infinite card_of_ordLeq_finite sup.idem)
```

```
  moreover have "|S2| \<le>o |S|"
  proof -
    have "|UNIV::nat set| \<le>o |S|" using b4 b6 ordLeq_transitive by blast
    moreover have "|S2| \<le>o |UNIV::nat set|"
    proof -
      have "\<forall> \<alpha> \<in> S2. \<alpha> <o \<omega>_ord \<and> \<alpha> \<in> \<O>" using b5 a4
       ordLess_ordIso_trans by blast
      then have d1: "\<forall> \<alpha> \<in> S2. \<alpha> =o natLeq_on (card (Field \<alpha>)) \<and> \<alpha>
       \<in> \<O>" using lem_wolew_nat by blast
      obtain A where d2: "A = natLeq_on ` UNIV" by blast
      moreover obtain f where d3: "f = (\<lambda> \<alpha>::'U rel. natLeq_on (card (Field \<alpha>)))" by blast
      ultimately have "f ` UNIV \<subseteq> A" by force
      moreover have "inj_on f S2"
      proof -
        have "\<forall> \<alpha> \<in> S2. \<forall> \<beta> \<in> S2. f \<alpha> = f \<beta> \<longrightarrow>
       \<alpha> = \<beta>"
          proof (intro ballI impI)
            fix \<alpha> \<beta>
            assume "\<alpha> \<in> S2" and "\<beta> \<in> S2" and "f \<alpha> = f \<beta>"
            then have "\<alpha> =o natLeq_on (card (Field \<alpha>))" and "\<beta> =o natLeq_on (card (Field \<
       beta>))"
              and "natLeq_on (card (Field \<alpha>)) = natLeq_on (card (Field \<beta>))"
              and "\<alpha> \<in> \<O> \<and> \<beta> \<in> \<O>" using d1 d3 by blast+
            moreover then have "\<alpha> =o \<beta>"
              by (metis (no_types, lifting) ordIso_symmetric ordIso_transitive)
            ultimately show "\<alpha> = \<beta>" using lem_Oeq by blast
          qed
          then show ?thesis unfolding inj_on_def by blast
        qed
        ultimately have "|S2| \<le>o |A|" using card_of_ordLeq[of S2 A] by blast
        moreover have "|A| \<le>o |UNIV::nat set|" using d2 by simp
        ultimately show ?thesis using ordLeq_transitive by blast
      qed
      ultimately show ?thesis using ordLeq_transitive by blast
    qed
    ultimately have b7: "|S1 \<union> S2| \<le>o |S|" using b4 ordLeq_transitive by blast
    have "{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>} \<subseteq> S1 \<union> S2" using b1 b5 a1 a3 by
         fastforce
    then have "|{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |S1 \<union> S2|" by simp
    moreover have "\<forall> \<alpha> \<in> \<O>::'U rel set. \<alpha>0 \<le>o \<alpha> \<and> \<alpha> <o \<kappa
        > \<longrightarrow> \<alpha> \<le>o (nord \<circ> f) \<alpha> \<and> (nord \<circ> f) \<alpha> \<in> S"
    proof (intro ballI impI)
      fix \<alpha>::"'U rel"
      assume c1: "\<alpha> \<in> \<O>" and c2: "\<alpha>0 \<le>o \<alpha> \<and> \<alpha> <o \<kappa>"
      then have c3: "(nord \<circ> f) \<alpha> \<in> S" using b1 b3 by blast
      moreover have "\<alpha> <o f \<alpha>" using c1 c2 b1 b2[of \<alpha>] unfolding sc_ord_def by blast
      then have "\<alpha> \<le>o f \<alpha>" using ordLess_imp_ordLeq by blast
      then have "\<alpha> \<le>o (nord \<circ> f) \<alpha>" using lem_nord_le_r by simp
      then show "\<alpha> \<le>o (nord \<circ> f) \<alpha> \<and> (nord \<circ> f) \<alpha> \<in> S" using c3 by
        blast
    qed
    ultimately show ?thesis using b7 ordLeq_transitive by blast
  qed

  lemma lem_rcc_uset_rcc_bnd:
  assumes "s \<in> \<UU> r"
  shows "\<parallel>r\<parallel> \<le>o \<parallel>s\<parallel>"
  proof -
    obtain s0 where b1: "s0 \<in> \<UU> r \<and> |s0| =o \<parallel>r\<parallel> \<and> |s0| \<le>o |s| \<and> (
        \<forall> s' \<in> \<UU> r. |s0| \<le>o |s'| )"
      using assms lem_rcc_uset_ne by blast
    have "CCR s" using assms unfolding \<UU>_def by blast
    then obtain t where b2: "t \<in> \<UU> s \<and> |t| =o \<parallel>s\<parallel> \<and> ( \<forall> s' \<in> \<
        UU> s. |t| \<le>o |s'| )"
      using lem_Rcc_eq1_12 lem_rcc_uset_ne by blast
    have "t \<in> \<UU> r" using b2 assms lem_rcc_uset_tr by blast
    then have "\<parallel>r\<parallel> \<le>o |t|" using lem_rcc_uset_mem_bnd by blast
    then show "\<parallel>r\<parallel> \<le>o \<parallel>s\<parallel>" using b2 ordLeq_ordIso_trans by blast
  qed

  lemma lem_dc2_ccr_scf_lew:
  fixes r::"'U rel"
  assumes a1: "CCR r" and a2: "scf r \<le>o \<omega>_ord"
  shows "DCR 2 r"
  proof -
    have "\<exists> s. s \<in> \<UU> r \<and> single_valued s"
    proof (cases "scf r <o \<omega>_ord")
      assume "scf r <o \<omega>_ord"
```

```
    then have b1: "Conelike r" using a1 lem_scf_ccr_finscf_cl lem_fin_fl_rel lem_wolew_fin by blast
    show ?thesis
    proof (cases "r = {}")
      assume "r = {}"
      then have "r \<in> \<UU> r \<and> single_valued r"
        unfolding \<UU>_def CCR_def single_valued_def Field_def by blast
      then show ?thesis by blast
    next
      assume "r \<noteq> {}"
      then obtain m where c2: "m \<in> Field r \<and> (\<forall> a \<in> Field r. (a,m) \<in> r^*)"
        using b1 unfolding Conelike_def by blast
      then obtain a b where "(a,b) \<in> r \<and> (m = a \<or> m = b)" unfolding Field_def by blast
      moreover obtain s where "s = {(a,b)}" by blast
      ultimately have "s \<in> \<UU> r" and "single_valued s"
        using c2 unfolding \<UU>_def CCR_def Field_def single_valued_def by blast+
      then show ?thesis by blast
    qed
  next
    assume "\<not> (scf r <o \<omega>_ord)"
    then have "scf r =o \<omega>_ord" using a2 ordLeq_iff_ordLess_or_ordIso by blast
    then obtain s where b1: "s \<in> Span r" and b2: "CCR s" and b3: "single_valued s"
      using a1 lem_sv_span_scfeqw by blast
    then have "s \<in> \<UU> r \<and> single_valued s" unfolding Span_def \<UU>_def by blast
    then show ?thesis by blast
  qed
  then obtain s where b1: "s \<in> \<UU> r \<and> single_valued s" by blast
  moreover have "DCR 1 s"
  proof -
    obtain g where "g = (\<lambda> \<alpha>::nat. s)" by blast
    moreover then have "DCR_generating g"
      using b1 unfolding \<DD>_def single_valued_def DCR_generating_def by blast
    ultimately show ?thesis unfolding DCR_def by blast
  qed
  ultimately have "DCR (Suc 1) r" using lem_Ldo_uset_reduc[of s r 1] by fastforce
  moreover have "(Suc 1) = (2::nat)" by simp
  ultimately show ?thesis by metis
qed

lemma lem_dc3_ccr_refl_scf_wsuc:
fixes r::"'U rel"
assumes a1: "Refl r" and a2: "CCR r"
    and a3: "|Field r| =o cardSuc |UNIV::nat set|" and a4: "scf r =o |Field r|"
shows "DCR 3 r"
proof -
  obtain \<kappa>::"'U rel" where b0: "\<kappa> = |Field r|" by blast
  have b1: "\<omega>_ord <o (scf r) \<and> regularCard (scf r)"
   and b2: "\<omega>_ord <o |Field r|"
    using a3 a4 lem_cardsuc_inf_gwreg ordIso_transitive by blast+
  then obtain Ps f
      where b3: "f \<in> \<N> r Ps"
        and b4: "\<And>\<alpha>. \<omega>_ord \<le>o |\<LL> f \<alpha>| \<and> \<alpha> <o \<kappa> \<and>
      isSuccOrd \<alpha> \<Longrightarrow>
                  CCR (Restr r (\<W> r f \<alpha>)) \<and> |Restr r (\<W> r f \<alpha>)| <o \<kappa>
                \<and> (\<forall>a \<in> \<W> r f \<alpha>. wesc_rel r f \<alpha> a (wesc r f \<alpha> a))"
    using b0 a1 a2 a4 lem_ccr_rcscf_struct by blast
  have q0: "\<And> \<alpha>. \<omega>_ord \<le>o \<alpha> \<and> \<alpha> <o \<kappa> \<and> isSuccOrd \<alpha>
      \<Longrightarrow> \<not> Conelike (Restr r (f \<alpha>))"
  proof -
    fix \<alpha>::"'U rel"
    assume "\<omega>_ord \<le>o \<alpha> \<and> \<alpha> <o \<kappa> \<and> isSuccOrd \<alpha>"
    then have "Conelike (Restr r (f \<alpha>)) \<longrightarrow> Conelike r"
      using b3 b0 unfolding \<N>_def \<N>3_def \<N>12_def clterm_def using ordLess_imp_ordLeq by blast
    moreover have "Conelike r \<longrightarrow> False"
    proof
      assume "Conelike r"
      then have "finite (Field (scf r))" using a2 lem_scf_ccr_finscf_cl by blast
      then show "False" using b2 a4
        by (metis Field_card_of infinite_iff_natLeq_ordLeq ordIso_finite_Field ordLess_imp_ordLeq)
    qed
    ultimately show "\<not> Conelike (Restr r (f \<alpha>))" by blast
  qed
  have q1: "\<And> \<alpha>. \<omega>_ord \<le>o \<alpha> \<and> \<alpha> <o \<kappa> \<and> isSuccOrd \<alpha>
      \<Longrightarrow>
                \<omega>_ord \<le>o |\<LL> f \<alpha>| \<and> scf (Restr r (f \<alpha>)) =o \<omega>_ord"
  proof -
    fix \<alpha>::"'U rel"
    assume c1: "\<omega>_ord \<le>o \<alpha> \<and> \<alpha> <o \<kappa> \<and> isSuccOrd \<alpha>"
    have "Card_order \<omega>_ord \<and> \<not>finite (Field \<omega>_ord) \<and> Well_order \<omega>_ord"
```

```
    using natLeq_Card_order Field_natLeq by force
  then have "\<not> isSuccOrd \<omega>_ord"
    using card_order_infinite_isLimOrd wo_rel.isLimOrd_def wo_rel_def by blast
  then have "\<omega>_ord <o \<alpha>" using c1 using lem_osucc_eq ordIso_symmetric
    ordLeq_iff_ordLess_or_ordIso by blast
  then obtain \<alpha>0::"'U rel" where c2: "\<omega>_ord =o \<alpha>0 \<and> \<alpha>0 <o \<alpha>" using
    internalize_ordLess[of \<omega>_ord \<alpha>] by blast
  then have c3: "f \<alpha>0 \<subseteq> \<LL> f \<alpha>" unfolding \<LL>_def by blast
  obtain \<gamma> where c4: "\<gamma> = scf (Restr r (f \<alpha>))" by blast
  have "\<not> Conelike (Restr r (f \<alpha>))" using c1 q0 by blast
  moreover have "CCR (Restr r (f \<alpha>))" using c1 b0 b3 unfolding \<N>_def \<N>6_def
    using ordLess_imp_ordLeq by blast
  ultimately have "Card_order \<gamma> \<and> \<not> finite (Field \<gamma>)" and c5: "\<not> finite (Restr r
    (f \<alpha>))"
    using c4 lem_scf_ccr_finscf_cl lem_scf_cardord lem_Relprop_fin_ccr by blast+
  then have c6: "\<omega>_ord \<le>o \<gamma>"
    by (meson card_of_Field_ordIso infinite_iff_natLeq_ordLeq ordIso_iff_ordLeq ordLeq_transitive)
  have "\<omega>_ord \<le>o |\<LL> f \<alpha>|" using c1 b0 b3 unfolding \<N>_def \<N>12_def using
    ordLess_imp_ordLeq by blast
  moreover have "scf (Restr r (f \<alpha>)) =o \<omega>_ord"
  proof -
    have "|f \<alpha>| \<le>o \<alpha>" using c1 b0 b3 unfolding \<N>_def \<N>7_def using ordLess_imp_ordLeq
    by blast
    then have "|Restr r (f \<alpha>)| \<le>o \<alpha>" using c1 lem_restr_ordbnd by blast
    then have "\<gamma> \<le>o \<alpha>" using c4 c5 lem_rel_inf_fld_card[of "Restr r (f \<alpha>)"]
      lem_scf_relfldcard_bnd ordLeq_ordIso_trans ordLeq_transitive by blast
    then have "\<gamma> <o cardSuc |UNIV::nat set|" using c1 b0 a3
      using ordIso_iff_ordLeq ordLeq_ordLess_trans ordLess_ordLeq_trans by blast
    moreover have "Card_order \<gamma>" using c4 lem_scf_cardord by blast
    ultimately have "\<gamma> \<le>o |UNIV::nat set|" by simp
    then show ?thesis using c4 c6 using card_of_nat ordIso_iff_ordLeq ordLeq_ordIso_trans by blast
  qed
  ultimately show "\<omega>_ord \<le>o |\<LL> f \<alpha>| \<and> scf (Restr r (f \<alpha>)) =o \<omega>_ord"
    by blast
qed
obtain is_st::"'U rel \<Rightarrow> 'U rel \<Rightarrow> bool"
  where q3: "is_st = (\<lambda> s t. t \<in> Span s \<and> t \<noteq> {} \<and> CCR t \<and>
                    single_valued t \<and> acyclic t \<and> (\<forall>x\<in>Field t. t``{x} \<noteq> {}))"
    by blast
obtain st where q4: "st = (\<lambda> s::'U rel. SOME t. is_st s t)" by blast
have q5: "\<And> s. CCR s \<and> scf s =o \<omega>_ord \<Longrightarrow> is_st s (st s)"
proof -
  fix s::"'U rel"
  assume "CCR s \<and> scf s =o \<omega>_ord"
  then obtain t where "is_st s t" using q3 lem_sv_span_scfeqw[of s] by blast
  then show "is_st s (st s)" using q4 someI_ex by metis
qed
obtain \<kappa>0 where b5: "\<kappa>0 = \<omega>_ord" by blast
obtain S where b6: "S = {\<alpha> \<in> \<O>::'U rel set. \<kappa>0 \<le>o \<alpha> \<and> isSuccOrd \<alpha>
    \<and> \<alpha> <o \<kappa>}" by blast
obtain R where b8: "R = (\<lambda> \<alpha>. st (Restr r (\<W> r f \<alpha>)))" by blast
obtain T::"'U rel set" where b11: "T = { t. t \<noteq> {} \<and> CCR t \<and> single_valued t \<and>
                                      acyclic t \<and> (\<forall>x\<in>Field t. t``{x} \<noteq> {}) }" by
    blast
obtain W::"'U rel \<Rightarrow> 'U set" where b12: "W = (\<lambda> \<alpha>. \<W> r f \<alpha>)" by blast
obtain Wa where b13: "Wa = (\<Union>\<alpha>\<in>S. W \<alpha>)" by blast
obtain r1 where b14: "r1 = Restr r Wa" by blast
have b15: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> Restr r (\<W> r f \<alpha>) = Restr r1 (W \<
    alpha>)" using b12 b13 b14 by blast
have b16: "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> Restr r (\<W> r f \<alpha>) \<in> \<UU> (Restr
    r (f \<alpha>))"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  have d1: "\<not> finite r" using b2 lem_fin_fl_rel by (metis infinite_iff_natLeq_ordLeq ordLess_imp_ordLeq)
  moreover have "\<alpha> <o scf r" using c1 b0 b6 a4 using ordIso_symmetric ordLess_ordIso_trans by blast
  moreover have "\<omega>_ord \<le>o |\<LL> f \<alpha>|" using c1 b5 b6 q1 by blast
  moreover have "isSuccOrd \<alpha>" using c1 b6 by blast
  ultimately show "Restr r (\<W> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))"
    using b3 a1 a2 lem_der_qw_uset[of r f Ps \<alpha>] by blast
qed
have "\<kappa> =o cardSuc |UNIV::nat set|" using b0 a3 by blast
moreover have "Refl r1" using a1 b14 unfolding refl_on_def Field_def by blast
moreover have "S \<subseteq> {\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}" using b6 by blast
moreover have b17: "|{\<alpha> \<in> \<O>::'U rel set. \<alpha> <o \<kappa>}| \<le>o |S|
            \<and> (\<exists>h. \<forall>\<alpha>\<in>\<O>::'U rel set. \<kappa>0 \<le>o \<alpha> \<and> \<
    alpha> <o \<kappa> \<longrightarrow> \<alpha> \<le>o h \<alpha> \<and> h \<alpha> \<in> S)"
proof -
```

```
  have "Card_order \<kappa>" using b0 by simp
  moreover have "\<omega>_ord \<le>o \<kappa>" using b0 b2 ordLess_imp_ordLeq by blast
  moreover have "\<kappa>0 <o \<kappa>" using b0 b2 b5 by blast
  moreover have "\<kappa>0 =o \<omega>_ord" using b5 ordIso_refl natLeq_Card_order by blast
  ultimately show ?thesis using b6 lem_oint_infcard_gew_sc_cfbnd[of \<kappa> \<kappa>0 S] by blast
qed
moreover have "\<forall> \<alpha> \<in> S. \<exists> \<beta> \<in> S. \<alpha> <o \<beta>"
proof -
  have "Card_order \<kappa>" using b0 by simp
  moreover have "\<omega>_ord \<le>o \<kappa>" using b0 b2 ordLess_imp_ordLeq by blast
  ultimately show ?thesis using b6 lem_oint_infcard_sc_cf[of \<kappa> S \<kappa>0] by blast
qed
moreover have b18: "Field r1 = (\<Union>\<alpha>\<in>S. W \<alpha>)"
proof -
  have "SF r = {A. A \<subseteq> Field r}" using a1 unfolding SF_def Field_def refl_on_def by fast
  moreover have "Wa \<subseteq> Field r"
    using b0 b3 b6 b12 b13 lem_qw_range[of f r Ps _] ordLess_imp_ordLeq[of _ \<kappa>] by blast
  ultimately have "Field r1 = Wa" using b14 unfolding SF_def by blast
  then show ?thesis using b13 by blast
qed
moreover have "\<forall>\<alpha>\<in>S. \<forall> \<beta>\<in>S. \<alpha> \<noteq> \<beta> \<longrightarrow> W
     \<alpha> \<inter> W \<beta> = {}"
proof (intro ballI impI)
  fix \<alpha> \<beta>
  assume "\<alpha> \<in> S" and "\<beta> \<in> S" and "\<alpha> \<noteq> \<beta>"
  then have "Well_order \<alpha> \<and> Well_order \<beta> \<and> \<not> (\<alpha> =o \<beta>)" using b6
    lem_Owo lem_Oeq by blast
  then show "W \<alpha> \<inter> W \<beta> = {}" using b12 lem_Der_inf_qw_disj by blast
qed
moreover have "\<And> \<alpha>. \<alpha> \<in> S \<Longrightarrow> R \<alpha> \<in> T \<and> R \<alpha> \<
    subseteq> Restr r1 (W \<alpha>) \<and> |W \<alpha>| \<le>o |UNIV::nat set|
                              \<and> Field (R \<alpha>) = W \<alpha> \<and> \<not> Conelike (Restr r1 (W \<
    alpha>))"
proof -
  fix \<alpha>
  assume c1: "\<alpha> \<in> S"
  then have c2: "CCR (Restr r (\<W> r f \<alpha>)) \<and> scf (Restr r (f \<alpha>)) =o \<omega>_ord" using b4
     q1 b5 b6 by blast
  moreover have c3: "scf (Restr r (\<W> r f \<alpha>)) =o \<omega>_ord \<and> |\<W> r f \<alpha>| \<le>o |UNIV
    ::nat set|"
  proof -
    have d1: "\<not> finite r" using b2 lem_fin_fl_rel by (metis infinite_iff_natLeq_ordLeq ordLess_imp_ordLeq
    )
    have "Restr r (\<W> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))" using c1 b16 by blast
    then have d2: "\<parallel>Restr r (f \<alpha>)\<parallel> \<le>o \<parallel>Restr r (\<W> r f \<alpha>)\<
    parallel>" using lem_rcc_uset_rcc_bnd by blast
    have "scf (Restr r (f \<alpha>)) =o \<omega>_ord" using c1 b5 b6 q1 by blast
    moreover have "CCR (Restr r (f \<alpha>))"
      using c1 b0 b3 b6 unfolding \<N>_def \<N>6_def using ordLess_imp_ordLeq by blast
    ultimately have "\<omega>_ord =o \<parallel>Restr r (f \<alpha>)\<parallel>"
      using lem_scf_ccr_scf_rcc_eq ordIso_symmetric ordIso_transitive by blast
    then have d3: "\<omega>_ord \<le>o \<parallel>Restr r (\<W> r f \<alpha>)\<parallel>" using d2
    ordIso_ordLeq_trans by blast
    have "|Restr r (\<W> r f \<alpha>)| <o |Field r|" using d1 c1 b0 b3 b6 lem_der_inf_qw_restr_card by blast
    then have "|Restr r (\<W> r f \<alpha>)| <o cardSuc |UNIV::nat set|" using a3 ordLess_ordIso_trans by
    blast
    then have d4: "|Restr r (\<W> r f \<alpha>)| \<le>o |UNIV::nat set|" by simp
    then have "\<parallel>Restr r (\<W> r f \<alpha>)\<parallel> \<le>o \<omega>_ord" using
    lem_Rcc_relcard_bnd
      by (metis ordLeq_transitive card_of_nat ordLeq_ordIso_trans)
    then have "\<parallel>Restr r (\<W> r f \<alpha>)\<parallel> =o \<omega>_ord" using d3 using
    ordIso_iff_ordLeq by blast
    moreover have "|\<W> r f \<alpha>| \<le>o |UNIV::nat set|"
    proof -
      have "\<W> r f \<alpha> \<subseteq> f \<alpha>" unfolding \<W>_def by blast
      then have "|\<W> r f \<alpha>| \<le>o |f \<alpha>|" by simp
      moreover have "|f \<alpha>| <o |Field r|" using c1 b3 b5 b6 b0 unfolding \<N>_def \<N>7_def
        using ordLess_imp_ordLeq ordLeq_ordLess_trans by blast
      ultimately have "|\<W> r f \<alpha>| <o cardSuc |UNIV::nat set|"
        using a3 ordLeq_ordLess_trans ordLess_ordIso_trans by blast
      then show ?thesis by simp
    qed
    ultimately show ?thesis using c2 lem_scf_ccr_scf_rcc_eq[of "Restr r (\<W> r f \<alpha>)"]
      by (metis ordIso_symmetric ordIso_transitive)
  qed
  ultimately have c4: "is_st (Restr r (\<W> r f \<alpha>)) (R \<alpha>)" using q5 b8 by blast
  then have c5: "R \<alpha> \<in> Span (Restr r (\<W> r f \<alpha>))" using q3 by blast
  then have "Field (R \<alpha>) = Field (Restr r (\<W> r f \<alpha>))" unfolding Span_def by blast
```

```
  moreover have "SF r = {A. A \<subseteq> Field r}" using a1 unfolding SF_def refl_on_def Field_def by fast
  moreover have "\<W> r f \<alpha> \<subseteq> Field r" using c1 b0 b3 b6 lem_qw_range ordLess_imp_ordLeq by
    blast
  ultimately have "Field (R \<alpha>) = \<W> r f \<alpha>" unfolding SF_def by blast
  then have "R \<alpha> \<subseteq> Restr r1 (W \<alpha>) \<and> Field (R \<alpha>) = W \<alpha>"
    using c1 c5 b12 b13 b14 unfolding Span_def by blast
  moreover have "R \<alpha> \<in> T" using c4 q3 b11 by blast
  moreover have "\<not> Conelike (Restr r1 (W \<alpha>))"
  proof -
    obtain s1 where d1: "s1 = Restr r (\<W> r f \<alpha>)" by blast
    then have "scf s1 =o \<omega>_ord \<and> CCR s1" using c2 c3 by blast
    moreover then have "\<not> finite (Field (scf s1))"
      by (metis Field_natLeq infinite_UNIV_nat ordIso_finite_Field)
    ultimately have "\<not> Conelike s1" using lem_scf_ccr_finscf_cl by blast
    then show ?thesis using d1 c1 b15[of \<alpha>] by metis
  qed
  ultimately show "R \<alpha> \<in> T \<and> R \<alpha> \<subseteq> Restr r1 (W \<alpha>) \<and> |W \<alpha>|
    \<le>o |UNIV::nat set|
                  \<and> Field (R \<alpha>) = W \<alpha> \<and> \<not> Conelike (Restr r1 (W \<alpha>))"
    using c3 b12 by blast
qed
moreover have "\<And> \<alpha> x. \<alpha> \<in> S \<Longrightarrow> x \<in> W \<alpha> \<Longrightarrow>
          \<exists> a. ((x,a) \<in> (Restr r1 (W \<alpha>))^* \<and> (\<forall> \<beta> \<in> S. \<alpha> <o
    \<beta> \<longrightarrow> (r1``{a} \<inter> W \<beta>) \<noteq> {}))"
proof -
  fix \<alpha> x
  assume c1: "\<alpha> \<in> S" and c2: "x \<in> W \<alpha>"
  moreover obtain a where "a = wesc r f \<alpha> x" by blast
  ultimately have "wesc_rel r f \<alpha> x a" using b4 b0 b5 b6 b12 q1 by blast
  then have c3: "a \<in> \<W> r f \<alpha> \<and> (x,a) \<in> (Restr r (\<W> r f \<alpha>))^*" and
    c4: "\<forall>\<beta>. \<alpha> <o \<beta> \<and> \<beta> <o |Field r| \<and> (\<beta> = {} \<or>
    isSuccOrd \<beta>) \<longrightarrow> r``{a} \<inter> \<W> r f \<beta> \<noteq> {}"
  unfolding wesc_rel_def by blast+
  have "(x,a) \<in> (Restr r1 (W \<alpha>))^*" using c1 c3 b15 by metis
  moreover have "\<forall> \<beta> \<in> S. \<alpha> <o \<beta> \<longrightarrow> (r1``{a} \<inter> W \<beta>)
     \<noteq> {}"
  proof (intro ballI impI)
    fix \<beta>
    assume d1: "\<beta> \<in> S" and "\<alpha> <o \<beta>"
    then obtain b where "(a,b) \<in> r \<and> b \<in> W \<beta>" using c4 b6 b0 b12 by blast
    moreover then have "b \<in> Wa" using d1 b13 by blast
    moreover have "a \<in> Wa" using c1 c3 b12 b13 by blast
    ultimately have "(a,b) \<in> r1 \<and> b \<in> W \<beta>" using b14 by blast
    then show "(r1``{a} \<inter> W \<beta>) \<noteq> {}" by blast
  qed
  ultimately show "\<exists> a. ((x,a) \<in> (Restr r1 (W \<alpha>))^*
                  \<and> (\<forall> \<beta> \<in> S. \<alpha> <o \<beta> \<longrightarrow> (r1``{a} \<inter> W
     \<beta>) \<noteq> {}))" by blast
qed
ultimately obtain r' where b19: "CCR r' \<and> DCR 2 r' \<and> r' \<subseteq> r1"
                          and "\<forall> a \<in> Field r1. \<exists> b \<in> Field r'. (a,b) \<in> r1^*"
  using b11 lem_cfcomp_d2uset[of \<kappa> T r1 S W R] by blast
then have b20: "r' \<in> \<UU> r1" unfolding \<UU>_def Span_def by blast
moreover have "r1 \<in> \<UU> r"
proof -
  have "\<forall> a \<in> Field r. \<exists> \<alpha> \<in> S. a \<in> f \<alpha>"
  proof
    fix a
    assume d1: "a \<in> Field r"
    obtain A where d2: "A = {\<alpha> \<in> \<O>::'U rel set. \<kappa>0 \<le>o \<alpha> \<and> \<alpha> <o \<
    kappa>}" by blast
    have d3: "a \<in> f |Field r| \<and> \<omega>_ord \<le>o |Field r|" using d1 b3 b2
      unfolding \<N>_def \<N>9_def using ordLess_imp_ordLeq by blast
    moreover have "Card_order |Field r|" by simp
    ultimately have "\<not> ( |Field r| = {} \<or> isSuccOrd |Field r| )" using lem_card_inf_lim by blast
    moreover have "|Field r| \<le>o |Field r|" by simp
    ultimately have "(\<nabla> f |Field r| ) = {}" using b3 unfolding \<N>_def \<N>2_def by blast
    then have "f |Field r| \<subseteq> \<LL> f |Field r|" unfolding Dbk_def by blast
    then obtain \<gamma> where d4: "\<gamma> <o \<kappa> \<and> a \<in> f \<gamma>" using d3 b0 unfolding \<LL
    >_def by blast
    have "\<exists> \<alpha> \<in> A. a \<in> f \<alpha>"
    proof (cases "\<kappa>0 \<le>o \<gamma>")
      assume "\<kappa>0 \<le>o \<gamma>"
      then have "nord \<gamma> \<in> A \<and> nord \<gamma> =o \<gamma>" using d4 d2 lem_nord_le_r
    lem_nord_ls_l
        lem_nord_r lem_nord0_le_r ordLess_Well_order_simp by blast
      moreover then have "f (nord \<gamma>) = f \<gamma>" using b3 unfolding \<N>_def by blast
      ultimately have "nord \<gamma> \<in> A \<and> a \<in> f (nord \<gamma>)" using d4 by blast
```

```
      then show ?thesis by blast
    next
      assume "\<not> \<kappa>0 \<le>o \<gamma>"
      moreover have "Well_order \<kappa>0 \<and> Well_order \<gamma>"
        using d4 b5 natLeq_Well_order ordLess_Well_order_simp by blast
      ultimately have "\<gamma> \<le>o \<kappa>0" using ordLeq_total by blast
      moreover have "\<kappa>0 <o \<kappa>" using b0 b2 b5 by blast
      moreover then obtain \<alpha>0::"'U rel" where "\<kappa>0 =o \<alpha>0 \<and> \<alpha>0 <o \<kappa>"
        using internalize_ordLess[of \<kappa>0 \<kappa>] by blast
      ultimately have "\<gamma> \<le>o \<alpha>0 \<and> \<kappa>0 \<le>o \<alpha>0 \<and> \<alpha>0 <o \<kappa
    >"
        using ordLeq_ordIso_trans ordIso_iff_ordLeq by blast
      then have "\<gamma> \<le>o nord \<alpha>0 \<and> \<kappa>0 \<le>o nord \<alpha>0 \<and> nord \<alpha>0 <
    o \<kappa> \<and> nord \<alpha>0 \<in> \<O>"
        using lem_nord_le_r lem_nord_le_r lem_nord_ls_l lem_nord0_le_r
          ordLess_Well_order_simp by blast
      moreover then have "f \<gamma> \<subseteq> f (nord \<alpha>0)"
        using b3 b0 ordLess_imp_ordLeq unfolding \<N>_def \<N>1_def by blast
      ultimately have "a \<in> f (nord \<alpha>0) \<and> nord \<alpha>0 \<in> A" using d4 d2 by blast
      then show ?thesis by blast
    qed
    then obtain \<alpha> \<alpha>' where "\<alpha>' \<in> S \<and> \<alpha> \<le>o \<alpha>' \<and> \<alpha>
    \<in> A \<and> a \<in> f \<alpha>" using d2 b17 by blast
    moreover then have "\<alpha>' \<le>o |Field r|" using b6 b0 using ordLess_imp_ordLeq by blast
    ultimately have "\<alpha>' \<in> S \<and> a \<in> f \<alpha>'" using b3 b0 b0 unfolding \<N>_def \<N>1_def
     by blast
    then show "\<exists> \<alpha> \<in> S. a \<in> f \<alpha>" by blast
  qed
  moreover have "\<forall> \<alpha> \<in> S. f \<alpha> \<subseteq> dncl r (Field r1)"
  proof
    fix \<alpha>
    assume d1: "\<alpha> \<in> S"
    show "f \<alpha> \<subseteq> dncl r (Field r1)"
    proof
      fix a
      assume "a \<in> f \<alpha>"
      moreover have "f \<alpha> \<in> SF r" using d1 b0 b3 b6
        unfolding \<N>_def \<N>5_def using ordLess_imp_ordLeq by blast
      ultimately have "a \<in> Field (Restr r (f \<alpha>))" unfolding SF_def by blast
      moreover have "Restr r (\<W> r f \<alpha>) \<in> \<UU> (Restr r (f \<alpha>))" using d1 b16 by blast
      ultimately obtain b where "b \<in> Field (Restr r (\<W> r f \<alpha>)) \<and> (a, b) \<in> (Restr r (f
    \<alpha>))^*"
        unfolding \<UU>_def by blast
      then have "b \<in> \<W> r f \<alpha> \<and> (a,b) \<in> r^*"
        unfolding Field_def using rtrancl_mono[of "Restr r (f \<alpha>)" r] by blast
      moreover then have "b \<in> Field r1" using d1 b12 b18 by blast
      ultimately show "a \<in> dncl r (Field r1)" unfolding dncl_def by blast
    qed
  qed
  ultimately have "\<forall> a \<in> Field r. \<exists> b \<in> Field r1. (a, b) \<in> r^*" unfolding dncl_def
     by blast
  moreover have "CCR r1" using b20 lem_rcc_uset_ne_ccr by blast
  moreover have "r1 \<subseteq> r" using b14 by blast
  ultimately show "r1 \<in> \<UU> r" unfolding \<UU>_def by blast
qed
ultimately have "r' \<in> \<UU> r" using lem_rcc_uset_tr by blast
then show "DCR 3 r" using b19 lem_Ldo_uset_reduc[of r' r 2] by simp
qed

lemma lem_dc3_ccr_scf_lewsuc:
fixes r::"'U rel"
assumes a1: "CCR r" and a2: "|Field r| \<le>o cardSuc |UNIV::nat set|"
shows "DCR 3 r"
proof (cases "scf r \<le>o \<omega>_ord")
  assume "scf r \<le>o \<omega>_ord"
  then have "DCR 2 r" using a1 lem_dc2_ccr_scf_lew by blast
  moreover have "r \<in> \<UU> r" using a1 unfolding \<UU>_def by blast
  ultimately show "DCR 3 r" using lem_Ldo_uset_reduc[of r r 2] by simp
next
  assume "\<not> (scf r \<le>o \<omega>_ord)"
  then have "\<omega>_ord <o |Field r|" using lem_scf_relfldcard_bnd lem_scf_inf
    by (metis ordIso_iff_ordLeq ordLeq_iff_ordLess_or_ordIso ordLeq_transitive)
  then have "|UNIV::nat set| <o |Field r|" using card_of_nat ordIso_ordLess_trans by blast
  then have "cardSuc |UNIV::nat set| \<le>o |Field r|" by (meson cardSuc_ordLess_ordLeq card_of_Card_order)
  then have b0: "|Field r| =o cardSuc |UNIV::nat set|" using a2
    using not_ordLeq_ordLess ordLeq_iff_ordLess_or_ordIso by blast
  obtain r1 where b1: "r1 = r \<union> {(x,y). x = y \<and> x \<in> Field r}" by blast
  have b2: "Field r1 = Field r" using b1 unfolding Field_def by blast
```

```
  have "r \<in> \<UU> r1" using b1 b2 a1 unfolding \<UU>_def by blast
  then have b3: "CCR r1" using lem_rcc_uset_ne_ccr[of r1] by blast
  have "(\<not> (scf r1 \<le>o \<omega>_ord)) \<longrightarrow> scf r1 =o |Field r1|"
  proof
    assume "\<not> (scf r1 \<le>o \<omega>_ord)"
    then have "\<omega>_ord <o scf r1"
      using lem_scf_inf by (metis ordIso_iff_ordLeq ordLeq_iff_ordLess_or_ordIso)
    then have "|UNIV::nat set| <o scf r1 \<and> Card_order (scf r1)"
      using lem_scf_cardord by (metis card_of_nat ordIso_ordLess_trans)
    then have "cardSuc |UNIV::nat set| \<le>o scf r1" by (meson cardSuc_ordLess_ordLeq card_of_Card_order)
    then have "|Field r1| \<le>o scf r1" using b0 b2 by (metis ordIso_ordLeq_trans)
    then show "scf r1 =o |Field r1|" using lem_scf_relfldcard_bnd[of r1]
      by (metis not_ordLeq_ordLess ordLeq_iff_ordLess_or_ordIso)
  qed
  moreover have "scf r1 \<le>o \<omega>_ord \<longrightarrow> DCR 3 r1"
  proof
    assume "scf r1 \<le>o \<omega>_ord"
    then have "DCR 2 r1" using b3 lem_dc2_ccr_scf_lew by blast
    moreover have "r1 \<in> \<UU> r1" using b3 unfolding \<UU>_def by blast
    ultimately show "DCR 3 r1" using lem_Ldo_uset_reduc[of r1 r1 2] by simp
  qed
  moreover have "scf r1 =o |Field r1| \<longrightarrow> DCR 3 r1"
  proof
    assume "scf r1 =o |Field r1|"
    moreover have "Refl r1" using b1 unfolding refl_on_def Field_def by force
    ultimately show "DCR 3 r1" using b0 b2 b3 lem_dc3_ccr_refl_scf_wsuc[of r1] by simp
  qed
  ultimately have "DCR 3 r1" by blast
  moreover have "\<And> n. n \<noteq> 0 \<Longrightarrow> DCR n r1 \<Longrightarrow> DCR n r" using b1
      lem_Ldo_eqid by blast
  ultimately show "DCR 3 r" by force
qed

lemma lem_dirs_wrt_ch_un:
fixes \<rho> \<A>
assumes a1: "\<forall> A\<in>\<A>. is_dirset_wrt A \<rho>" and a2: "\<A> \<noteq> {}" and a3: "\<forall> A\<in
      >\<A>. \<forall> B\<in>\<A>. A \<subseteq> B \<or> B \<subseteq> A"
shows "is_dirset_wrt (\<Union> \<A>) \<rho>"
proof -
  have "(\<Union> \<A>) \<noteq> {}" using a1 a2 unfolding is_dirset_wrt_def by simp
  moreover have "\<forall>x\<in>(\<Union> \<A>). \<forall>y\<in>(\<Union> \<A>). \<exists>c\<in>(\<Union> \<A>)
      . \<rho> x c \<and> \<rho> y c"
  proof (intro ballI)
    fix x y
    assume "x\<in>(\<Union> \<A>)" and "y\<in>(\<Union> \<A>)"
    then obtain A B where "A \<in> \<A> \<and> B \<in> \<A> \<and> x \<in> A \<and> y \<in> B" by blast
    moreover then have "(x \<in> A \<and> y \<in> A) \<or> (x \<in> B \<and> y \<in> B)" using a3 by blast
    ultimately show "\<exists>c\<in>(\<Union> \<A>) . \<rho> x c \<and> \<rho> y c" using a1 unfolding
      is_dirset_wrt_def by (meson UnionI)
  qed
  ultimately show "is_dirset_wrt (\<Union> \<A>) \<rho>" unfolding is_dirset_wrt_def by blast
qed

lemma lem_dirs_in_ch_un:
fixes X \<rho> \<A>
assumes "\<forall> A\<in>\<A>. is_dirset_in A X \<rho>" and "\<A> \<noteq> {}" and "\<forall> A\<in>\<A>. \<
      forall> B\<in>\<A>. A \<subseteq> B \<or> B \<subseteq> A"
shows "is_dirset_in (\<Union> \<A>) X \<rho>"
  using assms lem_dirs_wrt_ch_un[of \<A> \<rho>] by blast

lemma lem_diam_el_maxds:
fixes X \<rho> x0
assumes a1: "is_preorder X \<rho>" and a2: "x0 \<in> X"
shows "\<exists> M. ( is_maxdirset_in M X \<rho> \<and> x0 \<in> M )"
proof -
  obtain \<A> where b1: "\<A> = { A. is_dirset_in A X \<rho> \<and> x0 \<in> A }" by blast
  obtain U where b2: "U = { x \<in> X. \<rho> x0 x }" by blast
  have b3: "{x0} \<in> \<A>" using a1 b1 a2 unfolding reflexive_on_def is_dirset_wrt_def by blast
  have "\<forall> C. pred_on.chain \<A> (\<subset>) C \<longrightarrow> (\<exists>B\<in>\<A>. \<forall>X\<in>C.
      X \<subseteq> B)"
  proof (intro allI impI)
    fix C
    assume c1: "pred_on.chain \<A> (\<subset>) C"
    have "C \<noteq> {} \<longrightarrow> (\<Union>C) \<in> \<A>"
    proof
      assume d1: "C \<noteq> {}"
      moreover have "\<forall>A\<in>C. is_dirset_in A X \<rho>" using b1 c1 unfolding pred_on.chain_def by blast
```

```
      moreover have "\<forall>A\<in>C. \<forall>B\<in>C. A \<subseteq> B \<or> B \<subseteq> A" using c1 by (
      simp only: b1 pred_on.chain_def, fast)
      ultimately have "is_dirset_in (\<Union>C) X \<rho>" using lem_dirs_in_ch_un[of C] by simp
      moreover have "x0 \<in> (\<Union>C)" using b1 c1 d1 unfolding pred_on.chain_def by fast
      ultimately show "(\<Union>C) \<in> \<A>" using b1 by blast
    qed
    then show "\<exists>B\<in>\<A>. \<forall>X\<in>C. X \<subseteq> B" using b3 by blast
  qed
  then obtain M where b4: "M \<in> \<A> \<and> (\<forall>X \<in> \<A>. M \<subseteq> X \<longrightarrow> X = M)"
       using subset_Zorn[of \<A>] by blast
  then have b5: "x0 \<in> M" using b1 by blast
  have "is_dirset_in M X \<rho>" using b1 b4 by blast
  moreover have "\<forall> B. is_dirset_in B X \<rho> \<and> M \<subseteq> B \<longrightarrow> M = B" using b1
      b4 b5 by blast
  ultimately have "is_maxdirset_in M X \<rho>" unfolding is_maxdirset_in_def by blast
  then show ?thesis using b5 by blast
qed

lemma lem_diam_maxds_upset:
fixes X \<rho> A
assumes a1: "is_preorder X \<rho>"  and a2: "has_diamprop X \<rho>" and a3: "is_maxdirset_in A X \<rho>"
shows "is_up_closed A X \<rho>"
proof -
  have b1: "A \<subseteq> X" using a3 unfolding is_maxdirset_in_def by blast
  moreover have "\<forall>a\<in>A. \<forall>x\<in>X. \<rho> a x \<longrightarrow> x \<in> A"
  proof (intro ballI impI)
    fix a x
    assume c1: "a \<in> A" and c2: "x \<in> X" and c3: "\<rho> a x"
    let ?U = "{x' \<in> X. \<rho> x x'}"
    have "is_dirset_in (A \<union> ?U) X \<rho>"
    proof -
      have "\<forall>b1\<in>A \<union> ?U. \<forall>b2\<in>A \<union> ?U. \<exists>c\<in>A \<union> ?U. \<rho>
      b1 c \<and> \<rho> b2 c"
      proof (intro ballI)
        fix b1 b2
        assume "b1\<in>A \<union> ?U" and "b2\<in>A \<union> ?U"
        moreover have "b1 \<in> A \<and> b2 \<in> A \<longrightarrow> (\<exists>c\<in>A \<union> ?U. \<rho> b1 c
       \<and> \<rho> b2 c)"
          using a3 unfolding is_maxdirset_in_def is_dirset_wrt_def by (meson UnCI)
        moreover have "b1 \<in> A \<and> b2 \<in> ?U \<longrightarrow> (\<exists>c\<in>?U. \<rho> b1 c \<and> \<
      rho> b2 c)"
          proof
            assume f1: "b1 \<in> A \<and> b2 \<in> ?U"
            then obtain t where f2: "t \<in> A \<and> \<rho> a t \<and> \<rho> b1 t"
              using a3 c1 unfolding is_maxdirset_in_def is_dirset_wrt_def by meson
            have f3: "a \<in> X \<and> x \<in> X \<and> b2 \<in> X \<and> \<rho> a x \<and> \<rho> x b2" using b1
      c1 c2 c3 f1 by blast
            moreover then have "\<rho> a b2" using a1 unfolding transitive_on_def by blast
            moreover have "t \<in> X \<and> \<rho> a t" using b1 f2 by blast
            ultimately obtain c where f4: "c \<in> X \<and> \<rho> t c \<and> \<rho> b2 c" using a2 unfolding
      has_diamprop_def by blast
            moreover then have "\<rho> b1 c" using a1 b1 f1 f2 unfolding transitive_on_def by blast
            moreover have "\<rho> x c" using a1 f3 f4 unfolding transitive_on_def by blast
            ultimately show "\<exists>c\<in>?U. \<rho> b1 c \<and> \<rho> b2 c" by blast
          qed
          moreover have "b1 \<in> ?U \<and> b2 \<in> A \<longrightarrow> (\<exists>c\<in>?U. \<rho> b1 c \<and> \<
      rho> b2 c)"
          proof
            assume f1: "b1 \<in> ?U \<and> b2 \<in> A"
            then obtain t where f2: "t \<in> A \<and> \<rho> a t \<and> \<rho> b2 t"
              using a3 c1 unfolding is_maxdirset_in_def is_dirset_wrt_def by meson
            have f3: "a \<in> X \<and> x \<in> X \<and> b1 \<in> X \<and> \<rho> a x \<and> \<rho> x b1" using b1
      c1 c2 c3 f1 by blast
            moreover then have "\<rho> a b1" using a1 unfolding transitive_on_def by blast
            moreover have "t \<in> X \<and> \<rho> a t" using b1 f2 by blast
            ultimately obtain c where f4: "c \<in> X \<and> \<rho> t c \<and> \<rho> b1 c" using a2 unfolding
      has_diamprop_def by blast
            moreover then have "\<rho> b2 c" using a1 b1 f1 f2 unfolding transitive_on_def by blast
            moreover have "\<rho> x c" using a1 f3 f4 unfolding transitive_on_def by blast
            ultimately show "\<exists>c\<in>?U. \<rho> b1 c \<and> \<rho> b2 c" by blast
          qed
          moreover have "b1 \<in> ?U \<and> b2 \<in> ?U \<longrightarrow> (\<exists>c\<in>?U. \<rho> b1 c \<and>
      \<rho> b2 c)"
            using a1 a2 c2 unfolding transitive_on_def has_diamprop_def by blast
          ultimately show "\<exists>c\<in>A \<union> ?U. \<rho> b1 c \<and> \<rho> b2 c" by blast
        qed
        moreover have "?U \<noteq> {}" using a1 c2 unfolding reflexive_on_def by blast
        ultimately show ?thesis using b1 unfolding is_dirset_wrt_def by blast
```

```
    qed
    then show "x \<in> A" using a1 c2 a3 unfolding reflexive_on_def is_maxdirset_in_def by blast
  qed
  ultimately show ?thesis unfolding is_up_closed_def by blast
qed

lemma lem_diam_maxds_dnset:
fixes X \<rho> A
assumes a1: "is_preorder X \<rho>"  and a2: "has_diamprop X \<rho>" and a3: "is_maxdirset_in A X \<rho>"
shows "is_dn_closed A X \<rho>"
proof -
  have b1: "A \<subseteq> X" using a3 unfolding is_maxdirset_in_def is_dirset_wrt_def by blast
  moreover have "\<forall>a\<in>A. \<forall>x\<in>X. \<rho> x a \<longrightarrow> x \<in> A"
  proof (intro ballI impI)
    fix a x
    assume c1: "a \<in> A" and c2: "x \<in> X" and c3: "\<rho> x a"
    have "is_dirset_in (A \<union> {x}) X \<rho>"
    proof -
      have "\<forall>b1\<in>A \<union> {x}. \<forall>b2\<in>A \<union> {x}. \<exists>c\<in>A \<union> {x}. \<rho
      > b1 c \<and> \<rho> b2 c"
      proof (intro ballI)
        fix b1 b2
        assume "b1\<in>A \<union> {x}" and "b2\<in>A \<union> {x}"
        moreover have "b1 \<in> A \<and> b2 \<in> A \<longrightarrow> (\<exists>c\<in>A \<union> {x}. \<rho> b1
      c \<and> \<rho> b2 c)"
          using a3 unfolding is_maxdirset_in_def is_dirset_wrt_def by (meson UnCI)
        moreover have "b1 \<in> A \<longrightarrow> (\<exists>c\<in>A. \<rho> b1 c \<and> \<rho> x c)"
        proof
          assume "b1 \<in> A"
          then obtain c where "c \<in> A \<and> \<rho> a c \<and> \<rho> b1 c"
            using a3 c1 unfolding is_maxdirset_in_def is_dirset_wrt_def by meson
          moreover then have "\<rho> x c" using a1 b1 c1 c2 c3 unfolding transitive_on_def by blast
          ultimately show "\<exists>c\<in>A. \<rho> b1 c \<and> \<rho> x c" by blast
        qed
        moreover have "b2 \<in> A \<longrightarrow> (\<exists>c\<in>A. \<rho> x c \<and> \<rho> b2 c)"
        proof
          assume "b2 \<in> A"
          then obtain c where "c \<in> A \<and> \<rho> a c \<and> \<rho> b2 c"
            using a3 c1 unfolding is_maxdirset_in_def is_dirset_wrt_def by meson
          moreover then have "\<rho> x c" using a1 b1 c1 c2 c3 unfolding transitive_on_def by blast
          ultimately show "\<exists>c\<in>A. \<rho> x c \<and> \<rho> b2 c" by blast
        qed
        moreover have "\<rho> x x" using a1 c2 unfolding reflexive_on_def by blast
        ultimately show "\<exists>c\<in>A \<union> {x}. \<rho> b1 c \<and> \<rho> b2 c" by blast
      qed
      then show ?thesis using b1 c2 unfolding is_dirset_wrt_def by blast
    qed
    then show "x \<in> A" using a3 unfolding is_maxdirset_in_def by blast
  qed
  ultimately show ?thesis unfolding is_dn_closed_def by blast
qed

lemma lem_diam_maxds_eq_or_disj:
fixes X \<rho> A B
assumes a1: "is_preorder X \<rho>" and a2: "has_diamprop X \<rho>"
    and a3: "is_maxdirset_in A X \<rho>" and a4: "is_maxdirset_in B X \<rho>"
shows "A = B \<or> (\<forall>x\<in>A.\<forall>y\<in>B. \<not> \<rho> x y)"
proof -
  have b1: "A \<subseteq> X \<and> B \<subseteq> X" using a3 a4 unfolding is_maxdirset_in_def is_dirset_wrt_def
      by simp
  have "\<forall> x y a. (x\<in>A \<and> y\<in>B \<and> \<rho> x y) \<longrightarrow> a \<in> A \<longrightarrow> a \<in> B"
  proof (intro allI impI)
    fix x y a
    assume c1: "x\<in>A \<and> y\<in>B \<and> \<rho> x y" and c2: "a \<in> A"
    then obtain t where c3: "t \<in> A \<and> \<rho> x t \<and> \<rho> a t"
      using a3 unfolding is_maxdirset_in_def is_dirset_wrt_def by metis
    moreover have "is_up_closed B X \<rho>" using a1 a2 a4 lem_diam_maxds_upset by blast
    moreover have c4: "is_dn_closed B X \<rho>" using a1 a2 a4 lem_diam_maxds_dnset by blast
    ultimately have "t \<in> B" using b1 c1 unfolding is_up_closed_def is_dn_closed_def by (metis contra_subsetD
      )
    then show "a \<in> B" using b1 c2 c3 c4 unfolding is_dn_closed_def by blast
  qed
  moreover have "\<forall> x y b. (x\<in>A \<and> y\<in>B \<and> \<rho> x y) \<longrightarrow> b \<in> B \<
      longrightarrow> b \<in> A"
  proof (intro allI impI)
    fix x y b
    assume c1: "x\<in>A \<and> y\<in>B \<and> \<rho> x y" and c2: "b \<in> B"
```

```
    then obtain t where c3: "t \<in> B \<and> \<rho> y t \<and> \<rho> b t"
      using a4 unfolding is_maxdirset_in_def is_dirset_wrt_def by metis
    moreover have "is_up_closed A X \<rho>" using a1 a2 a3 lem_diam_maxds_upset by blast
    moreover then have "y \<in> A" using b1 c1 unfolding is_up_closed_def by blast
    ultimately have "t \<in> A" using b1 unfolding is_up_closed_def by blast
    moreover have "is_dn_closed A X \<rho>" using a1 a2 a3 lem_diam_maxds_dnset by blast
    ultimately show "b \<in> A" using b1 c2 c3 unfolding is_dn_closed_def by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_diam_decomp:
fixes X \<rho> A
assumes a1: "is_preorder X \<rho>" and a2: "has_diamprop X \<rho>"
shows "\<exists> \<A>. (X = (\<Union> \<A>)) \<and> (\<forall> A\<in>\<A>. \<forall> B\<in>\<A>. A\<noteq>B \<
      longrightarrow> (\<forall>x\<in>A.\<forall>y\<in>B. \<not>\<rho> x y)) \<and> (\<forall> A\<in>\<A>. (
      is_dirset_wrt A \<rho>))"
proof -
  obtain \<A> where b1: "\<A> = { A. is_maxdirset_in A X \<rho> }" by blast
  have "X = (\<Union> \<A>)"
  proof
    show "X \<subseteq> (\<Union> \<A>)" using a1 b1 lem_diam_el_maxds[of \<rho>] by blast
  next
    show "(\<Union> \<A>) \<subseteq> X" using b1 unfolding is_maxdirset_in_def by blast
  qed
  moreover have "\<forall> A\<in>\<A>. \<forall> B\<in>\<A>. A \<noteq> B \<longrightarrow> (\<forall>x\<in>A.\<
      forall>y\<in>B. \<not>\<rho> x y)"
    using a1 a2 b1 lem_diam_maxds_eq_or_disj by blast
  moreover have "\<forall> A\<in>\<A>. is_dirset_wrt A \<rho>" using b1 unfolding is_maxdirset_in_def by blast
  ultimately show ?thesis by blast
qed

lemma lem_conflars_rel: "is_confl_ars X (\<lambda> x y. (x,y) \<in> r) = (confl_rel r \<and> Field r \<subseteq>
       X)"
proof
  assume a1: "is_confl_ars X (\<lambda> x y. (x,y) \<in> r)"
  show "confl_rel r \<and> Field r \<subseteq> X"
  proof -
    let ?\<sigma> = "\<lambda> x y. (x,y) \<in> r"
    have "Field r \<subseteq> X" using a1 unfolding is_confl_ars_def is_ars_def Field_def by blast
    moreover have "confl_rel r"
    proof -
      have "\<forall> a b c. (a,b) \<in> r^* \<and> (a,c) \<in> r^* \<longrightarrow> (\<exists> d. (b,d) \<in>
      r^* \<and> (c,d) \<in> r^*)"
      proof (intro allI impI)
        fix a b c
        assume d1: "(a,b) \<in> r^* \<and> (a,c) \<in> r^*"
        moreover have "a \<noteq> b \<and> a \<noteq> c \<longrightarrow> (\<exists> d. (b,d) \<in> r^* \<and> (
      c,d) \<in> r^*)"
        proof
          assume "a \<noteq> b \<and> a \<noteq> c"
          then have "a \<in> Field r \<and> b \<in> Field r \<and> c \<in> Field r" using d1 lem_rtr_field[of _
      _ r] by blast
          then have "a \<in> X \<and> b \<in> X \<and> c \<in> X"
            using a1 unfolding is_confl_ars_def is_ars_def Field_def by blast
          moreover have "?\<sigma>^** a b \<and> ?\<sigma>^** a c" using d1 Transitive_Closure.
      rtranclp_rtrancl_eq by metis
          ultimately obtain d where "d \<in> X \<and> (?\<sigma>^** b d) \<and> (?\<sigma>^** c d)" using a1
      unfolding is_confl_ars_def by blast
          then show "\<exists> d. (b,d) \<in> r^* \<and> (c,d) \<in> r^*" using Transitive_Closure.
      rtranclp_rtrancl_eq by metis
        qed
        ultimately show "\<exists> d. (b,d) \<in> r^* \<and> (c,d) \<in> r^*" by blast
      qed
      then show "confl_rel r" unfolding confl_rel_def by blast
    qed
    ultimately show ?thesis by blast
  qed
next
  assume a1: "confl_rel r \<and> Field r \<subseteq> X"
  show "is_confl_ars X (\<lambda> x y. (x,y) \<in> r)"
  proof -
    let ?\<sigma> = "\<lambda> x y. (x,y) \<in> r"
    have "is_ars X ?\<sigma>" using a1 unfolding is_ars_def Field_def by blast
    moreover have "\<forall>a\<in>X. \<forall>b\<in>X. \<forall>c\<in>X. (?\<sigma>^** a b \<and> ?\<sigma>^** a
       c \<longrightarrow> (\<exists>d\<in>X. ?\<sigma>^** b d \<and> ?\<sigma>^** c d))"
    proof (intro ballI impI)
      fix a b c
```

```
      assume c1: "a \<in> X" and c2: "b \<in> X" and c3: "c \<in> X" and "?\<sigma>^** a b \<and> ?\<sigma>^** a
       c"
      then have "(a,b) \<in> r^* \<and> (a,c) \<in> r^*" using Transitive_Closure.rtranclp_rtrancl_eq by metis
      then obtain d where c4: "(b,d) \<in> r^* \<and> (c,d) \<in> r^*" using a1 unfolding confl_rel_def by blast
      have "d \<in> X" using a1 c3 c4 using lem_rtr_field[of _ _ r] by blast
      moreover have "?\<sigma>^** b d \<and> ?\<sigma>^** c d" using c4 Transitive_Closure.rtranclp_rtrancl_eq
       by metis
      ultimately show "\<exists>d\<in>X. ?\<sigma>^** b d \<and> ?\<sigma>^** c d" by blast
    qed
    ultimately show ?thesis unfolding is_confl_ars_def by blast
  qed
qed

lemma lem_cr_dp:
fixes r X \<rho>
assumes a1: "confl_rel r" and a2: "Field r \<subseteq> X" and a3: "\<rho> = (\<lambda> x y. (x,y) \<in> r^*)"
shows "is_preorder X \<rho> \<and> has_diamprop X \<rho>"
proof -
  let ?\<sigma> = "\<lambda> x y. (x,y) \<in> r"
  have "is_confl_ars X ?\<sigma>" using a1 a2 lem_conflars_rel by blast
  then have "has_diamprop X ?\<sigma>^**" unfolding is_confl_ars_def has_diamprop_def by blast
  moreover have "?\<sigma>^** = \<rho>" by (simp only: a3, metis Transitive_Closure.rtranclp_rtrancl_eq)
  ultimately have "has_diamprop X \<rho>" by blast
  moreover have "is_preorder X \<rho>" using a3 unfolding reflexive_on_def transitive_on_def by force
  ultimately show ?thesis by blast
qed

lemma lem_Cprf_conf_ccr_decomp:
fixes r::"'U rel"
assumes "confl_rel r"
shows "\<exists> S::('U rel set). (\<forall>s\<in>S. CCR s) \<and> (r = \<Union> S) \<and> (\<forall> s1\<in>S.
      \<forall>s2\<in>S. s1 \<noteq> s2 \<longrightarrow> Field s1 \<inter> Field s2 = {} )"
proof -
  let ?X = "Field r"
  obtain \<rho> where b1: "\<rho> = (\<lambda> x y. (x,y) \<in> r^*)" by blast
  then have "\<exists>\<A>. (?X = (\<Union> \<A>)) \<and> (\<forall> A\<in>\<A>. \<forall> B\<in>\<A>. A\<noteq>
      B \<longrightarrow> (\<forall>x\<in>A.\<forall>y\<in>B. \<not>\<rho> x y)) \<and> (\<forall> A\<in>\<A>.
      is_dirset_wrt A \<rho>)"
    using assms lem_diam_decomp lem_cr_dp[of r ?X \<rho>] by blast
  then obtain \<A> where b2: "?X = (\<Union> \<A>)"
                 and b3: "\<forall> A\<in>\<A>. \<forall> B\<in>\<A>. A\<noteq>B \<longrightarrow> (\<forall>x
      \<in>A.\<forall>y\<in>B. \<not>\<rho> x y)"
                 and b4: "\<forall> A\<in>\<A>. is_dirset_wrt A \<rho>" by force
  obtain S where b5: "S = {r'. \<exists> A \<in> \<A>. r' = r \<inter> (A \<times> A) }" by blast
  have "r = \<Union> S"
  proof
    show "r \<subseteq> (\<Union> S)"
    proof
      fix p
      assume d1: "p \<in> r"
      moreover obtain x y where d2: "p = (x,y)" by (metis surjective_pairing)
      ultimately have "x \<in> Field r \<and> y \<in> Field r" unfolding Field_def by blast
      then obtain A B where d3: "A \<in> \<A> \<and> B \<in> \<A> \<and> x \<in> A \<and> y \<in> B" using b2 by
       blast
      moreover have "\<rho> x y" using b1 d1 d2 by blast
      ultimately have "p \<in> A\<times>A" using b3 d2 by blast
      moreover have "r \<inter> (A \<times> A) \<in> S" using b5 d3 by blast
      ultimately show "p \<in> (\<Union> S)" using d1 by blast
    qed
  next
    show "(\<Union> S) \<subseteq> r" using b5 by blast
  qed
  moreover have "\<forall> r1\<in>S. \<forall>r2\<in>S. Field r1 \<inter> Field r2 \<noteq> {} \<longrightarrow>
       r1 = r2"
  proof (intro ballI impI)
    fix r1 r2
    assume c1: "r1\<in>S" and c2: "r2\<in>S" and "Field r1 \<inter> Field r2 \<noteq> {}"
    then obtain x0 where c3: "x0 \<in> Field r1 \<and> x0 \<in> Field r2" by blast
    obtain A1 A2 where c4: "A1 \<in> \<A> \<and> r1 = r \<inter> (A1 \<times> A1)" and c5: "A2 \<in> \<A> \<and>
       r2 = r \<inter> (A2 \<times> A2)"
      using b5 c1 c2 by blast
    moreover then have "x0 \<in> A1 \<and> x0 \<in> A2 \<and> \<rho> x0 x0" using b1 c3 unfolding Field_def by
      blast
    ultimately have "A1 = A2" using b3 by blast
    then show "r1 = r2" using c4 c5 by blast
  qed
  moreover have "\<forall>r'\<in>S. CCR r'"
  proof
```

```
    fix r'
    assume c1: "r' \<in> S"
    then obtain A where c2: "A \<in> \<A> \<and> r' = r \<inter> (A \<times> A)" using b5 by blast
    then have c3: "is_dirset_wrt A \<rho>" using b4 by blast
    have "\<forall>a\<in>Field r'. \<forall>b\<in>Field r'. \<exists>c\<in>Field r'. (a, c) \<in> r'^* \<and> (b
       , c) \<in> r'^*"
    proof (intro ballI)
      fix a b
      assume d1: "a \<in> Field r'" and d2: "b \<in> Field r'"
      then have d3: "a \<in> A \<and> b \<in> A" using c2 unfolding Field_def by blast
      then obtain c where d4: "c \<in> A \<and> \<rho> a c \<and> \<rho> b c" using c3 unfolding
       is_dirset_wrt_def by blast
      then have "(a,c) \<in> r^* \<and> (b,c) \<in> r^*" using b1 by blast
      moreover have "r^* \<inter> (A \<times> A) \<subseteq> r'^*"
      proof -
        have "\<forall> n. r^^n \<inter> (A \<times> A) \<subseteq> r'^*"
        proof
          fix n
          show "r^^n \<inter> (A \<times> A) \<subseteq> r'^*"
          proof (induct n)
            show "r^^0 \<inter> (A \<times> A) \<subseteq> r'^*" by force
          next
            fix k
            assume g1: "r^^k \<inter> (A \<times> A) \<subseteq> r'^*"
            show "r^^(Suc k) \<inter> (A \<times> A) \<subseteq> r'^*"
            proof
              fix p
              assume "p \<in> r^^(Suc k) \<inter> (A \<times> A)"
              moreover obtain x y where h1: "p = (x,y)" by (metis surjective_pairing)
              ultimately obtain t where h2: "x \<in> A \<and> y \<in> A \<and> (x,t) \<in> r^^k \<and> (t,y) \<
      in> r" by force
              moreover then obtain A' where "A' \<in> \<A> \<and> t \<in> A'" using b2 unfolding Field_def by
      blast
              ultimately have "t \<in> A" using c2 b1 b3 by force
              then have "(x,t) \<in> r'^* \<and> (t,y) \<in> r'^*" using c2 h2 g1 by blast
              then show "p \<in> r'^*" using h1 by force
            qed
          qed
        qed
        then show ?thesis using rtrancl_imp_relpow by blast
      qed
      ultimately have "(a, c) \<in> r'^* \<and> (b, c) \<in> r'^*" using d3 d4 by blast
      moreover then have "c \<in> Field r'" using d1 lem_rtr_field[of _ _ r'] by blast
      ultimately show "\<exists> c \<in> Field r'. (a, c) \<in> r'^* \<and> (b, c) \<in> r'^*" by blast
    qed
    then show "CCR r'" unfolding CCR_def by blast
  qed
  ultimately show ?thesis by blast
qed

lemma lem_Cprf_dc_disj_fld_un:
fixes S::"'U rel set" and n::nat
assumes a1: "\<forall> s1\<in>S. \<forall>s2\<in>S. s1\<noteq>s2 \<longrightarrow> Field s1 \<inter> Field s2 =
      {}"
    and a2: "\<forall> s\<in>S. DCR n s"
shows "DCR n (\<Union> S)"
proof -
  obtain gi::"'U rel \<Rightarrow> nat \<Rightarrow> 'U rel"
    where b1: "gi = (\<lambda> s. (SOME g. DCR_generating g \<and> s = \<Union>{r'. \<exists>\<alpha>'<n. r' = g
       \<alpha>'}))" by blast
  obtain ga where b2: "ga = (\<lambda> \<alpha>. if (\<alpha> < n) then \<Union>s\<in>S. gi s \<alpha> else {})"
       by blast
  have b3: "\<And> s. s \<in> S \<Longrightarrow> DCR_generating (gi s) \<and> s = \<Union>{r'. \<exists>\<alpha
      >'<n. r' = gi s \<alpha>'}"
  proof -
    fix s
    assume "s \<in> S"
    then obtain g where "DCR_generating g \<and> s = \<Union>{r'. \<exists>\<alpha>'<n. r' = g \<alpha>'}"
      using a2 unfolding DCR_def by force
    then show "DCR_generating (gi s) \<and> s = \<Union>{r'. \<exists>\<alpha>'<n. r' = gi s \<alpha>'}"
      using b1 someI_ex[of "\<lambda> g. DCR_generating g \<and> s = \<Union>{r'. \<exists>\<alpha>'<n. r' = g
      \<alpha>'}"] by blast
  qed
  have "\<forall>\<alpha> \<beta> a b c. (a, b) \<in> ga \<alpha> \<and> (a, c) \<in> ga \<beta> \<
      longrightarrow>
       (\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> ga \<alpha> \<beta> \<and> (c, c', c'', d) \<in>
      \<DD> ga \<beta> \<alpha>)"
  proof (intro allI impI)
```

```
  fix \<alpha> \<beta> a b c
  assume c1: "(a, b) \<in> ga \<alpha> \<and> (a, c) \<in> ga \<beta>"
  moreover have "\<alpha> < n" using c1 b2 by (cases "\<alpha><n", simp+)
  moreover have "\<beta> < n" using c1 b2 by (cases "\<beta><n", simp+)
  ultimately obtain s1 s2 where c2: "\<alpha> < n \<and> s1 \<in> S \<and> (a,b) \<in> gi s1 \<alpha>"
                          and c3: "\<beta> < n \<and> s2 \<in> S \<and> (a,c) \<in> gi s2 \<beta>" using c1
    b2 by fastforce
  then have "(a,b) \<in> s1 \<and> (a,c) \<in> s2" using b3 by blast
  then have "s1 = s2 " using c2 c3 a1 unfolding Field_def by blast
  then obtain b' b'' c' c'' d
    where c4: "(b, b', b'', d) \<in> \<DD> (gi s1) \<alpha> \<beta>" and c5: "(c, c', c'', d) \<in> \<DD> (gi
    s1) \<beta> \<alpha>"
    using c2 c3 b3[of s1] unfolding DCR_generating_def by blast
  have "(b, b', b'', d) \<in> \<DD> ga \<alpha> \<beta>"
  proof -
    have d1: "(b, b') \<in> (\<LL>1 (gi s1) \<alpha>)^* \<and> (b', b'') \<in> (gi s1 \<beta>)^= \<and> (b'',
    d) \<in> (\<LL>v (gi s1) \<alpha> \<beta>)^*"
      using c4 unfolding \<DD>_def by blast
    have "\<LL>1 (gi s1) \<alpha> \<subseteq> \<LL>1 ga \<alpha>"
    proof
      fix p
      assume "p \<in> \<LL>1 (gi s1) \<alpha>"
      then obtain \<gamma> where "\<gamma> < \<alpha> \<and> p \<in> gi s1 \<gamma>" unfolding \<LL>1_def by
    blast
      moreover then have "p \<in> ga \<gamma>" using c2 b2 by fastforce
      ultimately show "p \<in> \<LL>1 ga \<alpha>" unfolding \<LL>1_def by blast
    qed
    then have d2: "(b, b') \<in> (\<LL>1 ga \<alpha>)^*" using d1 rtrancl_mono by blast
    have "gi s1 \<beta> \<subseteq> ga \<beta>" using c2 c3 b2 by fastforce
    then have d3: "(b', b'') \<in> (ga \<beta>)^=" using d1 by blast
    have "\<LL>v (gi s1) \<alpha> \<beta> \<subseteq> \<LL>v ga \<alpha> \<beta>"
    proof
      fix p
      assume "p \<in> \<LL>v (gi s1) \<alpha> \<beta>"
      then obtain \<gamma> where "(\<gamma> < \<alpha> \<or> \<gamma> < \<beta>) \<and> p \<in> gi s1 \<gamma
    >" unfolding \<LL>v_def by blast
      moreover then have "p \<in> ga \<gamma>" using c2 c3 b2 by fastforce
      ultimately show "p \<in> \<LL>v ga \<alpha> \<beta>" unfolding \<LL>v_def by blast
    qed
    then have "(b'', d) \<in> (\<LL>v ga \<alpha> \<beta>)^*" using d1 rtrancl_mono by blast
    then show ?thesis using d2 d3 unfolding \<DD>_def by blast
  qed
  moreover have "(c, c', c'', d) \<in> \<DD> ga \<beta> \<alpha>"
  proof -
    have d1: "(c, c') \<in> (\<LL>1 (gi s1) \<beta>)^* \<and> (c', c'') \<in> (gi s1 \<alpha>)^= \<and> (c'',
    d) \<in> (\<LL>v (gi s1) \<beta> \<alpha>)^*"
      using c5 unfolding \<DD>_def by blast
    have "\<LL>1 (gi s1) \<beta> \<subseteq> \<LL>1 ga \<beta>"
    proof
      fix p
      assume "p \<in> \<LL>1 (gi s1) \<beta>"
      then obtain \<gamma> where "\<gamma> < \<beta> \<and> p \<in> gi s1 \<gamma>" unfolding \<LL>1_def by
    blast
      moreover then have "p \<in> ga \<gamma>" using c2 c3 b2 by fastforce
      ultimately show "p \<in> \<LL>1 ga \<beta>" unfolding \<LL>1_def by blast
    qed
    then have d2: "(c, c') \<in> (\<LL>1 ga \<beta>)^*" using d1 rtrancl_mono by blast
    have "gi s1 \<alpha> \<subseteq> ga \<alpha>" using c2 b2 by fastforce
    then have d3: "(c', c'') \<in> (ga \<alpha>)^=" using d1 by blast
    have "\<LL>v (gi s1) \<beta> \<alpha> \<subseteq> \<LL>v ga \<beta> \<alpha>"
    proof
      fix p
      assume "p \<in> \<LL>v (gi s1) \<beta> \<alpha>"
      then obtain \<gamma> where "(\<gamma> < \<beta> \<or> \<gamma> < \<alpha>) \<and> p \<in> gi s1 \<gamma
    >" unfolding \<LL>v_def by blast
      moreover then have "p \<in> ga \<gamma>" using c2 c3 b2 by fastforce
      ultimately show "p \<in> \<LL>v ga \<beta> \<alpha>" unfolding \<LL>v_def by blast
    qed
    then have "(c'', d) \<in> (\<LL>v ga \<beta> \<alpha>)^*" using d1 rtrancl_mono by blast
    then show ?thesis using d2 d3 unfolding \<DD>_def by blast
  qed
  ultimately show "\<exists>b' b'' c' c'' d. (b, b', b'', d) \<in> \<DD> ga \<alpha> \<beta> \<and> (c, c', c
    '', d) \<in> \<DD> ga \<beta> \<alpha>" by blast
qed
then have "DCR_generating ga" unfolding DCR_generating_def by blast
moreover have "\<Union> S = \<Union>{r'. \<exists>\<alpha>'<n. r' = ga \<alpha>'}"
proof
  show "\<Union> S \<subseteq> \<Union>{r'. \<exists>\<alpha>'<n. r' = ga \<alpha>'}"
```

```
    proof
      fix p
      assume "p \<in> \<Union> S"
      then obtain s where "s \<in> S \<and> p \<in> s" by blast
      moreover then obtain \<alpha> where "\<alpha><n \<and> p \<in> gi s \<alpha>" using b3 by blast
      ultimately have "\<alpha><n \<and> p \<in> ga \<alpha>" using b2 by force
      then show "p \<in> \<Union>{r'. \<exists>\<alpha>'<n. r' = ga \<alpha>'}" by blast
    qed
  next
    show "\<Union>{r'. \<exists>\<alpha>'<n. r' = ga \<alpha>'} \<subseteq> \<Union> S"
    proof
      fix p
      assume "p \<in> \<Union>{r'. \<exists>\<alpha>'<n. r' = ga \<alpha>'}"
      then obtain \<alpha> where "\<alpha><n \<and> p \<in> ga \<alpha>" by blast
      moreover then obtain s where "s \<in> S \<and> p \<in> gi s \<alpha>" using b2 by force
      ultimately have "s \<in> S \<and> p \<in> s" using b3 by blast
      then show "p \<in> \<Union> S" by blast
    qed
  qed
  ultimately show ?thesis unfolding DCR_def by blast
qed

lemma lem_dc3_to_d3:
fixes r::"'U rel"
assumes "DCR 3 r"
shows "DCR3 r"
proof -
  obtain g where b1: "DCR_generating g" and b2: "r = \<Union>{r'. \<exists>\<alpha>'<3. r' = g \<alpha>'}"
      using assms unfolding DCR_def by blast
  have "\<forall> \<alpha>::nat. \<alpha><2 \<longleftrightarrow> \<alpha> = 0 \<or> \<alpha> = 1" by force
  then have b3: "\<LL>1 g 0 = {} \<and> \<LL>1 g 1 = g 0 \<and> \<LL>1 g 2 = g 0 \<union> g 1
      \<and> \<LL>v g 0 0 = {} \<and> \<LL>v g 1 0 = g 0 \<and> \<LL>v g 0 1 = g 0 \<and> \<LL>v g 1 1 = g 0
      \<and> \<LL>v g 2 0 = g 0 \<union> g 1 \<and> \<LL>v g 2 1 = g 0 \<union> g 1
      \<and> \<LL>v g 2 2 = g 0 \<union> g 1 \<and> \<LL>v g 0 2 = g 0 \<union> g 1 \<and> \<LL>v g 1 2 = g 0 \<
      union> g 1"
    unfolding \<LL>1_def \<LL>v_def by (simp_all, blast+)
  have "r = (g 0) \<union> (g 1) \<union> (g 2)"
  proof
    show "r \<subseteq> (g 0) \<union> (g 1) \<union> (g 2)"
    proof
      fix p
      assume "p \<in> r"
      then obtain \<alpha> where "p \<in> g \<alpha> \<and> \<alpha> < 3" using b2 by blast
      moreover have "\<forall> \<alpha>::nat. \<alpha><3 \<longleftrightarrow> \<alpha> = 0 \<or> \<alpha> = 1
      \<or> \<alpha> = 2" by force
      ultimately show "p \<in> (g 0) \<union> (g 1) \<union> (g 2)" by force
    qed
  next
    have "(0::nat) < (3::nat) \<and> (1::nat) < (3::nat) \<and> (2::nat) < (3::nat)" by simp
    then show "(g 0) \<union> (g 1) \<union> (g 2) \<subseteq> r" using b2 by blast
  qed
  moreover have "\<forall> a b c. (a,b) \<in> (g 0) \<and> (a,c) \<in> (g 0) \<longrightarrow> jn00 (g 0) b c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> (g 0) \<and> (a,c) \<in> (g 0)"
    then obtain b' b'' c' c'' d where "(b, b', b'', d) \<in> \<DD> g 0 0 \<and> (c, c', c'', d) \<in> \<DD> g 0
      0"
      using b1 unfolding DCR_generating_def by blast
    then show "jn00 (g 0) b c" unfolding jn00_def \<DD>_def \<LL>1_def \<LL>v_def by force
  qed
  moreover have "\<forall> a b c. (a,b) \<in> (g 0) \<and> (a,c) \<in> (g 1) \<longrightarrow> jn01 (g 0) (g 1)
      b c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> (g 0) \<and> (a,c) \<in> (g 1)"
    then obtain b' b'' c' c'' d where
      "(b, b', b'', d) \<in> \<DD> g 0 1 \<and> (c, c', c'', d) \<in> \<DD> g 1 0"
        using b1 unfolding DCR_generating_def by blast
    then show "jn01 (g 0) (g 1) b c" unfolding jn01_def \<DD>_def \<LL>1_def \<LL>v_def by force
  qed
  moreover have "\<forall> a b c. (a,b) \<in> (g 1) \<and> (a,c) \<in> (g 1) \<longrightarrow> jn11 (g 0) (g 1)
      b c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> (g 1) \<and> (a,c) \<in> (g 1)"
    then obtain b' b'' c' c'' d where "(b, b', b'', d) \<in> \<DD> g 1 1 \<and> (c, c', c'', d) \<in> \<DD> g 1
      1"
        using b1 unfolding DCR_generating_def by blast
```

```
    then show "jn11 (g 0) (g 1) b c" unfolding jn11_def \<DD>_def apply (simp only: b3) by blast
  qed
  moreover have "\<forall> a b c. (a,b) \<in> (g 0) \<and> (a,c) \<in> (g 2) \<longrightarrow> jn02 (g 0) (g 1)
      (g 2) b c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> (g 0) \<and> (a,c) \<in> (g 2)"
    then obtain b' b'' c' c'' d where c1: "(b, b', b'', d) \<in> \<DD> g 0 2 \<and> (c, c', c'', d) \<in> \<DD>
      g 2 0"
        using b1 unfolding DCR_generating_def by blast
    then have "(c, c') \<in> (g 0 \<union> g 1)^* \<and> (c',c'') \<in> (g 0)^= \<and> (c'',d) \<in> (g 0 \<
      union> g 1)^*"
      unfolding \<DD>_def by (simp add: b3)
    moreover then have "(c',c'') \<in> (g 0 \<union> g 1)^*" by blast
    ultimately have "(c, d) \<in> (g 0 \<union> g 1)^*" by force
    then show "jn02 (g 0) (g 1) (g 2) b c"
      using c1 unfolding jn02_def \<DD>_def apply (simp add: b3) by blast
  qed
  moreover have "\<forall> a b c. (a,b) \<in> (g 1) \<and> (a,c) \<in> (g 2) \<longrightarrow> jn12 (g 0) (g 1)
      (g 2) b c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> (g 1) \<and> (a,c) \<in> (g 2)"
    then obtain b' b'' c' c'' d where c1: "(b, b', b'', d) \<in> \<DD> g 1 2 \<and> (c, c', c'', d) \<in> \<DD>
      g 2 1"
        using b1 unfolding DCR_generating_def by blast
    then have "(c, c') \<in> (g 0 \<union> g 1)^* \<and> (c',c'') \<in> (g 1)^= \<and> (c'',d) \<in> (g 0 \<
      union> g 1)^*"
      unfolding \<DD>_def apply (simp only: b3) by blast
    moreover then have "(c',c'') \<in> (g 0 \<union> g 1)^*" by blast
    ultimately have "(c, d) \<in> (g 0 \<union> g 1)^*" by force
    then show "jn12 (g 0) (g 1) (g 2) b c"
      using c1 unfolding jn12_def \<DD>_def apply (simp only: b3) by blast
  qed
  moreover have "\<forall> a b c. (a,b) \<in> (g 2) \<and> (a,c) \<in> (g 2) \<longrightarrow> jn22 (g 0) (g 1)
      (g 2) b c"
  proof (intro allI impI)
    fix a b c
    assume "(a,b) \<in> (g 2) \<and> (a,c) \<in> (g 2)"
    then obtain b' b'' c' c'' d where c1: "(b, b', b'', d) \<in> \<DD> g 2 2 \<and> (c, c', c'', d) \<in> \<DD>
      g 2 2"
        using b1 unfolding DCR_generating_def by blast
    then show "jn22 (g 0) (g 1) (g 2) b c" unfolding jn22_def \<DD>_def apply (simp only: b3) by blast
  qed
  ultimately have "LD3 r (g 0) (g 1) (g 2)" unfolding LD3_def by blast
  then show ?thesis unfolding DCR3_def by blast
qed

lemma lem_dc3_confl_lewsuc:
fixes r::"'U rel"
assumes a1: "confl_rel r" and a2: "|Field r| \<le>o cardSuc |UNIV::nat set|"
shows "DCR 3 r"
proof -
  obtain S where b1: "r = \<Union> S"
             and b2: "\<forall> s1 \<in> S. \<forall> s2 \<in> S. s1 \<noteq> s2 \<longrightarrow> Field s1 \<
      inter> Field s2 = {}"
             and b3: "\<forall> s \<in> S. CCR s" using a1 lem_Cprf_conf_ccr_decomp[of r] by blast
  have "\<forall> s\<in>S. DCR 3 s"
  proof
    fix s
    assume "s \<in> S"
    then have "CCR s \<and> Field s \<subseteq> Field r" using b1 b3 unfolding Field_def by blast
    moreover then have "|Field s| \<le>o |Field r|" by simp
    ultimately have "CCR s \<and> |Field s| \<le>o cardSuc |UNIV::nat set|" using a2 ordLeq_transitive by blast
    then show "DCR 3 s" using lem_dc3_ccr_scf_lewsuc by blast
  qed
  then show "DCR 3 r" using b1 b2 lem_Cprf_dc_disj_fld_un[of S] by blast
qed

lemma lem_cle_eqdef: "|A| \<le>o |B| = (\<exists> g . A \<subseteq> g`B)"
  by (metis surj_imp_ordLeq card_of_ordLeq2 empty_subsetI order_refl)

lemma lem_cardLeN1_eqdef:
fixes A::"'a set"
shows "cardLeN1 A = ( |A| \<le>o cardSuc |{n::nat . True}| )"
proof
  assume b1: "cardLeN1 A"
  obtain \<kappa> where b2: "\<kappa> = cardSuc |UNIV::nat set|" by blast
```

```
  have "cardSuc |UNIV::nat set| <o |A| \<longrightarrow> False"
  proof
    assume "cardSuc |UNIV::nat set| <o |A|"
    then have c1: "\<kappa> <o |A| \<and> |Field \<kappa>| =o \<kappa>" using b2 by simp
    then have "|Field \<kappa>| \<le>o |A|" using ordIso_ordLess_trans ordLess_imp_ordLeq by blast
    then obtain B where c2: "B \<subseteq> A \<and> |Field \<kappa>| =o |B|"
      using internalize_card_of_ordLeq2[of "Field \<kappa>" A] by blast
    moreover have "|UNIV::nat set| <o \<kappa>" using b2 by simp
    ultimately have c3: "B \<subseteq> A \<and> |UNIV::nat set| <o |B|"
      using c1 by (meson ordIso_imp_ordLeq ordIso_symmetric ordLess_ordLeq_trans)
    then obtain C where c4: "C \<subseteq> B \<and> |UNIV::nat set| =o |C|"
      using internalize_card_of_ordLeq2[of "UNIV::nat set" B] ordLess_imp_ordLeq by blast
    obtain c where "c \<in> C" using c4 using card_of_empty2 by fastforce
    moreover obtain D where c5: "D = C - {c}" by blast
    ultimately have c6: "C = D \<union> {c}" by blast
    have "\<not> finite D" using c4 c5 using card_of_ordIso_finite by force
    moreover then have "|{c}| \<le>o |D|" by (metis card_of_singl_ordLeq finite.emptyI)
    ultimately have "|C| \<le>o |D|" using c6 using card_of_Un_infinite ordIso_imp_ordLeq by blast
    then obtain f where "C \<subseteq> f ` D" by (metis card_of_ordLeq2 empty_subsetI order_refl)
    moreover have "D \<subset> C \<and> C \<subseteq> B \<and> B \<subseteq> A" using c3 c4 c5 c6 by blast
    ultimately have "(\<exists>f. B \<subseteq> f ` C) \<or> (\<exists>g. A \<subseteq> g`B)" using b1 unfolding
       cardLeN1_def by metis
    moreover have "(\<exists>f. B \<subseteq> f ` C) \<longrightarrow> False"
    proof
      assume "\<exists>f. B \<subseteq> f ` C"
      then obtain f where "B \<subseteq> f ` C" by blast
      then have "|B| \<le>o |f`C|" by simp
      moreover have "|f`C| \<le>o |C|" by simp
      ultimately have "|B| \<le>o |C|" using ordLeq_transitive by blast
      then show "False" using c3 c4 not_ordLess_ordIso ordLess_ordLeq_trans by blast
    qed
    moreover have "(\<exists>g. A \<subseteq> g`B) \<longrightarrow> False"
    proof
      assume "\<exists>g. A \<subseteq> g`B"
      then obtain g where "A \<subseteq> g`B" by blast
      then have "|A| \<le>o |g`B|" by simp
      moreover have "|g`B| \<le>o |B|" by simp
      ultimately have "|A| \<le>o |B|" using ordLeq_transitive by blast
      then show "False" using c1 c2
        by (metis BNF_Cardinal_Order_Relation.ordLess_Field not_ordLess_ordIso ordLess_ordLeq_trans)
    qed
    ultimately show "False" by blast
  qed
  then show "|A| \<le>o cardSuc |{n::nat . True}|" by simp
next
  assume "|A| \<le>o cardSuc |{n::nat . True}|"
  then have b1: "|A| \<le>o cardSuc |UNIV::nat set|" by simp
  have "\<forall> B \<subseteq> A. ( \<forall> C \<subseteq> B . ((\<exists> D f. D \<subset> C \<and> C \<
      subseteq> f`D ) \<longrightarrow> ( \<exists> f. B \<subseteq> f`C )) )
                  \<or> ( \<exists> g . A \<subseteq> g`B )"
  proof (intro allI impI)
    fix B
    assume "B \<subseteq> A"
    show "(\<forall> C \<subseteq> B . ((\<exists> D f. D \<subset> C \<and> C \<subseteq> f`D ) \<
      longrightarrow> ( \<exists> f. B \<subseteq> f`C ))) \<or> ( \<exists> g . A \<subseteq> g`B )"
    proof (cases "|B| \<le>o |UNIV::nat set|")
      assume d1: "|B| \<le>o |UNIV::nat set|"
      have "\<forall> C \<subseteq> B . ((\<exists> D f. D \<subset> C \<and> C \<subseteq> f`D ) \<
      longrightarrow> ( \<exists> f. B \<subseteq> f`C ))"
      proof (intro allI impI)
        fix C
        assume "C \<subseteq> B" and "\<exists> D f. D \<subset> C \<and> C \<subseteq> f`D"
        then obtain D f where e1: "D \<subset> C \<and> C \<subseteq> f`D" by blast
        have "finite C \<longrightarrow> False"
        proof
          assume "finite C"
          moreover then have "finite D" using e1 finite_subset by blast
          ultimately have "|D| <o |C|"
            using e1 by (metis finite_card_of_iff_card3 psubset_card_mono)
          moreover have "|C| \<le>o |D|" using e1 using surj_imp_ordLeq by blast
          ultimately show "False" using not_ordLeq_ordLess by blast
        qed
        then have "|B| \<le>o |C|" using d1 by (metis infinite_iff_card_of_nat ordLeq_transitive)
        then show "\<exists> f. B \<subseteq> f`C" by (metis card_of_ordLeq2 empty_subsetI order_refl)
      qed
      then show ?thesis by blast
    next
      assume "\<not> |B| \<le>o |UNIV::nat set|"
```

```
      then have "|A| \<le>o |B|" using b1 lem_cord_lin
        by (metis cardSuc_ordLeq_ordLess card_of_Card_order ordLess_ordLeq_trans)
      then have "\<exists> g . A \<subseteq> g`B" by (metis card_of_ordLeq2 empty_subsetI order_refl)
      then show ?thesis by blast
    qed
  qed
  then show "cardLeN1 A" unfolding cardLeN1_def by blast
qed

lemma lem_cleN1_eqdef:
fixes r::"('U\<times>'U) set"
shows "   ( |r| \<le>o cardSuc |{n::nat . True}| )
      \<longleftrightarrow> ( \<forall> s \<subseteq> r. (   ( \<forall> t \<subseteq> s . ((\<exists> t' f. t'
      \<subset> t \<and> t \<subseteq> f`t') \<longrightarrow> (\<exists> f. s \<subseteq> f`t )) )
                      \<or> ( \<exists> g . r \<subseteq> g`s )
                     ) )"
  using lem_cardLeN1_eqdef[of r] cardLeN1_def by blast

(* ------------------------------------------------------------------------ *)

section \<open>Main result\<close>

(* ------------------------------------------------------------------------ *)

theorem thm_main:
fixes r::"('U\<times>'U) set"
assumes "\<forall> a b c . (a,b) \<in> r^* \<and> (a,c) \<in> r^* \<longrightarrow> (\<exists> d. (b,d) \<in> r
      ^* \<and> (c,d) \<in> r^*)"
    and "|r| \<le>o cardSuc |{n::nat . True}|"
shows "\<exists> r0 r1 r2 . (
           ( r = (r0 \<union> r1 \<union> r2) )
         \<and> ( \<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r0
               \<longrightarrow> (\<exists> d.
                         (b,d) \<in> r0^=
                       \<and> (c,d) \<in> r0^= ) )
         \<and> ( \<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r1
               \<longrightarrow> (\<exists> b' d.
                         (b,b') \<in> r1^= \<and> (b',d) \<in> r0^*
                       \<and> (c,d) \<in> r0^* ) )
         \<and> ( \<forall> a b c. (a,b) \<in> r1 \<and> (a,c) \<in> r1
               \<longrightarrow> (\<exists> b' b'' c' c'' d.
                         (b,b') \<in> r0^* \<and> (b',b'') \<in> r1^= \<and> (b'',d) \<in> r0^*
                       \<and> (c,c') \<in> r0^* \<and> (c',c'') \<in> r1^= \<and> (c'',d) \<in> r0^* ) )
         \<and> ( \<forall> a b c. (a,b) \<in> r0 \<and> (a,c) \<in> r2
               \<longrightarrow> (\<exists> b' d.
                          (b,b') \<in> r2^= \<and> (b',d) \<in> (r0 \<union> r1)^*
                        \<and> (c,d) \<in> (r0 \<union> r1)^* ) )
         \<and> ( \<forall> a b c. (a,b) \<in> r1 \<and> (a,c) \<in> r2
               \<longrightarrow> ( \<exists> b' b'' d.
                          (b,b') \<in> r0^* \<and> (b',b'') \<in> r2^= \<and> (b'',d) \<in> (r0 \<union> r1)^*
                        \<and> (c,d) \<in> (r0 \<union> r1)^* ) )
         \<and> ( \<forall> a b c. (a,b) \<in> r2 \<and> (a,c) \<in> r2
               \<longrightarrow> (\<exists> b' b'' c' c'' d.
                          (b,b') \<in> (r0 \<union> r1)^* \<and> (b',b'') \<in> r2^= \<and> (b'',d) \<in> (r0 \<
      union> r1)^*
                        \<and> (c,c') \<in> (r0 \<union> r1)^* \<and> (c',c'') \<in> r2^= \<and> (c'',d) \<in> (r0
       \<union> r1)^* ) )
        )"
proof -
  have b0: "|r| \<le>o cardSuc |UNIV::nat set|" using assms(2) by simp
  obtain \<kappa> where b1: "\<kappa> = cardSuc |UNIV::nat set|" by blast
  have "|Field r| \<le>o \<kappa>"
  proof (cases "finite r")
    assume "finite r"
    then show ?thesis using b1 lem_fin_fl_rel by (metis Field_card_of Field_natLeq cardSuc_ordLeq_ordLess
      card_of_card_order_on card_of_mono2 finite_iff_ordLess_natLeq ordLess_imp_ordLeq)
  next
    assume "\<not> finite r"
    then show ?thesis using b0 b1 lem_rel_inf_fld_card using ordIso_ordLeq_trans by blast
  qed
  moreover have "confl_rel r" using assms(1) unfolding confl_rel_def by blast
  ultimately have "DCR3 r" using b1 lem_dc3_confl_lewsuc[of r] lem_dc3_to_d3 by blast
  then show ?thesis unfolding DCR3_def LD3_def
    jn00_def jn01_def jn02_def jn11_def jn12_def jn22_def by fast
qed

end
```

References

References[1] Radhakisan Baheti and Helen Gill. Cyber-physical systems. The impact of control technology, 12(1):161–166, 2011. [2] Nicolas Behr and Jean Krivine. Rewriting theory for the life sciences. In International Conference on Graph Transformation, pages 185–202, 2020. [3] Jasmin Christian Blanchette, Andrei Popescu, and Dmitriy Traytel. Cardinals in Isabelle/HOL. In Lecture Notes in Computer Science, volume 8558, pages 111–127, 2014. doi:10.1007/978-3-319-08970-6_8. [4] Mirna Bognar. A point version of decreasing diagrams. In Proceedings Accolade 1996. Dutch Graduate School in Logic, pages 1–14, 1997. [5] F. Bonchi, F. Gadducci, A. Kissinger, P. Sobociński, and F. Zanasi. String diagram rewrite theory I: Rewriting with Frobenius structure. Journal of the ACM, 69(2):14:1–14:58, 2022. doi:10.1145/3502719. [6] N.G. de Bruijn. A note on weak diamond properties. Memorandum 78–08, Eindhoven Uninversity of Technology, 1978. [7] R. Diestel and O. Pikhurko. On the cofinality of infinite partially ordered sets: Factoring a poset into lean essential subsets. Order, 20:53–66, 2003. doi:10.1023/A:1024449306316. [8] Jörg Endrullis and Jan Willem Klop. De Bruijn's weak diamond property revisited. Indagationes Mathematicae, 24:1050–1072, 2013. [9] Jörg Endrullis, Jan Willem Klop, and Roy Overbeek. Decreasing diagrams with two labels are complete for confluence of countable systems. In 3rd International Conference on Formal Structures for Computation and Deduction (FSCD 2018), pages 14:1–14:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi:10.4230/LIPICS.FSCD.2018.14. [10] Jörg Endrullis, Jan Willem Klop, and Roy Overbeek. Decreasing diagrams for confluence and commutation. Logical Methods in Computer Science, 16:23:1–23:25, 2020. doi:10.23638/LMCS-16(1:23)2020. [11] A.A. Fraenkel, Y. Bar-Hillel, and A. Levy. Foundations of Set Theory. Elsevier, 1973. [12] Jean Goubault-Larrecq. Non-Hausdorff topology and domain theory: Selected topics in point-set topology. Cambridge University Press, 2013. [13] N. Hirokawa, A. Middeldorp, C. Sternagel, and S. Winkler. Abstract completion, formalized. Logical Methods in Computer Science, 15(3):19:1–19:42, 2019. doi:10.23638/LMCS-15(3:19)2019. [14] Isabelle proof assistant. http://isabelle.in.tum.de. [Online]. [15] J.R. Isbell. The category of cofinal types. II. Trans. Amer. Math. Soc., 116:394–416, 1965. [16] Ievgen Ivanov. Generalized Newman's lemma for discrete and continuous systems. In Leibniz International Proceedings in Informatics, volume 260, pages 9:1–9:17, 2023. doi:10.4230/LIPICS.FSCD.2023.9. [17] Ievgen Ivanov. Formal proof of completeness of the decreasing diagrams method for proving confluence of relations of the least uncountable cardinality (supplementary material for this paper). https://doi.org/10.5281/zenodo.14254256, 2024. [Online]. [18] Ievgen Ivanov. On Newman's lemma and non-termination. In CEUR-WS.org, volume 3624, pages 14–24, 2024. URL: https://ceur-ws.org/Vol-3624/Paper_2.pdf. [19] Ievgen Ivanov. On non-triviality of the hierarchy of decreasing Church-Rosser abstract rewriting systems. In Proceedings of the 13th International Workshop on Confluence, pages 30–35, 2024. [20] Ievgen Ivanov. On the cofinality property in the context of the hierarchy of decreasing Church-Rosser abstract rewriting systems. In Information Technology and Implementation, November 20–21, 2024, Kyiv, Ukraine, 2024. [21] Ievgen Ivanov. Completeness of decreasing diagrams for the least uncountable cardinality. Archive of Formal Proofs, April 2025. https://isa-afp.org/entries/Completeness_Decreasing_Diagrams_for_N1.html, Formal proof development. [22] Ievgen Ivanov. Formalization of a confluent abstract rewriting system of the least uncountable cardinality outside of the class DCR2. https://doi.org/10.5281/zenodo.14740062, 2025. [Online]. [23] John L. Kelley. General topology. Van Nostrand, 1955. [24] Jan Willem Klop. Combinatory Reduction Systems. PhD thesis, Rijks-universiteit Utrecht, 1980. [25] Alexander Krauss and Andreas Schropp. A mechanized translation from Higher-Order Logic to set theory. In Lecture Notes in Computer Science, volume 6172, pages 323–338. Springer, 2010. doi:10.1007/978-3-642-14052-5_23. [26] Ondřej Kunčar and Andrei Popescu. Comprehending Isabelle/HOL's consistency. In Lecture Notes in Computer Science, volume 10201, pages 724–749. Springer, 2017. doi:10.1007/978-3-662-54434-1_27. [27] Ondřej Kunčar and Andrei Popescu. A consistent foundation for Isabelle/HOL. Journal of Automated Reasoning, 62:531–555, 2019. doi:10.1007/S10817-018-9454-8. [28] Edward A. Lee. Fundamental limits of cyber-physical systems modeling. ACM Transactions on Cyber-Physical Systems, 1(1):1–26, 2016. doi:10.1145/2912149. [29] George Markowsky. Chain-complete posets and directed sets with applications. Algebra universalis, 6(1):53–68, 1976. [30] Elliott Mendelson. Introduction to Mathematical Logic. CRC Press, 2015. [31] Eric C. Milner and Maurice Pouzet. On the cofinality of partially ordered sets. Ordered sets, pages 279–298, 1982. [32] Eric C. Milner and Karel Prikry. The cofinality of a partially ordered set. Proc. London Math. Soc., 46:454–470, 1983. [33] Chuangjie Xu Nicolai Kraus, Fredrik Nordvall Forsberg. Type-theoretic approaches to ordinals. Theoretical computer science, 957, 2023. doi:10.1016/J.TCS.2023.113843. [34] Tobias Nipkow, Markus Wenzel, and Lawrence C Paulson. Isabelle/HOL: a proof assistant for

higher-order logic. Springer, 2002. doi:10.1007/3-540-45949-9. [35] Andrew Pitts. The HOL logic. In Introduction to HOL: A Theorem Proving Environment for Higher Order Logic, pages 191–232. Cambridge University Press, 1993. [36] Raymond Smullyan. Set theory and the continuum problem. Clarendon Press, 1996. [37] Christian Sternagel and Rene Thiemann. Abstract rewriting. Archive of Formal Proofs, June 2010. [38] Stevo Todorcevic. Directed sets and cofinal types. Trans. Amer. Math. Soc., 290:711–723, 1985. [39] John W. Tukey. Convergence and uniformity in topology. Princeton University Press, 1940. [40] Vincent Van Oostrom. Confluence by decreasing diagrams. Theoretical computer science, 126(2):259–280, 1994. doi:10.1016/0304-3975(92)00023-K. [41] Vincent Van Oostrom. Confluence for Abstract and Higher-Order Rewriting. PhD thesis, Vrije Universiteit Amsterdam, 1994. [42] Vincent Van Oostrom. Confluence by decreasing diagrams – converted. In Lecture Notes in Computer Science, volume 5117, pages 306–320, 2008. [43] Shang-Zhi Wang and Bo Yu Li. On the minimal cofinal subsets of a directed quasi-ordered set. Discrete Mathematics, 48:289–306, 1984. doi:10.1016/0012-365X(84)90189-4. [44] Freek Wiedijk. The seventeen provers of the world: Foreword by Dana S. Scott, volume 3600. Springer, 2006. doi:10.1007/11542384. [45] Harald Zankl. Confluence by decreasing diagrams – formalized. In 24th International Conference on Rewriting Techniques and Applications, pages 352–367. Leibniz International Proceedings in Informatics, 2013. doi:10.4230/LIPICS.RTA.2013.352. [46] Harald Zankl. Decreasing diagrams. Archive of Formal Proofs, 2013.